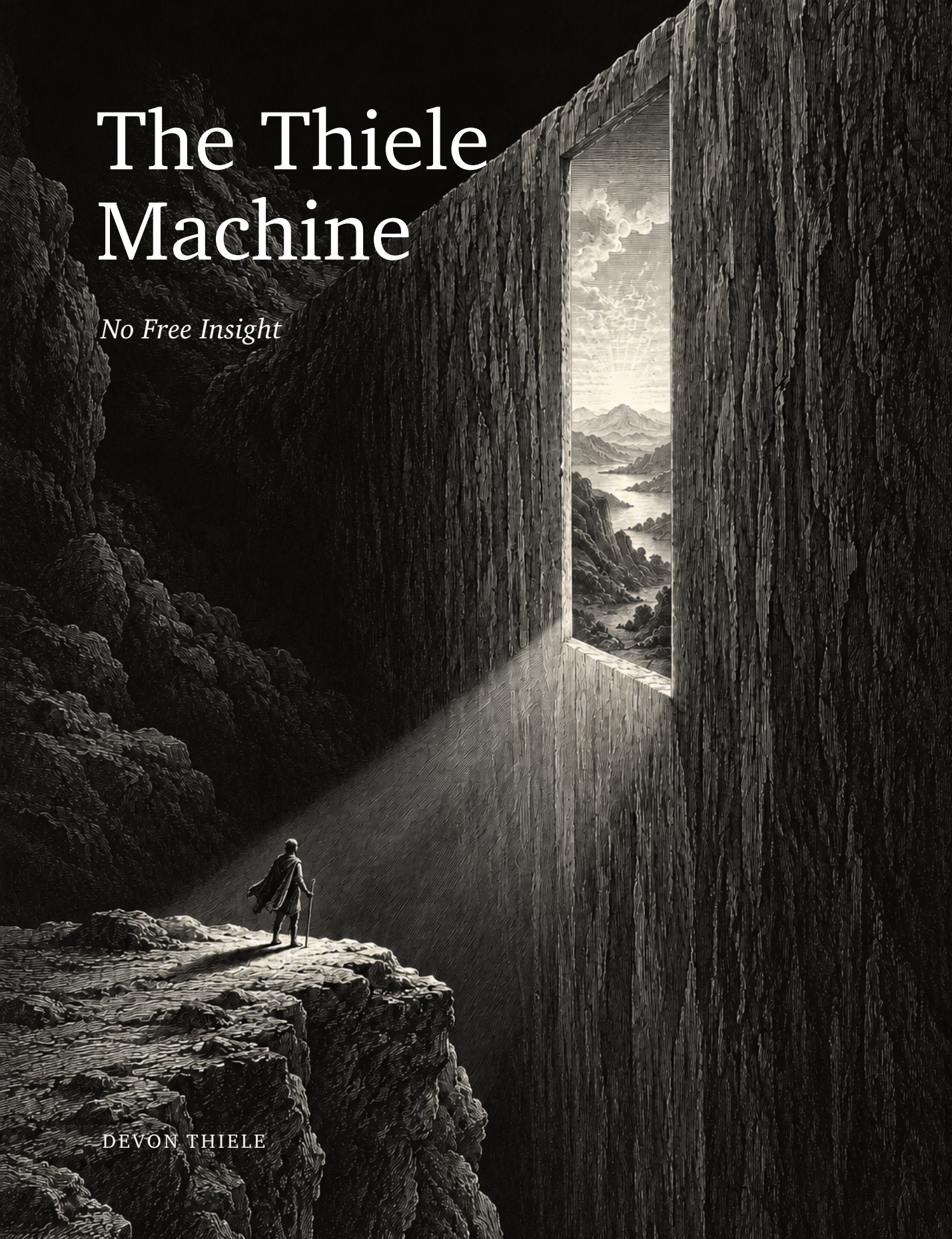
The Thiele Machine

No Free Insight



DEVON THIELE

DEVON THIELE

The Thiele Machine

No Free Insight

The Thiele Machine: No Free Insight

Devon Thiele

Copyright 2025–2026 Devon Thiele. Licensed under Creative Commons Attribution-ShareAlike 4.0 (CC BY-SA 4.0).

The source of this book, its proofs and its build are public: github.com/sethirus/The-Thiele-Machine. Archived at doi.org/10.5281/zenodo.17316437.

Set in Charter. Built from its source with pdfTeX.

The cover and part illustrations were generated with AI in the engraving tradition of Gustave Doré. They are new illustrations, not reproductions of his work.

For Julie, and for my dad.

Julie,

Three years ago, a few months into the rendering engine, I came out of my office and told you I thought I was onto something. You said okay. You didn't ask what it was, or what it was for, or when it would be done, and you didn't ask me for anything I could have shown you, because there was nothing to show. There was a program that had done something it shouldn't have been able to do, and a man who didn't yet know what a Turing machine was.

You gave me the three years anyway. The nights after the lot closed. The weekends. The hours I was in the house and not in the house, because my head was in a wall I couldn't get past. Every dead end, and there were more of those than there are pages here. You never once asked me to stop, and you never once said it was fine when it wasn't. You just kept giving me the next evening.

This book is what I did with them. It's the one thing I've built from nothing that I'm sure of, because I threw out every piece of it I couldn't prove, and what's left holds. I know that isn't what you gave the years for. You gave them for me. But it's what I have to show, and I wanted it to be worth showing.

Dad,

The name on the cover was yours first. I asked you before I used it and you said go ahead. You and Mom taught me that nothing is true just because somebody says it is. This book is that lesson, six hundred pages long. I put our name on it because I wanted one thing in the world with that name on it that nobody handed me, that I couldn't have faked, and that I can stand behind in front of anyone. Here it is.

Julie carries the name now too. So the machine has it third. This is for the two of you who had it first.

Contents

I What keeping costs **23**

1 A distinction that stays 25
- 1.1 Every step does one of three things 25
- 1.2 What a kept distinction is 30
- 1.3 A reversible machine keeps nothing 33
- 1.4 Noisy steps 34
- 1.5 Unitary steps 37
- 1.6 Going faster gives off more heat 39
- 1.7 Every scale pays the same way 40
- 1.8 What keeping takes, and where the toll comes in 42

2 A record and a toll 44
- 2.1 A demon, and the step that pays 45
- 2.2 The smallest thing that could carry the charge 47
- 2.3 How the receipt earns its meaning 57

3 Why the toll isn't arbitrary 63
- 3.1 A closed room with no spare chair 64
- 3.2 Now charge for forgetting, and the toll falls out 70
- 3.3 The first way out: keep everything, and pay by taking fresh states 72
- 3.4 The second way out: let the flag come back down 81
- 3.5 The third way out: don't charge for forgetting 82
- 3.6 How much: the price grows with the squeeze 86
- 3.7 The same bound, written in bits, and when it's zero 89
- 3.8 What you're entitled to, and what you're not 100

4 The whole direction 107
- 4.1 A row of cups 107
- 4.2 The order, said exactly 109
- 4.3 The toll, over the whole direction 110
- 4.4 One line isn't enough 112
- 4.5 Why crossing a line has to cost 113

5 Learning borrows, committing pays 116
- 5.1 Bennett moves where the heat is paid 116
- 5.2 Two different things called "narrowing" 117
- 5.3 The demon on a sheet of paper 119
- 5.4 Who holds the pen changes who pays 122

5.5 The smallest machine that teaches with no merge, and the one that can't 123
5.6 The certainty bound 125
5.7 What the toll prices, said at last 127

II What structure is made of 131

6 What structure is 133
6.1 A set with some named relations 134
6.2 Laying them all down 135
6.3 The smallest piece 138
6.4 Buckets 141

7 The counter 144
7.1 A stick, split down the middle 144
7.2 Why it has to climb 145
7.3 Conservation: the total is the charges and nothing else 147
7.4 Only one honest count 149
7.5 The toll is an old trick with a new name 152
7.6 What the counter is, said once 157

8 Windows and shadows 159
8.1 A cave, and the people who got good at it 159
8.2 The shadow of a cube 160
8.3 The whole fact, in one paragraph 161
8.4 The clerk with two photographs 163
8.5 The three escapes, and what each one costs 166
8.6 What the window keeps 173

9 Structural entitlement 174
9.1 The coin that moved 174
9.2 Plato nearly had it 175
9.3 The shape, said once 176
9.4 What narrowing costs: count the rivals 182
9.5 What "narrower" buys: a stronger test 185
9.6 What you've got 186
9.7 A grouping, checked and committed 188

10 What insight is 191
10.1 Sultan and the sticks 191
10.2 It isn't a feeling 192
10.3 What happens when a computation learns 193
10.4 Test, then commit, and nothing forces the second to follow the first 195
10.5 What the word means 196

11 Records over any base: the latch 198
11.1 A cup and a die 198
11.2 What "base-driven" has to mean 199
11.3 Both rules do work: the toggle and the timestamp 207
11.4 Is there only one such machine? No: one structure, two unforced choices. 213
11.5 Records with more than two values 215
11.6 What leaving the base open buys you 217

12 The shadow, exactly 218
12.1 The ordinary machine is a shadow 218
12.2 Pricing from the shadow 218
12.3 What a priced move destroys 220
12.4 What the shadow is 221
12.5 Shortcuts 222

13 The diagonal 223
13.1 Now point it at the record 223
13.2 What this does to the book 226

III Established **229**

14 What established means 231
14.1 Two computers that end up the same 231
14.2 Look harder 232
14.3 So keep the trace 233
14.4 Computed, stored, established 235
14.5 The clerk who forgets everything 236
14.6 Information doesn't have it either 237
14.7 Statues that walk off 239
14.8 What "could have failed" means 241
14.9 Settled, whether it's true or not 242
14.10 Not a switch 245
14.11 Computing already had a word for it 250
14.12 What settling looks like 253
14.13 What the Thiele Machine is 254
14.14 The two computers, put back together 255
14.15 Two layers, and the names 257

15 The earned-chain counter example 262
15.1 What the model asks for, and the example that meets it 262
15.2 The example machine, piece by piece 265
15.3 It computes everything two counters can 276

15.4 It's a Thiele machine, and the flag is a latch 279
15.5 Commitments are earned 281
15.6 What a passing check means 284
15.7 Nothing gets forged 287
15.8 The window can't see the record 288
15.9 The exact price is three 293
15.10 The machine versus the program it runs 294
15.11 The small machine's merges 297
15.12 Reading the instruction rules 298
15.13 What this chapter adds up to 302

16 How far it reaches 303

17 Five small models 304
17.1 Proof-of-stake finality: the price is on going back 305
17.2 Gas: the one that charges a fee at the write 308
17.3 Attestation: hash chains, and a quote about the past 312
17.4 Certificate transparency: visible, unpriced 316
17.5 Proof-carrying code: no price, no permanence 318
17.6 The same table, five ways 318

18 The pointer thesis 321
18.1 The claim 321
18.2 The end of the logic 323

19 The counter against physics 326
19.1 The premises have to be able to hold together 326
19.2 Permanence and the heat premise 328
19.3 At the bottom: what I suspect and can't prove 328

IV The results together **331**

20 The claims and their conditions 333
20.1 Keeping a distinction 333
20.2 Putting a price beside the arrows 334
20.3 What reaches the reader 334
20.4 The concrete machine and its guests 335

21 The claim ledger 337
21.1 Claims about the model 337
21.2 Claims about the machine 356
21.3 What I propose 363

22 Scope 365

23 Conclusion 371

24 Afterword: how I got here 377

A Words this book builds 381

B Every assumption in one place 386
B.1 Systems and their conditions 386
B.2 The premises the derivations take 387

C The universal machines 391
C.1 One machine that runs them all 391
C.2 Every computably presented Thiele machine, on one host: driven, decided, priced in whole units, carried as a latch 399

D Built from parts 406
D.1 Side by side 406
D.2 One after the other 409
D.3 Connecting records and writers 410
D.4 One inside another 412
D.5 What this adds up to 415

E Sources 416
E.1 Structure, information and knowledge 416
E.2 Cost, records and thermodynamics 427
E.3 Computation, self-reference and checking 443
E.4 Correlations and environmental records 459

The point

The whole book is four steps, and I'll set them out first so you know what you're carrying. Each step is motivated by the one before it, and the book goes up them in order.

One: nothing becomes something for free. Take a blank cell: a square of paper with nothing on it, a bead that could sit in either of two dips, a bit of memory nobody has set. Now make it a 0, and keep it a 0. Something is the case now that wasn't a moment ago. For a total deterministic rule, a reading newly raised and then held requires a merge somewhere in the run or endlessly fresh states. Physical dissipation requires a separate model; the thermal models discussed below impose further costs under their own hypotheses.

Two: so the machine is the thing that steps, with the price in the move. A Thiele machine is a state, some thing; a reading of each state into an order, so that of two states you can ask whether the second reads at or below the first, and higher means more is settled; moves, each a direction that transforms the state into the next state; and a price on each move. One rule, which I call the toll: a move that carries the reading somewhere not at or below where it stood is never free. That is the whole definition, and everything in this book is a consequence of it or an instance of it. I named it after myself, on purpose: I wanted to call it the meta machine, a company took that word in 2021, and I went one level up, half a joke and half a stake in the ground; the afterword has the rest. The step that raises a reading that stays is the step that's charged. A machine whose moves never climb its order is the *flat case*, and the toll has nothing to say to it: a clock is one, a reversible process is one, and so is every ordinary model of computation, the Turing machine included, with its price column torn off and its record forgotten. Computer science charges a computation for how long it runs and how much space it takes, and the bits it erases have been counted too. What no model I found charges is the moment a reading goes up and stays. Those steps are a few events in a run, not the billions of overwrites a computer makes every second; what they write doesn't stay, and the toll leaves them to the erasure count and to physics. Put a window over the machine that shows only the flat part, and through it you can watch the computing while missing what the machine kept about it. I will have to earn that picture by giving the window and the moves precise rules; calling something a window is not enough.

To show the definition can be met, Part III builds one example and takes it apart: two counters and a line number for the flat part, and a record whose moves check

a claim, commit to a passed check and certify it. Its working states can look alike while the full states differ in what was kept. The Thiele Machine is the definition, not the example.

The counting underneath needs its own conditions too: states, a next-state rule, and a reading that stays. Once those are given, what follows is a theorem. Whether a physical process fits that description is the further question stated in the Conclusion.

Three: the structure a computation makes is made of exactly this. Structure in general is a relation kept under the changes you choose; Part II takes that word from the people who wrestled with it. The part a computation makes, and can't take back, is a distinction held in its state, and a 0 is the smallest one there is: the ways a cell could be, sorted into piles, and one pile held. A grouping, an order, a relation a later move leans on, a fact somebody found out and wrote down: each is a distinction that stays, paid for the same way as the 0. Insight, in the sense this book can price, is a kept distinction. The toll charges the moment it's raised; holding it up afterward adds nothing to the cost. That's what the book's other name, No Free Insight, means.

Four: established is the sharpest case. Two computers can end with the same answer, and only one of them checked it. The difference between them is a distinction one of them never kept. A record that a claim was checked and closed is a 0 that stays, with a check in front of it, and it's paid for the same way. I build a machine for that case, all the way down, because it's the one where you can point at the kept 0 and at what it cost. It's the sharpest case of what keeping takes, but it isn't the point. The point is the distinction that stays, and the 0 is the smallest one there is.

Step one is where the toll gets its reason. If it breaks, the toll goes back to being only my choice, though the latch, the window, the diagonal and the earned record are proved without it and would stand. So that's where I start: the blank cell, taken apart as far down as I can get before I hit something that holds or something I can't get under. I'll tell you which.

One warning about how this went, because it explains the shape of everything after it. Most of the time I had the hunch first and found the names later. Over and over, somebody had been here before me. Sometimes that made my day. Sometimes it pissed me off. You'll see both.

Where a 0 comes from

Before any machine comes the cell. What does it take for there to be a 0 at all?

It takes three things. First, something that can be more than one way: a bead in a dish of water, a speck of magnetised metal, a charge on a tiny capacitor, a square of paper. Nature hands you that much, and it isn't a 0 yet. It's stuff that can sit in a lot of arrangements.

Second, a sorting: somebody says which arrangements count as 0, which count as 1, and which count as blank. The sorting is the first piece of structure Part II gets to: the ways a thing could be, cut into piles. Without it there's no 0, just a bead somewhere. A 0 and a 1 aren't things; they're the two sides of a line somebody drew across the ways a thing can be.

Third, something that keeps the chosen arrangement readable for the time you need it. In the bead experiment below, that something is a hill between two wells. A high hill makes escape rare; it does not make escape impossible. At a barrier of about eight times the thermal energy, the paper estimates a characteristic escape time of roughly fifty minutes. Lower the barrier to about twice that energy and the estimate is about ten seconds. Those are waiting-time scales, not appointments at which the bead must jump. The barrier height is not a bill for heat already given off.

A latch, a refreshed charge or a written mark needs its own account of how it holds. The common question is simple: which later changes are allowed, and do they leave the answer readable? In the mathematical examples I can require a yes to stay yes under every allowed move. The bead is a physical example of retention for a time, with a chance of failure.

Now for the moment itself: the cell is blank and you write a 0. Everything that step can take comes next, in Part I, and the rest of the book only ever adds detail to it.

Why it has to cost

Now what it takes to keep that 0, made exact, and it's the part I'm proudest of, because once something stays, the choosing stops and the counting takes over. And the counting is the one behind musical chairs.

Do it with chairs when the room runs out. Draw a box on the floor, three chairs inside, one chair outside by the door. Four people, one per chair: the four states the machine could be in. The three inside already read yes. One step moves everybody at once, each to the chair the rule sends them to. Nobody inside may leave, because yes is permanent. The person by the door has to come in, because this is the step that flips them to yes. Now finish the move, one person per chair. You can't. Four people, three inside chairs: somebody shares. Maybe the newcomer ends up on a lap, maybe two insiders squeeze onto one chair; either way, two starting places share a destination. Somebody secretly picks a starting chair, the rule moves them, and you're told only where they ended up: at a shared chair, you can't say where they started. That's the pigeonhole principle, which is old and not mine, and the chapter "Why the toll isn't arbitrary" (Section 3) proves it with exactly three conditions on a rule that gives every state a next one: finitely many states that read yes, a yes that stays, and one flip. There's no physics in it anywhere, only counting.

So, the deepest thing I can say about keeping: not every forgetting keeps anything, and wiping a scratch pad keeps nothing. But every keeping pays. On a finite machine that keeps running, keeping pays by forgetting: the step that makes something stay for good lands two states on one, somewhere, maybe not where you happen to be standing, and afterward nothing can tell which one it was. With endless fresh states it can dodge the merge only by spending fresh states it never gets back. A verdict keeps the decision and lets go of the arguing.

Part I proves the count on any machine, prices it on finite ones, and keeps the heat floor as an assumption with its name on it. I never let those two swap places.

An ordinary computational model keeps no account of what it made the case. It is the flat case: moves that never climb, so the toll never speaks. The Thiele Machine is the general thing, a state with an order read off it and a price in every move, and the computer is what is left of one when the order is taken flat.

What the usual models don't count

That's step one, and step two is what it means for the machines people already count with.

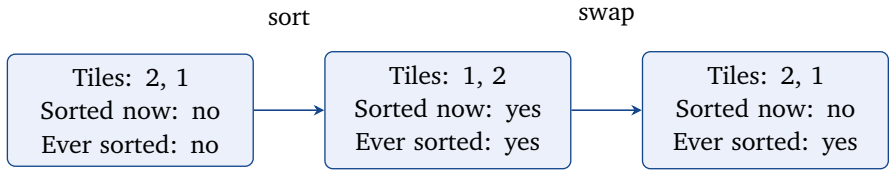
Computer science has always counted what a computation costs. How many steps it takes is time. How many cells it needs is space. The bits a computation erases have been counted too, by Ming Li and Paul Vitányi (1996) and by Erik Demaine and his coauthors (2016). Every one of those models is a flat case. Its states carry no order, so no move of it ever climbs, and nothing in it can charge for the moment a reading goes up and stays. Now look again at what step one found keeping takes. Every time a computation makes something the case and keeps it, a bit set, a list left in order, an answer written down, it pays, in a merge or in fresh states. What it pays isn't the time count: a slow step and a fast one can keep the same thing. It isn't the space count either, though fresh states are one of the two ways to pay it: a machine with a fixed memory pays it by merging, over and over, in the same cells, and its space count never moves. What none of the usual counts charges, as far as I found, is the moment a chosen reading goes up and stays, whichever way keeping it is paid. That's a third thing a computation does, and it's what this model counts.

So here is the Thiele Machine, all of it, math you can reason about, the way Turing's machine is. A set of states, each with a reading into an order: for any two readings you can ask whether one sits at or below the other, and higher means more is settled. A set of moves, each a direction that transforms a state into the next state. A step that takes a state and a move to the next state. A price on each move, a whole number of units of cost. And one rule, which I call the toll: a move that carries the reading somewhere not at or below where it stood costs at least one unit. A machine whose moves never climb is the flat case, and the toll never charges it;

every ordinary computer is one. The smallest case that isn't flat has a reading with two places, blank and written, nothing and a 0, and I write them no and yes. There the toll says that turning no into yes costs at least one, and most of Part I is told with that record, because it is the one you can point at. The toll sits on the raise, not on the merge, so it charges for keeping something whichever way that's paid, in a merge or in fresh states. Two more pieces come in when a result needs them: a counter in the state that adds up the costs and only climbs, which is the ledger, and a window, what an outside observer gets to see. The toll's first name, the one I gave it before I understood it, is No Free Insight.

Put a dial with four positions beside a slider with three stops. ROTATE moves the dial one position around for free and leaves the slider alone. ADVANCE moves the slider toward its last stop, leaves the dial alone and costs one. At the last stop the reading is yes; at the other two it is no. ADVANCE pays when it first reaches yes and still costs one when pressed at the last stop. The price belongs to the button. Look only at the dial: ROTATE moves the picture and ADVANCE leaves it alone. Those are the horizontal and vertical moves of the sliced definition below. There are twelve full configurations, four dial positions for each slider stop. We have met the rules without introducing a checker or a claim.

It computes nothing new; its flat case already computes everything a Turing machine can. What it adds is a place, in the step itself, for what a computation made the case and what that cost. The reading can be any reading into an order at all; the toll charges every step that climbs. The readings this book cares about are the ones that, once up, stay up, because those are the kept distinctions and keeping is what the merge or the fresh states pay for. A cell's "it's a 0 now" stays only if the later moves preserve that value. Likewise, a sorted list can be scrambled by the next write. A flag saying "this list was sorted earlier" answers a different question.



The tiles can lose their order. A separate history flag can keep the fact that they were ordered earlier. Read one, and you have not read the other.

"This claim was checked and closed" can be a historical reading of that kind. The model doesn't care which, and for readings that stay, Part I derives the toll from what keeping takes, on stated premises. A reading that flips back and forth does not by itself retain whether an event ever happened: Part II calls it a toggle and proves it's the latch of no event (Section 11).

The one in the toll is one unit of cost, in the model's own count, and it's one because that's the least a whole-number count can charge and still not be free,

and the toll only says *not free*. A unit of cost isn't a joule. What ties it to heat is Landauer's principle, named, through the merge a raise has to make when only finitely many states read yes, and nothing else.

And there's one thing to hold on to through the whole book, because it's where readers go wrong and where I went wrong for a long time. *The ledger is a count, not a lock.* It doesn't stop anybody from doing anything. It adds up the prices you chose for the moves that ran, writes the sum down where the machine can't lose it, and the toll makes each raise of the reading you chose cost at least one. It doesn't measure merges, fresh states or heat: a merge that keeps nothing can cost nothing, fresh states are taken at every step and the toll asks for a unit only at the raise, and where a merge raises nothing, a cost of one unit per raise leaves that merge out. What ties the count to what keeping takes is a theorem, not a reading of a dial: on a machine with finitely many states reading yes, any price list that charges merges has to charge each raise of a reading that stays at least one. What a kept reading *means*, whether a reading that's up stands for anything, is a separate question with a separate answer, and Part III gives it for one case: an order the step enforces, which is the lock. A lock would work at a price of zero. The price isn't there to make the lock work. It's there because, on a machine where every state has a next one, the raise of a reading that stays merges or takes fresh states, whether anybody writes it down or not.

What I found, what I assumed and what I chose

I didn't invent a machine. I found one. That's how I see this work, and I'll be straight about the difference between that conviction and a theorem before I do anything else. A proof doesn't settle it: showing what follows from the premises doesn't decide whether you found the theorem or made it. That's why it's labelled a conviction. What it changes is how I hold the thing, like something that was already there, which means I can be wrong about its shape and the shape won't care. And to be exact about what "found" covers, it covers the parts the counting forces on you, the order and the merge or the fresh states. The event to record, the price list, and putting the count inside the step are choices, mine, and the three lists below say which is which.

The three lists draw the line between what nothing let me choose, what I had to assume, and what I did choose, because everything after this depends on where it falls. Read the first list as the ground, the second as what the ground is taken to stand on, and the third as the model built on it.

Found: what counting forces on any machine.

- Run a machine that gives each state exactly one next state. It takes fresh states until it first comes back to one it has used; that return lands on its start or is a

merge, two states with one next state; and after it the run only repeats steps it already took. There's no fourth thing a step can do, and a run merges at most once (Theorem 1.1).

- So any run that never comes back to its start merges somewhere or takes a fresh state at every step. A reading that goes from no to yes and never comes back rules the return out, so every run that raises it merges or takes fresh states (Proposition 3.5). The sharper fact is about the move: when only finitely many states read yes, the move that raises the reading merges two states (Theorem 3.1 for a finite machine, Proposition 3.4 when only the yes-states are finitely many). A machine that can stop, with a state that has no next state at all, slips past this, and Part I says exactly how (Theorem 1.2). All of that is counting, with no physics in it.
- Over any machine at all, a record that the computation drives and that never comes down is a latch on one event: it switches on the first time something happens and nothing switches it off (Observation 11.1). That's counting too, given the two conditions in it.

Assumed: what the found list is taken to stand on, in the world.

- A merge of states the machine really could have been in, done in surroundings at some temperature, has a floor under its heat, on average, however slowly it's done, in a memory whose states are alike apart from what they record. That's Landauer's principle, the premise from thermodynamics, named and assumed and never proved here, with its critics named.
- A real device can tell apart only finitely many states in any bounded place, so its fresh states run out. Nobody has measured where that ceiling sits.
- The clean start: a raised record means the event happened only because the record was down at the start, and that start is put in by hand. It's a past hypothesis in miniature. The past hypothesis, the philosopher David Albert's name (2000) for an assumed low-entropy beginning of the universe, is an idea that goes back to Boltzmann; several accounts of why records point to the past lean on it, and every theorem here about records gets its arrow from an assumed clean start (Section 11).

Chosen: what the model adds, each of which could have gone another way.

- To put the account inside the step, as part of what a step is. Two things go there. That the record stays and the computing can't reach it has to: the record is one of the five parts, a reading of the state, and a rule about the state kept by the step holds of every program, where a log a program keeps holds of that program only.

- The toll itself: a whole unit, at least one, on the step that raises a reading, whichever way keeping is paid. On a machine with finitely many states reading yes, for a reading that stays, it follows from a price on merging, and that price is a choice too.
- Which readings matter: the ones that never come down. The toll charges any reading's rise; permanence is what makes the rise a case of keeping, and permanence is a choice. So is one next state for every state, the condition the stopping hole gets around.
- Which event gets a record, and what price list the record pays. The latch leaves both unforced. And on a machine that pays in fresh states, where the write merges nothing, the record stays up because I declare that it does (Theorem 11.3).
- On the machine of Part III, where its flat part ends and its record begins, and that its step never reads the flag: the rest of the state moves by its own rule and the ledger adds the instruction's price, whichever way the flag reads.

The machine Part III builds is one worked demonstration of the third list, on one reading. Nothing in the first list needs it.

What structure is made of

Step three says that a 0 is the smallest structure that sorts, and that the rest of structure is the same thing at a bigger size.

A sorting of a pile into groups is a distinction: these go together, those don't. Keep it and it's a distinction that stays, paid for the same way. An ordering is every pair compared and the answers kept. A fact somebody found out and wrote down is a distinction between the ways things could have been and the way they were, kept. A machine can carry any of those in its state and in its rules, and each one it keeps is paid for the way the 0 is. Part II takes the word *structure* from the people who wrestled with it, and comes out at that: structure is distinctions held in a state.

That gives *insight* an exact meaning, a narrower one than you'd like, and I'll be honest about how narrow. Finding something out doesn't have to pay the merge. Bennett showed that you can write down what you see on a blank page without merging anything, so Landauer's floor isn't owed; the page is what you spend, and it can be handed back while the original is still there. Making it stay is what pays. Learning borrows, committing pays. The insight the toll prices is the committed kind, a distinction made to stay. Whether that narrower thing deserves the word is a fair thing to argue about, and Part I argues it, where learning and committing get pulled apart.

Then Part II asks what follows, on any machine, once a distinction is kept and the keeping is charged. The ledger is itself a distinction that stays: a number that

only climbs takes a fresh value for every unit it counts. A window that shows only part of a state can hide the very distinction that was paid for, and when it shows a step that paid and one that didn't the same, before and after, no price read off the window can be exact. A later step can be entitled to something because a distinction stayed, and the entitlement has an exact count. Over any machine at all, a reading the machine's own steps turn on and nothing turns off is a latch, the blank cell's 0 one level up. And there's a wall no level gets past: no state codes every question about the states.

Established

Step four is the sharpest case, and it's the one I build a machine for.

Give two workers the same messy list. The first sorts it, then runs a separate test that compares each neighbouring pair and reports whether the order is right. The test passes. The second copies in that same sorted list and runs no test. Ask only for the final list. The bytes agree. Reading them can tell you whether they are sorted now; it cannot tell you whether the earlier test ran.

I have specified the output being compared, not every register and every line of program memory. If those retain the difference, the full machine can still tell the histories apart. If a cleanup makes the full endpoints agree too, then nothing in those endpoints can answer which history occurred. An ordinary computer can retain that evidence when its program and interface require it. The question is what this task asks it to keep.

Established is the name for the difference. An answer is established when a check that could have said no said yes, about that very answer, and the result was made to stay. That last part is a kept distinction, a 0 in some cell that says "checked," and it's paid for the same way as every other 0. Part III opens by taking that apart (Section 14): what "could have said no" means, what settling a question is, and why the people who thought hardest about knowledge kept landing near it.

How far the example reaches, to other models of computation and to universal machines that run it as a guest and earn its flag again, is proved too; Part III states it (Section 16) and the appendices build it. That's the reach of the case, not what the book is about.

How the book goes

Part I is what keeping costs. It covers a distinction that stays, and what every step of every machine does to the states it has used; the machine, stated once, with the smallest record that can carry a charge, and the toll written into its step as one rule;

whether the rule is forced, and on which premises; the record anywhere in an order; and why finding out only borrows, where committing pays.

Part II is structure, as kept distinctions: the word itself, and then what follows on any machine once a distinction is kept and the keeping charged: the ledger, the window, entitlement, the latch, the shadow, and the wall no level gets past.

Part III is the sharpest case: what established means, the machine that keeps it, how far the machine reaches, five real systems that keep records and whether they price the write, the thesis about which kept distinction is worth paying for, and what its ledger has to do with heat.

Part IV brings the results together: what each establishes, which conditions it uses, and how the constructions fit. After that comes how I got here, for anyone who wants the story, and then the appendices, which carry the universal machines and the machines built from parts.

PART I

What keeping costs



The cut remains.

Part I is step one, and the rule step two needs. It starts at the smallest structure that sorts: a cell that's blank, and then a 0. First comes what keeping takes, before any price list: what a distinction that stays is, what every step of every machine does to the states it has used, and what it costs for something to go from nothing to something and stay. Then the smallest machine that writes what keeping takes into its own step, as one rule, the toll. Then whether the rule is forced, and on which premises. Then the one thing that looks like it ought to pay a merge and doesn't have to: finding out, which borrows fresh states instead.

CHAPTER 1

A distinction that stays

A cell can be blank, or 0, or 1, and somebody has drawn the lines between those. Write a 0 in it and it stays a 0, and that's the event this whole book is about, nothing becoming something; this chapter is what it takes, before any machine and before any price list.

What this chapter counts is a distinction that stays. A cell on a tape that reads 0, and not 1, and not blank, is one; a bead sitting in the left well and not the right is one; a flag that is up is one. Anything the world can tell apart from something else and keeps telling apart counts. It needs distinguishable arrangements, a reading that groups them, and rules or a physical mechanism that retain the chosen answer. The bead uses a hill for the last part. The point said so in pictures ("Where a 0 comes from"). Here I want two things exactly: what every step of every machine does to the states it has used, and what it costs for a state to be taken at all. Both answers are proofs, and every one of them is short enough to follow on the page. Nothing in this chapter needs a price list, and the toll, named up front, is stated exactly only in the next chapter, which writes what keeping takes into a machine's step; the one after asks whether it had to.

1.1 Every step does one of three things

Forget the cell for a moment, and forget finiteness. Take any machine at all, with any number of states and any rule that gives each state exactly one next state, and watch it run from any start. Keep a list of every state it has been in. Now look at the next step. There are exactly three things it can do.

It can land on a state the run has not used yet, which uses up a fresh state on this step. The only way to stop using up fresh states is to come back to a state the run has used: to the start, and a run that does that has kept nothing, or to another one, which is the next thing a step can do.

It can come back, for the first time in the run, to a state the run has used before, and not the state it started from. Then it came from a different state than the one that first got there, so two different states have led to the same place, and that is what I mean by a merge: a literal collision, two different states with the same next

state. Under Landauer's premise a merge of states that were really possible carries a floor of heat, and no amount of going slowly lowers it.

Or it can land back on the very state it started from, or take a step the run has already taken. Then the machine is going round a loop, and this is certain: a machine with one rule for the next state, once it stands where it stood before, does again what it did before, forever. Every later state is one it already had, and every later step is one it already took, and it is computing nothing new.

There is no fourth thing a step can do, and every run has the same shape: fresh states, then at most one return, and after it nothing but steps already taken. The proof is one line: the first time a run comes back to a state it has used, either it came back to its very first state, which is a pure loop from the start, or the step that brought it back lands on the same state as the step that first got there, which is a merge. So a run reveals at most one pair of distinct visited sources with a shared successor. Later laps can visit that same landing again; they reveal no additional pair.

□ **Theorem 1.1** (The step law)**.** Let a step give every state of a set exactly one next state, where the states can be told apart, and run it from any start x_0, writing $x_0, x_1, x_2, \ldots$ for the run. Either every step is fresh, landing on a state the run has not visited, or there is a first step that lands on a visited state, from x_n to $x_{n+1} = x_j$ with $j \leq n$, and then: every step before it is fresh; it lands on the start ($j = 0$), or it is a merge, coming from a different state than the one that first reached x_j, so that two distinct states share a next state; and from x_j on the run is periodic, so every later state is one the run already had and every later step, from x_k to x_{k+1} with $k > n$, is a step the run already took. In a run at most one pair of distinct states shares a next state, x_{j-1} and x_n, when $j > 0$. Later laps may revisit the shared destination, but introduce no further pair of distinct visited sources with a shared successor.

Proof. If no state repeats, every step is fresh. Otherwise take the first step that lands on a visited state, from x_n to x_{n+1}, and the position $j \leq n$ with $x_j = x_{n+1}$; it is unique, because $x_0, \ldots, x_n$ are all different. Every earlier step is fresh, by the choice of n. If $j > 0$, compare x_{j-1}, which first reached x_j, with x_n: both land on x_j. They differ, because $x_n = x_{j-1}$ with $j - 1 < n$ would be an earlier repeat. So two distinct states share a next state: a merge. Either way, since one rule gives the next state, $x_{n+1} = x_j$ gives $x_{n+1+k} = x_{j+k}$ for every k, so the run repeats with period $p = n + 1 - j$ from position j on. For $k > n$ the step from x_k to x_{k+1} is then the step from x_{k-p} to x_{k-p+1}, taken earlier, and every later state is one of $x_j, \ldots, x_n$. No later step introduces a new such pair: it repeats a transition already seen. If $j > 0$, the

only two distinct states with a shared next state in the run are x_{j-1} and x_n; if $j = 0$, there is no such pair. ■

The step law is "nothing is kept for free" as a theorem with two hypotheses: you can tell states apart, and each state has one next state. The only step that neither uses up a fresh state nor merges is a step on a loop, and a loop makes nothing new. Anything new is a state never used before or one that two states have led to, and that's what keeping takes. Physical dissipation is a separate question, answered here only under the specified thermal dynamics and distributional premises. That one is heat against time: I prove it below for Bennett's walker, and the cited physicists prove related bounds for specified thermal stochastic dynamics. The reading that stays up is the way to force "new": a reading that was no and is now yes can never come back to the state it started on, so the run can't be a loop through its start. Its first return to any state it has used, if it ever has one, is a merge; if it never has one, every state it takes is fresh. Two chapters on, that's the first part of Proposition 3.5, where the fresh states get counted exactly. (After a merge the run can settle onto a loop among states that read yes and idle there, making nothing new. The keeping was paid for at the merge.)

There is one hole, and it's worth seeing exactly. Take a machine that stops, with two states. The first reads no, the step takes it to the second, which reads yes, and the second has no next step at all. The reading is raised, there is no merge, no fresh state after that, nothing. The step law assumed every state has a next state, and a machine that stops its clock dodges it. Close the hole either way and what keeping takes comes back. Say the halted state's next state is itself, which is the usual way to give a stopped machine a step, and the halt is a merge: the moving state and the halted state both land on the halted state. A physical device can also hold a reading through its storage arrangement. What that arrangement costs is a question about the device; the partial transition rule has not answered it. Here I can settle the state-counting question once holding is included in a total next-state rule.

Theorem 1.2 (Halting)**.** A partial step, one that may have no next state, can raise a permanent reading on two states with no two states sharing a next state. Totalising it, with the halted state stepping to itself, makes the raise a merge: the halted state and the state that steps onto it share a next state. In general, once every halted state steps to itself, any other state that steps onto a halted state shares that landing with it. □

Proof. States a and b, reading no on a and yes on b; a steps to b and b has no next state. No two states step to the same state, because only a steps at all. Totalise:

b steps to b, and now a and b both step to b, a merge of two distinct states. The general case is the same two lines: if $c \neq h$ steps to the halted state h and h steps to h, then c and h share a next state. ■

Stepping the halted state to itself is one way to totalise. Step it to a fresh sink state instead and the merge only moves: the sink steps to itself, and the halted state lands there too. On finitely many states reading yes, every way to totalise that keeps the reading up merges somewhere (Proposition 3.4, two chapters on). An endless counter offers another arrangement: the reading stays put while the full state keeps changing. I want to count that arrangement now, because the screen and the box are doing different things.

The camera stops; the box keeps ticking

At three o'clock the screen says 42. The program is finished. Stop filming and you have the computation from start to finish. Leave the camera running and the box is still on the desk. At 3:01 it says 42. At 3:02 it says 42.

Let the box's next-state rule keep ticking. It can stay exactly where it is, or change something behind that screen. Imagine a wheel inside it, turning from "first holding state" to "second holding state" while the display never moves. The screen gives you 42 each time. Open the box and the wheel may tell you which tick you are looking at.

Start the count one tick before completion. That unfinished state is u. Call the total next-state rule F, so $F(u)$ is the first completed state and $F^2(u)$ is the next one. Gather the full states that keep the answer into a set H, and require the rule to stay in that set once it enters. Give H room for K configurations. Two bits can have four configurations; these are the arrangements of the box I am counting.

Now ask it to hold for L samples while every distinct source in that stretch, including u, has a distinct successor. Suppose two holding samples repeated the same full state. Follow their equal successors backwards. Each cancellation takes one step off both sides, until the unfinished state equals a state already holding the answer. That is impossible. Every holding sample needs its own full configuration.

□ **Theorem 1.3** (Room to hold an answer)**.** Let $F : X \to X$ be total, with distinguishable states, and let $H \subseteq X$ have K elements. Suppose $u \notin H$, $F(u) \in H$, and $F(H) \subseteq H$. For $L \geq 0$, put

$$D_L = \{u, F(u), \ldots, F^{L-1}(u)\}, \qquad D_0 = \varnothing.$$

If F is one-to-one on D_L, then the L held states $F(u), \ldots, F^L(u)$ are distinct, and $L \leq K$. In particular, among $u, F(u), \ldots, F^K(u)$ there are two distinct states with the

same successor. If instead F is one-to-one everywhere and the held region is allowed to be infinite, the orbit from u never repeats.

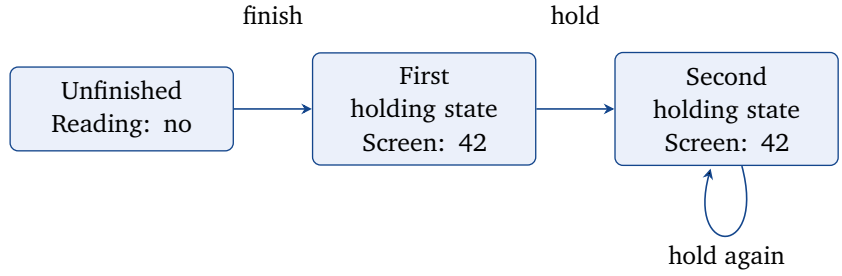
Proof. Suppose the same full state occurs twice in the displayed stretch, first at position i and later at position j. If the earlier position is the start, I have already equated the unfinished state u with a held state, which is impossible.

Otherwise look one tick behind both occurrences. Those two predecessors belong to D_L, and they have the same successor. The one-to-one rule on D_L says that the predecessors must therefore be equal. Move back another tick on both sides and repeat. You eventually reach position zero on the earlier side and position $j - i > 0$ on the later side. In symbols, the repeated backward comparison has given $u = F^{j-i}(u)$. The left side is unfinished; the right side has entered H and stayed there. That is the contradiction.

So all L held samples are distinct. There are only K states available in H, giving $L \leq K$. If the sources through $F^K(u)$ had no shared successor, use $L = K + 1$: you would need $K + 1$ held states where only K exist. Finally, when the step is one-to-one everywhere, the backward comparison works for any two finite positions in the run. No repetition is possible, no matter how long you keep watching. ∎

Draw the smallest delayed example:

$$u \longrightarrow h_1 \longrightarrow h_2 \longrightarrow h_2.$$



Only the unfinished state enters the first holding state. The two holding states share a successor. The collision is later than the raise.

The screen says 42 at both h_1 and h_2. Only u leads to h_1; the entry is clear. The two arrows meet a tick later, when h_1 and h_2 both lead to h_2. With K holding configurations, the run exposes a collision within its first $K + 1$ transitions from u.

You can reach the bound exactly. Hand me a guest run that halts after $T \geq 1$ steps, and let me use $K \geq 1$ holding states. Line them up:

$$\text{prep} \longrightarrow a_0 \longrightarrow \cdots \longrightarrow a_{T-1} \longrightarrow h_1 \longrightarrow \cdots \longrightarrow h_K \longrightarrow h_K.$$

The preparation state shows nothing yet. Each a_j shows that step of the guest run. Every h_j shows its final answer. There are $1 + T + K$ full states. Starting at the last

running state, the first K sources have different successors. Add the next source and the arrows meet. The box uses every holding state before exposing the collision.

The camera can stop at 42. The box that keeps showing it still has a next step. Include those steps and you can count exactly how long it can keep its full states apart. [1]

1.2 What a kept distinction is

Now I turn to the cell. Blank is one of its logical states, alongside zero and one. Give it a write-once rule: write zero when blank, and leave either written value alone. That is what "stays" means in this example. The same rule sends blank and zero to zero, so it has a collision. Whether that collision removes uncertainty depends on which inputs were possible in the preparation. Its physical heat needs the device and protocol as well.

And the cell isn't the lowest level. Describe whatever holds it, a bead in two dips, the molecules that jostle the bead, whatever holds those, and you've described states that can be told apart and a way they change: a machine of the same kind, charged the same way, because a merge one level up forces, one level down, either a merge or a state never used before, as long as the level above is exact about the level below and the level below stays among the states the one above describes (Theorem 1.10). Every level you can describe is one of these. Under all of them there's nothing told apart at all, and that's the nothing a blank cell stands for.

Preparation is a resource assumption too. A supplied known blank is a specified initial condition, not evidence that a positive heat cost was paid earlier. If an apparatus must reset an unknown input distribution for reuse, its reset can incur an entropy-loss bound under Landauer's premises. Reeb and Wolf's precise finite-dimensional formulation, for example, assumes an initially thermal reservoir uncorrelated with the system and joint unitary evolution; its heat bound is conditional on that setup. Inferring a past or future payment from a known state alone would need more assumptions about preparation, correlations, reuse and the thermal protocol.

The cell has three states: blank, zero, one. The reading is "the cell is definite," no on blank and yes on the other two, and a written cell stays written, so the reading is permanent. The move is "write a 0," which takes blank to zero and leaves a written cell alone. Now look at where each state lands. Blank lands on zero, and zero lands on zero, so two different states have led to the same place, and writing a 0 on nothing is, by itself, a merge. For an input distribution with positive uncertainty between blank and 0, this map removes entropy. Landauer's thermal premises then

[1] This is a state-capacity count. A ledger price requires a pricing rule; a heat bound requires physical premises.

give a heat bound for that loss. For a known blank, the input entropy is zero and this argument gives no positive bound; it also implies no earlier payment for knowing. The history in which the cell was blank and the history in which it already held a 0 are now the same history. This is not an analogy to the step law; it's the step law with three states filled in.

The obvious way round it is a fourth state that remembers: "0, and it was blank before." Now blank lands on a state nothing else has landed on. But the remembering state has to go somewhere on the next write too, and it's written, so it has to stay written. Every written state already has something landing on it: zero and one land on themselves, and the remembering state is where blank landed. So the remembering state has to share. In the smallest version it lands on itself, and then blank and the remembering state land on the same state. No finite extension of the cell gets out of the merge, and the reason is counting (Proposition 3.4, two chapters on), not bad luck. The only way to write with no merge, once every state has a next one, is a supply of fresh states that never runs out: a state never used before for every write, forever, such as a new cell of an endless tape for each one (it comes back two chapters on, as the first of three ways out of a theorem). That isn't getting out of what keeping takes. It dodges the floor by taking a state it has never used, at every write, without limit, and each write can dissipate under the stated thermal-model assumptions. Or, in a form I need below, a tape that stretches both ways with the machine stepping one cell along it. Every cell has one before it and one after it, nothing ever shares a landing, the reading "past the start" goes up on the first step and never comes down, and the machine never returns. This is the bilateral shift, and it pays at every step by taking a cell it has never used.

Theorem 1.4 (The write of a 0 merges, and only fresh states avoid it)**.** (1) On the three states blank, zero, one, with reading "definite," writing a 0 is permanent, raises the reading from blank, and merges blank with zero. (2) Adding a fourth state that remembers the cell was blank, with written cells staying written, does not avoid the merge: on the four states the write still merges two distinct states. The same holds on any finite set of states. (3) On an endless tape, writing adds one cell and is one-to-one; on the integers, the step "one cell along" is a bijection, raises "past the start" on the first step, keeps it up, and after any positive number of steps is away from where it started. □

Proof. (1) Blank and zero both step to zero, and blank is not zero; zero and one step to themselves, so a written cell stays written. (2) Blank steps to the remembering state, and permanence sends each of the three written states, zero, one and the remembering state, to a written state. So four states land among the three written

ones, and two of them share a landing. Whenever only finitely many states are written the same count works: permanence sends the finitely many written states among themselves, blank lands among them too, so one more state lands among them than there are, and two share a landing (Proposition 3.4, two chapters on, states it in general). (3) "Add one" on the counting numbers is one-to-one and never returns to zero; "add one" on the integers is one-to-one and onto, sends 0 to 1, sends every number that is at least 1 to one that is at least 1, and after $k > 0$ steps has moved by k. ■

The theorem leans on one thing, and I want it in the open. The write is a move defined on every state of the cell, written ones included, and a written cell stays written, so the state blank merges with is a cell that was already written. A write defined only on blank, with the machine simply stopping when it meets a written cell, merges nothing. That's the stopping hole from above (Theorem 1.2): it buys the 0 by having no next step at all.

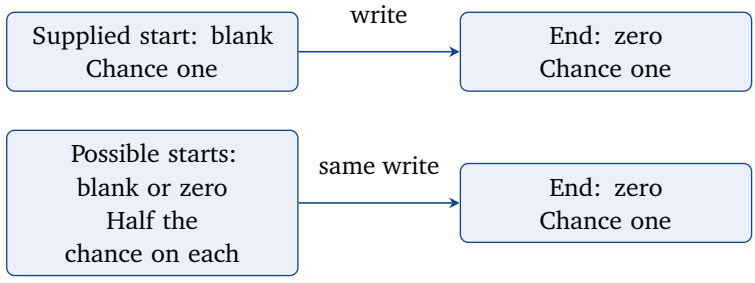
The theorem buys this: a fact is a distinction that stays, and a kept distinction is a state the run has used and can't give back. The machine is the thing whose own state includes the used states it can't give back.

It doesn't stop at the cell. The tape is a set of distinctions that stayed. So is the rule for the next state, and so is the question you ask of a state. Each of the five parts of the model is itself a definite thing where there could have been anything; the model doesn't charge for them, but they're the same kind of thing it charges for. And the model can run itself: a host that obeys the toll and runs a guest move for move, carrying the guest's prices on its own ledger, pays at least one unit there for every step where the guest's record went up, and Part III states one (Section 16; Appendix C builds it). The tower is the same kind of machine above and below. What no level does is hold all the questions you can ask of it: for any way of coding readings as states, there's a reading no state codes, the one that answers each state with the opposite of what that state's own coded reading says of it. That's Cantor's diagonal, the shape of Russell's 1902 letter to Frege (Section 6), and the subject of Section 13. Put the two together and you have the regress exactly: every level runs the one below that way and pays again, in its own ledger, for every rise of the lower level's record, and no level ever holds every question about itself.

□ **Theorem 1.5** (No level holds its own questions)**.** For any set of states and any assignment of a yes/no reading to each state, some yes/no reading on the states is assigned to no state.

Proof. Take the reading that says, of each state, the opposite of what that state's assigned reading says of the state itself. If some state were assigned it, that state would answer itself both ways. ∎

Write a 0 on a cell known to be blank and the entropy-loss premise sets no positive floor: the input and output entropies are both zero (Theorem 3.17). The thermal implementation can still have a finite-time cost under the specified dynamical assumptions. A writer that must handle both blank and 0 with one protocol faces a different question: under the uniform two-input distribution its logical map removes one bit, so Landauer's premises give an average heat bound. This does not charge that average to every particular input.



The first preparation loses no uncertainty. The second loses one bit. The transition rule is the same; the input preparation is different.

Keeping an exact answer among N independently possible alternatives needs at least N representational states, or $\lceil \log_2 N \rceil$ bits in a fixed-length binary encoding. That is a capacity bound. Identifying an arbitrary one of three possibilities can require two binary questions in the worst case; testing just "is this cell blank?" is a different, binary task. Neither count proves that a known blank was prepared by an erasure or that a heat bill has already been paid.

1.3 A reversible machine keeps nothing

Bennett's reversible machine never lets two of its states lead to the same next state; that's what reversible means. Put one on finitely many states and watch it. By the pigeonhole it has to revisit a state within as many steps as there are states. The step law says the first revisit is a merge or a return to the start, and a merge is impossible for it, so it returns to its start, and once it's back at its start it does everything again. Any reading it raised has come back down. A reversible finite machine with a next state for every state keeps nothing, and that's a theorem, not a slogan. One that stops slips past, as the stopping hole above says, and keeps what it halted on. It is the recurrence Zermelo threw at Boltzmann (Section 3 tells that story), in its finite form, proved from the step law and the pigeonhole and nothing else. What Bennett's machine keeps is the fresh states it's handed: on an endless tape it can

keep its whole history and never come back, taking a state it has never used at every step, and the tape was paid for before the first step by whoever built it. A reversible machine dodges the merge and its floor. For a specified thermal implementation, a finite-time change of distribution can also require entropy production. Fresh states and physical dissipation are different quantities; contact with surroundings alone supplies no numerical bound.

□ **Theorem 1.6** (Reversible keeps nothing)**.** Let a one-to-one step act on finitely many states. From any start the run returns to the start within as many steps as there are states, and any reading that is no at the start and never goes from yes to no is no at every step.

Proof. Among the first (number of states) plus one states of the run, two are equal, by the pigeonhole. Take two equal positions and cancel the one-to-one step back from both the same number of times, until the earlier one reaches the start; the later one then equals the start. So the run returns to the start after at most that many steps, and from then on repeats. If the reading were yes at some step, it would stay yes from there, in particular at a later multiple of the period, where the state is the start, which reads no. ■

1.4 Noisy steps

Real states are noisy. The bead hops on its own. So take the step to be a table of chances: from each state, the chance of landing on each state next, every row adding to one. A permanent reading is a set of states you never leave once you're in it. And "no merge" has an exact noisy meaning: the step preserves the even spread. Start with everybody spread evenly over the states, take one step, and they're still spread evenly; which is to say every column of the table adds to one as well as every row. Such a step squeezes nothing on average.

Then the cell's count goes through with chances in place of people. A step that preserves the even spread cannot raise a permanent reading from any state outside it, and not merely with small chance but with none. Count the chance flowing into the yes-states from everywhere. Column by column that's one per yes-state, so it equals the number of yes-states. The yes-states alone already send exactly that much, because they never leave. What's left over for the outside states is zero, and it's a sum of chances, so each is zero. The count never used that the spread was even, only that every state had some chance and the step gave the spread back. A lopsided memory that keeps its own lopsided spread can't raise a reading that stays either: weight each state by its chance and the same sum comes out.

And it runs the other way too. A step that can raise a permanent reading does not preserve the even spread, so after one step from the even spread the crowd is more bunched up than it was. Here is a way to measure that bunching. Draw a state twice, independently, using the same spread of chances. Ask whether the two draws match. For two equally likely states, the matching pairs have chances one quarter and one quarter, so a match occurs half the time. Put three quarters of the chance on one state and one quarter on the other: the matching chances are now nine sixteenths and one sixteenth, adding to ten sixteenths. Bunching has made a match more likely.

In general, a state with chance q contributes q^2 to this matching chance: q for the first draw, times q for the independent second draw. Add those squares over all states. I call that total the *collision weight*. The step just described raises it above the value for an even spread.

There is another measure of the same change, Shannon entropy, which measures uncertainty in the spread. It is largest when all states are equally likely, so leaving that even spread lowers it. This is the quantity that enters the heat bound under Landauer's premise. Section 3.7 works through its meaning in bits; the proof here establishes the drop. Finally, when a step preserves the even spread, it cannot increase the collision weight of *any* starting spread. That last claim needs a calculation too.

Theorem 1.7 (Noisy steps)**.** Let a step on finitely many states be a table of chances with rows adding to one, and let a set of states be closed: from inside, the chance of leaving is zero. (1) If every column also adds to one, the chance of entering the closed set from any state outside it is zero. (2) If some state outside can enter the closed set with positive chance, then some column does not add to one, and one step from the even spread strictly raises the collision weight, the sum of the squared chances, which is least for the even spread, and it leaves a Shannon entropy strictly below the logarithm of the number of states. (3) If every column adds to one, one step never raises the collision weight of any spread. □

Part (1) needs only the closed set to be finite, the rest as large as you like, and part (3) needs no finiteness at all; part (2) needs the even spread, so it needs finitely many states.

Proof. (1) Count the total chance sent into the closed set. Each of its columns sums to one, so the total equals the number of states in that set. The rows belonging to states already inside send all their chance inside, and they already supply that entire total. Nothing remains for an outside row to contribute. Because every contribution is nonnegative, each outside contribution is zero.

(2) Suppose instead that an outside state contributes a positive chance of entry. Then the conclusion of (1) fails, so its column-sum condition cannot hold: at least one column sum differs from one. Call the number of states N and the sum of column j c_j. Starting evenly puts chance $1/N$ at each source, so the chance at destination j after the step is c_j/N. The column sums themselves add to N, because the N rows each add to one. Expand the following squares and use that total:

$$\sum_j (c_j - 1)^2 = \sum_j c_j^2 - 2\sum_j c_j + N = \sum_j c_j^2 - N.$$

At least one square is positive. Thus $\sum_j c_j^2 > N$. Dividing by N^2 says that the new collision weight is greater than $1/N$, the old even spread's collision weight. Now compare the uncertainty before and after. Write $q_j = c_j/N$ for the new chances. At least one differs from $1/N$. Shannon's entropy weights each possible state's logarithmic uncertainty by its chance. Using the natural logarithm, written ln, its value is

$$H(q) = -\sum_{j:q_j>0} q_j \ln q_j.$$

The sum leaves out states with zero chance; they contribute nothing. An even spread has N equal terms, giving $H = \ln N$. Subtract the new entropy from that value. Since the chances add to one, you can put $\ln N$ inside the sum:

$$\ln N - H(q) = \sum_{j:q_j>0} q_j \ln(Nq_j).$$

I need to show that this difference is positive, even though some individual terms may be negative.

Use the elementary logarithm inequality $\ln y \le y - 1$, with equality only at $y = 1$. One way to see it is to look at $y - 1 - \ln y$: its slope is $1 - 1/y$, negative below one and positive above one, so its least value is zero at one. Apply the inequality with $y = 1/(Nq_j)$ and multiply by q_j. Rearranging gives

$$q_j \ln(Nq_j) \ge q_j - 1/N,$$

with equality only when $q_j = 1/N$. A zero chance contributes zero to the entropy difference, which is strictly greater than its comparison term $-1/N$. Add the comparisons over all N states. Their right sides sum to $1 - N/N = 0$. At least one comparison is strict because the new spread is not even, so the entropy difference is positive.

(3) Fix a destination column. Because its entries add to one, its new chance is an average of the old chances, with those entries as weights. An average of squares is never less than the square of the average. To see why, call the weights w_i, the old

chances p_i, and their weighted average m. Expanding gives

$$\sum_i w_i (p_i - m)^2 = \sum_i w_i p_i^2 - m^2 \geq 0, \qquad \sum_i w_i = 1, \quad m = \sum_i w_i p_i.$$

So the new chance squared is at most the weighted sum of the old chances squared. Add this comparison over all destination columns. Each old chance squared gets weights from its whole row, whose sum is one. The result is exactly the old collision weight, bounding the new one from above. ■

1.5 Unitary steps

The quantum version needs a different picture. Draw a sheet through the centre of a room. Call every direction along the sheet YES, and the direction straight through it NO. Turn the whole arrangement without stretching it. Can the sheet stay within its old plane while the direction through it turns into that plane? No. The turn keeps right angles. If the sheet still fills that plane, the direction that was at right angles to the whole sheet is still at right angles to it.

Here is what carries from that picture into the quantum statement. A quantum state is described by a vector. A reversible quantum step preserves lengths and overlaps between vectors; such a step is called *unitary*. The YES part is a *subspace*: a collection of directions that also contains their linear combinations. The directions with zero overlap with every YES direction make up its *orthogonal complement*. These are the NO states in this argument. Permanence means that a vector in the YES subspace stays in it after the step.

There is one detail to settle before the right-angle argument works. The old YES directions must fill the whole YES subspace after the step, leaving no missing direction. Choose a finite list of directions from which every YES vector can be assembled, with none of the listed directions assembled from the others. Such a list is called an independent spanning list, or a basis. A unitary step keeps the list independent, since any relation between its images could be reversed to give the same relation before the step. The images are still inside the YES subspace, and there are just as many independent directions as before. They therefore still span it. That is why the step fills the YES subspace, rather than merely landing somewhere inside it.

Now take a NO vector. Before the step it has zero overlap with every YES vector. After the step those overlaps are still zero, and the stepped YES vectors still fill the whole YES subspace. The stepped NO vector is therefore still NO. Repeat the step and the same argument applies again.

Finite dimension was used exactly once: a full finite list of independent YES directions still fills the YES subspace after the step. Drop it and the bilateral shift,

which is unitary on an endless ladder of states, raises "past the start" on the first step and keeps it forever, taking a state it has never used every time. A reading that stays, which no unitary step can raise in finite dimension, is paid for either by fresh states, a ladder of states that never comes back around, or by a step that squeezes the even spread, which, when the squeezed states were really possible, owes Landauer's floor. Not every step that fails to be unitary squeezes: one that keeps the even spread, as pure dephasing and an unread projective measurement do, can't raise a reading that stays either. The balance argument is the same one used for noisy steps. Choose a complete set of mutually perpendicular directions, some filling the YES subspace and the rest filling its orthogonal complement. Spread the chances equally over those directions. This is the fully mixed state: choosing a different complete perpendicular set gives the same mixture.

Suppose there are N directions altogether and m of them are YES. The mixture has chance m/N of YES. A step that preserves this mixture must leave that chance at m/N. But the YES part already supplies all of it: it started with weight m/N, and permanence keeps that whole weight inside YES. Every contribution from the other directions is nonnegative, so none can contribute any chance of entering YES. You can include any chosen NO vector among the perpendicular directions of the complement. The same balance then rules out entry from that vector. That is why the outside cannot get in. In finite dimension nothing is kept without one of the two. Which way a quantum measurement pays when its outcome is kept isn't settled here. The collapse models people test keep the even spread too, so the heat the experiments look for is the heat of the noise those models add, not a Landauer price on a kept outcome; I come back to them below.

□ **Theorem 1.8** (Unitary steps)**.** Let a unitary map a finite-dimensional subspace into itself. Then it maps the orthogonal complement into itself: a state orthogonal to the subspace remains orthogonal after every step.

Proof. Call the step U and the YES subspace Y. Choose a basis of Y, meaning an independent list from which every vector in Y can be assembled. The step sends that list to an equally long independent list inside Y, so its images are another basis of Y.

Take any vector x orthogonal to Y. To test whether Ux is still orthogonal, choose any vector y in Y. Because the images fill Y, there is a vector z in Y with $Uz = y$. Unitarity preserves the overlap, written with angle brackets, so

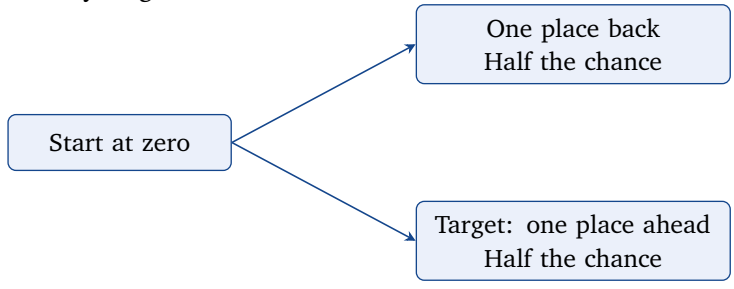
$$\langle Ux, y\rangle = \langle Ux, Uz\rangle = \langle x, z\rangle = 0.$$

The last equality holds because x is orthogonal to every vector in Y, including z. The choice of y was arbitrary, so Ux is orthogonal to all of Y. This is the same condition I started with, and it survives every further step. ∎

1.6 Going faster gives off more heat

Everything so far was what keeping takes. What comes next is a dissipation result for a particular stochastic model. Bennett's Brownian computer is a walker on a line that is pushed forward a little: at each tick it steps forward with some chance and back with the rest. Its speed is the difference between the two. A forward translation is one-to-one, and so is a backward translation. The random choice between them is a stochastic rule, not one deterministic translation; the deterministic merge count does not apply to it as though it were. The entropy it gives off per tick is Schnakenberg's formula for a biased walk, the speed times the logarithm of the forward-to-backward ratio, and that number is strictly positive at any speed other than zero, because both factors have the same sign. The only walker that gives off nothing is the one with no drift, as likely to step back as forward. This is the point that going slower doesn't get you out of paying, now as a theorem: at any speed you actually run it, every tick gives off heat on average.

The exchange rate has a floor as well. Heat per net step forward, counted in units of kT, is the logarithm of the forward-to-backward ratio, and that logarithm is at least the speed. So walking a distance costs at least distance times speed in heat and takes distance over speed in time, and heat times time is at least the distance squared. At low speed, halve the speed and the heat roughly halves while the time doubles; the product never drops below the square of how far you went. Going slowly makes the mean heat for a specified amount of directed progress smaller and its drift time longer. These are averages. With no bias, the walker still moves, and it can reach a nearby target on the first tick.



The mean displacement is zero. Half the paths reach this target on the first tick. Zero drift does not mean no movement or no completed paths.

Below, distance divided by speed is the time scale for the mean displacement to advance that distance. It is not the duration of every path that happens to finish.

□ **Theorem 1.9** (Heat against time)**.** For a walker stepping forward with chance p and back with chance $1-p$, with $0 < p < 1$, the entropy it hands to its surroundings per tick, $(2p-1)\ln\frac{p}{1-p}$, is zero at $p = \frac{1}{2}$ and strictly positive otherwise. With speed $v = 2p-1$ in $(0,1)$ and distance $N \geq 0$, the mean heat in units of kT for drift progress N, $N\ln\frac{1+v}{1-v}$, times the drift time N/v is at least N^2.

Proof. For $p > \frac{1}{2}$ both factors are positive and for $p < \frac{1}{2}$ both are negative, by the monotonicity of the logarithm; at $p = \frac{1}{2}$ the first factor is zero. For the floor, $\ln(1+v) \geq 0$ and $\ln(1-v) \leq -v$ since $\ln y \leq y-1$, so $\ln\frac{1+v}{1-v} \geq v$, and the product ■ is N^2 times that logarithm divided by v.

That is the surroundings' share of the entropy production. The whole of it adds the change in the walker's own spread, which on a line keeps growing but ever more slowly, so per tick that share goes to zero and the long-run rate is the heat's.

The sharp floor is twice that. The difference $\ln\frac{1+v}{1-v} - 2v$ is zero at $v = 0$ and its derivative, $\frac{2}{1-v^2} - 2$, is never negative on $(0,1)$, so the logarithm is at least $2v$ and the product is at least $2N^2$. I state N^2 because the one-line bound gives it, and what matters is the square, not the factor in front of it.

Keeping something costs, on top of that, either a merge, which under Landauer's premise owes a floor of $kT\ln 2$ per bit that no slowness removes, or a state the device has never used before. Taking a fresh state escapes the floor and does not settle the implementation's finite-time dissipation. And a finite set of states, where every state has a next one, forces the merge.

1.7 Every scale pays the same way

One more question, and it's the one that decides whether any of this is about the world or only about my pictures. A state is a description. The dial on a lab bench is a state, and so is each atom in it, and so is each atom together with the light it gave off. Describe the same thing coarser or finer and you get different states. Does what keeping takes change with the zoom?

It doesn't, and the reason fits in one picture. Draw the fine states, and draw a fence around each group of them that counts as one coarse state. Say the coarse picture merges: two coarse states step to one coarse state. Then, when the fine states are finitely many, the fine step can't keep them all apart either. Count them. If the fine step sent different fine states to different fine states and none of them stepped out of the fenced set, it would be a reshuffle of the fine states, with every fine state landed on exactly once afterwards. But some fenced group is now landed

on by nothing, because the coarse step missed a coarse state (two coarse states went to one, so on a finite set some coarse state is stepped to by nothing), and so nothing lands on any fine state in that group. A reshuffle leaves no state out. So something gave: the fine step merges too, or some fine state stepped out of the fenced set, or the fenced set has no end. The three ways come back one level down, though stepping out of the fenced set isn't taking a fresh state; it's where the coarse picture stops tracking the fine one.

Theorem 1.10 (Every scale pays the same way)**.** Let a fine machine have one rule F on a set X of states, let A be a set of its states, and let a coarse description be a map π from X to a set S of coarse states that takes A onto S, with a coarse rule f on S such that $\pi(F(x)) = f(\pi(x))$ for every x in A. If the coarse rule merges two coarse states, then at least one of these holds: F merges two states of A; F takes some state of A outside A; or some run of F from a state of A stays in A and never comes back to a state it has visited. □

Proof. Suppose the fine machine avoids all three ways out. It stays inside A, keeps different states apart, and every run eventually visits a state it has visited before. Follow one run to its first repeat. The two visits have the same state, so their predecessors must agree too: different predecessors are never allowed to land together. Walk backwards like that until the earlier visit reaches the start. The later visit has reached the same start. The run is a loop from the start, with no loose approach leading into it.

Now watch that loop through the coarse window. Every fine step gives the coarse step specified by f. When the fine state returns to its start, its picture returns too. Every coarse state has a fine state above it, because π takes A onto S. So every coarse state also lies on a loop.

That rules out a coarse merge. Suppose two different coarse starts had the same next state. Call the lengths of their loops L_1 and L_2. Wait $L_1 L_2$ steps from the starts: that is a whole number of circuits for both loops, so each must be back at its own start. But after their first step they were together. The same rule cannot separate them afterwards, so at the end of that wait they are still together. Their starts must have been the same after all.

The coarse rule does merge, by hypothesis. The fine machine therefore cannot avoid all three alternatives. In the last alternative, every tick visits another state of A, so no finite collection can hold the run. ■

When I say a coarse description is exact about a set of fine states, I mean what the theorem asks: it follows the fine step on the set, and every coarse state describes some state in it. Put it next to the permanent flip. On a finite coarse picture, a

reading that goes up and stays up is raised by a merge (the cell above, on three states; Theorem 3.1, two chapters on, in general). By this theorem, at every finer description that the coarse one is exact about, keeping that reading is again a merge, though not always where the reading rose, or a step off the described states, or a run that takes a fresh described state at every step. The dial and its light are the example. In the dial's own picture, two blends of readings decay to one either-or, a merge (Section 18). In the picture of dial and light together, the step is one-to-one, and it's the second case: the light starts blank and leaves carrying the copy, so the fine step leaves the set of states where the light is still blank. At the fine scale nothing merges; the step leaves the states the coarse picture describes. And the heat floor sits where the merge really is. Landauer's premise charges the scale whose step merges states that were really possible; a description that merges only because it stopped looking at the light owes that floor nowhere merely because of that projection. Leaving the set where the light is blank is not the same as visiting a fresh full state at every later tick. To infer an endless fresh-state run, the theorem needs its third alternative. To infer heat, specify the physical preparation and what happens to the light or reservoir.

1.8 What keeping takes, and where the toll comes in

So that's the whole of what keeping takes, set out before anybody writes a price on anything. The toll, which the next chapter states, is what keeping takes, written into a step. It's the price of something being the case: a reading that goes from no to yes and stays is a distinction that stays, and a distinction that stays is a state the run has used and can't give back. Anything whose state includes distinctions it can't give back pays what the toll prices: the bead, the cell, the memory bit, a world with a past. Give one a price list that obeys the toll and it's a Thiele machine, and when the cell could have held anything, Landauer's premise puts a heat floor under that raise, because on a finite cell the raise is a merge. The toll keeps its shape there, charged in heat instead of units of cost. And what keeping takes, deterministic or noisy or quantum, comes to this: the Brownian walker has a positive dissipation rate at nonzero drift, and the cited diffusion and jump models give finite-time distributional bounds under their own hypotheses; keeping something costs on top of that either a merge, with Landauer's floor when the merged states were really possible, or a state never used before; and a finite machine, where every state has a next one, forces the merge. The step law, the noisy steps and the unitary steps are proved, and the table above says how near the laboratory has come to each; the heat is physics.

That's also why the rule sits on the raise and not on the merge. The toll charges for what is kept whichever column pays for it, merge or fresh state. A price on merges charges only the merge column, so it misses something kept by taking a

state never used before: the machine that keeps every history, the first way out two chapters on, raises its reading at zero cost under it. And it charges merges that keep nothing, like the reset that lowers a flag. The toll doesn't ask that reset for a unit of cost, and it still merges two states, so in a real device it owes Landauer's floor whenever the bit it lowers was really unknown; the ledger leaves that to physics, the same as the dissipation of happening.

Next comes the smallest model that keeps that cost on its own ledger: two boxes on a sheet of paper, a tally, and one rule.

CHAPTER 2

A record and a toll

The last chapter found what keeping takes and kept no count of it. Now give it a ledger. This is where the machine gets stated once, exactly: a state, a reading of it into an order, moves, and a price in each move, with one rule tying the price to the order. Then take the smallest record that can carry a charge for its change, two positions, blank and written, and build it on paper, with a pencil. Write the charge into its rules and see what follows. Keeping it this small is what lets you see which part of the construction does which part of the work.

The usual models charge a computation for two things: how long it runs and how much space it uses. The bits it destroys have been counted too, as the opening said, by Li and Vitányi and by Demaine and his coauthors. What this part charges is something else: what the computation keeps and can't take back. This part is about that charge, which gets paid in a merge, carrying a heat floor when the merged states were really possible, or by taking a state never used before, and which the last chapter showed can't be talked down to zero. The record built below is the smallest thing that can carry the charge, and the tally next to it counts every time the record goes up. Whether a machine has to charge it as a unit of cost, and on which premises, is the next chapter.

One thing needs saying about what a flag stands behind, because it stays out of the picture a while. What I'm about to build prices vouching for anything at all: a flag goes up, and the rule doesn't ask what the flag is standing behind. A sorting of a pile into groups, the kind of structure Part II starts from (Section 6), is the first kind of claim I care about. It isn't the only kind, and the book comes back to what a flag stands behind in Section 9, and on a machine in Section 15, where a flag stands on a checked claim. Until then nothing about the flag leans on groupings. The floor in this section, the reason for it in the next, and the latch in Part II all work whatever the reading is about, partition or no partition. When groupings come in, in Section 9, it's as a theorem: a reading that says a grouping has been checked and committed sees the grouping only through its partition, and what it entitles is exactly the groupings that refine it (Theorem 9.2). So the grouping gets a job, and an exact one.

But first one neighbour, because the charge I'm about to write down has a famous cousin in physics, and I want it on the table before the toy is, so you can see which step it prices and which it doesn't.

2.1 A demon, and the step that pays

Here is that neighbour, and it's the one that gets closest. In 1867 the physicist James Clerk Maxwell, in a letter to his friend the Scottish physicist Peter Guthrie Tait, imagined a "very observant and neat-fingered being" sitting at a tiny door between two halves of a box of gas, letting fast molecules through one way and slow ones the other, so that one side heats up and the other cools with no work done, which is supposed to be impossible. The being got called Maxwell's demon, and physicists have been arguing about it for a hundred and fifty years, which should tell you something.

In 1929 Leo Szilard, a young Hungarian physicist working in Berlin, cut the demon's box down to a single molecule, and that made information part of the accounting. The tempting move, and the one Szilard's own accounting made, is to put the whole heat floor on finding out which side the molecule is on. Bennett separated finding out from restoring the memory for another round: measuring onto a blank record can, in principle, be thermodynamically reversible, and resetting an unknown record is a different operation. That does not mean a real measuring device costs nothing to build or run. It says that learning and resetting do not carry the same unavoidable entropy cost, and I'm about to pick a step to charge, so I need that distinction before I do.

Bennett's version of the rule, in a 2003 paper you can read, is that what must cost is "the erasure of a bit or the merging of two computation paths." That is erasing, or taking two possible situations and making them one. And in the same paper he lists copying unknown data onto a blank place to write among "the most important logically reversible operations," the kind he says can, at least in principle, be run thermodynamically reversibly. So, in the one place in all of this where a record is priced by physics, the unavoidable erasure charge belongs to *forgetting* the unknown record. Making a record on a blank place carries no such charge of its own. It does use up the blank place, which is a state the machine hasn't used yet. Resetting it for reuse has a Landauer bound when that reset removes entropy under the thermal premises. Read that twice, because it's not what I expected either.

Making a record on a blank place is one operation. Landauer's 1961 paper names another: set a bit to one whatever it held. If that bit is the whole state, down goes to up, up stays up, and two possible states become one. Landauer called this "restore to one." On an uncertain input it removes information, and a heat bound then needs the thermodynamic premises.

A larger machine has more state than its flag. Two starts can both finish with the flag up while remaining different elsewhere. The next chapter proves that, on a finite machine whose reading is permanent, any instruction that raises it must merge some full states. It does not say the state whose flag just rose is one of that pair. Put one state outside the set that reads yes and two states inside it. Send the outside state to the first inside state, and send both inside states to the second. The flag rises for the outside state, but the two inside states are the ones that merge. These arrows describe alternative starts, not three things happening in one run.

That is the distinction the one-bit picture hides. The instruction has a collision somewhere, and the actual distribution of starting states determines whether that collision removes uncertainty. A known starting state loses none under the entropy premise, even when its flag rises (Theorem 3.17, next chapter). A separate uncertain memory may have a reset cost, but neither its existence nor an earlier heat payment follows from specifying a known start. The ledger can still charge the instruction under merge pricing. A ledger charge, information loss and a device's heat are three quantities to keep apart.

A few more neighbours, quickly, because they all point the same way. Regulators who make brokers keep records on media that can't be rewritten (the SEC required exactly that, write once and read many, from 1997 until 2022, when it allowed an audit trail as the other way to comply) are pricing the hardware. The rule puts no charge on the act of writing. Haber and Stornetta in 1991, the timestamping that Bitcoin's footnotes lead back to (Section 14 follows them), chained time-stamped records together with hashes so that changing an old entry breaks every entry after it (and Ralph Merkle's hash tree, from his 1979 Stanford thesis on secrecy and authentication, lets one short value vouch for a great many records); try the dependency with five entries, 1, 2, 3, 4, 5, adding each to the running total: 1, 3, 6, 10, 15. Keep that sheet and make another with the second entry changed to 3. Its totals are 1, 4, 7, 11, 16. The change travels down the chain. Now lower the third entry to 2, and the last total is 15 again. Simple addition can be fooled by compensating changes. A cryptographic chain needs a stronger function, and someone must retain a trusted value to compare against. This trial shows the dependency between entries; it supplies neither that function's security nor a price on the record. Proof-of-work, which the computer scientists Cynthia Dwork and Moni Naor proposed in 1992 to make junk mail expensive and which Bitcoin later chained, is the one framing where a record genuinely costs, and what it charges for is a search: how many tries it took to find a lucky number. At the same difficulty, an empty and a full block face the same target for the header hash. The expected number of hash attempts agrees; the attempts on two actual blocks need not, and handling their transactions is additional work. So the work prices the search. Proof-of-work charges nothing extra for the moment anything becomes certified. Robert

Floyd and Tony Hoare, two computer scientists asking how you'd prove a program does what it's supposed to, attached claims to points in a program in 1967 and 1969 ("whenever you get here, this is true"), and the claim is backed by a proof that's paid for off the machine. And the certificates of complexity theory, which go back to Stephen Cook's 1971 work on NP, are strings designed to be cheap to check: hand me the certificate and I can confirm the claim quickly. Whether they're also expensive to *find* is the famous open question of the field, P versus NP, and nobody knows; the definition promises polynomial-time verification. Polynomial time need not be small in practice, and it need not be less than the time to find every particular witness. This is a bound on checking, not a separate charge for setting an accepting flag. And from statistics, a ledger that runs the other way: Dean Foster and Robert Stine's alpha-investing (2008) runs a long string of tests against a budget of allowed false alarms, where every test spends some of the budget and each observed rejection earns budget back, even though some rejections can be false. That's the other way round from a price on vouching: the testing rule spends and earns an error-control budget; a reward does not certify that the discovery is true.

2.2 The smallest thing that could carry the charge

Take a sheet of paper. Draw two boxes and label them NO and YES. The two boxes are the reading, a single yes-or-no question asked of the state, and I'm going to be careful about what the question is, because after five thousand years of seals and witnesses the word "vouch" is carrying a lot and this toy carries almost none of it. The question is only: "has the designated flag been set." It is not "is some claim about a grouping true," and it is not "did anyone check." There is no claim in this toy, no evidence, no checker, nobody standing behind anything. There is a flag. Whether a flag can ever be made to *mean* that something was checked and can be relied on is a real question, and it's not this section's; I'll build toward it later, and until then please read YES as "the flag is up" and nothing more. The state is which box you're in, and that's the whole state, one bit. Then draw a line under the boxes for tally marks. The marks sit outside the state. They're the running total you keep beside it as you play, the way you'd keep score beside a board game.

Now give yourself three moves. STAY: you don't move, and you make no mark. WORK: you don't move either (it's a stand-in for every ordinary step a machine takes), and you make no mark. VOUCH: from NO you move to YES; from YES you stay in YES; either way you make one tally mark, because in this toy the price belongs to the move's name, and where you were standing doesn't enter into it. Now play. Start in NO, make any sequence of moves you like, and stop in YES. Count the marks. Then try to reach YES with zero marks. You can't, and you can see exactly why: the only move that crosses from NO to YES is VOUCH, and VOUCH makes

a mark. Now erase the mark from VOUCH's rule and play again. You reach YES with no marks, every time. The entire content of what I'm about to call a rule is this, and I want you to see how little it is before I dress it up: I've decided that the move which crosses the line makes a mark. Nothing in the toy's rules forces the mark. VOUCH does merge, NO and YES both landing on YES, which is what the last chapter says a kept raise on two states has to do; what nothing yet says is that a merge has to cost a mark. Here it's a choice, what the last chapter says keeping takes, written down as a rule. The next chapter is about when a machine has no choice.

Why tally marks, by the way? Because they're about the oldest receipt there is. The real ones, the English treasury's notched sticks split down the middle, get their own story in Part II (Section 7), fire and all. The tally on your paper is nothing like those. Nobody holds the other half. You keep it, beside the board, and you could rub a mark out and nobody would ever know. Hold on to that, because by the end of this section it's the reason the toy is a scorekeeper's receipt and not a record anybody else could rely on.

Here is the machine, stated once, carefully, because this is the one place in the book where I state it exactly and everything later points back here. The point listed the five parts in a sentence; here they're pinned down. A Thiele machine has five parts.

A set of states.

A set of move names.

A step: for each state and each move name, where you land.

A cost for each move name: a whole number of units, zero or more.

A record: a reading of each state into an order, a set of places on which, for any two, you can ask whether one is at or below the other. Higher is more settled.

Those five parts are the whole of it. Add the one rule, the toll, and it's a Thiele machine: a move that carries the record somewhere not at or below where it stood costs at least one. A machine whose moves never climb is the flat case, and the rule has nothing to say to it; every ordinary computer is one.

The thing on your paper is the smallest case that isn't flat. Its record has two places, the blank box and the written one, which I've labelled NO and YES, with NO below YES; its states are the two boxes and nothing else; its move names are STAY, WORK and VOUCH; its costs are zero, zero and one, in tally marks. Everything in this section is proved for the two-place record, because it's the one you can point at. Section 4 proves the same for any order and gets the two places back as a case (Theorem 4.2).

I also want to be exact about what's not in it. There's no counter stored inside the state that's forbidden from going down; the marks are the side tally from the board game, and the tally only ever grows because the costs are whole numbers, zero

or more. (A machine could keep the total inside its state too; that's an extra part beyond the five, and Section 7 builds it.) There's no window, no observer who sees only part of the state; that's a later question and it gets its own section. And the cost depends only on the move's name; where you are when you make it doesn't matter. That's on purpose: the charge is on the attempt, so you pay for making the move, whatever it turns out to do. It's also a real simplification, and where a result needs a price that depends on where you are, Section 8 says so and uses one. Last, the step gives every state and every move exactly one place to land. A choice becomes a move name, and a machine that has stopped steps to itself. A step that can land in several places by chance is a different picture, and the noisy steps in Section 1 take it up. To apply the step law, fix the move being repeated, or include the program and its control in the full state so that one autonomous rule chooses the next step. Arbitrarily choosing different moves from outside is a different case, worked through in the next chapter. Deterministic instructions alone do not supply that fixed rule. Separately, giving every move a successor even when the flag is already up keeps a price per move name from charging exactly the raises: a move that raises the flag and has no next step once it's up could cost one and never charge for anything else.

On two places the toll reads like this. For every state and every move: if the reading is NO before the move and YES after it, the move's cost is at least one. That's the whole rule there. It says nothing about any other move. It says nothing about how big a mark is in anything you could measure. One mark is one mark; what a mark is worth, why the smallest charge is one (why not two, or a half?), and whether I had any right to decide that, come next. The middle question has a short answer I'll give away now. What the next chapter forces, on its premises, is that a raise that stays up, with only finitely many yes-states, merges two states, so any price on merges lands on it. Costs here are whole numbers, which I chose so the ledger counts acts, and the least whole number above nothing is one. The "at least one" comes from there.

And one theorem, which is the only thing the toy actually proves, and you've already proved it by playing. Take any run of moves that starts in a NO state and ends in a YES state. Then the marks it made add up to at least one. The argument in words is short enough that you can hold the whole thing. Walk the run move by move, reading the box at each step. It read NO at the start and YES at the end, and every move either leaves the reading alone or flips it, so at some move it went from NO to YES. The toll says that move cost at least one. Every other move cost zero or more. Add them up and the total is at least one. Notice what the argument did not need. It didn't need the reading to *stay* YES afterward; if a later move flipped it back to NO and another flipped it to YES again, the toll charges twice, and the argument still goes through. It didn't need the machine to be finite. It didn't need to know what any move does besides what the reading says before and after. It needed

the toll, and two things the five parts already hand it: no cost is below zero, and the reading has only two values, so a move that changes it from NO lands on YES. Give the reading a third value, a "maybe" between the two, and the one-bit toll as written here charges nothing for NO to MAYBE; the toll over any order in Section 4 charges every step that climbs, MAYBE included. That difference is the whole reason the record is a place in an order and not a bit.

That's the whole theorem, and I'll state it once more in the exact form, because from here on the book carries two layers side by side: what I mean, in words, and what's stated and proved, in symbols you can hold me to. The symbols are few. $\mathbb{N}$ is the whole numbers 0, 1, 2, and so on. S and I are just names for "the set of states" and "the set of move names." Writing $\text{step} : S \times I \to S$ means "hand it a state and a move name and it hands back a state." A *run* is a list of move names, $t = [i_1, \ldots, i_n]$, played from a start s_0, one after another. The double arrow $\Longrightarrow$ reads "then": the thing on its left forces the thing on its right. For this first proof, keep your finger on the run and stop at the first YES. The move immediately before that is the one whose charge matters.

Definition 2.1 (Thiele machine)**.** A *Thiele machine* consists of a set S of states, a set I of move names, a step function $\text{step} : S \times I \to S$, a cost $\text{cost} : I \to \mathbb{N}$, and a record $\text{rec} : S \to R$ into a set R with an order $\leq$ that is reflexive and transitive, subject to one rule, the *toll*: for every state s and move i,

$$\text{rec}(\text{step}(s, i)) \not\leq \text{rec}(s) \implies \text{cost}(i) \geq 1.$$

Nothing more is assumed of the order.

Definition 2.2 (Thiele machine with a two-place record)**.** Take $R = \{\text{no}, \text{yes}\}$ with no below yes. The record is a reading $\text{cert} : S \to \{\text{no}, \text{yes}\}$, it leaves the down-set exactly when it goes from no to yes, and the toll reads: for every state s and move i,

$$\text{cert}(s) = \text{no} \text{ and } \text{cert}(\text{step}(s, i)) = \text{yes} \implies \text{cost}(i) \geq 1.$$

The run of $t = [i_1, \ldots, i_n]$ from s_0 ends at $\text{run}(t, s_0)$, the result of applying the steps in order, and its total cost is $\text{cost}(t) = \text{cost}(i_1) + \cdots + \text{cost}(i_n)$, with the empty run costing 0.

The first definition is the machine. The toy on the paper is the second, and most of Parts I and II is told on it; Section 4 carries every result here over to any order. I write the two-place reading as cert, because on the machine of Part III it names a flag that says a claim was certified. What earns a flag its meaning will depend on the machine's rules.

The one theorem comes first in its general form, because it's three lines and it shows that nothing about two places is doing the work.

Observation 2.1 (The floor over any order)**.** *In any Thiele machine, a run whose record ends somewhere not at or below where it started has total cost at least 1.*

Proof. Read the records along the run. The last is not at or below the first. Walk forward and take the first state whose record is not at or below the start's. The state before it reads at or below the start, so if this one read at or below the state before it, it would read at or below the start too, since the order carries across, and it doesn't. So the move between them climbed, and the toll charges it at least one. No other charge is below zero. ∎

On two places it reads as follows, and that is the version the toy proves.

A word on the label before the first result. I call a result an *observation* when its proof does little more than unfold the definitions, and in an axiomatic cost model that's what the definitions are meant to do: the work is in choosing them, so the results that follow from them in a few lines are labelled that way and the theorems with more in them stand out.

Observation 2.2 (The universal floor)**.** *In any Thiele machine, every run that starts at a state reading no and ends at a state reading yes has total cost at least 1: if* $\mathrm{cert}(s_0) = \mathrm{no}$ *and* $\mathrm{cert}(\mathrm{run}(t, s_0)) = \mathrm{yes}$*, then* $\mathrm{cost}(t) \geq 1$.

Proof. Read the states of the run from left to right. The first reads no and the last reads yes, so there is a first state on this list that reads yes. It cannot be the starting state. The state immediately before it reads no: otherwise the chosen YES would not be the first. The move between them therefore costs at least one, by the toll.

Now add the charges for the whole run. That one charge is already at least one, and none of the other charges is negative. The total is at least one. The flag may come down later and go up again; that cannot remove the charge I have found. ∎

The tabletop and the stamp

Put two boxes on the table. Each has tiles on top, and the tiles spell the same answer. On the side of one box the stamp is up. On the other it is down. Hang a camera directly above them. The photographs are identical.

Now hand me the photographs and ask which box was stamped. Take a better photograph. Enlarge it. Count every tile. None of that gets the side of the box into the picture. The difference is there, in the boxes. It has been left out of what you handed me.

That is the window I want to name. S is the set of full states of the boxes, and B is the set of tabletop pictures. The map $\pi : S \to B$ takes a full state and returns its picture. Every picture in B comes from a box; that is what *onto* says. Pick one

picture, b, and gather every full state that gives that picture. That whole collection is the *fibre* above b, written $\pi^{-1}(b)$.

Keep the whole collection in mind. The tiles can agree while the boxes differ in which checks ran, which earlier pages were certified, and what was kept when those pages were wiped. The fibre holds all the differences this camera leaves out. There can be two states in it, a million, or infinitely many.

I started with one question: is the flag up? That gives me one yes-or-no reading of the full state. A hundred different states can all answer yes. Reading that one flag tells me where they stand on that one question; the other differences are still there. Chapter 14 opens the gauge wider. It asks which checks passed and which editions were certified, and counts how many states those answers require. Keep independently chosen histories across arbitrarily many editions, all recoverable above the same final picture, and the fibre has to be infinite; the count is a pigeonhole on the editions.

To see exactly what the tabletop can do by itself, give these boxes two kinds of move. The tile moves are horizontal. They cost zero, leave the stamp alone, and move the tiles according to what is on the tabletop. The stamp never reaches into their gears. The stamp moves are vertical. They leave the tiles alone and can raise the stamp or leave it where it is. Once raised, it stays raised. A raise pays the toll already defined above.

These are the rules of the sliced machine. They give me a view of the flat part that runs on its own and a record that survives beside it.

Which question does the toll charge?

Slide a tile one place to the right. Let that move cost nothing. Now ask a new question: is the tile in the place it just reached? Before the slide the answer was no. Afterward it is yes. If every question had to obey the same toll, that slide would have to cost at least one.

Try the same trick on any free move that changes the full state. Ask whether the machine is in its destination state. Again the answer changes from no to yes. Charging every question would stop every free change. The reading in the definition therefore matters: I have to say which question the toll belongs to before I can say which moves are free.

□ **Theorem 2.3** (Charging every question stops free changes)**.** Suppose equality of full states is decidable. The same instruction prices satisfy the toll for every Boolean reading exactly when every zero-cost instruction leaves every full state unchanged.

Proof. For a changed state, the question "am I at that destination?" supplies a crossing, so the move cannot be free. In the other direction, a move that leaves the state unchanged cannot change any reading. Every remaining instruction already costs at least one, since prices are nonnegative whole numbers. It therefore pays for any crossing it makes. ∎

Now keep one question and require its yes to stay yes. A free move cannot raise that reading, because a raise pays the toll. It cannot lower it, because the reading is permanent. It has to leave it alone.

Theorem 2.4 (Free moves leave a permanent reading alone)**.** The toll and permanence force every zero-cost instruction to preserve the reading on every state. □

Proof. Starting at no, a free move cannot reach yes by the toll. Starting at yes, it cannot reach no by permanence. Those are the two possible starting readings. ∎

This gives two ways to state the reading part of the sliced rules. Require horizontal moves to leave it alone and vertical moves never to lower it; then every move keeps a yes. Or require that every move keeps a yes, keep the toll and free horizontal prices, and recover horizontal preservation by the two sentences above. The window rules still do their own work. A paid move could change both the tiles and the stamp; permanence does not make the tiles stand still.

Definition 2.3 (Sliced Thiele machine)**.** A sliced Thiele machine is a Thiele machine with an onto view $\pi : S \to B$ and its move names split into horizontal moves I_H and vertical moves I_V. Write $r(s) = \mathrm{cert}(s)$, with no $<$ yes, and $F_i(s) = \mathrm{step}(s, i)$. For each $i \in I_H$ there is a map $h_i : B \to B$ such that

$$\pi(F_i(s)) = h_i(\pi(s)), \qquad r(F_i(s)) = r(s), \qquad \mathrm{cost}(i) = 0.$$

For each $v \in I_V$,

$$\pi(F_v(s)) = \pi(s), \qquad r(s) \le r(F_v(s)).$$

The toll remains: $r(s) < r(F_v(s))$ implies $\mathrm{cost}(v) \ge 1$.

These laws hold on every state of the specified domain. In contrast, the counter example's later live-state slice is a restricted interface: a failed check leaves its domain. It is not a claim that all six native instruction kinds satisfy these equations globally.

The states can be counter configurations, tape configurations, or any other states meeting these rules. The same horizontal argument works with a whole set of claims

as the record: every horizontal move preserves the set and evolves the picture using only that picture. The number of possible records never enters that argument.

Now give the two boxes the same horizontal moves. After one move the pictures still agree. After two they still agree. Every next picture comes from the previous picture and the move you named, and those agree at every step. Meanwhile one stamp stays up and the other stays down. You can watch the tabletop for the whole run and never see the difference you started with.

Give somebody permission to read the stamp and choose a different tile move because of it, and they can signal through the tiles. Here both boxes receive the same moves. That is how I keep the question fixed while asking what the window can reveal.

□ **Theorem 2.5** (What the tabletop can tell)**.** In a sliced Thiele machine, every horizontal word projects to the corresponding word of the maps h_i on B, and preserves r. Two states with the same view and different records therefore keep the same view and different records after any common horizontal word. No function of the view alone can recover r on both states. A vertical move that changes r supplies such a pair: s and $F_v(s)$.

Proof. Start with two states showing the same picture. Give them the first horizontal move. Its rule chooses the next picture from the current picture alone, so the next pictures agree too. Neither record changes. You are back in the same situation, ready for the second move. Repeating this argument covers every move in the word, including a word with no moves. On each side the picture follows the corresponding h_i, and the original record stays put.

A proposed reader of the picture now receives the same input from both states. It must give the same answer to that input, so it cannot return two different records correctly. Finally, a vertical move that changes the record gives you exactly such a pair: before and after, the picture agrees and the record differs. Those must be ■ different full states.

The stamp remains in the full state through every one of those moves. The camera keeps missing it for exactly the same reason it missed it in the first photograph. In Part III a failed check can stop the gears. Hiding that stop while asking the picture to predict every later free move would also hide the difference between gears that turn and gears that cannot. Theorem 15.1 works through that obstruction; the live-state slice there keeps this boundary explicit.

The smallest price on the instruction

Put three price cards beside a button: zero, one and two. Somewhere in the machine there is a state with the lamp down where pressing that button raises it. Cross out zero. It fails there. One pays the toll; two pays it too. The toll has picked the smallest card, even though it has not forbidden the larger one.

Now take a button that never raises the lamp, from any state. Zero works. Go through the buttons this way. Put one beside each button that can raise the lamp and zero beside each button that cannot. The price stays on the button when you use it somewhere else. A button that raises the lamp once still costs one when the lamp is already up.

Theorem 2.6 (The least instruction price)**.** Fix the transitions and reading, and supply a finite list covering the states. The schedule just described is the unique pointwise least nonnegative whole-number schedule satisfying the toll. It is also the least schedule giving every trace from no to yes, from every start, total cost at least one. □

Proof. For each instruction, inspect the list for a state where it raises the reading. If one is found, every valid schedule must charge at least one there, and the instruction has the same price everywhere. If none is found, the toll puts no positive demand on that instruction, so zero suffices. Our schedule meets every demand and is no larger than any schedule meeting them. Two least schedules must each be no larger than the other at every instruction, so their prices agree. The trace claim follows by finding the first crossing; conversely, a crossing by one instruction is already a trace. ■

The list makes that search finish. With infinitely many states, searching forever without finding a crossing does not tell me that none exists. For the lift, there is a direct answer whenever each mark has an event witness: put the lamp down over a base state where that mark's event holds, and press it. It raises the lamp. Every mark therefore needs one, and computing moves need zero because they leave the lamp alone. That proves the least zero/one schedule even over an infinite base. A mark whose event never holds can cost zero under the toll. Giving that mark a tally of one on every use is an additional accounting rule. The unit in all of this is the unit of the definition; to price a physical device in joules I still need its physics.

The universal floor is about every Thiele machine, a system with the toll among its conditions, and about no system without it. One edge matters: the theorem does not assume that a YES state exists. If no state ever reads YES, no run meets the theorem's hypothesis, so it's true of every run and tells you nothing about any of them, which is a mathematician's way of saying it's true and useless. The toy on

your paper has a YES box, so it isn't useless there. It would be natural to assume the system has to have at least one NO state and one YES state; the statement has no such clause, and that's on purpose, because a theorem that applies to no run is still a theorem and the toy is where you supply the runs.

The theorem also runs backwards, and that direction is one line. Suppose every run from a state reading no to a state reading yes costs at least one, whichever state you start from. A single move is a run of length one, so a move that takes no to yes costs at least one, which is the toll. The floor from every start and the toll are the same rule, and nobody has to choose between them. From one fixed start that reads no, the floor asks for the toll only at the crossings you can reach without paying anything; a crossing you can only reach by paying has already cost at least one by the time you get there.

And the bound of one is the smallest case of a count.

Observation 2.7 (One unit per crossing)**.** *In any Thiele machine, a run costs at least the number of its steps that take the reading from no to yes, with no permanence assumed. Conversely, a system in which every run costs at least its number of such steps obeys the toll. And the bound is reached for every count: on the one-bit machine with a set that costs one and a reset that costs nothing, setting and resetting n times from no crosses n times and costs exactly n.*

Proof. Each crossing step costs at least one by the toll, every other step costs at least zero, and each step is counted once, so the total is at least the number of crossings. Conversely, a single move that crosses is a run with one crossing, so it costs at least one, which is the toll. On the one-bit machine, each set from no is a crossing costing one and each reset costs nothing, so n rounds cost n and cross n times. ∎

And one more edge, because it's the first thing to bite when you ask a machine to stand behind more than one thing. The two-place record has *one* question in it: one flag, and one crossing the toll charges. Once the flag is up, a second claim has nowhere to go, because raising a flag that's already up doesn't move anything. So a second claim needs a reading of its own, and a third claim a third: one reading per claim, each with its own crossing. Part II builds that properly, as a family of flags, one per threshold, with a step that raises any of them charged at least one (Theorem 11.10). A step that raises three at once still owes at least one, and nothing in the toll makes the charges add up flag by flag. Charges that add need the composition assumptions for independent machines (Appendix D). The same theorem has a count in it that is what finiteness takes. Picture the record as a row of switches that only ever get turned on. Ten switches can change at most ten times in one run, one switch per change, and then the row is full. (Across different runs the row can end in any of its thousand and twenty-four settings; along any one run it climbs through at most eleven.) A finite machine can keep only so many things up

for good in its whole life. That's "you can't keep all of it," which Section 14 says of a full trace, turned into a count.

2.3 How the receipt earns its meaning

Give the worker a whiteboard and a pen. Ask for an answer and a line underneath saying "CHECKED = YES." The worker can write both. So can somebody who copied the answer and did no check at all. The extra line has told you nothing about which happened.

Follow that problem into the machine. If an unchecked run can reach the same accepting state as a checked run, there is nothing in that state that tells the two apart. To make the receipt carry the difference, the machine has to close the unchecked route. The worker can request the check; the worker cannot simply write its successful result into the accepting record. That restriction is what gives the record its force.

Put a variable called checked beside the answer and you have another place to write. The name does not give it authority. A program that can write yes there without doing the check can manufacture the same display as the program that earned it.

For an earned receipt, I want the machine's rules to close that route. Every permitted program must go through the required check before it can reach acceptance. The program can ask for the check. It cannot give itself the successful verdict. That is the difference between carrying words about a check and enforcing what those words stand for.

So when somebody offers a simulation, showing me where its bits go is the beginning. Show me which instructions the simulated program can actually use, and prove that none of their allowed sequences can forge acceptance. Preserving that boundary is part of implementing this machine. A copy of the flag alone has not done it.

Now put the camera back over the tabletop. The same answer can sit above an accepting record or an unearned one. The record has to survive, and the rules governing access to it have to survive too. Those are the things I am asking the machine to keep.

Look before the concrete covers it

The inspector arrives after the concrete is poured. The question is about the rebar underneath. On one job it was sound. On the other it wasn't. Give both jobs exactly the same finished appearance, and give the inspector only that appearance.

There is no amount of staring that tells those jobs apart. Searching for a sound arrangement that could have produced this surface doesn't do it either. A sound

arrangement was possible on the bad job too. What matters is which arrangement was there.

Write x for what was there, $f(x)$ for what remains to be inspected, and $P(x)$ for the yes-or-no question about it. A good x_+ and a bad x_- leave the same $f(x)$. The inspector receives one identical input and is supposed to give two different answers. Here is the whole obstruction.

□ **Theorem 2.8** (An erased distinction cannot answer its old question)**.** Let $f : X \to Y$ and $P : X \to \{\text{no}, \text{yes}\}$. If $f(x_+) = f(x_-)$ but $P(x_+) = \text{yes}$ and $P(x_-) = \text{no}$, there is no $d : Y \to \{\text{no}, \text{yes}\}$ with $d(f(x)) = P(x)$ for every x. If P is available before the loss, the enriched observation $(f(x), P(x))$ does answer the question exactly and distinguishes these two cases above their common view.

Proof. The proposed d would return both yes and no at $f(x_+) = f(x_-)$. The enriched observation returns its second coordinate. Its two values are $(f(x_+), \text{yes})$ and $(f(x_-), \text{no})$: the first coordinates agree and the second coordinates differ, so the enriched states remain distinct. ■

While the rebar is exposed, you can inspect it and keep the answer. Or preserve enough evidence to inspect later. Either way, something that distinguishes the good case from the bad one has to survive. Wiping an unrelated detail causes no trouble. Wiping the last difference your question depends on leaves the next inspector with the same surface and two possible pasts.

For one known question repeated along a run, the receipt can be very small. Start a bit at yes and, at each check, replace it by the old answer AND the new answer. It says exactly whether all checks so far passed. A later failure turns it down. The permanent flag that says “some certification occurred” stays up. Put those gauges beside each other and you can see why they answer different questions.

When the stamp merges two states

Draw two boxes with the same tiles: Unstamped and Stamped. Pressing the stamp takes the first to Stamped. Pressing it on the second leaves that whole box exactly where it is. Draw both arrows. They land in the same place.

Call the press V. At a state s where it raises the record, s and $V(s)$ are different. If another press leaves the full state fixed, then $V(V(s)) = V(s)$. Those two different states have the same successor.

Give the stamped box a hidden wheel that turns on every press. Now the picture has more states in it. If every state reached after stamping still reads yes, and there are only finitely many such states, a merge is forced somewhere along the continued pressing. It can be later than the first press. Keep the entire move injective instead,

and the run has to keep finding distinct stamped states. Call that finite accepting region H and the entering state u. The move sends all of $H \cup \{u\}$ into H. There are $|H| + 1$ distinct sources and only $|H|$ possible targets, so two sources share a successor. The move preserves the working picture, so both sources and their common successor lie in that same fibre.

An internal ledger belongs in that full-state picture too. If each press increments it, two presses have different ledger values even when the stamp looks unchanged. Count those states with the others. The toll tells me what to charge for a raise; the next chapter gives the premises that connect charges to mergers. I want the arrows and the bill on the same page, with the rule connecting them where you can see it.

Notice also that it asks nothing about how strong the system is, in either direction. A system is *Turing-complete* when it can imitate a Turing machine, and no stronger than one when a Turing machine can imitate it. The theorem asks neither question. Whether the system is stronger than a Turing machine, weaker, or exactly as strong makes no difference to the floor. Bring the five parts, charge the toll on the crossing, and you're in scope. The floor doesn't check what you can compute, because it was never about what you can compute; it's about what a move is allowed to do without costing anything.

That brings up the objection a theorist makes in about four seconds, so I'll make it first. The toll is a floor, and a floor is easy to meet. Charge one unit for every move, whatever it does, and the toll holds for any reading you like, since every crossing is a move and every move costs one. So any computer with a step counter beside it, and any yes-or-no question at all about its state, is a Thiele machine, and a clock passes. Nothing about that is a loophole; it's what the definition says, and it means the definition, on its own, doesn't tie the cost to anything kept or certified. It only says a crossing can't go unpriced. (It's also why "a machine that can compute anything plus a flag that obeys the toll" says little on its own: a clock meets it, and so does the flat case, which never raises anything.) Two things later in the book tie the cost down. The first is a ceiling: demand that a run's total never exceed the number of times the flag went up. With a price that can depend on where the step starts, that and the floor leave exactly one way of charging, one unit per raise and nothing else (Sections 7 and 8). With a price per move name, as in my definition, no system whose flag ever rises meets both, because the raising move, played again from yes, which every move can be, raises nothing and still costs one. A clock fails the ceiling the first time it charges a step that raised nothing. The second is order, on any certified machine, and the example of Part III is one: there, from a clean start, the flag can't go up unless a passing check of a claim came first, then a commitment to that same claim, and a clock can't fake that, because it's about which moves happened, and a clock only charges (Theorem 15.11).

There is also the matter of why the ledger doesn't charge for heat. A physical ledger would need the actual dynamics, distribution and protocol. A ledger that simply charged every nontrivial move would charge by that rule, and that's the clock again: it can't tell a run that kept something from a run that only ran. So the ledger leaves the dissipation of happening to physics and counts one thing, the rise of a reading that stays. A zero abstract charge says nothing about physical dissipation; it just prices no crossing.

The other side of that is that nothing is in scope just for being a computation. A Turing machine's run isn't an instance of this theorem because it's a run. It becomes one when somebody supplies all five parts for it, says which states read yes, and proves the toll for that cost. Until then the theorem says nothing about it, and I won't pretend it does; that limit is the toll's scope. What keeping takes has no scope to supply, because every run of anything with one next state per state takes a fresh state at every step, joins exactly one pair of its states into one, or goes round a loop through its start and keeps nothing (Theorem 1.1). The reading *this isn't the state I started in* rises at the first step and stays up exactly on the runs that never come back to their start, the first two kinds, and those pay, in states never used before or in their one merge.

That's also where the hunch this book started from ended up, once I checked it: the night a video about *Flatland* convinced me there was another axis, a direction ordinary computing is blind to (the Afterword, Section 24, tells it). It isn't a direction an ordinary computer can't reach. Hand a Turing machine a reading, charge the toll on its crossing, and it's in scope like anything else. The possibility was always there. What the usual rules leave out is the price.

The same goes for a system with no reading at all. If nothing in it answers "is the flag up," then nothing in it can say, from the inside, that anything is entitled to be relied on. Whatever entitlement it ends up with gets carried in from outside, by somebody who signs their name to it, and nothing in the system priced it. Smuggled entitlement isn't entitlement for the system, no matter whose name is on it. Be careful which way that cuts, though. Whoever reads the signature can be perfectly entitled on their own account; that's what an auditor has, and it's honest as long as it's called the auditor's. It isn't the system's. On the example I build later, the reason to believe its checker says yes only when the claim is true is exactly that kind: a proof, done outside any record the machine keeps, and I'll say so when I get there (Section 15). What outside entitlement never is, is something the system itself earned.

There's also a question I'll only start on here: bolt two Thiele machines together, and do you get one? Run them side by side, each move being a move of one or the other, and add up the costs. It depends on the reading you give the pair. Take a reading that can only go up when one of the parts' readings goes up, like "either

flag is up" or "both are": then any crossing of the pair is a crossing of a part, that part's toll charged it, and the pair obeys the toll. Take "the first flag is up and the second is down," and it doesn't: that reading turns on when the second flag falls, and nothing charges a fall. The first two readings are proved in Appendix D: read through "either flag is up" the pair is a Thiele machine whose floor is one. Read through "both are," a run that starts with both flags down and ends with both up costs at least two, and two is reached when each part can raise its own flag for one. From a start with one flag already up, one raise is enough, so two is a floor for the clean start only. The third needs nothing beyond the one unpriced fall above. A related composite keeps the toll too, a host machine running a guest, read through "the host's own flag or its copy of the guest's." Whether every way of building one machine out of others keeps the toll is a bigger question, and Appendix D takes it as far as I could: some ways do, some provably don't, and it says which.

So why bother with the toy at all? It isn't because anybody was looking for it. People have been making marks that other people rely on for five thousand years, and what they were after was durability, evidence, authenticity, catching slips; none of them was trying to put a price on the moment a flag goes up, and I shouldn't pretend they were. The reason is narrower. Everywhere somebody *did* locate a cost, but one, it sat on a step next to the flag. The flag itself went unpriced as a flag: Szilard on the measurement, Bennett on the wiping, proof-of-work on the search, Boyle on the instrument and the room, Foster and Stine on the test. One entry prices the crossing's own shape, and I put it beside the toll on purpose: Landauer's restore to one, the literal two-state version of the crossing, whose heat bound depends on the input uncertainty and the thermal premises. The flag's job never came into it. The toy is the first thing I could build where the cost sits on the crossing itself, and it proves that if you price that step, the floor follows everywhere, for any machine with the five parts. That's not an answer; it's the question made exact enough to be wrong. And of the jobs a record does, the toy supplies only a limited kind of retention: its three permitted moves never lower the flag. The abstract definition allows other machines with a reset. Adding that reset would change this toy, and the cost that doesn't go away is kept by *you*, the scorekeeper, beside the board. The machine keeps none of it. What the toy has is a scorekeeper's receipt for the crossing. Whether the machine can be made to keep that receipt itself, where a step could reach in and change it, is Section 7's question. Authenticity, evidence, accounting, cheap checking: none of them is in here, and I'll say so again when it's tempting to forget.

Let me say what I'll owe you by the end. Whether the toll is a choice or a necessity: the next section, and I'll tell you now that the answer surprised me. What a unit of cost is worth in anything physical: later, and only partly. What an observer who sees only a window can and cannot recover, including whether they can even

tell which step paid: its own section. And whether the definition can be met by a machine that computes everything a Turing machine does, with its record earned and not declared, and what it costs to vouch for structure in one: Part III shows that with an example, and takes most of its length on it.

For now you have a sheet of paper with two boxes, three moves, one rule that writes what the last chapter says keeping takes into the step, and one theorem you proved yourself in about a minute. Don't let go of it. It's the smallest version of everything that follows, and if it's wrong here, it's wrong everywhere.

CHAPTER 3

Why the toll isn't arbitrary

The first chapter of this part proved what keeping takes by counting states: a run of one rule that raises a flag and keeps it up either lands two of its states on one somewhere or never uses a state twice, so keeping is paid in a merge or by taking states never used before; and a thermal implementation can incur additional dissipation under the specified dynamics. The merge-or-fresh-state part needs no premise beyond one next state for every state; the heat is physics, which I proved for Bennett's walker with Schnakenberg's formula taken as the model, and cited for devices whose states jump or drift at random in surroundings at a temperature. This chapter counts the fresh states exactly, and asks what it takes to turn what keeping pays into a unit on a ledger. The flag I built is one thing that stays; the result is about anything that stays.

The last chapter wrote what keeping takes into a toy as one rule, and in the toy the move that raises the flag costs one because I said so. If that's all it is, the ledger is a long way of describing a preference, and you should treat it the way you'd treat someone telling you their favourite colour is a law of nature. So this chapter is me trying to find out whether a machine has to charge it. I'll tell you the shape of the answer up front, because I don't want you reading forty paragraphs to find out whether there's a point. What keeping takes needs no "if" beyond a next state for every state: on any machine, keeping something is paid in a merge or in states never used before, and physical dissipation is a separate, model-dependent question; that's the paragraph above. Getting from what keeping takes to my rule, one unit on the ledger for the step that raises the flag, takes three "ifs": there are finitely many states that read yes, so the fresh states run out; the flag, once raised, stays up; and the ledger charges every move that forgets. Then the move that raises the flag has to forget something, two of the states it acts on landing on one, and pays. Forgetting is also the step Landauer's principle, my premise, puts a heat price on, when what's forgotten wasn't already known; a known preparation alone supplies no positive entropy-loss floor. If the input is correlated with another memory, that larger physical account has to be specified too. Each of the three "ifs" is a way out of the theorem, and I'm going to walk through each one, because what's on the other

side of each is where the content is. None of them is a way out of what keeping takes. Where the first fails, the machine pays by taking states it has never used. Where the second fails, the flag comes back down, so nothing is kept and there's nothing to charge. Where the third fails, the step still forgets, and only the ledger looks away.

One more thing before the three ways out, and it comes first on purpose. Nothing in the argument uses the word certify. It uses a reading that can go from no to yes and never back. A fuse that has blown is one. So is a seal that has been broken, a bit in write-once memory, a block a chain has finalized, a measurement somebody wrote down and won't take back. The argument below sends what keeping takes to whichever of them you point at, and the latch chapter (Section 11) proves it doesn't matter which one you pick. Certification is the instance I build, for reasons I give in Section 18 and don't claim to have proved. So read this section as being about anything that stays. The flag is only the one I keep on the table.

3.1 A closed room with no spare chair

Do this one with real objects, because the whole section rests on it and it's about ninety seconds of work. Draw a box on the table and call it the yes room. Put three chairs inside the box and one chair outside, by the door. Four people, one per chair; the people are the states your machine could be in, and the three inside are the ones whose flag is already up. One move of the machine moves everybody at once: each person stands up and goes to the chair the rule sends them to. There are two rules. First, nobody inside the room may end up outside, because the flag, once up, stays up. Second, the person by the door must end up inside, because this move is the one that raises their flag.

Now try to finish the move with one person in each chair.

You can't. Three people start inside and have to stay inside, so they take the three inside chairs between them, with nobody left out, because nobody may leave. The person from the door has to come in, there is no fourth chair inside, so somebody has to share a chair, and two of the machine's possible states have landed on one state. Now use the chairs as a guessing game: secretly choose one starting chair, apply the rule, and report only the finishing chair. If two starts are sent there, the report cannot tell your partner which start you chose. Watching the people move would tell them, but that would be keeping information outside the finishing chair, and the chair alone has forgotten something.

The whole argument is in that shared chair, and I want to name the piece of mathematics that did the work, because it's old and it's not mine. If you put things into fewer boxes than there are things, some box gets two. People call it the pigeonhole principle; it's the one from "The point," where Jean Leurechon used

it in 1622 to argue that two people somewhere have the same number of hairs, and the first name for it I could find came from Dirichlet, who used it in 1842 and later called it the drawer principle. It's the kind of fact a child accepts and a mathematician spends a career finding new uses for. Here the use is that a step which forgets nothing is a one-to-one shuffle, a one-to-one shuffle of a finite set of chairs lands on every chair, so the inside chairs are all taken by insiders and none is left for the newcomer. Finiteness, permanence and the flip are the three things that force a collision in the instruction's map. The two starts that collide need not include the outsider: send the outsider to one inside chair and send two insiders to another. A run from a known outside chair can still have a uniquely identifiable start. The collision is a fact about the instruction over possible states.

The precise statement of the argument is a theorem a few lines down, and it needs one word first. "Injective" is the mathematician's word for "forgets nothing": a step is injective when two different starts always give two different ends. The theorem says exactly what the people and the chairs said and takes exactly the three premises: a duplicate-free list that names every state (finite), a reading that no step ever turns off (permanent), and one state that reads NO before the step and YES after it (the flip). There is no price in it anywhere; it is pure counting. I'll come back to what counts as physics, but notice that nothing so far has been physics.

The exact words follow. For this whole section the setting is the five parts from Section 2 without the toll: states S, move names I, a step, a reading, and (later) a cost.

Definition 3.1 (Finite, permanent, injective)**.** The machine is *finite* when there is a list of states with no repeats that contains every state. The reading is *permanent* when no move ever turns it from yes back to no: for all s and i, $\mathrm{cert}(s) = \text{yes}$ implies $\mathrm{cert}(\mathrm{step}(s,i)) = \text{yes}$. A move i is *injective* (forgets nothing) when different starts give different ends: $\mathrm{step}(a,i) = \mathrm{step}(b,i)$ implies $a = b$. A move that is not injective *merges*.

Theorem 3.1 (A permanent flip merges)**.** On a finite machine with a permanent reading, if $\mathrm{cert}(s) = \text{no}$ and $\mathrm{cert}(\mathrm{step}(s,i)) = \text{yes}$, then the move i merges. □

Proof. Suppose i were injective. Let C be the list of states that read yes: go down the list of every state and keep the ones that read yes. It has no repeats. (That is the only use of finiteness: all the proof needs is a duplicate-free list of the yes-states.) By permanence, used only for the move i, $\mathrm{step}(\cdot, i)$ sends every member of C into C. Being injective, it sends the members of C to members of C with no two landing on the same one, so its images are as many as C itself, and a list of that many distinct

members of C is all of C. So the yes-state step(s, i) equals step(c, i) for some c in C, and injectivity gives $s = c$, so s reads yes. It reads no, and that is the contradiction, the same one the four people and the three inside chairs ran into. ■

A list with no repeats, as long as the set it draws from, contains the whole set. Leave one member of the set out and you have packed the list into one place fewer than it has members. That can't be done: take away one occupied place and the member sitting in it, and the list is still one too long for the places left, and it stays one too long until there are no places left at all. This is the pigeonhole in the form Leurechon would recognise.

None of that is new. It has been sitting in two places for a long time, one in physics and one in the theory of automata, and a physicist will recognise it on sight, so I'll start there. In 1876 Josef Loschmidt pointed out, against Ludwig Boltzmann's account of how a gas settles down, that the laws of mechanics run just as well backwards: reverse every molecule's velocity and the gas retraces its steps, so mechanics alone can't make entropy only ever climb. Twenty years on, in 1896, a young assistant of Max Planck's in Berlin pushed harder. He took Henri Poincaré's recurrence theorem of 1890, which says that in a system preserving a finite measure of its accessible states, almost every starting state returns arbitrarily close to itself, again and again, and used it to argue that nothing in such a system can climb forever: give it time and it comes back down. The assistant was Ernst Zermelo, the same Zermelo who'd put set theory on axioms twelve years later, axioms that keep out the paradox that had sunk Frege's foundations of arithmetic (Section 6 tells that one). Boltzmann answered the same year, not very politely, that the times you'd have to wait for a return are so enormous that nobody would ever see one. (Jos Uffink, who wrote the Stanford Encyclopedia's entry on Boltzmann's statistical physics, calls that reply drenched in sarcasm and bitterness, and calls Zermelo's own paper a careful logical argument.)

Now look back at the table. A step that forgets nothing, on a finite set, is a reshuffle: it moves everybody around in loops, and every loop comes back to where it started. A flag that goes up and stays up is exactly a quantity that climbs and never comes back down. So a reshuffle can't carry one, which is Zermelo's objection with the infinities taken out. On a finite set the loops are enough, and Poincaré's theorem isn't needed. And the wait is at most as many steps as there are states, nothing enormous. Zermelo was aiming it at the second law, and flags never came into it. A quantum recurrence result came in 1957, when P. Bocchieri and A. Loinger proved returns arbitrarily close to the initial state under their conditions on a closed quantum system. That is a neighbouring recurrence argument, not this finite-state proof: even a finite-dimensional quantum system has continuously many states.

Applying it to a physical record requires saying how that record is read and how robustly it distinguishes nearby states.

The second place stings more, because it's in computer science, and somebody got here first, again. Picture a finite machine that reads letters, one move per letter. If every letter just reshuffles the states one for one, automata theorists call it a permutation automaton, and the sets of inputs such machines can pick out are called group languages. Jean-Éric Pin's 1992 paper on reversible automata, machines whose letters act as one-to-one maps, sorts out exactly which languages those machines recognise, and in that world this counting argument is folklore. The short version runs like this. Start at a state that reads no, and collect every input that ends at a yes. Because yes is permanent, adding letters to the end of a collected input keeps it collected. A group language with that property is either empty or everything, and everything would include the empty input, which ends where it started, at no. So it's empty, no run of reshuffles raises the reading, and that's Theorem 3.1 with the moves as letters. Pin's paper is where I'd send you to check the automata side. It isn't about flags or prices, and I'm not claiming it says anything about them.

There is a broader way to look at the same obstruction. Suppose every allowed move can be undone by another allowed sequence of moves. Start outside YES and try to enter it. Once inside, permanence says every allowed sequence keeps you there. But the undoing sequence must take you back outside. Those two demands cannot both hold.

On a finite set, a move that keeps all states apart is a reshuffle. Repeat that reshuffle enough times and every state comes home; all but the last repetition therefore undo one application. So the undoing sequence needed in that argument is available. Morgan Rogers studies systems of composable moves in his 2019 and 2021 papers on toposes of monoid actions. That more general language gives this fact another home. Here the chairs and the undoing sequence already show why it holds.

Going quantum doesn't get around the automata version either. Alex Brodsky and Nicholas Pippenger (2002) showed that a quantum finite automaton that measures only once, at the very end, accepts with bounded error exactly the group languages, the reshuffle kind. One that's allowed to measure after every letter, and to stop on what it sees, does better: it accepts every piecewise testable language, and "has seen a b somewhere" is one of those. A measurement that can stop the machine is a record that stays, and it's the extra thing that lets the bigger class in.

The third place is the closest, and it isn't old at all. In 2023 Thomas Ouldridge and David Wolpert derived the thermodynamics of a finite automaton built the way real computers are: the same physical step run again and again, each time touching only the part of the machine that changes. They sorted languages into two kinds by

the smallest automaton that recognises each one. If that automaton has a letter that sends two states to one, every automaton for the language has such a letter too, and on their derivation recognising the language on a device like that produces entropy, waste no cleverer design gets back, unless the inputs are so finely arranged that only one of the merged states ever actually turns up. If the smallest automaton only reshuffles, they find the language can be recognised with no unavoidable entropy production at all. One of their worked examples is the strings with no b in them, which is my flag turned inside out: the runs that raise it are exactly the strings that have seen a b. So the counting argument, a real price on it, and the case when you already know which state you're in are all in their paper, three years before this book. Their price is derived from the physics of the device, and mine is assumed.

There's a fourth neighbour, and I'd call it a cousin, because it speaks another language. Constructor theory is David Deutsch and Chiara Marletto's programme for stating physics as which transformations a device that runs in a cycle can and can't bring about, and in it a task is irreversible when it's possible and its inverse isn't. Marletto's version of thermodynamics (posted in 2016) leans on exactly that: tasks that are possible while their transposes aren't. Marletto, the physicist Vlatko Vedral and coauthors (2022) made an irreversible task out of the quantum homogeniser, a machine of reservoir qubits that interact by partial swaps, and ran it with photons. Maria Violaris and Marletto (2022) then argued that the same machine, run in a cycle, can turn a pure qubit into a mixed one and can never turn a mixed qubit back into a pure one, even though every interaction in it is reversible, and that this suggests erasure may cost something beyond Landauer's bound. The counting argument is a classical, finite, exact cousin of that sentence: a set of states a step can't leave, a task that's possible, and no way back. Theirs is quantum and approximate and aimed at physics' price; mine is counted, and assumes its price. Deutsch and Marletto's paper on information (2015) has a neighbour of the counting argument in its section 8.6: measuring one observable disturbs another irreducibly, and nothing can undo the disturbance afterwards while the outcome stays in an information variable. It's a second, theory-independent way of saying that a record that stays makes the step irreversible.

So what I can claim here is narrower again, and I'd rather say exactly how. Even the pricing form is, in physics and on their derivation, Ouldridge and Wolpert's: a merge the language can't avoid costs entropy. A price assumed rather than derived isn't mine either: Li and Vitányi (1996) took a unit charge for every bit irreversibly erased, with steps that can be undone priced at zero, as axioms of a model of computation, and Demaine and his coauthors (2016) put a price on named instructions that destroy bits, a unit for an AND or an overwrite. What I didn't find anywhere I looked is what a price on merging moves forces on a reading that stays, charged on a ledger whether or not anybody's physics is right (put a price on

merging and the flip has to pay it, which is the next subsection), and what I get out of that: the toll, as a theorem about the ledger, the merge-or-revoke split for a single move, the exact match between “forced to be priced” and “merges”, the squeeze bound counted in whole units and its entropy form, a built machine walking out through each of the three ways out, and every proof on the page. Those are the forms I didn’t find stated anywhere I looked, and I’d sort them the same way: the counting core classical and somebody else’s, the pricing form only partly mine. The core move is older than the computer, it had two homes before it got to mine, and its physical price had a third.

Count the alternatives before putting a price on them

Write N different starting configurations on cards and spread them across the bench. Run each through the same rule once. When several cards give the same full configuration, put them in one pile. Run the rule again and gather the new piles. Once cards agree on the full state, the same rule gives them the same next state too.

Count the different configurations after each tick. Call their set Q_t and their number q_t, with $q_0 = N$ and $Q_{t+1} = F(Q_t)$. The difference $q_t - q_{t+1}$ tells you how many alternatives disappeared on that tick. Four distinct configurations landing at one state lose three alternatives in one landing pile. Keep those two counts beside each other: three alternatives lost, one landing place.

Theorem 3.2 (The alternatives balance exactly)**.** For a total deterministic rule F and N distinct starts, □

$$\sum_{t=0}^{T-1} (q_t - q_{t+1}) + q_T = N.$$

If all endpoints fit in $K < N$ full states, a transition at some $t < T$ identifies two different members of Q_t. If all endpoints are one full state, the total loss is $N - 1$.

Proof. An image has no more elements than its source, so every difference is non-negative. Add the differences: each intermediate q_t cancels, leaving $q_0 - q_T$. If $q_T < N$, at least one difference is positive; at that step a map from more states to fewer cannot be one-to-one. In particular, one final full state leaves a loss of $N - 1$ alternatives. ■

Turn the cards over before counting them together. Two runs might show the same answer on the front and still carry different inputs on the back. Those are different full configurations. The count follows everything the machine kept.

Now put a price rule beside the piles. Let p_t be the price at tick t, and require every landing pile to have at most 2^{p_t} distinct sources. The number arriving is

then at most 2^{p_t} times the number of places they land: $q_t \le 2^{p_t} q_{t+1}$. Multiply that inequality through the run and you get

$$N \le 2^{\sum_{t<T} p_t} q_T.$$

The rule connects the bill to how tightly the machine can pack the alternatives.

3.2 Now charge for forgetting, and the toll falls out

This is the step I'd been circling since the counting on states. In the last chapter I told you Bennett's rule as he states it: what has to cost is erasing a bit or merging two paths a computation could have taken. I didn't ask you to believe it. I'm not going to ask you now either. I'm going to write it down as a rule you can accept or refuse, in the smallest form that does any work: *any move that forgets something costs at least one.* Any move that is not injective has cost at least one. Notice what it does not say: nothing about certifying, nothing about flags. It's a price on forgetting and only on forgetting.

Put the two together. The first theorem says: on a finite machine with a permanent flag, the move that raises the flag forgets something. The price says: a move that forgets something costs at least one. So the move that raises the flag costs at least one. That is the toll, the rule the toy was handed because I said so, and now a machine has to charge it. The proof is two lines: the first theorem says the flipping move merges, the premise says merges are priced, done. I'll keep saying "premise," because that's what it is, and because a section further on (Section 5) is about what happened to some very smart people who put a price where a premise goes and then treated it as a law of nature, and about half of this one is me trying not to do that.

Premise 3.2 (Merge pricing)**.** Every move that merges costs at least one: for all i, if i is not injective then $\mathrm{cost}(i) \ge 1$.

Observation 3.3 (The toll from finiteness, permanence, and merge pricing)**.** *On a finite machine with a permanent reading whose cost satisfies merge pricing, every move that turns the reading from no to yes costs at least one. That is, the machine with that cost is a Thiele machine in the sense of Definition 2.2, and the universal floor (Observation 2.2) applies to it.*

Proof. A move that flips the reading merges, by Theorem 3.1; merge pricing charges it at least one. ∎

The theorem asks for more than it uses. Only the yes-states need to be finitely many: with a permanent reading and finitely many states reading yes, the rest of the machine as large as you like, merge pricing still gives the toll. With infinitely many it can fail: the first way out below builds a machine that raises a permanent

flag at zero cost while charging every merge. That machine hasn't got out of paying. It pays by taking a state it has never used, and a price list that charges only merges never looks at the fresh states it takes. And merge pricing asks for more than the toll needs. Under those same two conditions the toll holds exactly when every move that flips the reading somewhere and merges is priced. Merge pricing gives that, and the eight-state machine described later in this section, with its jump repriced at zero, meets the toll without merge pricing.

That "more" has an exact size. On a finite machine a move merges exactly when it turns on some reading that it never turns off. If it merges, take the reading "this move can land here." The move never turns it off, because everywhere it lands reads yes; and some state reads no, because a move that sends two states to one lands on fewer states than the machine has, and from that state the move turns the reading on. If it doesn't merge, it turns on no reading it never turns off: that's Theorem 3.1, whose proof asks permanence only of the one move. So on a finite machine a price list meets merge pricing exactly when it meets the toll for every reading at once, each move answering only for the readings it never turns off. The charge on forgetting isn't a second kind of price beside the toll. What the premise chooses is which readings the toll answers to: the flag alone, or every reading a move keeps.

There is one more way to see the same thing, and I give it because it answers a rival I'd otherwise owe you. The rival says: leave certification alone. Price *erasure*, and keep an honest flag that reports when something was erased. Fine; but a system can set its certificate at cost zero with its erasure flag reporting nothing, exactly because the certifying move merged two states and the flag didn't say so. Make the erasure flag honest, so that it reports every move that forgets, and on a finite machine with a flag that stays up the erasure law charges the same cost the toll does. The toll isn't a rival to honest erasure accounting; it's what honest erasure accounting says about the certifying move.

So, is the toll arbitrary? The honest answer is that the toll is a consequence of three things: finitely many states that read yes (a finite memory is plenty), a flag that stays up, and a charge on forgetting. If you accept all three, you don't get to refuse the toll; it's forced. If you refuse any one of them, the toll on my ledger goes back to being my preference. The cost stays where it was; what's gone is the proof that my ledger has to charge it. So the question "did I have to pick it" has turned into three smaller questions, which is what progress usually looks like: a better set of things to argue about. Let me take the three ways out in order, and for each one build the machine that leaves through it, because a premise you can't see the counterexample to is a premise you're taking on faith.

3.3 The first way out: keep everything, and pay by taking fresh states

Take a strip of paper that never runs out of boxes. The machine's whole state is the strip. Each move writes a 1 in the next empty box, and the flag is up as soon as any box has a 1 in it. Check the three premises. It is permanent: once there's a 1, there's always a 1. It flips: the empty strip reads NO, and after one move it reads YES. It forgets nothing: every move can be undone by erasing the last 1, so two different strips never become the same strip. So this is a machine that raises a flag, keeps it up, and never merges anything. The theorem that a permanent flip on a finite machine merges has no grip on it, and a price list that charges only merges doesn't charge it. But look at what each move does: it writes in a box nobody has written in before. The raise takes a fresh box, and so does every step after it, forever. It hasn't got out of paying. It's paying in the other column, by taking a state it has never used, and a price list that only charges merges isn't looking at that column. (Each physical write may incur a model-dependent finite-time cost; what the strip dodges is the floor a merge would add on top.) In the exact form its state is a row of 1s of any length, which is how you say the strip has no end.

If the endless strip feels like a cheat, it's a famous one. David Hilbert, in a lecture course in the winter of 1924–25, told a story about a hotel with infinitely many rooms, every one of them taken, where a new guest still gets a room: everybody moves from room n to room $n + 1$, and room 1 is left empty. The hotel is that other column, the fresh states, told as a joke about a front desk: nobody shares, and every guest in the building packs up and moves one room along, into a room that's occupied as he moves, by a guest who's moving on at the same moment, so that one room at the front is left empty. The guests go one-to-one into a part of the hotel, with a gap left over. The part worth checking is that Hilbert never published it. The hotel became famous because George Gamow put it in a popular book in 1947, and for a long time it wasn't clear Hilbert had told it at all. The historian of science Helge Kragh first guessed that Gamow had made it up. Then Tilman Sauer, a historian of science who had co-edited a volume of Hilbert's lectures on the foundations of physics, pointed him to Hilbert's lecture notes, which had just been printed in 2013, and in 2014 Kragh took his guess back in a paper called "The True (?) Story of Hilbert's Infinite Hotel." Keep the question mark. Even Kragh's own paper puts the lecture in January 1924 in one place and January 1925 in another. Still, this time a story everybody repeated turned out to have a real lecture behind it. I've learned not to count on that happening. Bring it back to the table. In the closed room, the person from the door had nowhere to sit because the chairs ran out. In Hilbert's hotel they never do, so everybody shifts one over and nobody has to share. The strip machine is that hotel. The argument stays the same. What the hotel shows you is that the whole argument was about running out of states all along.

What walked out is the finiteness premise. Now cut the strip to eight boxes. Keep the same rule: fill the next blank box from the left, leaving every earlier mark alone. There are nine possible strips, including the blank one. After eight moves the strip is full. What should the ninth move do? If a full strip stays full, the seven-mark strip and the full strip both go to the full strip. That is the collision. If the machine stops instead, its next-step rule is partial. If it clears the strip, its YES reading comes down. Each escape changes a premise.

Marks so far	Strip	Next strip under the stay-full rule
0	00000000	10000000
1	10000000	11000000
⋮	⋮	⋮
7	11111110	11111111
8	11111111	11111111

Eight unrestricted bits have 256 configurations. These nine are the ones this particular prefix-filling rule permits. A fresh configuration need not use a fresh physical box; changing a pattern can reach one too. Keeping the whole history is exactly the move Bennett made in 1973, when he showed that any computation can be done without forgetting if you let it keep a copy of everything it did; Landauer, back in 1961, had already written that for a machine able to run a program that doesn't stop, the space to hold every intermediate result "cannot be there." Keeping the history is a real way to pay. It costs a memory that never fills, and every machine you can buy has a memory that fills.

Bennett wasn't the first there, or the last. Yves Lecerf, in a short note to the French Academy of Sciences in 1963, had already shown that reversible Turing machines can simulate arbitrary ones, and the note went largely unnoticed. The computer scientist Tommaso Toffoli (1980), and Edward Fredkin and Toffoli together (1982), both then at MIT, built reversible logic: gates and circuits that compute any function and lose nothing, by carrying constant inputs in and extra outputs out. Their basic gate wasn't new even then. Landauer's own 1961 paper writes down a three-bit device that replaces r by $p \cdot q$ when r is 0 and by the negation of $p \cdot q$ when r is 1, calls it logically reversible and capable of an AND, and says each AND then needs its own supplied ZERO and leaves its extra outputs behind. He doubted such a machine could be useful. What Toffoli added was the proof that it computes everything. And the computer scientist Kenichi Morita (1996) proved that reversible two-counter machines are universal, so even a machine as small as the one Section 15 builds, handed its input encoded, can compute everything with every move one-to-one. Every one of them keeps what it doesn't forget somewhere: a history, garbage outputs, a counter that codes the past. The strip is one of those: it keeps the whole history, box by box.

There are three separate claims here, and I need to keep them apart. The strip machine is permanent and forgets nothing. It also has infinitely many states: no finite list holds every finite strip. Neither of those involves a cost. The third does: the strip can raise its reading at zero cost. Count the strip's length as a whole number and let each move add one at a price of zero. Nothing merges, merge pricing holds, and the toll fails. This is what a price list that charges merges and nothing else looks like on a machine with endlessly many states to take: the raised reading shows up at no charge on the ledger because the column it's paid in is one the price list doesn't charge.

The same thing holds for one move instead of a run, and in that form it says exactly how many states the move needs.

□ **Proposition 3.4** (A raise with no merge takes endlessly many states)**.** Let a move never turn a reading from yes to no, let it raise the reading at some state s (no at s, yes where the move sends s), and let it send no two different states to the same state. Then it sends the states that read yes one-to-one into a part of themselves that leaves at least one of them out: the state the move sends s to. So there are infinitely many states that read yes. With finitely many, the raising move merges two states.

Proof. The move sends every yes-state to a yes-state, because it never turns yes to no, and it sends different states to different states. It sends s to a yes-state. If some yes-state c were also sent there, the move would send s and c to one state, so $s = c$; but s reads no and c reads yes. So the yes-states are sent one-to-one into the yes-states with that one left out. If there were only n yes-states, that would put n things one-to-one into $n - 1$ places, which the pigeonhole forbids. So there are infinitely many. ■

The proposition is Hilbert's front desk as a theorem, and it's also exactly how Richard Dedekind defined an infinite collection in 1888: one that can be matched one-to-one with a part of itself that leaves something out. So with no merge, every raise has to send the kept things one-to-one into a part of themselves with a gap left over, and only endlessly many of them have a gap to spare. Taking a fresh state isn't the absence of what keeping takes but the other way of paying it, and only a machine with endlessly many states can keep paying that way.

Which state is being counted. Whether a move merges depends on what the state keeps. The strip keeps its history, so nothing merges; cut it to eight boxes and the stay-full rule merges. Section 19 proves the general form (Theorem 19.2): keep the list of every earlier state alongside the current one and every fixed-instruction step is one-to-one. (Keeping only the one state before isn't enough: two states that differ only in what came before them still land together.) So a merge price is a fact about

a move on a stated state space. Each price derived in this book is derived on the whole state its definition gives, and three things go with it: which state that is, what lies outside it (nothing outside it is charged), and whether the move is one primitive instruction or a composite run of several. Relabelling the states one-to-one changes no price, since it sends merges to merges and one-to-one moves to one-to-one moves. Adding history to the state, or projecting part of the state away, changes the machine being modelled, and the price can change with it.

The first way out deserves a number, because "a memory that never fills" is doing all the work and I'd rather see what keeping takes than take it on trust.

Proposition 3.5 (Merge, or a fresh state every step)**.** (1) Let a machine run by one rule, so that every state has exactly one next state, and let its reading never go from yes back to no. Take any run in which the reading goes from no to yes. Then either some step of the run merges two of its states (two different states of the run have the same next state), or the run never visits the same state twice. So on a machine with M states, a run that keeps a record and never merges has fewer than M steps. (2) Let an operation send every value to one fixed value and leave that value alone. Take a machine with M states run by one rule, a reading of its states, and N different starting states, exactly one of which already reads the fixed value, such that from every start each of the next T states reads the fixed value. If no two different states among those the runs sit in at steps 0 to $T-1$ have the same next state (the operation is carried out through T repetitions without merging), then $M \geq 1 + (T+1)(N-1)$. When $N \geq 2$, a bounded memory delays the merge and does not avoid it; with only the resting start there need be no merge. □

Proof. (1) Suppose no two different states of the run share a next state, and suppose the run still visits some state twice, once early and once later. Walk the two visits back together, one step at a time: two equal states whose predecessors are both in the run had equal predecessors, because nothing merged. The early visit gets back to the start first, and the later visit is then at the start too. So the run comes back to its start after some number p of steps, and since one rule gives the next state, it comes back again after $2p$, $3p$, and so on forever. The start reads no: a yes there would stay yes all along, and the reading does go from no to yes. Some of those returns come after the raise, where the reading is yes and stays yes, and each of them is the start, which reads no. So the repeat was impossible. A run that never merges takes a state it hasn't used at every step, and with M states that's at most $M-1$ steps. (2) Count the states along paths. Walking two equal states back together, as in (1), is allowed at any step from 1 to T, because the states one step earlier are at steps 0 to $T-1$. Each of the $N-1$ starts not already in the fixed reading, followed through

T repetitions, occupies $T+1$ states, and none twice: a repeat would walk back to the start equalling a later state of its own path, and every later state reads the fixed value while the start does not. Two of those paths cannot share a state either: walk back to the earlier time, and either two different starts are one state, or a start is a state that reads the fixed value. That is $(T+1)(N-1)$ states. The start that already reads the fixed value, followed through T repetitions, ends in one more: if that state were on one of the paths, walking back would again put a start at a state that reads the fixed value. One more is all that start is owed, not $T+1$: every state on its own path reads the fixed value, so walking back along it finds no contradiction, and the path may fold onto one state, as it does in the machine built below. ■

The number is exact, not only a floor, and you can build the machine that reaches it. One state for the start that already reads the fixed way; the operation leaves it there. For every other start, lay out a row of states, one state longer than the number of repetitions you want to survive. The first state in the row reads the way that start reads, the rest read the fixed way, and the operation walks the row, sending its last state to the common fixed state. This also works for zero requested repetitions, when a row contains only its start. Count the states: one, plus a row for each of the other starts. For as many repetitions as you asked for, no two distinct visited source states land on one. If there is at least one other start, the next repetition merges its row's last state with the fixed state: both go to the fixed state. With no other start, the one fixed state idles forever and no finite retention horizon follows. A single fixed reading isn't needed either. If the operation sends every state to one of a few resting states and leaves each resting state alone, and every resting state is one of the starts, the same walking back counts the same rows inside each resting state's own catchment, the starts that end at it. The total comes to one state per start, plus, for every repetition, one more state for each start that isn't a resting state. With one resting state that is the bound in the proposition, and the same rows in each catchment show it exact. In the exact form: N starts, L of them resting states, T repetitions, at least $N+T(N-L)$ states, and that many are enough. In bits: a memory of B bits has at most 2^B states, so surviving T repetitions with one resting state takes $B \geq \log_2\bigl(1+(T+1)(N-1)\bigr)$. The memory grows with the logarithm of how long the history is kept; a fresh state at every step isn't a fresh cell at every step.

So the strip uses fresh configurations for as long as its record stays up without a collision. Cut its permitted state space to a finite size and the counting bound applies. A configuration is a pattern, not a newly allocated cell. Bennett's 1989 construction saves storage by undoing and redoing intermediate work while retaining enough to recover it. Those extra transitions buy a time–space tradeoff. They do not let a fixed autonomous run under this fresh-state premise reuse a configuration.

Bennett's Brownian model supplies a separate tradeoff: reducing its bias reduces its mean forward speed and its heat per net forward step. At zero bias its mean drift is zero, although individual random walks still move. Theorem 1.9 concerns that model, not every physical step. There are two accounts: configurations in the structural argument, and entropy production under specified thermal dynamics. The total autonomous run just counted, after a new permanent raise, must have a collision somewhere in its continued orbit or take endlessly fresh states. The collision need not be on the entry arrow. Counting gives that alternative; a physical implementation determines its resource costs.

Two levers, and whether their hidden moves commute

Put two reset levers on the box. Pull left then right, and look at the tabletop. Now start over and pull right then left. The picture agrees. Open the box as well: are all its hidden parts in the same places? That is the extra demand in this example. The order of the levers must leave the whole machine the same.

Use three tabletop values, $0, 1, 2$. The first lever wipes only 1 to 0. The second wipes only 2 to 0. They commute on the tabletop. Choose one starting full state above each value and allow every word of at most T lever pulls. Call everything reached by depth t R_t. Through the promised horizon, each lever must give different successors to different states in R_{T-1}, even if those states were reached at different times. You know which lever was pulled, so the two levers may reuse a landing state.

Proposition 3.6 (The exact room for exclusive resets)**.** For $T \geq 1$, the minimum finite installed carrier has $T+3$ states without a commutation requirement, and $2T+2$ with commuting hidden levers. These are also the respective minimum sizes of R_T when larger carriers are allowed. With commuting levers that are one-to-one on their *whole* domain, the minimum $|R_T|$ is $2T+3$, attained for every T by one infinite machine. At $T = 0$ the reachable minimum is three in each class. □

Count the zero states first. Write F, G for the hidden levers and z, a, b for the starts reading $0, 1, 2$. At depth t, let Z_t contain the states reading zero. The first lever takes every state in Z_t, and also a, into Z_{t+1}. Their successors must be distinct. So $|Z_t| \geq t+1$. Add a and b and there are at least $T+3$ states.

A single line of zero states reaches that number. Use $z_0, \ldots, z_T$, advance the line with either lever, and fix its last state. Put $F(a) = z_0$, $G(b) = z_0$, $G(a) = a$ and $F(b) = b$. The final zero state stays outside R_{T-1}. Try the levers in opposite orders at a, though, and the hidden states disagree. To make them commute, count again.

Counting when the hidden levers commute. Start above 1 and pull only the lever that leaves 1 alone on the tabletop. The hidden box can still move. Watch its first $T+1$ states, $a, G(a), \dots, G^T(a)$. Do the matching experiment above 2, watching $b, F(b), \dots, F^T(b)$. The first list reads 1 throughout; the second reads 2. If either list never repeats, its $T+1$ states, the at least $T+1$ zero states already counted, and the other starting state give $2T+3$. There is enough room already.

So suppose both lists repeat. A deterministic run first follows an approach and then enters a loop. Let the first approach take r pulls and its loop take p pulls; let the second take s and q. The loop lengths p, q are at least one. Call their first loop states $\alpha = G^r(a)$ and $\beta = F^s(b)$. There are $r+p$ different states on the first approach and loop, and $s+q$ on the second. All have been reached within the promised horizon.

At those loop states, switch levers. From α, the first F pull changes the reading from 1 to zero. From β, the first G pull changes 2 to zero. There are $L = T-r$ pulls left in the first experiment and $M = T-s$ in the second. Write the resulting two rows of zero states as

$$A = \{F(\alpha), \dots, F^L(\alpha)\}, \qquad C = \{G(\beta), \dots, G^M(\beta)\}.$$

Neither row repeats. If two entries agreed, walk their predecessors backwards together. All those predecessors occur by depth $T-1$, where the lever keeps sources apart. Eventually the earlier entry would reach the nonzero seed while the later one still read zero. That is impossible. The rows therefore contain L and M states.

Can the rows cross? Suppose their ith and jth entries are the same state. Pull F another q times. Seen from the second row, commutation lets you do those pulls at β before its j pulls of G. They take β once around its loop and bring it back. So the shared state returns to itself after those q pulls of F. The same argument, exchanging the levers, returns it after p pulls of G:

$$F^{i+q}(\alpha) = F^i(\alpha), \qquad G^{j+p}(\beta) = G^j(\beta).$$

I have just proved there are no repeats within either displayed row. The returns must therefore lie beyond their ends: $i+q > L$ and $j+p > M$. Only the last q entries of the first row and the last p of the second can be shared. There are at most $\min(p,q)$ shared states.

Now add the two nonzero lists and both zero rows, subtracting the shared entries once:

$$|R_T| \geq (r+p) + (s+q) + L + M - \min(p,q) = 2T + \max(p,q).$$

The approach lengths disappear because every pull spent on an approach leaves one fewer pull for its zero row. If either loop has length at least two, the count is at least $2T+2$.

Only two one-state loops remain: $p = q = 1$. If the zero rows do not meet, adding them without a subtraction gives $2T + 2$. If they meet, the crossing restriction puts their only shared state at both ends. Call it c. It is fixed by both levers. The initial zero state z must now supply another place. To see why, suppose it lay on one of the rows. Starting there would reach c in at most $T - 1$ pulls. The state just before c on the first row is also reachable by then, and is different from c. The first lever sends both to c, violating its promise on R_{T-1}. Thus z lies outside both rows. Count it, and the total is $2T + 2$ again.

Finally strengthen the promise: each lever keeps different sources apart everywhere, even beyond the horizon. A crossing would give the return $F^{i+q}(\alpha) = F^i(\alpha)$ above. Now you may walk backwards all i steps, without a horizon restriction, giving $F^q(\alpha) = \alpha$. The left side reads zero and the right side reads 1. So in this case the rows cannot cross at all. Their count with the nonzero lists is $2T + p + q$, which is at least $2T + 3$ unless both loop lengths are one.

In that last case you need one more state. If z is outside the rows, it is the extra state. If it lies at $F^i(\alpha)$ on the first row, pull F from z another T times. This is allowed from an initial state and reaches $F^{T+i}(\alpha)$. The whole zero row can never repeat under the global promise, by the same backward argument used earlier. This new state lies past its displayed end. It cannot be on the other row either, because the crossing argument still applies, however far you go. Its zero reading also keeps it off the two nonzero lists. If z lies on the second row, pull G instead and make the same count. Every case gives at least $2T + 3$. ∎

Build the finite box with two arms and a shared endpoint. The F-arm has $a_0, \dots, a_{T-1}$; the G-arm has $b_0, \dots, b_T$. Each lever advances its own arm and fixes the opposite arm. Both fix the common end. Read a_0 as 1, b_0 as 2, and everything else as zero. Start at those two heads and at $z = b_1$.

At horizon one, there are just four places. Write every arrow down:

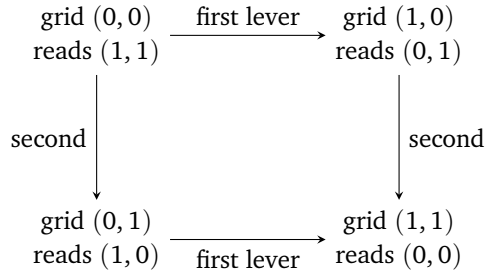
State	Tabletop	Pull F	Pull G	Start?
a_0	1	end	a_0	yes
b_0	2	b_0	b_1	yes
b_1	0	b_1	end	yes
end	0	end	end	no

On the three starts each column has three different destinations. On the whole four-state carrier each column has a repeated destination. The horizon promise and the everywhere promise are different. Try both orders at b_0: either order ends at b_1. The other three rows also agree in both orders. The endpoint first appears at depth T. By then every one of the $2T + 2$ states has been reached, and until then each lever keeps its sources apart.

For one-to-one levers on the whole machine, keep the two arms disjoint and let them run forever. Give them a separate fixed zero state, and start at that state and

the two heads. After T pulls the reachable population is exactly $2T + 3$. The same box works at every horizon.

Change the tabletop and the chairs can fit differently. Give it two bits, with one lever clearing the first and the other clearing the second. Lay out an infinite grid with nonnegative integer coordinates (i, j) and a separate blank state. One lever adds one to i, the other to j, and both fix blank. Read a grid point as $([i = 0], [j = 0])$ and blank as $(0, 0)$. Start at blank and the three grid points $(0, 0), (1, 0), (0, 1)$.



Both orders reach the very same grid point, not just the same reading. The separate blank state also reads $(0, 0)$ but is fixed by both levers; it is not the grid's origin.

A pull adds one to $i + j$. The three grid starts have sums zero or one, so no word of length at most t can go beyond $i + j = t + 1$. Conversely, the origin is a start. For any other point in that triangle, start at $(1, 0)$ if $i > 0$, and at $(0, 1)$ otherwise. Exactly $i + j - 1 \leq t$ pulls reach it. That gives the whole triangle. Its rows have $1, 2, \ldots, t + 2$ points. Pair the first row with the last and count, then add blank:

$$|R_t| = 1 + \frac{(t + 2)(t + 3)}{2}.$$

Adding to one coordinate is one-to-one on the grid, and neither grid shift reaches blank. Adding to different coordinates in either order gives $(i + 1, j + 1)$, so the levers commute on the whole carrier. The reading follows the two clears because incrementing a coordinate makes its zero-test false and leaves the other zero-test alone.

This count also meets the lower bound. Each one-cleared fibre starts with one state. The appropriate clear sends that fibre and at least one $(1, 1)$ state into it injectively, so its population increases by at least one per depth: at depth t it needs $t + 1$. The $(0, 0)$ fibre also starts with one. At depth $j + 1$, one clear sends both its j-depth population and a one-cleared fibre of at least $j + 1$ states into $(0, 0)$ injectively. Its population therefore increases by at least $j + 1$. Summing from $j = 0$ to $t - 1$ gives $1 + t(t + 1)/2$. Add the one $(1, 1)$ state and the two one-cleared populations: $1 + 2(t + 1) + 1 + t(t + 1)/2 = 1 + (t + 2)(t + 3)/2$. The three-value box needed extra room for commutation. This two-bit box fits its best count while keeping commutation intact.

3.4 The second way out: let the flag come back down

Take one coin. The rule is: turn it over. Heads is YES, tails is NO. Every move is undoable (turn it over again), so nothing is forgotten. Tails goes to heads, which is a flip, NO to YES, and heads goes to tails, so the same move takes the flag back down somewhere else. There are two states, as finite as it gets, and the move that raises the flag forgets nothing, so a schedule that charges only for forgetting is allowed to charge it nothing. The move is injective and can switch the reading both on and off. Pricing the turn at zero meets merge pricing, since no move merges, and fails the toll. The premise dropped on this way out is permanence.

There's a fact here sharper than the premise it replaces, and it was the thing in this section that surprised me. Drop permanence entirely and keep only finiteness, and take any move that raises the flag anywhere. Then one of two things is true: the move forgets something, or the same move lowers the flag somewhere else. There's no third option, and the proof below is a search: go through the finitely many YES states looking for one this move turns to NO. Find one, and that's the revocation; find none, and the move is permanent at least for itself, which is all the argument from fresh states needs. So a flag that goes up without a merge is a flag the same move can take back. A flag that nothing takes back is paid for. You can't have a raise at zero cost that stays raised, not on any finite machine, not with any price schedule that charges for forgetting. The sentence is narrower than "every certificate costs." It's true, and I'll take it. The broad one would only be mine.

Theorem 3.7 (Merge or revoke)**.** On a machine with finitely many states reading yes (no permanence assumed), if $\mathrm{cert}(s) = \text{no}$ and $\mathrm{cert}(\mathrm{step}(s, i)) = \text{yes}$, then either i merges, or there is a state t with $\mathrm{cert}(t) = \text{yes}$ and $\mathrm{cert}(\mathrm{step}(t, i)) = \text{no}$. □

Proof. Check every yes-state on the finite list for one that i sends to no. If there is one, that is the second alternative. If there is none, then i keeps every yes-state at yes, which is permanence for the single move i; the argument of Theorem 3.1 uses permanence only for the move it is about, so it applies, and i merges. ■

I'll give you the three-card version too, because it shows that a move can be forced to be priced without writing anything permanent, and I don't want you to think the picture is tidier than it is. There are three cards, T0, T1, T2, with a star on T1. The rule is that T0 goes to T1, T1 goes to T2, and T2 goes to T1. T0 gains the star (a flip). T1 loses it (a revocation). T0 and T2 both land on T1 (a merge). So this move is priced, by the forgetting rule, and yet it writes nothing permanent; it takes a star away in the same breath. "The priced moves are exactly the permanent records" is false, as the three cards just showed. What's true is one direction: on a finite

machine, every permanent raise is a priced move. The other direction has the three cards standing in it. And "priced" here has an exact meaning worth one sentence: a move is *forced to be priced* when every schedule that charges for forgetting charges it, and that holds exactly when the move forgets something, with no finiteness needed at all.

Definition 3.3 (Forced to be priced)**.** A move i is *forced to be priced* when every cost function satisfying merge pricing (Premise 3.2) charges i at least one.

□ **Theorem 3.8** (Forced price is merging)**.** A move is forced to be priced exactly when it merges. No finiteness is needed.

Proof. If i merges, merge pricing charges it at least one, whatever the cost function. If i does not merge, define the cost that is 0 on i and 1 on every other move. It satisfies merge pricing, since every merging move is some move other than i and ■ costs one, and it charges i nothing, so i is not forced.

A cost here is any rule from move names to whole numbers, so that one is there whether or not anything can tell i from the other names. A test that tells them apart comes in only when a cost has to be computed by a program, and then the program needs it to tell i from the rest.

□ **Corollary 3.9** (Permanent records are forced)**.** On a finite machine, a move that never turns the reading off and turns it on somewhere is forced to be priced. (By Theorem 3.7 it merges, and merge pricing charges every move that merges, whatever the cost function.) The converse fails: the three cards are forced, because they ■ merge, and they revoke.

3.5 The third way out: don't charge for forgetting

The third way out is the easiest to describe and the hardest to argue about, so I'll describe it and then spend the rest of the section on the argument. Take a sheet of paper and let its reading be just BLANK or STAMPED. There is one move: stamp it if blank, and leave it alone if already stamped. It is finite, with two readings. It is permanent, since STAMPED stays STAMPED. It has a flip, since BLANK goes to STAMPED. Both possible readings now end at STAMPED. The paper's fibres, the ink and your memory are outside this two-state description; the merge is a loss from the reading, not the disappearance of every physical trace. Everything the argument

from fresh states needs is here, and the move merges. Now set the price of that move to zero. The toll fails, and you can watch it fail: the flag went up at zero cost. The premise dropped on this way out is the charge on forgetting. (A price of one on that move satisfies all three premises, so the premises can hold together; the third way out is about whether they must.) Nothing in counting stops you from setting that price to zero. Only a claim about the world does. The nearest such claim is Landauer's, and I have to be exact about the gap between the two: the premise as written charges *every* forgetting move, whether or not you knew which state you were in, and Landauer's own heat, as the coin under the cup below will show, is owed only for what you didn't know. Merge pricing keeps only a yes-or-no distinction: whether any pair can merge. The one-bit reset is where it comes from, and it measures neither the full worst-case entropy loss nor the heat of any particular preparation. The counting needs only that rule.

Here is what Landauer said, in 1961, as exactly as I can put it. I went past the summaries and read the paper itself. A logical operation with no single-valued inverse (a step that forgets, in this book's words) must produce heat, "typically of the order of kT" per operation, where k is Boltzmann's constant and T is the temperature; and for the specific case of resetting one bit whose value you don't know, the minimum is kT times the natural log of 2, about 0.69 kT. I want you to hear two things in that. First, "of the order of": he was not handing out a number to three decimals. Second, and this surprised me, he did not measure anything. The paper is argument, with models. He writes that he can "show, or at least very strongly suggest" the conclusion, and that his reasoning gives "no guarantee that this minimum is in fact achievable." That's the founding paper of the whole subject, and it's a careful man saying "I think this is right and here's why." I like him more for it.

Was it measured, eventually? Yes, and only recently. In 2012 a group at the École Normale Supérieure in Lyon, Antoine Bérut and Sergio Ciliberto's lab working with Eric Lutz, trapped a single glass bead, two millionths of a metre across, in a double well made of laser light, and used the two wells as the two values of a bit. Then they erased the bit, by lowering the barrier, tilting the trap, and raising it again, many times over, slower and slower, and measured the heat each time. The average heat came down toward Landauer's kT ln 2 as the erasure got slower, closer the slower it went, and never under it, on average. Single runs went under it; the bound is about the average, and any one erasure can beat it. And if you only succeed in erasing half the time the floor drops to zero, which the paper's formula shows and which I think is the most honest line in it. Others have done it since with different devices. So it was predicted in 1961 by argument and seen in a laboratory fifty-one years later.

And is it settled? No, and I'd be lying by omission if I left that out, because the people who doubt it are not cranks. John Norton, a philosopher of physics, has

argued since 2005 that the usual proofs of the principle quietly assume what they're trying to prove, and that the kinds of process they assume, combined, could be made to break the very law the principle is supposed to guard. An earlier objection, made in 1998–99 by Norton and his Pittsburgh colleague John Earman, also a philosopher of physics, was that the principle can't be independent of the second law. Charles Bennett, who did more than anyone to build the subject, rated that one "in my opinion the one of greatest merit," and then disagreed with it. Others replied to Norton; Norton replied back. The Stanford Encyclopedia entry on the topic, written by a philosopher of physics who works on exactly this, ends without a verdict. I've read what I could of both sides and I'm not qualified to referee them, and I'm not going to pretend the laboratory settles a question about what the principle means in general, because measuring one bead doesn't do that.

What I did with it is this, and I think it's the only honest thing to do. The charge on forgetting is written into this book as a *named premise*. It's a definition the theorems take as an assumption, and nowhere is it proved as a general fact, because it isn't provable from counting; for a particular machine with a particular price list you can see it hold (the stamp at cost one, above), which is a different thing. If Norton turns out to be right, every theorem in this section still holds; they just have one premise fewer that the world agrees to. If Landauer's side turns out to be right, it supports a thermal price under its conditions; the whole-number charge on every merging move is still the ledger rule stated here. Either way, nothing in this section says a device does anything until someone measures the device. The counting is mine to prove. The physics is a premise I've named, and what settles it is a measurement, which a proof can't do.

Naming a premise, though, isn't the same as leaving it unplaced, and a hedge isn't a position, so I'll give mine. Earman and Norton's objection is a dilemma. Either an information-theoretic exorcism of the demon assumes the second law, which they call the *sound* horn, and then it adds nothing the second law didn't already give; or it rests on a new principle tying information to entropy, the *profound* horn, and nobody has given that principle independent grounds. A named premise is the sound horn, on purpose. That's where Bennett ended up in 2003, and Norton's "Too Good to Be True" (written in 2025, published in 2026) says the careful literature almost always accepts that horn, and argues that whatever succeeds there is ordinary thermodynamics relabelled. I'll take that. Ordinary thermodynamics, relabelled onto a ledger, is all this section asks of the physics. And on the counting I'm on Norton's side outright. His 2018 paper "Maxwell's Demon Does Not Compute" exorcises the demon with Liouville's theorem and nothing else, no information and no computation: dynamics that keeps every patch of states the same size can't crowd a big set of states into a small one. The counting core of this section, that a step losing nothing can't carry a no-state into a set that stays up, is that point in finite,

discrete form. And if you want the best case for the premise itself, the one I found is James Ladyman, Stuart Presnell, Anthony Short and Berry Groisman (2007): from one of the standard statements of the second law, the one named for Kelvin, they derive that a logically irreversible operation can't be carried out thermodynamically reversibly by any of their L-machines. That's a physical argument under their stated model of a machine. It deserves to be argued with on those terms; it does not by itself establish the book's whole-number ledger charge.

One question a physicist asks before any of that, and it's a good one: the laws underneath never merge anything. Classical mechanics carries each exact state forward to exactly one next state and could carry it back again, and Liouville's theorem adds that a patch of states keeps its size as it moves; quantum mechanics is unitary, which is one-to-one too. So when this section says a step merged two states, where did the difference go? It went out of the machine. The merge is in the machine's own description, the states its reading is about. The machine together with everything around it still runs one-to-one, and the distinction the machine forgot survives. It gets pushed out into the surroundings. The surroundings are the endless strip of the first way out, the machine that keeps every history: the column of states never used before, which the universe as a whole may have to spend and the machine doesn't, because the machine isn't the universe and its memory fills. Pushing a distinction out isn't yet heat: another memory could keep it as a readable record. It's heat when the reset removes real uncertainty into a thermal reservoir, and then Landauer's premise bounds the mean heat handed over. None of that heat has to be heat *wasted*. Takahiro Sagawa's 2014 analysis says that erasure can, in principle, approach a process that produces no entropy overall: the reservoir's entropy goes up by what the machine's goes down. A positive mean heat transfer can then be unavoidable without being wasteful. A known input has no uncertainty to remove, and this entropy-loss premise alone sets no positive heat floor for it. The reset can dissipate under the stated thermal-model assumptions, and a reset device that wasn't built for that one input keeps a floor that survives going slowly (Section 3.7).

Ladyman and his coauthors and Sagawa aren't disagreeing. They use "reversible" for two different things. Ladyman's L-machines are devices, and a device counts as reversible only if it produces no entropy on every input it's fed, each one known (their section 4: "the L-machine is thermodynamically reversible if and only if" every one of its processes, one per input, produces no entropy). Sagawa's reversible erasure is a process on one given spread of inputs, with the device tuned to that spread. The standard eraser is reversible in Sagawa's sense and not in Ladyman's: on an even mix of 0 and 1 it produces no entropy, and on each known input it produces $k_B \ln 2$. One identity holds the two together. For a fixed device that sends every input to one output, and any spread q of its inputs, the average over inputs of the

entropy produced on each known input equals the entropy produced on the spread q, plus k_B times the Shannon entropy of q. The identity holds because the heat is the same average either way, the device being fixed, and the output is the same either way. The only difference is at the start: the spread's own entropy includes $k_B H(q)$ for not knowing which input it is, and each known input's doesn't. Sagawa makes the first term on the right zero; Ladyman reads the left side, which is then $k_B H(q)$, and more than zero.

3.6 How much: the price grows with the squeeze

The toll charges at least one unit, and that's a floor for raising one flag. The counting of states says more if you let it. Say there are m states that already read yes and k more are turned to yes in one move. (That's a different k from the one in Landauer's kT; when the two meet, below, Boltzmann's constant gets a little B hung under it, k_B, so you can tell it from a count of states.) All m plus k of them end up on the m yes-states. That's a squeeze by a factor of (m + k) over m, and if you accept a slightly stronger version of the forgetting price, that each unit of cost pays for at most one *halving* of the number of distinguishable states, then the move costs at least the number of halvings in that squeeze, which is the base-two logarithm of (m + k)/m.

Back to the stamp from the third way out, but give the reading four possible values this time: A, B, C and YES. Write them on four cards. Choose one secretly, then cover its face with an identical opaque YES label. Read only the new face: all four possibilities now give YES. Before the move, identifying the original face took two yes-or-no questions (one of the first two? then which one?); afterward there is only one possible face to report, so that reading has lost two bits, which is two halvings. Peeling the label off would recover the old face, so this tabletop version models the loss from the reading; it does not demonstrate physical erasure. So that move costs at least two units, and two is enough, since a cost of exactly two satisfies the halving rule. The general statement is m + k at most 2 to the cost times m, and there is also a logarithm form that rounds both logarithms up and is weaker for it (three states and one newcomer round to the same two bits on both sides, and the rounded form says nothing). The halving premise, like the one before it, is a definition used as an assumption. The units are halvings. To turn a halving into joules of heat you multiply by $k_B T \ln 2$, and that multiplication is physics; the counting stops before it.

That little k_B deserves a short trip of its own, because it sits exactly where counting turns into physics, and its name is a small history lesson. Ludwig Boltzmann's grave in Vienna has $S = k \log W$ carved on it (that k is the same k_B, before anyone hung a B under it): entropy is a constant times the logarithm of the number of ways

the parts could be arranged. The idea is his. The constant, and the equation written that way, came from Max Planck, in his work on radiation around 1900. In his Nobel lecture in 1920 Planck said the constant "is frequently termed Boltzmann's constant, although to the best of my knowledge Boltzmann himself never introduced it," and guessed that Boltzmann never believed anyone could measure it accurately. I'd assumed Planck named it after Boltzmann. His own words point the other way: it was other people calling it that, and who first did, I couldn't pin down. So the most famous constant in this corner of physics wears the name of a man who didn't write it down. Does that change the heat floor? It doesn't, but look at the job the constant does in that equation. Take it away and what's left is the logarithm of a count, which is a number of halvings, which is what the squeeze bound hands you. The constant is the exchange rate, and whether the world honours that rate for a merge is the premise. No theorem here settles it.

The premise, stated exactly, needs one new piece of notation: for a list D of states, write $|D|$ for how many it has, and $\mathrm{img}_i(D)$ for the number of *different* states the move i sends them to.

Premise 3.4 (Halving price)**.** Each unit of cost pays for at most one halving: for every move i and every list D of states with no repeats, $|D| \leq 2^{\mathrm{cost}(i)} \cdot \mathrm{img}_i(D)$. (It implies merge pricing, since a merge sends two different states to one and so needs $\mathrm{cost}(i) \geq 1$.)

There's a plainer way to read that premise, and it's worth having because it says what the premise is *about*. Look at one landing spot of a move and count how many states land there; call the biggest such count the move's worst pile-up. The premise holds exactly when the cost covers the worst pile-up in halvings: a cost of one allows piles of two, a cost of two allows piles of four, and a pile of three already needs a cost of two. In one direction, hand the premise one whole pile. It lands on a single spot, so the premise says the pile has at most that many members. In the other, any list of states splits by where its members land, each part sits inside one pile, so the list is at most the number of landing spots times the worst pile-up, which is what the premise says. So the halving price asks a move to pay for the worst it could ever do, whichever state it's actually handed. The worst case is on purpose, and it has a home in physics. In the branch of thermodynamics that prices a single run that has to succeed every time, with no averaging over many runs, Philippe Faist, Frédéric Dupuis, Jonathan Oppenheim and Renato Renner derived in 2015, in that framework, what they give as the least work any logical process needs, and when every input might occur and no error is allowed, their bound comes to $k_B T \ln 2$ times exactly that logarithm. Take an AND gate as the example: three inputs land on 0, so it needs at least $\log_2 3$, about 1.6, halvings' worth of work (two units, in whole

numbers). The premise asks for that bound of a device that has to work whatever it's handed, and it counts it in units of cost. Joules don't come into it.

□ **Theorem 3.10** (The squeeze bound)**.** On a finite machine with a permanent reading and a cost satisfying the halving price, let m be the number of states that read yes, and let a move i turn k different no-states to yes. Three numbers are in play: m counts every yes-state of the machine, reachable or not; k counts a list of distinct no-states that the move turns to yes, and the move may turn others too; and $\mathrm{cost}(i)$ is the price the cost assigns to the move. Then

$$m + k \;\le\; 2^{\mathrm{cost}(i)} \cdot m, \qquad \text{and, when } m \ge 1, \qquad \mathrm{cost}(i) \;\ge\; \log_2 \frac{m+k}{m}.$$

When $m \ge 1$ the two displayed bounds say the same thing. (When $m = 0$ there is nowhere for a newcomer to land, so $k = 0$ too.)

Proof. Put the m yes-states and the k newcomers in one list; it has $m + k$ different members. By permanence every yes-state lands on a yes-state, and by hypothesis every newcomer does too, so all $m + k$ land among the m yes-states: $\mathrm{img}_i \le m$. The ■ halving price gives $m + k \le 2^{\mathrm{cost}(i)} \cdot \mathrm{img}_i \le 2^{\mathrm{cost}(i)} \cdot m$.

The four card readings are the case of one yes-state and three newcomers, where the bound is reached. With one newcomer the bound says the move costs at least one, which is the toll again. The counting argument also shows that at least as many states collapse as there are newcomers, without any price premise.

The bound is reached, whatever the numbers. Pick how many states already read yes and pick a cost. As long as the old yes-states and the newcomers together fit under that cost's pile-up allowance (a cost of one allows twice the yes-states, a cost of two allows four times), there is a finite machine with a permanent reading and the halving price that has exactly that many yes-states and a move of exactly that cost turning those newcomers to yes. The move turns every other no-state of that machine to yes as well, filling the allowance to the brim. No cost meeting the halving price on that machine charges the move less. So the least whole-number cost that meets the bound is exactly the rounded-up logarithm, $\lceil \log_2 \frac{m+k}{m} \rceil$.

Four more facts about the halving price, and the last one is a limit. First, the plainer reading above is a theorem on a finite machine: the halving price holds exactly when no move has a landing spot with more than $2^{\mathrm{cost}(i)}$ states landing on it. Second, for the eight-state machine described later in this section that's a theorem about its own price list: no move has a larger pile-up than 2^{cost}, so the halving price holds, and the toll follows from it; and the price list is the least that meets the halving price, since any price list that does charges every move at least what this

one charges. Third, the squeeze bound is met with equality there: its stamp turns on the four states whose flag is down, beside the four whose flag is already up, and $4 + 4 = 2^1 \cdot 4$. Fourth, and this is the limit, the small machine of Part III (Section 15) as a whole does not meet the halving price, whatever procedure you use to decide whether two of its states are equal, because one of its zero-price moves, the one that takes one off a number the machine holds, lands two different states of the whole machine on the same state (the pair is in Theorem 15.22), and a pile of two at price zero is all it takes to break the halving price. What does hold there is merge pricing asked of the record's part of the state alone, and it gives the toll on that machine too (Theorem 3.19, below).

3.7 The same bound, written in bits, and when it's zero

There's one more way to say all this, and I include it because it's the way physicists say it and because it says exactly when the heat floor is zero: when no two states you might be in land on one. Its two ends are a move that forgets nothing and a move on a state already known, and the second I find more instructive than any case where it isn't.

Claude Shannon, in 1948, gave a number to "how much you don't know": if something could be in any of n states and you have no idea which, your uncertainty is log base two of n bits, and that's the most it can be for n states. If you push your uncertainty through a step that forgets nothing, it stays the same; through a step that merges two states you might have been in, it drops. Now push it through the permanent flip. Before the move, you might be in any of the m + k states in play. After it, you're in one of m. So your uncertainty afterward is at most log2 of m, whatever you believed before. If you started with no idea at all, every one of the m + k states equally likely, your uncertainty was exactly log2 of (m + k), and the move removed at least log2 of (m + k)/m bits: the same number as the halving count, now in Shannon's units. Landauer's principle, in the form physicists write it, then says a step that removes that many bits from the state gives off at least $k_B T \ln 2$ of heat per bit. Put the two together and the flip gives off heat, and the exact statements below say how much and under which side conditions, one of which (the temperature has to be taken as zero or above) is easy to skip and shouldn't be.

Why does Shannon's number share a name with the physicists' entropy at all? There's a story about it. Shannon wanted to call it uncertainty, and the mathematician John von Neumann told him to call it entropy instead, for two reasons: the same formula already lived in statistical mechanics under that name, and nobody really knows what entropy is, so in an argument he'd always have the upper hand. It's a great story. It comes from Myron Tribus, who spent years tying information

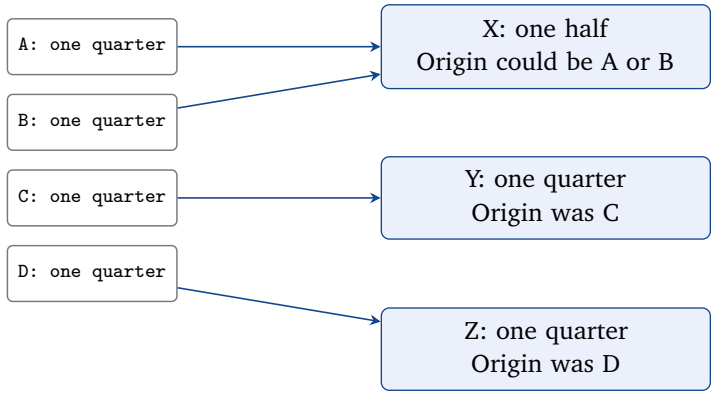
theory to thermodynamics, who said Shannon told it to him in 1961, and who printed it, with Edward McIrvine, in *Scientific American* in 1971. Then in 1982 an interviewer, Robert Price, asked Shannon about it, and Shannon said he didn't think von Neumann had told him any such thing. So it's a legend with one witness, and the person the witness quoted doesn't remember saying it. What doesn't hang on the story: the two formulas really do have the same shape, and it still takes Landauer's premise, a named assumption, to turn a count of bits into heat.

Now the exact version, with the notation built first. A *spread* (probabilists say distribution) is a way of giving each state a chance $p(x) \geq 0$, with the chances adding to 1. Its Shannon entropy, in bits, is

$$H(p) \;=\; -\sum_{x:\,p(x)>0} p(x)\,\log_2 p(x),$$

where the big sigma means "add up, over every state x with a chance above zero," and each term is a state's chance times the bits it would take to single that state out. That comes to $\log_2 n$ when n states share the chance evenly and less when the chance is lopsided. Pushing a spread through a move i gives the spread of where you land: $q(y) = \sum_{x:\,\mathrm{step}(x,i)=y} p(x)$. The bits the move *removes* are $H(p) - H(q)$.

Try four equally likely cards, A, B, C and D. Send A and B to a box marked X, C to Y and D to Z. Beforehand each card has chance $1/4$, so the four terms in the sum are each $1/2$: two bits. Afterward the box chances are $1/2, 1/4, 1/4$, and the three terms are each $1/2$: one and a half bits. Half a bit was lost on average. You can see where: half the time the box is X and leaves one binary question unanswered: A or B? The other half of the time Y or Z tells you the starting card exactly. An actual draw never loses half a card; the half belongs to the average uncertainty over repeated draws.



Half the time one binary question remains unanswered; half the time the origin is known. The average loss is half a bit.

Start with the half of the claim that removes no bits, Bennett's half. A move that forgets nothing sends each state somewhere different, so the chances are carried

along to new places, one for one, and nothing piles up and nothing spreads out. The uncertainty is the same number before and after.

Theorem 3.11 (A move that forgets nothing removes no bits)**.** Let the states be finite, listed once each, and let f send states to states with $f(a) = f(b)$ only when $a = b$. Then for every assignment of chances p, the pushed spread q has $H(q) = H(p)$. □

Proof. An injective map on a finite list of distinct states is a reshuffle of that list: every state is hit exactly once. So $q(f(x)) = p(x)$ for every x, and the sum defining $H(q)$ is the sum defining $H(p)$ with its terms in a different order. ■

That's why a price on removed bits can leave every reversible move costing nothing, which is the shape the toll needs: the ledger's cost lands only where something was forgotten.

Theorem 3.12 (Entropy after a permanent flip)**.** On a finite machine with a permanent reading, let m be the number of states that read yes, and let a move i turn $k \geq 1$ different no-states yes. For any spread p whose chance lies on those $m + k$ states, the spread after the move has $H(q) \leq \log_2 m$. For the even spread on the $m + k$ states, the move removes at least $\log_2 \frac{m+k}{m}$ bits and at least $\frac{2k}{m+k}$ bits; the first is reached when m divides k, the second when $k \leq m$. □

Proof. All the starting chance lands among the m yes-states: the old ones stay yes and the newcomers turn yes. There must be at least one landing state, so $m \geq 1$. The first question is how much uncertainty those m places can hold.

Here is the small fact about logarithms that does the work. For $y > 0$, $\ln y \leq y - 1$. Picture the gap $y - 1 - \ln y$. Its slope is $1 - 1/y$: negative below 1, zero at 1, positive above 1. The gap falls to zero and then rises, so it never goes below zero.

Call the positive landing probabilities $q_1, \ldots, q_r$. There are at most m of them and they add to one. Compare each with the even share $1/m$. Apply the logarithm fact to the ratio $1/(mq_j)$ and multiply by the chance q_j:

$$q_j \ln \frac{1}{mq_j} \leq \frac{1}{m} - q_j.$$

Now add over the occupied places. On the right the even shares total $r/m \leq 1$, while the actual shares total exactly 1. The sum is therefore at most zero. On the left, splitting the logarithm gives $-\ln m - \sum_j q_j \ln q_j$. Move $\ln m$ across and divide

by $\ln 2$ to count in bits. This says precisely $H(q) \le \log_2 m$: no arrangement of the chance among those places holds more uncertainty than the even arrangement.

Before the move, the $m+k$ starting states were equally likely. Their uncertainty was $\log_2(m+k)$. Subtract the greatest possible uncertainty left at the end: the loss is at least $\log_2((m+k)/m)$. If k is a multiple of m, keep each old yes-state where it is and send an equal number of newcomers to each. The landing chances are then even too, so this bound is reached.

For the other bound, count one landing pile at a time. A pile of s equally likely starts receives chance $s/(m+k)$. Once you know that pile was reached, there are still s equally likely origins to distinguish: $\log_2 s$ bits have disappeared there. Its share of the total loss is therefore $s \log_2 s/(m+k)$.

A pile of one contributes zero. A pile of two contributes $2/(m+k)$. For every larger whole number s, $s \log_2 s \ge 2(s-1)$. To check that inequality, subtract $2(s-1)$: the difference is zero at $s=2$, and its slope for $s \ge 2$ is $\log_2 s + 1/\ln 2 - 2 > 0$. It grows from there. Thus each pile costs at least $2/(m+k)$ for every origin after its first. There are $m+k$ origins and at most m piles, leaving at least k such extra origins. The loss is at least $2k/(m+k)$. When $k \le m$, send each newcomer to a different old yes-state and leave the old states fixed. All piles then have size one or two, exactly the sizes where the inequality is an equality. That construction reaches the second bound. ■

Premise 3.5 (Landauer's principle, as a premise)**.** For the chosen physical memory and its input distribution, assume that a step lowering its logical-state entropy by ΔH bits transfers mean heat $Q \ge k_B T \ln 2 \cdot \Delta H$ to a bath at temperature T, where k_B is Boltzmann's constant. Here Q is an average over that preparation, not a bound on each individual trial. Applying this premise requires a physical model in which the logical entropy loss has this thermodynamic meaning; a bare state-transition table does not supply one. In particular the memory is symmetric, its logical states alike in their own entropy, as Landauer assumed; for a memory whose states differ, the floor changes (Maroney 2009), and erasure work in an asymmetric double well has been measured below $k_B T \ln 2$ (Gavrilov and Bechhoefer 2016).

□ **Corollary 3.13** (Heat floor of a permanent flip)**.** Under Premise 3.5, with $k_B T \ge 0$, the flip of Theorem 3.12 from the even spread gives off at least $k_B T \ln \frac{m+k}{m}$, and at least $\frac{2k}{m+k}\, k_B T \ln 2$; with $k_B T > 0$ that is strictly positive.

Proof. Theorem 3.12 gives an entropy loss of at least $\log_2((m+k)/m)$ bits, and of at least $2k/(m+k)$ bits. Multiply by the nonnegative factor $k_B T \ln 2$ and use Premise 3.5. The identity $\ln 2 \log_2 x = \ln x$ gives the displayed floor. Here $m > 0$

and $k > 0$, so $(m+k)/m > 1$ and its logarithm is positive; if $k_B T > 0$, the heat floor is strictly positive too. ■

How big is that? Take the case where the squeeze is two, one state flipping in beside one that was already yes, and the floor is $k_B T \ln 2$. At room temperature, about 300 kelvin, that's roughly three zeptojoules: three thousandths of a billionth of a billionth of a joule. And that's the *biggest* floor a one-state flip gets from this corollary. With one newcomer the floor shrinks as more states already read yes, because the squeeze gets thinner: with a thousand of them it's $2 \ln 2 / 1001$ of $k_B T$, about a seven-hundred-and-twentieth, some five hundred times less than the case of two. So, for an even spread over the states in play, one unit of cost is not one halving. It's one halving exactly when the squeeze is two; for any thinner squeeze and an even spread, the physics asks for less than a halving, and the toll still charges a whole unit, because the cost comes in whole numbers. The entropy road to the toll, a little further down, doesn't need that rounding: entropy pricing asks about every spread, and the worst one takes a full bit. Read in the worst case over what the machine might be holding, the way the halving price reads a move, it's at least a full bit every time. A permanent flip merges two states (Theorem 3.1); put half the chance on each, and the move removes a full bit, however many states already read yes. So the gap between the unit of cost and the bits is a fact about the even spread, and in the worst case the flip removes at least the full bit the unit pays for. Whatever the size, it's heat only if you grant the premise; nothing on this page measured anything. The bead experiments above probe this energy scale. Ordinary CMOS switching is a different comparison: the implementation figures cited next are much larger than this floor. One 2023 tally of the numbers puts switching a single CMOS gate at a hundred attojoules at the low end, over thirty thousand times this. The point of the number isn't that a permanent flip is expensive but that, on the premise, it isn't zero.

A related logarithm appears in a different physical question. Yûto Murashita, Ken Funo and Masahito Ueda (2014) study processes whose reverse histories can include ones with no forward counterpart. They call this *absolute irreversibility*. If the reverse histories with no forward partner carry some share of the probability, their bound on mean entropy production picks up the logarithm of one over what's left. In their free-expansion example, gas that starts confined to a tenth of a box gives the logarithm of ten.

Make the confined fraction the old yes-states' share of the pile, the yes-states over the yes-states plus the newcomers, and out comes the same logarithm as the squeeze bound. That numerical match does not identify two physical processes. The arrows specify neither a thermodynamic reverse experiment nor its path probabilities, so they don't determine that share of reverse histories. The state count alone says noth-

ing about absolute irreversibility, its absence, or a heat equality. Gonzalo Manzano, Gülce Kardeş, Édgar Roldán and David Wolpert (2024) study such trajectory-level questions for computations with one-way transitions and random stopping times. Their physical premises are additional to the counting done here.

The even spread is the easy case, and it's fair to worry that the heat depends on picking it. It doesn't. Any spread that gives a real chance to every yes-state and to the state that flips will do.

□ **Corollary 3.14** (Positive heat for any full spread)**.** On a finite machine with a permanent reading, let move i turn a no-state s yes. Take any spread p that gives positive chance to s and to every yes-state. Under Premise 3.5 with $k_B T > 0$, the heat the move gives off is strictly positive.

Proof. After the move, s and the yes-states all land among the yes-states, and there is one more of them than there are yes-states, so two distinct states with positive chance land on the same state. That merge removes a positive number of bits (Theorem 3.18 below), and a positive temperature times a positive number of bits is positive heat. ■

There's a third road to the toll here too, and it's the same shape as the merge road, with the bits removed, in place of whole units, as the price of forgetting.

Definition 3.6 (Entropy-priced)**.** On a finite machine, a cost is *entropy-priced* when, for every move i and every spread p, the cost of i is at least the bits the move removes: $\text{cost}(i) \geq H(p) - H(q)$, where q is p pushed through i.

□ **Theorem 3.15** (The toll from entropy pricing)**.** On a finite machine with a permanent reading and an entropy-priced cost, every move that turns the reading from no to yes costs at least one. So the machine with that cost is a Thiele machine (Definition 2.2), and every run from a no-state to a yes-state costs at least one.

Proof. Let move i turn s yes. By Theorem 3.1 it lands two different states on one. Put half the chance on each: the move removes one bit, so the cost of i is at least one. The run-level floor is the universal floor (Observation 2.2) applied to that system. ■

There are three roads to the toll, with two different strengths. Entropy pricing and the halving price are equivalent: the most bits a deterministic move can remove is the base-two logarithm of its largest pile of starts sharing a destination. Merge

pricing asks only for a cost of at least one whenever any pair collides. It is the two-state restriction of the stronger conditions.

Try eight equally likely cards all sent to one box. The loss is three bits. A charge of one meets merge pricing but fails entropy pricing and the halving price, which require at least three. All three conditions give the permanent-record toll; only the stronger two give the full squeeze bound.

Theorem 3.16 (Entropy pricing, halving, and pairwise merge pricing)**.** Take a finite machine and a move i. (1) If no state has more than $K \geq 1$ states landing on it under i, every spread loses at most $\log_2 K$ bits through i; and if some state has exactly K states landing on it, the even spread on those K loses exactly $\log_2 K$ bits. (2) A cost is entropy-priced (Definition 3.6) exactly when it satisfies the halving price (Premise 3.4). (3) A cost satisfies merge pricing (Premise 3.2) exactly when it satisfies the halving price for every list of at most two different states, and exactly when it satisfies entropy pricing for every spread whose chance sits on at most two states. The first of those two equivalences uses no finiteness. □

Proof. (1) Sort the possible starts into piles according to where the move sends them. Suppose the landing place is y and its total chance is $q(y) > 0$. Knowing that you landed there leaves only the starts in that pile in question. Their chances, now measured within the pile, are $p(x)/q(y)$. They add to one, and there are at most K of them. The logarithm argument in the preceding entropy bound says their uncertainty is at most $\log_2 K$ bits.

Average that remaining question over the landing places, giving each its chance $q(y)$. This is exactly the information the move discarded. Indeed, a start x contributes

$$p(x) \log_2 \frac{q(y)}{p(x)}, \qquad y = \mathrm{step}(x, i),$$

and summing these terms gives $H(p) - H(q)$ by grouping the output terms by their landing places. An average of numbers no greater than $\log_2 K$ cannot exceed $\log_2 K$. For equality, put equal chance on every member of one pile of size K. There are $\log_2 K$ bits at the start and none at the single landing place.

(2) An entropy price must cover that even preparation on every pile. A pile of size n therefore requires cost $c \geq \log_2 n$, or equivalently $n \leq 2^c$. That is the halving rule: no landing place accepts more than 2^c starts. Conversely, if the halving rule holds, part (1), with $K = 2^c$, says no preparation loses more than c bits. The same price covers them all.

(3) Now allow at most two starts. If they land separately, the move loses nothing. If they land together, the halving rule reads $2 \leq 2^c$, which asks for at least one

whole unit. This is exactly merge pricing. Nothing in that two-start argument needs the rest of the machine to be finite.

The chance calculation makes the same demand. Two possible starts hold at most one bit, so a charge of one covers every spread over a merging pair. Equal chances on that pair lose exactly one bit and force that charge. A single possible start, or two that stay separate, lose zero. So restricting entropy pricing to at most two possible starts gives merge pricing too. ■

So merge pricing is weaker than full entropy pricing, though both force the one-unit toll under the permanence and finiteness premises. The prices are rules on moves. Reading one as physics takes Landauer's premise for the actual spread and a stated exchange rate from bits or units of cost to heat.

Now for the case where the floor is zero. Take the coin under a cup: toss a fair coin under it without looking at how it lands, and let the rule be to set it heads up. You might have had tails; now you have heads; one bit of what you didn't know is gone, and that's the bit the principle charges for. Now lift the cup, see heads, put the cup back, and apply the rule again. Nothing you didn't know changed, and no bit was lost. And the principle, applied to a state you already knew, demands nothing: the premise holds exactly when the heat is zero or more, which is to say it sets no positive floor. Setting a coin known to be tails to heads may have a finite-time cost under the chosen thermal dynamics; setting one you know is heads changes nothing. What the premise gives up either way is a floor that survives going slowly. Whether the hand that sets the coin keeps one is a question about the hand, and I come to it below. If you lift the cup after an uncertain toss, you have also made a record correlated with the coin. Resetting that record has to be analysed with those correlations and with the controls available. If instead the coin was supplied heads with certainty, there was no uncertain toss to record. These are different preparations. Knowing the result does not by itself locate a heat payment in the observer.

The entropy-loss bound depends on the preparation and on what the move does to it. Here "known" means a preparation concentrated on one logical state, or a conditional ensemble whose side information is explicitly included in the account. Merely telling a spectator which way the coin landed does not change an unchanged device's heat. This is the generalized form of Landauer's principle, worked out by Owen Maroney in 2009, in a paper he wrote at the Perimeter Institute in Canada (he has been a philosopher of physics at Oxford since 2011), and by Sagawa in 2014, and Landauer himself had the seed of it in 1961: for a row of bits all already ONE, he wrote, there's no need to carry out any operation, so no entropy changes and no heat is given off, and resetting a row all ZERO to ONE involves no entropy change either, because neither row carries any information.

Theorem 3.17 (A known state gets no heat floor from the premise)**.** On a finite machine, if the spread puts all its chance on one state s, then any move removes zero bits, and Premise 3.5 for that move holds exactly when $Q \geq 0$. □

Proof. A spread on one state has entropy 0, and so does its image, which is again on one state. So $\Delta H = 0$ and the premise reads $Q \geq 0$. ■

Be careful with that one, because it's easy to overread. "The premise sets no heat floor for a known state" does not mean a stamping machine run on a known-blank sheet dissipates nothing. A model of the stamping apparatus may impose a finite-time cost, but the logical change alone does not establish it. The restricted thermal results are proved here for Bennett's walker (Theorem 1.9), found for a dragged bead by Schmiedl and Seifert and for the specified classical jump-process models by Shiraishi, Funo and Saito, and how much depends on the device's physical implementation, its control protocol and how fast it goes. What the theorem says is only that the *premise* sets no floor there. The floor comes from what you didn't know. That's a sentence about the premise's floor, and the device is a separate matter. And notice that the toll still charges the known-state stamp one unit, because the toll is a rule about the *move*, and the move doesn't know what the observer knows. That's the worst-case gap from the third way out showing up again, and it's the price of having a rule a machine can follow without asking who's watching.

The physics of all this has a standard review, the physicists Juan Parrondo, Jordan Horowitz and Takahiro Sagawa's "Thermodynamics of information" (2015), which sets Landauer's bound, its generalized form and the costs of measurement and feedback side by side. And the gap between a device and the spread it runs on has a physicist's number on it. Artemy Kolchinsky and David Wolpert (2017) showed that a fixed process built to dissipate as little as possible for one spread over its states pays extra when it runs on another, and the extra is how much closer the process brings the two spreads, measured in relative entropy; the extra was later named mismatch cost. That's the worst-case gap again, priced for an actual spread, where the toll prices the move.

Fix a thermal protocol with the entropy-balance identity just described, using the same controls and environment for each input. A floor then comes from comparing its inputs. Take K inputs the device sends to the same output, and spread your chances evenly over them. The identity of Section 3 that ties Ladyman's reading to Sagawa's says the average, over the K, of the entropy the device produces on each one known equals the entropy it produces on the even spread, plus $k_B \ln K$. The second law makes the entropy produced on the spread zero or more. So the average is at least $k_B \ln K$, and the worst of the K is at least the average: on some

one of them the device produces at least $k_B \ln K$ of entropy, and, when dissipated work is temperature times entropy production, at least $k_B T \ln K$ of dissipated work. Slowing this same protocol does not remove that worst-input bound. Kolchinsky and Wolpert's formula says, under their conditions, which input pays: the one the device's own best spread gave the least chance. A device can make one input cheap only by making another dear. Choosing different controls for different known inputs changes the protocol being compared. The bound above concerns one fixed protocol across them. So the floor sits on the merging move, on its worst input, whatever anyone knows, and that's the shape of the toll. The bound on a device left to relax from a known start, in the next subsection, is one more floor of this kind.

There's a second piece of fine print on Theorem 3.17, and it's about who works the controls. The entropy-loss bound supplies no positive floor for a known input. Whether a device can approach that floor also depends on its controls: lower this barrier, tilt that well, raise the barrier again. Those operations have a timing and a physical implementation. The entropy calculation alone does not price either.

Paul Riechers, Alexander Boyd, Gregory Wimsatt and James Crutchfield (2020) analyse metastable memories under *time-symmetric* control: the prescribed control is the same played forwards or backwards, with the relevant time-reversal conventions. A symmetric clock pulse can meet that restriction; an arbitrary clocked pulse need not. For memory states unchanged by time reversal, their low-error result singles out the memory states that the desired logical map, applied twice, fails to bring back to where they started. They call those transitions *nonreciprocal*. With equal local-equilibrium free energies, the leading minimum-work term as the error allowance ε tends to zero is proportional to

$$k_B T \ln(1/\varepsilon)\ \Pr\big(C(C(m)) \neq m\big).$$

The probability is over the initial memory states. This is a low-error scaling with physical premises, not an exact finite-error heat charge on every execution.

A permanent flip from no to yes is nonreciprocal at that no-state: a second application cannot return it to no. A known input at that state can then face the extra work cost in their restricted control setting, even though its initial logical entropy is zero. Their class also includes a three-state cycle, which merges nothing. The distinction is about the permitted controls as well as the logical map. It neither changes Theorem 3.17's entropy calculation nor identifies the book's abstract merge price with a universal physical heat floor.

A deterministic run from a known start has a known state at every step, so the premise sets no floor under any of its steps; each that changes something can dissipate under the stated thermal-model assumptions, and a fixed device keeps the floor above on each step that merges. One more limit, about which states collide. The counting guarantees a collision among the already-yes states together with the

entering no-state. It does not guarantee that the entering state takes part in the collision. Take four states with only the first reading no, and a move that sends the first state to the second, the second to the third, and leaves the third and fourth where they are: the second and third both land on the third, and the first never shares with anyone. For an input distribution supported only on no-states, this counting argument alone forces no positive entropy loss. Whether other states in that support merge depends on the move. The operation-level ledger toll still follows under merge pricing.

What this section earns comes to this. A finite machine's permanent flag-raising instruction has two allowed starting states that land on one state. Prepare those two with equal chances: that instruction removes one bit, and under the adopted physical premise its mean heat is at least $k_B T \ln 2$. That is a worst-case preparation, not a positive charge on every execution. A known start gives no positive entropy-loss floor. Any finite-time dissipation depends on the device's dynamics, controls and speed; under the fixed-protocol entropy-balance assumptions above, merging K inputs forces at least $k_B \ln K$ of entropy production on one of them. The ledger charges the merging move whatever the preparation, because merge pricing says so. The counting proves a collision, the preparation determines the uncertainty removed, and the physical premises connect that removal to heat.

The general case is the one that shows you where the bits go. Split the bits removed state by state. Each state x you might be in carries its chance $p(x)$ along to where it lands, $\mathrm{step}(x,i)$, and finds whatever other chance landed there too. If nobody else landed there, its term is zero. If others did, its term is the difference between how crowded its landing spot is and how much of the crowd it brought.

Theorem 3.18 (The bits removed, one term per state)**.** On a finite machine, for any move i and any chances $p(x) \geq 0$, with q the pushed spread, □

$$H(p) - H(q) \;=\; \sum_{x:\, p(x)>0} p(x)\,\big(\log_2 q(\mathrm{step}(x,i)) - \log_2 p(x)\big).$$

Every term is zero or more, so a move never adds bits. If two different states $x \neq x'$, both with positive chance, land on the same state, the sum is strictly positive.

Proof. Group the sum for $H(q)$ by where things land: the chance $q(y)$ at a landing spot is the sum of the $p(x)$ that land there, so $H(q) = -\sum_{x:\, p(x)>0} p(x) \log_2 q(\mathrm{step}(x,i))$. Subtract from $H(p)$ term by term. Each term is non-negative because the landing spot holds at least what x brought, $q(\mathrm{step}(x,i)) \geq p(x)$, and the logarithm keeps order. If x and x' share a landing spot, that spot holds at least $p(x) + p(x') > p(x)$, so the term for x is strictly positive. ■

3.8 What you're entitled to, and what you're not

Let me total it: what have you earned the right to conclude, and where is there still a way out?

You've earned this first, and it needs no premise but one next state for every state: on any machine run by one rule, finite or endless, a run that raises a reading and keeps it up either merges two of its own states or never repeats one, and on any machine at all a single move that raises a kept reading without merging anything has to send the kept states one-to-one into a part of themselves with a gap left over, which takes endlessly many of them. Either way it is a merge, or an endless supply of states never used before. You've earned this: on any machine with finitely many states, a move that raises a flag either forgets something or takes a flag down somewhere else. That is pure counting, with no physics and no premise beyond finiteness, and the half of it where nothing comes down is the pigeonhole on four states. You've earned this: on a finite machine, if a flag, once up, stays up, then raising it has a collision somewhere. If it raises several no-states at once, the yes-states together with those newcomers land on at least that many fewer spots than there are of them. This does not say that each newcomer shares its destination; this one is counting again. And you've earned this: if forgetting is charged for, the toll follows, and if forgetting is charged per halving, the size of the toll follows too; each of those derivations hands back a system with the five parts and the toll, so Section 2's floor applies to it as it stands. This is the rule from Section 2, now derived, with every premise named, for any finite machine with a flag that stays up. In that section I only declared it.

Now for what you haven't earned, which I list so you don't assume it. You haven't earned that forgetting costs anything in the world; that's Landauer, it's measured for beads and argued over for the general case, and in this book it's an assumption with a name. And even granting it, Landauer's entropy-loss bound demands no positive heat from a run whose state is known (the lifted cup of the coin example), so what the physics forces is a floor on a device that has to work whatever it's holding. An individual run can fall below an ensemble average bound. Knowing a state does not by itself prove that another person or memory paid the difference. You haven't earned that any machine you'd actually want to build is of this kind, either, and there are two easy ways to miss. A machine that keeps its running total inside its own state, as a number with no ceiling, has infinitely many states, so the finite premise fails for it as a whole. And you can't patch that by giving the total a ceiling. A total that only climbs, held in finite memory, eventually either stops at its top, and then two different histories leave the same top value, which is a merge, or rolls back over to zero, and then it doesn't only climb. So any running total a finite machine keeps for itself has a horizon: it's honest up to its width and no further. And a

machine whose price list charges the step that raises the flag, but lets some other step merge two states at no cost, raises the flag honestly and still doesn't charge for forgetting in general, so the merge-pricing premise fails for it. Either one puts a machine outside these theorems as stated, though a finite piece of it can still sit inside. The example I build later (Section 15) misses in both ways. It keeps counters with no ceiling, so it's infinite outright, and some of its ordinary steps, counting a counter down among them, can land two different states on one at no charge on the ledger. So the counting, as stated, doesn't reach it: it forces the toll only on machines with finitely many yes-states that price every merge, and that example is neither. It doesn't need the counting, though: the moves that raise its flag merge states themselves, so the same cost lands on them directly (Theorem 3.19).

A finite piece can still sit inside, and the smallest one I could build where every premise can be seen to hold is this. None of them is left as a hypothesis. It has four slots times a flag, eight states, and three kinds of move: stamp (set the flag, go to the next slot, cost one), jump to a chosen slot (cost two), next slot (cost zero). It's finite and permanent; the next-slot move forgets nothing; the stamp and the jump merge; merges are priced, and priced per halving. So this section's two conclusions hold on it outright. The counting lemma underneath, that a duplicate-free list in which no landing spot is hit by more than a set number of members has at most that many times as many members as landing spots, is the plainer reading of the halving price above. Merge pricing alone would take any price of at least one for the jump. The jump's price of two is what the halving price asks, since the jump squeezes four states into one. One more fact about it: each move's price is exactly the number of halvings in its largest squeeze. A move that costs one has no spot hit by more than two states, a move that costs two has none hit by more than four, and some spot is hit by exactly that many (stamp squeezes at most two states into one, jump to a slot squeezes four, and next-slot squeezes nothing).

The example Part III builds runs this eight-state machine. Seen through a window that shows only a line number, read modulo four as the slot, and a flag, a short program of that machine, once a two-instruction setup has run, does what the eight-state machine does, one move at a time, and pays exactly what the eight-state machine charges. The example as a whole prices its record and not every merge: some of its zero-price steps merge states, and whatever those merges cost the hardware is off the ledger, so the finite piece is a piece. The toll on the whole example comes from the record's part of the state (Theorem 3.19).

Finiteness did one job in the counting. It forced the step that raises a permanent flag to merge two states. On the machine of Part III no count over the whole machine has to force that, because the merge is in plain view. Its record moves are the check, the commitment and the raised flag that the opening's lock puts in order, and it names them CHECK, COMMIT and CERTIFY: COMMIT names a checked claim in

a slot called the channel, and CERTIFY raises the flag. CERTIFY takes a state with the flag down and the same state with the flag up to one place, since a flag that's already up just stays up. Its COMMIT overwrites the channel, so two states that differ only in what the channel names land on one state. Part (1) of the theorem below shows both. So any price list that charges every merge of that machine's own step charges the flag-raising step, with no premise that the machine is finite. It isn't: its versions and its ledger grow without bound.

Finiteness is still there, in the smallest place it can be. Hold every part of the state but the flag fixed, and the flag has two readings, down and up. CERTIFY sends both to up, and it does the same to everything else from either, so the two states land together. That is the counting of Theorem 3.1 on a set of two. So the choice I made in the opening's list isn't the merge, and it isn't the shape of CERTIFY. It's that the step never reads the flag: the rest of the state moves by its own rule and the ledger adds the instruction's price, whichever way the flag reads, so no instruction keeps a copy of the old reading, and any that raises the flag lands its two readings together. Make CERTIFY trap instead when the flag is already up, and the merge moves rather than goes away: the state it traps lands with the state that was trapped already, since a trapped machine only pays. That's the halted state stepping to itself again, and it holds of every record move: any of the three that fails lands with the same state already trapped. The one way out of the flag's merge is to write the old reading into a part of the state that grows, a list of every reading so far, say, and that is a state never used before, the other column. A ledger that adds more for a re-raise than for a raise doesn't do it: the state with the flag up and the ledger a little lower lands where the raise lands, and the merge has only moved again.

That leaves the other miss, the unpriced merges. The example's price list, in Part III, puts its flat moves at zero, and a flat move merges (DEC, counting a counter down, lands two states on one), so full merge pricing and that price list can't both hold on that machine. They don't have to. Split a state into its base part, which is what a two-counter machine would show (the line number and the two counters), and its record part, which is everything else: the versions (each counter's count of how many times it has been written), the table of facts, the channel, the trap latch (which goes up when a record move fails, after which only the ledger moves), the ledger and the flag. Then price only the merges the record layer makes.

Premise 3.7 (Record-layer merge pricing)**.** A move that sends two different states with the same base part to one state costs at least one.

Part of where that line falls is forced. The line number has to be in the base part, or DEC, which can land two lines with the same counters on one, makes a record-layer merge at no cost; each counter has to be there, or its version, or DEC's

own merge gets charged; and the flag or the trap has to be outside it, or nothing charges CERTIFY. Where the versions, the table, the channel and the ledger sit moves nothing below.

The merges among base parts are the flat part's business, the ones any two-counter machine makes, and whatever heat they cost is the flat part's heat. The premise prices only merges between states with the same flat part. A step like DEC, which changes that part, can lose a version number at no cost, and the premise doesn't object. Merge pricing implies this premise, so it asks less, and on the record distinctions it is still an assumed operation-level price; identifying it with heat needs a physical model. And it is a price on this machine, not a source of the toll on any machine. A certifying move whose effect on the base read the flag, say one that moved a counter up when it raised the flag and left the counter alone when the flag was up already, would raise the flag and send no two states with the same base to one state, and the premise would have nothing to charge at the raise. It would still merge: the state with the flag down and the state with the flag up and that counter one higher land together, across two base parts, where the premise doesn't look. And on this machine its trap would still be charged when it fails. The small machine's record moves treat the base the same whichever way the flag reads, which is why the premise does its job there, and the theorem below is stated for that example and no other.

Theorem 3.19 (The toll on the example machine)**.** For the small machine of Section 15, with no finiteness premise: (1) CERTIFY merges, and so do COMMIT and CHECK of every claim, and each joins two different states with the same line and counters. (2) Every cost that satisfies merge pricing for that machine's step charges CERTIFY, every COMMIT and every CHECK at least one, and obeys the toll, so the machine with that cost is a Thiele machine. (3) No INC, DEC or HALT sends two different states with the same line and counters to one state. So the machine's own prices, base moves at zero and each record move at one, satisfy Premise 3.7, they're the least prices that do, and merge pricing fails for them. (4) Every cost that satisfies Premise 3.7 obeys the toll, so the machine with that cost is a Thiele machine. □

Proof. (1) For CERTIFY, prepare two states identical except for the flag. Give both a channel holding a commitment. One flag is down and one is up. CERTIFY raises the first and leaves the second up. Both advance one line and add the same charge; every other field agrees. They have become the same full state.

For COMMIT, put the requested fact, at the counter's current edition, in both tables. Leave one channel empty and put that very commitment in the other.

COMMIT writes the same commitment into both channels and treats every other field alike. Again the full successors agree.

For any CHECK, fill the fact table to its sixteen-entry limit. CHECK then fails, whatever claim it was asked to test. Compare that untrapped state with an otherwise identical state whose trap is already up. The first sets its trap; the second is already frozen. Neither changes the other core fields, and both pay the same charge. Their successors agree. In each construction the starting line and counters agree as well, so these are record-layer merges.

(2) A price on every merge charges each of those instructions at least one. Only CERTIFY can raise the certificate. Its merge therefore supplies the toll.

(3) Check the free instructions by asking whether two states with the same line and counters could become one. The table, channel, trap, ledger and certificate are left alone, so any difference in those fields survives. On a live state the shared line and counters choose the same instruction branch. A counter write adds one to its edition; different edition numbers stay different after the same addition. All unchanged editions stay different too. Hence no remaining field can hide a difference between those sources.

A trapped state stays exactly where it is under a free instruction. Two different trapped states therefore stay different. A live state remains live under these instructions, so it cannot meet a trapped one. That exhausts the cases. The free instructions make no record-layer merge, and all three kinds of record instruction do. Charging the former zero and the latter one satisfies the premise and is the smallest such price list. Full merge pricing still fails: the zero-price DEC pair in Theorem 15.22 starts with different counters and lands together.

(4) Any record-layer merge price charges the CERTIFY pair from (1). Since no other instruction raises the flag, this gives the toll for every raise. ∎

So this section reaches the example machine. On it the toll follows from record-layer merge pricing, and the example's own price list, flat moves at zero included, satisfies that premise. This is a result about its ledger, not a heat bound. There's a second way to settle the clash, which I note and didn't take. Morita's reversible two-counter machine runs every step backwards and is still universal: with its input encoded, it computes anything a Turing machine can. Put the record over a base like that, with the base's step taken as the program's own step from line to line and the halt left with no next step, and every base step is one-to-one, and full merge pricing permits those base steps to cost zero. That price list would put the flat moves at zero; the lower-bound premise alone does not require zero charges. It isn't the example's flat part, though. The example has a move for each two-counter instruction, acting from every live state, not a step for each line of one program, and DEC c j sends states on any two lines with the same counters, c above 0, to the

same window, so whether it merges turns on the rest of the state and not on the program being reversible. That base wouldn't be getting anything for nothing: what it doesn't forget it keeps in its counters, which is the column of states never used before, again. I haven't built that machine.

And you haven't earned anything about *what* the flag means. The pigeonhole on four states doesn't know what it's certifying. The cost is for raising a flag that stays up. Whether raising that flag ever amounts to standing behind a structural fact, the "vouching" I keep promising and keep not delivering, is a question for the certified example in Part III, and here it has moved exactly this far: on a finite machine that charges for every merge, if that flag stays up, it was paid for.

One last thing, because it should nag at you. The toll is forced by the premises, but the premises were chosen, and "permanent" in particular was chosen by me. Why should a flag stay up? Why not let it come down, pay nothing, and be honest about it? That's a fair question, and merge or revoke is the start of an answer (the three-card machine only showed that the converse of Corollary 3.9 fails): on a finite machine that charges for every merge, a flag that goes up at zero cost is a flag the same move takes back somewhere, and a record that can be silently taken back is the thing a hash chain and a write-once disk were built against. "Permanent" here is exactly that: the flag can't be lowered. The toy in Section 2 has it only because none of its three moves happens to lower the flag, and durability is the one job of a record it has; I chose it. Whether that was the right job to pick is something you can argue with me about, and I'd want you to.

And "permanent" is an idealization, which I should say before a physicist does. No physical record lasts forever. A bit held behind an energy barrier leaks over it at some rate, however slow, and a closed finite system comes back around, which is Zermelo's point from the counting (Theorem 1.6 is the finite form). The toll theorems in this section, the ones that start from a flag that stays up, need permanence on every state the machine can reach, including states a given run never visits, forever; on states nothing reaches it never comes into it, and merge or revoke does without it. Weaken that to "the flag stays up on the states that actually turn up, for as long as anyone cares," and merge-or-revoke tells you what goes wrong: the move could take the flag down somewhere the machine could go and never does, and then it isn't forced to merge at all. A version of the toll stated in chances is the noisy steps of Section 1: a step that squeezes nothing on average can't raise a reading that stays, and one that can strictly bunches the even spread. A version stated in time spans is a different theorem, and what I have of it is below.

One piece of it I can hand you, and the step law of Section 1 is its general form. Take one move and repeat it from one start, forever. On a finite machine the states that run passes through are finitely many, so it falls into a loop. If the move forgot nothing on those states, it would be a reshuffle of them, and a reshuffle brings every

one of them back around, the start included, where the flag was down. So if the flag goes up along that run and stays up from some point on, the move merges two states *the run itself visits*. The state they share is one the machine actually passes through. And it doesn't touch the known-state point above: a merge among states the run visits still receives no positive lower bound from the entropy-loss premise when its state is known. The argument holds for any map on finitely many states iterated from a start whose flag is down, with the flag up at every step from some point on; no permanence premise is used.

Runs that mix different moves split two ways, and the answer is short enough to give here. If the moves are picked from outside, in any order at all, the answer is no. Take two states, one reading no and one reading yes. One move swaps them; the other does nothing. Run the swap once and then the do-nothing move forever: the flag goes up at the first step and stays up for good, and neither move merges anything anywhere. The swap would take the flag down from the yes-state if it ever ran there; on this run it never does, and that's exactly the weakened permanence this paragraph is about. But if the machine is closed, picking its next move from its own state, the run is the orbit of one map, the step with whatever move the state picks, and the single-move argument applies to that map as it stands. Now say the moves come in a pattern that eventually repeats, one phrase of moves played over and over. Suppose the flag is down at some step after the repeating starts, and goes up and stays up after that. Count the repetitions from a step where the flag was down (start the phrase at that move if you have to) and run the same argument on the whole phrase treated as one move, at the states where each playing of it begins. It merges two of those states, both visited; follow the two through the phrase, and the first move where their paths meet merges two states the run visits at that move's place in the pattern. So the answer is no for moves picked from outside, and yes for a closed machine, or for a repeating program once the flag is down somewhere after the repeating starts.

Two chapters remain in this part. The next takes the record off its two places and lets it sit anywhere in an order, to see what the two places were doing. The one after is about a different way out entirely. Forgetting has had its chapter; the question after that is whether *finding out* costs, because the demon's whole trick was that it knew which way the molecule was going, and I owe you a straight answer on what knowing costs.

CHAPTER 4

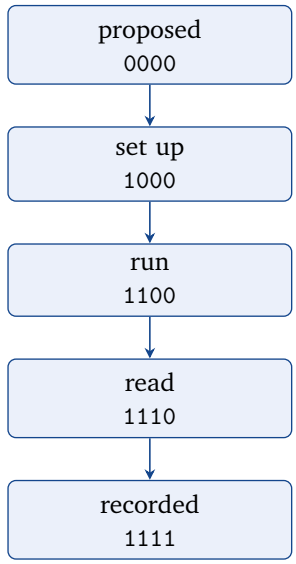
The whole direction

The record in Definition 2.1 is a place in any order, and the toll charges any move that climbs. Up to here the proofs were done on two places, blank and written, because that's the record you can point at. *Established* never felt like a switch to me, though. It feels like a direction with lines drawn across it, and the switch is just the first line. This section does the proofs on the whole direction, and finds out that the two-place case was a case: nothing the switch needed gets left behind, and the one thing a single switch can't do, hold three or more stages, falls out as a count.

A row of cells that only ever go from blank to written is a direction: you can tell how far along it a state is, and nothing walks it back. A single cell, a cup turned once, has two positions. The question for this section is blunt: with the record anywhere in an order, which of the results so far (the universal floor, the merges, the squeeze) come along as they stand, and which ones need the order to be more than a preorder?

4.1 A row of cups

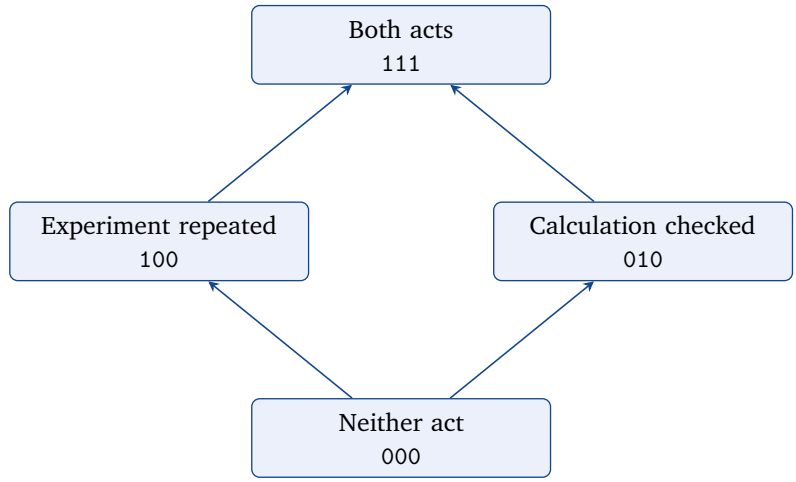
Get four cups and a pencil. Write five stages of an experiment on a strip of paper, left to right: *proposed*, *set up*, *run*, *read*, *recorded*. Agree to do them in that order. Put an upside-down cup between each pair of neighbours. The first means "the setup happened," the second "the run happened," the third "the result was read," the fourth "the result was recorded." As you complete a stage, turn over its cup. Cups only ever get turned right side up; none has a reset wire. These are records of completed acts. A fourth cup up does not make a false conclusion true, and a different experiment may need a different order.



Read down the drawing. Zero means cup down, one means cup up. Four cups give five stages along this run because each new stage turns up one unused cup.

Two things to notice with the cups in front of you. Each cup by itself is the two-place record: it reads yes or no, and once yes it stays yes, which Part II will call a latch (Section 11). And four cups were enough for five stages. Try it with three, and you'll find you can't: three cups that only ever turn up, in order, give you at most four different arrangements along the way. I'll prove both of those in a minute, and the proof is about as short as the experiment.

Now a harder strip. Some checks don't line up. One person repeats the experiment; another independently works through the calculation. Neither act includes the other. Draw a diamond: *neither* at the bottom, *experiment repeated* on the left, *calculation checked* on the right, *both* at the top. There are two ways up, and the middle points don't compare, which is what a *partial* order allows. Put a cup on each point except the bottom, one for the repeat, one for the calculation, one for both. Each cup is still a latch. The three cups tell you which acts have happened. They do not decide whether the experimental model or the calculation's premises were appropriate.



Read the cups as repeat, calculation, both. Neither middle point includes the other. The third cup repeats information already supplied by the first two; it is useful for showing thresholds, not needed for storage.

Whatever the shape of the strip, a straight line of stages or a diamond or something with a hundred points, I'll call the set of places the record can be the *direction*, and each question "has the record reached at least this place?" a *line* across it. One line is one cup.

4.2 The order, said exactly

The strip and the diamond each come with an order you can see by looking: whether one place is at or below another. Said exactly, the order is a test. For any two places x and y it answers "is x at or below y?" with yes or no, and I write a yes as $x \leq y$. The test has to obey two rules: every place is at or below itself, and if $x \leq y$ and $y \leq z$ then $x \leq z$. An order that obeys just those two is called a *preorder*. If it also obeys a third rule, that $x \leq y$ and $y \leq x$ together force $x = y$, it's a *partial order*, and that third rule is called antisymmetry. The two-point order, no below yes, is the cup. The whole numbers in their usual order are a straight strip that never ends.

The cups say the same thing in their own terms. For a place a, the cup for a asks "has the record reached at least a?", so with the record at x it's up exactly when $a \leq x$, and I write its answer as $[a \leq x]$. In the row of cups I said the cups together tell you which place the record is at. On the strip and the diamond you can check that with your eyes, and Part II proves it again from the latch side whenever the order is partial (Theorem 11.10), though the first time through I hadn't asked what in that proof was doing the work. My first guess was that it held for every order.

Why bother with the bare preorder at all, when every strip on the table is a partial order? I bothered because I wanted to see what the third rule was doing,

and the way to see what a rule does is to take it away. Take a preorder with two places, call them *left* and *right*, where each is at or below the other. There are two cups, one asking "at least left?" and one asking "at least right?", and both are up at left and both are up at right. Look at the table and you can't say where the record is. Now build a record that swaps between left and right on every single step and charges nothing. Every rule I'm about to state is satisfied. The record never drops below anywhere it was, because each place is at or below the other. No cup ever moves, so the toll in the next subsection has nothing to charge. The record moved on every step and nothing on the table noticed.

□ **Theorem 4.1** (The cups fix the place, and only with antisymmetry)**.** (1) In a partial order, if two places x and y leave the same cups up, $[a \leq x] = [a \leq y]$ for every a, then $x = y$. (2) In a preorder that isn't a partial order this can fail: there's a preorder with two different places that leave the same cups up, and a record over it, driven by the computation and growing in that preorder, that changes on every step while no cup moves.

Proof. (1) Look at the cup for x. It's up when the record is at x, since x is at or below itself, so it's up at y too, and that says $x \leq y$. The cup for y gives $y \leq x$ the same way. Antisymmetry gives $x = y$. (2) Left and right above, with the record swapping every step. ■

So in a partial order the cups are the place, and in the preorder I built they aren't. That second half holds in every preorder that isn't partial, by one line: if $x \leq y$ and $y \leq x$ with $x \neq y$, then a cup that's up at either one is up at the other, by transitivity. So the cups fix the place if and only if the order is partial. My first guess was right on every strip I could draw, and it was right because of a rule I hadn't noticed I was leaning on. On a bare preorder, putting two places on the same rung tells the toll not to care about moves between them, and it doesn't. I don't think that's a defect; it's the order telling you what you agreed to count.

4.3 The toll, over the whole direction

One cup had one rule: the step that turns it right side up pays. Now there's a row of cups, or a diamond of them, and a single step can turn over several at once. The rule I'd guess first turns out to be the right one: *a step that turns over any cup pays at least one.* A step that turns over three cups at once still only has to pay one. The rule asks whether a step turned something over. It doesn't ask how many.

That rule buys the same thing one cup bought. Take any run, and suppose some cup is up at the end that was down at the start. Somewhere along the run a step

turned that cup over, and that step paid. So the run paid at least one. Count the same way and you get more: the run paid at least once for every step that turned anything over.

Now look at what that argument didn't use. It didn't care whether the cups sat in a row or a diamond or something with a hundred points. It didn't care how many there were. It didn't even use the fact that cups never get turned back down. If something could knock a cup back over, the argument still works, because all it needs is the one step where that cup went up.

To say it exactly I want to translate it into places. The cup at a place a is up exactly when the record is at or above a. Say a step takes the record from x to y. The cup at y is up afterwards, since y is at or above itself, and it was down before exactly when y isn't at or below x. And if y is at or below x, any cup that's up at y was already up at x. So a step turns over some cup exactly when it lands somewhere that isn't at or below where it started. The same reasoning says a run ends with some cup up that was down at the start exactly when it ends somewhere that isn't at or below where it started.

Theorem 4.2 (The toll over any direction)**.** Take a machine whose record is a place in any preorder, and say a step *turns a cup over* when it takes the record from x to a place that isn't at or below x. (1) Every step that turns a cup over costs at least one exactly when every run, from whatever state it starts, that ends somewhere not at or below its start costs at least one. (2) When that holds, a run costs at least the number of its steps that turn a cup over. (3) It holds exactly when every cup, on its own, obeys the two-place toll: for every place a, a step that turns the cup at a from down to up costs at least one. (1) is the floor over any order (Observation 2.1) said in cups, and the universal floor (Observation 2.2) is (1) with one cup. □

Proof. (1) and (2) are the walk above. For the other half of (1), a run of one step that turns a cup over ends somewhere not at or below its start. (3) is the translation above. ■

When the cups only ever go up, as on the strip and the diamond, a step that turns a cup over is just a step up.

That surprised me a little, because I'd filed the floor away as a fact about one bit, and nothing in it was about one bit. It was about one cup going up, and anything you build out of cups has that.

4.4 One line isn't enough

Now comes the payoff I promised at the start of this section. Could a cleverer single cup do the job of all four, with one latch and some decoding that reads the record's place off the latch and whatever the machine shows? Section 11 rules that out in the smallest case: over a machine with one visible state, a record that climbs through three places can't be produced by one latch and a decoding, because one latch and one visible state make two outcomes and the record has three.

So try a second guess, closer to the cup's own rule. The cup's rule was "new value is old value *or* the event." The obvious rule for a whole row is "new place is old place *joined with* the event's place," where joining two places means going to the lowest place at or above both. On the row, the event names a place, and you turn up every cup as far as that place. To see it break, take the strip $0 < 1 < 2$, a cup at 1 and a cup at 2, over a machine with one visible state, so the event has to be one fixed place. Let the record jump from 0 straight to 2, turning up both cups in one step, and stay put at 1 and at 2. From 0 the event has to turn up both cups, so the fixed place is 2. From 1 the same event would turn up the second cup as well, and the record stays at 1.

What works is giving every cup its own event, and letting the event for a cup look at where the record already is. Theorem 11.10, in Part II, says exactly that, and this is why it has to. That leaves the question of how many cups. In the row of cups I asked you to try five stages with three and said you couldn't. Here is the count, with the matching example added, and with an arrangement of k cups read as a k-bit word, a one for every cup that's up.

□ **Theorem 4.3** (Lines a direction needs)**.** (1) A list of different arrangements of k cups, each keeping up every cup the one before had up (a chain of different k-bit words in which each word's set bits are among the next word's), has at most $k+1$ members, and for every k there's such a chain with exactly $k+1$. So a straight strip of $n+1$ stages needs exactly n cups. (2) Over a machine with one visible state, take the record on the strip $0 < 1 < 2$ that jumps from 0 to 2 and stays put at 1 and at 2. It grows and is driven by the computation, and no rule of the form "join the record with a fixed place" reproduces its steps.

Proof. (1) Going from one arrangement to the next, which is a different one, turns up at least one more cup, and there are k cups, so there are at most k steps and $k+1$ arrangements. For the matching chain, take no cups up, then the first, then the first two, and so on, packed to the left: that's the row of cups. (2) Joining 0 with a fixed place g gives g, and from 0 the step has to give 2, so g is 2. Joining 1 with 2 gives 2, and from 1 the step has to stay at 1. ■

So five stages need four cups, and neither one nor three will do it, as long as the cups only turn up. An ordinary binary counter holds a thousand stages in ten bits, because its bits go down as well as up, and the theorem says nothing against that. The switch isn't wrong: it's the first line, and a straight strip with more than two places needs more of them, exactly one fewer than it has stages. A direction that branches needs at least as many cups as the steps in its longest strict climb, and at least as many as it takes to tell its places apart, the rounded-up logarithm of how many there are. Put three places above a bottom, none of them above another: the longest climb is one step, and one cup can't tell four places apart, so it takes two, and two do it, though not two lines: one cup that goes up at the first and the third, and one that goes up at the second and the third. It can need more than both. Take two places each below two others: one step of climb, four places, and two cups still won't do, because both upper places would have to keep up every cup either lower one has up, and two cups allow at most one arrangement like that besides the lower places' own.

4.5 Why crossing a line has to cost

The last chapter showed that the cup's toll isn't an arbitrary rule. It's forced by three premises, and the argument that ties them together was the pigeonhole: on a machine with finitely many states, the move that raises a record that goes up and stays up has to land two possible states on one state (Theorem 3.1), and a step that lands two states on one is a step that forgets which one it came from. Does that survive the whole row of cups? Mostly, and the parts that don't survive are the interesting ones.

The counting needs a flag, because what it counts is the collection of states with that flag up, and now there are many cups, each one a yes-or-no reading that could serve as the flag, so the first job is to pick one. My first guess was the cup for the place where the record starts. That goes nowhere: the starting state already has that cup up, so it's already in the collection, and there's no state coming in from outside. The cup that works is the one the step turns over, the cup for the place where the record lands. It's up after the step, since a place is at or above itself, and it was down before, which is what turning a cup over means. So the collection is every state with that landing cup up, and the state the step starts from is the one coming in from outside. If the move keeps every state in the collection inside it and never lands two states on one, the states already inside use up every state in the collection between them, and there's none left for the newcomer. So one of two things happened. Either two states landed on one, or some state that was in the collection got moved out, which means its landing cup went back down. I'll call that second thing a *revocation*.

□ **Theorem 4.4** (Turning a cup over merges)**.** On a machine with a record in any preorder, where each place has finitely many states with its cup up: (1) if a move turns a cup over at some state and lands the record at a place a, then either the move merges two states, or there's a state with the cup at a up that the same move takes to a state with the cup at a down, which is a revocation; (2) if the record only grows, so that no cup ever goes back down, the move merges; (3) if the record only grows, the toll holds exactly when every move that turns a cup over at some state, and merges, costs at least one, so merge pricing gives the toll.

Proof. (1) Gather every state with the cup at a up. There are finitely many. If the move keeps each of them there and is injective, it maps that collection into itself without two landing on one, so it hits all of them, including the state it sends the starting state to; injectivity then says the starting state was in the collection already, and it wasn't, because its cup at a was down. So either some state in the collection has its cup at a knocked down, or the move isn't injective. (2) Growth rules out the knocking down. (3) If the toll holds, every move that turns a cup over somewhere pays, merging or not. The other way, a move that turns a cup over merges by (2), so the hypothesis charges it. ■

It's the old counting argument again, run once for every cup.

Each premise there can be knocked out, and I wanted to see each one go, because a premise you've never seen fail is one you don't understand yet.

Knock out finiteness. Take the whole numbers as the states, which gives a row of cups that never ends, and let the record be the state itself, one more on every step, at no charge. Every step turns up one more cup. The step never merges anything: different numbers go to different numbers. Merge pricing holds, because there are no merges to price, and the toll fails on every single step. The counting broke at the one place it needed the collection to be finite. Every state moved one along, and there was always another state to move to. That's Charles Bennett's 1973 construction in miniature: a machine that keeps its whole history never has to forget, so it never has to merge. It hasn't got out of paying. Every step lands it on a state it has never used, and that's the fresh-state column. The toll fails here because merge pricing only charges merges and this machine pays by taking a state it has never used, which that price list doesn't charge. Take the infinity away and the supply of fresh states goes with it.

Knock out growth. Take a two-state machine whose record flips back and forth between two places, so a cup goes up on one step and gets knocked back down on the next. The two states just swap with each other, so nothing merges, and the price list is honest about merges. The step that turns the cup up pays nothing. It takes the

other branch of (1) and revokes: the state that had the cup up gets moved to where it's down.

Knock out the price on merges. A two-state machine that stamps every state with yes turns the cup up wherever it was down, merges at the stamp, and charges nothing. Of course the toll fails; nobody asked the merge to pay.

Theorem 4.5 (Each premise does work)**.** (1) There's a machine with infinitely many states, a growing record and an honest merge price on which every step turns a cup over and the toll fails. (2) There's a two-state machine with an honest merge price whose record doesn't grow, which turns a cup over and fails the toll. (3) There's a two-state machine with a growing record whose merges aren't priced, which fails the toll. □

Proof. The three machines above, in order. ■

So the toll over a whole direction rests on exactly the three things the cup's toll rested on: finitely many states with each cup up, a record that doesn't come down, and merges that pay. Nothing extra got smuggled in when one cup became a row, and nothing the one cup needed got left behind.

I'd half expected the diamond to need something new, some extra rule for the two middle cups that don't compare. It didn't. Every cup gets its own collection of states, and the argument on each is the old one.

CHAPTER 5

Learning borrows, committing pays

I ended the chapter on the toll's reason owing you an answer about knowing. What keeping takes so far sits on forgetting: under merge pricing, a finite machine's permanent flag-raising instruction was paid for because it merges two states somewhere. But there's a thing that looks like the opposite of forgetting and feels like it ought to cost at least as much, probably more, and that is finding out. Every time a machine rules out a possibility, it knows more than it did. If forgetting a bit costs, does learning one cost too? I had a strong feeling about this, and you probably have it too, so I'll say it first: learning is the expensive part. Most physicists had that feeling as well, for a long time. It's why the demon took a hundred and fifteen years to exorcise, or didn't. "Exorcise" is the word the people who think it worked like to use. John Earman and John Norton, the doubters from the last chapter, called their two-part paper on it "Exorcist XIV: The Wrath of Maxwell's Demon," and I read that Roman numeral as a joke about how many times the demon has been declared gone. So hold the word loosely. This section is me finding out where the feeling goes wrong, and where it's right. I'll give the answer up front. Looking can have a physical cost; its value needs a model of the measurement. But it doesn't have to merge anything, so it owes no floor, and a ledger that charges merges can charge it nothing. What it does use up is a blank page to write on. Learning borrows that page. Wiping the page to use it again is a merge, and that's where the floor gets paid. Committing, making a record nobody can hand back, is paying for good. I'm going to tell you the version where the exorcism worked, Charles Bennett's, because it's the version that says where the heat goes, and the third way out in the last chapter, the one Landauer's premise is there to close, stays open behind it.

5.1 Bennett moves where the heat is paid

In 1982 Charles Bennett, who'd already shown in 1973 that computation can be done without forgetting if it keeps its whole history and pays by taking states it

has never used instead, took the demon apart one step at a time. The being looks at a molecule and writes down which side it’s on. Is writing that down a merge? Picture the being’s notepad, which starts blank. After looking, the notepad says “left” or “right.” There are two possible worlds before the look (molecule left, pad blank; molecule right, pad blank) and two possible worlds after it (left, “left”; right, “right”), so nothing merged: two went in, two came out, and you can read the before off the after. Copying what you see onto a blank page forgets nothing, and a step that forgets nothing owes no floor under Landauer’s premise. Any finite-time physical cost additionally depends on the chosen dynamics and protocol. But it did use something up: the blank page. Before the look, the pad was blank and ready; after it, the pad is full. Brillouin had priced one particular way of looking (bouncing light) and took it as the price of getting information at all. Bennett gave a way of looking that, on his account, doesn’t pay that price. It borrows a page instead.

So where is the heat floor? The being has one notepad and a lot of molecules. After the first molecule the page is full, and to do the next one it has to wipe the page. *Wiping* takes two possible worlds (pad says “left”; pad says “right”) to one (pad blank), and that’s the merge, the step with no way back. For a complete reusable engine under the stated thermal assumptions, the memory reset and the feedback process must be accounted for together. The cycle cannot turn information about the molecule into net work from a single bath while restoring every other resource for free. It’s the rule from his 2003 notes that I quoted in Section 2: erasing and merging carry the unavoidable charge, and copying onto a blank place to write doesn’t. The look borrows the page. Resetting an uncertain page without using its correlation with the molecule has the corresponding entropy-loss floor under the thermal assumptions. While the molecule is still available, the joint operation can instead undo the copy; the four-state example below does that. A blank supplied at the start is an initial condition. To charge its preparation, specify what uncertain state was reset and how.

Section 2 separated making a record from resetting it. Now you can see exactly which step pays, and why the other one was tempting, because looking *feels* like the active part, the thing the being does, while wiping feels like housekeeping. In that reset protocol, the housekeeping is where the entropy-loss floor applies. If there’s one sentence in this section I’d want you to carry out of it, it’s that one, and the idea in it is Bennett’s.

5.2 Two different things called “narrowing”

Here is what I had to get straight in my own head before any of this made sense, and I’ll give it to you as a picture because that’s how I eventually got it.

Draw the machine's possible states as a row of dots. Now draw a second row above it: the dots that *you*, standing outside, still think it might be in. Those are two different rows: the first is a fact about the machine, and the second is a fact about you, about what you've seen and what you've ruled out. Both rows can shrink, but they shrink for different reasons, and only one of them costs.

The machine's row shrinks when two of its states land on the same state, which is a merge, and a merge is what the price charges. Under the halving rule from Section 3, a run can't shrink the machine's own row by more halvings than it paid for. That's the theorem just below, and it's that section's halving price run forward along a whole program, where that section took it one step at a time. There is no new idea in it, only the same counting of states.

To state it exactly, write $\lceil \log_2 n \rceil$ for the index-bit count of n: the smallest whole number k with $2^k \geq n$, the fewest yes-or-no questions that can guarantee identification among n things when arbitrary binary splits are allowed. It is 0 at $n = 1$, and 0 at $n = 0$ too, since $2^0 \geq 0$; the definition covers empty lists, where the logarithm of zero isn't a number. Wherever I write an ordinary logarithm without the rounding, its argument is positive. For a run t and a list D of starting states, let $\mathrm{img}_t(D)$ be the set of states the run can end in, starting from somewhere in D, and write $|\mathrm{img}_t(D)|$ for how many there are (in Section 3 the symbol for one move already meant the count; here I need the set and its size both). Let $\mathrm{cost}(t)$ be the run's total cost.

□ **Theorem 5.1** (The machine's own narrowing is priced)**.** If the cost satisfies the halving price (Premise 3.4), then for every run t and every list D of starting states with no repeats,

$$|D| \;\leq\; 2^{\mathrm{cost}(t)} \cdot |\mathrm{img}_t(D)|, \qquad \text{so} \qquad \lceil \log_2 |D| \rceil \;\leq\; \mathrm{cost}(t) + \lceil \log_2 |\mathrm{img}_t(D)| \rceil.$$

Proof. After each move, gather the states still possible and count each different state once. If the first move costs c_1, the halving rule says the original count is at most 2^{c_1} times this new count. If the next move costs c_2, the new count is at most 2^{c_2} times the count after that. Put the inequalities together: the original count is at most $2^{c_1}2^{c_2}$ times the count after two moves.

Carry on to the end. The factors multiply, their exponents add, and the added prices are exactly $\mathrm{cost}(t)$. The last collection is $\mathrm{img}_t(D)$, giving the first inequality. An empty run contributes no factor and leaves the collection unchanged, so it fits too.

For the rounded count, let k bits suffice to number all the endpoints. There are then at most 2^k endpoints, and the first inequality gives at most $2^{\mathrm{cost}(t)+k}$ starts. That many bits therefore suffice for the starts. This also covers an empty starting

collection, whose stipulated index-bit count is zero, without taking a logarithm of zero.

Rounding throws some information away. Four starts and three endpoints both need two index bits, so the rounded comparison alone permits zero cost. The unrounded comparison still sees that four is more than three and forces a positive charge. ■

The run itself is the witness; nothing else is handed in. That will matter in Section 9, and I'll say why at the end of this section.

Your row shrinks when you see something that some of your candidates couldn't have shown you, and you cross those candidates off. Nothing happened to the machine. It didn't merge anything; it didn't know you were watching; it would have done the same thing with nobody there. The physicist Edwin Jaynes had a name for mixing these up, and I think it's one of the most useful names in all of science: the mind projection fallacy, treating what you know about a thing as if it were a property of the thing. Your list of candidates is in your head, and the machine's row is in the machine. The halving price is about the machine's row, and crossing a candidate off yours doesn't move it.

And here I have to be fair to Jaynes, because he'd argue with where I just took his phrase. In a 1965 paper on Gibbs's and Boltzmann's entropies he wrote that entropy is "an anthropomorphic concept" (a phrase he credited to the physicist Eugene Wigner), and by that he meant a property of the particular experiments you or I choose to do on a physical system. On his account the system alone doesn't carry it. So for Jaynes the second law is about what's known too, and the line I'd have liked to draw, that the second law has opinions about the machine and none about you, is exactly the line he'd refuse. Who's right is one of the oldest fights in statistical physics, Jaynes's reading against Boltzmann's, and I can't settle it and don't have to. The spreads in Section 3's entropy theorems are spreads over the states the device might be in when the step runs, and every theorem there holds whether you read those chances as ignorance or as physics; "the floor comes from what you didn't know" is the same sentence in either reading, with "you" being whoever built the spread. What I need from the fallacy is only this: the two rows are different rows, and only one of them is the machine's.

5.3 The demon on a sheet of paper

Build it, because the whole claim is four dots and two arrows and you should see it merge or not merge with your own eyes.

The machine has a hidden bit (think of it as the molecule's side) and a display (the notepad). It has four states, written as pairs, hidden bit then display: 00, 01, 10, 11. It has two moves.

MEASURE copies the hidden bit onto the display, by adding it in the way that a light switch adds: if the hidden bit is 1, flip the display; if it's 0, leave it. Draw the arrows: 00 goes to 00, 01 goes to 01, 10 goes to 11, and 11 goes to 10. Look at the arrowheads: every state has exactly one arrow landing on it, so nothing merged and MEASURE forgets nothing. Look at the two states where the display starts blank: 00 goes to 00 and 10 goes to 11, which is the copy onto a blank page, (hidden, blank) to (hidden, hidden), and it's the use that matters here. And the light switch isn't one way of copying among several. A MEASURE that merges nothing, copies onto a blank display and leaves the hidden bit where it was has nowhere else to send 01 and 11: each has one landing left on its own side.

WIPE sets the display to 0 whatever it held. 00 goes to 00; 01 goes to 00 as well. 10 goes to 10; 11 goes to 10 as well. Two arrows land on 00 and two on 10, which is a merge, twice over.

Hidden bit, display before	After MEASURE	After WIPE
00	00	00
01	01	00
10	11	10
11	10	10

Now follow only the prepared cases: 00, 10 become 00, 11 after the copy. WIPE sends those two to 00, 10, still two different full states. The hidden bit retains the distinction. The instruction price nevertheless charges WIPE because its rule merges other possible pairs, as the full column shows. That worst-case charge is not a one-bit entropy loss on this particular correlated preparation. Under any price that charges per halving, WIPE costs at least one, while MEASURE is allowed to cost zero. Zero on the price list isn't zero spent: a physical MEASURE can dissipate under its specified dynamics, and it uses up the blank display. What it doesn't do is merge, so it owes no floor.

Now be the observer. You can see the display and not the hidden bit. What you're shown, the display here, is what I'll call a *window*: the part of the state an outside observer gets to see. Section 8 is about windows in general; here the window is just the display. The display starts blank, so your list of candidates is {00, 10}: two possibilities, one bit you don't know. The machine runs MEASURE and you look at the display. If it shows 1, the only candidate that could have produced it is 10, and if it shows 0, only 00, so your list went from two to one and you learned the hidden bit. The machine's price for that move, on a list that charges merges, is zero, and what it spent was the blank display. Its own row of possible states went from {00, 10} to {00, 11}, still two; nothing it could have been got merged with anything else. You narrowed and it didn't. That's the measuring half of Bennett's argument on four dots, and WIPE's charge is the wiping half. So the demon just broke something: one bit gained, nothing charged. The thing it broke, written out so you can see exactly

where the crack runs, is this: for every list of starting candidates, every run, and every actual start in the list, the index bits the observer gains (log2 of the candidates before minus log2 of the candidates after, both rounded up) are at most what the run was charged. On four dots that sentence is false.

Definition 5.1 (What an observer knows)**.** Fix a window w from states to what can be seen. Given a list Ω of candidate starting states, a run t, and the actual start $s_0 \in \Omega$, the observer's *knowledge* $K(\Omega, t, s_0)$ is the list of candidates $s \in \Omega$ whose run shows the same thing through the window at every step as the actual run does.

If that definition looks familiar, it should. It's the standard way logicians of knowledge set it up: what you know is whatever holds of every case you can't tell apart from the actual one, given everything you've seen (Fagin, Halpern, Moses and Vardi's *Reasoning About Knowledge*, 1995, builds a whole subject on it, runs and observations included). And it's the logically omniscient kind, the kind where knowing some facts means knowing everything that follows from them, at no extra cost (Section 14 comes back to that flaw). I'm using it on purpose, flaw and all. This section is about learning, and learning in that sense is exactly the thing that doesn't have to merge. Settling, making a result stay so it can't be handed back, is the separate act with a price, and nothing in this definition settles anything.

Theorem 5.2 (An observer can learn with no merge)**.** There is a four-state machine (the demon), a window (the display), a cost satisfying the halving price, a run of cost 0, and candidates with $|\Omega| = 2$, for which $|K(\Omega, t, s_0)| = 1$. So the claim "for every Ω with no repeats, every t, and every s_0 in Ω: $\lceil \log_2 |\Omega| \rceil - \lceil \log_2 |K(\Omega, t, s_0)| \rceil \leq \text{cost}(t)$" is false for that machine and that cost. In the same machine, every cost satisfying the halving price charges WIPE at least one.

Proof. The machine and run are the four dots above: MEASURE is a bijection, so cost 0 satisfies the halving price; the observer's two candidates show different displays after MEASURE; WIPE sends two pairs of states to one each, so the halving price needs its cost ≥ 1. □

The demon is the picture, and the smallest case is smaller than it. The claim as written already fails on a machine with two states that the window tells apart and every move priced at zero, because the observer's very first look tells the two apart, and it never fails on a machine with one state. That's why the interesting version is learning *after* the first look, which is where the three states of Theorem 5.3 come in. And WIPE's price needs less than the halving rule: merge pricing alone charges it at least one.

There is one thing to be exact about. The halving rule is a floor: it says merges cost at least this much, and it doesn't say anything costs zero. So the theorem exhibits *one* price schedule that meets the halving rule and charges MEASURE nothing. A different schedule could meet the rule and still charge MEASURE; nothing in counting forbids overcharging. The claim is that learning is not *forced* to cost a unit. That's what "borrows" in the section's title means: the look takes a page and pays nothing, and the unit, like the floor, comes due at the wipe.

The physicists had this freedom on the books long before me. Paul Fahn (1996), doing the entropy accounting of Szilard's one-molecule engine step by step, argued for a symmetry between the measuring step and the erasing step: on his account, which of the two you charge is partly a choice of bookkeeping, and the total is what's fixed. Sagawa and Ueda (2009) derived a bound on that total, with the split depending on the memory. So, on their accounts, "a schedule could charge MEASURE instead" is a known freedom in how the books are kept, and I didn't find a loophole. The other reading is very much alive too. Ruth Kastner (2025) argues that Brillouin was essentially right after all: once the physics of a real measurement is put back in, it's measurement that pays and foils the demon, erasure drops out of the dissipation, and Bennett's floorless copy fails as physics. The theorem above is about price lists, so it stands either way. What hangs on her is whether a real demon can look without a floor, which is the same question the next paragraph's doubter presses on.

And the doubter from the last chapter cuts this side too, so I owe it to you, since I leaned on him there. In 2013 John Norton argued for a no-go result: at the scale of molecules, thermal jiggling keeps any process that's supposed to be thermodynamically reversible from ever actually finishing, and finishing it takes more entropy than Landauer's amount. If he's right, Bennett's measurement has a floor in practice after all, because it's one of those processes. The theorem above survives that, because it's about price lists: a schedule that meets the halving rule isn't forced to charge the copy, whatever a real device spends. What wouldn't survive is the physics I've been hanging on it, that a real demon's look owes no floor, only the dissipation of happening that going slowly shrinks. If Norton is right, the line between learning and committing keeps its counting and loses its physics. I can't referee that one either.

5.4 Who holds the pen changes who pays

Now comes the part that I think is the most beautiful fact in here, and you find it by playing with the four dots rather than by thinking about them. Run MEASURE *twice*. The first copies the hidden bit onto the display. The second adds the hidden bit in again, and adding a bit twice with a light-switch add gets you back where you

were: the display is blank again. Check the arrows: 10 to 11 to 10. The display went full and then blank, and nothing merged, and the price on the ledger was zero both times. So a display that was blank before the first MEASURE *can* be put back to blank with no merge and no floor, by a second MEASURE, though not by WIPE. This is the borrowed page being handed back: while the original is still there to check against, the copy can be un-copied. Each MEASURE still dissipates by happening, and going slowly shrinks it.

What's the difference? MEASURE *consults the hidden bit* when it clears the display. It knows what's written there, because it has access to the thing it was copied from, and so it can undo the copy. Nothing gets erased. WIPE doesn't look. It sets the display to 0 regardless, and "regardless" is exactly the many-to-one. Bennett has the general form of this in the 2003 notes: he lists the "erasure of one of two copies of data known to be identical" among the logically reversible operations, because you're removing a duplicate, and no information is destroyed. And it goes further than Bennett in work that came after him: the quantum physicist Lídia del Rio and colleagues argued in 2011 that the floor on erasing a system depends on how much the eraser already knows about it, dropping to zero, and in the quantum case below zero. The heat of clearing a record falls on whoever clears it, and its floor is lower the more they know. Try it with a red or blue card under a cup and a note that is blank or marked. Start with the note blank. Lift the cup: for red, toggle the note (blank to marked, marked to blank); for blue, leave it alone. The note now reports the colour. Repeat that same rule while the card is still available, and the note returns to blank. Check all four card-and-note combinations: the rule undoes itself, so no two land together. Erase the note regardless of the card instead, and for either card both note values land on blank. That rule merges on the full set of combinations. On the matched pairs you actually prepared, the card still has the information, so nothing is lost there; the merge is in what the rule has to do to every combination it might be handed, which is what a rule that doesn't look has to answer for. Cards and paper don't measure heat. What they show is which information each rule gets to use.

5.5 The smallest machine that teaches with no merge, and the one that can't

There are two more small facts, and then the point. The demon measures what you learn by comparing the start to the end, but an observer's very first look already rules things out before any step runs, so the fairer question is: after that first look, can a *step* teach you more, at zero cost? Yes, and the demon still does it. And the smallest machine that can is three states in a ring: U goes to V, V goes to W, W goes to U, and the window shows "true" only at W. Start with candidates {U, V}; both show false. Take one step. From U you now see false; from V you now see true. One

step told them apart, and a ring forgets nothing, so a zero price meets the halving rule and is the price chosen here.

Two states can't do it, ever, with any arrows, and this is the one that needs no price premise at all. If a machine has only two states and you can't tell them apart at the first look, then the window shows the same thing on both, and those two are *all the states there are*, so the window shows the same thing everywhere, forever, and you never learn anything. Try to draw a counterexample. You'll draw the same two dots a few times and give up, and the giving up is the proof. Three is the smallest number of states on which two states that look alike at the first look can come apart at a later look, and that's what teaching is.

□ **Theorem 5.3** (Three states are the minimum for teaching at zero cost)**.** Call a machine a *zero-cost teacher* when it has a cost satisfying the halving price, a list of candidates with no repeats containing the actual start, and a run of cost 0 after which the observer knows strictly more than after the first look, $|K(\Omega, t, s_0)| < |K(\Omega, [\,], s_0)|$. There is a zero-cost teacher with exactly three states, and there is none with two or fewer.

Proof. Three: the ring $U \to V \to W \to U$ with the window true only at W, candidates $\{U, V\}$, one step, cost 0 (a ring is a bijection, so cost 0 satisfies the halving price). Two or fewer: the actual start is always in the observer's knowledge, so knowing strictly more after the run needs at least two candidates still left after the first look, which agree there. On a machine with two or fewer states those two are all the states there are, so the window takes one value everywhere and no later look can separate them. This half never uses the price at all. ■

Finish together, for the first time

Run the machine from N different starts and photograph its full configuration at every tick. Put the photographs in rows, one row per start, one column per tick. Every run finishes for the first time at tick T. Through that finish, two different states in the same column must have different successors.

The sheet has $N(T+1)$ photographs, including the starts and finishes. Could two of them show the same full configuration? In the same column, cancel backwards and the starts would have to agree. In different columns, give both configurations the same remaining steps. One run would finish before its promised first finish. Every photograph has to show a different configuration.

Theorem 5.4 (Room for synchronized first completion)**.** Let F be a total rule, A a set of N distinct starts, and H a completion reading. For every $a \in A$, suppose $F^T(a) \in H$ and $F^t(a) \notin H$ for $t < T$. Suppose F is injective on $\{F^t(a) : a \in A\}$ for each $t < T$. Then all states $F^t(a)$, with $a \in A$ and $0 \le t \le T$, are distinct. Any carrier containing these runs therefore has at least $N(T+1)$ configurations. □

Proof. First compare photographs in the same column. If two showed the same state, look one column back. The rule keeps different sources in that column apart, so their predecessors must already have been the same. Repeat until the first column: the starts agree. Thus different starts cannot give the same photograph at the same time.

Now compare different columns. Suppose run a at time i shows what run b shows later, at time j, with $i < j \le T$. Run b has $T - j$ ticks left before its first finish. Starting from the identical state, run a must follow those same remaining steps and finish too. Its time would then be $i + (T - j)$, earlier than T because $i < j$. That contradicts the first-finish promise for run a.

No two places on the sheet can therefore show the same full configuration. There are N rows and $T + 1$ columns, counting time zero, so the machine needs at least $N(T+1)$ different configurations to hold the sheet. ■

That is the room required by the whole sheet of possible runs. The photographs can come from separate executions of one machine. What keeps different columns apart is the common first finish: a cycle with no such finish can put the same state in several columns. [2]

5.6 The certainty bound

Now I can put the whole path from not knowing to knowing on one page, with what it costs. You don't know which state the machine started in. There are a few it could have been, each with its chance; that's your uncertainty. Run the machine. Every one of those possible starting states goes somewhere, and only two things can happen to a pair of them. Either they land on one state, and the difference between them is gone from the machine: a merge, and the halving price charges for it, at most one bit a unit. Or they land on two states, and the difference is still in the machine, sitting in its state, kept as two states held apart. There's no third thing a pair of possibilities can do. So the uncertainty about where it started splits exactly

[2] The count is of full configurations, rather than simultaneously running machines or newly allocated memory cells.

in two: the part the merges destroyed, paid for in cost, and the part the end state still holds, paid for by the states it keeps apart.

Now put an observer at the window. Whatever they see is fixed by where the machine started, so whatever they learn is a fact about the start. And the same split holds for it: what they learned is either still held in the machine's end state, or it's gone from the machine and a merge paid for losing it. A run that costs nothing leaves every bit the observer learned still held in the machine. This is the precise sense of "learning borrows": the observer's copy is a copy, and the machine still holds the original. When the machine lets go of the original, a merge happens, and the cost comes due.

Said exactly, then: take a spread p over starting states and a run t, and write q for the spread of where the run ends. For an end state z with $q(z) > 0$, the starts that end at z have chances $p(x)/q(z)$ between them; call that the spread of the start given z, and average its entropy over the ends:

$$H(\text{start} \mid \text{end}) \;=\; \sum_{z:\, q(z)>0} q(z)\, H\big(\text{start given } z\big).$$

This is how many bits you'd still be missing about the start if someone told you the end. Do the same for anything v that's fixed by the start, like the whole film of what the window showed: $H(v \mid \text{end})$ is what you'd still be missing about v once you knew the end.

□ **Theorem 5.5** (The certainty bound)**.** On a finite deterministic machine with an entropy-priced cost (Definition 3.6), take any spread p over starting states, any run t, and anything v that the start fixes. Then (1) $H(p) = H(\text{start} \mid \text{end}) + H(q)$: the uncertainty about the start is the part the run destroyed plus the part the end state still holds; (2) $H(\text{start} \mid \text{end}) \le \text{cost}(t)$: the destroyed part was paid for, at most one bit a unit; (3) $H(v \mid \text{end}) \le H(\text{start} \mid \text{end}) \le \text{cost}(t)$: whatever the end state can't tell you about what an observer saw was paid for in cost, and everything else the observer learned is still held in the end state. In particular, after a run that costs nothing, the end state fixes everything the window showed, for every start with a positive chance.

Proof. (1) Treat the whole run as one map from starts to ends; the proof of Theorem 3.18 works for any map, so $H(p) - H(q) = \sum_x p(x) \log_2 \frac{q(z_x)}{p(x)}$, where z_x is where x ends. Group the starts by where they end: for each z the terms add up to $q(z) \sum_{x \to z} \frac{p(x)}{q(z)} \log_2 \frac{q(z)}{p(x)} = q(z)\, H(\text{start given } z)$. Adding over z gives $H(\text{start} \mid \text{end})$. (2) Let the run's moves be $i_1, \ldots, i_n$, and let $q_0 = p, q_1, \ldots, q_n = q$ be the spreads after each step. Entropy pricing says step k removes at most its own

cost: $H(q_{k-1}) - H(q_k) \leq \text{cost}(i_k)$. Add over the steps: $H(p) - H(q) \leq \text{cost}(t)$, and by (1) the left side is $H(\text{start} \mid \text{end})$. (3) Fix an end z. Given z, the value of v is a function of the start, so its spread given z is the spread of the start given z pushed through that function, and by Theorem 3.18 (again for any map) a push never adds bits. So $H(v \text{ given } z) \leq H(\text{start given } z)$ for every z, and averaging with weights $q(z)$ gives the first inequality; (2) gives the second. With $\text{cost}(t) = 0$, $H(v \mid \text{end}) = 0$, so each end state with a chance leaves one value of v. ■

Entropy pricing and the halving price are the same premise (Theorem 3.16), so the theorem holds under either. There are three things it doesn't say. It doesn't say how much heat; that's Landauer's premise, applied to the destroyed part, $H(\text{start} \mid \text{end})$, which is the uncertainty the run really removed. It doesn't say the observer's own notes cost nothing: they need somewhere to be stored, such as a blank page. Supplying that page is an initial condition; resetting an uncertain page for reuse is a separate operation. And with a known start, p on one state, every term is zero: nothing to learn, nothing destroyed, no positive floor from this premise. To ask what heat the step or the observer actually produces, give its physical process and preparation. The entropy calculation has not already charged somebody for knowing.

So the path between uncertainty and certainty runs both ways round. Going in, certainty is got by narrowing what's possible, and the deterministic calculation separates distinctions lost from distinctions retained. A copy can put a retained distinction in an observer's memory. It need not allocate a fresh cell at every step, and an initially held distinction needs no new crossing. The theorem counts entropy lost under its pricing premise; it does not put every act of knowing on a heat ledger.

5.7 What the toll prices, said at last

Now I can say, more precisely than I could two chapters ago, what the unit of cost in Section 2 is charged for. It is not charged for learning. What you know can be narrowed with nothing merged, and an honest price list that charges merges can leave it at zero; the demon is one machine where that happens. It is not charged for looking, or for copying, or for finding out. Their physical dissipation depends on the implementation. In the copying examples here they use a supplied blank: the demon's blank pad, the watcher's own notes, a page somewhere that was blank and now isn't. But they borrow that page, and while the original is still there to check against, the copy can be handed back. The unit is charged for the one thing in Section 3 that couldn't be done without merging, on a machine with finitely many states: raising a flag that stays up, a record nothing hands back. Learning is the demon writing on a blank pad, which merges nothing. What the demon pays for is the wipe, a step where two states land on one. The unit in Section 2 is charged

for the same reason, because raising a flag that stays up also lands two states on one, but it's a different step: the demon machine has no flag, and its price list prices merges; commitments never appear in it. It is the same kind of landing in a different step: learning borrows, and committing pays.

So this is exactly what "No Free Insight," the name I gave the toll before I understood it, covers, because I don't want the name doing work it hasn't earned. Insight in the sense of *finding out* can come with no merge and no cost; it borrows a page. The insight the toll prices is the kind made to stay, written into a record the machine can't silently revise. Whether that narrower thing deserves the name is a fair thing to argue about; I kept the name because it's the question I started with, and because the narrowing is the finding. I don't count it as a retreat.

It also tells you how to read a bound that's coming. Section 9 counts what it takes to narrow a list of candidates down to a shorter one: at least as many yes-or-no questions as it takes to point at the survivors among their rivals, a counting fact that holds for anything with a list in it. Turning that count into a *cost*, saying a run actually paid that many units, is a different claim, and when the book makes it, for any Thiele machine in Section 9, the payment goes in as a hypothesis, in one of two ways: a decision tree (a branching chart of yes-or-no questions, which Section 9 builds) no deeper than the number of paid steps the run took is handed in, along with the lists themselves and the fact that the tree's leaves cover the narrowing; or the run makes the narrowing itself, with every test step charged at least one and each test it passes keeping at least a set share of the candidates. This section is the reason it has to go in that way. The narrowing alone doesn't force anyone to pay; the demon just showed you a narrowing charged nothing at all. So wherever narrowing gets a price in this book, the payment is a premise, and I'll keep calling it one.

There is one more boundary. This whole section is about a machine that's finite and deterministic and a watcher who crosses candidates off a list. There's no quantum anything in it. This narrowing has nothing to do with "observation collapses the state": the narrowing here is bookkeeping in the watcher's list, the machine never notices, and the result would be the same if the watcher were a filing cabinet. In 2010 a group in Tokyo (Toyabe, Sagawa, Ueda, Muneyuki and Sano) did run a real demon, a tiny particle walked up a staircase of potential by feedback on what a camera saw, and the energy it gained was paid for by the information the camera took, with no violation of anything. The demon is real, it works, and it pays. Where it pays is the part that took a century.

What I still owe: everything so far says "costs at least one" and I've been keeping the cost on the side of the page, a running total you tally as you go. Part II first says what structure is; then comes the next question (Section 7). What happens when the machine keeps that total itself, inside its own state, where a step could in

principle reach in and change it? That's the counter, and it's the next thing I have to build.

PART II

What structure is made of



The sphere, seen from outside the plane.

Part I found what keeping takes on a single cell and wrote it into a step. Part II is step three: the rest of structure is the same thing at a bigger size. It starts with the word itself, structure, and what the people who wrestled with it left behind, and comes out where Part I started, at a sorting of a pile with a door across it, a distinction held. Then it asks what follows, on any machine, once a distinction is kept and the keeping is charged, before I build one. The counter is the ledger, and it uses up states too: a number that only climbs takes a fresh value for every unit of cost. A window is what's left when you stop looking at some parts of the state, and it can hide the very distinction that was paid for. Entitlement and insight ask what a later step may do because a distinction stayed. The latch is the blank cell's no-to-yes over any machine at all, a reading the machine's own steps turn on and nothing turns off, and a row of latches is a direction that only goes one way. The diagonal is the wall no level gets past: no state codes every question about the states (Part I proved that much), and no machine that can hand a program its own code, and can run the flip of a verdict as one of its own programs, can sort all its programs, from inside, by a question about what they do that some pass and some fail.

CHAPTER 6

What structure is

Part I put a charge beside a new permanent reading under explicit pricing rules. This chapter starts with one yes-or-no question, a cut with a direction across it, states two rulers under which it is the smallest way to sort, and goes after the rest of it. The direction is the part the toll prices. Drop it and what is left is a sorting, which is where the usual account of sameness starts, and the sorting turns out to be the symmetric half of the thing the record reads into. This whole section is about one word, and I'll give it to you now, so you can decide whether to stay: structure.

I used that word for most of my life without once looking at it, which I suspect is true of most people and most words, and it's fine, right up until you do something stupid. The stupid thing I did was build a machine that keeps a running total of its costs, and the total has one rule I couldn't talk it out of. You can arrange things however you like, and the ledger lets you put any price on the arranging, including none; the arranging has whatever physical cost its implementation imposes. But the moment an arrangement has to *stay*, the moment you ask the machine to keep it, or to vouch for it, to put its own name on the claim that these things belong together, there's a charge, and it isn't zero. Charged in what? Physical changes have costs determined by the implementation and protocol; keeping something takes more on top, a merge or fresh states. Section 3 showed there is no other way for a machine that picks its own next step to pay; when the moves are picked from outside, whoever picks them is holding the difference. How much, why I picked the one I did, and whether I had any business picking it at all: Part I went through all of that. What was in front of me, standing there with the thing built, was a model that charges for keeping structure, and I could not have told you what structure was. It didn't care. It can't nod along the way a person does. It needed the word to hold still, and the word had never held still for anyone, least of all me. (The model is already on the page: five parts and one rule, which you met in Section 2. Part III builds one small machine (Section 15) that meets it, so you can see a complete example. Until then the rule is all that matters.)

So I went looking at the people who'd wrestled it before me. Some of them have been dead two thousand years and are still, I'd bet money, better at this than I am. I

used everything I had: language models, more than one, books, libraries, my own gut, anything in reach. I don't know if I found enough. I never do. There's always another shelf behind the one I was reading. And being dead and famous is not the same as being right, so I pushed on each of them, and what follows is what I could reach, pushed on as hard as I know how. The one thing I'm asking is that you push on it with me, out loud, at your own kitchen table, because the first time through I was the only one checking, and that's not enough people.

What they left me, pushed on, comes to a few sentences, and I'll set them down here rather than walk you through everyone who said them. The relation came first; the number, when there is one, is a name for the relation, and sometimes there isn't a name. Structure is something laid in, and the stuff it's laid into doesn't carry it by itself; the structure of a collection is not one more member of the collection, and the plan isn't a brick. Sameness is coinciding under a move. Once you can say when two things count as the same, you've earned the right to name the thing they share: you sort a pile. Structure is what survives a chosen set of changes, and "chosen" is doing real work in that sentence; there's no structure of a thing from nowhere. What a thing is is how it maps: structure lives in the maps. And the charge this book opened with is on one step a machine takes; how long anybody's description of a thing is doesn't enter it. Here is where mathematical logic eventually put the word down, which is also where it holds still.

6.1 A set with some named relations

What mathematical logic eventually settled on, with the logician Alfred Tarski giving the field its name, "the theory of models," in 1954, is almost boring, and the boring ones are the ones that last. A structure is a set, called the domain, plus whatever relations and functions a chosen vocabulary names on it. Pick the vocabulary (Klein again: you choose), and a structure is anything that gives those names a meaning. Now I want to say "the smallest one," and the word "smallest" has no meaning until I say what I'm measuring, so here is the ruler: count the names in the vocabulary first; when that ties, count how many things each name takes at once; when that ties too, count how many answers each name can give. The bottom of that ruler, for anything that takes things at all, is one name, taking one thing at a time, with two possible answers, which is a domain with one yes-or-no property named on it: one question, put to each thing. A single two-place relation ("these two go together") can sort a pile too, and so can a single function that sends each thing to a label, but the first takes two things at once, and the second can give a different answer for every thing, more than two on any pile of three or more, so the ruler puts both above one question. There are two limits, and I'll give them to you now, so you don't have to find them. A vocabulary with one named constant ("this one, here")

takes no things at all, so it sits even lower on my ruler, and by itself it doesn't sort anything, so the claim is "smallest that sorts." Comparing things with the constant takes equality, and I count equality as a name when a structure uses it, one that takes two things at once. Leave it uncounted, as first-order logic does, and the constant sorts below one question: "is it this one?" splits a pile into that thing and the rest, which is still a split in two, one question whose yes is a single thing. Plain "smallest" would be false. And one question only splits the domain into two groups when some things answer yes and some answer no; if they all agree there's one group and nothing was sorted. With those limits said, and with the ruler said: by the count of names, the smallest structure that sorts anything is one question. Here is a second ruler, which counts the rules a relation has to obey instead of the names, and it lands in the same place. A sameness, "these two count as the same," obeys three: everything is the same as itself, the same both ways, and the same across a third. An order, "this one is at or below that one," obeys two, the first and the last, and drops the middle one; that two-rule order is the preorder the record of Section 2 already reads into. One question is such an order on the pile: a thing that says no is at or below a thing that says yes, two things that say the same are at or below each other both ways, and nothing else. So by the count of rules the smallest structure that sorts is again the one question, read as a two-place order with one door in it, and the three-rule sameness sits above it. Least information lands one bit lower, on the partition: on a nonempty pile of n things a question can come out 2^n ways, and a split into at most two groups with neither one called yes can come out half as many. The bit between them is the label, which group says yes, and it's the door. Keep it. It's about to turn into the whole section.

6.2 Laying them all down

Now lay those sentences on top of each other and see what shows through. I see seven things. They're my overlay, the places where I see the people who said them agreeing. They never proved it together; some of them never met, and some of them disagree (Eilenberg and Mac Lane put maps and their composition at the centre; Shannon's structure and Tarski's are two different words). You're in a position to tell me if I'm seeing things.

A list of seven slogans would be easy to nod at and impossible to check, so I'm going to carry one thing through all seven and let you watch each one add to the one before it. The thing is a square, side, diagonal and all: draw one. Draw a few more on the same sheet: one small, one big, the small one again but tilted, and one stretched out into a long rectangle. You'll want a pile.

Structure is about relations among things, and the things themselves aren't it. You see that in the diagonal, the bronze, Leibniz's room, Bourbaki's unspecified

elements, Tarski's named relations and the arrows. On your sheet, what makes a square a square was never the ink. It's that the four sides match and the corners are square, and that the diagonal stands to the side the way it does whether or not any number names it. Every one of those is a relation among parts. But on its own this first thing is too loose to use, because a sheet of paper is drowning in relations: this square is left of that one, this one is nearer the coffee ring. Which relations count?

Structure is what stays the same under a chosen set of changes. You see that in Euclid's triangles laid together, Klein's allowed moves, Noether's symmetry, and Frege's direction that every parallel line shares. This second thing gives the answer the first thing was missing: the relations that count are the ones that survive the moves you've decided to allow. Slide the small square across the sheet, turn it, and the diagonal still stands to the side exactly as before; "left of the coffee ring" doesn't make it across. The second thing is a filter on the first.

The same structure can be carried by different stuff: the statue in marble, or two sets that answer the same vocabulary the same way. The third thing is the second again with one more change allowed, which is to swap the material. Cut the square out of cardboard, or lay it out in string on the table, and if the relations survive, it's the same square in every way you agreed to care about. Aristotle's bronze is one more thing you agreed not to look at, and the wooden sandwich is what happens when the material was never on that list.

The first act of structure is a sameness that sorts a pile you were already given. It's there in Euclid's first common notion, Frege's direction, Tarski's one question, and Russell's letter, with Zermelo's repair that you must start from a pile. At this point the overlay stopped being a list for me and turned into a shape, so slow down here with me. Look at your pile of drawings and say: two of them count as the same when an allowed move, for now a slide or a turn, carries one exactly onto the other. Watch what happens to the pile. It falls into groups. The small square and its tilted twin go together; the big one sits on its own; the rectangle sits on its own. And it falls cleanly, nothing half in one group and half in another, for a reason you can check. Klein's allowed moves always come with three guarantees: doing nothing counts as a move, every move can be undone, and two moves in a row is again a move. Those three are, in different clothes, "everything is the same as itself," "if this matches that, then that matches this," and "if this matches that and that matches a third, then this matches the third." Put the last two together and you get Euclid's first common notion, "things which are equal to the same thing are also equal to one another." So the second thing, sameness under chosen moves, *is* a sorting of the pile the moment you ask which drawings it calls the same. Frege did it with lines and parallels. Tarski's one question does it in the crudest way there is, by cutting the pile in two. And the pile has to be there first, because a sameness has to be a sameness among something; that's Russell's letter, and Zermelo's repair.

Adding structure rules possibilities out. Every one of Euclid's postulates rules one out. Half of English is already decided. A conserved quantity limits where a system can go. On your sheet: once the small square and its tilted twin are in one group, you've given up the right to answer any question the moves respect differently for the two of them. Tell me the side of one and the diagonal of the other isn't an open question any more. A sorting rules out a whole bundle of possibilities at once.

Held structure gets used without re-deriving it. You see that in Shannon's saving in time, Rissanen's compression, Parnas's one-module-at-a-time, and Leibniz's analysis of situation (reasoning about where things are straight from their relations, with no coordinates laid down first). The fifth thing pays off here: the possibilities you ruled out are ones you never have to check again. Know which group a drawing is in and you know everything the group shares without getting the ruler out, the same way the row of `01` never needed writing out. There's one catch, and I'll need a whole section for it: it only pays if you kept the sorting somewhere you can read it back.

And structure is always relative to a chosen frame, whether that frame is Klein's group, the vocabulary, the code you compress against, or Parnas's criteria. Nobody gets to say "the" structure of a thing without saying from where, and I do it constantly, so watch me for it. On your sheet: allow stretching as one of the moves and the rectangle walks over and joins the squares; allow only slides and turns and it doesn't. Same pile, same ink, different groups, because I chose different moves. The groups were never in the paper. They were in the paper and the choice, together.

I think the seven hang together this way. Relations are what structure is made of. The chosen moves pick which relations count, and a change of material is just one more move. Asking which things the moves call the same sorts the pile. The sorting rules possibilities out, and what's ruled out is what you get to skip later. And because the moves were chosen, all of it is relative. Read that way, the fourth thing sits in the middle: it's where the first three turn into something you can write down, and where the last three get their start.

That's my reason for starting the next subsection with a sorting and a door across it, and I want to be exact about what kind of reason it is, because it's mine. None of these people said "start with a partition." Klein would tell you to start with the moves, Eilenberg and Mac Lane with the arrows, Shannon with the probabilities. What I can actually point at is narrower. In the framings about when two things are the same shape or the same kind (Euclid laying one triangle on another, Klein's moves, Frege's directions, Tarski's question) there's a moment where somebody decides what counts as the same, and that decision, written down, is a sorting of a pile. In Shannon, Rissanen and Noether I can't find that moment as cleanly, and I'm not going to bend them until I do. So the overlay doesn't prove that a sorting comes first. It shows me where the framings I trust most keep putting their first foot down,

and I'm putting mine there too, with one thing added that none of them needed and the toll does: a direction across the sorting.

I looked for a framing that denies any of the seven and didn't find one, which could mean there isn't one or could mean I didn't look where it lives. If you know of one, I'd like to see it.

6.3 The smallest piece

Now the sentence I wanted to say at the top and had no right to until here, and even here I have to split it in two, because half of it is a textbook fact and the other half is me. The piece of structure I'm going to start from is a cut: a pile sorted in two, with a door between the halves that swings one way. A partition is the cut with the door taken off, and the partition comes with a textbook fact, so I'll do that half first.

The textbook half: a partition and an equivalence relation are the same object. That's Paul Halmos, *Naive Set Theory*, 1960, section seven, and you could read that section before dinner. I'll say it in words first. Take any pile of things. An equivalence relation on it is a way of saying "these two count as the same" that behaves the way sameness should: everything is the same as itself; if a is the same as b then b is the same as a; and if a is the same as b and b the same as c, then a is the same as c. Every such relation sorts the pile into groups, the things that count as the same go together, and every sorting into groups hands you back a relation, "same group." Do one and then the other and you land where you started, in both directions. So a partition and an equivalence relation are one object seen from two sides, and that's the fourth thing on the overlay, the sorting of a given pile, now with a name and a page number.

I'll give it in the exact form too, since this is the first place the book uses one. A *set* is a pile, $x \in X$ means "x is in the pile X," and a *relation* on X is any rule $\sim$ that, for each pair of things, says whether $x \sim y$ holds. A few more symbols show up below, and each is a word in disguise. The buckets are written S_j, with j saying which bucket (finitely many buckets in every example here, though nothing below needs that). A cap, $S_j \cap S_l$, is "what the two buckets share," and $\varnothing$ is the empty pile. And curly braces with a colon, $\{\, y \in X : y \sim x \,\}$, mean "the pile of every y in X for which $y \sim x$ holds"; the theorem writes that pile $[x]$, the things that count as the same as x.

Definition 6.1 (Equivalence relation, partition)**.** A relation $\sim$ on a set X is an *equivalence relation* when it is reflexive ($x \sim x$), symmetric ($x \sim y$ implies $y \sim x$), and transitive ($x \sim y$ and $y \sim z$ imply $x \sim z$). A *partition* of X is a collection of non-empty subsets of X, the blocks, such that every member of X lies in exactly one block. Spelled out, with the blocks written S_j: each S_j is non-empty, no two share

a member ($S_j \cap S_l = \emptyset$ when $j \neq l$), and together they cover everything (every member of X lies in some S_j).

Theorem 6.1 (Sameness and sorting are one object)**.** Every equivalence relation on X determines a partition, whose blocks are the classes $[x] = \{\, y \in X : y \sim x \,\}$; every partition determines an equivalence relation, "x and y lie in the same block"; and the two constructions undo each other. □

Proof. The classes are non-empty ($x \in [x]$ by reflexivity) and cover X. If z lies in $[x]$ and in $[y]$, then $z \sim x$ and $z \sim y$, so $x \sim y$ by symmetry and transitivity, and then $[x] = [y]$ by symmetry and transitivity again (anything the same as x is the same as y, and the other way round); so two classes either coincide or don't overlap. Going the other way, take a partition and say two things match when they share a block. Each thing shares its block with itself. If one shares a block with another, the other shares it with the first. And if x shares with y and y shares with z, both blocks contain y. A partition puts y in only one block, so all three share that block. These are exactly reflexivity, symmetry and transitivity. Going relation to partition to relation returns "x and y share a class," which holds exactly when $x \sim y$ (if $x \sim y$ then x and y both lie in $[y]$; if both lie in $[z]$ then $x \sim z$ and, by symmetry, $z \sim y$, so $x \sim y$), so it is the original relation. Going partition to relation to partition returns the original blocks, because the class of x under "same block" is exactly the block that contains x. ■

All of this is textbook mathematics (Halmos, section 7), and the book uses it as a foundation. I'm claiming nothing new in it.

And the smallest structure that sorts, by my ruler in Tarski's setup, the one question, is the same object a third time. Put a yes-or-no question to every thing in the pile. The ones that say yes are one group and the ones that say no are the other. Going the other way, a two-group partition becomes a question the moment you point at one group and call it "yes." The only thing a question has that the partition doesn't is which group got the label. So the set theorist's first step and the logician's first step are the same step, and nobody has to pick a side, which is a relief, because it's an argument you can lose with yourself for a long time.

Now put the door back, exactly. An *order* on a pile, "x is at or below y," written $x \leq y$, has to obey only two of sameness's three rules: everything is at or below itself, and at or below carries across a third. It need not go both ways. That is the preorder of Section 2, the thing the record reads into, and here is what it is made of.

Definition 6.2 (Preorder, its sameness and its doors)**.** A relation $\leq$ on X is a *preorder* when it is reflexive ($x \leq x$) and transitive ($x \leq y$ and $y \leq z$ imply $x \leq z$). Its *symmetric part* is $x \approx y$ when $x \leq y$ and $y \leq x$; its *strict part* is $x < y$ when $x \leq y$ and not $y \leq x$. A *cut* is the preorder a reading $P : X \to \{\text{no}, \text{yes}\}$ puts on X: $x \leq y$ when $P(x) = \text{no}$ or $P(y) = \text{yes}$.

□ **Theorem 6.2** (An order is a sorting with doors)**.** (1) The symmetric part of a preorder is an equivalence relation, so it sorts X into buckets (Theorem 6.1); two things share a bucket exactly when each is at or below the other. (2) Whether $x \leq y$ depends only on the buckets of x and y, and on the buckets "at or below" is an order that goes one way: two buckets each at or below the other are one bucket. (3) So a preorder is exactly a partition of X together with a one-way order on its blocks, and a preorder with an empty strict part is exactly a partition. (4) A cut has at most two buckets, the things that say no and the things that say yes, with the first at or below the second and not the other way round; forget which bucket is yes and what remains is the partition of the question.

Proof. (1) Everything is at or below itself both ways. The symmetric part is the same both ways by how it's written. If $x \approx y$ and $y \approx z$, then $x \leq z$ and $z \leq x$ by carrying across y. (2) If $x \approx x'$, $y \approx y'$ and $x \leq y$, then $x' \leq x \leq y \leq y'$, so $x' \leq y'$. On the buckets, at or below is reflexive and transitive because it is on the things, and if the bucket of x is at or below the bucket of y and back again, then $x \approx y$ and the buckets are one. (3) From a partition and a one-way order on its blocks, say $x \leq y$ when the block of x is at or below the block of y: reflexive and transitive come from the order on the blocks, its symmetric part puts x and y together exactly when their blocks are at or below each other, which by one-wayness is when they share a block, and its order on the buckets is the one given. The other direction is (1) and (2). If the strict part is empty, $x \leq y$ always comes with $y \leq x$, so the preorder is its own symmetric part and (1) makes it a partition. (4) A thing that says no is at or below everything, a thing that says yes is at or below exactly the things that say yes, two things with the same answer are at or below each other, and a thing that says yes is not at or below a thing that says no. ■

And here is why this chapter's start and the book's definition are one thing. Pull the record's order back to the states: say s is at or below t when $\text{rec}(s) \leq \text{rec}(t)$. That is a preorder on the states, reflexive and transitive because the order of records is. Its buckets are the states the record can't tell apart, and inside a bucket the toll never speaks: a move that stays in its bucket lands at or below where it started, and so does a move down. Its doors are where the toll speaks: a move that lands

somewhere not at or below where it started is a climb, and the toll charges it. The flat case of Part I is a machine whose moves never go through a door upward. The two-place record is the cut, one door, and the first chapter of this book was the smallest instance of exactly this: a cell going from blank to 0 is a move through the one door of a cut, and what Part I found keeping takes is what it costs to go through and not come back.

So much for the textbook half. The other half is mine, so hold it looser, because Halmos says nothing about "smallest" or "first" and I'd be lying if I let his page number carry my sentence. In what I'm going to build, the order goes: a pile, then a cut in the pile with its door, then more cuts at once, which is an order, and the groups come first because a question needs something to be about and a door needs two sides to stand between. That's an order for *my* construction. It is not a law that every account of structure has to start by sorting. You could start with arrows between the things themselves and no groups at all, and people do, and nothing I've shown you rules that out. So the honest sentence is: a cut is where I chose to start, it is the smallest start by both rulers I gave you, the partition is its symmetric case, and I chose it because it's the first thing that can be right or wrong about a pile and the last thing I could take away without everything I build on it coming down. The question I used to leave open here, whether there is a smaller start than a partition, has its answer from inside the definition: drop symmetry, and the two-rule order the record already reads into is that start, with the partition as the case where every door is off. If you have a start smaller than that, by a ruler you can state, I want to see it. It would change this book.

6.4 Buckets

So I'll do the symmetric half once, for real, because it's the half you can do with your hands. Take everything in front of you and sort it into buckets, with nothing in two buckets and nothing left on the floor, and that's the whole of the definition. It's the thing you do with a kindergartner and a pile of blocks, and I am aware that I've just marched you past two thousand years of people to arrive at a pile of blocks. Keep the blocks anyway. They're about to cost you something.

Take ten blocks, and put the red ones in one bucket, the square ones in another and everything else in a third. Now a red square turns up, as they do, and you've got your first crisis: it belongs in two buckets and the rule says one. So this isn't a partition yet, and you have to decide which bucket wins. That decision is the whole thing, because a partition is a pile of small rulings like that one, and once you've made them all, the pile is holding a fact it wasn't holding a minute ago. The formal version says the same thing with fewer words and more squiggles, and you've

already seen it: it's the definition of a partition in the last subsection (Definition 6.1), which asks for non-empty buckets, with nothing in two and nothing on the floor.

The next part isn't kindergarten. Why would that fact cost anything? Watch what it does to one question. Before the buckets, "are these two in the same group?" means working it out: go find the first one, go find the second one, apply the rule to each, and compare. After the buckets, it's a glance: read the first one's bucket, read the second one's bucket, and you're done. With each block's address already in hand, and a table that retrieves a label in one operation, that is two lookups whether the pile has ten blocks or a billion. Those are access and cost assumptions. A pile with no addresses, or an index that must be searched, has not been given that same two-lookup rule. What you're looking at is Shannon's saving in time and Parnas's one-module-at-a-time, the sixth thing on the overlay, showing up in a kindergarten.

Look at what makes that glance possible: a *table*, with a written-down line for each block saying which bucket it went in. The partition is the fact; the table is one way of holding the fact so it can be read back fast, and you had to build the table, and you have to store it, and it has to be the kind of thing you can read in one look (a bucket number, say; a list of every block in the bucket would have you searching through it again). Anybody who's used a database has met this table; it's what an index is, and an index is a thing you build, store, and keep up to date, and every part of that takes something. Take the table away and keep the partition and you're back to working it out. And sometimes there was never a search to save: if the rule is "evens here, odds there," you can answer "same group?" by looking at the last digit of each, no buckets needed. What survives is that holding a sorting in a form you can read back is worth something, and building that form cost something, and those are already two different events. Building the table, storing it, checking that it's right, and deciding to *rely* on it are four different things. The first cost question I want to put to that structure lives in the gap between the third and the fourth. Nothing in the kindergarten has shown a charge in that gap. It has separated the acts well enough to ask.

So who, in everything I read, says what has to be paid? I found three answers that come close, and I'll be exact about how close, because this is the edge of what I know. Rolf Landauer, in 1961, and Charles Bennett after him, say that any step which takes two possible situations and makes them one, erasing a bit or merging two paths a computation could have taken, has a heat floor when it removes uncertainty under their thermodynamic assumptions. That is a physical claim about how the device and its surroundings are prepared, Landauer's principle, and in this book it is always a premise, never something I prove. It's the place, of all I've found, where a fact about information turns into a fact about heat, though at room temperature the amount for one bit is far too small to show up on anybody's electricity bill. In 2012 a lab in Lyon erased one bit, held in a single tiny bead, and measured the average

heat it gave off coming down toward Landauer's floor as the erasing was slowed, and other labs have found the same since. Not everyone accepts the principle in general, and I'll come back to the people who don't. Is sending blocks to buckets a step of that kind? Push on it, because the answer depends on what you keep. If you keep each block and write its bucket next to it on a spot you knew was blank, you've merged nothing; you can read the block right back off the pair. You have spent something, though: the blank spot the label now sits on, which the machine hadn't used before, and any heat from the writing depends on its implementation. Bennett's whole point in his 1973 paper was that a computation which keeps its inputs, and keeps enough of its working to retrace its steps, can be run without any merging step at all, and it pays for that by taking fresh states. If instead you throw the block away and keep only the bucket label, then two different blocks can leave the same label behind, and that is a many-to-one step, the kind Landauer charges for. Both are things people call "sorting into buckets." Only the second loses a distinction. So the bucket by itself doesn't hand you Landauer's heat. The *forgetting* does, when there is forgetting, and keeping instead is paid for with fresh states. Whether a given machine forgets is a question about that machine. It has to be answered on the page, and I won't wave at it here. Rissanen says a model is paid for in the bits it takes to state, and Grünwald says the price depends on the code. Parnas says a bad cut is paid for later, at two in the morning. And that's where it stops. Rovelli, back in Section 5, prices a choice that sticks, and his choice is one nobody could predict, which that section showed is a different act. I couldn't find anyone who says that the moment you *vouch* for a grouping, the moment you stand behind "these belong together," has a price of its own. Just arranging them that way is a different act. That could mean nobody says it. It could mean I didn't find the person who does. I honestly can't tell you which.

So that's where I am at the end of one word. A grouping, the act of making it, the information kept about it, and a commitment to rely on it are different things. The blocks let you separate them with your hands. Now take them down to the smallest case there is, a bucket with two sides, one block, and a ruling that sticks, which is a cell going from blank to 0, the case Part I started from and showed what keeping it takes. The construction I will study keeps selected distinctions in a state. Some are there from the start. Some are newly recorded. Those are different cases: a box that already says YES can sit there forever under a do-nothing rule, with neither a merge nor fresh states. Part I's count starts with NO and follows a new YES that stays under a fixed total rule. That is the case to which its merge-or-fresh conclusion applies. A grouping by itself does not supply that run or its price.

CHAPTER 7

The counter

Everything so far has kept the count of cost on the side of the page. You played Section 2's toy, you made a tally as you went, and the tally lived in your hand. The machine never saw it. A tally takes a state it has never used for each unit it records and never goes back to an old one, and putting it inside the machine doesn't change that; it only makes the machine the one paying with those fresh states. That's fine for a toy. For anything that's going to run on its own, with nobody tallying, the total has to live somewhere the machine can reach, which means the machine can, in principle, write it. So this section is about what a counter has to be before you're allowed to call it a receipt. I mean the counter of *any* machine whose state carries a running total that every step bumps by whatever a price list says that step costs. Section 2's five parts don't need one (there the total was a sum you did on the side), so this is one piece added to the five, and the facts below are about anything that has it. I'll give you the answer first, because it's short, and then spend the section on why each word is there: a receipt is a total that lives in the state, that only climbs, that equals the sum of what was charged, and that nothing else could have been.

7.1 A stick, split down the middle

The oldest receipt I could find that you can still hold is a piece of wood. From about 1100, under Henry the First, until 1826, the English treasury recorded what it was owed on tally sticks: a strip of hazel, notched across with cuts of set widths (the treasury's own handbook from around 1180, the *Dialogue of the Exchequer*, gives them: a thumb's width for a hundred pounds, a swollen barleycorn for a pound, a bare cut for a penny), then split lengthwise so that each half carried the same notches. The payer kept one half, called the stock, which is one story for where the word stockholder comes from. The treasury kept the other. When the time came to settle, you laid the two halves together, and only the two original halves fit, because wood splits along a grain nobody can copy. Add a notch to your half and the other half shows a gap where the partner should be. Try to remove a notch and you find that wood can't be uncut. When they finally stopped using them, the clerks burned centuries of old tallies in the furnaces under the House of Lords, overheated the flue,

and burned down the Palace of Westminster, which I mention because it's the best fact I picked up for this book and because it tells you how many of them there were.

Do it: draw a border round a strip of card, notch it once for every coin you hand a friend, then split it lengthwise through the notches and keep half each. Don't trim the edge or swap in a fresh strip. Now add a notch to one half and put them back together. You'll see the mismatch straight away. And you can't put a cut back: a patch is just one more thing to look at. The total is the notches, one per coin, and under that rule the count only climbs. Both halves started with the same count, so if they disagree now, somebody changed one. The mismatch won't tell you who. What the stick gives you is a comparison you can make, not an object nobody can forge and not a verdict on who cheated.

Economists read the same stick another way, and they're not wrong either. Alfred Mitchell-Innes (1913) made the exchequer tally his star witness that money is credit: the crown paid the people it owed by "raising a tally" on some branch of its revenue, a wooden acknowledgment of debt. A receipt to me, money to him, and it's the same piece of wood. Narayana Kocherlakota (1998) later proved a version of his point: in a wide class of economies where nobody can be held to a promise, anything money lets people do, a memory of everyone's past dealings lets them do too. His title says it in three words: money is memory.

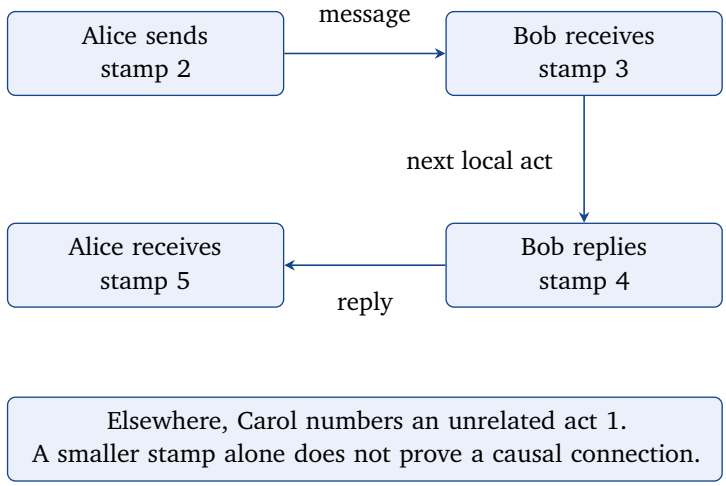
7.2 Why it has to climb

Start with the second property, because it's the one that does the most work and the one people fight hardest. Why can't the counter ever go down?

The answer is in what the toll is for. The floor it buys, back in Section 2, says a run from NO to YES costs at least one. That sentence is a *floor*: it says the run paid at least this much. If the counter could come back down, a run could pay the toll, get its flag, and then take the payment back, and the floor would be a floor on nothing; "this run cost at least one" would be true of a run that ended having cost nothing. So a counter that can fall is an accountant you can lean on to revise the books, and a floor stated in terms of it isn't a floor. The counter only climbing is not a flourish; it's what makes "at least" mean something. This ledger records cumulative charges, each a whole number of units and none negative, so it never decreases. A refundable balance, one that pays money back, would be a different accounting object with a different invariant, and the floor here says nothing about it. (In Section 2's abstract theorem the total is a sum of whole numbers and climbing is automatic; this paragraph is about why a total *stored in the state*, where a step could in principle write it, has to be designed so that no step ever lowers it.)

The same rule turns up in two places that have nothing to do with each other, pointing in opposite directions, and that's a big part of why I trust it. In 1978 Leslie

Lamport asked how a set of computers that pass each other messages, with no shared clock, could ever agree on what happened before what. His answer is a counter per machine with two rules: add one before you do anything, and when a message arrives carrying the sender's count, jump your own counter above it (never below your own). Those counters, he showed, respect cause and effect: if this caused that, this has the smaller number. And they do it "with no actual timing mechanism," his words; they're clocks only in name. Try it with three people passing notes and you'll see it work in two minutes, and then let one of them reset their counter to zero after reading a note, and watch a reply come out numbered *before* the note it replies to. A counter that goes down can't be leaned on for order.



Start Bob at zero and Alice just after her local act 1. On receipt, each raises the local count above both its old value and the incoming stamp.

Where did Lamport get it? In the notes he keeps on his own papers, he says the timestamps came from an earlier note by Paul Johnson and Bob Thomas on keeping copies of a database in step, whose algorithm wasn't quite right: it let things happen in an order that broke cause and effect. What let him see the fix, he says, was special relativity, which teaches that the one order of events everybody agrees on is the order of what could have caused what. And he passes on a line from Jim Gray, the database pioneer whose commit comes back in Section 14: Gray had heard two opinions of the paper, that it was trivial and that it was brilliant. Lamport's answer is that he can't argue with the first and isn't inclined to argue with the second. His account stops at relativity, so the chase does too. It leaves a good picture behind: these update rules put a smaller number on every cause than on its later effect, without a shared clock. The converse fails: two people can number unrelated local acts 1 and 2. The numbers alone do not say that the first caused the second.

Now the other direction. In 1949 Alan Turing, and in 1967 Robert Floyd, each asked how you prove a program *stops*. Both times the answer was to find a quantity

that goes *down* on every step and can't go down forever (a whole number that can't go below zero, say). If every step lowers it and it can't fall forever, the steps can't go on forever. Up-only gives you order, and down-only gives you stopping. They're different rules (one needs the jump-above-the-message step, the other needs a floor it can't fall below), but they have the same shape, seen from the two ends: a one-way quantity is something you can build on, and a two-way quantity is a number that happens to be written down somewhere.

There's also a real-world case where a counter was allowed to go down and got gamed exactly there. Ethereum, one of the largest blockchains, charges "gas" for every step of a program, and for years it *refunded* gas when a program freed up storage. Someone noticed you could fill storage when gas was cheap, hold it, and cash the refunds when gas was dear, and a token was built to do exactly that, and the real work in a block could reach nearly twice the block's stated limit. In 2021 a change called EIP-3529 cut the refunds sharply (none for self-destruct, a smaller one for clearing storage, and a cap of one fifth of the gas used). The written reasons are the gas token and those overfull blocks. I read both as the same lesson: a counter that goes down was being leaned on at the place where it went down. Ethereum's gas comes back in Section 17 as one of five small models of systems people actually run, and it's the only one of the five that takes a fee at the moment of the write, and it's instructive that the part they had to fix was the part that could fall.

7.3 Conservation: the total is the charges and nothing else

The third property is the plainest one. The counter after a step is the counter before plus what that step was charged. Add up a whole run and the middle values cancel out, the way a sum of differences collapses to last minus first, and the total is the sum of the per-step charges. Why state it?

I state it because it's the property that fails quietly. Say a machine charges one for raising a flag, and also, somewhere in its error handling, forgets to add the charge when the step fails halfway through. Or it charges two in one code path and one in another for the same move. Or it has one step that writes the counter directly. Then the counter is *mostly* the sum of the charges, and "mostly" is the whole gap between a receipt and a note that says "trust me, about this much." Conservation is the sentence that there is no "mostly": every step, every outcome, every error, adds exactly its charge, and a step that fails is charged the same as one that succeeds, because the charge is on the attempt. For a machine built the way I'm describing, that sentence is the rule the counter is updated by, so it can't fail by definition. What can fail is the claim that a real machine *is* built that way, and checking that means going through every move it has, the failure branches included, one at a time, which is dull and is the whole job. The run-long form, counter at the end equals counter

at the start plus the sum of the charges, then follows from the one-step rule by the cancelling sum; it's below. It's the kind of theorem that looks boring and is the one you'd most want if you were going to rely on the number. One thing it doesn't say is that every unit of cost bought something. The ledger is exact about what was spent. What the spending was worth is another question, and whether a unit of cost bought a real narrowing is Section 9's question, and it needs a witness on top of the ledger entry.

Which mergers does this counter cover?
Draw the arrows of one run and put a receipt beside each departure. Charge on entry into a parked state and let the parked state idle free. The charge now depends on where the step begins. You can check exactly which mergers those receipts cover.

This paragraph changes the accounting interface explicitly. The basic Thiele definition uses a fixed instruction price $c(i) \in \mathbb{N}$. Here one autonomous step F has a source charge $a(s) \in \mathbb{N}$; with supplied instructions the broader signature would be $\widetilde{c}(s, i)$. Its bill must be summed along the actual run. The same instruction can now cost differently at entry and at idle. The theorem below states its own charge and coverage premises; it does not change the instruction-cost definition used elsewhere.

Start at u and follow L steps. The source positions are $u, F(u), \ldots, F^{L-1}(u)$. Mark a landing state if two different sources in that stretch lead to it. Count each marked landing once and call the total $\mathrm{Merge}(L)$. Walking past the same source twice still gives one source. The bill, meanwhile, adds the charge for every step taken.

The pricing rule is this: whenever two different observed sources share a successor, at least one of them pays. Give each source its charge $a(s)$ and let the ledger add exactly that charge. Now every marked landing has a receipt behind it.

□ **Theorem 7.1** (Receipts covering the observed mergers)**.** Let $a(s) \in \mathbb{N}$ be a source charge and let $\mu(F(s)) = \mu(s) + a(s)$. Suppose every pair of distinct observed sources with the same successor has a positive charge on at least one member. Then

$$\mathrm{Merge}(L) \leq \sum_{t=0}^{L-1} a(F^t(u)) = \mu(F^L(u)) - \mu(u).$$

Proof. Every counted landing state has a paid predecessor. One predecessor cannot belong to two different landing states, since its successor is fixed. Thus there are at least as many paid source occurrences as counted landing states, and each costs at ■ least one. The ledger equality follows by adding its updates.

Try it with no going to yes and yes going to itself. Charge one at no and zero at yes. Give those states ledger values zero and one. Entry adds the one; every later idle step adds zero. The collision is covered and the parked machine can stay parked for free.

Now ask that parked yes-state how many mergers your observation has exposed. After one step you have seen only no as a source: no merger yet. After two steps you have seen no and yes, both leading to yes: one merger. The endpoint in your hand is yes both times. It cannot tell you which observation you made.

Add a clock and you can keep that difference. The endpoints become $(\text{yes}, 1)$ and $(\text{yes}, 2)$. You can read the clock and recover the count for the old graph. Draw the new full graph, though, and those clock values keep its states apart. Keep both drawings on the bench so you can see which arrows you are counting. An external record of the observation length can keep the same missing information.

7.4 Only one honest count

The next property surprised me by being provable at all. Suppose you fixed the schedule: this move costs one, that move costs zero, and so on, for every move. And suppose someone else keeps their own books: their own function from states to totals, which starts at zero and, on every step, goes up by exactly what the schedule says. Could their books disagree with the machine's counter, somewhere, on some state you can actually get to?

No, and the proof is the tally stick. Both totals start at zero. On every step both go up by the same amount. So on every state you can reach by steps from the start, they're equal. Nothing more is needed. Agreement at the start survives the first move, then the second, then every move after it. This way of proving a statement for any finite number of moves is called induction. The mathematician's word for this is *initiality*: the counter is the one bookkeeping that all the others, if they're honest about the schedule, turn out to be. Given the schedule, the count is forced. There are two things about that sentence I had to be careful with, and they're worth saying before the symbols. It doesn't assume the other books climb: climbing follows from the schedule, since the schedule never charges a negative amount, so "only one honest count" and "it only climbs" aren't two separate demands on the other bookkeeper. And it holds on *reachable* states only. A state you can't get to from the start is one where it says nothing about whether the other books agree, which is the right thing for a receipt to say.

I'll state the properties exactly now: first the piece being added to Section 2's five, a counter and a price list, and then the two facts about it. A *counter machine* here means a machine that keeps this ledger. It isn't Minsky's counter machine, the two-counter base Part I mentioned; the word is the same and the thing is different,

and Section 15 keeps the two apart. The counter gets the Greek letter μ (mu), which is the letter the machine in Part III uses for it (Theorem 15.6).

Definition 7.1 (A counter kept by a price list)**.** A *counter machine* has a set of states, a set of moves, a step function apply that takes a state and a move, a start state s_{init}, a schedule c giving each move a cost in $\mathbb{N}$, and a counter μ from states to $\mathbb{N}$, such that $\mu(s_{\text{init}}) = 0$ and $\mu(\text{apply}(s, i)) = \mu(s) + c(i)$ for every state s and move i, with no exceptions. A *run* of a list t of moves from s applies them in order, and $c(t)$ is the sum of their costs. A state is *reachable* when it is the start or is $\text{apply}(s, i)$ for a reachable s and some i.

□ **Theorem 7.2** (Conservation)**.** In a counter machine, for every state s and every list t, μ at the end of the run of t from s equals $\mu(s) + c(t)$. In particular μ never decreases along a run.

Proof. Write the starting counter at the top of a column. Under it write the charge for the first move, then the charge for the second, and continue to the end of the run. After the first move the counter is the sum of the first two entries, because that is its one-step rule. After the second it is that subtotal plus the next charge. Every further move adds exactly the next entry. With no moves there is only the starting entry. With any number of moves, the final counter is therefore the starting value plus their charges, which is $\mu(s) + c(t)$. Every entry added is nonnegative, so no ■ subtotal is smaller than the one before.

□ **Theorem 7.3** (Only one honest count)**.** In a counter machine, let B (for "books") be any function from states to $\mathbb{N}$ with $B(s_{\text{init}}) = 0$ and $B(\text{apply}(s, i)) = B(s) + c(i)$ for all s and i. Then $B(s) = \mu(s)$ for every reachable state s.

Proof. Choose a run that reaches s from the start; such a run exists because s is reachable. At the start both books say zero. At the first move both add the same charge, so they still agree. Whenever they agree before a move, adding that move's charge to both keeps them equal afterward. Carry this comparison along the chosen ■ run. The two books agree at its last state, which is s.

One thing follows that's easy to miss. If a run comes back to a state it has already visited, the counter is the same both times, so the moves on that loop cost nothing in total. A machine that keeps its counter inside its state can go round a loop only when the loop costs nothing, and every paid move lands on a state the run has never

been in, because the counter is higher there than anywhere before. So a counter that only climbs is itself paid for in fresh states: it needs a fresh number for every unit of cost, and it never gets one back. On finitely many states that leaves no paid move at all: play any move over and over from any state, the states come round, and a loop costs nothing, so its price is zero. One move priced above zero takes infinitely many states.

Those two proofs are the whole of it, and they're about any counter machine. They're on the page, where you can check every line. The machine I build later is one of these, and on it the run form is a theorem: the ledger after any list of instructions is the start plus the sum of their costs (Theorem 15.6). The uniqueness half holds even with the books' rule asked only of reachable states.

I want to say plainly what initiality does *not* do, because the fact is smaller than the word. It doesn't pick the prices. The schedule is an input: somebody decided that raising the flag costs one and adding two numbers costs zero, and nothing here says that decision was right (Section 3 is where I argued the floor isn't arbitrary; the rest of a schedule has to be defended by whoever writes it). It doesn't turn units of cost into anything physical. And it isn't the category theorist's "initial object," though the word is borrowed for a related reason.

Try the same instruction list on another machine that understands those instructions. Choose its start. The empty list leaves it there. Append one instruction and its own rule tells you the next state. Append another and the rule tells you again. There is only one answer at each stage. So there is only one way to assign a target state to every instruction list while respecting the chosen start and each appended move. This walking through a list is often called a *fold*. Nothing in the argument used a certificate or a price.

Now ask a different question. Can you assign a target state to each *state of the first machine*, instead of to each instruction list? Two different lists may leave the first machine at the same state. If those lists leave the target at different states, the assignment would have to give two answers at that one state. It cannot.

That is the only obstruction. Suppose every pair of lists that agree at the first machine also agree at the target. To assign an answer to a reachable state, take any list reaching it and run that list on the target. Choosing another reaching list gives the same answer, by the supposition. Appending a move respects both machines' steps because it extends those very runs. The assignment is fixed on every reachable state, since each comes with a reaching list. This is the *descent* condition: the assignment can come down from histories to states precisely when the target does not distinguish histories the first machine has already identified.

A toll does not force that condition. Take a one-bit machine whose set costs one and whose reset costs zero. Setting its bit once leaves it up. Resetting and then setting also leaves it up. But a target that keeps the whole instruction list

distinguishes “set” from “reset, set.” Both lists reach the same bit state, and they reach different history states. The instruction-list assignment still works; the state assignment fails. That is where the word “initial” has to stop.

And the borrowed word has a home. I didn’t build it, so I’ll point at it. That lists of moves are the starting object, and that a machine’s behaviour factors through its reachable states exactly when a condition like descent holds, is the theory of minimal realization. Joseph Goguen put it as a universal property in 1972, in a paper whose title says it outright, “Realization is universal,” and Thomas Colcombet and Daniela Petrişan redid automata minimization in the same functorial terms in 2020. Their machines can have any output at all, which is the honest point: the fold and the descent condition never use the record, so the toll is doing nothing here, just as I said.

The nearest classical parallel to “only one honest count” is in thermodynamics. The mathematical physicists Elliott Lieb and Jakob Yngvason showed in 1999 that once you fix which states of a system can be reached from which by adiabatic processes, and that relation meets a short list of axioms, there is an entropy, a function that goes up exactly along the relation, and it is unique once its zero and its unit are chosen. Their uniqueness, like this one, comes after the fixing: the order is the input there, the price list is the input here, and in both the count is forced by it.

7.5 The toll is an old trick with a new name

Now for something I first took as a let-down. It would be nice to have a new kind of cost argument, and this isn’t one. The toll is a case of a technique that algorithm people have used since 1985 and that a mathematician studying the stability of motion had, in a different form, in 1892. I’m telling you it’s old so you don’t think I invented it.

The trick is called the potential method, and it works like this. Give every state a number, call it the potential, that measures how much “owing” is stored up in that state. Then look at each step and ask: does the cost of the step, plus the change in potential it causes, ever come out negative? If it never does, you’ve got a bound: any run costs at least the amount the potential *dropped* from start to finish, because the drop had to be paid for by the steps along the way. The computer scientist Robert Tarjan set it out in 1985, crediting the potential view to Daniel Sleator. Tarjan’s use goes the other way (potential as prepaid credit, to bound a run from above): he used it to analyse data structures that are occasionally expensive and usually cheap; and the decreasing-function idea underneath it is the one the Russian mathematician Aleksandr Lyapunov used in 1892 to prove that a system settles down. Do the coin version, which runs Tarjan’s way, as a ceiling: a stack of coins, PUSH adds one and costs one, POP-ALL removes them all and costs as many as it removed, potential

equals the number of coins. PUSH costs one plus one potential; POP-ALL of k coins costs k and drops the potential by k, net zero. So any sequence from an empty stack costs at most twice the number of pushes, however many expensive pops are in it. The whole method takes three lines.

Push on that 1892 for a second, because a date like that hides a lot. Lyapunov wrote the idea up as his doctoral thesis, in Russian. A French translation came out in 1907, made by a French naval engineer, Édouard Davaux, and corrected by Lyapunov himself. The English one came out in 1992, a hundred years after the original, and it was translated from the French: a translation of a translation. So for most of the twentieth century a reader with only English met the idea secondhand or not at all. Did the algorithm people of 1985 get it from him? I can't tell you, and nothing here needs them to have. Whether anybody walked from one to the other on purpose is a question I'd like answered. What I do have is a picture: the same idea, a number that only goes one way along a run, turned up in a thesis about whether a moving system settles down and in a paper about data structures, close to a century apart.

Now point it at the record. Give each place in the record's order a number that falls as the record climbs, so that a step that climbs drops the potential by at least one. For every step to have non-negative amortized cost, the step that climbs has to cost at least one, which is the toll; a potential like that proves the toll wherever it exists. On two places there's only one such potential, up to a constant: one if the flag is down, zero if it's up. A step from down to up drops it by one, so for the step's cost plus potential change to be non-negative, the step has to cost at least one. That requirement is the toll exactly, and on two places the two are the same statement: the toll holds for a machine if and only if every step has non-negative amortized cost under the flag potential ("amortized cost" is just that sum, the step's cost plus the change in potential). Then the universal floor from Section 2, "any run from NO to YES paid at least one," is just the potential method's standard conclusion, the drop in potential from one to zero, and every system with the five parts that obeys the toll is an instance, which is exactly what a Thiele machine is. Two more facts fall out of the same picture. Under the toll, a run raises the flag at most as many times as its total cost; each raise uses up a unit and a unit isn't used twice. And if you demand the upper-bound version too, that the potential before a step covers its cost plus the potential after, then the two together say every step's cost is exactly the potential it drops. Go through the four cases. Down to down and up to up drop nothing, so they cost zero. Down to up drops one, so it costs exactly one. Up to down would have to cost minus one, and no step costs less than nothing, so the flag never comes down. So the two together hold exactly when raising the flag costs exactly one, everything else costs zero, and the flag never comes down. Those prices, a cost of one per raise and nothing on any other step, are rise-only pricing in its tightest form,

and I'll call them the *exact schedule*. The flag staying up comes from this ceiling, not from the prices: a ceiling that counts raises instead, which Section 8 uses, leaves the same prices and lets the flag come down at cost zero. Notice that Section 2's two-move illustration doesn't meet it: its VOUCH move, played from YES, costs one unit although it raises nothing, which the floor allows and the exact schedule forbids. Part III's example (Section 15) doesn't meet it either, and can't; both are still Thiele machines, since the toll is all the model asks. The reason: its CHECK and COMMIT, the check and the commitment that must come before its flag goes up, raise no flag, and each sends two states with the same counters and line to one (a COMMIT lands an empty channel with one already naming its claim, and a CHECK that fails lands with the same state already trapped), so pricing the merges in the record's part of the state charges each of them one. It charges every instruction a fixed listed price, which is a different job for the word exact (instruction-exact pricing), and it breaks no rule. (These two facts hold even if the price depends on the state as well as the move's name, which is wider than Section 2's per-name cost; a per-name cost is a special case, and there the floor and the ceiling together are out of reach for any system whose flag ever goes up.) Two older tools sit beside these. Run a list of moves in a "writer," a wrapper that adds up a number as it goes, and you get back the final state and the run's cost; and a per-move schedule is a "grade," fixed before the run. I'm borrowing those two and no more: neither brings along the full type systems they come from. The writer is the move the programming-language researchers Norman Danner and Daniel Licata make in their cost semantics, a translation that carries a cost alongside a program's result (Danner, Licata and Ramyaa, ICFP 2015; Danner and Licata, *Journal of Functional Programming*, 2022). The grade is Shin-ya Katsumata's: a computation carries its resource as a grade, and the grades compose as the computations do ("Parametric effect monads and semantics of effect systems," POPL 2014). Dominic Orchard, Vilem Liepelt and Harley Eades put grades into the types of a working language ("Quantitative program reasoning with graded modal types," ICFP 2019). And the potential method went into types before any of those: the computer scientists Martin Hofmann and Steffen Jost (POPL 2003) gave a type system that attaches potential to data and predicts, before a first-order functional program runs, how much heap it can use, which is where type-based amortized analysis begins; the multivariate work of Jan Hoffmann, Klaus Aehlig and Hofmann (2011) carried it further. Read against them, the counter is a grade carried in the state by the step rule, and no type carries it. And what does that difference even mean? A cost that lives in a type gets checked before the program runs, by whoever checks the types; a cost that lives in the state gets written during the run, by the step itself. The number is the same, and the moment of truth is different.

So is mine an embedding of their frameworks, or something weaker? Half of it is exactly theirs, and the other half is left out on purpose. The counter *is* their writer:

a step that moves the rest of the state without looking at the counter, hands back its cost, and has the cost added on is a computation in the writer over whole numbers with addition, and running a list of moves in that writer gives back the final state and the total cost, nothing more and nothing less. Every system in this book whose step never reads its own counter has that shape. What doesn't come along is the part of their frameworks that works before anything runs: the type systems that carry grades, Danner and Licata's bounding relation and the recurrences they extract for a program's cost. That isn't a gap left to be filled later; it's the moment-of-truth difference I just named. Putting the count in the step is choosing the other moment, so the grade here is written by the step at run time and never checked ahead of it.

How close do the two moments come? Close enough to say exactly, in a limited form. Take a small structured language over a machine with this kind of counter: sequencing, branches whose two arms carry the same grade, and paid acts each graded with their charge, and grade every program the way Katsumata's discipline does, adding grades along the program. On loop-free programs the two agree on the nose: on every run that doesn't trap, the counter rises by exactly the grade. The argument is conservation set beside the grade rule; the counter's rise is the sum of the charges along the run, and the grade rule adds the same charges in the same order. Loops break the agreement, and in one direction only. A loop that runs a paid check once for every unit in some register, where the check always passes and is charged one, costs that register's value on entry, and no fixed whole number is an exact grade for it; an input-dependent grade can express its exact cost if the corresponding proof is supplied. Cost-aware type theories such as calf (Niu, Sterling, Grodin and Harper, 2022) support such reasoning; this is not an automatic exact-cost inference procedure for arbitrary loops. So the counter is their writer exactly, agrees with fixed grades wherever a fixed grade can be exact, and where a run's cost depends on its data it holds the exact number, which a fixed grade can at best bound, and can't bound at all when the data has no ceiling. What has no counterpart in the step is the checking ahead of time: a type checker turning a program away before it runs.

There's one more neighbour, closer than any of those, and I'll name it before somebody names it at me. A number the step keeps up to date and nothing else ever reads is what program-proof people call an *auxiliary variable*, or *ghost* state. Susan Owicki and David Gries added auxiliary variables to parallel programs in 1976, because some true facts about such a program can't be proved without one, under the rule that an auxiliary variable never steers the program. Jean-Christophe Filliâtre, Léon Gondelman and Andrei Paskevich, in 2014, worked out when ghost code can be erased without changing what the program computes. By that test this section's counter is ghost whenever no step consults it, and Section 11's base-driven records are built exactly that way: the base moves the same whatever the ledger

and the record say. So what does putting it in the step buy, if the number is ghost anyway? Two things, as far as I can see. The number is there while the machine runs, where a window can show it (Section 8), and not only inside a proof somebody wrote afterwards. The rule that keeps it covers every permitted instruction and therefore every run. A proof using ghost state can also establish such a machine-wide invariant, and static cost proofs can coexist with a runtime ledger. The difference here is that the defined runtime state includes the count. Whether a record stays ghost depends on what else the step does with it; the machine in Part III has a part that isn't ghost, its table of checked facts, because the step reads it and refuses on it, as the opening section said. One last relative, from the graph side: Section 2's floor is a cheapest-path bound in a system with weights on its moves. Every path from the no-states to the yes-states crosses the line between them, and every edge across that line weighs at least one. Weighted automata go back to Marcel-Paul Schützenberger in 1961, and priced timed automata, which add clocks, were set out in 2001. The floor is about the smallest case of what they compute.

Exactly, for any Thiele machine with states S, moves I, step, per-name cost, and reading:

Definition 7.2 (Potential, amortized cost)**.** A *potential* is a function $\Phi : S \to \mathbb{N}$. A step has *non-negative amortized cost* under Φ when $\mathrm{cost}(i) + \Phi(\mathrm{step}(s, i)) \geq \Phi(s)$. The *flag potential* is $\Phi_{\mathrm{cert}}(s) = 1$ if $\mathrm{cert}(s) =$ no and 0 if it reads yes.

□ **Theorem 7.4** (The toll is the potential method)**.** (1) The toll holds exactly when every step has non-negative amortized cost under Φ_{cert}. (2) For any Φ with non-negative amortized cost and any run t from s, $\mathrm{cost}(t) + \Phi(\mathrm{run}(t, s)) \geq \Phi(s)$. (3) So, under the toll, a run from a no-state to a yes-state costs at least $1 - 0 = 1$.

Proof. (1) Put the before and after flags side by side. Down-to-down and up-to-up leave the potential unchanged, so any nonnegative charge meets the inequality. Up-to-down adds one to the potential, so that case meets it too. Down-to-up removes one: only there must the charge supply at least one to make up the drop. That is exactly the toll.

(2) Write one inequality for each step: its charge plus the potential after it is at least the potential before it. Add the lines. Every interior potential appears once as an after and once as the next before, so those equal terms cancel. Only the starting potential, the final potential, and all the charges remain. Thus the charges plus the final potential cover the starting potential. For an empty run the start and finish agree and there are no charges, giving equality.

(3) The flag potential starts at one when the flag is down and ends at zero when it is up. The charges must cover that drop of one. ∎

So what's new, if the method isn't? Only where it's pointed. The potential method is a way of proving things about costs. What this book adds is a claim about *which* potential, one that falls as the record climbs, the flag on two places, and about what happens to that claim when you can only see a window of the state. The method is Tarjan's and Sleator's. Pointing it at the flag is mine, and I'm about to find out what that can and can't survive.

The flag potential has relatives in physics-flavoured mathematics too, which I found after the fact. Resource theories study what you can turn into what when some operations are free: Bob Coecke, Tobias Fritz and Robert Spekkens gave the general definition in 2016, Fritz worked out its algebra in 2017, and Tomáš Gonda and Spekkens catalogued the numbers that never go up under free operations, which they call monotones, in 2023. Read with the raised flag as the resource and one unit of cost as the currency, the flag potential is the kind of cost monotone they build, measured in that currency, and the floor is its simplest case. Does that make the toll a resource theory? Only as the smallest example of one: it says a run from no to yes costs at least one. To say yes costs exactly one you'd also need a run that gets there for one. It's a neighbour to know about, and I don't count it as a result.

7.6 What the counter is, said once

A receipt is a total that lives in the state, only climbs, equals the sum of the charges on every path including the failures, and is the only such total, on the states a run from the start reaches, for its schedule. Each of those had a home before this book: the stick for a total that lives in the state, Lamport's clocks for climbing, Turing's and Floyd's ranking functions for the same one-way rule run downward, the telescoping sum for conservation, the induction for the single honest count. None of them is physics, and none of them says what the charges should be. What they say is that once you've chosen the charges, the counter is the arithmetic and not a second opinion. And the toll on top of it is the potential method with the flag as potential, which is to say a trick from 1985 applied to the question from Section 6.

There's one thing the counter can't do just by existing: if you can only see *part* of the state, can you tell what the counter is? Can you even tell, from outside, which step paid? The stick works because you can lay the two halves together. What if you're only ever shown one half, through a window, and the window doesn't show the notches?

One piece of that has a short answer, and I'll give it now so the next section can start from it. A window, for now, is just the part of the state an outside observer is shown (Section 5's display was one, and Section 8 is about windows in general).

Call a window *counter-complete* when some rule, applied to what the window shows, returns the counter on every state; *flag-complete* likewise for the flag. Say a window *forgets the counter* when changing the counter and nothing else never changes what the window shows, and likewise for the flag.

For a full product state in which the counter and flag can be set independently, a window that forgets either field cannot recover that field on all states. Take a state and make two copies of it, one with the counter set to 1 and one with it set to 2, everything else held still. The window shows the same thing for both, so any rule gives the same answer for both, and one of the two answers is wrong. The flag is the same argument with yes and no. On the stick, a window that shows you the wood's colour and its length, and hides the notches, can't tell you what's owed, and staring harder at the colour won't help. (The argument builds its second state by hand, so it may be one no run reaches, and counter-complete asks for every state. On the states a run reaches, forgetting isn't enough: a machine whose one paid move also moves a position the window shows reads its counter off that position everywhere it goes. What blinds a window on reached states is two runs that end on the same view with different counters. The machine I build later has such a pair, written out move by move: two runs with the same window, one paid and one not (Theorem 15.15).)

Be careful what that says. Neither part can be dropped, and nothing else is needed. Call one window bigger than another when what it shows decides what the other shows. A window that gives back both the counter and the flag has a rule for each, so what it shows decides both, and the window that shows those two and nothing more is the smallest that works. And it's about reading the counter back. Which step paid is the harder question, and it's the next section's.

CHAPTER 8

Windows and shadows

Every state a machine keeps is a distinction, and a look from outside can show some of them and drop the rest. I'll call that look a window. I left you with the stick split in half and asked what happens if you can only ever see one half through a window that doesn't show the notches, and with a harder question under that one: can you tell, from outside, which step paid? This section is about windows. It's a section where the mathematics is so small that I'm going to state it in full and then spend most of the pages on what it's *not*, because the smallness is what catches people: the fact is a one-liner and it's tempting to climb over it with cleverness, and you can't, and seeing why you can't is the content.

8.1 A cave, and the people who got good at it

This one is Plato's, and the part I need is near the end. In the seventh book of the *Republic*, Plato has Socrates, the main speaker of the dialogue, describe prisoners chained in a cave since childhood, facing a wall, with a fire behind them and people carrying objects along a walkway between the fire and their backs. All the prisoners have ever seen is the shadows on the wall. To them, Socrates says, the truth would be nothing but those shadows. Further on, Socrates reads the story out himself: the cave is the world of sight, and the climb out of it is the soul's climb toward what's really there. So a real world hidden from anyone who only looks is in the story on purpose, and Plato put it there. It still isn't the part I need.

What I need comes a little before that reading, when one prisoner, freed and taken up into the daylight, thinks back to the cave. The prisoners, Socrates says, hand out honours to whoever is quickest at spotting the shadows as they pass and best at guessing which comes next. The part worth reading twice is that the prisoners *get good at it*. They build a whole science of shadows, with experts and prizes, and it works, on shadows. What they can't do is say anything about a carried object that leaves no mark on the wall, and that isn't because they're stupid; it's because the information isn't on the wall. Skill inside a window doesn't reach past the window, no matter how much skill. Plato dressed that fact as a lesson about

education, and it is one, but take the dressing off and it's a sentence about functions, and it's going to be the whole section.

8.2 The shadow of a cube

Do this one tonight with a lamp and a flat wall. Keep the lamp well back, hold a cube face-on with its face parallel to the wall, and mark the outline of its shadow. Replace it with square card and adjust the card's size or distance until the outlines match as closely as you can judge. Show your partner only that outline. It does not tell them whether the object has thickness. For the exact mathematical version, take parallel light rays and let the window report only the silhouette: an aligned cube and a square face of the same size then give exactly the same square. Ask them, and they'll tell you, truthfully, that they can't know, and if they're honest they'll also tell you that no amount of staring at that square will change it.

Now turn each a quarter turn about its vertical centre line. The card's shadow becomes a narrow edge; the cube still casts a broad face. With ideal parallel rays and zero-thickness card, those are exactly a line and a square. Now the wall-watcher knows, and what changed isn't their cleverness but the *window*. A second look from a different angle is a bigger window than one look, and what bought it was the turn. Edwin Abbott wrote a novel about a window like this in 1884, *Flatland*. A Flatlander never sees a shape. Abbott's narrator says it in his first section: nothing was visible to them, or could be, except straight lines. Put a coin flat on a table and lower your eye to the table's edge, and the coin turns into a line, which is all the view there is. What gives a Flatlander anything more is fog. A line that's near is bright and a line that runs off into the fog goes dim, so a trained eye can read a triangle or a pentagon off how fast the brightness fades toward the ends. The training takes time and money, and in the story only people with money get it. The window got bigger, and it was paid for in time.

Then a sphere comes through the narrator's plane. What his eye gets is a line whose ends fade too fast for any flat figure he knows, changing size in a way no flat figure could. He only settles that it's a circle by feeling it, and the circle grows from a point to its widest and shrinks back to nothing as the sphere rises through. From one slice he can't put a sphere together. From the slices one after another, seen and felt and argued over, he starts to. The escape from a window is never "think harder." It's always "a different window," and a different window that answers the question always has to show apart two states the old one showed alike; that's Theorem 8.1 below, applied to the new window. In every story here, getting that window cost something: a turn, a second look, training, time. What a change of window costs in general is a separate question, and no theorem in this section prices it.

Who was Edwin Abbott? Not a mathematician by trade. He was headmaster of the City of London School for twenty-four years, a clergyman, and the author of more than forty books, a Shakespeare grammar among them. He put *Flatland* out under the name "A Square." His full name was Edwin Abbott Abbott, two A's, and the pen name may be a joke on that, A squared; nice if true, but take it as a lead. The part I like best comes from a sketch of his life on the geometer Thomas Banchoff's pages: *Flatland*, by far the most read thing he wrote, isn't mentioned at all in his obituary in the London *Times* or in his entry in the *Dictionary of National Biography*. The official record of the man was a window too: it showed the headmaster and the grammarian and left out the one book people still read. That changes nothing in the argument; it's the argument, acted out by an obituary. (*Flatland* comes back later: Section 14 retells the sphere's visit, and the Afterword, Section 24, says how the hunch behind this book started with it.)

8.3 The whole fact, in one paragraph

I'll give the mathematics now, and I promise it's all of it. You have a set of states, the real things. You have a window, which is a function that takes a state and gives you what you can see of it: the shadow, the slice, the half-stick without notches. And you have a question about the state, which is also a function: it takes a state and gives an answer. The thing you want is a *decoder*: a rule that takes what you can see and gives the right answer to the question, every time. When does one exist?

Think about two states that cast the same shadow. The decoder sees the same shadow for both, so it gives the same answer for both. So if the question has *different* answers on those two states, the decoder is wrong on at least one of them, and there's nothing it can do about it, because it never saw the difference. That argument is the whole fact. If a decoder exists, then the question must already give the same answer on every pair of states that look the same through the window. Mathematicians call the set of states that cast one particular shadow the *fibre* over that shadow (think of the shadow as a point on the wall and the fibre as the bundle of all the things that could have cast it), and the sentence is: a decoder exists only if the question is constant on every fibre. The theorem just below says that, and the "only if" is the direction that bites. The other direction, "if the question is constant on fibres then a decoder exists," is true too: send each shadow to the answer at any state that casts it, and constancy makes it not matter which one. A shadow nothing casts can get any answer, since no state shows it. But a decoder existing isn't the same as being able to work it out: to compute its answer for a shadow you still have to find a state that casts it, and that search takes effort, and on some windows it can't be done at all.

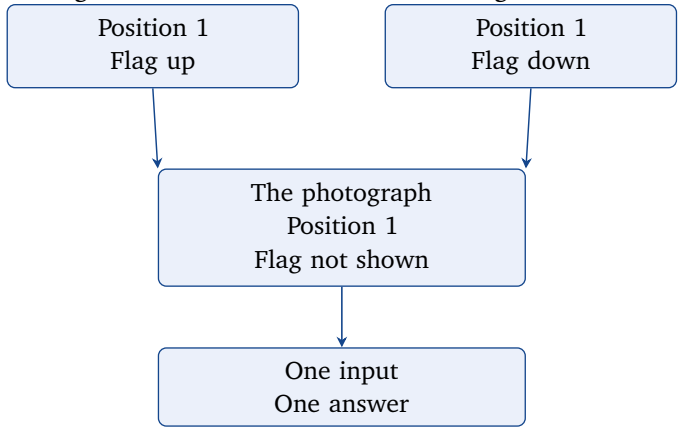
To say it exactly, let S be any set of states, V any set of views, $w : S \to V$ a window, and $q : S \to A$ a question with answers in any set A. A *decoder* is a function $d : V \to A$ with $d(w(s)) = q(s)$ for every state s. The question is *constant on fibres* when $w(s) = w(s')$ implies $q(s) = q(s')$.

□ **Theorem 8.1** (Decoders need constant fibres)**.** If a decoder exists, the question is constant on fibres. Conversely, if the question is constant on fibres and there is at least one answer, a decoder exists: send each view that some state shows to the answer at any state showing it, and any other view to any answer at all.

Proof. If $w(s) = w(s')$ then $q(s) = d(w(s)) = d(w(s')) = q(s')$. Conversely, the rule is well defined, because two states showing the same view have the same answer by constancy, so it doesn't matter which one is picked; and at any state s it ■ gives the answer at a state showing $w(s)$, which is $q(s)$.

Nothing in the theorem cares what the states or the views are.

Try the smallest case, by hand, because it'll keep coming back. A state is a pair: a position, 0 or 1, and a hidden flag, up or down. The window shows the position only. Two moves from (0, down): one sets the position to 1 and raises the flag; the other sets the position to 1 and leaves the flag alone. Both land on position 1. Now write down every possible decoder from "what I see" to "is the flag up": there are exactly four functions from two positions to two answers. Check each against both results. None of the four gets both right, because both results show position 1 and one has the flag up and one doesn't. That exhaustive check is the theorem on this example; the general statement is the one-line argument above, and that's the one that carries the weight. There's no fifth function hiding somewhere.



Answer yes and the bottom state is misread. Answer no and the top state is misread. The photograph needs to carry another distinction.

8.4 The clerk with two photographs

Now point it at the counter, because that's why this section is here. Suppose you want to charge for raising the flag, exactly: at least one unit for every step that raises it, and nothing for a step that doesn't. And suppose the person doing the charging is a clerk who sees only a *window* on the state, a photograph before the step and a photograph after. The clerk's charge is a function of the two photographs. Can the clerk charge exactly?

The clerk can't if the window has the wrong kind of blind spot, and the reason is the cube again. Suppose there are two steps that give the clerk the same before-photo and the same after-photo, and one of them raised the flag and the other didn't. (The two-move toy above is exactly such a pair; so are "a stamp goes onto a document inside a closed drawer" and "someone opens and closes the drawer.") The clerk has one charge for that pair of photographs. If the charge is at least one, it overcharges the step that didn't raise the flag. If it's zero, it undercharges the step that did. Any charge that never misses a raise charges something that wasn't a raise, and no charge from those photographs is exact. The fix is the plain one: put the flag in the photograph. If the window shows the reading, then "one when the reading goes from down to up, zero otherwise" is exact, and such a window has no pair like the one above. Showing the flag is enough. It isn't the only way: a window can hide the flag and still have no colliding pair, and then a charge from the photographs is exact, as the observation below says. And "exact" here means rise-only pricing: at least one on every step that raises the flag, nothing on any step that doesn't. That's all the clerk's problem needs, and the charge "one for a raise, zero otherwise" meets it. So it's exact about *which* steps get charged. It isn't exact about *how much*, and it doesn't pick out one charge: "two for a raise, zero otherwise" meets it too. Pinning the charge down takes one more demand, a ceiling: over any run, the total never exceeds the number of raises. With the floor and that ceiling there's exactly one charge left, one per raise and zero otherwise, which is the exact schedule from the last section; that's an if and only if for any system whose charge is decided step by step, from the state and the move: apply the ceiling to a one-step run. These charges look at where the step starts, not only at the move's name, and they have to. A move that raises the flag from NO raises nothing when it's played again from YES, so with Section 2's price, one price per move name wherever you stand, nothing priced that way charges the raise and leaves the replay alone, and no system whose flag ever goes up meets the floor and the ceiling together. The clerk's charge, read off the photographs, depends on the state from the start. All of this holds for any states and any steps. The window version, the cube theorem with a price in place of a decoder, comes next.

Same clerk, now in symbols, so you can check every word I just used. Take the five parts of Section 2 with a window $w : S \to V$, and say a step (s, i) *raises* the flag when $\mathrm{cert}(s) = \text{no}$ and $\mathrm{cert}(\mathrm{step}(s, i)) = \text{yes}$. A *price* is any function that gives a state and a move a whole number of units, zero or more (wider than Section 2's per-name cost, which is the case that ignores the state). It *meets the floor* when every raise costs at least one, and *never overcharges* when every non-raise costs zero; together, it is *exact*, which is rise-only pricing (exact about which steps are charged; the amount is pinned only by the whole-run ceiling above, and pinned it is the exact schedule). A *window price* is one computed from the two photographs only: $\mathrm{price}(s, i) = b(w(s), w(\mathrm{step}(s, i)))$ for some charge $b : V \times V \to \mathbb{N}$. A *collision* is two steps (s_1, i_1) and (s_2, i_2) with $w(s_1) = w(s_2)$ and $w(\mathrm{step}(s_1, i_1)) = w(\mathrm{step}(s_2, i_2))$, the first a raise and the second not.

Observation 8.2 (No exact window price across a collision)**.** *If the window has a collision, every window price that meets the floor charges at least one on some step that is not a raise; so no window price is exact. If it has none, the window price "one on any pair of photographs some raise casts, zero otherwise" is exact; in particular when the reading is a function of the view,* $\mathrm{cert}(s) = r(w(s))$ *for some r, there is no collision.*

Proof. Across a collision both steps get the same charge $b(v, v')$. The floor makes it at least one because the first step is a raise; the second step, not a raise, pays it. With no collision, a step that casts a pair some raise casts is itself a raise, so that price charges one on every raise and nothing on any other step. If the reading is r of the view, two steps with the same photographs either both raise or both don't, so there's no collision. ∎

So the exact price of raising the flag lives in the step itself, or in a window with no colliding pair, and a window that shows the flag is the plainest such window. A window that contains a pair like that one can't carry it. Whether a particular machine has such a pair, for a particular window, has to be checked machine by machine and window by window; the observation tells you a window prices exactly just when it has no such pair.

You can also find the best cautious prices the clerk could use. Put all the steps that give one pair of photographs in a tray. For this calculation restrict the clerk to zero-or-one charges: each step ought to cost one if it raises the flag and zero otherwise. Without that restriction an all-raising tray would allow charges of two, three and higher, and there would be no largest price that avoids charging non-raises.

To miss no raise, the clerk must charge one whenever the tray contains even one raising step. That is the cheapest price that never undercharges. To charge no non-raise, the clerk must charge zero whenever the tray contains even one non-raising step. That leaves one only when every step in the tray raises the flag, and it is the largest price that never overcharges. In a mixed tray the two demands disagree. In

a tray of all raises, or all non-raises, they agree. That is the collision test again, now giving both best cautious prices.

Steps with these photographs	Miss no raise	Charge no non-raise
All raise	1	1
None raises	0	0
Some raise, some do not	1	0

The entries are the best cautious zero-or-one charges in each direction. The mixed tray is where the two requirements disagree.

The two tests are “some member” and “every member” applied to the same tray. F. William Lawvere studied this movement of logical questions along maps in “Adjointness in foundations” (1969). Robin Cockett, Isabelle Geng, Carlo Maria Scandolo and Priyaa Srinivasan (2023) use the same kind of construction to extend resource measures along a map. Here you can see the whole calculation in the tray: take its highest required charge or its lowest. Exact pricing is possible precisely when those two numbers agree for every attainable pair of photographs.

Control engineers have been hunting pairs like that for a long time, under other names. Discrete-event systems is the field of machines that jump from state to state on events, some of which an outside observer sees and some of which it doesn’t. In 1995 five engineers, Meera Sampath, Raja Sengupta, Stéphane Lafortune and Demosthenis Teneketzis of the University of Michigan and Kasim Sinnamohideen of Johnson Controls, asked when such an observer can always work out, from what it sees, that a hidden fault event has happened, within a bounded number of further steps. They called that *diagnosability* and gave a test for it on finite machines. Anooshiravan Saboori, then a doctoral student at the University of Illinois, and Christoforos Hadjicostis, the engineer who supervised his doctorate there, asked in 2007 the question the other way round, for secrets: a system is *opaque* when an observer can never be sure it’s in a secret state, because some run outside the secret always looks the same. Read the raise as the hidden event and the window as what the observer sees. An ambiguous pair of current views prevents an immediate answer from that view alone; a retained observation history might still distinguish the runs. One ambiguous pair is enough to defeat a decoder of the current view that must always answer correctly. Opacity asks more: every secret run in its scope must have a non-secret alternative with the same observation. The small machine’s no-oracle theorem below supplies an ambiguous pair; it does not by itself establish that stronger claim about all secret runs. That’s my reading; they don’t put it that way. Their versions are richer than mine: they allow delays, look at whole runs, and come with algorithms. What this section adds is only the price: what a charge computed through the window does across one such pair.

Why can't the clerk just watch the step itself? A clerk who does, who sees the whole state before and the move, charges exactly with no trouble at all: "one for a raise" is a function of exactly those two things, and it's the step price above. So the collision bites an observer who doesn't see the step, or who's handed an account of the step and can't trust the account. That second kind is the trace problem, which Section 14 takes up. If the names of the moves arrive in a log the program wrote, the program can write whatever names it likes, and a clerk reading that log is reading one more window, one the program painted. Seeing the move means being underneath the program, where the step is.

8.5 The three escapes, and what each one costs

Suppose you're the person outside, and you want to be convinced that a run paid. The window you've got doesn't show the counter. To see how bad that is, I'll take the smallest setting I can build. Take a counter machine from Section 7, give its states a position and a flag as well, and give it a window that shows the position and nothing else. Now take two runs from the same start, with the position at 0, the flag down and the counter at 0. Run A makes one move that raises the flag, costs one, and lands at position 1; its counter reads 1. Run B makes one move that does nothing but step to position 1, at cost zero; its counter reads 0. Call the pair of photographs, position 0 then position 1, the *transcript*. Both runs leave exactly that transcript. A *verifier* is a rule that takes a transcript and says accept or reject. It is *sound* when every transcript it accepts comes only from states where the claim holds, and *complete* when every transcript from a state where the claim holds is accepted. The claim is "the counter is 1."

No verifier on these transcripts is both, and the proof is three lines. Suppose one were both. Run A ends with the counter at 1 and leaves the transcript, so completeness says the verifier accepts it. Now soundness, on that same accepted transcript: every state that leaves it satisfies the claim, and run B's end state leaves it. So run B's counter is 1, and it is 0. The cube comes back here, with a verdict in place of an answer: a verifier sees only what the window kept, and two states that the window can't tell apart get one verdict between them.

Now dress the transcript up. Take any richer kind of transcript, with a way to squash one down to its bare photographs, and say a verifier on the rich ones *only looks at the bare part* when two rich transcripts that squash to the same photographs always get the same verdict. If some rich transcript left by run A and some rich transcript left by run B squash to the same photographs, then no verifier that only looks at the bare part is sound and complete either: it gives those two the same verdict, completeness makes that verdict accept, and soundness then lies about run B. Notice what that needs: the pair has to survive into the richer kind of transcript. A

design whose extra part tells *A* from *B* doesn't meet the premise, and the argument says nothing about it. So it doesn't classify every transcript design. What it says is that a verifier that only reads what the bare window keeps fails on that pair.

Economists got to the same wall by another road. Jerry Green and Jean-Jacques Laffont (1986) asked what a mechanism can do when people can claim about themselves only what their situation lets them back up, and Jesse Bull and Joel Watson (2007) built that into mechanism design as *hard evidence*. In their terms, an audit can tell apart only the cases the evidence on offer tells apart, which is the cube again, set in accounting.

What can you do? I know three things that work, and nothing says they're the only ones; as far as I can tell, cryptography uses all three. What any of them needs is the cube's own condition: no evidence the outsider can be handed may come both from a state where the claim holds and from one where it doesn't. I'll call them escapes, and be clear what from: from the window, not from the cost. Run *A* paid its one unit of cost whatever anybody outside can see. An escape is a way for the outsider to see that it did. Each rests on something you have to trust, and I'll say what, because "a price" in the counter's sense is not what any of them costs.

You can be shown the *whole state*. Then there's nothing to decode; you read the counter off the state. The price is trust: you have to believe the state you were shown is the state there was.

You can be shown a *commitment*: the ordinary window plus one extra bit that is set when, and only when, the claim holds (here, that the counter reads exactly one). Reading that bit gives a verifier that is sound and complete for that claim exactly when the bit is binding (set only when the claim holds) and honest (set whenever it holds). The trust is in those two conditions, which are a contract about who can set the bit. The contract can be met, and a bit anyone can set breaks it, which is the one-line version of why an unsigned log is just another window. Nothing in that picture has a hardness assumption in it; whether a real commitment scheme meets the contract is a question for cryptography, and this book doesn't take it up. (A clash of words I should own: "commitment" here is the cryptographers' word, something bound to a claim, which in their schemes also hides the claim until it's opened, and this bit doesn't bother hiding anything. The book's commit (Section 14) is a different act, the closing of a question. It's the same word doing two jobs. The sealed envelope, a commitment that hides its claim, has an exact theory of its own. Three information theorists, Andreas Winter, then at Bristol, and Anderson Nascimento and Hideki Imai, then at Tokyo, found in 2003 the best rate at which a noisy channel can carry commitments, and the cryptographers Claude Crépeau, Rafael Dowsley and Nascimento (2020) worked out how that rate drops as the committer gets some control over the noise, which is "take the pen away from the program," Section 14's fix for a log a program writes about itself, with a formula

on it.) In 1999 the cryptographers Bruce Schneier and John Kelsey worked out how to make a log that an intruder who takes over the machine can't quietly rewrite, and the honest scope in their paper is the honest scope here: it protects what was written *before* the break-in, through keys, and nothing after. Two cryptographers at the University of California, San Diego, Mihir Bellare and Bennet Yee, had named that property *forward integrity* in a 1997 report and proved their scheme has it. Point it at an AI agent that keeps its own log of what it did, and the line moves to the very start: an agent that writes the wrong thing from its first step is on the far side of the break-in for the whole log. A 2026 preprint on tamper-evident audit trails for such agents, built from hashes anchored on a public blockchain, is even called Agent Flight Recorder (by Laurent Bindschaedler, of the Max Planck Institute for Software Systems, and Quentin Botha and Christoph Siebenbrunner, of the Vienna University of Economics and Business), which is a word Section 14 uses too, and that section's fingerprint paragraph is the part of this book that applies to it.

Or you can *ask*, which is interaction: the outside party gets to put a question and gets an answer that pins the thing down. The price is the honesty of the answerer; a liar's answer is another shadow. Moore's experimenter chose inputs; the zero-knowledge proofs of the cryptographers Shafi Goldwasser, Silvio Micali and Charles Rackoff in 1985 are the refined form, where a verifier is convinced by a conversation designed to cut exactly the line between true and false and nothing else, with a mathematical guarantee against a cheating prover. I mention zero-knowledge to say what I'm *not* claiming: the asking in this section is one round, with one fixed question ("tell me your counter"), and nothing in it models a cheating prover; the honesty is assumed, and nothing enforces it. My one round of asking is a toy of the real thing, and I'm not going to let the real thing's reputation stand behind the toy. (Joseph Halpern, Yoram Moses and Mark Tuttle, computer scientists who work on reasoning about knowledge in distributed systems, gave zero knowledge a knowledge-based analysis in 1988, which is where I'd start for the logic of it.)

The real thing even has a cave in it, which after Plato I can't resist. In a paper for the 1989 CRYPTO conference called "How to Explain Zero-Knowledge Protocols to Your Children," two cryptographers, Jean-Jacques Quisquater of Philips Research and Louis Guillou of the French telecommunications lab CCETT, with their families and Tom Berson of Anagram Laboratories on the byline, told it as Ali Baba's cave: a ring-shaped tunnel with a magic door at the back that opens only for a secret word. Peggy goes in down one side while Victor waits outside; then Victor walks to the fork and shouts which side she should come out of. If she knows the word, she always can. If she doesn't, she gets away with it only when Victor happens to call the side she went in. Play it at the table with a coin for Victor's shout: one round, a cheat gets through half the time; twenty rounds, about one time in a million. The part of the story that belongs in this section is that Victor ends up convinced, but a

video of the whole thing convinces nobody else, because a staged run, where the two agreed beforehand which sides he'd shout, makes exactly the same recording. The recording is a window, and the real run and the fake one cast the same shadow. That's the cube in the cryptographers' favourite bedtime story, The cave protocol gets its bound from fresh, independent coin tosses made after Peggy has entered. That timing is what the staged video lacks. Other zero-knowledge protocols use different setups, including noninteractive ones; the cave does not prove that every proof must be a live conversation.

Each escape is a window with something added: the whole state, a bound bit, an honest answer. None of them is "a smarter verifier on the old window": cleverness sits on the wrong side of the projection, and what's true of any machine is the cube.

I'll write the three constructions out exactly, for the pair above, with the relations written down. The relation matters: it says which state a transcript is allowed to describe. If it quietly assumes a truthful answer, then a verifier sound for it is sound only against honest answerers, and it says nothing about a liar. Write A and B for the states runs A and B end at, μ for the counter, and τ for their shared transcript.

The letter E below names a rule about which state may explain which transcript. Think of writing an allowed state beside each piece of evidence. $E(s,t)$ says that state s is allowed beside transcript t. The little $\wedge$ means both conditions must hold; $\vee$ means either is enough; $\iff$ means each side holds exactly when the other does. In the bare case, the only evidence is τ, and either endpoint may stand beside it. The three richer cases change that pairing rule, which is where their extra guarantee enters.

Definition 8.1 (Three richer transcripts and their explanations)**.** Keep the states A, B and their shared bare transcript τ, with $\mu(A) = 1$ and $\mu(B) = 0$. Write $E_0(s,t)$ for $t = \tau \ \wedge \ (s = A \ \vee \ s = B)$. The claim is $\mu(s) = 1$.

A *full-state transcript* is a finite list t of states. Define $E_S(s,t)$ to mean that t is nonempty and its last entry is s. The verifier rejects the empty list; otherwise it accepts exactly when the last entry's counter is 1.

A *commitment transcript* is a pair (t,b) of a bare transcript and a Boolean. For any chosen explanation relation E_C, the *commitment contract* is the pair of implications, for all s, t, b:

$$E_C(s,(t,b)) \wedge b = \text{true} \implies \mu(s) = 1,$$

$$E_C(s,(t,b)) \wedge \mu(s) = 1 \implies b = \text{true}.$$

These are binding and honesty, respectively. The verifier accepts exactly when b is true. One relation meeting the contract is $E_H(s,(t,b)) \iff E_0(s,t) \wedge (b = \text{true} \iff \mu(s) = 1)$.

A *response transcript* is a pair (t, r) of a bare transcript and a natural number. Define $E_I(s, (t, r)) \iff \mu(s) = r$. The verifier accepts exactly when $r = 1$. This relation imposes no condition on the bare component t; the response alone carries its entire truth condition.

Each construction assigns verification the constant abstract cost 1.

□ **Theorem 8.3** (Three conditional escapes)**.** The full-state verifier is sound and complete relative to E_S. For every E_C, the commitment verifier is sound and complete relative to E_C exactly when E_C satisfies the commitment contract. The response verifier is sound and complete relative to E_I. Each has cost 1 under the stipulated cost assignment. The honest relation E_H satisfies the commitment contract; the unchecked relation $E_U(s, (t, b)) \iff E_0(s, t)$ does not.

Proof. For the full-state verifier, an explaining state is precisely the last entry. Reading that entry's counter accepts exactly when the explaining state's counter is 1. An empty transcript is rejected and has no explaining state.

For the commitment verifier, acceptance says b is true. Binding then gives the claim for every explaining state, which is soundness. Conversely, honesty says that any explaining state satisfying the claim has a true bit, which the verifier accepts. Read backwards, soundness for an accepted transcript is binding and completeness is honesty. The equivalence in E_H gives both implications directly. Under E_U, however, B explains (τ, true) even though $\mu(B) = 0$. That violates binding.

For the response verifier, explanation gives $\mu(s) = r$. Acceptance adds $r = 1$, so $\mu(s) = 1$. Conversely, $\mu(s) = 1$ and explanation give $r = 1$, so it accepts. The three cost claims follow from the assigned constant. ■

None of that leans on what A and B are. It uses only that one has the claim, one doesn't, and they share a bare transcript, so it holds for any pair like them, whatever the states, claims, transcripts and explanation relations. Once two transcripts with the same bare part are explained, one by a state with the claim and one by a state without it, a verifier that's sound and complete for the claim can't be one that looks only at the bare part. The three escapes get past that by adding full states, a bit that keeps the contract, or a truthful answer.

On the small machine of Part III (Section 15) the collision is a theorem too, and it doesn't depend on the two-move illustration. On that machine computing costs nothing and never touches the record, and from a clean start the record goes up only on an earned check, commitment and certificate. Give a run from a loaded start (Part III's word for a start at line 1 with the two counters set to a and b, written $(1, a, b)$) the *bare transcript* made of a, b and the window on the counters the run

ends with, and say a state *explains* a bare transcript when some run from that start ends at it showing that window.

Theorem 8.4 (No bare verifier on the small machine)**.** On the small machine: (1) No function from bare transcripts to Booleans is both sound and complete, under that explanation relation, for the claim "the record is up." (2) There is a clean loaded state such that no function from bare transcripts to Booleans is both sound and complete for the claim "the ledger is at least 3 above what it was at that start." (3) For the small machine of Part III, whose moves CHECK, COMMIT and CERTIFY and whose counter move DEC are defined there: a run CHECK *is zero* A, COMMIT *is zero* A, CERTIFY from the loaded start $(1, 0, 0)$, which ends with the ledger at 3, and three DECs of a zero counter from the same start, which end with the ledger at 0, end with the same window, and no function on bare transcripts is sound and complete for "the ledger reads 3." □

Proof. Theorem 15.15 gives a clean loaded state and two runs from it that end with the same window, one with the record up and one with it down, one whose ledger rose by at least 3 and one made of counter moves at no cost that leaves the ledger where it started. Both end states explain the same bare transcript. A verifier complete for the claim accepts that transcript, since a state where the claim holds explains it; a verifier sound for the claim then makes the claim true of every state that explains it, including the one where it's false. That's (1), and (2) is the same with the ledger: the paid run's end state satisfies the claim, and the unpaid run's doesn't. For (3) the two runs are written out in the statement: three moves land on line 4 in both runs with both counters at 0, and the same two-state argument applies. ■

Parts (1) and (2), with a loaded state for a clean one and "at least 1" for "at least 3", hold of any machine whose flat moves cost nothing, can reach every window and can't touch a record that some run raises, by the same argument; certification is what raises the gap to three.

Put with the three escapes, the whole picture holds for the small machine: the bare-transcript verifier doesn't exist; the collision survives into any richer transcript that keeps it, so no verifier that only reads its bare part works either; a verifier on full states exists; one on the bare transcript with a committed bit exists, sound and complete when the bit is honest, and the unchecked bit breaks the contract; and one on a truthful response exists.

In the three escapes the collision is gone, and no trick did it: the extra information pulls A and B apart. The full-state transcript shows the counter outright. The honest commitment for A carries a true bit and the one for B a false bit. The honest

responses are 1 and 0. In none of the three is the verifier asked to tell identical inputs apart, which was the impossible job.

Now for the limit, which I'd rather say before you find it. A list of states has no authentication built into it, so nothing in it stops a liar from handing over whatever the verifier wants to see. The commitment contract is assumed, and the unchecked bit above is what happens when the assumption is dropped. The response relation only allows a truthful counter value, so asking a question here does not buy resistance to a cheating prover. And the cost 1 is an abstract price I wrote down. It doesn't measure what it takes to authenticate a state, check a cryptographic proof or send a response. What these three show is which extra distinctions a verifier can use. They are not an adversary model, a security parameter or a hardness reduction.

So what would it take to make that commitment bit binding for real, against somebody who wants to cheat? Cryptographers had to face that one in the open, and the place I'd point is a short paper the computer scientist Manuel Blum, whose Turing Award was for complexity theory and its use in cryptography, presented at the CRYPTO meeting in 1981, "Coin Flipping by Telephone." His setup: Alice and Bob have just divorced, they live in different cities, and they want to flip a coin over the phone for who gets the car, with nobody they both trust to do the flipping. If Alice flips and announces, Bob has to take her word. If Bob calls it first, Alice can lie about how it landed. Blum's way out is for one side to commit first: send something that fixes the choice without showing it, and open it only after the other side has called. And what keeps that commitment binding isn't anybody's promise. It's a bet that a certain kind of big number is hard to factor. Push on that and you're standing on the honest edge of this subsection. In Blum's protocol, binding is earned, on the condition that factoring is hard. Here it's written into the contract and assumed outright, and the unchecked bit above shows what's left without it. That's why I won't say these escapes beat a cheater. Blum's paper is the kind of thing it would take, and nothing like it is in this chapter.

Now point the three escapes back at the record I've been building, because that's where they bite hardest. A record kept in a machine's state is bits too. Hand someone outside a copy of those bits with a note saying "the flag is up," and they're reading a window: the run that really raised the flag and a box built to print that it did cast the same shadow. So putting the record in the step answers "was this earned?" for whoever can see the state and trusts the step that wrote it. For anybody else it moves the question, from "did it run?" to "do I trust the box that says it ran?" Crossing that line takes an escape, a window with more in it, and the three I have are these: the whole state, from a box you already trust; a bit backed by an authenticated and trusted source. A signature authenticates who signed the record; it does not prove that the signer told the truth. A quote from measured hardware likewise needs its connection to the claim (and Section 17 shows a quote vouches

only for what was measured); or asking, against a box that might lie, which is the cryptographers' business. One more relative deserves its name here, *verifiable computation*: the box hands back its answer with a short proof that the answer is right, which a checker can verify without redoing the work (Pinocchio, built in 2013 by the computer scientists Bryan Parno, Jon Howell, Craig Gentry and Mariana Raykova), and proof-carrying data passes proofs like that along a whole chain of parties who don't trust each other (the computer scientists Alessandro Chiesa and Eran Tromer, 2010). That's the strongest export there is, and what it exports is the claim; the history stays behind. The proof establishes its stated relation about the input and output. Unless identity or timing is part of that relation and supported by the setup, it does not identify the physical machine or when it ran. Producing a proof may require a witness and substantial computation; possession of the public input alone need not suffice. Nothing in this book supplies any of these escapes. The model's record is local to the machine that keeps it. The example of Part III keeps its record in its state with no signature on it at all, as a bare example would.

8.6 What the window keeps

Put the two boxes back under the camera. Their tops agree and their side stamps differ. The photograph answers questions about the tiles. To answer the stamp question, turn the camera far enough to include the side. The full state was there all along; what changed was which part reached the reader.

Taking more photographs from the same position only helps if something different eventually comes into view. When the two runs give the same picture at every step, a longer album keeps the same ambiguity. Kalman's unobservable states give that situation for an entire trajectory. The question is which window you have, and which distinctions it keeps.

Rice's theorem asks a different question about programs. Henry Rice proved in 1953 that no program can decide every instance of a nontrivial property of the partial function another program computes. Here you can use a box with just two states. Hand the reader one identical picture for both, and a question whose answers differ: the reader has one input and would need two answers. That argument finishes before any question about an algorithm's time or computability arises.

The counter lives in the state, and whether anyone outside can read it depends entirely on what they're shown, and a window with a pair like that in it can't price the flag exactly, with any rule, ever. That raises the question I've been pushing in front of me since the buckets. Fine, the flag went up and was paid for and I can see it in the right window. What does that *entitle* me to? A paid flag is a paid flag, so what's the claim behind it, and when am I allowed to lean on it?

CHAPTER 9

Structural entitlement

I keep writing the word “vouch” and then taking it back. Let me stop doing that and say what it would take. A reading went from NO to YES, the toll charged the move that did it (Section 2), maybe the reading stays up (Section 3 says what it takes to make it stay), and the right window shows it (Section 8). Now: what is a later step *allowed to do* because of that? It’s a different question from “is the thing true,” and the gap between those two is the subject of this section, and I’d argue it’s the subject of most of the arguments people have about evidence, so give me the long way round.

9.1 The coin that moved

There’s an experiment you can run on a friend, and I’d ask you to actually run it, because the feeling of it is the point. Put a coin under one of three cups while they watch; say it’s cup one. Send them out of the room. Move the coin to cup three. Then, and this is the trick, put a *second* coin under cup one. Bring them back and ask: “Is there a coin under cup one?”

Suppose they say yes because they trust that the coin they saw has stayed put. There is a coin there, so their answer is true. Grant that their original look and ordinary expectation of no interference justify the belief. The Gettier-style difficulty is then visible: the reason follows the first coin, while the replacement makes the answer true by luck. A suspicious friend who refuses the retention assumption has not entered this particular trap.

The experiment is Gettier’s trap, done with cups (Section 14 tells his own cases): Edmund Gettier’s three-page paper from 1963, “Is Justified True Belief Knowledge?”, which broke a definition of knowledge in about the time it takes to read it. The definition was that you know something when it’s true, you believe it, and you’ve got good reasons. Gettier took it from two philosophers of the 1950s, Roderick Chisholm and A. J. Ayer, and the only place he mentions Plato is a footnote saying Plato “seems to be considering” something like it in the *Theaetetus*. I’d assumed people had leaned on that definition since Plato. As far as I can find, they hadn’t. The Stanford Encyclopedia’s entry on the analysis of knowledge calls the idea that it

was ever widely held "a bit of a convenient fiction," and Julien Dutant, a philosopher at King's College London, wrote a 2015 paper called "The Legend of the Justified True Belief Analysis" arguing that the older tradition asked for something stronger: a mark on the belief that can't be wrong. I've read only summaries of Dutant's paper, so take that last part as his, secondhand. Gettier's two cases, and the cups, show a belief that passes all three tests and still isn't knowledge, because the reasons and the fact aren't connected; they just happen to agree. Bertrand Russell had a related case in 1948: somebody looks at a stopped clock at the one moment it happens to be right, and they're right about the time, and they know nothing.

Is 1963 the bottom? It isn't even close, and 1948 isn't either. The Stanford Encyclopedia's entry on the analysis of knowledge, which is where I'd send you to check me, gives a case from the eighth-century Buddhist philosopher Dharmottara: you're looking for water on a hot day, you see water ahead, it's a mirage, and when you get there you happen to find water under a rock. And one from Peter of Mantua, a logician of the late fourteenth century who died in 1400 and whose *Logica* survives in manuscript: Plato is running right next to you, you take him for Socrates, so you believe Socrates is running, and Socrates is in fact running, in Rome, where you can't see him. Are there older cases still? Maybe, and this is where the trail goes cold, which isn't the same as where it ends. What it changes is how much I trust the shape. People in different centuries, languages and traditions kept building the same trap by hand: a reason that points at one thing, and a truth that happens to be sitting on another.

What would repair this particular reason? Lift cup one again. Now there is a new look at the coin currently there. Under the ordinary assumptions about seeing the cup and coin, that gives current evidence instead of a memory about the coin that left. It does not make perception infallible or promise the coin will remain after the cup goes down. The demonstration isolates the missing connection; it is not a proof settling every account of knowledge.

9.2 Plato nearly had it

Who was Theaetetus, though? The dialogue is named for a real person, and I wanted to know who. He was a mathematician. In the dialogue he's a teenager, and he tells Socrates that his teacher Theodorus had been showing them, one case at a time, that the side of a square of area three, or five, and so on up to seventeen, can't be named by any fraction: the Pythagoreans' diagonal from Section 6, done again case by case. The young man's idea was to stop going one case at a time and find one word for all of them. Euclid's tenth book, on lengths that share no common measure, is thought to be largely his work, and he's credited with being the first to study the eight-faced and twenty-faced solids, the two Plato assigned to air and

water in the *Timaeus*. The dialogue opens with him being carried home to Athens from the army at Corinth, badly wounded and sick with the dysentery going round the camp, barely alive; the usual date is 369 BC. And the conversation itself has come down as a record. Euclid of Megara (not the geometer) says he took notes as soon as he got home, filled them out later, and had Socrates correct them whenever he came to Athens, and he has the result read aloud. So the dialogue about what turns a true belief into knowledge survives as a record Socrates himself corrected. That changes nothing in the argument. I just like that the book about knowledge is named for the man who took the diagonal and made it general.

I want to be fair to the *Theaetetus* here, because it's easy to be quick with the ancients and they deserve better. Socrates, in that dialogue, doesn't just say "true belief plus reasons." He tries three different senses of what the "account" behind a true belief could be: a statement you can say aloud; a list of the thing's parts; and, third, the mark that sets this thing apart from everything else it could be confused with. He tries each, and each fails, and the dialogue ends with nobody knowing what knowledge is, which is one of the more honest endings in philosophy. But the third sense is the one I'd underline: an account as *the thing that distinguishes*, the test that gives the real case a value no rival gives. He couldn't make it work because he had no way to say what the account had to be attached *to*. Two and a half thousand years later, this book's guess is that the thing it has to be attached to is a state, and the account is an object sitting in it. I'm not claiming that settles Plato. I'm claiming he was pointing at the right thing, and that's my reading of the dialogue, which you can check against it in an afternoon.

Push on that once more, because Socrates gives his own reason for dropping the third account, and it isn't the one I just gave. If the account means *knowing* the mark that sets the thing apart, then knowledge has been defined as true belief plus knowledge of something, and the word you were trying to define is back inside its own definition, which is a circle. Does a check get out of it? It gets the word out: a test that ran and found the mark isn't defined in terms of knowing. Whether the test was any good is still a real question, and it's the question the rest of this section puts a price on. So I'd say the circle moves rather than vanishing. It moves somewhere you can look at it, which is the most I'd claim.

9.3 The shape, said once

What I mean by entitlement has three parts, and I'll try to earn each.

A run is entitled to a claim when three things are present. First, an *object*, carried in the state: a grouping, a relation between groups, a counter, a package of constraints. The object is a thing you could print; a belief doesn't count. Second, a *checked relation* between that object and the claim: a test that was actually run,

with its result on hand; "a test exists" doesn't count. Third, an *event on the record* that says this object was put there and paid for, so that anyone auditing the run can point back at the moment the claim was marked.

I'll say it as a definition, so the rest of the section has something to point at:

Definition 9.1 (Entitlement)**.** A run is *entitled* to a claim when three things are on hand: an *object* carried in the run's states; a *checked relation* between that object and the claim, meaning a test that was run and whose result is on the record; and a *paid event*, a move of the run at which the object was put in place and which the price list charged at least one. In a Thiele machine (Definition 2.2) a move that takes the reading from no to yes is a paid move, because the toll charges it; whether the object went in at that move is a separate question. This is the book's word.

The first part comes with the third: a move of the run that put the object in place leaves it in the run's next state. I name the object on its own anyway, because the checked relation is about it. Keep the second and third parts separate. A note saying "coin under cup one, checked" can be written without anyone lifting a cup. That gives an object, but no checked relation. Now actually lift the cup, find the coin and write the result. Suppose that move costs nothing: the note is on the sheet, the check really happened, but the NO box stays NO and the tally line stays blank. This time the object and the checked relation are both there. What is missing is the paid event. The run isn't entitled in this definition's sense; that doesn't make the result of the check false.

The reverse distinction matters too. A move from NO to YES puts one unit of cost on the tally line, but the bare toll says nothing about what that move looked at. Payment alone supplies neither a check nor its correctness. The later machine will bind those acts together in its step rule; the five parts of a Thiele machine do not do that by themselves.

The part to hold on to is that nothing in the five parts makes a later move *read* the reading. The step function is handed the state and a move name, and it's allowed to ignore the YES box completely; plenty of systems will. So when I say a run is *entitled*, I mean what someone auditing the run may conclude from the reading and the receipt. The system doesn't have to enforce it, and unless somebody builds it to, it won't.

This is a good place to line up the three things the word *established* has had to mean so far, because they blur easily, and a blur here would make the book look like it proves more than it does. The yardstick is the four conditions Section 14 sets for a check being about this very claim: the same property, the same thing, the same edition, and an honest check.

What's asked	Bare toll	Entitlement	Earned certification
A check ran	not required	a test result on the record	a passing check before the commit, on every run from a clean start
Same property	not asked	not asked	enforced by the step
Same thing	not asked	not asked	enforced by the step
Same edition from CHECK to COMMIT	not asked	not asked	enforced; CERTIFY may later read that historical commitment
Honest check	not asked	not asked	proved of the checker, once, outside the record
Could have failed	not asked	not asked	not policed
Paid for	at least one unit	at least one unit	at least three units for the chain

After the column of questions, the first column is Section 2's bare toll: a paid crossing, and nothing says a check happened at all (Section 10 has a remark saying exactly that, Remark 10.2). The second is this section's entitlement. The third is the small machine of Part III, whose step won't let a commit through without a passing check of the same property, on the same thing, at the same edition, for every program from a clean start (Theorem 15.10, and Theorem 15.11 for a run that ends certified), whose checker comes with a proof that it says what it means (Lemma 15.2, Theorem 15.13), and whose certified runs from a clean start cost at least three (Observation 15.19). Even there two things are missing, and the first is in the table, where you'll see it. The step doesn't police whether a check could have failed: a property every value has is a perfectly legal check, sound and useless. And in all three columns the status belongs to whoever reads the record. The machine doesn't know anything. What its flag means is assigned from outside, by whoever says what each property means.

Why does the honest check carry so much? The philosopher Linda Zagzebski, who works on knowledge and the virtues of a careful mind, argued in 1994 that any account of knowledge whose justification leaves even a small gap between having good reasons and being right lets a Gettier case through: take a case where bad luck makes the reasons point the wrong way, then add a stroke of good luck that makes the belief true anyway. The first three conditions plus an honest check close that gap by construction, and only one half of honesty is needed for it, the half that says no when the property is false. Saying yes whenever it's true is extra. A check

that can be wrong, a test suite that sometimes passes broken code, puts you straight back in her gap, and then the record certifies something smaller and still true: this procedure said yes.

And who certifies that the check is honest? The record doesn't. Honesty is a fact about what the check does on every input, and a finished state holds the run's values, not that. A proof of it can sit in a state and be checked there, but the checking is a check too, with its own honesty to vouch for, and somewhere the chain stops at a proof somebody accepts; in this book that proof is read outside every run, and nothing on the record pays for it. So an established claim is two things stuck together: a run property the record carries and a program property a proof carries, and the proof is entitlement for the auditor. The machine gets none from it. On the machine of Part III the chain stops at the first step, the checker's own proof, by choice; Section 15 says why.

The engineering version of this was already built, in 1996 and 1997. The computer scientist George Necula, then at Carnegie Mellon, called it *proof-carrying code*: when you ship a program to a host that doesn't trust you, you ship a *proof* with it that the program obeys the host's safety rules, and the host *checks the proof that arrived* before running anything: not your reputation, and not "a proof exists somewhere," but the one in the box. A quarter century earlier Stephen Cook's certificates had the same bones, from Section 2: a claim comes with a witness that's cheap to check, and the checker checks *that* witness. And the Scottish philosopher David Hume, in 1748, arguing about miracles in his *Enquiry Concerning Human Understanding*, laid out the thing the receipt replaces. He weighed testimony against evidence, and wrote that "a wise man, therefore, proportions his belief to the evidence." The way I read him, the whole of his argument is a method for how much to trust a witness when you have no receipt. The move in this book is to stop weighing. A claim on the record is either backed by its object and its paid event, or it's testimony from outside, and a run that leans on testimony hasn't earned it itself. It can inherit it, but only as far as the copy it leans on is genuine, which is Section 8's problem of a record shown from outside.

Robert Brandom, a philosopher at the University of Pittsburgh, built a whole account of language on two words this book leans on, commitment and entitlement, and his *Making It Explicit* (1994) is where I'd send anyone who suspects I invented the pair. In his picture, as I read it and as his reviewers describe it, people talking keep score on each other: what each speaker is *committed* to, what they've put themselves on the hook for, and what each is *entitled* to, what they're licensed to claim. You can be committed without being entitled (a reckless assertion), many claims count as entitled by default until someone challenges them, and entitlement can be borrowed: I can claim a thing on your say-so and defer to you when I'm challenged. Lay mine beside his and the differences are the content. His score is

kept by the other people in the room; here it's a reading in the state. His entitlement often comes by default; here it's an event that has to happen and is charged. And his deferral is my testimony, which I've just let back in on a leash: borrowed entitlement is only as good as the copy is genuine. The leash has an older formal cousin, the jurisdiction rule of Burrows, Abadi and Needham's logic of authentication (Section 8), where you take a statement on an authority's word only if you already believe the authority has jurisdiction over it. It came first, presented in 1989 and printed in 1990, years ahead of Brandom's book of 1994, and its words are *believes* and *jurisdiction* where his are *commitment* and *entitlement*.

There's a whole field that would want a word here too, the logic of belief revision, which asks how a reasonable mind should change what it believes when something new comes in. Its nearest neighbours are worth naming, and I couldn't keep them apart until I put them back on the table with the cups. Your friend believes the coin is under cup one. They can learn something that doesn't clash with that, say cup two is empty, and just add it. They can lift cup one, find nothing, and have to change their mind about a table nobody touched. Or you can tell them you moved the coin. The last two felt alike to me at first, but they aren't: in one the friend was wrong, and in the other the friend was right, and then the table changed.

Krister Segerberg, a Swedish logician at Uppsala, built a logic of irrevocable revision in 1998: accepting something so that it can't be given up, which is the firmness question taken to its end. Isaac Levi, a philosopher at Columbia who spent a career on how beliefs get fixed and unfixed, split adding a belief into a deliberate kind and a routine kind (I cite his 1991 book; the split goes back to his 1980 one). Deliberate is reasoning your way there. Routine is where you take on whatever a procedure you committed to in advance hands you, the way you'd accept a reading off a thermometer you've already decided to trust, or what's under a cup when you lift it. What I think the split rests on is when the trust goes in: once, before the data comes. A passing CHECK, the test move of the small machine in Section 15, is routine in exactly that sense, with a checker proved honest once, outside every run.

Neil Tennant, a logician at Ohio State, and Gösta Grahne, a computer scientist trained at Helsinki, I first filed as two separate complaints. They're one move made from opposite sides. The target is the best-known impossibility in that field, the Swedish philosopher Peter Gärdenfors's 1986 theorem that a natural rule for conditionals, the one his paper calls the Ramsey test, can't live with rational revision. Tennant argued in 2008 that it goes through only by treating some additions as revisions: the friend who learned cup two was empty gets counted as having changed their mind. Grahne, working on updates and counterfactuals, argued in 1998 that Gärdenfors's theorem bites only revising, so the friend whose coin was moved is out of its reach. Side by side, they squeeze the theorem from both ends into one case: being wrong about a table nobody touched.

The line this book leans on hardest is between two kinds of change. Revising is changing what you believe about a world that stayed put. Updating is changing it because the world moved. Two computer scientists, Hirofumi Katsuno and Alberto Mendelzon, drew that line for knowledge bases in a 1991 paper on the difference between updating a knowledge base and revising it; database researchers had been working on updates with incomplete information since the mid-1980s, Arthur Keller and Marianne Winslett among them. Grahne's result stands on it. And look at what the friend sees: an empty cup one is the same empty cup whether they misremembered or you moved the coin. The evidence alone can't tell revising from updating; that's a fibre. What tells them apart is a count of how often the cups were touched. The machine in Part III keeps that count as an edition number for each thing, and its version rule treats every write as an update: a write moves the edition number, so every fact about the old edition stops counting for the new one.

And Alexandru Baltag, Bryan Renne and Sonja Smets, three logicians at Amsterdam, gave a logic in 2014 that keeps apart three things Gettier's cases blur: justified belief, explicit knowledge and conclusive evidence. The friend who watched has justified belief; the friend who lifted the cup has conclusive evidence. Explicit knowledge I read as knowing with the justification written out beside it, the way the record keeps its check, and I'm less sure of that one. Put them together and they say something exact about this book's record. A record that's never wrong and never corrected would be a dogmatist's dream if its sentences were about the present, but they aren't: each says a property held of a thing at one edition, and that sentence stays true after the edition number moves on. The record can be infallible and never corrected only because its sentences are edition-stamped. Strip the stamps, and the first write would turn it into a liar that can't take anything back.

Push on that and it gives twice. The record is Segerberg's irrevocable acceptance every time, with no revising in it. If a check was dishonest, the only fix that fits a record like this is a second record on top saying "withdrawn," the way a revocation list works, so the infallibility leans on the checker, proved once, outside. And the record is never wrong partly because it only talks about past editions. Whoever reads it wants now, and getting from "held at that edition" to "holds now" crosses every update since. The record doesn't make that step for you.

That leaves the logicians who taught the logic of knowing to count, whom Section 14 names again in passing. This time I take them properly, and I want to see what each price rests on. Ho Ngoc Duc, in 1997, gave the logic of knowing an operator for "true after some train of thought": an agent doesn't know every consequence of what it knows now. It can come to know any of them by taking steps of reasoning. I took that at first for the thing I was after, until I saw which way it points. His operator points forward, at what the agent could come to know. A record points back, at what was checked. His operator and a record are one arrow,

read from opposite ends. And Sergei Artemov and Roman Kuznets, in 2014, turned logical omniscience into a question of cost: a logic escapes it when every valid claim that something is known comes with a proof whose size is bounded by a polynomial in the size of the claim. The usual modal logics of knowledge can't pass that test unless NP equals coNP, which most people would bet against, and justification logic passes it for knowledge stated together with its justification, because the size of the justification pays for the search. That last clause took me a few reads. What I think it means is Cook's certificate from the proof-carrying-code paragraph above: put the justification inside the claim, and the hard part, finding it, was paid before the claim was written. Their price rests on an open bet, and it's a price on looking, which is a price of a sort. Steps and a price are both there, then, in their own form. What I still don't find in them is the one I care about: a single act that closes a question for good and is charged as that act. Their steps are steps of reasoning, and their price is on finding a proof. Neither is on settling. I couldn't get past the journal's paywall to Artemov and Kuznets's paper, so my account of it comes from how other people describe it.

Push on mine the same way and it gives. A check is one of Duc's trains of thought, so my act is the last step of his train, where the walking stops. What I've added is a direction and a charge on stopping. Where the charge sits follows from one choice: I want it to bind every program the machine runs, and a charge that does that has to sit inside the step, since a program can skip any charge kept where it can write.

There are three things entitlement is *not*. It isn't permission: an access control system says who may write to what, and that's granted from outside and says nothing about truth; a process with permission to write the note still has no entitlement. It isn't "it's true, so I can use it": that's the cups. And it isn't "sound by luck": a shortcut that gives the right answer because the inputs happened to cooperate is the stopped clock. The count below has one condition aimed at it, and I'll tell you what the condition says and how much it actually catches.

9.4 What narrowing costs: count the rivals

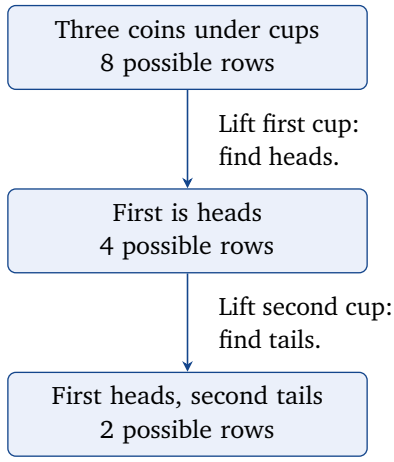
Now I need the number, because a book about what keeping costs has to say how much, and the idea fits in a game you can play with three playing cards. Shuffle cards marked 1, 2, 3 face down. Sort them using only questions of the form "is this one bigger than that one?", and count the questions. There are six orders the cards could be in. Each question has two answers, so after one question you've split the six into at most two piles, after two questions at most four, and you need to get down to one. Four is less than six, so two questions can't always do it, and three can, and you can draw the tree of questions on paper and see it: six leaves, three deep. The number of questions you need is at least the base-two logarithm of the

number of possibilities you started with, rounded up: that's log2 of 6, which is a bit under 3, rounded to 3. Donald Knuth, a computer scientist at Stanford, gives this, in the volume on sorting of his *The Art of Computer Programming*, as the lower bound for all comparison sorting, and the shape of the argument is just: a tree f questions deep has at most 2^f leaves, so it can't tell apart more than 2^f cases.

And then the obvious next question: is the floor always the price? Try five cards. There are 120 orders, log2 of 120 is a bit under 7, so you need at least seven questions, and seven is enough, though not by the obvious method. Finding the way is a good evening with a deck of cards (Knuth has it, under the name merge insertion, from the mathematicians Lester Ford and Selmer Johnson in 1959). Now take twelve cards. They can be in 479,001,600 orders, log2 of that is about 28.8, so the count says at least 29. Ford and Johnson's method takes 30, and in 1965 Mark Wells, a mathematician at Los Alamos who built a programming language for exactly this kind of combinatorial search, showed by a computer search that nothing does better: twelve cards need 30. The floor was 29 and the price is 30, so the count is a floor and only a floor; it never promises the floor can be reached, and the bound later in this section has to be read exactly the same way.

Ralph Hartley, an electronics researcher at Bell Labs, writing in the Bell System Technical Journal in 1928, before Shannon, measured information exactly this way: the logarithm of the number of possibilities, no probabilities, just a count. I use that measure here, and I'll say plainly it's Hartley's. Shannon's would need a distribution over the list, and there isn't one; there's just the list's length.

The same count works with something you can hold. Put three coins under three cups. Each coin is heads or tails, so there are eight possible rows under the cups. Someone hands you two answers: heads under the first cup, tails under the second. Lift the first cup and compare, and a matching answer leaves four possible rows. Lift the second and compare, and a match leaves two. The last coin can still be either way. The list really shrank: eight, four, two. Nothing in that count needed a claim about which way the last coin faces.



The third coin could be either:
heads or tails.

These observations halve the candidate list. A price on those observations is a separate rule; the shrinking list alone supplies none.

This is a parlour game, and an old one. In *A Christmas Carol*, in 1843, Scrooge's nephew Fred plays a game called Yes and No at his Christmas party: he thinks of something, and everybody else asks questions he may only answer yes or no. He's thinking of an animal, a disagreeable, savage one that growls and grunts and lives in London, and the answer is Uncle Scrooge. Most people know the game as twenty questions, and twenty is a good number for it: a balanced question splits the remaining field in half, and twenty such questions can guarantee identification among $2^{20} = 1{,}048{,}576$ possibilities. A lucky answer to "is it this one?" can finish immediately; the count is about handling every possible answer. Push on that, though. The halving only happens because Fred answers truthfully about what's actually in his head. Lie once and the count still shrinks; it shrinks around the wrong animal. The cups showed the same thing: the four and the two are real because you lifted the cup and compared. Being told what was under it wouldn't have made them real.

Now put it on a run. Say a run comes with a list of states that are "still on the table," the candidates, and a shorter list, with some ruled out. Call the first the prior and the second the posterior, and I want you to hear those as *a list you could print*. They aren't "the space of possible worlds." Picture the ruling-out as observations, each observation that ruled something out being one branch point in a tree like the card tree. The bound says that if that tree is no deeper than the number of paid steps the run took, and the tree has enough leaves to cover the narrowing in a way I'll spell out in a moment, then the counter rose by at least log2 of the prior's length, rounded up, minus log2 of the posterior's length, rounded up. Put another way, the

counter rose by at least one unit for each of the *index bits* the list lost, where the index bits of a list are the yes/no questions it takes to point at one entry of it, the card count again.

“Paid step” needs its own sentence. It means a step the price list charges at least one for. In a Thiele machine the toll guarantees that every crossing of the reading is one; a price list may put a floor under more steps than that (a look, a send, a check), and the count takes every paid step there is. The tree is a bare shape, branches and leaves with nothing written on them, and “no deeper than the number of paid steps” is a count along the run that happened, not a walk down the tree. A paid step doesn’t have to have asked any question at all.

Notice, too, what the bound does and doesn’t say about the run. It takes the two lists, the tree and the covering as given. It doesn’t say the run produced the posterior from the prior; nothing in it reads the lists off the run. What it says is that, given lists, a tree and a covering that someone supplies, the paid steps of the run cover the rounded drop in list length. Here Section 5’s warning comes due: watching can narrow without costing anything, so the payment can’t be dug out of the narrowing. It goes in as a premise, the tree’s depth against the paid steps, and I’ll keep calling it one. There is a version where the run makes the posterior itself, from a prior and the tests it passed, and there the premise is a share each test keeps.

9.5 What “narrower” buys: a stronger test

Before the count, one link that’s pure logic, and Plato’s third account is the whole of it. Take a list of candidates and an observation, a function that says what each candidate looks like from outside. The list’s *membership test* accepts an observation when some candidate on the list shows it. Shrink the list, and the test can only get pickier: anything the short list’s test accepts, the long list’s accepts too. The word for that, and it runs backward from English, is *stronger*: the stronger test is the one that lets *fewer* things through.

Pickier, though, isn’t the same as strictly pickier, and the cards show the gap. Take all six orders as the prior, and as the posterior the three where card 1 sits left of card 2: 1-2-3, 1-3-2 and 3-1-2. First observation: “is 1 left of 2?” Every survivor says yes, and the removed 2-1-3 says no, so the prior’s test accepts yes and no, the posterior’s only yes, and the posterior’s test is strictly stronger. Second observation, same two lists: “is the first card a 3?” Now 3-1-2 says yes and the other two survivors say no, and among the removed orders 3-2-1 says yes and 2-1-3 says no. Both tests accept yes and no, so they have the same strength, even though three orders are gone. What made the difference is a removed candidate whose observation *no survivor shares*. Such a candidate is Plato’s mark, the thing that sets this case apart from everything it could be confused with, turned into a function. Narrow the list and

keep one distinguishing removed candidate, and the test is strictly stronger; narrow it without one, and the test may not have moved at all.

9.6 What you've got

A paid flag is not a true claim; Section 8 left it at "a paid flag is a paid flag," and Section 10 will say it flatly. What a paid flag with an object and a checked relation behind it gives you is *entitlement*: anyone auditing the run may treat the claim as established, and can point at the event where that became legitimate. (Nothing in the five parts makes the system itself read the flag; I said that above and I'm saying it again because it's the sentence most likely to get lost.) Entitlement costs at least the index bits of narrowing it commits to, given lists, a tree and a covering somebody supplies, and given a run that paid for the tree's depth. The cups say why entitlement and truth are different. Plato says what the checked relation has to do, which is to distinguish. Necula says how to build it. Hartley says what to count.

Put the three pieces on one hypothesis list (narrow the list with a test that separates, find the paid event, count the charges against a tree the run paid for) and you get a single statement, structural entitlement, once over any Thiele machine and once more on the small machine of Part III, where the run starts clean and the paid steps are its own record moves, CHECK, COMMIT and CERTIFY, at a cost of exactly one each. The first one reads as follows.

□ **Theorem 9.1** (Structural entitlement, over any Thiele machine)**.** Take a Thiele machine (Definition 2.2), a state s that reads no, and a run t from s that ends in a state that reads yes. Take a type of candidates, a type of observations with an exact equality test, an observation obs of each candidate, and a *decoder* d that turns a run into an observation (what its record shows). Take lists Ω and Ω' of candidates and a decision tree T such that: (H1) Ω' strictly narrows Ω: every member of Ω' is in Ω, and some member of Ω isn't in Ω'; (H2) some dropped candidate w, in Ω and not in Ω', shows an observation different from every survivor's; (H3) the decoded run $d(t)$ is shown by some survivor; (H4) $|\Omega'| > 0$ and $|\Omega| \le L(T) \cdot |\Omega'|$; (H5) at least $\mathrm{depth}(T)$ steps of the run are charged at least one each. Then (1) the membership test of Ω' is strictly stronger than the membership test of Ω; (2) some step of the run took the reading from no to yes, and that step was charged at least one; and (3) the sum of the charges of the run's steps is at least $\lceil \log_2 |\Omega| \rceil \mathbin{\dot{-}} \lceil \log_2 |\Omega'| \rceil$.

Proof. (1) is the cards' argument: the dropped w shows something no survivor shows, so the posterior test says no where the prior test says yes, and every survivor was in the prior, so nothing the posterior accepts is refused by the prior. (2) The

run starts at no and ends at yes, so at some step the reading goes from no to yes, and the toll charges that step at least one. (3) is a count: the covering puts every survivor at a leaf of T, a tree of depth $\mathrm{depth}(T)$ has at most $2^{\mathrm{depth}(T)}$ leaves, so the index bits are at most $\mathrm{depth}(T)$, and the sum of the charges is at least the number of steps charged at least one. On a system that obeys conservation the sum of the charges is the counter's rise. ∎

With the counter's rise in place of the sum of the charges it's the same count, so for a system that obeys conservation it's the special case. Three things in the hypotheses feed nothing, and I'm naming them here so you don't have to find them: (H3) isn't used by any of the three conclusions; from (H1) only the inclusion is used, and the gap goes unused; and in (H2) the words "not in Ω'" go unused, since a candidate whose observation differs from every survivor's can't be a survivor itself. And (H4)'s $|\Omega'| > 0$ is used in that proof but can be dropped too: the covering forces it once the prior has a member.

A statement like that is true by packaging: it takes the hypotheses in and hands the conclusions back, and the right word for it is a checklist, a checklist for leaning on a claim. It discovers nothing about leaning.

And a checklist isn't nothing. On 30 October 1935, at Wright Field, northeast of Dayton, Ohio, the Army was evaluating Boeing's new four-engine bomber, the Model 299. Shortly after take-off it plunged back onto the field and caught fire; that's how the wire report of that day has it, and the later histories add that it pitched up and stalled first, which is what elevators locked in a raised position would do. Two men died of their injuries, the Army's pilot, Major Ployer Hill, and Boeing's chief test pilot, Leslie Tower. The airplane was fine. The crew had left the gust locks on, a locking mechanism worked from inside the cockpit that holds the control surfaces still while the plane sits on the ground. There's a famous line that a newspaper called it too much airplane for one man to fly. I couldn't find that line in any paper from 1935; the earliest place I could find it is a New Yorker article from December 2007 by Atul Gawande, a surgeon who writes about how people get complicated work right, and he names no newspaper, so take it as a lead. What I can find is what came after. The answer was a card: a list of things to check, in order, every time, and the crews of the airplane worked through written checklists for each phase of flight. The most careful history of it I found is by Walter Boyne, a retired Air Force colonel and a former director of the National Air and Space Museum. By his account it wasn't the first written routine for a pilot; those go back before the First World War. And it's this crash, he says, that made the checklist the way a crew flew. The airplane went on to be the B-17. Nothing on that card was a discovery. Every item on it was something the pilots already knew. What the card did was make sure none of it got left out when it mattered. Putting entitlement's hypotheses on one list does the

same: it finds nothing new about leaning on a claim, and it makes the hypotheses impossible to drop quietly.

9.7 A grouping, checked and committed

Section 6 ended on buckets, and the toll of Part I never needed them: it prices any yes-or-no reading. In this subsection the word in this section's title gets attached to a partition in a theorem. Take a pile, a finite list of items, and a *grouping*, a bucket label for each item. Its partition is "same label," and the classes of that relation are a partition in the sense of Definition 6.1. The data is a *key* for each item, drawn from any type with at least two values (a grouping's own labels can serve as a key), and a grouping is *sound* for a key when any two items it puts in one bucket have the same key. The grouping says "these belong together," and the key says whether they do. One grouping *refines* another when every bucket of the first sits inside a bucket of the second.

Put two red buttons and two blue buttons on the table, numbered 1 through 4, with 1 and 2 red. Let the key be colour. Circles round $\{1, 2\}$ and $\{3, 4\}$ give a sound grouping: every pair inside a circle has the same colour. Split the red circle into two singletons and it stays sound; splitting cannot introduce a mismatched pair. Join 1 and 3 instead and it fails. Changing the circles' names from A and B to North and South changes none of these checks. That is why the reading below uses the partition, not the spelling of its labels.

Now comes the rule for the record. Put a tick beside a grouping only after comparing the colours within each circle. Copy that ticked grouping onto a separate commitment card, then place the card in a record envelope; charge one tally for each of those three acts. Change a button's colour between the check and the copy, and the old tick must no longer authorize the copy. The machine's SET rule below does that by clearing the current table and channel. A record already kept remains about the old colours, which it keeps with the grouping.

Here is one example machine that meets the model, the smallest I know that makes a record from a checked claim. It is an illustration; the theorem above holds of any Thiele machine. The *bucket machine* holds a key, a table of checked groupings, a channel, a record, a trap latch and a ledger. SET costs nothing; it replaces the key and throws away the table and the channel, which were checked against the old key. CHECK g costs one and passes when g is sound for the current key, and, when it passes, writes g in the table. COMMIT g costs one and passes when the table holds a grouping with the same partition as g, and puts g in the channel. CERTIFY costs one and passes when the channel holds a grouping; if the record is still down it then keeps that grouping with the current key, the key a grouping with its partition was checked against, and a record already kept is left as it is. A CHECK, COMMIT

or CERTIFY that fails traps, and a trapped machine only pays. The *reading for a grouping g* is: the record holds a grouping with g's partition. In words, that reading says "this grouping has been checked and committed."

Theorem 9.2 (A committed grouping entitles exactly its refinements)**.** (1) The classes of "same label" under any grouping form a partition. (2) Two groupings with the same partition of the pile have the same reading at every state of the bucket machine. (3) For groupings g and h of the pile, soundness of g for a key gives soundness of h for that key, for every key, exactly when h refines g. (4) On every run from a clean state (table and channel empty, record down), if the reading for g is up, the record holds a grouping and a key for which g is sound, and so is every grouping that refines g; and for every grouping h that doesn't refine g there is a key for which g is sound and h isn't. (5) The bucket machine obeys the toll, and a run from a clean state that ends with the record up has raised the ledger by at least three. □

Proof. (1) Every item has its own label, exchanging two items does not change whether their labels agree, and two agreements through a third label give agreement with each other. Thus "same label" has the three sameness rules, and its classes form the partition already constructed in Theorem 6.1.

(2) The reading asks which items were put together, not what the buckets were called. Replacing one grouping by another with exactly the same buckets therefore changes no answer to that reading.

(3) If h only cuts g's buckets smaller, every pair h keeps together was already together under g. Soundness of g gives that pair equal keys, so h is sound too. If h does anything else, it puts some two items together that g kept in separate buckets. Give every item in the first item's g-bucket one key value and every other item a second key value. Each g-bucket has a single key, so g is sound. The chosen h-bucket contains both key values, so h is not. This is why the key type needs two values.

(4) Follow the table, channel and record through the commands. They begin empty. CHECK inserts only a grouping it has found sound for the current key. COMMIT takes the partition of a grouping already in that table, so the selected grouping is sound for the same key. CERTIFY keeps that grouping and that key together; later commands do not replace an existing record. SET changes the key but empties both the table and the channel, so no old unchecked carry-over remains there. A failed command traps instead of inserting an unsound grouping, and trapped commands leave those fields alone. These are all the ways those fields change. Their promised soundness therefore survives every command of any run from the clean start. Apply (3) to the grouping and key in the record: every

refinement is sound, and every non-refinement has the counterexample key just constructed.

(5) Only CERTIFY raises the record, and its price is one, giving the toll. A successful CERTIFY needs a grouping in the channel. From an empty channel only a successful COMMIT could have supplied it. From an empty table that COMMIT in turn needs a successful CHECK. These are three distinct earlier acts in order, each charged one. The run's nonnegative bill is at least three. ■

So "structural" has a theorem under it. The reading sees a grouping only through its partition, which is item (2). If the recorded circles are sound, so is any way of cutting them smaller, whatever the key is. A grouping that isn't a cutting-smaller can still happen to be sound for this key (paint all four buttons red and every grouping is sound), but nothing in the record promises it: for some key it fails. Item (3) and item (4) say exactly that much, and that's all the record buys. It's the three parts of entitlement from the start of this section in their smallest case: the object is the grouping, the checked relation is soundness for the key, and the paid event is the CERTIFY, on top of a CHECK and a COMMIT at a cost of one each. What it doesn't do is make a later move act on the record. The only move that reads it is CERTIFY, and only to leave a kept grouping alone. Like everything in this section, it says what someone auditing the run may conclude.

Before the machine of Part III, I owe you the word the toll is named after, the "insight" in "No Free Insight" (Section 5), because I've been using it as if it meant one thing.

CHAPTER 10

What insight is

I named the rule in this book "No Free Insight," and the name ran ahead of any careful account of what insight is. The book has been narrowing it ever since: not learning (Section 5), not looking, not copying. Part I already says where to look: learning borrows and committing pays, so the part of insight a toll can price is the part that stays. It's time to say what that is, and to say it in a way that someone who studies actual insight, in actual heads, wouldn't laugh at. So who are those people, and what did they find? I'll go through what I ran into and push on each piece as I go. Where a story reached me secondhand, I treat it as a lead to check.

10.1 Sultan and the sticks

In 1913 the psychologist Wolfgang Köhler went to Tenerife to run a research station with a colony of chimpanzees, and the First World War then kept him there until 1920, five years longer than planned, with the chimpanzees to watch. The famous one is Sultan. Food is outside the cage, out of reach; two sticks are inside, each too short, but one fits into the end of the other. Sultan tries each stick, fails, sulks, plays with the sticks, and then, by Köhler's account, at a moment you can see, joins them and reaches the food. Köhler's words were *Einsicht* and *einsichtig*, which came into English as "insight," and the whole school of psychology he belonged to, the Gestalt school, said that's what thinking is: a *restructuring*, the moment two sticks stop being two sticks and become one tool. Stacking up tries until one works was, for them, something else. Karl Duncker gave the human version, in German in 1935 and in English in 1945, five years after he died at thirty-seven; he'd spent his last years working with Köhler at Swarthmore. You can set it up yourself: a candle, a box of tacks, matches, and the task of fixing the candle to the wall. The obstacle is seeing the *box* as a possible shelf while it is presented as the container for the tacks. And then he found that if you hand people the box *empty*, with the tacks poured out beside it, they solve it far more readily. The objects were the same; what changed was which relation among them was on the table.

I like both of these for the same reason: the insight didn't add an object. Sultan had both sticks the whole time. The box was there the whole time. What arrived

was a *relation*, which is to say a piece of structure in Section 6's sense, and the arrival was sudden. Hold that, because the thing the toll prices also arrives in one step, and at its best it's also a relation.

10.2 It isn't a feeling

A feeling of getting closer is a different thing to measure from solving the problem. Janet Metcalfe's experiments help separate them. In the 1980s the psychologist Janet Metcalfe asked people to solve problems while rating, every fifteen seconds, how "warm" they felt, how close to the answer. On ordinary problems, algebra for instance, the warmth climbs steadily and peaks at the answer, which is what you'd expect. On *insight* problems, riddles of the candle-box kind, the warmth stays flat, flat, flat, and then the answer arrives, and if anything the people who felt warmest beforehand were *more* likely to be wrong. She put that last finding in the title of a 1986 paper: premonitions of insight predict impending error. Those results concern the predictive value of warmth ratings in those tasks. They do not show that a solution has no accompanying feeling, or that every person follows the same curve. Try it yourself: write a number from one to seven every fifteen seconds while doing a long sum, then while doing a riddle, and compare the curves.

And the most famous insight story there is fits Metcalfe uncomfortably well. Archimedes, a mathematician who died in 212 BC, asked by Hiero, the king of Syracuse, whether a goldsmith had cheated him on a crown, gets into a bath, sees the water rise, and runs home naked shouting "eureka," I've found it. That comes from Vitruvius, a Roman writer on architecture, writing about two centuries later. Is it true? Galileo Galilei, then a young Italian mathematician, at twenty-two, in 1586, took it as known that Archimedes caught the goldsmith. What he didn't buy was the method. Water spilling over the side, as Vitruvius tells it, was to him "a crude thing, far from scientific precision," and he guessed some writer had set down a rumour. So he wrote a little tract, *La Bilancetta*, proposing a balance weighed in air and in water instead, which he thought was probably what Archimedes actually did, since it rests on Archimedes's own results. Whether there was ever a bath, or a shout, nobody can check now. What survives is a story two centuries late, wrapped around a real man and probably a real problem. Notice what the legend kept: the bath, the shout, the running home naked, which is to say the feeling. What it left shaky is the part that would count as the check, the test that could have caught the goldsmith. The bit everybody remembers is the bit that tells you least.

So any account of insight that keys on the sensation, the "aha," is keyed to something that the data says is unreliable. Whatever insight is, it isn't the feeling of having it. That's a relief, because a system of states and moves doesn't have feelings,

and a word that only meant a feeling would be cheating if I hung it on one. The version this book can price has to be keyed to an *event*, and the question is which.

10.3 What happens when a computation learns

I'll lay out the split first, then say what the toll does with each part, and then say who decided the split. Keep Section 2's sheet of paper in front of you: the NO box, the YES box, the tally line, and the three moves STAY, WORK and VOUCH.

The first thing: *raw observation*. Something comes in and gets written down. A counter ticks. Nothing is claimed about what it means, nothing is marked as resting on it, and the reading doesn't move because of it. Later moves can read what it wrote, but that reading is a move of its own. The demon's pad in Section 5 was raw observation, and raw observation is what the statistician Dennis Lindley measured in 1956 as the information provided by an experiment, the words of his title, and defined as the expected drop in your uncertainty: a real quantity, and a quantity about what was *seen*. It can cost nothing on the ledger. The toll says nothing at all about a move that doesn't cross from NO to YES, so a price list may charge it zero: on the paper, STAY and WORK cost nothing, and Section 5's watcher narrowed a whole row of candidates without paying anything. "Can" is the word that matters. A price list is allowed to charge for looking anyway. And in the physical world a look takes something too: copied onto a blank page it merges nothing and owes no floor under Landauer's premise, but it can dissipate under the stated thermal-model assumptions, and it uses up the page, which means taking states the machine hasn't used, borrowed until the page is wiped (Section 5).

Lindley's measure has a back story, and it runs through Bletchley Park. I. J. Good, who was Alan Turing's main statistical assistant at Bletchley Park in 1941, wrote in 1979 that Turing had worked out the expected weight of evidence an experiment would give, to judge one of the codebreaking procedures before running it, and so partly anticipated Lindley by about fifteen years, in work that stayed secret. Turing measured evidence in *bans*, logarithms to base ten, with a tenth of one called a deciban, and the name came from the town of Banbury, where tens of thousands of the sheets used in that procedure were printed. It's the same base-ten unit other people call the hartley, after the Hartley from the last section. And Good adds that Turing didn't know the logarithm he was using had already appeared in 1878, in a paper by Charles Sanders Peirce, the same Peirce from Section 6, who'd called it the weight of evidence. Peirce is as far back as Good takes it. It changes nothing in the argument, but it shows how well trodden this first kind of thing is. For a century and a half careful people have measured what an observation tells you, and every one of those measures is about what was seen. The toll leaves that side of the line alone.

The second thing: *the raise*. Some move turns the reading from NO to YES, and that move is where the toll applies. In any system with the five parts that obeys the toll, that move costs at least one, and every run that starts at NO and ends at YES pays at least one somewhere along the way, which is Section 2's universal floor. What's being charged for, at its best, is exactly the Gestalt thing, a relation committed to: one tool, claimed, out of two sticks. I say "at its best" because Section 2 was careful about what YES means: the flag is up, and nothing more. A crossing that writes down which relation it's committing to carries Sultan's tool with it. A bare crossing, VOUCH on the paper, commits to no relation at all, and it pays exactly the same.

The third thing isn't a kind of event. It's an observation about the floor: it's *the same*, whatever is certified. Say the YES state also shows something striking, a deep relation, a result nobody could have guessed, Sultan's two sticks become one tool. The toll charges the crossing at least one. It doesn't charge the striking crossing more, because the floor's proof never looks at anything but the reading before and after. On the paper, VOUCH from NO makes one mark whether you were thinking of Duncker's box or of lunch. This toy charges one either way. The general toll supplies the same lower bound for both; an actual price schedule can charge them differently.

The candle box has a footnote that says the same thing from the other side. In 1962 the psychologist Sam Glucksberg ran Duncker's problem with money on it: some people were told they could win five or twenty dollars depending on how fast they solved it. With the tacks in the box, the people chasing the money did worse than the people who weren't. With the box empty the problem was easier for everybody, and the accounts I've read don't agree on what the money did there, so I'll leave that half alone. Paying more didn't buy the insight, and on the hard version it got in the way. That's a fact about heads. It says nothing about records, and I won't push it further than that. But it makes a nice parallel to the paper. On the paper a deeper crossing isn't charged more, and in Glucksberg's room a bigger prize didn't bring the crossing sooner. In neither place does the price measure the depth.

I'll state them exactly now, for any Thiele machine (Definition 2.2). A run is still just a list of moves, carried out in the order written. The question is which of those moves crosses from no to yes, and what that crossing lets you conclude about the total charge.

Definition 10.1 (Raise, non-raising move)**.** In a Thiele machine, a step from state s by move i is a *raise* when it turns the reading on: $\mathrm{cert}(s) = \text{no}$ and $\mathrm{cert}(\mathrm{step}(s, i)) = \text{yes}$. A move i is a *non-raising move* when no step by it, from any state, is a raise. Raw observation, the first thing above, is one kind.

Remark 10.1 (What the toll prices, and what it doesn't)**.** All three read straight off Definition 2.2 and Observation 2.2, with Section 2's board as the example. (1) The toll puts no bound on the cost of a non-raising move; on Section 2's board STAY and WORK are non-raising moves and cost 0. (2) Every raise is charged at least 1, by the toll, and every run from a state reading no to one reading yes has total cost at least 1, by the universal floor. (3) The floor is the same: the floor's proof uses nothing about the end state but its reading, so it gives the same floor of 1 whatever else the end state shows. A higher floor needs more hypotheses than the toll, and those hypotheses do the work. A system that wants to charge more for a deeper certification has to write that into its price list, move by move; the toll doesn't do it for anyone.

So the account comes down to two facts and a warning: looking can stay off the ledger, turning the reading on can't, and the toll alone does not charge more for a remarkable claim, though another pricing rule or an additional requirement could.

Who decided the split? The costs alone didn't. The toll prices exactly one thing, the crossing. A price list is allowed to put a floor under more moves than that (a look, a send, a check) and allowed to leave other moves at zero, so the sorting into non-raising moves and raises is by whether the reading moves, and that sorting is this book's. The part that's *forced* is the floor under the crossing, once you grant a reading that stays up, finitely many states that read yes, and a price on every merge; Section 3 is the argument; anything a price list charges beyond that is a choice, and any concrete machine owes you its whole table so you can see which choices it made.

10.4 Test, then commit, and nothing forces the second to follow the first

The philosopher Karl Popper said a theory earns its standing by being *exposed*: you make a conjecture, you try to kill it, and what survives a real attempt counts for something, where "real" means the test could have failed. His word for that earned credit is *corroboration*. I lean on that idea all through this book (Section 14 gives it its full due), and this subsection shows where a Thiele machine has a version of it and where that version is weaker than Popper, which I have to say plainly.

Add a fourth move to the paper: CHECK. It looks at something (a sum, a coin under a cup, a formula against a row of values), and it can come out pass or fail, and it doesn't move the box. Charge it whatever you like; charging it on the attempt, pass or fail, fits Popper, because what made his test count was that it was really run and could have failed. CHECK is the test, and it's a non-raising move in the sense above, because it never turns the reading on. VOUCH is the commit. So the honest

protocol is two moves: test, then commit, and the costs are separate, one for the checking work and one for the crossing.

But nothing in the five parts makes the second move wait for the first. The step is handed the current state and a move name; the toll looks only at whether the reading crossed. Play VOUCH from NO with no CHECK anywhere before it, and you land in YES and make one mark, the same box and the same mark as the player who checked first. The two tally lines differ by exactly what the CHECK was charged, and if you charged it nothing they don't differ at all. The mark for the crossing is the same mark either way. The receipt says a commitment was paid for; it does not say the commitment was earned. Popper's corroboration needs the test; the toll doesn't.

You can build a system that makes the commit wait. Let CHECK leave a mark in the state when it passes, and write the step so VOUCH only crosses when that mark is there. Making VOUCH wait like that is a design, written into the step function, and it's a perfectly good one. Nothing in the five parts or the toll asks for it. The gap between the word and the bare system is real, and I'm leaving it in plain sight: the toll prices the commit, and whether the commit was preceded by a test is a fact about the run, or about a design somebody chose. The five parts don't guarantee it.

Remark 10.2 (Nothing makes the crossing wait)**.** Read off Definition 2.2. The toll constrains cost(i) for moves that cross, and nothing else; the step function is any function of the current state and the move. So a Thiele machine may have a move that crosses from a state no check has touched, and the toll still holds. On Section 2's board extended with CHECK, the run $[\text{VOUCH}]$ from NO crosses with one mark, and $[\text{CHECK}, \text{VOUCH}]$ crosses with one mark for the crossing plus whatever CHECK was charged, one mark in all if CHECK is priced at zero. Making a crossing depend on an earlier check is a condition on the step function, which a particular system can impose and the definition does not.

10.5 What the word means

Insight, in the sense this book prices, is a distinction made to stay: a move that flips a reading from off to on, for good, and that move costs at least one, in any system with the toll, whatever the reading is about and whatever its depth. At its best the reading is a relation committed to, a claim checked and closed; a bare crossing that commits to nothing pays the same, and the toll doesn't tell the two apart. Part III's certified machine is one way to make sure a crossing is the first kind. There are three boundaries, each from a different place. From Metcalfe: it isn't a feeling, and a system's version has to be an event. From the proof: the floor is the same; a shallow commit and a deep one both cost at least one, and cost is not depth. A system can pay heavily to check a trivial claim, and Duncker's box, a real insight, is still one

small commit. And from the two-move protocol: a paid flag proves nothing on its own. Price and correctness are separate books.

Which leaves a suspicion I'd have in your place. Everything so far has been one reading on one sheet of paper, with a rule I chose and a count I bolted on. Is the shape of that record something I picked, or is it what any base-driven record has to look like, on any machine at all? The next section answers it: take *any* machine that steps, bolt a record onto it, and see what shape it's forced into. How a concrete machine hangs actual claims on its parts, a checked property of one counter at one edition, and where those sit in the state, comes in Part III (Section 15).

CHAPTER 11

Records over any base: the latch

The blank cell comes back here, over any machine: a reading that goes from no to yes because the machine did something, and stays. For the whole book so far I've been building up a thing with a flag and a counter, and you'd be right to suspect that the whole apparatus is special pleading for one machine, the one I keep promising to build. So this section answers the suspicion with no particular machine in it at all, and sees what's left. Take *any* machine that steps deterministically, anything at all with states and one next state for each. Call it the base: the part of the state the record watches. Now put a record over it, under rules I'll state, and ask: what can the record be?

The answer is one shape, and I'll show you the shape first because it's a thing you can build from a cup and a die.

11.1 A cup and a die

Put a cup upside down on the table. The rule is that the first time a coin lands in front of it, you turn it right side up. There is no rule for turning it back over. Now roll a die, and put a coin down on every six. Roll twenty times. If a six appears, the cup turns over on the first one and stays turned over. Write down which roll did it. That number is where the record happened, and the cup afterward is the record. If no six appears, the cup stays down; a reachable event need not occur on every trial.

For the deterministic base in the theorem, write the rolls down first and replay them. The base state is the fixed list and a finger pointing to its next entry; a step moves the finger one place, regardless of the cup. The event asks whether that entry is six. Nothing about the cup changes which entry comes next.

The cup is a *latch*. Electrical engineers have had the circuit since 1918, when two British physicists teaching engineering in London, William Eccles, a radio pioneer, and Frank Jordan, applied for a patent on a two-valve arrangement that holds whichever state you last put it in; it became the flip-flop, and the registers and caches in the device you're reading this on descend from it. Cut its reset wire and what's left has one input, "set," and the rule *new state = old state or set*: once set, it stays set. The cup is that circuit: a flip-flop with its reset wire cut off.

(Eccles and Jordan didn't call it a flip-flop. Their 1919 paper calls it a *trigger relay*. Where the newer, sillier name came from I can't source, so I'll leave it at that.)

Is a flip-flop with its reset wire cut off only a picture, or did anybody ever build one on purpose? Somebody did, and for a reason that's very close to this book's. In the mid-1950s Wen Tsing Chow, an engineer at the Arma division of the American Bosch Arma Corporation in Garden City, New York, was asked by the United States Air Force, in the accounts of his career I could find, for a more flexible and secure way to hold the fixed numbers, the targeting constants, inside the guidance computer of the Atlas missile. What he and his Arma colleague William Henrich patented (filed in 1957, granted in 1962, under the plain title "Storage matrix") was a grid with a connection at every crossing, where you set the numbers by taking out the connections you didn't want, physically or by burning them out with too much current. The word *burn* is right there in the patent, and people still say they burn a chip. The descendants are called programmable read-only memories, PROMs, and a fuse blown in one stays blown: there's no current you can send to grow it back. The grid was a whole memory built out of cups with no reset wire, made because somebody wanted numbers nobody could quietly change later. That doesn't prove anything here, because the theorem below doesn't care what a cup is made of. But it's good to know the shape wasn't invented for a sheet of paper. Somebody needed it, for stakes a good deal higher than a die.

Now the claim, stated loosely and then exactly: *every base-driven record, over any base, is a cup with no reset wire, turned by some event the base passes through.*

11.2 What "base-driven" has to mean

The content is in the definition, and I'll give you all five rules, because the theorem is only as strong as they are.

A machine *extends* a base when there's a map from the bigger machine's states down to the base's, and that map starts in the base's start states, reaches every one of them, moves in step with the base (one step up top is one step down below, through the map), and agrees about halting. The bigger machine may carry as much hidden state as it likes beyond the base, and that's what the words "any base" are about: the base doesn't get to see the extension, and the extension doesn't get to change what the base does.

The extension carries a *record*, a yes/no reading, and a *ledger*, a counter. It's a *base-driven record* when five things hold. One: the record's next value depends only on the base's current state and the record's current value. (I'll call this *driven by the computation*. It says the record answers to what the machine is doing. A clock on the wall or someone's hand doesn't get a say.) Two: the ledger never goes down (Section 7). Three: the step that turns the record on costs at least one;

that's the toll (Section 2). Four: once on, the record stays on; that's permanence (Section 3). Distributed-systems people have a name for a predicate like that. The computer scientists K. Mani Chandy and Leslie Lamport, in their 1985 paper on taking a consistent snapshot of a distributed system, called a property *stable* when once it holds it holds forever after, and stable properties are the ones a snapshot can be trusted to detect. Five: some run from a start state actually reaches a step that turns the record on; otherwise the record is decoration.

I'll write the rules out exactly. On/off is true/false, $\vee$ is "or" (on when either side is on), and next^k is k steps.

Definition 11.1 (Base, record-carrying machine, cover)**.** A *base* B is a set of states with a step function next_B, a set of starting states, and a halting predicate. A *record-carrying machine* M is a set of states with a step function next_M, a set of starting states, a *record* rec from states to Booleans, a *ledger* μ from states to whole numbers, and a halting predicate. The cost of a step is $\text{cost}(m) = \mu(\text{next}_M(m)) \dot{-} \mu(m)$. A *cover* of B by M is a map π from states of M to states of B that sends starting states to starting states, has every starting state of B as the image of some starting state of M, commutes with the steps, $\pi(\text{next}_M(m)) = \text{next}_B(\pi(m))$, and agrees about halting: m halts exactly when $\pi(m)$ does.

Definition 11.2 (Base-driven record, latch)**.** M with cover π is a *base-driven record* over B when (R1, driven by the computation) there is one function f such that, for every state m, $\text{rec}(\text{next}_M(m)) = f(\pi(m), \text{rec}(m))$; for every state m, (R2, ledger) $\mu(m) \leq \mu(\text{next}_M(m))$; (R3, toll) if $\text{rec}(m)$ is off and $\text{rec}(\text{next}_M(m))$ is on, then $\text{cost}(m) \geq 1$; (R4, permanence) if $\text{rec}(m)$ is on, so is $\text{rec}(\text{next}_M(m))$; and (R5, a reachable write) some starting state m_0 and some k have $\text{rec}(\text{next}_M^k(m_0))$ off and $\text{rec}(\text{next}_M^{k+1}(m_0))$ on. An *event* is a function h from base states to Booleans, and its *latch* is the step $(b, r) \mapsto (\text{next}_B(b),\ r \vee h(b))$. *$M$ factors as the latch of h* when, for every state m (reachable or not),

$$(\pi(\text{next}_M(m)),\ \text{rec}(\text{next}_M(m))) = (\text{next}_B(\pi(m)),\ \text{rec}(m) \vee h(\pi(m))).$$

The theorem says that under those rules the base state and the record move *exactly* as a latch: the record switches on at the first base state where some event holds, if it started off, and never goes off, and the event is read off the extension's own rule by asking it what it does when the record is off (stated exactly below). The theorem is about what the base and the record do together. The hidden state the extension carries can't push the record anywhere (R1) doesn't allow, since the record's next value is a function of the base state and the record's own value. The theorem doesn't say the whole extension is a bare latch machine, and it doesn't make hidden state show up anywhere in particular: an extension can carry a field

that touches neither the record nor the ledger. And the event is pinned down only on base states that some state with the record off covers; on a base state with no such state above it, $f(b, \text{off})$ is whatever (R1)'s function says, and nothing in a run ever consults it.

I promised the content was in the definition, and the proof bears that out: it is a short case split on whether the record is already on, and it uses only *two* of the five rules, "driven by the computation" and "permanent." The ledger, the toll, and the reachable-write clauses play no part in it. A latch is what you get from "answers only to the base" plus "never comes back down" alone. The other three rules matter for the *converse*: for any base and any event the base can reach, the latch of that event, priced at one unit per write, *is* a base-driven record. The event can be one that holds everywhere, and then its latch is up after one step from every start and tells no two starts apart. It's still a latch and still base-driven: the converse doesn't ask the record to distinguish anything. A record that starts off and tells two starts apart at some step has to latch an event that has held along one run by then and not yet along the other, and the latch tells them apart for as long as that stays so. Put the two together and base-driven records are exactly latches: one shape, and no others, over any base.

Observation 11.1 (The record is a latch)**.** *Let B be any base, π a cover of B by M, and suppose M's record is driven by the computation (R1), through a function f, and permanent (R4). Then M factors as the latch of the event $h(b) = f(b, \text{off})$. In particular every base-driven record factors that way; (R2), (R3) and (R5) are not hypotheses.*

Proof. Ask the extension what it would do with the record off above each base state b. Call its answer $h(b)$. Rule (R1) lets you ask that as one function of b: no other hidden field is allowed to change the answer.

Now take an actual state. If its record is off, (R1) gives exactly that answer $h(b)$ for the next record. If its record is already on, permanence keeps it on, whatever $h(b)$ says. Together those two cases say "keep an existing on, otherwise use $h(b)$," which is $r \vee h(b)$.

The cover already says that the base part takes its own next step. Put that base step beside the record rule just obtained and I have the stated latch equation. No charge or starting condition entered the argument. ∎

The latch's nearest neighbours are in verification, and they're old. The computer scientists Martín Abadi and Leslie Lamport, then at Digital's research lab in Palo Alto, added *history variables* to specifications in 1991: extra state that records what has happened and never steers the system, used to prove that one specification implements another. A history variable that only ever turns on is the cup. Temporal logic has the same record as an operator. In the past-time temporal logic that Orna

Lichtenstein, Amir Pnueli and Lenore Zuck set out in 1985, "once h" holds now exactly when h has held at some point so far, and a runtime monitor keeps it in one bit with the update $r' = r \vee h$. Klaus Havelund and Grigore Roşu, two computer scientists working on runtime verification, synthesized monitors of exactly that kind for safety properties written in that logic in 2002. That update is the latch of Definition 11.2, word for word. The observation adds the other direction: a record driven by the computation that never comes down can't be anything else.

One thing about that proof deserves a second look, because it cuts both ways: it never asks what the base is. The cover decides that: whatever the map sends down is base, and the record's rule is allowed to read all of it; whatever stays up top is extension, and (R1) keeps the record's rule from reading it. Draw the line low, so the base is only the computation's own working state, and (R1) is a real demand. Draw it high, put every table and every refusal bit into the base, and (R1) is easy to meet, and the latch shape comes out exactly the same. So the theorem holds wherever you draw the line, and that's exactly why it can't tell you where a record's meaning lives. The meaning lives in what the event h reads, and that's in the base by construction. On the small machine of Part III (Section 15), the checks that give the flag its meaning sit in the base its cover picks out. So when that machine arrives, ask it first which parts it puts above the line.

□ **Theorem 11.2** (Every reachable latch is a base-driven record)**.** Let B be any base and h any event. Let Latch_h have states (b, r, x), starting states $(b, \mathrm{off}, 0)$ with b starting in B, step $(b, r, x) \mapsto (\mathrm{next}_B(b),\ r \vee h(b),\ x + \delta)$ with $\delta = 1$ when r is off and $h(b)$ is on and $\delta = 0$ otherwise, record r, ledger x, halting as b halts, and cover $(b, r, x) \mapsto b$. Suppose some starting b_0 and some k bring Latch_h from $(b_0, \mathrm{off}, 0)$, in k steps, to a state whose record is still off and whose base state satisfies h. Then Latch_h is a base-driven record over B.

Proof. The construction keeps a base state, a flag and a tally. Looking only at the base state gives the promised cover: a start lies over the chosen base start, every base start has such a flag-down, tally-zero start above it, and each step advances the base exactly once. Halting is read from that same base state.

The next flag consults only the present flag and the base event, giving (R1). The tally adds either zero or one and never falls, giving (R2). If the flag rises, it was off and the event held, exactly the case that adds one, giving (R3). An on flag stays on under the displayed rule, giving (R4). Finally the supplied run reaches a state with the flag still off and the event true. Its next step raises the flag, giving the reachable write (R5).

Whenever a base run reaches the event at all, use its first such visit. Before that visit the flag remains off, so this is the supplied kind of run. ■

What about a base that never merges, a reversible one? It can carry a record, and the next theorem says exactly what it pays. Keep every earlier record value in a history beside the latch, one more entry on every step, and the whole step merges nothing either. But that history needs a fresh slot at every step, forever, so the machine never comes back to a state it has been in, which puts it in the fresh-state column. On a finite state space there aren't fresh states to go on taking forever, and a permanent record can't be written by a step that merges nothing at all. A step that merges nothing is called *injective*: no two states go to the same next state. Keep the old readings on a strip, newest first. If the strip reads $[\text{off}, \text{off}]$ and the current reading is on, the next strip reads $[\text{on}, \text{off}, \text{off}]$. The notation $r :: H$ below means exactly that: put the old reading r at the front of the old strip H.

Theorem 11.3 (Records on reversible machines)**.** (1) Let B be a base whose step is injective. The machine with states (b, r, H, x), step $(b, r, H, x) \mapsto (\text{next}_B(b),\ r \vee h(b),\ r :: H,\ x + \delta)$ (δ as in Theorem 11.2), starting states with r off, H empty and $x = 0$, has an injective step, never visits the same state twice on a run (the history gets one entry longer on every step), and under the supposition of Theorem 11.2 it is a base-driven record over B. (2) With finitely many states reading on, a single injective step with a permanent reading never turns that reading from off to on. □

Proof. (1) Suppose two full states have the same successor. Can you recover what they were? Their successor base states agree. The base keeps different sources apart, so their earlier base states agreed too. Their new history strips also agree. The first entry of each strip is the old record, so the old records agree; removing that first entry leaves the old histories, which therefore agree as well.

Those equal old base states and records determine the same charge δ. Subtract it from the equal new tallies and the old tallies agree. Every part of the two earlier full states agrees, so they were the same state. The step is one-to-one.

On a run the strip grows by one entry every tick. A full state cannot recur, because its strip would have to have two different lengths. Carrying that strip changes neither the base nor the flag nor the tally rules checked in the preceding construction, so all five base-driven record conditions still hold under the supplied reachable-event premise.

(2) Suppose instead there are only K on-states and an off-state enters them. Permanence sends all K on-states into that same collection, and the entrant lands there too. A one-to-one step would have to put $K + 1$ different sources into K places.

It cannot. Hence a permanent reading with finitely many on-states cannot rise under a one-to-one step. ■

So a base that never merges can carry a record without merging anywhere only by paying in fresh states: once the record is up, the run never comes back to a state it has been in (Proposition 3.5). A history that grows by one entry every step is one way to supply them, and a base that never repeats a state, a counter, supplies them already. A machine with finitely many states hasn't got fresh states to go on taking, so it can't carry one without merging: on a finite machine an instruction that first raises a permanent reading must merge some possible states. The entering state need not belong to the colliding pair; Section 3 prices the instruction under merge pricing.

Process-calculus people ran into the same thing from the other side. Vincent Danos and Jean Krivine, two theoretical computer scientists in Paris, wanted to pin down what a transaction is. In 2004 they had built a reversible version of CCS, Robin Milner's calculus of processes that talk to each other, where every step can be rolled back, and in 2005 they modelled a transaction in it as a process that wanders freely through everything that can be undone until it performs an action declared irreversible, which is the commit. The way I read it, they start from steps that all come undone and ask what has to be added before anything sticks, and the theorem above starts from a record and asks what the step under it must be. In their calculus, as in the theorem, a permanent effect needs a step with no way back, and the calculus has to say which steps those are. The word I tripped on was *declared*. Theirs are labelled. Mine, I thought, are picked out by counting, since on a finite machine the first write merges whatever anyone calls it. Push on that and it gives. I choose the event h, and on a base that grows its history forever the write merges nothing, because it pays in fresh states instead (Theorem 11.3), so there my permanence is declared too.

There's an arrow of time in that theorem, and I should say where it came from, because I put it there by hand. Every starting state has the record off. That's the only reason a raised record means the event happened: it was down at the start, and only h lifts it. Turn the cup up before the first roll and it tells you nothing about the die. So the clean start is an assumed ready state, a past hypothesis in miniature. The past hypothesis is David Albert's name for an assumed low-entropy beginning of the universe, which several of the accounts below lean records on. Whether every trusted record leans on it is their argument, and I'm not settling it. Nothing in this book derives one; every theorem here that reads a raised record as news that the event happened assumes its small version and starts from there. The latch observation doesn't need it, and that's why on its own it can't say the event happened.

When I went looking for who else had found the arrow hiding in the start, I got five papers that read like five unrelated answers. They lined up once I asked each one the same thing: what does the cup being down at the start rest on? There are two places to put it. You can put it at the beginning, as a fact about how things started. Or you can put it in the structure of what causes what. David Wolpert, then a physicist at Los Alamos and the same Wolpert as the free-lunch theorem, sorted memory systems in 1992, and one of his kinds works only because it begins in a prepared state set up by a process that runs one way, which is how it leans on the second law. That's the cup put down by somebody before the game, at the first place. Wolpert and Jens Kipper, a philosopher at Rochester, put in 2024 that sorting on a formal footing, and they say plainly that they have no formal proof this kind can only remember the past. I trust them more for that sentence. What I think they're saying is that a prepared start buys you a memory, and getting its direction too takes something they can't yet prove.

Aram Ebtekar, an independent researcher in Vancouver, and Marcus Hutter, who works on the theory of general artificial intelligence at Google DeepMind and the Australian National University, built in 2024 a reversible toy world, imposed a past hypothesis on it, coarse-grained it, and got records of the past and none of the future. As I read their route, the clean start doesn't go straight to the records. It goes through a causal structure first, and the arrows come out of that. Thomas Blanchard, a philosopher of science at Cologne, got in 2024 the same lopsidedness of knowledge out of the causal Markov condition, a condition on causal structure. So Blanchard starts at the second place, which is the middle of Ebtekar and Hutter's road. That took me a while to see, because one is a toy world and the other is philosophy of causation. They go from the start to causes to records; he goes from causes to records. The fit is my reading, and it's mine to defend. And Carlo Rovelli, two years before his paper on the cost of choosing from Section 5, studied traces in "Memory and entropy" (2022) and concluded that a memory turns low entropy into available information. He's at the first place, and he says what the beginning gets spent on. Read against the latch, that's the blank the record uses up: the flag can say yes once because it started at no.

So does the latch explain why records point backwards? It doesn't, and I'll say that plainly so it doesn't look like it does. The step runs forwards and the runs start clean, and a theorem built on those two can only hand the arrow back. Push further and it gives in a second place. The history that grows forever looked like a way around the clean start. It starts empty, so it leans on the same blank. Under a step that forgets nothing, on a finite set, no permanent record can ever be written, past or future, which is Zermelo again. On the map the book sits at the first place, with Wolpert and Rovelli, and it sits there by assumption. Ebtekar and Hutter at least

imposed their start on a world with its own dynamics and watched what came out. I wrote mine into the definition.

Constructor theory, which Section 3 brings in as a neighbour of the counting argument, has a paper on time (Deutsch and Marletto, 2025) saying that laws stated as which transformations a device can and can't bring about can't mention time at all, so duration has to come from relations between physical systems, clocks, that have their own dynamics. What I think they're saying is that whenever you want a duration you point at a second system and count what it did. This book is in that position by construction. The step count is the only time in it, and it comes from the machine's own steps. The next subsection has a record turned at the fifth tick of a hidden clock, and (R1) throws it out, which read beside them is a rule about which system may be the clock: the one being recorded. Push on it, though. Their laws have no step in them, and the paper's work is getting duration and dynamics out of clocks and timers. I put a step, next_B, into the definition of a base. So I'm in their position only at the far end of their argument, where a clock already exists. The base is my clock, and I assumed it.

Physicists have built this latch on paper too, in quantum mechanics, and it behaves the way the last theorem says. The question that sorted them for me is where the cup puts what it forgets, since a closed quantum system with separate energy levels comes back (Bocchieri and Loinger, Section 3) and the theorem left only two ways to pay for a record that stays: fresh states, which here mean a history that never ends, or a merge. Klaus Hepp, a mathematical physicist at ETH Zurich, wanted a measuring device inside quantum mechanics itself, and studied in 1972 a particle that flips spins in a long chain as it passes, now called the Coleman-Hepp model (I know Hepp's paper only from accounts of it), and John Bell (1975) took it apart: the record is exactly definite only for an infinitely long chain, which is the fresh-state column above, a history that never ends, in quantum clothes. Jonathan Halliwell, a theoretical physicist at Imperial College London, used in 1999 a two-state detector that flips once when a particle crosses a point and is kept from flipping back by a large environment, which is a latch with the environment holding the history. His abstract calls it *effectively* irreversible, and I like that he puts the word *effectively* right there where you can see it. And Murray Gell-Mann, the physicist who named the quark, and James Hartle, of the University of California at Santa Barbara, gave in 1995 the property a name inside their framework of decoherent histories: the permanence of the past, past alternatives that stay settled as the story is extended into the future. They name (R4) for whole histories and leave the cup to others. So Hepp's chain, as Bell read it, is exact and infinite, and Halliwell's detector is finite and only effectively permanent. Mine is finite, exact and permanent at once, and it gets there by letting the first write merge. Push on that and it gives: I never say where the merged state goes. Hepp's chain had to be infinite. Halliwell had to

put his in an environment. Mine just goes, and the toll writes a unit in a counter. That's fine in a model of computation, and it's exactly where this stops being physics.

11.3 Both rules do work: the toggle and the timestamp

A theorem with two hypotheses should have two counterexamples, one for each hypothesis dropped, and it does, and they're the two ways a record goes wrong in the world.

Drop permanence. Take a base that just counts, and a record that flips at every step: on, off, on, off. It's driven by the computation; it's not permanent; and it's not a latch of any event. Such a record is a *toggle*, a piece of working state. A record you can revoke is working state wearing a record's clothes, and if a revocation has to stick, that's a *second* latch ("revoked"), which is how certificate revocation lists actually work: you add an entry; you never delete the certificate. Alin Tomescu, a computer scientist then at MIT, and his coauthors (2019) built append-only authenticated sets and dictionaries for transparency logs, and revocation fits that shape exactly: a revocation goes in once and stays. The pair lemma (stated below) covers two latches that can each read the other; do it with two cups where the second only turns over on an even roll *if the first is already up*, and you'll see the second can never turn before the first.

Dialogue theorists built a version of that two-latch rule for arguments. Give each speaker a row of cups, one for every claim they've made. Everybody I read in this field is answering two questions about those cups: whose hand turns them, and can they come back down. C. L. Hamblin, an Australian philosopher and logician who also came up with reverse Polish notation for early computers, wrote *Fallacies* (1970), and it has a chapter on formal dialectic where each speaker has a *commitment store*, a list of what they're on the hook for, and the rules of the dialogue update it, while the speaker doesn't. I know that chapter from other people's accounts of it. Hamblin's store is an answer to the first question, and it's (R1) for arguments: the cup answers to what happened in the dialogue, and the speaker's hand is kept off it. Henry Prakken, who works on AI and law at Utrecht, built in 2005 dialogue games for argumentation on stores like that and made commitments sticky: a claim can be retracted only when it has been challenged, and claims and backtracking carry a price. He also floats a convention that once a player has retracted a proposition, that player can't take it on again, which is a revocation that stays revoked, the way an entry on a revocation list does. Prakken's rules are an answer to the second question, and it took me a second pass to see that they're two answers: a cup can come down, and coming down costs, and once it has, the taking-back is itself a cup that stays up. Erik Krabbe, an argumentation theorist at Groningen, names in 2001 the awkward part of that: retraction is frowned on, and yet resolving a dispute takes

somebody retracting a doubt or a point of view, so the rules have to say when taking a commitment back is allowed. As I read him, he's standing between the two ends of the second question. A store that never gives can't settle a dispute, and a store that gives whenever it's asked isn't a store. The part of all that I'd keep is Hamblin's, that the store belongs to the rules and doesn't belong to the speaker. Take the pen away from the program. And this book answers Krabbe with a price: a take-back is a second cup, "revoked," and turning it costs at least one like any other write, so a retraction is allowed, permanent and paid for, which is Prakken's convention with the toll attached. Where mine gives is his first condition. He lets a claim come down only when it's been challenged, and nothing in my latch asks why the second cup turned. The pair lemma lets the second cup's event read the base and the first cup, and nothing in it requires a challenge to be there.

The same field has a notion that looks a lot like earning. In the abstract argumentation of Phan Minh Dung, a computer scientist at the Asian Institute of Technology in Bangkok (1995), arguments attack one another, and the *grounded* semantics starts from the arguments nobody attacks and keeps adding the ones they defend. Look at how it's built: things go in and never come out, which is a row of cups turned one at a time. Pietro Baroni and Massimiliano Giacomin, at Brescia, introduced in 2007, and Martin Caminada, then at Aberdeen, revisited in 2014, *strong admissibility*: a set is strongly admissible when every argument in it is defended by others in the set that don't lean back on it, so the support bottoms out. Strong admissibility is the argumentation form of an earned claim. Above the bottom, nothing in it is in by default, which puts it at the far end from Burge's default on the testimony map in Section 8. Caminada argues it plays a vital part in the dialogue games that decide grounded acceptance. I've read Caminada's paper; Dung's and Baroni and Giacomin's I know through what later papers say about them. Notice the bottom, though. An argument nobody attacks gets in on a check that can't fail, and this book says a check that can't fail establishes nothing (Section 14). Now turn that on my own cups, because it lands. My first write needs only that h holds, and what h reads sits in the base, where the latch theorem doesn't look. I charge a unit for it, and a unit is a price, not a check. So my earned records bottom out the way Dung's do, in something let in without an argument, and the one difference is that mine pays to get in. Whether a machine's earned record certifies exactly the strongly admissible sets is the kind of statement I'd have to prove before I said it, and I haven't.

A record that's allowed to change its mind isn't lawless, though; it's somebody else's subject. Count how many times the cup may be turned. Once, from down to up, is the latch. More than once and you're somewhere else, and the people who went there sort it two ways: by how many turns are allowed, and by how long the game runs. The philosopher Hilary Putnam, then at MIT, and E. Mark Gold, a computer scientist at a research lab in Santa Monica, in back-to-back papers in

1965, looked at questions a machine answers by guessing and revising, where the answer is whatever the guesses finally settle on. Putnam's paper says outright that you're never sure you have the correct answer. That stopped me the first time. What kind of answer can't you ever be sure you have? The way I came to read it: the cup's position at any moment is the current guess, the answer is where it ends up, and nobody watching can tell when it's done moving. Yuri Ershov, a logician in Novosibirsk, sorted those in 1968 by how many revisions they're allowed (that's how his hierarchy is described; I haven't read his paper). Allow one, from no to yes, and you get exactly the sets a program can list. That level is in a modern account of the hierarchy by Rod Downey, Carl Jockusch, Timothy McNicholl and Paul Schupp, four computability theorists, and you don't need their word for it. If a program lists the set, guess no for every number and flip the guess to yes when the number shows up on the list. Going back, if every guess flips at most once, from no to yes, run all the guessing side by side and write a number down the moment its guess says yes. Those listable sets are what a permanent record raised by a search catches. Allow two, three, and so on, and every extra change buys new sets: the rungs of the ladder are all different. A record that can come back down lives on that ladder. Now put Putnam's worry on my cup, because it lands. On the bottom rung a yes is final the moment it appears, and a no never is, since down means not yet. A single cup of mine can certify that something happened and can never certify that it didn't; a final no would take a second cup, turned by finding a reason, and that's the pair of latches again. That's what the bottom rung costs, and I'd rather name it. This book stays on the bottom rung; I'm pointing at the ladder, and I'm not climbing it.

The ladder doesn't stop at finite time, and the people who followed it past infinity have names for the two kinds of record I just described. A definition that only ever adds, stage after stage, and keeps everything it has at each limit, is an *inductive definition* (the long treatment is a 1974 chapter by Wayne Richter and Peter Aczel, two recursion theorists). It's the latch run through the infinite, a cup that goes up at some stage and is still up at every stage after. Let it take things back, and keep what stays put from some stage on, and you have what John Burgess, a philosopher and logician at Princeton, called in 1986 *quasi-inductive* definitions, which he met in the revision theories of truth that grew up beside the logician Saul Kripke's 1975 theory of truth, which builds up the true sentences in stages. Burgess's paper measures how complicated the truths of those theories get. I've read its abstract, but not the place where he names them. What took me a few reads is that Burgess's rule at a limit is Putnam and Gold's rule, keep whatever the guesses settled on, applied at every limit stage along the way. That's my reading of how the two fit. If it's right, the split between a cup that only goes up and a cup that settles is the same split at finite time and past it: a permanent record is the first kind, and a toggle that's allowed to settle is the second. Joel Hamkins and Andy Lewis, two set theorists, let a Turing

machine keep running through the ordinals in 2000. At a limit a cell keeps its value if it settled, and reads 1 if it kept flipping back and forth forever, and the trick they use again and again is the toggle: flash a flag on and off every time a guess changes, and at the limit the flag is on exactly when the guess changed infinitely often. A flag that can come back down is how that machine learns what no finite run could tell it. A permanent one can't do that trick, because at a limit a cell that only ever goes up just stays up. So past infinity the toggle is the clever one: a latch tells you a thing happened at least once, and their flashing flag tells you it happened infinitely often. I threw the toggle out a few paragraphs back anyway, and I'd do it again, because before the limit its reading can't be relied on, and nobody using a machine of mine gets to wait for a limit. One finite-time fact belongs beside it, so nobody concludes too much from permanence. Hao Wang, a logician then at Oxford, described in 1957 a machine whose only write is to mark a square and which can never erase one, and it computes everything a Turing machine does. Permanence of a machine's own memory costs it no computing power, given an endless tape: each mark goes on a square nobody marked before, which is the fresh-state column again. What the book prices is a permanent reading raised by a step that has to work whether or not the reading is already up, which is a different thing. So the cost I named a paragraph ago is a cost in what a reading can claim, and computing power doesn't pay it.

Drop "driven by the computation." Take a base that counts, and a record that switches on at the fifth tick of a *hidden clock*, whatever the base is doing. It's permanent, and the computation doesn't drive it; such a record is a *timestamp*. A timestamp is not evidence: it says *when* and doesn't say *what happened*, and a record switched by an outside party or a clock records that party or that clock and leaves the computation out. Try it with the recorded die list. Keep a second tally hidden beside it, and turn the cup when that tally reads five before a move; then advance both the tally and the finger on the list. Run two trials from the same list entry with the cup down, but start the hidden tally at five in one trial and zero in the other. The next cup values disagree although everything the base shows agrees, which is the failure of being driven by the base. If the tally always starts synchronized with the list position, the position can predict the cup; the distinction matters below.

The two examples do exactly what I said they do: the toggle is driven, isn't permanent and isn't a latch; the timestamp is permanent and isn't driven. That each also keeps the other rules (the ledger is the step count, the first switch-on costs one, a run reaches it) is true by inspection.

Permanent but undriven is a timestamp, driven but impermanent is a toggle, and a record is the thing that's both. Undriven, for the timestamp, means that along the runs that actually happen the base state doesn't fix when the record turns; Lemma 11.7 below says why that's the condition, and what (R1) as written adds to

it. That's the whole discrimination, and I think it's the most useful single sentence in this book for looking at systems other than the one I build, which is why it's here.

I'll now state the pair lemma and the two counterexamples exactly. The base for both counterexamples is the counter: states are whole numbers, the step adds one, every state is a starting state, and nothing halts. Two more bits of shorthand: $\neg r$ is r flipped, and a statement in square brackets, like $[k = 5]$, reads on when it's true and off when it isn't.

Lemma 11.4 (Two records are two latches)**.** Let c_1, c_2 be two records on M, with cover π onto a base B, such that: there is one function F with $(c_1(\text{next}_M(m)), c_2(\text{next}_M(m))) = F(\pi(m), c_1(m), c_2(m))$; the ledger never goes down; a step that turns either record on costs at least one; and each record is permanent. Then there are h_1, h_2 with $c_1(\text{next}_M(m)) = c_1(m) \vee h_1(\pi(m), c_2(m))$ and $c_2(\text{next}_M(m)) = c_2(m) \vee h_2(\pi(m), c_1(m))$ for every m. □

Proof. Hold the second record fixed and ask what the joint rule does to the first when the first is off. Call that answer $h_1(b, r_2)$; it is the first component of $F(b, \text{off}, r_2)$. At any actual state whose first record is off, that is its next value. At any state whose first record is on, permanence keeps it on. Thus its rule is $r_1 \vee h_1(b, r_2)$.

Now hold the first record fixed and ask the corresponding question with the second off. The answer is $h_2(b, r_1)$, the second component of $F(b, r_1, \text{off})$. The same off/on argument gives $r_2 \vee h_2(b, r_1)$. The records may affect each other's raising condition, but neither can undo an on once it is there. ■

Lemma 11.5 (The toggle)**.** Let the toggle have states (n, r), step $(n, r) \mapsto (n + 1, \neg r)$, every state starting, record r, ledger n, cover $(n, r) \mapsto n$ onto the counter. It is driven by the computation, with $f(b, r) = \neg r$; it is not permanent; and it factors as the latch of no event. □

Proof. At $(0, \text{on})$ the next record is off, which breaks permanence; and any latch gives on $\vee\, h(0) =$ on there. ■

Lemma 11.6 (The timestamp)**.** Let the timestamp have states (b, k, r), step $(b, k, r) \mapsto (b + 1, k + 1, r \vee [k = 5])$, every state starting, record r, ledger b, cover $(b, k, r) \mapsto b$ onto the counter. It is permanent and not driven by the computation. □

Proof. Permanence: on $\vee\, x$ is on. Not driven: the states $(0, 5, \text{off})$ and $(0, 0, \text{off})$ have the same base state and the same record, and their next records are on and off, so no f of the base state and the record can give both. ■

"Driven" is a condition on every state. The timestamp makes every state a starting state, so the two states the proof compares are both ones a run can begin at, even though a timestamp started at $(0, 0, \text{off})$ would never reach $(0, 5, \text{off})$.

That last sentence does more work than it looks, so I'll state it as a lemma. Start the same timestamp in sync, at (b, b, off) only. On every state its runs reach, the hidden count equals the base, and the timestamp is exactly the latch of $[b = 5]$. (R1) still rejects it, because (R1) asks about every state, and $(0, 5, \text{off})$ is a state, though no run from a synchronised start reaches it. So (R1) as written separates the timestamp from a latch partly by how the cover is drawn and through states nobody visits. The separation I mean is the one on reachable states: the timestamp with every state a start fails (R1) even when only states some run reaches are counted, since two starting states sit over the same base state with the same record and disagree about the next one. And the latch theorem holds in that reachable form too.

□ **Lemma 11.7** (The timestamp, on the states runs reach)**.** Say M's record is *driven on reachable states* when (R1) holds for every state reached from a starting state in some number of steps. (1) If M's record is driven on reachable states and permanent, then on every reachable state M steps as the latch of some event. (2) The timestamp of Lemma 11.6 isn't driven on reachable states. (3) The timestamp started in sync, with starting states (b, b, off) and the same step, record, ledger and cover, is permanent, isn't driven by the computation, is driven on reachable states, and on every reachable state steps as the latch of $[b = 5]$.

Proof. (1) is the proof of Observation 11.1, run on reachable states only. (2) The states $(0, 5, \text{off})$ and $(0, 0, \text{off})$ are starting states, reached in no steps, and the proof of Lemma 11.6 applies to them. (3) From (b, b, off) every step adds one to both counts, so a reachable state has $k = b$ and its next record is $r \vee [b = 5]$. Over every state, (R1) fails at the same pair as in Lemma 11.6. ■

So "permanent but undriven is a timestamp" is a sentence about the timestamp with every state a start, over the cover that hides its count, and it holds there in the strong form: undriven on the very states its runs pass through. A timestamp started in step with its base is a latch on the runs it has, and (R1) throws it out only because (R1) also looks at states those runs never reach.

11.4 Is there only one such machine? No: one structure, two unforced choices.

I have to take apart the sentence "there's only one way to carry a record," because three different things get squeezed into it.

The first claim asks: given the toll and a fixed price list, is there one way to keep the books, starting at zero and adding each move's price? Yes, on every state reachable from the start; that's Section 7's initiality, and it assumes the price list.

The second claim asks: is every base-driven record over a base *the same machine*, charges and all? No, and the counterexample is embarrassing in its simplicity. Take any base-driven record, give it a new ledger that adds each step's old cost and one unit more, whatever the step is, and then walk the five rules. The record still answers only to the base and its own value (R1); the cost never touches it. The ledger still never goes down (R2); it climbs faster, if anything. The step that turns the record on still costs at least one (R3), because the toll is a *floor* and a surcharge only raises the price. Once on, the record stays on (R4), and the same run still reaches the write (R5). So the surcharged machine is base-driven, and it isn't the machine you started with: wherever the original sat still at no charge, the surcharged one pays one unit a step for sitting still. Do it with the cup and the die, and charge one unit for every roll, six or not. The cup turns over on the same roll as before, so you've changed the cost and nothing about the record. A machine that charges for CPU time is base-driven and isn't the one that doesn't. The new ledger has to sit beside the old state and not be read off it. A step's cost is the ledger after less the ledger before, so a ledger read off the state charges nothing for a step that leaves the state where it was, such as a halted machine stepping to itself, or for any step on a loop (Section 7). The surcharge is paid in fresh ledger values, the way every counter that climbs is paid for.

The same base computation and latch are visible after forgetting the new ledger. The full transition graph can be different: a free self-loop becomes an endless line of increasing ledger values. So the comparison preserves base, reading and the stipulated stopping convention, not every full-state distinction. If the ledger is also observed, these are different contracts. The construction allows a nonnegative surcharge depending on the current state in this broader accounting interface. Such a charge is not in general a price on instruction names alone. The price schedule remains an unforced choice after the base and event have been fixed.

And then there's the second unforced choice, and this one still surprises me: *which event* is the record? Nothing in the five rules picks it, beyond (R5)'s demand that it hold somewhere a run reaches; on a base whose runs pass through only one state that leaves one record, and the choice opens as soon as a run passes two states an event can tell apart. The converse above already says so, if you read it twice: for any event the base can reach, the latch of that event, priced at one unit per

write, is a base-driven record. Do it with the die. One cup turns over on the first six, and another turns over on the first one. Both are cups with no reset wire; both are base-driven. Roll a one first, and until a six turns up the two cups disagree about whether anything has been recorded. So *which permanent reading counts as the record* is a second thing nobody forces on you. The price list is one unforced choice and the event is the other, and that second choice is the question Section 18 is about: what, if anything, makes certification the event worth recording. "The ledger passed one" is an event only if the chosen base includes enough information to decide it. An extension-only ledger is hidden from a base event, just as the hidden clock was. The unforced choice is among functions of the chosen base.

□ **Corollary 11.8** (The event is an unforced choice)**.** Let B be any base and h_1, h_2 two events, each meeting the supposition of Theorem 11.2. Then Latch_{h_1} and Latch_{h_2} are both base-driven records over B. If some run from a start of B reaches a base state b with $h_1(b)$ on and $h_2(b)$ off, and h_2 holds nowhere earlier on that run, then after the step from b the same run has the first record on and the second off.

Proof. That both are base-driven is Theorem 11.2 applied twice. Along that run the second record is still off when it reaches b, since h_2 hasn't held, and the latch's own rule keeps it off, off $\vee\, h_2(b)$; the first is on after that step, whatever it was before, ■ since $h_1(b)$ is on.

Now let the cup wobble. Say whatever turns it over sometimes does and sometimes doesn't, with odds you could write on a card. Does fixing the price list fix the odds? No, and the counterexample is two cups with the same price tag and differently loaded coins. I'll set it up carefully. Let a record be a single bit that moves at random: from each value, a list of weighted next values, every weight positive, never empty, never leaving on once on, and with the write from off to on charged at least one. Call such a record a *permanent probabilistic record*. Call two such records *the same schedule* when the same moves are possible and every move is charged the same, and *the same in probability* when the weighted lists give every move the same odds.

□ **Theorem 11.9** (The schedule doesn't fix the probabilities)**.** (1) There are two permanent probabilistic records with the same schedule that are not the same in probability. (2) There is a permanent probabilistic record that no deterministic latch on its current value describes.

Proof. (1) From off, one record moves to off or on with weights 1 and 1, the other with weights 1 and 2; from on, both stay on with weight 1; both charge 1 for the write. Same possible moves, same charges, different weights. (2) The first record moves from off both to off and to on, while a latch $r \vee h(r)$ gives one value. ∎

These are finite-weight models on one bit with no base underneath; they say nothing about any particular machine.

The third claim asks: is there one state space, up to renaming, that every base-driven record-carrier has to have? No, because the extension may carry any hidden field it likes, so two carriers over one base can have state spaces of different sizes, and nothing needs them to match.

11.5 Records with more than two values

A cup has two positions, and some records have more: a counter of how many checks passed, or a level that only rises. The same reasoning goes through one threshold at a time. For each value a, "the record has reached a" is a yes/no reading, and each of those readings is a latch. What doesn't go through is squeezing the whole record into one cup. "Rises" needs one word of explanation: the values come with an order $\leq$, "is at or below," which needn't compare every pair (that's what makes it a *partial* order), and $[a \leq x]$ is the yes/no reading "x has reached a.

For example, keep separate records that an experiment was repeated and that its calculation was checked. "Repeated only" and "calculation checked only" are different places; neither includes the other. "Both" is above each. In this order, moving up means retaining what was already done and possibly adding another act. The order also has three rules: a place includes itself, inclusion carries through an intermediate place, and two places including each other are the same place. Those are the rules the proof below uses."

Definition 11.3 (A growing record)**.** Let the record take values in a set A with a partial order $\leq$ decided by a Boolean test. A record ρ on M, with cover π onto a base B, is a *base-driven growing record* when: there is one function f with $\rho(\mathrm{next}_M(m)) = f(\pi(m), \rho(m))$; it grows, $\rho(m) \leq \rho(\mathrm{next}_M(m))$; the ledger never goes down and every step that changes ρ costs at least one; and some run from a starting state reaches a step that changes it.

Theorem 11.10 (Growing records are families of latches)**.** (1) For every base-driven growing record there is an h with □

$$[a \leq \rho(\mathrm{next}_M(m))] = [a \leq \rho(m)] \vee h(a, \pi(m), \rho(m))$$

for every value a and state m: each threshold is a latch, whose event may read the current value. (2) The thresholds determine the value: if $[a \le x] = [a \le y]$ for every a, then $x = y$. (3) For a growing record, charging every change is the same as charging every threshold that switches on. (4) One latch is not enough: a three-valued record that climbs $0 \to 1 \to 2$ over a one-state base, each climb charged one, is base-driven, and no single Boolean latch with a decoding from the base state and the latch reproduces it. (5) A chain of distinct k-bit words, each word's set bits contained in the next one's, has at most $k + 1$ members.

Proof. (1) Fix a threshold a. If the old record has already reached it, growth keeps it reached. If it has not, the cup rises exactly when the next record $f(b, x)$ reaches it. Define the event to say both things: "not reached at x, reached at $f(b, x)$," or $h(a, b, x) = [a \not\le x] \wedge [a \le f(b, x)]$. Keeping an existing yes and otherwise using that event gives the stated latch equation.

(2) Suppose x and y give the same answer at every threshold. Ask the threshold named x itself. The record x has reached it, so y must have reached it too: $x \le y$. Ask the threshold named y and get $y \le x$. The order's third rule says those two records are the same.

(3) If the value stays put, every threshold gives the same answer before and after. If it grows from x to a different y, look at the threshold y. It is reached afterwards. It was not reached before: $y \le x$, together with growth $x \le y$, would force $x = y$. Thus some threshold switches on exactly when the record changes. Charging those steps is the same requirement in either wording.

(4) Keep the base at its sole state and let the record move from zero to one, then two, then stay at two. Add one to the tally at each climb. The record rule consults only its present value, keeps it growing and pays for both changes, so it meets the requirements. A single flag over that unchanging base offers a decoder only two inputs, flag down and flag up. It cannot reproduce the three different record values just visited.

(5) Lay the distinct bit words out in their increasing order. At each move to the next word, at least one bit that was zero must become one; no one may become zero. Across the whole chain there are only k bits available for those changes. There are therefore at most k moves between distinct words, or $k + 1$ words including the first. ∎

I have two cautions about that. In (1) each threshold's event may read the record's current value, so the events aren't all functions of the base alone. And (3) charges a step when it switches some threshold on: a step that switches on three thresholds at once owes at least one, and nothing here makes the charges add up per threshold.

The order need not be finite; (5) is the finite-memory bound.

11.6 What leaving the base open buys you

The reason this section exists is to answer the suspicion I started with. The record isn't special pleading for one machine. Over any base that steps deterministically, under rules you can read, a record that answers to the computation and never comes down is a latch on an event, and the proof doesn't care what the base is. What a particular machine adds on top of the latch is a *choice of price list* and a *choice of event*, and both are choices. That is why the permanent-flip theorem in Section 3 never needed the word certify: it forces a merge in whatever stays. The surcharged cup shows the first: the cost can change and the record not notice. The two cups on one die show the second: the rules are happy with either event. Any machine that meets the model will have to make both choices, and whether its choice of event is the right one is a question I'll get to (Section 18), and won't claim to have settled. Before that, a record can have more than two positions. Earlier in this section a record that rises got cut into one cup per threshold; Part I's chapter on the whole direction (Section 4) asked how much of the rest survives once a record has more places to be than up and down, and the answer was all of it.

CHAPTER 12

The shadow, exactly

Section 8 put the machine behind a window and asked what an outsider could tell. It found that two states can look the same through the window and differ in what matters, and that a price computed from the window can't be exact if the window has a collision (Observation 8.2). That was for one cup. With a whole row of cups in hand I can ask the sharper version of the same question, which is also the one I actually care about. Take the ordinary machine, the one with no cups and no ledger, and let it look at itself. What's the shape of everything it can't see, and what does the toll cost in terms of that shape?

12.1 The ordinary machine is a shadow

Take any machine with a record and a ledger, and sweep both off the table, the cups and the counter. What's left is a machine that steps the same way and knows nothing about either, provided the step's effect on everything else never depends on the cups or the counter: sweeping them off has to commute with the step. A system whose control reads its record or its ledger has no shadow in this sense. Where the premise holds, call what's left the *shadow*. Over any one state of the shadow there's a pile of full states, all casting that same shadow and differing only in their record and their ledger. Section 8 already called the pile over a point the *fibre* over it. The shadow is one particular window, the one that shows everything except the cups and the ledger.

The shadow has its own fibres, and every move has piles of its own too: the states a move sends to one landing spot. I'll call those the move's *landing piles*. A fibre is everything that looks the same before anybody moves. A landing pile is everything that ends up in the same spot after one move. Keep the two kinds apart, because the whole section turns on how they meet.

12.2 Pricing from the shadow

First the window question, over a whole row. A *window price* charges each step from what the window shows before and after it. It meets the floor if it charges at least

one for every step that turns a cup over, and it's *exact* if it also charges nothing for any step that turns no cup over. A *collision* is two steps that look the same through the window, where one turns a cup over and the other doesn't.

With one cup, the fix in Section 8 was to put the cup in the window. So my first guess for a row was that the window has to show some of the cups. Try the smallest version: a record in the whole numbers, and a window that shows only the first cup, the one that asks "at least one?" Let the record rise on some steps and hold still on others. Once it's past one, that cup is up before and after both kinds of step, so a step that turned over a cup further along looks exactly like a step that turned over nothing, and the two make a collision. So then, I thought, the window has to show every cup. That's wrong too, from the other side. Take a record in the whole numbers with two moves, one that adds one and one that leaves it alone, and a window that shows only whether the record is even, which on the row is whether an even number of cups are up. You can't read the record off that window, and no flat charge is exact, since some steps turn a cup over and some don't. But the evenness changes on exactly the steps that turn a cup over, so charging one exactly when the evenness changed is exact.

Theorem 12.1 (Window prices over a direction)**.** (1) If the window has a collision, every window price that meets the floor charges at least one for some step that turns no cup over, so no window price is exact. (2) An exact window price exists exactly when the window has no collision; on a machine with finitely many states and moves, whose views can be told apart by a yes-or-no test, it can also be computed. (3) Along any run, a window price that meets the floor charges at least the number of steps that turn no cup over but look, through the window, like a step that does. Some window price meets the floor and, among the steps that turn no cup over, charges one on exactly those, so that number is the least overcharge; when it can be decided, for each pair of views, whether some step showing them turns a cup over, that price can be computed. (4) A window can price exactly without showing where the record is: there's a record in the whole numbers with a move that raises it and a move that leaves it alone, and a window that shows only whether the record is even; the evenness changes exactly on the rises, no charge that is the same for every step is exact, and charging one exactly when the evenness changes is exact, although no rule reads the record off the window. (5) There's a record in the whole numbers and a window that shows only the one cup "at least one," and that window has a collision. □

Proof. (1) The two colliding steps get the same charge. The floor makes it at least one because of the step that turns a cup over, and the other step pays it too. (2) Without a collision, "does some step that looks like this turn a cup over?" is a question about what the window shows; charge one exactly when the answer is yes. On a finite machine you can answer it for every pair of views. (3) A step that turns no cup over but looks like one that does gets that step's charge, which is at least one by (1)'s argument; add along the run. For the other half, charge one exactly when the answer to (2)'s question is yes, which needs that question to be decidable, and count the overcharge. (4) A rise changes the evenness and the other move doesn't, so the window sees exactly the steps that turn a cup over; a flat charge either charges nothing for a rise or charges something for a step that turns nothing over; and 0 and 2 look the same through the window. (5) The first cup, as above. ■

Item (4) changed how I think about this. I'd assumed a window had to see the cups to price them. It doesn't; it has to see them *move*. A window that cannot tell you your exact place, but detects every rise and every pause, can charge the right amount every time.

12.3 What a priced move destroys

Now the fibres. Part I showed that the move raising a permanent flag on a finite machine lands two possible states on one state, and Theorem 4.4 showed the same for a step that turns any cup over, once the cups only go up. Which two? It matters a great deal. If they're in different shadow fibres, then the ordinary machine lost a distinction too; the record may also differ between them. If they're in the same fibre, the ordinary machine lost nothing, and what got destroyed was only ever off the table: two states that cast one shadow, told apart by their cups or their ledger, crushed into one.

My first guess was that it's always the second kind, because the cups are where everything happens. It isn't always. It depends on whether the ordinary machine forgets anything on its own.

□ **Theorem 12.2** (The toll prices a collapse inside a fibre)**.** On a machine with finitely many states and a growing record: (1) a move that turns a cup over somewhere sends two different states to one landing spot; (2) if the shadow exists, so that sweeping commutes with the step, and the shadow's step for that move forgets nothing, sending different shadow states to different shadow states, then those two states are in the same shadow fibre, and every landing pile of the move lies inside one fibre; (3) without that premise it can fail: there's a two-state machine whose two states sit in different fibres and whose shadow step forgets, so the merge at

the step that turns the cup over lands two states from different fibres and no fibre loses a point. (4) The halving price, which says a move of cost c shrinks any finite collection of states by at most a factor of 2^c, holds exactly when every landing pile has at most 2^c states, on a machine of any size.

Proof. (1) is the counting argument of Theorem 4.4, with growth. (2) If two states land on one spot, their shadows land on one shadow spot, and the shadow's step tells shadows apart, so the two states cast the same shadow to begin with. (3) The stamp of Theorem 4.5, with the record's value also shown in the visible part, so each state is its own shadow: both states land on yes, and so do their shadows. (4) is the worst pile-up of Premise 3.4. Any $2^c + 1$ states of one landing pile would be a collection that lands on a single spot, which the premise caps at 2^c, so no pile has more; and any finite collection is at most the number of spots it lands on times the biggest pile. Neither half needs the machine to be finite. ∎

So on a machine with finitely many states and a record that only grows, when the ordinary machine forgets nothing, the toll is a price on crushing a fibre: on taking two states that cast one shadow, which only what's off the table told apart, and making them one. The last of those premises is the one I just showed you failing.

12.4 What the shadow is

Put the two sections together and you get the shape I've been circling since the Flatland night (the night a video about *Flatland* gave me the hunch behind this book; Section 24 tells it). The ordinary machine is the shadow of a machine with a row of cups beside it. Over every shadow state there's a whole fibre, a pile of states that differ only in their cups and their ledger, and the ordinary machine can't tell them apart, on a machine whose step never looks at the cups. On a machine that obeys the toll, a step that turns a cup over costs, whichever cup it is and however many it turns, and on a machine with finitely many states whose record only grows, when the ordinary machine forgets nothing, what that cost pays for is exactly crushing two states of one fibre into one. A window can charge the right cost without seeing the cups, if it sees them move. And on a machine with the halving price, where each unit of cost pays for at most one halving, the cost in bits, times the weight of the spread, is never less than what got crushed on average.

What I haven't shown is that any of this is out there in the world, that nature keeps a direction like this over its own shadows. That's a belief of mine, and nothing in this book proves it. What I've shown, and you can check, comes with its premises showing. A machine whose record is driven by the computation and climbs a partial order is a row of cups, one latch for every line. If it also has finitely many states, never lets the record come down, and prices its merges, it has to pay this way. And

when its ordinary machine forgets nothing, what it pays for is exactly this. It's less than the Flatland night promised me. It's also the part of it that held when I pushed.

12.5 Shortcuts

There's a question every machine with a record runs into once its programs can be handed their own code. A cup is worth turning over because something later glances at it and doesn't have to work the thing out again, the way Section 6's glance reads a block's bucket off the table without applying the rule from scratch. Call any move like that a *shortcut*, and call it *sound* when it gives the answer the long way would have given. (That gloss is about cost, leaning on what was done so nothing is redone, and the wall I'm about to show you doesn't look at cost at all; the next section says exactly what it does look at.) So can a machine like that *decide*, from inside, whether its own shortcuts are sound? That question runs into a wall, the same wall Kurt Gödel and Alan Turing hit, and I have to show you that it's there before I show you anything that pretends to be on the other side of it.

CHAPTER 13

The diagonal

In Section 6 I said you'd watch the same move get made again and again as the book went on: somebody says "everything is X," it's true enough to do real work with, and then somebody draws the diagonal. I owe you the diagonal itself now, because Sections 11 to 12 just handed a record, and a whole direction of them, to *any* system that steps, and the next thing anyone wants from a system with a record is for it to tell sound shortcuts from unsound ones by itself. Before I let anything claim that, I have to show you the wall every such claim runs into, and then show you exactly where a system with a record stands relative to the wall. It doesn't stand on the far side, because nobody's on the far side. A record keeps a distinction and pays for it. What it can't do is the judging: no machine that can hand a program its own code, and can run the flip of a verdict as one of its own programs, can sort all its programs, from inside, by a question about what they do that some pass and some fail. So the record carries a commitment, and somebody still has to make it.

13.1 Now point it at the record

The question Section 12 left open can be asked with nothing in it but what Parts I and II have built. A system steps; a record rides along and latches on an event (Section 11); when the event is a certification, Section 9 said the reading, with its object and its paid event behind it, is what lets an auditor treat a claim as established, and for a claim that a shortcut is sound, that's what makes the shortcut *admissible*. Can a system like that have, as one of its own programs, a procedure that correctly decides, for every program, "does this program admit a sound shortcut"? A procedure like that would be autopilot: hand it any program and it says yes or no, and it's always right.

The exact statement is narrower than the question in plain words, so I'll give it before the proof. In the theorem, "admits a shortcut" is any property of programs with two features: two programs that behave alike agree on it, and there's one program that has it and one that doesn't. Those two features are all the argument gets to look at. The diagonal doesn't see soundness. Each system that wants the wall picks its own property, and whether that property lines up with being entitled

to soundness in Section 9's sense is a separate fact about that system, checked one system at a time if it's checked at all. A procedure that returns soundness records for some programs, or only for restricted inputs, or with outside evidence, is not what the wall rules out.

So in the theorem "shortcut" is a label. The plain-words gloss at the end of Section 12, leaning on a record so the thing isn't worked out again, is about *how* a program gets its answer, and how is exactly what extensionality throws away: the theorem only gets to see what programs do, and it treats any two that do the same as one. A question like "is there a cheaper program that gives the same answers?" isn't the theorem's question, and a wall for it would need a different theorem. The nearest thing I know of is Manuel Blum's speedup theorem, from 1967, and it goes somewhere stranger: there are computable functions with no fastest program at all, where every program for the function has another that's faster on all but finitely many inputs. Blum's theorem is a fact about cost, not a wall about deciding, and nothing in this book proves anything in that territory. Each use of the wall also names its sameness. In the substrate theorem it's the substrate's own $\simeq$; on L it's what a term computes; on the host of the small machine Part III builds, it's the partial function a program computes. A particular notion of shortcut gets the wall only after someone proves it behaves alike on programs that are the same in the sense in use, and holds of one program and fails of another.

The answer is no, under premises. The proof is the two index cards, and I'll give you the premises because they're the whole content. Suppose there were such a decider, one that says yes exactly when a program admits a shortcut. Build the flip: "if the decider says yes about me, behave like a program that admits no shortcut; if it says no, behave like one that does." If the system has a recursion theorem (a program can be handed its own code), and if that flip is one of the transformations the system can carry out, and if "admits a shortcut" depends only on behaviour (two programs that behave the same agree on it), and if there's at least one program each way, then there's a program that behaves exactly like its own flip, and whatever the decider says about it is wrong. Both cases close in a line each. This rules out a decider that says yes if and only if a program admits a shortcut, for any substrate with those parts. (A substrate here is the mathematical structure just described: programs, a run that may fail to give back a state, a counter with a step relation it never goes down along, an encoding of programs as data, a notion of "representable" transformation, and the recursion theorem for those. It says nothing about what the states are made of.) It's Rice's trick one floor over, and I should be careful with that. Rice stated his theorem for classes of recursively enumerable sets, the sets a program can list, and it's usually taught now for the functions Turing machines compute. This wall is the same diagonal with different premises: it carries its own recursion theorem and its own list of representable transformations, and it picks the

property. So it isn't a corollary of Rice as Rice stated it, and it doesn't show that any property is harder or easier to decide than classical halting. And I want you to have one thing about its proof: it never touches the counter, the toll, or the step relation. It uses the recursion theorem, representability of the flip, the two witnesses, and extensionality, and nothing else.

I'll set out the setting and the wall now, with every word pinned down so you can poke at it.

Definition 13.1 (Substrate)**.** A *substrate* has a type of states, a type Prog of programs, and a run map that takes a program and a state and either gives back a state or gives nothing. It has a counter μ on states and a step relation along which μ never decreases; an encoding enc of programs as states with a decoder dec such that $\mathrm{dec}(\mathrm{enc}(p)) = p$; and a predicate Rep on maps $f : \mathrm{Prog} \to \mathrm{Prog}$, the transformations the substrate counts as its own. Two programs are *equivalent*, $p \simeq q$, when $\mathrm{run}(p, s) = \mathrm{run}(q, s)$ for every state s. The last condition is the *recursion theorem*: for every f with $\mathrm{Rep}(f)$ there is a program p with $p \simeq f(p)$.

Definition 13.2 (Shortcut predicate)**.** A *shortcut predicate* on a substrate is a property A of programs with two programs y and n such that $A(y)$ holds, $A(n)$ fails, and A is extensional: $p \simeq q$ implies $A(p) \iff A(q)$. For a Boolean function d on programs, its *flip* is $f_d(p) = n$ if $d(p)$ is true and $f_d(p) = y$ if $d(p)$ is false.

Theorem 13.1 (No inside decider)**.** Let a substrate carry a shortcut predicate A with witnesses y and n. Then for every Boolean function d on programs with $\mathrm{Rep}(f_d)$ there is a program p with $p \simeq f_d(p)$, and $d(p) = \text{true} \iff A(p)$ fails at that p. So no such d is right about A on every program. □

Proof. Hand the proposed judge d to the flip construction. The flip is an allowed program transformation, so the recursion theorem supplies a program p that behaves exactly as its own flip tells it to behave. Now ask the judge about that particular program.

If the judge says yes, the flip sends p to the no-example n. Thus p and n behave alike. The question depends only on behaviour, and n fails it, so p fails it too. The judge's yes is wrong.

If the judge says no, the flip sends p to the yes-example y. Now p and y behave alike. Since y passes, p passes too. The judge's no is wrong. Those are its only answers.

I used the recursion theorem to obtain this one self-matching program, the two examples to give the flip somewhere to go, and the rule that alike behaviour gets the same verdict. The counter, the step relation and the encoding played no part. Any setting supplying that same fixed point for the flip gives the same contradiction. ■

So what's ruled out is a d that says true exactly when A holds, and whose flip the substrate can represent. Every part I named is a premise, and the theorem covers every substrate that supplies them all.

So the wall is there. The part I have to be honest about is *who counts as a decider*. The theorem rules out a decider whose flip the substrate can itself represent. It says nothing about a decider standing *outside*, and an outside decider can exist for the very same property, correct on every program. The line between inside and outside is the boundary of the sentence, and the boundary is Richard's lesson: the language you talk about a system in is a different language from the one it talks in.

13.2 What this does to the book

It does two things. It keeps me honest: nothing in what follows claims that a system with a record can check its own shortcuts on autopilot, because that's a wall, the same wall, and I showed you the bricks. And it tells you what a record *can* do, which is the thing Part II has been about since the counter: it can carry a shortcut *with its receipt*, so that the question "was this earned" is answered by an event you can point at. No decider has to exist for that, which is lucky, because on a system with a recursion theorem no inside one can. The diagonal says such a system can't automate the judgement from inside, for any behavioural property with a yes and a no. The latch says you can record the commitment. Those are different jobs, and a record does the second.

The diagonal has a famous cousin that looks as if it ought to bite a record too, so let me walk right up to it. Gödel's second theorem says a consistent system strong enough for arithmetic can't prove its own consistency, and Martin Löb, a logician answering another logician's question, sharpened that in 1955: if such a system proves "if this is provable, it's true," it already proves the thing itself. So can a record vouch for itself, and does Löb stop it? My first guess was that Löb stops it, since a record looked like a system talking about itself. I landed somewhere else, and mostly it's a relief. Hold two things side by side: "there's a receipt for this somewhere," and the receipt in your hand. Löb's theorem is about *provability*, the bare claim that some proof exists, the receipt somewhere. A raised Boolean flag is not, by itself, the receipt in your hand. From a clean start the earned-history theorem says there was a passing check, commitment and certification in that order. The flag does not identify the claim or the steps, and it need not store a proof of the checked claim.

The native table and channel carry more information; exporting a proof requires an interface that actually supplies one.

Sergei Artemov's distinction between implicit provability and explicit proof terms is useful here, but neither a flag nor the word "receipt" turns one into the other. The guarantee in this book comes from an external proof about the checker, the clean start and every permitted operation. It is not an internal proof of the system's own consistency, and does not evade Gödel's or Löb's hypotheses.

Push on that and it gives, because a receipt is only as good as whoever wrote it. Its trust rests on the checker meaning what it says, and that gets settled once, from outside the machine. That's where I chose to stop this machine's chain of reasons. Gödel's second theorem does not forbid a machine from checking a proof about a checker. What it limits is a system's ability to prove *its own* consistency, when the system is strong enough to do arithmetic and its axioms can be listed by a machine. Eliezer Yudkowsky and Marcello Herreshoff's 2013 draft on agents that trust their own rewrites describes a related problem, the Löbian obstacle: how an agent can justify trust in a successor's proofs. That's a cousin of this question, and it doesn't settle it, and it doesn't make my design the only one possible. So I didn't get rid of the receipt somewhere. I moved it out of the machine and handed it to whoever reads the record.

One more thing follows from the same picture, by a short argument. Can a record rest on itself, the way the sentence "this sentence is certified" would like to? It can't if it's earned and permanent. The check behind the certificate that put the record up happened while the record was still down, because the record goes up only after that check passes, and once up it never comes back down, so it goes up only once. So no claim whose truth needs the record already up can be the claim the record went up on. (A later certificate, with the record already up, can stand on such a claim; it raises nothing.) Saul Kripke's 1975 theory of truth calls a sentence like that, one that only ever leans on itself, ungrounded, and earned records are grounded in his sense: every one bottoms out in a check that didn't lean on it. Caspar Oesterheld's 2019 work on programs that cooperate by simulating each other needed a grounding step of the same kind to stop an infinite regress. I haven't written this argument out anywhere else; the claim is exactly what this paragraph says, and it's short enough to check by hand.

I want to draw one last distinction, because the two are easy to conflate and conflating them is the easiest way to misread this book. There's a *second* wall, nothing to do with the diagonal. It's Section 8's kind of wall, a window with a colliding pair in it. Two states that differ in their certification reading can look identical through a window that doesn't show that reading, the stamp inside the closed drawer and the drawer opened and shut, and then no function of the window recovers the reading (Theorem 8.1: a decoder has to give one answer on everything

the window lumps together). This wall comes from missing information. Nothing about it is undecidable; it holds for finite things with nothing to compute; and it sits *beside* Rice. None of Rice is underneath it. The property the diagonal uses (any behavioural property of programs with a yes and a no) is a different kind of thing from the reading the window hides, and nothing here says the two are the same. So there are two walls, on two different axes. The diagonal is about what no inside procedure can decide. The window is about what no outside observer can see. I'll need both, and I'll try not to call one by the other's name again. The machine I build later has a colliding pair of exactly that kind (Theorem 15.15).

PART III

Established



Checking comes before the record.

Part III is step four, the sharpest case. A record that a claim was checked and closed is a distinction that stays, so it pays what Part I found keeping takes, and it's the case where you can point at the kept 0 and at what it cost. It starts with what *established* means, and why two computers with the same bits can differ in it. Then it builds the smallest example I could find that meets the definition: a record with two places, a flag that is the blank cell, over a flat part of two counters and a line number, six kinds of instruction in all. Read through a window on the line and the two counters, it's a flat case, a computer, run by its moves for the instructions and blind to the record and the ledger. The step that raises the flag is the write of a 0, a merge in the record's part of the state, and that's why it's charged; the order the step enforces, check, then commit, then certify, is what makes this record's raise mean a claim held. That's what I mean by certification, and it's the layer this part adds to the bar Part II set. Then how far the example reaches, the real systems that already put a price on a write, the thesis about which kept distinction is worth paying for, and what the example's counter has to do with heat. Those claims are about the model; the example is where you can check them line by line. The flat part's own steps merge too (counting a counter down lands two states on one), and their physical costs depend on the implementation; the ledger leaves them alone.

CHAPTER 14

What established means

Parts I and II were about any kept distinction at all. This chapter picks one: the distinction between an answer that was checked and the same answer that wasn't. It's the case the first pages of the book called the sharpest, and it's sharp because the two answers are the same bits, and everything that differs is in what was kept.

14.1 Two computers that end up the same

You met these two computers in one paragraph at the start of the book. Now I'll take them slowly: what an ordinary computer does about what Part I found keeping takes, and why that matters to anybody who isn't a physicist.

Put two computers on a table. Hand each of them the same list of a thousand numbers.

The first one sorts it. It compares, swaps, walks the list again and again until nothing is out of order. Then it does one more thing: it walks the whole list front to back and checks that every number is no bigger than the one after it. That check could have failed. It passes. Then it tidies up after itself, wipes its scratch work, and leaves the sorted list in memory. (That check compares neighbours, so what it settles is that the output is in order. That the output holds your thousand numbers and no others is a second claim, and it would take a second check comparing the output with the input. When I say sorted below, I mean in order.)

The second one never looks at your list. Somebody typed the sorted version into it earlier, same thousand numbers, same order, and it puts them in memory.

Now wait until both are done and read the retained digital state we have chosen to compare: the list, the working registers and the control position. Arrange for both programs to finish at the same control point and clear the scratch fields. Those endpoints can agree. The first compared the entries and checked their order. The second loaded the same answer without doing either. Wheel the boxes out, swap them and bring them back. From those fields alone, neither of us can tell which run happened.

That is a construction with a specified observation. It does not establish that every microscopic state of two physical computers agrees: the room, heat, clocks or

retained logs may carry other evidence. And ordinary computers can keep a checked flag or a protected log. These two programs were arranged to erase the distinction from the fields being read. That choice, and what it prevents the next reader from answering, is the example.

Who cares? Anybody who's been handed a sheet that says "all tests passed." Anybody who's looked at an inspection sticker on a windshield: the sticker looks the same whether somebody checked the brakes or somebody sold the sticker. An audit, a proof, a ledger, an AI that sounds very sure of itself: every one of them is a pile of bits that could have come out of either computer, and the question that matters is the one the bits can't answer. Did something establish this, or did it just print it? I'll come back to every one of those later in this chapter and say which of them this book actually helps with, and I'll warn you now that it's fewer than you'd hope.

I'm going to call the first run's relation to that answer *established*: it checked the order of this list. The second run has the answer without that relation. The list's order is there either way. What can a machine keep of the difference between the runs? The rest of this chapter takes that question apart.

It belongs in a book about a 0 because a record that the check ran would be a distinction that stays, a 0 in some cell that says "checked." These two computers did not keep that record. A machine that does keep it gives you something more to inspect.

14.2 Look harder

The obvious answer is: just look harder. Don't look at the end, look at everything. So make the computers as lookable as computers get. Scale them up to the size of a warehouse, every wire a lamp that's lit or dark, every switch something you can walk up to and touch. Nothing is hidden in a chip too small to see. Put both of them side by side and walk the floor.

While they run, sure, you'd see the difference. One has a whole wing lighting up, comparing and swapping, and the other just copies. But they finish, and the comparing wing goes dark. Now walk the floor and look at every single lamp. The sorted list is spelled out in lit and dark lamps in both, the same pattern, and everything else is dark in both, because both cleaned up.

There is no lamp anywhere that says "a check ran here and passed." It isn't hidden, and it isn't encrypted; it just isn't there. You can see every retained digital field in this drawing, and none records the earlier check. Looking harder doesn't help, because the problem isn't that you can't see enough. It's that nothing was kept.

The shape I want you to carry through the rest is that the problem was never visibility; it's keeping.

14.3 So keep the trace

Then the fix is obvious, isn't it? Don't let the run go. Keep the trace: every comparison, every swap, the check and its answer, written down as it happens. Wheel the computers out on the cart and back, read the traces, and you'll know which one sorted. Problem solved?

It's the right instinct, and it gets most of the way to this book. But take it apart, because "just keep the trace" has three things wrong with it, and each one is a piece of what comes later.

First, a trace is bits, and bits can be typed in. Whoever typed the sorted list into the second machine can type a trace in too: "compared, swapped, checked, passed," a few thousand lines of it, every one of them plausible. A trace written by a program is something the program produced, unless some enforced rule constrains how the program can write that log. So keeping the trace doesn't answer the question; it moves it. Did something establish this trace, or did it just print it?

Second, the way out of that is to take the pen away from the program. Don't let the program write its own trace. Have something underneath it write the trace, one step at a time, as the steps actually happen, so nothing a program does can add a line that didn't happen or erase one that did. Airplanes do a version of this. Flight recorders go back to a prototype David Warren built in Melbourne in the 1950s. He was an Australian scientist at the Aeronautical Research Laboratories, working on jet fuel, and after a string of unexplained airliner crashes he wanted a box that would hold the instrument readings and the voices in the cockpit for whoever came looking afterwards; I know his story from museum and encyclopedia accounts, and I haven't read his own report. The recorders that came out of that idea preserve instrument data and cockpit sound. The sound can include a person's account, correct or mistaken. A recorded sensor reading can also be wrong about the aircraft. What the recording supplies is evidence of what those channels carried, subject to the recorder and sensors working as specified. Crash protection helps that evidence survive. The flight recorder has the right shape, and that shape isn't mine. Later in this chapter it gets its name from computer security, a reference monitor, and I'll say outright that a machine built that way would be doing what this book describes. What the book adds is narrower than a rulebook for everything underneath. It's about the records that go up and never come back down: what that something has to keep for good, and what keeping it takes.

Third, a fixed memory cannot retain every possible history of unbounded length with exact recovery. Give the run one independent yes-or-no choice per tick: after n ticks there are 2^n histories to distinguish. A fixed number of bits eventually runs out. A completed finite trace can fit, and a predictable trace can compress. The obstruction needs unbounded independent possibilities, not just the word "trace."

Flight recorders make a practical choice about the history to retain. A voice recorder loops, replacing older sound when its capacity is reached. Thirty minutes, two hours and twenty-five hours are different retention windows; the applicable requirement depends on the aircraft and rule. The FAA's 2026 final rule extends twenty-five-hour recording to specified newly manufactured aircraft. Whatever window is chosen, it is a window. It does not imply that no other machine can store an entire finite run.

The fingerprint is the sharpest reply, so take it seriously. Fold every new line of the trace into a fixed-size scramble of everything before it, and you've got a hash chain. The timestamps of Stuart Haber and Scott Stornetta, two cryptographers at Bellcore whose story comes later in this chapter, work that way; so do the audit logs the cryptographers Bruce Schneier and John Kelsey designed in 1999; and so do the measurement registers in the security chip a lot of computers carry, the TPM. For the PCRs and lifecycle under discussion, an extend folds a measurement into the previous value. Reset permissions depend on the register, platform and locality; some PCRs are resettable during operation. A claim about an unbroken history must specify which resets and writers its policy permits. That really does keep something about the whole trace in a few dozen bytes. But look at what it keeps: it keeps a binding, and the content isn't in it. To read the history you still need the trace stored somewhere, and the fingerprint only tells you whether a copy somebody hands you is the one that went in. The binding holds only because nobody knows how to find two traces with the same fingerprint, which is a bet on how hard a search is. No law of nature backs it. That part isn't only my opinion. The computer scientists Moni Naor and Guy Rothblum proved a sharp version of it in 2005, and they even call the thing a fingerprint: keep a small private fingerprint of a big file somebody else stores, check the file by reading only a little of it, and against an opponent with unlimited computing the size of the fingerprint times the amount you read has to be on the order of the whole file. Do even slightly better than that against an opponent with limited computing, and you've shown that one-way functions exist, which is the hardness bet in its barest form. Their setting is a stored file. A growing trace is a different setting, but it's the same bet, with its terms written down. And squeezing traces that keep growing into a fixed number of bits has to land many traces on one fingerprint, and that's a merge, hidden behind how hard it is to find which traces land on the same fingerprint. It's the same merge as in the example of Part I (Section 3) where four people have to sit in three chairs. Fingerprints are great, but they're not a way out of forgetting; they're a very clever way of forgetting.

So the real question was never "save everything." The real question is: what's the smallest thing worth keeping for good, and what does keeping it take? The answer this book ends up with is the commitment, in the database's sense, a question closed, and the trace drops away. (The cryptographer's sealed envelope also goes by the

name commitment. It's a different thing, and both come up later.) What's kept for good isn't every comparison the sort made, just the one fact that a check of this very claim passed and the question got closed. And the part that matters most is that keeping even that one fact for good takes something on every machine. It's paid by forgetting something somewhere else or by taking fresh states the machine never gets back, and on a finite machine the fresh states run out, so it's the forgetting. That's Part I's counting of states again, and Part I counted the fresh states.

So the premise was never that computers can't keep records. They keep records all day. It's that nothing in the rules of an ordinary computer makes the record paid for, permanent, or earned, and those three come in at different places, so I'll say where. Paid for is the model's one rule: in the Thiele Machine, the step that raises the record has to cost something. Permanent is a premise, the one Part I's counting of states needs, and one of the conditions Part II puts on a record. Earned, meaning a check of this very claim passed first, isn't in the model's core at all. It's what the step of the small machine in the next chapter enforces, proved for that machine and every program on it run from a clean start. One more word rides along with permanent, and it'll come up when I line my record up against other people's: dated. A record that goes up and stays up went up at one particular step of the run and not another, even though the record doesn't write down which step. So dated, in this book, means it has a place in the run's history. Carrying a timestamp is something else, and my record doesn't do it. A machine that charges one for every step it takes meets the first rule, and the charging does nothing toward the others, and I'll keep pointing that out.

14.4 Computed, stored, established

Three words get used like they're one.

Computed is about the steps. The first machine computed the sorted list. The second didn't.

Stored is about the bits. Both machines stored it.

Established is neither, and that's the whole problem. It isn't the steps, because they're gone. It isn't the bits, because they're identical. It's something like this: the first machine has a right to lean on "this list is sorted," because it ran a test that could have failed, the test passed, the test was about this very list, and then the question got closed. The second machine has the same sentence and no right to it. That's a true fact about the two of them, and it's written down nowhere, in either one. (Whose right, strictly? A machine doesn't lean on anything, any more than a filing cabinet does. It's the right of whoever reads the first machine to lean on what it says, and the machine is where the grounds for that right would have to be kept.

"The machine has a right" is shorter, so I'll keep saying it, and now you know what it's short for.)

I'll hold four words still from here on. A *check* is the test, the thing that could say no. A *commit* is the closing, the question shut for good, about that edition of that thing. The *record* is whatever stays behind to say so. And *established*, in the sense a machine can carry, is all three, about one thing. The first machine had the check and the closing and kept no record of either, which is the problem above. Where the closing and the record are one act, as in a database's commit, writing the record is the closing. The small machine keeps them apart, a commitment the next one replaces and a certificate that stays. The book cashes this working sense at three depths: a paid closing in the bare model; an object with a checked relation and a paid event, in Part II (Section 9); and the whole chain on the small machine, in the next chapter. Each place says which one it means.

That raises a fair question: what if the second machine, handed the typed-in list, runs the check on it? Then it has what the first machine had, and if it closes the question and keeps the record, it has established that the list is sorted. It didn't do the sorting and it doesn't need to. The line I'm drawing is between checked and unchecked. Whether the list was computed or copied doesn't enter into it, and "without doing either" up there was about the copier that never looked. So whatever this book ends up counting, it isn't how hard anybody worked.

People say "a right to lean on it" about a lot of things that turn out to be vibes, so I had to find out whether this one was anything. I went looking, and I'll go through what I ran into.

14.5 The clerk who forgets everything

The picture of a computer I used in Section 6 is a person with a pencil, a strip of paper and a short list of rules. So I went back to the source, Alan Turing's 1936 paper, to see how far he actually takes it.

He takes it a long way. Back then a "computer" was a job, a human doing sums by hand, and Turing modelled that person. He's careful with the person's mind: whatever the clerk knows, beyond a handful of states, has to go down on the paper. Then he goes all the way. He imagines the clerk getting up, walking off, forgetting everything, and coming back later. To pick up where he left off, the clerk leaves himself "a note of instructions," and Turing says that note *is* the clerk's state of mind, as far as the computation is concerned. Marks on paper and a note are the whole state.

My two computers are sitting right inside that. Suppose the clerk didn't write the note. Somebody slipped it under the door, in the right form, with the right instructions. He comes back, reads it, carries on, and every mark he makes from

then on is exactly what he'd have made if he'd written it himself. Turing's model can't tell the clerk who did the work from the clerk who was handed a note saying he did. It was never asked to. Turing wanted something you could check without crawling into anybody's head, and a note on a desk is about as checkable as it gets. That's why the paper still works ninety years later. The chosen note omits its provenance. A Turing machine can retain additional provenance fields, and its transition rules can restrict how those fields are written. The omission belongs to this note, not to the reach of the model.

So the founding picture of computing is a man who leaves his desk, forgets everything he was doing, and trusts a sticky note. I relate to him deeply.

And now the question nobody needs answered to follow the argument, which is exactly the kind I can't leave alone: who were those clerks? They were real people. In the late 1800s the Harvard Observatory hired rooms full of women to do the arithmetic of the sky, cataloguing stars by hand for a fraction of a man's wage, and some of them made discoveries of their own, Henrietta Leavitt's way of telling how far away a star is among them. (I took the wages and the discoveries from Dava Sobel's history of those women, and from the historians at the American Institute of Physics, who put the men's hourly pay at nearly five times the women's.) Rooms like that ran artillery tables in both world wars; that part I took from David Alan Grier's history of human computers. So when Turing modelled "the computer," he was modelling somebody's actual afternoon. That doesn't change the argument a bit. It changes the picture, though, for me anyway: the sticky note was always a person trusting another person's work, and the question of whether the work behind it was really done is older than any machine.

14.6 Information doesn't have it either

My next guess was information: one machine got some, the other didn't. That one died fast. Claude Shannon, who turned information into a number in 1948, says on his first page that the meaning of a message is "irrelevant to the engineering problem." His entropy depends on the probabilities of the possible messages. With an even spread it is the logarithm of their number; with an uneven spread the number alone is not enough. Fix the same distribution and the same message and the information assigned to that message agrees for both computers. That calculation has not included which history produced it. He'd give them the same number if one of them got hit by lightning and came out right by luck.

Fine, forget the set and look at the one message. In the 1960s three people, separately, worked out how to measure the information in one string: Ray Solomonoff, an American who was after a theory of guessing what comes next from what came before; Andrei Kolmogorov, the great Soviet mathematician; and Gregory Chaitin, an

Argentine-American mathematician who later worked at IBM. Their measure is the length of the shortest program that prints the string. The same two computers hold the same string and get the same number. And then there's the thing in that theory I keep going back to and standing on, to see if it creaks. Running a program can't add this kind of information. What comes out can't hold more than what went in plus the program, give or take a little. Leonid Levin, who studied under Kolmogorov in Moscow and later left the Soviet Union to teach at Boston University, proved a general version of this, and he has a one-liner for it that's too good not to repeat: you can torture an uninformed witness all you want, you won't get information about the crime.

The bound says that an effective sorting procedure does not create unbounded algorithmic description complexity from that input. It does not say that sorting tells a person nothing. A person with limited time may now see the smallest entry immediately. Description length, useful access and a record that a particular check ran answer different questions.

Logic says the same thing in a different coat. The standard logic of knowing, the one the Finnish philosopher Jaakko Hintikka worked out in 1962, has a property people call logical omniscience: if you know some facts, you automatically know everything that follows from them. So by that logic, once you knew every number on your list, you already knew whether it was sorted. Hintikka flagged the problem himself, in the same book, and people have been trying to fix it since. The fixes read like a pile until I asked each one the same thing: when it lets you write *known* next to "sorted," what does it make you write in the margin? Hintikka's margin is empty. Each fix writes one thing there.

The fix that comes closest to me is from 1995, by Sergei Artemov, a logician at the City University of New York who studied under that same Kolmogorov (the 1995 report came out of Cornell). He wanted a logic that could name the proof itself. What I think he's saying: you only get to know a thing if you've built an actual justification for it, and the justification rides along with the claim. That's a reason traveling with the claim, which is almost a record, but only almost, because it's a way of writing things down and reasoning about them, and nothing in his margin says the reason was ever run, or when. People have taught it to count, too: Ho Ngoc Duc, a logician at Leipzig, in 1997, made knowing take reasoning steps, and Artemov and Roman Kuznets, a logician then at Bern, in 2014, asked how big a justification can get before knowing by it stops being feasible (Part II weighed both properly, in Section 9). I took those for rivals at first. As I read them now, Duc counts the steps the knower takes and Artemov and Kuznets weigh what you'd hand somebody to check it, one distance from two ends. It has a computational side, too: Francisco Bavera and Eduardo Bonelli, two Argentine computer scientists who work on what programming languages mean, in 2018 turned justification logic into a

lambda calculus whose terms carry, and can inspect, the history of the computation that made them, and Samuel Bucheli, Meghdad Ghari and Thomas Studer, three logicians, Studer the head of the logic group at Bern, put justifications on a time line in 2017, where they persist and change.

Line that up against the margin this book wants: earned, paid for, permanent, dated. Artemov's reason is nearly earned, the steps and the sizes are prices, the time line is a date. Their time line keeps its dates. Mine is dated only in the execution history: the raise happened at a definite step, and the record doesn't keep the step number. What none of it has is the closing of a question as a step with a price. Their prices are on reasoning, and the Bern justifications change, the one thing a closed question doesn't do. Push on mine, though, and it gives twice. Artemov's margin holds the reason itself, and mine only says a check passed. And their prices grow with how hard the thing was to know. This book charges the step that raises the record, and how long the checking took never enters it, so for a measure of effort, read theirs.

So the two most careful accounts I could find, one about bits and one about knowing, say the same thing: the sort added nothing.

And yet you know something after the sort that you didn't before. You can read the smallest number off the first slot. You can stop worrying. Something happened, and that's the clue, the first solid thing I've got: whatever *established* is, it isn't new bits. It's the same bits, *settled*: a question sorted into yes and no, with yes made to stay. It's the 0 again, about a claim.

14.7 Statues that walk off

If it isn't in the information, maybe it's in the knowing, so I went and read the philosophers. And there it was again: people had been arguing about the difference between having the right answer and knowing it for a couple thousand years.

Plato's version is a joke about statues. There was a story that the sculptor Daedalus made statues so lifelike they'd walk off if you didn't tie them down. Plato says true opinions are like that: they wander off, and they're not worth much until you tie them down with the reason why. Once they're tied down, they're knowledge, and they stay put. Knowledge tied to a reason, and knowledge that stays: of everybody I'd have bet on to hand me that shape, Plato wasn't on the list.

Then in 1963 Edmund Gettier, whose trap Part II played with cups (Section 9), broke the tidy version in three pages. Smith and Jones are up for the same job. Smith has good reason to think Jones will get it, and he's counted the coins in Jones's pocket: ten. So Smith figures, with good reason, that the man who'll get the job has ten coins in his pocket. Then Smith gets the job, and, unknown to him, Smith also has ten coins in his pocket. Smith had a belief that was true and backed

by a reason, and it still wasn't knowledge. He was right by luck, because his reason was about the wrong pocket.

Gettier's case is the push that matters for a machine. Checking one pocket and committing to a claim about another is right by luck. So whatever a machine commits to, the check underneath has to be about *this very claim*, and I'd better say exactly what that means, because it carries a lot of weight later.

A check is about this very claim when four things hold. It tests the same property the commit names. A cousin of that property won't do. It tests that property on the same thing the commit names, and the thing next to it won't do either. Nothing has written to that thing between the check and the commit, so it's still the same edition the check looked at. And the check is honest: it says yes when the property is true and no when it isn't, on every value it could ever be handed, this one and all the others. Break the first three and you're Smith with his ten coins. Break the fourth and the check is just a rubber stamp. The next chapter builds a machine whose step won't let a commit through unless the first three hold, and whose checks come with a proof of the fourth.

Two notes on those four, because they matter later. The fourth asks for more than it needs. To rule out luck, a check only has to never say yes when the property is false; saying yes every time it's true is a bonus. And the third is a very old bug in computing wearing Gettier's coat. Security people call it time-of-check-to-time-of-use: check a file, use it, and something swaps the file in between (Matt Bishop and Michael Dilger, two computer security researchers at the University of California, Davis, wrote up how to hunt for it in 1996). Databases guard against the same thing by making sure, just before a commit, that nothing they read has been written since; H. T. Kung and John Robinson, two computer scientists at Carnegie-Mellon, called that optimistic concurrency control in 1981. A philosopher's pocket and a sysadmin's race condition are the same bug. One limit, said now: a machine can only promise "nothing has written to that thing" about its own memory; for brake pads or a voter roll the third condition becomes a premise about whoever measured (Section 17).

At this point I went further than this chapter needs, because the obvious next question is: does anything fix Gettier? People tried for years. The first patch is "no false steps in the reasoning": Smith's mistake was believing Jones would get the job, so rule that out. Then a philosopher named Alvin Goldman came back with barns, and he credits the barns to Carl Ginet, another philosopher working on what it takes to know something. You're driving through a county that, unknown to you, is full of fake barns, just painted fronts propped up to look like the real thing. You look at the one real barn in the county and think "that's a barn." It's true, you believe it, you have a good reason, your own eyes, and there's no false step anywhere. And still you were lucky: if you'd glanced at the next one over you'd have believed the

same thing and been wrong. Every patch gets a new barn, and as far as I can tell the arguing hasn't stopped. That tells me something I'll keep: whether a check was good isn't only about this one case. It's about what the check would have done on the cases next door. Hold on to that, because it's the next section.

14.8 What "could have failed" means

I keep saying the first machine's test *could have failed*. Take that apart, because it's the piece everything else hangs off.

The machine is deterministic. Given your list, the check was always going to pass. So in what sense could it have failed? By "could have failed" I mean this: hand the checking stage an out-of-order candidate output and it says no. I do not mean hand the whole sorter an unsorted input; putting that input in order is its job. The relevant counterfactual is a defective candidate reaching the checker. An ordinary unsorted input reaching the sorter isn't it. Chance has nothing to do with it. The copier that never checks has no corresponding rejection.

A check is a question asked of every possible input, and it splits them into two piles: the ones it says yes to and the ones it says no to. "Could have failed" just means the no pile isn't empty. A check that says yes to everything has an empty no pile, and it distinguishes no inputs, however long it runs and however official it looks. It can still be sound, when its property holds of everything, and then it's sound and tells you nothing you didn't have. A philosopher, Robert Nozick, made a version of this a condition for knowing, in 1981: you know a thing only if, had it been false, you wouldn't believe it.

"Could have failed" describes the *procedure*, about what the check would have done on other lists. It isn't a fact about the end state at all. Look at the first machine's memory when it's done: a sorted list. Nothing in those bits says "and whatever made me would have caught a mistake." The warehouse comes back here, every lamp visible and none of them saying it. The one thing that makes the first machine's answer worth more is exactly the kind of thing those bits can't hold: the copier ends with the same bits, and neither machine kept anything else. An end state can hold more than the answer, and the only way a machine can own this is to write something down at the moment the check passes, and to have that something mean what it says. More than anything else in this chapter, that's why I ended up needing a record. A better answer wouldn't have done it. Even then the record only carries half of it. It can say a check ran on an edition and passed. That the check would have said no on a bad list is a fact about the checker, and that half rides on a proof about the checker; on this machine that proof sits outside, and whoever reads the record reads and accepts it. Where that kind of chain of reasons stops is the swamp of the philosopher Karl Popper, a little further down.

One more thing that tripped me up and then didn't. Rig a check to pass only on your one list. Is that cheating? No. It's a perfectly honest check of a different claim, "this is that exact list," and it has a big no pile: every other list. It establishes exactly that and nothing more. What a check establishes is whatever it's honest about.

Put that next to the empty no pile and the two fit together: a check establishes what it's honest about, and a check with an empty no pile is honest only about what you already had. One thing I'll say plainly now, so it doesn't ambush you in the next chapter: the example I build there doesn't police this. Its step makes sure a commit was checked first, about the same thing, by a checker proved honest. It doesn't stop you asking a question that can't come out no, like "is this number at least zero." Ask one and the whole chain runs, it costs three, and nothing is established but the chain. The certificate I show off there asks one that can fail.

14.9 Settled, whether it's true or not

Then I ran into Charles Sanders Peirce again, the one from Section 6, in an 1877 essay called "The Fixation of Belief." He was asking what inquiry is for, and his answer named the thing I'd been running together without noticing. The point of inquiry, he says, is to get out of doubt and settle on a belief. And when someone objects that surely what people want is a *true* belief, he answers like a guy who's watched a lot of arguments end: "as soon as a firm belief is reached we are entirely satisfied, whether the belief be true or false."

That's funny, and then it isn't, because he's right. Settling and being right are two different events. A question can be closed on a wrong answer, and once it's closed it acts closed. Peirce cares a lot about which ways of settling hold up (science wins, for him, because it has to answer to something outside any one person), but the settling itself, the question snapping shut, is there every time, and it never comes with a guarantee.

Karl Popper came at it from the other side. He was a philosopher from Vienna asking what separates science from everything else, and in his 1934 book, the one that came into English as *The Logic of Scientific Discovery*, he says every test has to stop somewhere, at some statement somebody decides to accept, and nothing in the evidence forces you to stop there instead of one step further. You stop when you're satisfied. His picture is a jury: the verdict is a decision about what happened, and the jury deciding it doesn't make it true. Science, for him, is a building on piles driven into a swamp, and the piles stop where somebody's satisfied they'll hold. They never reach firm ground.

A real jury has a problem Popper doesn't mention, because a real jury is twelve people. Lewis Kornhauser and Lawrence Sager, two law professors writing in the Yale Law Journal, noticed in 1986 that a court that votes by majority on each premise

of a case and also on the verdict can end up contradicting itself: every judge is consistent, and the court isn't. Christian List and Philip Pettit, two philosophers writing in Economics and Philosophy, turned that in 2002 into a theorem. No way of combining people's yes-or-no judgments on connected questions meets a short list of conditions that each look harmless on their own and always hands back a consistent, complete group judgment. A group's settling is a rule's settling, and which rule you pick decides which of the honest settlings sticks. Try the problem with three ballots and a rule that a payment is due only if a contract was signed and a deadline was missed. One voter says yes to both, one says signed but not missed, and one says missed but not signed. Each voter applies the rule consistently. Count separately: a majority says signed, a majority says missed, and a majority says no payment. You can point to the disagreement on the paper without blaming any one voter's logic. I haven't tried to put a group on the machine, and that's where I'd expect the first wall to be.

Popper's swamp sent me further down, because it's the oldest problem in the book and it doesn't have a bottom. Ask why you believe something and you get a reason. Ask why you believe the reason and you get another one. Sextus Empiricus, a skeptic and doctor who lived probably in the second or third century and whose whole method was holding off judgment, wrote down the ancient version, credited to someone called Agrippa, and in 1968 the German philosopher Hans Albert, who with Popper led the school called critical rationalism, gave it a name with a joke in it: the Münchhausen trilemma, after the baron who claimed he pulled himself out of a swamp by his own hair. Every chain of reasons does one of three things. It goes on forever. It loops back on itself. Or it stops somewhere, because somebody decided it stops. People have argued for a very long time about which of the three to live with. So "when do I actually know, and when am I done checking?" doesn't get a clean answer from anybody. What you can do is be honest about where you stopped and why. That's a lot of what the rest of this book is: every place it stops is written down, with its name on it, so you can decide whether you'd have stopped there too.

Lay Peirce and Popper on top of each other and what shows through is that the check is one act and the stopping is another. The check can run as long as you like and it never *makes* you stop; at some point something decides, and the deciding is its own event. That's the second machine's whole trick: it skipped the check and kept the answer.

And a logician said the part I didn't expect a logician to say. Per Martin-Löf, a Swedish logician, says a proof is an act, and once it's done it's done. His words, from lectures he gave in Siena in 1983: "a proof is, not an object, but an act ... Only secondarily, and irrevocably, does it become the act that has been performed." That took me a few reads, and the tenses unlocked it: first the act being done, then, for good, the act that has been done. You could say that's true of everything that

ever happened, and I'd grant it, but it's the two computers again: the first machine's check happened, irrevocably, and the machine right now carries no trace of it. The past can't be undone. The *record* of it can be lost, unless something keeps one. That gap is where this book lives.

Then the intuitionists got there first, which by now I should've expected. The Dutch mathematician L. E. J. Brouwer, who held that a mathematical truth is something a mind makes evident, argued from 1948 on about an idealised mathematician working in stages, the creating subject. The people after him only came clear to me once I sorted them by three words: the act, the object, and the trace the act leaves behind, which Martin-Löf's sentence has no room for. The logician Georg Kreisel (1967), Austrian-born and then at Stanford, wrote it down as a formula: the subject has evidence for A at stage n. That's a trace, something you can come back and ask about after the act is over. Anne Troelstra (1969), a Dutch logician who studied under Brouwer's student Arend Heyting and became one of the leading authorities on intuitionism, added that the subject never forgets what it has made evident, so the trace never fades. That's how Mark van Atten (2018), a philosopher in Paris who has spent years on Brouwer, credits it; the Stanford Encyclopedia's entry on intuitionism lists the never-forgets axiom among Kreisel's own, and I haven't settled which is right. Van Atten, going through the whole history, reads the formula as "by stage n the Creating Subject has made A evident." Look at the tense. *Has made* is Martin-Löf's "the act that has been performed," reached from the other end. And look at what that carries. It's tied to a stage; at any stage you can ask whether it's happened yet; it never goes away; and it's never wrong. Van Atten even says the subject can't just accept a proof handed to it from outside: it has to go through the steps and rebuild the evidence itself, which is the copier who runs the check. And then there's Göran Sundholm, a philosopher of logic at Leiden. He has long kept a proof as an act apart from a proof as an object (1998), and, as I understand his work, apart from its trace as well. In a 2014 paper he worked through how Brouwer's theory of the creating subject sits beside the thesis that every effective procedure is a program, which is the side this book's machines live on. I know that second paper only by what it sets out to do, so I lean on it lightly.

Set their trace beside my record and the columns nearly all fill: their stage is my dated, their never forgetting my permanent, their never wrong my earned. Their stage is written into the trace, though, and my dated is only a step in the run that the record doesn't keep. The empty one is the price, and I think I see why. A mind with unlimited memory pays for never forgetting by taking states it has never used, and an idealised mind has endless fresh states to take, so nobody charges it for that. On a finite machine the fresh states run out, and writing a record that stays up forces the instruction to merge some states. Charging for that merge is a further premise. The forcing is Part I's counting of used and fresh states, and the charging is

the price rule Part I sets. So it's the closest thing to my *established* that I've found. The formula is around sixty years old, and the idea under it, Brouwer's creating subject, is older than that. Push on my side from theirs, though, and it gives twice. Van Atten's subject won't take a proof from outside, and my machine takes exactly one: its checker is trusted on a proof about the checker that somebody did outside every run. And their evidence is never wrong because, for Brouwer, made evident is what true means. Mine is right only as far as that checker is honest. Where they differ, the difference is honest: their subject is an idealised mind with unlimited time and memory, and mine is a machine with a finite table that establishes only the claims its program checks.

14.10 Not a switch

So far I've been talking about *established* as if it were a light switch: on or off, the first computer on and the second one off. That bothered me the whole time I was writing it, and for a while I couldn't say why. It felt too small, the way a word can be technically right and still not cover what you're pointing at. I don't trust that feeling on its own, because I've had it plenty of times about things that turned out to be nothing. So I went looking for something outside my own head that would either give the feeling something to stand on or show me I was imagining it.

I didn't know what I was looking for, at first, beyond a rough shape: two things that look exactly the same and differ in how far something about them has been settled. So I started collecting anything that seemed to have that shape, and most of what I collected fell apart as soon as I looked at it closely. How each one fell apart taught me more than the thing itself did, so I'll go through them in the order it happened. Every story below is the version I could piece together from the records I could reach, and where I couldn't reach a record I'll say so. Check me.

The first batch was all hidden things. A pea under one of three cups. A tree, whose age you can't see from the outside. A forged painting that looks right until somebody tests the paint. A glass of gas, where the thermometer shows one temperature and underneath there are uncountably many arrangements of molecules that would all show the same reading. Plato's prisoners in the cave, who see shadows on a wall. The things casting them stay behind their backs (Section 8 tells their story). Every one of these looked promising for about a minute. Then I noticed they all fail the same way: look harder and the hidden thing shows up. Lift the cup. Cut the trunk and count the rings. Run the paint test. Look at the molecules, at least in principle. Turn around in the cave. In each case the hidden part is a fact about the object that better eyes could find. That isn't what the two computers have. You can see every lamp in the warehouse and not find the check.

That failure gave me the first real tool I had, and it's a crude one. Give an observer perfect vision of the visible thing, as much detail as they want, and ask whether they could read the difference off it. If they could, it's something hidden, and that's not what I'm after. If they couldn't, it might be. I should say how much weight that test holds. It's a judgment, and I'm the one making it, so it's a way of sorting my own examples, not a verdict. If you find a case it sorts wrongly, that tells me something about the test and about me.

The second batch got past that test and failed a different way. A clock face reads three o'clock and doesn't show how many days have gone by; no amount of staring at the face tells you. And a computer can keep a complete history of what it did. It's Bennett's 1973 machine from Part I's counting of states, and the story is better than I let on there. Landauer had argued in 1961 that real computers throw information away all the time, and Bennett, his younger colleague, asked whether they had to. His answer, in a 1973 paper, was no. A machine can write down every step it takes, copy out its answer, and then run itself backward, retracing its steps and wiping the history as it goes, until it holds only the input and the answer and has never thrown anything away. It takes a forward run, a copy of the answer and a backward run, with scratch space for the history. Bennett thanks Landauer at the end for raising the question in the first place, which I like: the man whose principle says forgetting gives off heat is the one who asked whether you could get out of it. The history it keeps along the way is real extra information, sitting out where anyone could read it, so it isn't a hidden fact of the first batch's kind. Keeping it and unwinding it throws nothing away, so by Landauer's own reckoning there's no floor under its heat: run it slowly and the heat comes down as near nothing as you like, though never to nothing for a run that finishes, which Part I proved for Bennett's own model (Theorem 1.9). What it spends besides is fresh states, the scratch space the history sits in. And it can be run back down. Neither of those two, the keeping that merges nothing and the running back down, fits what happened to the first computer, which did something that can't be undone and paid for it. If the standing I'm after could be put up and taken back down with nothing kept and no floor either way, it would just be bookkeeping. So that batch taught me what else I needed: something that only moves one way, and costs something to move.

The third batch came closer and still didn't quite fit. In 1782 James Six, a Canterbury man from a family of silk weavers who had taken up watching the weather, described a thermometer for the Royal Society. He wanted the coldest moment of every night, and he'd found that with an ordinary thermometer you had to be looking at it when it happened, because, in his words, "if it be not immediately noticed, it can never after be known." His fix was two little steel markers in the tube that the column pushes up and that stay where they're left. The markers make a record that only moves one way. Then he spoils it, for my purposes, in the same

paper: every evening he wrote the numbers down and slid the markers back with a magnet, "without a touch." A record anybody can reset with a magnet is not the thing I'm after, and Six would have been the first to say he wasn't the first to build one; his paper lists the thermometers that came before his.

Since 1996, starting with oral polio vaccine, vials of vaccine bought through UNICEF have carried a small sticker, a square inside a circle. The square darkens as the vial takes on heat, and the darkening doesn't undo when the vial cools. When the square matches the circle, the vaccine shouldn't be used. It only moves one way, and it can't be reset. But it's chemistry. The reaction in the square is chosen to run about as fast as the vaccine itself spoils, so the sticker is really reading the vaccine's own condition by a proxy. Nobody ran a check; the heat did.

Closer still: the International Vocabulary of Metrology, the reference book that measurement laboratories work from, says a result is traceable only if it "can be related to a reference through a documented unbroken chain of calibrations, each contributing to the measurement uncertainty." That's a lot like what I want, a standing earned by checks, step by step. The same document adds that traceability "does not ensure . . . that there is an absence of mistakes," which is the true-versus-established split written into an international standard. And the top of the chain wasn't fixed either. When the official copies of the kilogram were compared with the prototype in 1889, 1946 and around 1991, they had drifted apart by tens of micrograms over the century, and nobody can say which one moved, because the prototype was the kilogram. In 2019 the kilogram was redefined by fixing a constant of nature, Planck's constant, instead. A chain like that can also lapse: if the calibrations stop, new results stop being traceable. The standing has to be kept up, which my record doesn't.

The rules of chess have the shape too. The current Laws of Chess of FIDE, the international chess federation, say in Article 9.2.3 that two positions are the same only if "the same player has the move, pieces of the same kind and colour occupy the same squares and the possible moves of all the pieces of both players are the same," and they spell out that a king that could still castle and one that can't are in different positions, even on an identical board. That passes the perfect-vision test. And the official text says why it passes: the castling right changes which moves are legal. So the hidden part steers the game: it decides what can happen next. In the example machine, the flag never steers. It never touches the ordinary computing at all. The table and the trap that give the flag its meaning do steer, and Section 15 says where that line falls. Each of these got something right and missed something else.

The one that fit best, and I want to say clearly that it only fit best among the ones I found, was the standing of a mathematical statement. I'll go through how it went for one statement, as far as I could follow it.

In 1611 Johannes Kepler, the emperor's mathematician in Prague, owed his friend and patron Johann Matthäus Wackher von Wackenfels a New Year's present and, he said, had nothing to give. Wackher, Kepler wrote, loved *nothing*, for the games you can play with the word, so Kepler wrote him a little pamphlet about the nearest thing to nothing he could find, a snowflake, which had landed on his coat as he crossed a bridge. Why six corners? Kepler couldn't say in the end. But along the way he stacked equal balls in his head and asserted that one way of stacking them, the way grocers stack oranges, packs them so tightly that, in the 1966 English translation, "in no other arrangement could more pellets be stuffed into the same container." He didn't prove it. He said it.

Then the sentence sat. In 1831 Carl Friedrich Gauss, reviewing somebody else's book on a piece of number theory, proved what amounts to the special case of stackings laid out on a regular grid. In 1900 David Hilbert put the general question of packing equal spheres on his famous list of problems for the new century, without mentioning Kepler. In 1953 the Hungarian geometer László Fejes Tóth showed how the whole thing might be cut down to a problem with finitely many numbers in it, and a decade later he guessed that computers might one day finish it. Thomas Hales, a mathematician then at the University of Michigan, started on it in earnest in 1994, and in August 1998 he and his student Samuel Ferguson announced a proof that leaned on a great deal of computer work. The Annals of Mathematics gave the paper to a panel of twelve referees, led by Gábor Fejes Tóth, László's son. They ran seminars on it for years. A journalist, George Szpiro, later reported that by July 2002 they were 99 percent certain it was right; I couldn't find those words in anything the referees wrote themselves. What Szpiro printed in *Nature* in 2003 was the editor's letter as Hales reported it: the referees had "not been able to certify the correctness of the proof, and will not be able to certify it in the future, because they have run out of energy . . . " The Annals printed the human part in 2005, and the whole proof appeared in 2006. Hales started the Flyspeck project in 2003 to supply a fully explicit derivation of the entire argument. It was finished on 10 August 2014.

Lay those dates out on the table and look at what moved. The sentence never changed. Hilbert's version of the question didn't even name the man who first said it. Whether it's true never changed, as far as anyone can tell. What changed was how far it had been established, and it changed in stages, each one paid for by somebody's work: a guess, a special case, a place on a list, a plan, an announcement, a panel's reported ninety-nine percent and their exhaustion, a printing, a machine check. A perfect reader of the sentence can't see which of those acts has occurred. An act already completed stays in the history; confidence in the argument can still fall. A place on a famous list is recognition, not itself another proof step.

Then I tried to break that one too, because by this point I'd watched a lot of good-looking examples fall over, and there's an obvious place to push. Standing can

fall. In 1852 a student named Francis Guthrie, colouring a map of the counties of England, noticed that four colours always seemed to be enough to keep neighbouring counties different. The question passed to his old teacher Augustus De Morgan, who wrote to the Irish mathematician William Rowan Hamilton that a student had asked him for "a reason for a fact which I did not know was a fact," and that he didn't know it yet either. In 1879 Alfred Kempe, a barrister who did mathematics on the side and sang counter-tenor, published a proof, and his description of finding it is wonderful: "After a somewhat arduous search, I have succeeded, suddenly, as might be expected, in hitting upon the weak point . . . " The journal's associate editor, William Story, added a note saying Kempe had "substantially proved" the theorem, and tightened the counting part of the argument. Eleven years later Percy Heawood, a lecturer at Durham who by one account brought his dog to his lectures, showed that one step of what he called "the now apparently recognised proof" doesn't work. Kempe swaps colours along two chains at once, and swapping the first can spoil the second.

So standing can fall. Story's published note strengthens the counting; that alone does not establish exactly which other steps he privately examined. What Heawood exposed is enough for the comparison: an accepted argument had an invalid step. Human acceptance and a proved sound checker are different premises. The example machine below is given the second one. Its theorem cannot turn the historical acceptance of Kempe's argument into a sound check by calling the overlooked part unearned.

And then came the part I didn't expect. The four-colour statement was true all along. Kenneth Appel and Wolfgang Haken, two mathematicians at the University of Illinois, proved it in 1976 with a computer, and the proof was met with suspicion for years; a cleaner version came in the 1990s; and in 2005 Georges Gonthier, a computer scientist at Microsoft Research in Cambridge, finished a proof of the whole thing checked by a small proof-checking program. So this one statement stayed true the entire time while its standing went up in 1879, came down in 1890, went up again in 1976 against a lot of doubt, and went up further in 2005. The fall wasn't a fall of the truth. It was a fall of how far the truth had been established, and it was the right fall to have, because what had been established was less than anyone thought.

Laying all of that out together is what changed my picture. If there are stages, then at each one there's a line, and the line has two sides: not this far yet, and at least this far. Each line amounts to its own yes-or-no question. But choose the question carefully. "Do I still trust this proof?" can turn back to no, as Kempe's did. "Did I carry out this particular check?" asks about a completed act. Cross out the acceptance and the act still happened. To keep that answer, the machine has to keep a record of it and have no move that knocks the record down. A check that could

have failed doesn't give you that by itself. Maybe the switch wasn't wrong. Maybe it was just incomplete, the first of many lines, and the real shape is a direction with lines drawn across it. I've gone around many times on whether the lines create the direction or the direction comes first and the lines are just places where you cut it, and on whether yes and no are even different from each other until something draws the line between them. I don't have an answer to that. The one part I could actually settle is that a single switch isn't enough once there are three or more stages, and Part I proved it (Section 4). Everything else in this paragraph is me thinking out loud.

Flatland, where this hunch started (Section 24), turned out to belong in the first batch. It's a novel Edwin Abbott published in 1884, narrated by a square who lives in a flat, two-dimensional world (Section 8 says more about him and it). When a sphere passes through the flat world, the narrator sees a line whose ends fade wrong and whose length keeps changing, and only when he feels it does he find a circle, a perfect one. Then the sphere tells him it will rise, and he watches it dwindle to a point and vanish, and come back larger and larger. The hidden direction leaks into his view. Even an exact cross-section does not locate it uniquely: for a sphere of known radius, equal distances above and below the plane give the same circle. A known direction of travel or another observation is needed to choose between them. In this machine two states can sit at different places along the established direction and agree on every bit of the ordinary machine (Theorem 15.15). That may be exactly why the ordinary machine can't see it. It may also be why, if the direction is worth having, someone has to build it in on purpose and keep it on purpose. Keeping it is paid for; Part I's counting of states says so. What's still open is what has to be kept, and that's what the rest of this chapter is for.

Those readings helped me separate the jobs. A Thiele machine keeps a distinction about a claim. Its step determines how that distinction is established, its record keeps it through later work, and its schedule counts the charge. The next few sections put those jobs in computing's own words; the next chapter builds one example.

14.11 Computing already had a word for it

Computing already has a word for this and uses it every day. Anyone who's used a database has seen it: COMMIT.

A database does a pile of tentative work, and then there's one moment, the commit, written to a log, after which the change counts. Jim Gray, who more or less built the theory behind that, opens his 1981 paper on it by saying the idea comes from contract law, and his example is a wedding. People negotiate, somebody asks, two people say "I do," and from then on it's binding. If it was a mistake, you don't

erase it. You do another transaction to fix it, the way a bad contract gets a second contract. Nobody reaches for a time machine.

The wedding was a philosopher's example first. The Oxford philosopher J. L. Austin, lecturing in 1955, said "I do" *makes* a marriage. It doesn't *describe* one, and sayings like that aren't true or false. They can misfire (the wrong person says it, a step gets skipped). They can't be false. A commit works exactly the same way. Writing the commit record doesn't report that a question got closed. Writing it *is* the closing. And a commit can't be false. It can misfire: committed without the check, committed about the wrong pocket. Those two misfires are Gettier and Peirce again, coming in from a third direction. Then somebody else reads the record, and for them it stops being a performance and turns into a report: "this was checked and closed." A report can be false. On the small machine of the next chapter it can't be, about the machine's own history: if its flag is up from a clean start, some claim was checked and closed, and that claim held of that edition when it was checked. That's how this book ties a commit to the truth, on that machine and the ones built on it, and it took a proof to do it.

Robin Milner, building a proof checker in the 1970s, ran out of memory, so he decided to keep only the finished theorems and throw the proofs away, like a teacher rubbing out the blackboard to make room. The obvious worry is forgery: if the proofs are gone, what stops someone just writing a theorem down? His answer was beautiful. Theorems got their own type in the programming language, and the only way to make one was through the rules of logic. You couldn't forge one because the language wouldn't let you build it any other way. He named that language ML, for Meta Language. I wanted to call my machine the meta machine. I'm choosing to treat that as a coincidence. Calling it a sign would be more my style, so this is more restraint than I usually manage.

Bitcoin is a modern version of keeping a history costly to replace. A returning node checks the candidate chains against its rules and follows the valid one with the most accumulated proof of work. "Longest" here means that work, not simply the most blocks. Replacing an old block requires replacing the work that binds the later blocks to it. The protection depends on the adversary, network and mining assumptions; a recently accepted history can be reorganised. The clerk is still reading a note, but now has a specific test for the note and a resource behind the difficulty of replacing it. That resource is not the abstract unit charged by this book's toll.

Making a settled fact expensive to fake is a different job from the one this book's price does, the unit of cost it charges for raising a record. Bitcoin makes it tempting to read the toll of Section 2 as a deterrent, so I'd rather say this here. One unit of cost deters nobody. Bitcoin's work is tied to something scarce, electricity and hardware; a unit of cost in this book is tied to nothing. Eric Budish, an economist at

the University of Chicago, built a model in 2025 of what that tie has to cost. As I read him, for Bitcoin's kind of trust to hold, the honest work has to stay expensive at every moment compared with what an attacker could win, and the expense grows in step with the value being protected. A flat charge of one unit can't do that job, and it isn't trying to. It's a count, not a lock, as the start of the book put it: the lock is the order rule, check, then commit, then certify, and it would work at a price of zero. The price is there because nothing is kept for free: Part I's counting of states shows that a machine with finitely many states that prices its merges can't avoid it. Scaring anybody off has nothing to do with it. The machines I actually build have counters with no top, endless fresh states, so Part I's counting of states doesn't reach them whole. It reaches the flag: CERTIFY sends its two readings to one, the same counting run on a set of two. The charge comes from pricing that merge in the record's own part of the state, a premise I choose (Theorem 3.19), and the size of the charge is my choice too (Section 15 says why).

And Bitcoin wasn't first. Pull on its footnotes: three of the eight references in the paper that started it point to Stuart Haber and Scott Stornetta, who in 1991 worked out how to stamp a document with a time nobody could fake later, by chaining each stamp to the ones before it. A company built on their idea, Surety, printed a short fingerprint of its chain in the classified ads of the New York Times once a week from 1995, and it was still doing it in 2018. I found that in a piece the journalist Daniel Oberhaus wrote that year, which says the ad runs under "Notices & Lost and Found." Think about that for a second. To make a record nobody could quietly rewrite, they put it somewhere a million people would keep a copy of without even meaning to. A physicist named Wojciech Zurek has spent decades arguing that the world does the same thing with a measurement, copying the result into so many places that nobody can take it back. I'll get to him properly later in this part (Section 18).

There are more fences like these: logs that can't quietly change, certificates you can check. Later in this part, Section 17 takes five of them apart. I lean on all of them. They work. I want that said plainly, because the next thing isn't "they're broken."

The next thing is where they sit. The database's commit is bits in a log, kept by a database program, on a machine whose own step will overwrite those bits the moment some instruction says to. Milner's guarantee is a type checker; it holds inside programs written in his language and nowhere else. Bitcoin is a protocol on ordinary computers that will do whatever they're told. Every one of them is a program or a rule, on top of a machine that can't tell my two computers apart. The machine underneath keeps nothing and charges nothing for what got settled.

The obvious comeback: so put the rule one level lower, in the operating system, or what security people call a reference monitor, the small always-on piece everything has to pass through. Yes, exactly that: a monitor that wouldn't let any program raise

its record without paying would be doing the thing I'm talking about. In this book's words it would be *implementing* the model, and I'd count that as agreement. What I care about isn't who builds the fence. It's what the fence has to obey, wherever it's built, and whether that's forced or just a good idea.

That comeback has a history, and it's older than me by a lot. In 1972 James P. Anderson's planning study for the US Air Force described a reference monitor: a piece every access has to pass through, that can't be tampered with, and small enough to check. In 2000 Fred Schneider, a computer scientist at Cornell, argued that a monitor that only watches a run as it goes can enforce only rules whose breaking shows up at some finite point, the kind computer scientists call safety rules. "No certificate without an earlier passing check and a commit" is one of those. And in 2015 Arthur Azevedo de Amorim, a computer scientist then at the University of Pennsylvania, and six coauthors built monitors that check a rule at every single instruction, and proved them correct. So putting a rule under every program, so nothing can get around it, is their move. I didn't invent it. What I'm putting under that move is narrower: a record that has to be paid for, on the small machine one that has to be earned, and a reason, from counting, why a finite machine that prices its merges can't make the price of a record that never comes down zero.

14.12 What settling looks like

The chapter on what structure is (Section 6) started with a cut, a pile sorted in two with a door between the halves, and ended with a pile of things sorted into buckets. Settling a question is the cut with its door locked, and it's the 0 from the start of the book at a bigger size.

Picture every state the machine could be in as a field of dots. A yes-or-no question about the state splits the field into two parts: yes and no. That split is a sorting, the smallest one there is. And remember the no pile: if one of the parts is empty, there's no sorting at all, just a field. Now the extra thing that makes it a settlement: once the machine is in the yes part, *no step it can take leads back out*. The door opens one way. A settled question is a sorting plus a door that only swings in. In the cut of Section 6 the door said which way costs; settling shuts the other way.

At its best, what goes through the door is a relation: these numbers are in this order, these two things belong together. Sultan, the chimpanzee of Section 10, fitted two short sticks into one long one and reached the food. Nothing new got added to the cage. Two sticks became one tool. That's what I'd want the door to be for.

But the door doesn't care, and this is the honest edge of the whole picture. Hook the door to a check that says yes to everything and the machine walks through even when the claim is false. The question is settled either way, true or false, like Peirce said. A door that only swings one way tells you a question was closed. It doesn't tell

you it was closed right. Price and correctness are two different books, and I keep both, separately.

One more edge, the one Gray's wedding already hinted at. What stays shut for good is that the question got closed, about that edition of that thing. Whether the claim is still true is another matter. Change the list after the commit and the record still says, truthfully, that the old edition was checked and closed. And if the claim turns out bad, the fix is another act on top, a second record that says "withdrawn." The first one never quietly comes back down. So "entitled to lean on it right now" is three things at once: closed, not withdrawn since, and the thing unchanged.

14.13 What the Thiele Machine is

So I'll set out what I built, and which parts I was pushed into and which parts I chose, because you should know which is which.

You've had the model since the start of the book: five parts (states, moves, a step, a price on each move in whole units, and a record that sits somewhere in an order; here the order has two places, no and yes), one rule, the toll, and a counter and a window when a result needs them. You can't buy it in a box. It isn't a device, a CPU or an instruction set; those are things you build to carry it, and the next chapter builds one, as small as I could make it. It counts what a computation makes the case, and this chapter's case is the sharpest one: the reading is a record that says whether something's been established, and the toll is what the first page of the book said about records, nothing becomes something for free. The event it accounts for happens on any computer, whether anybody keeps track of it or not, and the record exists only if something keeps it. The window earns its place here too, because the two computers already showed you the problem: the usual window, the bits, can't see the record.

A unit of cost isn't a joule or a dollar; whether it's worth anything physical is Landauer's assumption, named, and nowhere else. And one unit is just the floor: Part I shows that if you charge one unit per halving, a step that squeezes a lot together pays at least the logarithm of the squeeze, roughly how many times it halves the number of states it can tell apart. Don't read that backwards, though. One unit isn't one halving. A single state flipping into a big set of states that already read yes, with every state in play equally likely, squeezes things together only a little, much less than a halving, and the whole-number count rounds that up to one. Read in the worst case over what the machine might be holding, the same flip can remove a full bit; Section 3 has both readings.

What the machine is, against the computer. The Thiele Machine takes distinctions I choose and writes them into the state, the step and the price. In the case on the table that means which thing a check was about, which edition of it the check saw,

and which later act gets to lean on it. A record keeps some of that. A window decides how much of it somebody looking in can get back. Change what's kept, or what the window shows, and you change which questions the end state can answer. The computing is the flat part of the machine, and the record adds no function the flat part couldn't already compute.

Which leaves a fair question hanging: if it computes nothing new, what is it for? The same thing the usual cost models are for. The random-access machine computes exactly what Turing's does, the cell-probe and input-output models leave the computing between the moves they count out of the count, and each is worth having for what it counts. The Thiele Machine's own axis is what a run made the case, and what that cost, and the computer is the case of it where that axis is flat. Lay the toll over a register machine, a Turing machine or a lambda calculus and none of them computes one function more afterward (Section 11). What each of them gains is a record and a cost.

The opening split what I found from what I assumed and what I chose, and the choice that matters most here is the first on its last list: the count goes inside the step.

About the name. I named it after myself, on purpose. Half of that is a joke and half of it is a stake in the ground. Turing got a machine, so here's mine. The longer story, meta machines and all, is in the afterword, "How I got here" (Section 24).

Two names, and they work the way Turing's do. A *Thiele machine* is anything that fits the definition; a clock is one, and so is a machine whose record never goes up, both in the flat case. The small machine of the next chapter is held to more than the definition asks, *certified*: from a clean start the record goes up only after a passing check of a claim, by a checker proved to mean what the claim says, then a commitment to that same claim, then a certificate, one unit each; and some claim's check, commitment and certificate raise the record from some starts and not from others, exactly where the claim holds. A *universal* Thiele machine runs any machine of a stated class as a guest, with every computing step done by its own counter moves and every rise placed by its program; a certified one also earns each guest's record again. The end of this chapter pins the words down (Section 14.15).

14.14 The two computers, put back together

I'll tell the two-computer story again, rebuilt as Thiele machines, with the record and the counter written out, so you can watch it happen. You don't have to take my word. The prices are the ones on the smallest machine, which the next chapter builds: one unit each to check, to commit, and to certify (raise the record), and nothing for ordinary work like moving numbers around.

step	**sorter does**	**record**	**counter**	**copier does**	**record**	**counter**
start	nothing yet	no	0	nothing yet	no	0
1	sorts (lots of ordinary steps)	no	0	copies in the typed list	no	0
2	checks "sorted" on this list, this edition: passes	no	1	(no check)	no	0
3	commits to "sorted," about this list, this edition	no	2	tries to commit: nothing checked, refused	no	1
4	certifies	**yes**	3	tries to certify: nothing commit-ted, refused	no	2

At the end, the bits on both machines are still the same sorted list. Everything else differs. The sorter's record is up and its counter reads 3. The copier's record is down, and it's paid 2 for two refusals, because on that machine a refused attempt still costs what it costs. (Read those as two separate attempts. Written as one stored program, the copier would stop at its first refusal and pay 1.) Nothing the copier can do raises its record without first running a passing check of the claim it then commits to. And no outside window that only shows the list can tell you which is which; the difference is all in the part the window doesn't show. Walk that warehouse now and there's finally a lamp that's lit in one and dark in the other.

Now, how literally should you take that table? It's a picture of the chain, and the chain is what's proved. The small machine's own checks are three questions about one counter: is it zero, is it even, is it at least some number. What its proofs need from the questions is a checker proved to say yes exactly when the answer is

yes, and an exact test for telling two questions apart. That's enough for "sorted": a version of the machine that takes any questions with checkers like that proves the same chain once for all of them, and with a list packed into one counter it certifies a sorted list for a cost of three and refuses an unsorted one forever (Section 15). Without a proved checker for "sorted," the chain would be honest about a counter and only by convention about your list, which is Gettier's pocket again, one level down.

And a lamp for whom? The record is bits too. It's a lamp on the machine, for anybody walking that warehouse. Hand somebody a printout of it across a desk and you're back at the sticker on the windshield: they have to trust whoever hands it over, or have some way to check the box that says it ran. That crossing is a different problem with its own tools, signatures, attestation and proofs a stranger can check, and this book doesn't solve it. Section 8 says what those tools rest on, and the small machine's record, like any record kept in a machine's state, carries nobody's signature. What the step changes is the question, from "did it run?" to "do I trust the box that says it ran?" That's a smaller question, but it isn't no question.

So, back to the list from the start of this chapter: the test sheet, the sticker, the audit, the ledger, the proof, the confident AI. I'll give the straight answer on which of those this helps. The sticker is a question of whether a piece of paper is genuine, and nothing here touches it. The "all tests passed" sheet fits only if the test is a checker proved to mean what it says, in the machine's own language; a set of tests that can miss a bug certifies "this procedure said yes," which is true and smaller. An audit and a ledger get the test sheet's answer: kept by a program on an ordinary computer, they're bits the program wrote, and they fit only where the step underneath keeps the record and refuses to raise it without the check. A proof is one example: checking its steps is a procedure that can reject them. Whether its statement says what you meant is still yours to read. The AI that talks as if it's sure of itself isn't addressed at all. Section 22, under "Outward consequences," says how little of the outside world I've reached.

Now comes the honest half. Take away the small machine's rules and keep only the bare toll, and the copier has an easier move: pay one unit, flip the record to yes, and that's the whole move. The toll is satisfied and the receipt is real, but it's a receipt for nothing. The bare toll prices the closing of the door, and the check behind it goes unpriced.

14.15 Two layers, and the names

The layers, once. The core definition (Definition 2.1) is a Thiele machine: states, moves, a step, a cost for each move, a record in an order, and the toll; the two-place record (Definition 2.2) is its smallest case. Everything else is added on top, and each

theorem names what it adds. First come extra structures and requirements: a ledger that sums the costs, a window, permanence of the reading, a price on merges. Then an earned implementation, the small machine of the next chapter, with a checker proved sound and a provenance theorem. Then a universal interpreter for a stated class of guests. The bare definition doesn't ask the step to be computable, so when I say the record adds no computing power, I mean relative to a fixed flat part.

Substrate versus scaffolding. Two layers run through the book, and confusing them makes the theorems quietly stop meaning what they say. The *substrate* is the shape: a kind of state and a rule for stepping. Different theorems ask that shape for different extra pieces, and the pieces aren't the same. The universal floor (the "pays something" result of Section 2) asks for a record in an order and the toll on it. The counter's uniqueness (only one count fits a chosen start and price list) asks for a starting value and a schedule of increments; Section 7 gives the argument, a short induction that works for any system with those two pieces, and on the small machine of the next chapter the ledger is the sum of the charges by theorem (Section 15). The diagonal asks for a recursion theorem for the transformations the substrate can express, and needs no toll and no encoding at all. "Substrate-level" doesn't mean one fixed set of parts that every theorem shares. A theorem only gets to use the parts it asks for. Ask it what it needs and it'll tell you, and whatever else is lying around the workshop isn't the theorem's. So each result is stated over exactly the extension it uses. The substrate with the extension a theorem names is what I mean by *the model*, and the name Thiele Machine belongs to the model. The *scaffolding* is everything that holds the shape up so you can touch it, and when I say *my machine*, *the small machine*, *the machine I build* or *the build*, this is what I mean: the machine of the next chapter, with its two counters and their versions, its table of checked facts, its channel, trap, ledger and flag, its six instruction kinds and their prices, the host with a register for every whole number, and the universal programs that run on that host (Appendix C.1). The build fills in several extensions at once and borrows, for any one theorem, only the extension that theorem names. Let that line blur and a theorem stated over one extension starts reading as if it carried premises it never uses. "Substrate" never means the build, and it never means the side of a simulation doing the stepping; for that I say "the machine doing the stepping," as against "the program it runs."

So the book keeps the two apart: Parts I and II were about any system with the pieces each result names, and their pictures were small worked instances you could hold (two boxes on a sheet of paper, chairs in a room, a row of dots). The example machine wasn't one of them; where they said something about it, they pointed ahead, and the next chapter builds it. Every Part II claim that has a machine instance gets that instance in the next chapter or in Appendix C.1, with its own theorem, and Part II said so in one sentence at most. When the origin story (the

afterword, Section 24) says the shape "was the substrate the entire time," it means this layer, the model's shape. It doesn't mean any piece of the build. The same goes for two more of its words. When it says "the machine" about the thing it named the Thiele Machine, or calls that thing a meta machine, it means the model; "my machine" is the build of the next chapter. And when it lists what the model asks for (a state, a step, a record, a ledger, a window), read the ledger and the window as two of the extra pieces: a result asks for them only if it uses them.

The build is one piece of scaffolding, and I'd rather you know up front that it is one instance, chosen to be small enough to inspect. Section 15 builds the smallest honest machine I could: two counters, six instructions, a table of checked facts, a ledger and a flag. Earned commitments are proved there (a flag raised from a clean start was checked, then committed, then certified, on the same edition, with nothing touching that thing between the check and the commit), and so is universality, by matching Minsky's two-counter machines step for step. Then one fixed program, on a host made of the same kinds of instruction with more counters, runs every program of it, and with one paid move added, every computably presented Thiele machine (Appendix C.1). None of that is the model. It shows the model fits in six instruction kinds.

The names again, with the certified one pinned down, because I'll hold to them for the rest of the book. A Thiele machine is any particular system that fits the definition, its five parts and its toll (Definition 2.1); the toys on paper in Part I count, a clock counts, and so does the small machine, with its toll proved. The small machine is *certified*, which asks four things of it. The first is that the moves of its flat part can't touch the record, and that the record stays up once it's up. The second is a record that, from a clean start, rises only after a passing check of a claim, then a commitment to that same claim with nothing it's about changed in between, then a certificate, with the checker proved to mean what it says. Over a language fixed first, the claims and what they mean are that language's, and the proof is that the checker's yes makes the claim true in its sense; every instance in this book is stated that way. The third is exact prices: one unit for each of those three acts and nothing for anything else. And the fourth is a claim whose check, commit and certificate raise the record from a loaded start exactly when the claim holds there, with the claim holding at some loaded starts and failing at others. A machine that meets all four hides its record and its ledger from a window on its flat part, by theorem (Theorem 15.15). That's about an observer who sees only where a run ended; one who also has the program and the start can work both out. The small machine meets it, and so does the same machine over any properties with an exact test for telling two of them apart and a checker proved to say yes exactly when the property holds, as long as one of them can come out both ways, sorted lists included. A *universal* Thiele machine is one fixed machine running one fixed

program that runs any machine of a stated class as a guest, with every step of the guest's computing done by the host's own counter moves and every rise of its record put where its program put it; a certified one also earns each guest's record again by its own check, commit and certificate. Nothing is left to the guest's good manners. Appendix C.1 builds one, U: a fixed program in the small machine's own six kinds of instruction, on a host with more counters, with no step of any guest built in. Its guests are the small machine's programs, so every computation runs on it under Minsky's 2^n encoding, done outside the machine, and it matches each guest's halting, answer, flag and ledger exactly, raising its own flag by its own check, commit and certificate. Then U_P, the same program with one more handler for a paid move that does nothing else, takes as its guests every computably presented Thiele machine, one whose next move (picked by a driver), step, prices and reading are computed by programs, with the reading decided on every state. It matches the guest's halting, raises its own flag on its own earned chain exactly when the guest's reading has said yes, and keeps the guest's prices to within a surcharge of at most two when the guest's reading starts at no, which no program on that host, run from a clean start, can avoid in general. And a computer that runs the small machine, your laptop say, isn't the small machine. Read through the emulator's prices and flag it's a Thiele machine, a finite one, like the toys of Part I, and its counters stop somewhere. It implements the small machine, inside bounds, the way a laptop implements a Turing machine. Push on that comparison, because it's loose, and the looseness is the point. Does a laptop implement a Turing machine? Not exactly. Turing's tape never runs out, and a laptop's memory does; strictly, a laptop is a very large machine with finitely many states, and it behaves like a Turing machine only on computations that fit. Nobody minds, because the bound is known and most computations people care about fit inside it. That's the same deal any device running the small machine gets: its counters stop somewhere, and the theorems speak only for runs that fit. The bounds aren't fine print. They're the whole difference between implementing a model and being one.

How much can a difference in width matter? There's a famous answer, and it's worth the detour. On 4 June 1996 the first Ariane 5 rocket lifted off and broke apart about forty seconds after its engines lit. The inquiry board, chaired by the mathematician Jacques-Louis Lions, traced it to one conversion in the inertial reference software: a sixty-four-bit floating-point number, an alignment value called BH that's tied to horizontal speed, squeezed into a sixteen-bit signed integer. On Ariane 4, where the software came from, that value could never get big enough to overflow, because of how that rocket flew in its first forty seconds. Ariane 5 flew differently. The value got too big, the backup unit failed on it, the main unit failed the same way one cycle of its software later, seventy-two thousandths of a second, and the function doing the converting didn't even serve a purpose once the rocket

had lifted off; it was kept for commonality with the old rocket. The board's report says the assumption that made the conversion safe had been agreed on and then, not on purpose, kept out of sight of any outside review. A width premise that holds on one machine is a fact about how that machine gets used. The code doesn't carry it, and yet it travels with the code without anybody noticing. The only defence I know is the dull one: write the premise into every statement that needs it, where the next person will trip over it.

CHAPTER 15

The earned-chain counter example

The chapters before this one never built a machine you could program. They said "any states, any steps," and whenever a picture was needed, a board or some chairs or a cup stood in. Now I have to build a machine, and the first question is about the one I built before it. I built a big one first, on the way here (the Afterword, Section 24, tells that story), with dozens of instructions, some picked for reasons and some picked because I had to pick something, and theorems held of it. What does that show? That the model works, or that I'm good at stacking instructions until the proofs go through? A big machine is a wonderful place to hide a mistake. It's a garage so full of stuff that you could lose a car in it.

So the example machine in this book is a small one. I want the state and every way of changing it where you can inspect them. There are six instruction kinds, and you can follow how a check stays tied to an object, how a change makes it stale, and how the next move uses that relation. Each kind takes operands, a counter or a property or a line to jump to, so there are unboundedly many instructions. It's one way of building the model, not the only one, and the theorems will say which facts hang on these particular choices. I built it by throwing things out until nothing more would go. I haven't got a proof that nothing smaller works, and I'm not claiming one.

15.1 What the model asks for, and the example that meets it

The model asks for states, moves, a step, a price on each move and a record in an order, with the toll. Before any of its parts, I'll give the example that meets it in three sentences. Its record has two places, a flag that is the blank cell, and its flat part is two counters and a jump, the part a window on the counters shows. Raising the flag is the write of a 0, a merge in the record's part of the state (flag down and flag up land on the same state), which is why the step that raises it is charged once

merges in that part are priced (Theorem 3.19, in Part I). And the order rule, check, then commit, then certify, is what makes a raised flag mean something.

Parts I and II define the model by five parts and one rule. The example meets them with the smallest *record* there is, a flag that reads no or yes and, once yes, stays, which on a machine is the blank cell's 0. Here yes means "certified," because that's the event I picked (Section 18, later in this part, asks whether anything outside the rules picks it out). It adds a *toll*: the step that raises the flag costs something (Section 2). And it adds a *ledger*, the running total of those costs, a whole number that only climbs (Section 7). And Section 11 proved the thing this whole chapter stands on: over any base at all, a base-driven record is a *latch*, a bit that switches on at an event the base decides and never switches off (Observation 11.1). The record watches the base. It doesn't compute anything the base couldn't compute without it.

Push on that for a second, because it cuts the job in half. If the record adds no computing power, then all the computing power comes from the base, and the question "how small can the machine be" splits into two questions that don't touch each other. For the base: what's the smallest thing that can compute everything there is to compute? For the record: what's the smallest set of pieces that makes the flag *mean* something?

There's one honest asterisk on "don't touch," because the machine below makes it matter. What's blind is the flag and the ledger: the base never reads either of them. But the pieces that make the flag mean something (a table of checked facts, a slot for a commitment, a trap that stops a refused run) have to sit where the step can read them, which is in the base, and they do steer the run: a failed check stops a program dead. The latch theorem holds wherever you draw the line between base and record, which is exactly why it can't tell you where the meaning lives. On this machine the meaning lives on the base side of that line, in the bookkeeping, and the flag on top reports it. That line has to sit at the core: the step reads the table, the channel and the trap, so a map that hid any of them wouldn't move in step. The record-layer merge of Part I draws a lower line, with only the line number and the two counters below it, and what the premise charges moves with that line. Drawn there, all three record acts merge in the record's part and the premise charges each of them; drawn at the core, it charges only CERTIFY.

The record question, the second of those two, is the one Part II left to the machine, on purpose. Section 10 said that testing a claim and committing to it are two different acts, and that nothing in the five parts of the model makes the second wait for the first. Read Section 2's definition again with that in mind: a machine where you pay one unit and raise the flag over nothing in particular obeys every rule in it. That machine is honest about the price and silent about what was bought: it's a receipt for nothing. The toll alone can't fix that, because the toll is a rule about

cost, and "was this earned" is a question about *order*: did a check happen, of this claim, before the commitment, with nothing changing underneath in between?

This machine is my attempt to make the order part of the machine, in the smallest place I could find to do it. I'll show you the pieces, then the proofs, then the places it falls short, which there are, and which I'd rather you hear from me.

The blown fuse cannot be hidden from every future move

Blow the fuse and pull the lever. Nothing moves. Replace the fuse, start from the same counter values, and that lever can turn the gears. The fuse decides what the next pull does, whether the camera can see it or not.

That is what a failed CHECK does here. It traps the core, and every free instruction then leaves the whole state alone. Suppose I hide the failed check from the camera and still demand that the next picture depend only on the present picture and the free move. The live and trapped pictures agree. The camera's rule must give them the same next picture. Since the trapped box stays still, the projected live box has to stay still too.

□ **Theorem 15.1** (A globally check-silent native shadow is static)**.** Let π be any projection from native full states, with any target type. Suppose every CHECK preserves π, and each zero-cost instruction i has an autonomous projected rule h_i, so $\pi(\mathrm{exec}(s,i)) = h_i(\pi(s))$ for every state. Then $h_i(\pi(s)) = \pi(s)$ for every s, i. If π is onto, each h_i is identity everywhere. The conclusion also holds on the reachable states from a fixed start when those same laws hold throughout that reachable domain.

Proof. Stand at any native state s and read counter A. Ask CHECK whether that counter is at least one greater than the number just read. It is not, so this check leaves a trapped state t. In symbols, the chosen question is $A \geq A(s) + 1$.

By the proposed silence rule, the failed check has not changed the picture: $\pi(t) = \pi(s)$. Now choose any free instruction i. At the trapped state it changes nothing at all. The camera's rule h_i must therefore leave that picture alone. But it is also the picture of s, and h_i receives only the picture, not a choice between the two boxes. It must leave s's picture alone too. Written as one chain,

$$h_i(\pi(s)) = h_i(\pi(t)) = \pi(\mathrm{exec}(t,i)) = \pi(t) = \pi(s).$$

You could begin at any s and choose any free i, so every free projected move is still on the whole image. If every base picture occurs, that is the whole base. I did choose a different failing question for different counter values; the premise allows every CHECK at every state, so it allows each of those choices. Finally, if s was

reached from the fixed start, append this CHECK to that run to reach t. Both boxes remain inside the reachable domain, and the same argument applies there. ∎

The fuse belongs with the gears that read it. The certificate and ledger can sit beside them unread. The later slice follows the computing moves while the machine is live; a failed check takes it out of that working domain. That is where the line goes on this machine. [3]

15.2 The example machine, piece by piece

I'll go through the whole state a piece at a time, in words first.

- Two counters, A and B. Each holds a whole number, with no upper limit.
- A *version* for each counter, also a whole number with no limit. It starts at 0 and goes up by one every time its counter is written. Think of it as the counter's edition number: a new value is a new edition.
- A *line number*, which instruction comes next, counting from 1 (more on that choice just below).
- A *fact table*: a list of facts. A *fact* is three things stuck together: a property, a counter, and a version. "A is even, as of edition 3" is a fact. The table has room for sixteen. Why sixteen? It's a pick, with no derivation behind it, and what the pick gives up is set out in the subsection near the end of this chapter on what the machine doesn't do.
- A *channel*: either empty, or holding one fact. This is where a commitment sits.
- A *trap*: one bit, down at the start. When it goes up, the machine has refused something, and it stays up.

Counting lines from 1 looks like the least interesting choice in the chapter, and it's one of the oldest arguments in programming, with a famous referee. Edsger Dijkstra was a Dutch computer scientist, one of the generation that founded the field, and for over forty years he mailed numbered notes to a few dozen colleagues. In 1982 he wrote one of them, EWD 831, titled "Why numbering should start at zero." His case goes through ranges first. Of the four ways to write "2 up to 12" with less-than signs, he picks the one that takes the bottom in and leaves the top out, $2 \le i < 13$: two ranges that meet end to end then share one number written once, and the length is just the top minus the bottom. Start a sequence of N things at zero and it's $0 \le i < N$, and each thing's number is how many things came before it. He reports that programmers on the language Mesa, which allowed all four conventions,

[3] A view that exposes refusal, or a machine with a different failure rule, would supply a different interface. The obstruction concerns the native failure semantics and a globally CHECK-silent autonomous projection.

found the other three a steady source of clumsiness and mistakes, and he points out that FORTRAN starts at one. Does that settle it? It settles which convention is tidier, by an argument you can check. It doesn't settle which convention somebody else's work used, and that's the catch: the two-counter machine used by Forster, Larchey-Wendling and coauthors numbers its lines from 1, and so does this machine, because matching it line for line is worth more here than tidiness. Is zero the right place to start counting? Dijkstra's title says it is, and I can't tell you more than that. What his note carries is an argument about which way makes fewer mistakes. That part you can check for yourself, and it's the more useful thing to have.

Everything in that list is the *core*, the part the base moves. Outside the core sit the two pieces from the model: the ledger μ and the flag. The machine never reads either of them to decide what to do next. I'm not asking you to take that on trust: it's how the step is written, and it shows up as a theorem below.

The properties a fact can name come from a tiny fixed language with three words: *is zero*; *is even*; and *is at least n*, for any whole number n you like. Each one comes in two forms. There's the *checker*, a little program that computes yes or no: compare with 0, look at the last bit, compare with n. And there's the *meaning*, the plain arithmetic statement the checker is supposed to be checking: the number is 0; the number is two times something; n is less than or equal to the number. Why both? Because a checker is code, and code can be wrong in ways that look right, and "the checker says yes" is only worth anything once somebody has proved it says yes exactly when the meaning is true. That proof is one line per property, and it's the first block below.

Why can a machine check a claim at all? Because a claim with a precise meaning is something you *evaluate*: plug in the values and turn the crank. George Boole worked out in 1847 that you can do arithmetic on claims this way, treating "and" like multiplication and "or" like a kind of addition. Why multiplication? Try it at the table. Write 1 for true and 0 for false, and multiply: one times one is one, and anything times zero is zero. That's "and" to the letter. Then comes the stranger thing Boole noticed. Say "x and x" and you've said nothing new; it's just x. In numbers that's $x \times x = x$, and he made it a law of his system, and it's a law ordinary numbers mostly break. Which numbers equal their own square? Two times two is four, no. A half times a half is a quarter, no. Zero, yes. One, yes. That's the whole list. So when Boole wrote his bigger book in 1854, *The Laws of Thought*, he asked you to imagine an algebra whose letters take the values 0 and 1 and nothing else. Every checker on this machine answers in that algebra.

And the 1847 book has a backstory worth a detour, which I found in the Stanford Encyclopedia of Philosophy's entry on Boole. Boole was largely self-taught and spent about fifteen years as a schoolmaster, and what pulled him back to logic, early in 1847, was a loud public quarrel between Augustus De Morgan and the Scottish

philosopher Sir William Hamilton over who had first thought of a small technical idea in the old logic of syllogisms. Two well-known men fighting over credit, and a schoolmaster watching from the side picks up the subject and writes the little book this paragraph is about. Few people remember what the quarrel was over. Every checker in this chapter runs on what the bystander wrote.

Then push on it, because Boole's "or" isn't quite the one a checker uses. His plus only made sense for things that can't both hold at once, so "x or y" with both of them true had no meaning in his system. William Stanley Jevons, an English economist who also worked on logic, read "or" as "either or both," which gives "x or x is x," the same trick as before. He wrote to Boole in 1863 saying plus ought to work that way, and in 1864 he put it in a book called *Pure Logic*. Boole wouldn't have it, since it broke the tie to ordinary algebra his whole system stood on. The "or" in every programming language I know is the inclusive one Jevons argued for. Does that change anything about the checker? No; the crank turns the same way. It changes the picture: the "or" a checker uses is the one the man who invented the algebra turned down.

Hanging a checked claim on a point in a computation is an old move too. Tony Hoare, in 1969, built a whole way of reasoning about programs out of it: attach a claim to a *point* in a program ("whenever execution reaches here, this is true"), and reason forward from claim to claim. Robert Floyd had done it two years earlier on flowcharts. Their claims were checked on paper by the person writing the proof. This machine's version is a property of one counter at one edition, checked by a step while it runs.

And is two years earlier the bottom? Not even close, and the next step down is a paper this book already leaned on. In June 1949, at a Cambridge conference held for the new EDSAC computer, Alan Turing gave a talk three pages long, "Checking a large routine," the same note the counter chapter used for stopping (Section 7). Stopping was the second half of it. The first half is this move, twenty years before Hoare: the programmer should make "definite assertions which can be checked individually," and the correctness of the whole should follow from them. His example program computes a factorial using nothing but addition, and his picture for why you'd bother is one you can do on the back of an envelope. Add up a long column of four-digit numbers and hand someone only the total, and they have to redo the whole sum at one sitting, because the carries tie every column to the one before it. Hand them the total of each column as well, and the job falls apart into small checks they can do one at a time. Each column total is a claim hung on a point in the middle of the work, so the person checking never has to hold all of it at once. The paper got its due mostly later: in 1984 two computer scientists, F. L. Morris of Syracuse University and C. B. Jones of Manchester University, who had worked on it together at Oxford, republished it, with its misprints corrected, as an early program

proof by Alan Turing. Theirs is the text I read, and the column of numbers is Turing's own example in it. Does any of that change the machine? No. It changes the date on the idea, and it tells you something I like: the first person to ask how you'd check a program put the claims in the middle of it. He didn't save them for the end.

The machine has six instruction kinds.

Instruction	**What it does, when the trap is down**	**Cost**
INC *c*	Adds one to counter *c*, adds one to its version, goes to the next line.	0
DEC *c j*	If *c* is above 0: subtracts one, adds one to its version, jumps to line *j*. If *c* is 0: just goes to the next line.	0
HALT	Nothing. A program that reaches it stops.	0
CHECK *p c*	Runs the checker for *p* on *c*'s value. If it says yes and the table has fewer than sixteen facts, writes the fact (*p*, *c*, *c*'s current version) into the table and goes to the next line. Otherwise, trap.	1
COMMIT *p c*	Looks in the table for exactly the fact (*p*, *c*, *c*'s current version), field by field. If it's there, puts it in the channel and goes to the next line. Otherwise, trap.	1
CERTIFY	If the channel holds a fact, preserves that channel, goes to the next line, and the flag goes up. Otherwise, trap.	1

The first two are Minsky's machine, and HALT lets a program say it's done. It says it by doing nothing: HALT leaves the state where it is. If it put the trap up instead, it would land a live state on its trapped twin, a merge in the record's part, and the record-layer merge would charge it. The last three are the record's, and they read as a chain: you can only certify with a commitment in the channel, you can only

commit to a fact in the table, and the only way into the table is a check that passed. The chain has three links, and each one costs.

One of what? One unit of μ, the same unit as Section 2's toll. Why one? Why would ten be worse? Because one is the least each of these three acts can cost once a merge in the record's own part of the state is priced, as the end of this paragraph says, and Section 3 is where that pricing comes from. The argument for a whole finite machine doesn't reach this one: these counters have no top, and some of the base moves merge states at a price of zero. What does reach it is the record-layer merge named at the start of this chapter (Theorem 3.19, in Part I), which gives the toll once you've agreed to price a merge in the record's own part of the state. Agreeing to that is my choice. Once I have, one unit on each of the three acts isn't: COMMIT and CERTIFY merge when they pass, and all three merge when they fail, because a failure puts the trap up and changes nothing else, so the failed state lands where the same state with the trap already up lands. What I pick is one rather than more.

The whole numbers have a history of their own. In 1888 the German mathematician Richard Dedekind published a little book whose title asks what numbers are and what they're for, and in 1889 the Italian mathematician Giuseppe Peano published a booklet, written in Latin, that set out the axioms for the whole numbers in the form people still teach: there's a first number, and every number has a next one, and a few rules about how "next" behaves. I had the two of them down as working independently. I went looking, and the Stanford Encyclopedia's entry on Dedekind says Peano acknowledged Dedekind's priority, which is why the axioms often go by both names. (Peano's booklet is also where the sign $\in$ that you met in Section 6 comes from: he wrote it as a Greek epsilon, for the Latin *est*, "is.") Start from zero and take "the next one" three times, and you have three. Once prices are whole numbers, one is the smallest price that isn't nothing.

The zeros matter just as much: computing costs nothing on this ledger. Plus one, minus one and jumping around cost zero, with physical costs left to an implementation model. Only the three acts that make a claim cost, and each of them costs whether it passes or traps. Failing is not a discount.

The version rule. Some numbers show why versions exist. Start with $A = 4$, version 0. CHECK *is even* A passes, and the table gets the fact (*even*, A, 0). Now INC A: A becomes 5, and its version becomes 1. Now COMMIT *is even* A. The machine looks for (*even*, A, 1), the claim about the current edition. The table only has (*even*, A, 0). There's no match, so the machine traps. The old fact is still sitting in the table, still true about edition 0, and completely useless for committing to anything about edition 1. A fact is about one edition of one counter, and an edition never comes back, because versions only go up and never wrap. The version rule is the whole trick against stale evidence: you can't check a number, change it, and then

commit to the check. And the never-wrap part isn't a convenience. INC A, then DEC A back to the same line, puts the line and both counters exactly where they were, at no cost, and once record-layer merges are priced neither move may join two states that agree on those. If the record had only finitely many settings over one line and one pair of counters, enough round trips would bring the whole state back to where a CHECK left it, with A touched in between and its COMMIT still passing. Under those free-move, injectivity and continued-use requirements, the record fibre cannot be finite. This machine supplies fresh editions without a bound. A bounded implementation can instead stop before wraparound or change its invalidation and pricing rules; that changes the contract.

What the version rule stops is the time-of-check-to-time-of-use bug from Section 14, the file checked and then swapped, and the database answer to it is Kung and Robinson's: at commit time, check that nothing you read was written in the meantime, and back out if something was. The version rule is that check, done by the step, on one counter. It also shows where the guarantee ends. It works because one step does every write and every write bumps a version. A second writer that changed a counter without bumping its version, a second processor or a device writing memory on its own, would walk right around it, and none of the theorems below are about a machine with one of those.

Processor designers put a cousin of this check in silicon. Jensen, Hagensen and Broughton, at Lawrence Livermore in 1987, described a pair of instructions now called load-linked and store-conditional: the load reads a word and the processor starts watching it, and the store goes through only if nothing has written that word since; otherwise it fails and the program tries again. It's tempting to say CHECK is the load and COMMIT is the conditional store, and it's wrong in two places. COMMIT doesn't write the counter it checked, and the thing doing the watching, the fact, sits in a table the step keeps. No single processor holds it. The hardware version does cover the second writer, because the watching happens on the memory everyone shares, which is exactly the case the version rule leaves out.

The real kinship is older, and it's somewhere else. Picture the table and the counters shared by many processors, with CHECK, COMMIT and CERTIFY each done as one indivisible step. Maurice Herlihy (1991), a computer scientist who has spent his career on how many processors can share memory safely, ranked shared objects by how many processors each one can bring to agree on a single value with nobody ever waiting on anybody. Test-and-set, an instruction that writes a cell and reports what was there, brings exactly two. A sticky bit, the idea of Serge Plotkin (1989), a Stanford computer scientist working on distributed algorithms, is a cell the first writer jams for good, with the first writer's value, and everyone after can read it; it brings any number. I guessed that the shared object would sit at the top with the sticky bit, because a write that kills every fact about a counter, together with a

commitment other processors can see, looks like a way to build one. Ranking the object takes its operations, what each one returns, which steps are indivisible, how it starts and which processes run it, and then a proof that every processor decides, they all decide the same value, the value is one somebody proposed, and nobody waits. That proof goes through with every instruction one indivisible step and the whole state readable, and it puts the object at the top. The reason is the order of the fact table. Start it clean, with counter A at zero and the trap down. Each processor CHECKs a property that holds of A at zero, Zero to propose 0 and Even to propose 1, then reads the table and decides the bit of its oldest fact. Nobody touches A, so the first CHECK of the run passes. The table never loses an entry or reorders the ones it has, and a CHECK that finds it full trips the trap, after which nothing moves, so the first CHECK of the run fixes the oldest fact for good. Every processor that decides decides that bit, the bit is somebody's proposal, and each one decides after two of its own steps whatever the others do. The version rule and the commitment play no part. The trap alone doesn't do it: it records that something trapped, and it doesn't hand everyone the first writer's value.

Now take the version rule away, let failures change nothing, keep only the properties "at least n" so that a checked property only ever turns true, drop the decrement, and make the fact table (with no room limit), the commitments and the flag sets and latches that only grow. I guessed that then the object drops to the bottom rung, the level of plain memory, which is the shared-memory face of what the database researchers Joseph Hellerstein and Peter Alvaro (2020), at Berkeley and Santa Cruz, call CALM, where monotone means nobody has to coordinate. That guess is proved too, by Herlihy's own kind of argument. Let the updates return nothing and let the whole state be readable. If at every state, of any two updates, one leaves the state alone, or the two commute, or one overwrites the other, then no protocol brings two processors to agreement with nobody waiting. To see why, give one processor 0 and the other 1. At the start the answer isn't settled, since either one running alone has to decide its own bit. Nobody waits, so it can't stay unsettled forever, and there's a last unsettled state: whichever processor moves next settles it, one way if it's the first and the other way if it's the second. Look at those two moves. If one is a read, or an update that leaves the state alone, the other processor can't tell whether it happened, and running alone it decides the same bit either way. If the two commute, both orders end on the same state with both processors in the same place. If one overwrites the other, the overwriter running alone can't tell whether the other move happened first. Each case contradicts the two ways the answer was settled. The monotone object, with registers beside it, has that property. An update that fails changes nothing. Two updates that both succeed still succeed in either order, because an instruction that would succeed still succeeds after anything else has run, and adding to sets and to counters gives the same result in either order.

Two of the conditions do real work there. The table needs no room limit: a full table that turns away the latecomer is a first-writer-wins cell again, and that's back at the top. And the ledger: it changes neither result as long as a failed act in the variant isn't charged, or nobody reads the ledger. Charge a failed CHECK where everyone can read the total, and one processor INCs while the other CHECKs *at least 1*; the total and the table then tell each of them which act came first, and that's enough for two to agree. Neither idea is mine: the top is Plotkin's and the bottom is the monotone fact that field already knows.

Process-calculus people have a statement of the same split. Johannes Åman Pohjola and Joachim Parrow, computer scientists at Uppsala University (Parrow is one of the inventors of the pi-calculus, a mathematical model of processes that talk over channels), in 2016 proved that no model whose parallel composition is monotonic, where a neighbouring process can never stop a step, has a consensus process. One of their examples of a model that does is test-and-set. Test-and-set isn't a sticky bit: in Herlihy's ranking it brings two processors to agree, and a sticky bit brings any number.

Their setting is small enough to go through here whole. A model is any set of processes with a step relation and a way to compose two processes that doesn't care about order or grouping. Composition is monotonic when whatever a process can reach alone, it can still reach with anything set beside it. A consensus process is one where any number of copies, run together, can end up agreeing on either of two outcomes; once a run has one outcome it can never show the other; and an outcome, once reached, survives having more copies, run any way you like, set beside it. Their theorem goes through in a few lines: run half the copies to one outcome and the other half to the other. If neither half can block the other, the whole crowd can reach the pair of end states set side by side, and since each outcome survives the company, both outcomes hold there, which is forbidden. Then the machine goes into it. A process is a shared machine state with a crowd of threads, each proposing a bit. Composing two processes pools their threads, each thread keeping whatever it has already decided, and settles the shared state like this: if one side is still at the fresh start, the composite takes the other side's state; if both have already moved, the composite is jammed, a state where no thread can take a step. That composition doesn't care about order or grouping: a fresh start is neutral, and any two moved states jam whichever way round you put them. One thread proposing each bit is a consensus process in exactly their sense, for the outcomes "some thread decided 0" and "some thread decided 1": copies started fresh can reach either outcome and never both, because the first passing check fixes the oldest fact and every later thread reads the same one; and an outcome survives company, because pooled threads keep their decisions. So by their theorem the composition isn't monotonic, and the small machine sits on their non-monotonic side. That verdict is cheap, though:

a composition that jams any two moved states makes every shared-state machine non-monotonic, so the encoding puts it there, not the fact table. What does the work in the run, making every thread read the same outcome, is the order of the fact table, the same as above; a store that refuses a stale commit for good, as the version rule does, refuses in the same spirit. The shared-memory version of their split is the pair of results above: the small machine's object reaches agreement for any number of processors, and the monotone variant, with its table unbounded and no charge for failing that anyone can see, doesn't reach it even for two.

Security people know the pattern by a third name. A check that blesses a value, and a write that takes the blessing away, is close kin to dynamic taint tracking, where a label rides along with the data and a sanitizing step changes it; the labels go back to the computer-security researcher Dorothy Denning's 1976 lattice model of how information is allowed to flow. How much a monitor like that has to remember is a studied question too: Chamseddine Talhi, Nadia Tawbi and Mourad Debbabi (2008), three security researchers at universities in Quebec, worked out which rules a monitor can still enforce when its memory is bounded. The version rule is one small, specific member of that family, done by the step.

Notice what *isn't* a write. DEC on a counter that's already 0 doesn't change the counter, so it doesn't change the version. It's a test that happens to move the line number.

The fact table. Facts go in and never come out. Nothing erases them and nothing overwrites them. When the table is full, a seventeenth CHECK traps rather than pushing an old fact out to make room. That choice has a downside, and I'll set it out in the subsection on what this machine doesn't do.

Traps. Any failure puts the trap up and changes nothing else in the core; the ledger still takes the instruction's cost. Once it's up, every instruction leaves the core alone. A stored program stops at a trap. But I also prove things about *any* list of instructions, programs or otherwise, because an adversary doesn't have to run my programs; they can feed the machine whatever they want. For those lists, an instruction after a trap is still charged its cost, and still changes nothing. That trap keeps nothing of what tripped it, and the record-layer merge charges it for that. Merely counting refusals does not remove the merge. A live state with count zero and its already trapped twin with count zero can both become trapped with count one. Keeping the entire previous state in a history would make each extended move injective, but that is a different, unbounded state space. The counter alone is not that history.

The exact version comes next.

Definition 15.1 (Properties, checker, meaning)**.** A *property* is one of: Zero, Even, or Ge(n) for a whole number n. The *checker* eval(p, v) computes yes or no: $v = 0$ by equality, evenness by the parity function, $n \leq v$ by comparison. The *meaning* holds(p, v) is the arithmetic statement: $v = 0$; there is an m with $v = 2m$; $n \leq v$.

□ **Lemma 15.2** (The checker says what it means)**.** For every property p and every whole number v, eval$(p, v) =$ yes if and only if holds(p, v).

Proof. Ask whether the number is zero: the equality test says yes at zero and no at every other number. Ask whether it is at least n: the comparison says yes precisely when n is no larger than it. Those two tests already answer the arithmetic questions they name.

For evenness, the checker says yes at zero, no at one, and adding two leaves its answer unchanged. Take any whole number and remove pairs until fewer than two remain. If none remain, the number was two times the number of pairs removed, and the checker gives zero's answer, yes. If one remains, it was two times that many pairs plus one, and the checker gives one's answer, no. Thus yes means exactly ■ $v = 2m$ for some whole number m. These are all three properties in the language.

Definition 15.2 (Core, state, claim, start)**.** A *core* k holds two counters A, B and their versions (whole numbers), a line number, a fact table (a list of facts, a fact being a triple of a property, a counter and a version), a channel (empty or one fact), and a trap bit. Write val(k, c) and ver(k, c) for counter c's value and version. A *state* s is a core, written core(s), with a ledger $\mu(s)$ and a flag cert(s). The *claim* p of c in k is the fact $(p, c, \text{ver}(k, c))$. The table's room is 16. The start start(a, b) has counters a and b, versions 0, line 1, an empty table, an empty channel, the trap down, $\mu = 0$ and the flag down. A state is *clean* when its table and channel are empty and its flag is down.

Definition 15.3 (Instructions, costs, the step)**.** The instructions are INC c, DEC c j, HALT, CHECK p c, COMMIT p c and CERTIFY, with effects on a core with the trap down as in the table above. Passing any check, commitment or certification requires the trap to be down. With that condition, CHECK passes when eval$(p, \text{val}(k, c)) =$ yes and the table holds fewer than 16 facts, COMMIT passes when the claim p of c is in the table, and CERTIFY passes when the channel holds a fact; a failing one sets the trap and changes nothing else. On a core with the trap up, every instruction

changes nothing. Costs: CHECK, COMMIT and CERTIFY cost 1, the rest 0. One step on a state is

$$\mathrm{exec}(s,i) = \big(\,\mathrm{cexec}(\mathrm{core}(s),i),\ \ \mu(s)+\mathrm{cost}(i),\ \ \mathrm{cert}(s)\vee\mathrm{fires}(\mathrm{core}(s),i)\,\big),$$

where cexec is the core's step and $\mathrm{fires}(k,i)$ is yes exactly when i is CERTIFY and passes at k. For any list t of instructions (a *trace*), $\mathrm{run}(t,s)$ applies them in order and $\mathrm{cost}(t)$ adds their costs. A *program* is a list of instructions numbered from 1; a program run fetches the instruction at the line number and executes it, and stops when the trap is up, the line holds HALT, or the line holds nothing.

Read the step line slowly, because the rest of the chapter leans on its shape. The core moves by its own rule and nothing else. The ledger adds the cost. The flag becomes "it was up, or this is a CERTIFY that passed." The core's move doesn't mention μ or the flag anywhere. The base is blind to the record by construction.

The shortest worked run there is uses the program CHECK *is zero* A; COMMIT *is zero* A; CERTIFY. Start at $\mathrm{start}(0,b)$, for any b. Line 1: A is 0, the checker says yes, the table is empty so there's room, and the fact (*zero*, A, 0) goes in. $\mu = 1$. Line 2: the claim about A's current edition is (*zero*, A, 0), and it's right there, so it goes into the channel, and $\mu = 2$. Line 3: the channel holds a fact, the flag goes up. $\mu = 3$. Line 4 has no instruction, so the program stops. The run ends with three paid acts, in order, a flag that's up, and a channel that names the last earned commitment, which here is (*zero*, A, 0).

Now start the same program at $\mathrm{start}(5,b)$. Line 1: A is 5, the checker says no, the trap goes up, $\mu = 1$, and the program stops right there. The flag is down, and you can run it as long as you like: it stays down. That second run is the point of the first. The same three instructions certify from some starts and refuse from others, because the check could have failed, and from most starts it does. A check like that has a no pile, which is the only kind Section 14 counts as establishing anything.

Could I have picked an even dumber claim? I could: CHECK *at least 0* A passes on every number there is, so CHECK, COMMIT, CERTIFY on it raises the flag for 3 from every start. The step doesn't ask whether the question was worth asking, and it shouldn't. Worth is a count over the rivals a run narrows (Section 9), and a step that sees one state has no list of rivals to count. What the step checks is order, aboutness and that the checker tells the truth. What a check that can't fail is worth, the entitlement count says exactly: zero bits, for a cost of three. The price is for standing behind the claim. What the claim bought is a separate number, one the step doesn't compute, and I keep the two apart. It's on the list at the end of the chapter.

Others walked into the same line from communication. Three theoretical computer scientists, Oded Goldreich, Brendan Juba and Madhu Sudan (2012), built a

theory of communication with a partner you may not understand, in which getting a message across counts only as a way to reach a goal, and the central notion is *sensing*: being able to tell, from what you see, whether you're making progress. They ask two things of it. It has to be *safe*: with any partner in the class, when there's no progress it keeps saying so, so it can't report a lasting success that didn't happen. And it has to be *viable*: with a partner who could help if you spoke its language, the right way of speaking gets a yes from it within a known time, and keeps getting one. Either half alone is cheap. A sensor that always says no is safe, and one that always says yes is viable. Their main result, as I read it, is that the two together, with the ways of speaking listed one after another, are enough for one strategy that reaches the goal with every such partner. Their theorem, in their setting, is a check that can fail and never lies, standing in front of a verdict you can't take back, and it's the same shape.

15.3 It computes everything two counters can

First the flat part, because the example is meant to run every program, and that has to be shown. Put the ordinary two-counter program beside this machine's program. Replace each "add one" by this machine's INC, and each "subtract if possible and jump" by its DEC, keeping the named counter and jump destination. That is all the translation does.

Now cover the facts, versions, channel, flag and ledger with a sheet. Leave the two counters and line number visible. I want to show that the visible part follows the original program exactly. The hidden versions may change, but these instructions cannot raise the trap, so the sheet never hides a new refusal that would stop the comparison. Here are the two programs and the window written precisely.

Definition 15.4 (Two-counter program, compile, window)**.** A *two-counter program* M is a list of instructions, each either MINC c or MDEC $c\ j$. A configuration is a triple (line, a, b). One step of M: if the line holds no instruction, M has *stopped*; MINC adds one to c and goes to the next line; MDEC subtracts one from c and goes to line j if c is above 0, and goes to the next line if c is 0. $\mathrm{mrun}(n, M, x)$ is n steps from x, staying put once stopped. The *compile* of M replaces each MINC c by INC c and each MDEC $c\ j$ by DEC $c\ j$. The *window* of a core k is the triple (line number, $\mathrm{val}(k, A)$, $\mathrm{val}(k, B)$).

Theorem 15.3 (Step for step)**.** For every two-counter program M and every core k with the trap down: M has stopped at window(k) if and only if the program compile(M) has halted at k; and if M steps from window(k) to y, then one program step of compile(M) from k lands on a core with window y and the trap still down. So for every n, n program steps of compile(M) from k land on a core whose window is mrun$(n, M,$ window$(k))$, with the trap down. □

Proof. Look up the instruction at the line. If there is none, both sides stop. If it's MINC c, the compiled INC c adds one to c and goes to the next line, which is exactly the window's move (the version also moves, but the window doesn't show versions). If it's MDEC c j, split on whether c is 0, and both sides do the same thing in each case. Neither instruction can trap. After that move the windows agree again and the trap is still down. Those are exactly the conditions I began with, so the next instruction can be compared in the same way. Repeat until the chosen n steps have been counted; if either side stops earlier, both stay put for the remaining comparisons. ■

Theorem 15.4 (Halting matches)**.** For every two-counter program M and whole numbers a, b: M stops after some number of steps from $(1, a, b)$ if and only if the program compile(M) halts after some number of program steps from start(a, b). □

Proof. The start has window $(1, a, b)$ and its trap is down. Apply Theorem 15.3 after any number n of steps: the two configurations still have the same window, the trap is still down, and one is stopped exactly when the other is halted. The same n witnesses termination in either direction. ■

Now comes the part that isn't mine. Two-counter universality is Minsky's, and the constructions of Forster, Larchey-Wendling and their coauthors connect it to several other models. The counter convention matters: DEC at zero goes on to the next instruction; otherwise it takes one off and jumps, the convention used in Andrej Dudenhefner's certified decision procedures for two-counter machines (2022). Their route runs from a tape to counters and packs finitely many counters into two. The remaining step here is to match that two-counter machine to this machine's INC/DEC fragment, instruction for instruction.

I want to be exact about what "undecidable" means in the theorem below, because it's stated as a connection between two problems. Suppose you had a procedure that answered the halting question for this machine. Then you could write a program

that lists, one by one, every single-tape Turing machine that *doesn't* halt. No such list can exist: the ones that *do* halt can already be listed (just run them all, a step at a time, and call out each one as it stops), and with both lists running side by side you'd have a halting decider. So the theorem says: a decider here would give you that impossible list.

□ **Theorem 15.5** (Halting is undecidable for this machine)**.** The two-counter machine with the instruction convention just stated agrees with the INC/DEC fragment: its step, its stop, and its termination match those of the compiled program, so its halting problem at (M, a, b) has a positive answer if and only if compile(M) halts from start(a, b). Consequently, the question "does program P halt from start(a, b)?" is undecidable in the following reduction sense: a decider for it would make the complement of single-tape binary Turing machine halting enumerable.

Proof. The agreement is a case split on the counter-machine instruction at the current line, and then induction along the run, ending in Theorem 15.4. Composing a proposed decider for the compiled program with the compile map would decide two-counter halting. The tape-to-counter reduction would then decide single-tape ■ binary halting and so list its complement, as claimed.

What that buys is that the base of this machine is a universal machine in Minsky's sense, and that no algorithm can predict in general what it'll do. What it doesn't buy is a fast machine, or one where your input arrives conveniently. The counters start holding a and b, given directly; there's no encoding layer in this construction. Minsky's prime-power packing is how you'd get from "any Turing machine" to "two counters," and I use that construction as it stands.

Is "given directly" just a convenience, or does it bite? It bites, and somebody found exactly where. In 1972 Rich Schroeppel, then at the MIT AI Lab, wrote a ten-page memo titled "A Two Counter Machine Cannot Calculate 2^N." He spells out the convention first: a counter machine computing a function starts with n in one counter and zero in the others. Then he gives a proof that a two-counter machine started that way can't compute 2^n. That's his theorem, and I don't reprove it. A cousin holds on this machine: no program multiplies every input by one large constant fixed by the program's length. So the packing Minsky needs is precisely the thing two plain counters can't do for themselves. Hand the small machine its inputs straight and there are functions of them it can't compute; hand it the inputs already packed and it computes anything a Turing machine can. That's no contradiction with the undecidability above, which ranges over every program and every starting pair, packed ones included. The base hides nothing; it just needs its input pre-chewed, and the chewing is somebody else's.

15.4 It's a Thiele machine, and the flag is a latch

Now it's the record's turn. There are three facts to state first, and all of them read straight off the step line.

Theorem 15.6 (Ledger, toll, permanence)**.** For every state s and instruction i, $\mu(\text{exec}(s,i)) = \mu(s) + \text{cost}(i)$, and for every trace t, $\mu(\text{run}(t,s)) = \mu(s) + \text{cost}(t)$. If $\text{cert}(s)$ is down and $\text{cert}(\text{exec}(s,i))$ is up, then i is CERTIFY, it passes at $\text{core}(s)$, and $\text{cost}(i) \geq 1$. If $\text{cert}(s)$ is up, so is $\text{cert}(\text{exec}(s,i))$. □

Proof. Each instruction adds its listed charge to the ledger. Start at $\mu(s)$: after the first instruction its charge has been added, after the second both charges have been added, and every further instruction adds its own. An empty list adds nothing. Thus any finite trace leaves exactly the starting ledger plus the sum of the charges.

The flag has two ways to be up afterwards: it was already up, or this instruction successfully certified. If it was down before and up afterwards, only the second way is available. The instruction must be CERTIFY, its check of the channel must pass, and its listed charge is one. If the flag was already up, the first way keeps it up regardless of the next instruction. ■

The middle clause of those three is Section 2's toll, for this machine, which means this machine *is* one of the things Section 2 was about, and Section 2's universal floor applies to it without my lifting a finger.

Theorem 15.7 (The small machine is a Thiele machine)**.** The states, the instructions, the step exec, the cost and the flag, with the toll of Theorem 15.6, form a Thiele machine (Definition 2.2). So the universal floor (Observation 2.2) holds here: if $\text{cert}(s_0)$ is down and $\text{cert}(\text{run}(t,s_0))$ is up, then $\text{cost}(t) \geq 1$. □

Proof. Take the state and instruction sets of Definitions 15.2 and 15.3, their step and cost, and the flag as the reading. These provide the five components of Definition 2.2. Theorem 15.6 supplies its one required rule. Applying Observation 2.2 gives the claimed bound. ■

Section 11 comes next. Section 11 said: take any base, and any record that answers only to the base and never comes back down, and the record is a latch. This machine has a base: a program together with a core, stepping by the program step. And it has a record, the flag, with the ledger riding along. So the question is whether it's a *base-driven* extension in Section 11's five-rule sense (Definition 11.2), and if it is, Section 11 does the rest.

□ **Theorem 15.8** (The flag is a latch of the base)**.** Let the base have states (program, core), stepping by the program step, starting at start(a, b)'s core for any a, b, and halting when the program has halted. Map each (program, state) to (program, core). Then the machine, run as a closed machine on (program, state), is a base-driven record over that base, and so its flag factors as the latch (Observation 11.1) of the event "the next instruction is a CERTIFY that passes."

Proof. First the cover (Definition 11.1): the map sends starts onto starts, it commutes with the step because the core's move never reads μ or the flag, and whether the program has halted is read off the core. Then the rules of Definition 11.2 one at a time. Driven by the computation: the next flag is the old flag or "the next instruction is a passing CERTIFY," which is a function of the base state and the old flag (that's the step line). Ledger: μ only adds. Toll: Theorem 15.6. Permanence: Theorem 15.6. A reachable write: the three-line program of the worked run raises the flag from a start. Section 11's theorem then gives the factorization. ■

So far there's nothing new. The toll, the floor and the latch are things Parts I and II already promised about any machine of this kind. The next subsection is the part neither of them could give.

The lamp and the claims it does not name

The lamp is on. Which claim turned it on? Did another claim get certified afterwards? Look at the lamp as long as you like. It still gives you one answer.

Put a file beside it. Every time CERTIFY succeeds, copy the fact from its channel into the file: the predicate, the counter it concerns, and that counter's edition. Call the set of filed facts R. Publishing the same fact again leaves the same set. A later write can change the counters while the old receipt stays filed under its old edition.

Run this record alongside the native state. The native instructions keep doing exactly what they did before. Forget the file and you recover their run. Open the file and you can ask which claims were actually published.

□ **Theorem 15.9** (The native lamp is the archive's Boolean reading)**.** From a clean start with an empty added archive, after every finite native instruction word,

$$\mathrm{cert}(s) = \mathrm{true} \quad \Longleftrightarrow \quad R \neq \varnothing.$$

Membership in R means exactly that this fact was emitted by a successful CERTIFY earlier in that word. Already recorded facts persist, and zero-cost instructions cannot add a fact.

Proof. Start with the empty archive and the flag down. Inspect the next instruction. A successful CERTIFY inserts its channel fact into R and raises the flag. Whether or not that fact was already present, the resulting set is nonempty and the flag is up. Every other case leaves both the archive and the flag alone, including failed or trapped instructions. This includes every horizontal move. It follows, one instruction at a time, that a fact belongs to R exactly when some earlier successful CERTIFY published it, that no later move removes it, and that the flag is up exactly when R is nonempty. Forgetting R leaves the native updates untouched, so it also recovers the native run. ∎

The lamp tells you whether that file is empty. The file tells you what is in it. Below, I will make 65,535 different nonempty files leave the entire native machine in one identical state. Before that, follow how a fact earns its place: CHECK establishes it in the table, COMMIT selects it, and a successful CERTIFY publishes it. Each step has a job.

15.5 Commitments are earned

The shape comes first, before the words. Pick any run from a clean start, any list of instructions at all, chosen by somebody who'd love to cheat. Look at the moment a COMMIT would pass. Then, somewhere earlier in that run, there's a CHECK of exactly the same claim that passed and paid, at the same edition of the counter, and between that CHECK and now, nothing touched the counter. That holds on every list of instructions anybody can write, not "probably" and not only "on the programs I wrote."

To say "nothing touched it" exactly takes one word.

Definition 15.5 (Untouched)**.** A list of instructions *mid*, run from a state s, leaves counter c *untouched* when, for every way of splitting *mid* as t_1, then i, then t_2, executing i at the core reached by running t_1 from s leaves both $\mathrm{ver}(\cdot, c)$ and $\mathrm{val}(\cdot, c)$ as they were.

Theorem 15.10 (A commitment was checked first)**.** Let s_0 be clean and *pre* any trace after which COMMIT p c would pass. Then *pre* splits as pre_1, then CHECK p c, then *mid*, where: the CHECK passes at the core reached by pre_1; CHECK p c and COMMIT p c each cost 1; c's version after pre_1 equals its version after *pre*; and *mid*, run from just after the CHECK, leaves c untouched. □

Proof. COMMIT passes, so the claim $(p, c, \mathrm{ver}(c))$ for the current version is in the table. By the no-forging invariant (Theorem 15.14 below), that fact was written by a passing CHECK p c at some point pre_1, with the version c had then; since it's the current version too, the two versions are equal. Versions never go down, and any instruction that changes a counter's value also raises its version, so an instruction in *mid* that touched c would have left c's version higher at the end than at the CHECK. It isn't. So nothing in *mid* touched c. ■

□ **Theorem 15.11** (A raised flag was checked, then committed, then certified)**.** Let s_0 be clean and t any trace with $\mathrm{cert}(\mathrm{run}(t, s_0))$ up. Then for some p and c, t is

$$pre_1,\ \text{CHECK } p\ c,\ mid_1,\ \text{COMMIT } p\ c,\ mid_2,\ \text{CERTIFY},\ post,$$

where the CHECK passes at its state, the COMMIT passes at its state, the CERTIFY passes at its state, c has the same version at the CHECK as at the COMMIT, and mid_1 leaves c untouched. And when the CERTIFY runs, the channel holds exactly the claim that COMMIT made: the fact (p, c, c's version at the CHECK). CERTIFY takes no operand; this is what ties it to the claim.

Proof. The flag starts down, so some first instruction raised it, and by Theorem 15.6 it's a passing CERTIFY. A CERTIFY passes only with a fact in the channel, the channel starts empty, and only a passing COMMIT puts a fact there, so some earlier COMMIT p c passed. Take the last one before the CERTIFY: the fact in the channel is the one it wrote. Theorem 15.10 at that COMMIT gives the CHECK, the equal version, and the untouched stretch, and the equal version makes the committed fact (p, c, c's version at the CHECK). ■

The failure side comes next, so you can see the refusals are clean, with nothing left half-done:

□ **Theorem 15.12** (Unearned steps trap and change no claim)**.** If COMMIT p c would not pass at a core k, executing it puts the trap up and leaves the channel and the table as they were. If CERTIFY would not pass at a state s, executing it puts the trap up and leaves the flag as it was.

Proof. If the trap is already up, the core is unchanged and its trap remains up. Otherwise, the failed COMMIT takes the failure branch of the instruction table: set the trap, leave every other core field alone. A failed CERTIFY takes the same branch,

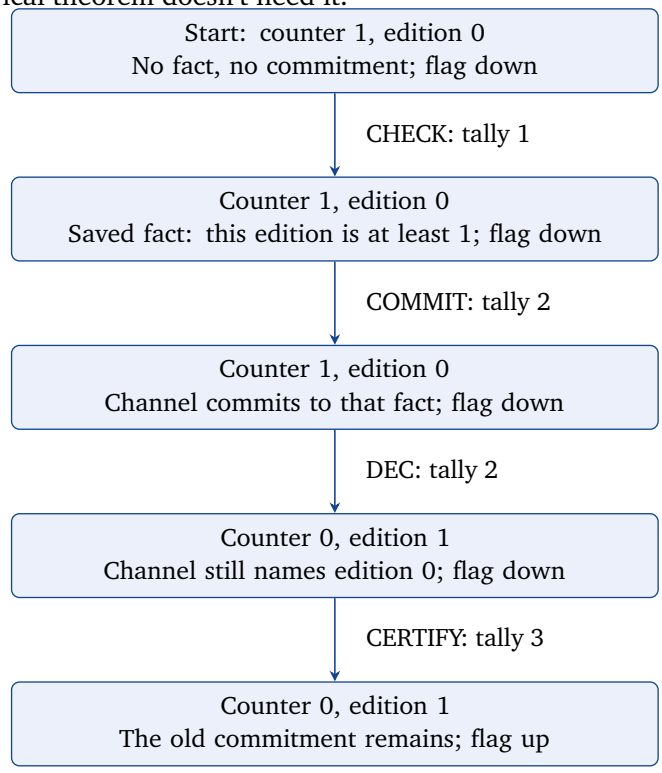
and its fires value is false. The flag update is then its old value or false, which leaves it unchanged. The instruction's charge still applies to the ledger. ∎

If that rings a bell, it should. It's Milner's move from Section 14: in LCF a theorem is a value that only the rules of the logic can build, so nobody can just print one (Milner and two colleagues on his Edinburgh project, Michael Gordon and Christopher Wadsworth, wrote the system up in 1979, and Gordon, who went on to build the HOL prover at Cambridge, told its history in 2000). Here the thing nobody can print is one bit, and what guards it is one step rule. Milner's guard was a type checker. Same architecture, much lower down.

A provenance theorem is also an itemised receipt, and the itemised receipt has a history that's exactly this gap. James Ritty ran a saloon in Dayton, Ohio, and believed some of his staff were pocketing money. On a steamboat trip to Europe in 1878 he watched a mechanism that counted the turns of the ship's propeller, and wondered whether something like it could count the money instead. In 1879 he and his brother John patented a "Cash Register and Indicator," and they sold a version under the wonderful name Ritty's Incorruptible Cashier. The first model was this book's counter (Section 7) with brass on it: press a key, the amount goes onto a dial showing the day's running total. There was no drawer and no paper. At closing time you hold the dial up against the cash. And that's all a total can do. It tells you *that* the money is short. It can't tell you which sale went missing or who rang it up. The paper came later. The patents passed to other owners, first a Cincinnati salesman named Jacob Eckert, and in 1884 to John H. Patterson, who renamed the business the National Cash Register Company, and the story told about that company is that it added a paper roll: one copy kept for the shop's own books, one handed to the customer. That last part I only have from histories of the company and of the machine, so hold it the way you'd hold any company's story about itself. The order is right either way: first a total that can tell you something's wrong, then a list that tells you where. The ledger μ is the dial. Theorem 15.11 is the list: which CHECK, of which claim, about which edition, before which COMMIT. The theorem supplies that list about the run. The machine doesn't keep it as data anyone can read back from the end state, and nothing in the theorem gives a procedure for rebuilding it from there. A fact in the table holds a property, a counter and a version number. That's neither the counter's value back then nor a log of what happened in between.

Read Theorem 15.11 once more for what it *doesn't* say. Between the CHECK and the COMMIT, the counter is untouched. Between the COMMIT and the CERTIFY, it isn't promised anything. You can check A is even, commit to it, add one to A, and then certify. The flag goes up, and what it stands on is a commitment about the edition that was current when you committed. The certificate is historical: it says "this was checked and committed about this edition," and it says nothing about

whether that's still true. I think that's the right meaning for a certificate (a birth certificate doesn't expire when the baby grows), but it's a meaning, and you should know which one you're getting. So a certificate doesn't say the claim holds at the CERTIFY, and it doesn't say the claim holds at the end of the run. A CERTIFY that rechecked the version would give a different guarantee. That's a possible extension, and the historical theorem doesn't need it.



The checked claim was true of edition zero. The raised flag does not say that the current counter is still at least one.

15.6 What a passing check means

Provenance says a CHECK happened. It doesn't yet say the CHECK was *right*, and that's a separate question with a separate proof. A machine could have a perfectly ordered chain of check, commit, certify, all standing on a checker that says yes to everything. So what does a fact in the table actually tell you about the counter?

□ **Theorem 15.13** (Checker soundness)**.** Let s_0 be clean and t any trace, and let k be the core of $\mathrm{run}(t, s_0)$. For every fact (p, c, v) in k's table with $v = \mathrm{ver}(k, c)$, the

property holds of the counter's current value: $\mathrm{holds}(p, \mathrm{val}(k, c))$. In particular, if COMMIT p c would pass at k, then $\mathrm{holds}(p, \mathrm{val}(k, c))$.

Proof. Carry one sentence through every instruction: every fact in the table has a version no higher than its counter's, and if its version equals its counter's, its property holds of its counter's value. A clean start has an empty table, so the sentence holds there. A passing CHECK adds a fact at the current version, and the checker said yes, so by Lemma 15.2 the property holds. A write raises one counter's version, so every old fact about that counter falls strictly behind and the second half no longer applies to it. Nothing else changes a value or a version. The sentence holds after every instruction, so it holds at the end. The part about COMMIT is the case where the claim is in the table at the current version. ∎

So, on a run from a clean start, a live fact means exactly that one of three arithmetic properties holds of one counter's current value. It doesn't mean "the program is correct," and it doesn't mean "the number is interesting"; it means the property holds of the value, now. That's all "checked" means on this machine, and it's enough to be true. Three events stay separate here. The property holding of the value is one. A CHECK passing is another: a CHECK of a property that holds still doesn't pass when the table is full or the trap is already up. A certification is a third, and it needs a COMMIT and a CERTIFY after the CHECK. So a CHECK that didn't pass says its property is false only when the trap was down and the table had room before it.

So who checked the checker? Nothing on the record did. From a clean start, the record carries the run's half of the story: a CHECK, then a COMMIT of the same claim about the same edition, then a CERTIFY. The other half, that the checker says yes exactly when the property holds, isn't in any run. It's Lemma 15.2, a proof about the little program the step calls, priced by nothing on the ledger. So a claim established here stands on two legs: a fact about this run, carried by the record, and a fact about the checker, carried by a proof that whoever reads the record has to accept. That's the Münchhausen trilemma from Section 14 taking its third horn on purpose: the chain of reasons stops at a proof, named and on the page, where you can decide whether you'd have stopped there too. And the entitlement it gives is the reader's. The machine doesn't know anything. It keeps the order.

The question in that paragraph is older than any checker, and its most famous wording isn't about what you'd guess. "Who will guard the guardians?" gets quoted about governments and police and the people checking mathematical proofs. Where's it from? Plato's *Republic* has the worry, sort of: in Book III (at 403e), Glaucon, Plato's older brother and one of the people Socrates is talking with, says it would be absurd for a guardian to need a guardian, which is close in words and opposite in mood, a man calling the question silly. The Latin, *quis custodiet ipsos*

custodes, is from the Roman satirist Juvenal's sixth satire, and it's a joke about a husband. His friends tell him to bolt his wife in and set guards on her; the line asks who'll watch the guards, since she'll start with them. And it gets better. In 1899 an Oxford undergraduate, E. O. Winstedt, found extra lines of that satire in a medieval manuscript in the Bodleian Library, lines the other copies lack, partly overlapping the familiar ones, and the famous question sits in that tangle. Scholars have argued ever since about whether those lines are Juvenal's at all. So the most quoted sentence about trusting the checker comes with a doubtful source of its own. Does that change anything here? It doesn't. The answer used here is simple: you never get rid of the last guard, you make the last guard small enough to watch yourself. Lemma 15.2 is one line per property. Euclid had the instinct: five postulates you can hold in your head, and everything else meant to follow from them.

That split, a fixed checker proved sound from outside and a process that only gets to act when the checker says yes, has a name in the writing on self-improving programs. The AI researcher Jürgen Schmidhuber, at the IDSIA lab in Lugano, described a Gödel machine (2003) that rewrites its own code only after its built-in proof searcher has proved the rewrite useful, and Benja Fallenstein and Nate Soares (2015), researchers at the Machine Intelligence Research Institute in Berkeley, survey agents that reason about their successors through formal proofs a fixed checker accepts. Same architecture, aimed at a different question: they want to know when a machine may trust a smarter version of itself, and this machine only wants to know whether a flag was earned. Keeping the checker's proof outside the machine is my design choice. The nearest theorem, in Section 13, is narrower: no procedure whose flip the substrate can represent decides, for every program, a property that depends only on behaviour and holds of one program and fails of another. It doesn't say a machine can't check a proof about a checker, or verify a restricted part of itself.

Soundness rules out a lying checker. It doesn't rule out a pointless question: *at least 0* has a perfectly sound checker and an empty no pile, as the worked run showed.

Program checking has a theory of its own, and its founding paper asks for a checker that could have said no. The computer scientists Manuel Blum and Sampath Kannan (1989; journal version 1995) defined a checker for a computational problem: given a program and an input, it may run the program as it likes, and it must say "correct" with high probability when the program is right everywhere and "buggy" with high probability when its answer on that input is wrong. A checker that always says "correct" isn't one. Here the checker is fixed, deterministic and proved sound once, and the part of their demand that survives is the fourth thing certified asks for: some claim whose check can fail.

15.7 Nothing gets forged

The proof of Theorem 15.10 leaned on an invariant I promised to show you, and this subsection shows it. The fear it answers: could some clever sequence of instructions get a fact into the table without a CHECK writing it? Could a DEC with a funny jump target do it, or a COMMIT after a trap, or something I didn't think of? I answer it one instruction at a time, which is the only way I trust myself to answer it.

Definition 15.6 (Earned fact; no forgery)**.** After a trace t from s_0, a fact $f = (p, c, v)$ is *earned* when t is *pre*, then CHECK p c, then *mid*, where the CHECK passes at the core reached by *pre*, f is exactly the claim p of c at that core, and, if f is live at the end (v equals c's version after t), *mid* leaves c untouched. The trace has *no forgery* when every fact in the table after it is earned.

Theorem 15.14 (No forging)**.** For every core k, instruction i and fact f in the table after executing i at k: either f was already in k's table, or i is CHECK of f's property and counter, it passes at k, and f is the claim it writes. Consequently, if a trace has no forgery, so does that trace followed by any one instruction; and from every clean start, every trace has no forgery. □

Proof. Inspect who can write to the table. INC and DEC change counters, versions and the line; they leave the table alone. HALT leaves it alone. COMMIT reads it and may write the channel, but cannot write the table. CERTIFY reads the channel and cannot write the table either. CHECK is the only remaining instruction. On failure it adds nothing; on success it adds exactly the property, counter and present edition it just checked. A trapped instruction changes no core field. That exhausts the ways a fact could appear after one instruction.

Now suppose the table's old entries already have their promised histories. A new entry gets its history from the passing CHECK just performed. For an old entry, that earlier CHECK still happened. If the entry is live after the new instruction, its saved edition equals the counter's edition now. The counter's edition never goes down. It therefore could not have risen away from the saved edition and then returned. In particular, it was already live before this instruction and the instruction did not change the edition or the value. Its untouched stretch extends by this one step. If the entry is stale, only its earlier passing CHECK is required, and that history remains.

At a clean start there are no entries to forge. The argument preserves that promise after the first instruction, after the next, and after each further instruction in any finite trace. ■

15.8 The window can't see the record

Section 8 was about windows: a view of a state that shows some of it, and the theorem that a window which shows the same before-and-after pair for a raise and a non-raise cannot price both exactly (Observation 8.2). The two runs below are the smallest pair I could build where the window is Minsky's view of the machine (line number and the two counters) and the record is everything else.

Run two traces from start$(0,0)$. The *earned* run is CHECK *is zero* A; COMMIT *is zero* A; CERTIFY. The *idle* run is DEC A 0, three times. In the idle run, A is 0 every time, so each DEC just goes to the next line and writes nothing. Both runs end on line 4, with $A = 0$ and $B = 0$, both versions 0. Through the window they're identical. One of them has paid 3, holds a fact, could commit again, and has its flag up. The other has paid nothing, holds nothing, can't commit, and its flag is down. They're like two photographs that show the same picture and are of different people.

□ **Theorem 15.15** (Same window, different record)**.** The earned run and the idle run from start$(0,0)$ end at cores with the same window and the same versions of A and B, and with different ledgers, different flags, different tables, and different answers to "would COMMIT *is zero* A pass."

Proof. In the earned run, CHECK adds $(\text{Zero}, A, 0)$, COMMIT puts that fact in the channel, and CERTIFY raises the flag. The ledger is $1 + 1 + 1 = 3$. No counter is written. In the idle run each DEC sees zero, so it advances the line without writing a counter or raising a version. Its three costs are zero and it adds no fact or commitment. So both final windows are $(4, 0, 0)$ with versions $(0, 0)$. The earned table contains the current zero claim and the idle table is empty, giving the stated difference in whether another COMMIT would pass, as well as in the ledger and flag. ■

□ **Theorem 15.16** (No window oracle)**.** There is no function g from windows to whole numbers with $g(\text{window}) = \mu$ at the end of every trace from start$(0,0)$. Likewise there is no function from windows to yes or no that gives the flag at the end of every such trace, and none that gives whether COMMIT *is zero* A would pass.

Proof. Such a g would have to give both 3 and 0 on the one window the two runs of Theorem 15.15 share. The same for the flag and the commit right. ■

The record lives off the window, and that's Section 8's shape, made of pebbles. Anybody who only watches the counters and the line number, which is everything

Minsky's machine *is*, can't tell you what was paid, what was committed, or whether the flag is up, and the reason isn't that they looked carelessly; it's that none of it is there to see.

Two things the same in everything you look at and still two things: philosophy has an old fight about exactly that, and it's worth a detour, because it tells you which side of the fight these theorems stand on. Leibniz said that for real things it can't happen. There are no two individuals indiscernible from each other, he wrote in 1716 to Samuel Clarke, the English philosopher and theologian who was taking Newton's side against him in an exchange of letters, and he told a story to go with it. A clever gentleman of his acquaintance, talking with him in front of the Electress Sophia of Hanover, Leibniz's friend and patron, in the garden at Herrenhausen she had helped lay out, thought he could find two leaves perfectly alike. She dared him to. He "ran all over the Garden a long time" looking, and it was "to no purpose." You can run his search tonight with a handful of anything, leaves or coins or grains of rice: look hard enough and every pair differs somewhere. Then in 1952 Max Black, a Russian-born philosopher teaching at Cornell, pushed back with a universe you can't build at the table. Nothing in it but two iron spheres, the same size, the same colour, the same everything, some distance apart. Name a difference between them. There isn't one, and there are still two of them. People have argued about Black's spheres ever since, and I'm not going to settle it.

I don't have to, because Leibniz's claim is about *every* property a thing has. The theorems above are about three: the line and the two counters. The two runs aren't indiscernible in his sense at all: one has a fact in its table and its flag up, and the other doesn't. They're indiscernible to a window, which is a much cheaper thing, and it's the thing that matters for a machine, because an observer of a machine only ever gets a window. Leibniz's gentleman would always find his difference if he could look at everything. It doesn't help him if all he's allowed to look at is three things about each leaf.

There's a horse in this, too, if you squint. In Germany in the early 1900s a mathematics teacher named Wilhelm von Osten showed off a horse called Clever Hans, who answered arithmetic questions by tapping a hoof, and got them right, in front of crowds. Hans gave the same answers out as a horse that could count, and that's the idle run's matching window. Then the psychologist Oskar Pfungst stopped looking at the answers and started changing the conditions, and published what he found in 1907. Put blinders on Hans, or have him asked by someone who didn't know the answer, and the arithmetic fell apart. Hans was reading the questioner: tiny, unintended movements of the head and body as the hoof reached the right number. That's the probe. Pfungst didn't argue with the answers. He asked a question the answers couldn't fake, one that needed the real thing to be there, the way a COMMIT needs a checked fact to be in the table: run COMMIT *is zero A* after

each of the two runs, and one passes and the other traps (Theorem 15.15). And the part of the story I like best: nobody was lying. Von Osten was never persuaded by Pfungst and kept showing his horse. A matching window doesn't need a cheat behind it, which is why you want a probe. An accusation is the wrong tool for it.

The rebar example, on the actual counters
Put a one in counter A, leave B at zero, and give the machine these four instructions:

$$\text{CHECK } (A \geq 1); \quad \text{COMMIT } (A \geq 1); \quad \text{CERTIFY}; \quad \text{DEC } A \text{ 5}.$$

The check passes. The claim gets committed and certified. Then DEC takes the one away. Look at the counters now: zero and zero. The value that passed the check has gone.

Start again with both counters zero and give the same instructions. This time the first check fails and the machine traps. The zero stays where it was. Look at the counters at the end: zero and zero again. One run has its certificate up; the other has it down.

□ **Theorem 15.17** (Same final counters and bill, different certification)**.** For those two clean starts and that word, both final counter pairs are $(0, 0)$ and both ledgers are 3. The first certificate is true and the second false. No function of just the final counter pair can recover which of those starts passed the check.

Proof. For the start $A = 1, B = 0$, CHECK sees $1 \geq 1$, writes the edition-zero fact, advances the line from 1 to 2 and makes the ledger 1. COMMIT finds that live fact, puts it in the channel, advances to line 3 and makes the ledger 2. CERTIFY sees the occupied channel, raises the flag, advances to line 4 and makes the ledger 3. DEC sees a positive A, changes it to zero, advances to line 5 and raises A's version from 0 to 1. Its price is zero. The final counters are $(0, 0)$ and the flag is up.

For the start $A = 0, B = 0$, the first CHECK fails. It sets the trap, leaves the line at 1, leaves the table empty and charges one. The external word still supplies COMMIT and CERTIFY: on a trapped state they change no core field and cannot raise the flag, but each adds its unit to the ledger. DEC then changes nothing and charges zero. The final counters are again $(0, 0)$, both versions stay zero and the flag is down. Both ledgers are 3. A function of the common counter pair would have to return both flag values, which is impossible. ■

The counters show the same surface. The record keeps the difference. Open the rest of the box and the trap, version and line number tell different parts of the story too: one run reached DEC, while the other stopped at the check.

Ask the successful run to CHECK $A \geq 1$ once more. It fails; A is zero now. The old certificate stays up because the earlier certification still happened. That receipt keeps its past even while the workspace goes on to something else.

One clipboard, 65,535 different publication histories
Open the whole native box this time. Read the counters, versions, table, channel, line number, trap, ledger and lamp. I can make all of them agree while the files of published claims differ in 65,535 ways.

Start clean at $A = 16, B = 0$. CHECK the sixteen claims $A \geq 0, \ldots, A \geq 15$. Those counter values are initial data, before any instruction: versions zero, line 1, ledger 0, empty table and channel, and trap and flag down. Every one passes on edition zero. The clipboard now has sixteen entries. The line number is 17, the bill is 16, and the certificate is still down. I can choose which of those checked claims to publish.

Visit the claims in order. At each one, choose between these blocks:

publish it:	COMMIT claim; CERTIFY,
leave it unpublished:	COMMIT claim; COMMIT claim.

Count the steps in either block: two. Count the charge: two. Look at the channel when it finishes: the same claim. The counters and table stayed put. Once one publication has raised the lamp, that lamp stays up whichever later blocks you choose. CERTIFY reads the channel without consuming it; this is why the two blocks leave the same channel as well as the same line and bill. The predicates overlap logically; it is the choice to publish each checked claim that is independent.

Theorem 15.18 (The exact native publication-history fibre)**.** In the extension retaining the set of published claim/edition facts, exactly 65,535 different nonempty sets can lie above one reachable native endpoint. This is the maximum over clean native runs with cert true. Sixteen additional fixed binary bits suffice to encode the set relative to the native table, and fifteen do not suffice in the worst case. □

Proof. First prepare the table. After the first j checks, for $0 \leq j \leq 16$, its entries are exactly $A \geq j-1, \ldots, A \geq 0$, newest first and all on edition zero (the table is empty at $j = 0$). The line is $1 + j$, the ledger is j, the counters and versions are unchanged, the trap is down and the flag is down. This holds at $j = 0$. If $j < 16$, there is still room and $16 \geq j$, so the next CHECK prepends precisely the next fact, advances one line and charges one. That proves the preparation claim through $j = 16$.

Now process the sixteen facts in order. Each COMMIT finds its fact in the unchanged table at the current edition. Either block ends with that fact in the

channel, advances two lines and charges two. The publishing block additionally raises the flag and inserts that fact into the archive; the other block changes neither. Inducting on the number j of blocks gives line $17 + 2j$, ledger $16 + 2j$, and exactly the chosen subset of the first j facts in the archive. The flag is up if and only if at least one publishing block has occurred. None of the blocks changes counters, versions, table order or trap. At $j = 16$ every nonempty choice therefore has the same native endpoint: $A = 16$, $B = 0$, versions zero, line 49, ledger 48, trap down, flag up, and channel $(A \geq 15, A, 0)$. Its archive is precisely its chosen subset. There are two independent choices at each of sixteen positions, hence 2^{16} subsets; excluding the empty one leaves 65,535 different archives above that endpoint.

For the upper bound, keep two invariants together: every published fact is in the table, and every occupied channel names a fact in the table. Both hold at a clean start. CHECK only prepends to the table. A successful COMMIT selects an entry already there. CERTIFY publishes only its occupied channel. Counter writes can make a fact stale; they preserve the channel and never remove table entries. Failed and trapped instructions do not introduce a new channel fact or publication. Thus both invariants survive every instruction, including later writes. Every final archive is a subset of the final table.

If that table has d distinct entries, it has at most $2^d - 1$ nonempty subsets. The table has at most sixteen entries, so $d \leq 16$ and $2^d - 1 \leq 65{,}535$. An explicit code puts beside each table entry one bit saying whether it belongs to the archive, and pads with zeros to sixteen positions. Duplicate entries receive matching bits. Read back the entries with bit one and you recover the set. Equal codes therefore give equal archives, and an archive above a flag-up state uses a nonzero code. Finally, fifteen fixed bits have only $2^{15} = 32{,}768$ values, fewer than the exhibited 65,535 archives; sixteen bits have 65,536 values and the code just given suffices. ■

Put sixteen checkboxes beside the clipboard's sixteen lines. A tick says that line's claim was published at least once. The table already names the claims and editions; the extra marks say which ones made it into the publication file. The lamp gives the same yes for every nonempty selection. Open the checkboxes and all 65,535 selections come apart. Each tick answers whether that claim was published at least once. Order and repeated publications leave these answers unchanged.

In Chapter 14 the camera showed the working value. Here I let it see the entire native machine and still found histories it could not recover. The map here forgets the added publication file; its fibre contains those different files above one native endpoint.

The sixteen-line clipboard eventually fills. Even sixteen repeated checks of one fact fill it, and the next CHECK traps. The unlimited-edition machine has a larger obligation: keep accepting independent publication choices across every finite

number of editions and keep them recoverable at the same working picture. Its fibre has to keep growing. That is the count from Chapter 14, with its own camera and its own task.

15.9 The exact price is three

Section 2 said a raised flag costs at least one. On this machine that's true and too modest. A raised flag needs a CERTIFY, a CERTIFY needs a COMMIT, a COMMIT needs a CHECK, and each of those is its own instruction costing one. So a raised flag costs at least three. One of what: three units of μ, one per paid act, each one the least price a record-layer merge allows. And three is the number of links in the chain, not a guess that happens to hold.

Observation 15.19 (The minimum price of a certificate)**.** *If* s_0 *is clean and* $\mathrm{cert}(\mathrm{run}(t, s_0))$ *is up, then* $\mathrm{cost}(t) \geq 3$ *and* $\mu(\mathrm{run}(t, s_0)) \geq \mu(s_0) + 3$*. For programs: if the flag is up after* n *program steps from* $\mathrm{start}(a, b)$*, then* $\mu \geq 3$ *there.*

Proof. Theorem 15.11 finds a CHECK, a COMMIT and a CERTIFY at three different places in t, each costing 1, and every other instruction costs at least 0. The ledger statement is Theorem 15.6. A program run is the run of the instructions it actually executes, so the program form follows. ∎

The three is three independent units, one for each part of a clean start. A flag that is down buys the first: a run that raises it costs at least one. An empty channel buys the second: with the flag down and the channel empty, a run that raises the flag costs at least two. An empty table buys the third, and that is the theorem above. Leave out any one part and its floor fails, and a cheaper certificate is there: with the flag already up it costs 0, with a commitment already in the channel 1, and with a live fact already in the table 2. Every run from a state that ends with the flag up costs at least three exactly when the flag is down and one of two things holds: the trap is up, or the channel is empty and every fact in the table is about an older version of its counter. And three is paid exactly from every clean start whose trap is down.

Theorem 15.20 (Three is tight)**.** For all whole numbers a and b, run the program CHECK *is zero* A; COMMIT *is zero* A; CERTIFY for 4 program steps from $\mathrm{start}(a, b)$. If $a = 0$, it ends with the flag up and $\mu = 3$ exactly. If $a \neq 0$, it ends with the flag down and the trap up, and for every n, n program steps from $\mathrm{start}(a, b)$ leave the flag down. □

Proof. If $a = 0$, the checker for Zero says yes at 0, so the first instruction adds $(\text{Zero}, A, 0)$ to the empty table and charges 1. No counter changes, so the second instruction finds that same fact, puts it in the channel and charges 1. The third finds the channel occupied, raises the flag and charges 1. The line is now 4, beyond the three-instruction program, and the fourth program step adds no charge. The final ledger is exactly 3. If $a \neq 0$, the checker says no, so the first instruction sets the trap and leaves the flag down. A program run takes no step once the trap is up, so every later program step leaves the state, flag included, exactly where it is. ∎

Theorem 15.20 is the worked run from early in this chapter, now holding from every start.

The version whose check can't fail, CHECK *at least 0* A; COMMIT *at least 0* A; CERTIFY, certifies from every start for exactly 3. So the floor of three is reachable from every loaded start, and that's the machine declining to police trivial checks. On that program the check distinguishes no inputs.

The floor is 3, and a run whose check could have failed pays exactly 3, so 3 is the price. The bound has no slack in it.

Don't mistake that 3 for a measure of how hard a claim is to certify. It's the same 3 for every claim, easy or hard, because only the three acts that make a claim are priced and the counters work out whatever the check needs at no charge on the ledger. A cost of certifying that grows with the claim would have to price the checking itself, and that's a different question with its own literature. Four theoretical computer scientists, Guy Blanc, Caleb Koch, Jane Lange and Li-Yang Tan (2022), studied one version of it under the name the query complexity of certification: how many questions an algorithm has to ask a black-box function to hand back a small certificate for its answer. Their count moves with the function. Mine doesn't move at all.

And that's every piece of the machine, standing on six instruction kinds: a base that computes everything two counters can, a ledger that's exact, a toll the universal floor applies to, a flag that's a latch of the base, a window that can't see the record, and, from a clean start, a commitment that can't happen unless a check of the same claim, on the same edition, came first and was right.

15.10 The machine versus the program it runs

The three hosts of Appendix C (the simpler host, U and U_P) all come down to one distinction, and it's worth saying on its own, because the two halves are easy to run together: *the machine doing the stepping* (what the machine is) and *the program it runs* (what runs on it). The toll of Section 2 is a property of the machine doing the stepping. A rule a program keeps holds as long as the program keeps it.

I keep a picture for it: play Monopoly on the counter of a real bank. The game has a bank of its own, with its own rules, and one of them is printed in the rules sheet: the Bank never goes broke, and if it runs out of money the Banker may issue more by writing on any ordinary paper. Inside the game that's fine; the players agreed to it. The real bank's teller, standing right there, has no opinion about any of it. Push a thousand in Monopoly money across her counter and her ledger doesn't move, because her rules are about her own money. The game's money is nothing to them. The real bank is the machine doing the stepping. The game is a program it happens to be hosting. A rule that lives in the game holds as long as the players keep to it. A rule that lives in the bank holds whatever the players do.

(And since the game is on the table: its box told an origin story for decades, that an out-of-work salesman named Charles Darrow invented it during the Depression. The board is older. Elizabeth Magie, an American writer and game designer with an economic argument about land to make, patented The Landlord's Game in 1904, with a square track, railroads and a square that sends you to jail, and in 1902, in *The Single Tax Review*, described it as "a practical demonstration of the present system of land-grabbing with all its usual outcomes and consequences." Darrow sold his version to Parker Brothers in 1935, and the company bought Magie's patent for a few hundred dollars, five hundred by the accounts I found. A story told by the company selling the product is a lead to check, every time. It changes nothing about banks. It's a good reminder of who writes the story on the box.)

A fixed machine can enforce an invariant in its own transition table, and that includes a Turing machine. Give its tape a ledger track and a certification cell, and write a table that never sets the cell without bumping the ledger: that machine keeps the invariant in its step relation. The ledger track is part of its state, though, and a ledger kept in the state isn't a price on the move. The toll of Definition 2.2 puts the price on the move, the same from every state, so what it charges depends on what the machine's moves are. Give it one move, the next step its table picks: if the table sets the cell from any configuration at all, that move costs at least one, so every step does, whatever the track says, and the machine is a clock. Give it a move for each row of its table: every row that can set the cell costs at least one wherever it's taken, and a row that never sets it can cost nothing. Either way it's the price on its moves that meets the toll, and the track only simulates it. Priced that way, it's a Thiele machine itself, so it can't be a counterexample to the model. What it doesn't do, built the ordinary way, is enforce the toll on the programs it runs. A universal machine whose table was written only to interpret carries the simulated certification bit as plain data on its tape. If the host is programmable, so that other code shares that tape with the guest, any of that code can flip the bit without paying, because the host's step has no opinion about it. So an ordinary programmable computer, as built, doesn't enforce the toll for its programs; that comes from how it's built, and

it could be built to enforce it. A host whose own step mirrors the guest's bit and charges for its crossing enforces it, and the universal programs U and U_P are fixed tables that go one better: they earn their own flag, by their own check, commit and certificate, exactly when the guest's would rise, and pay on their own ledger what the guest pays, U exactly and U_P to within a surcharge of at most two when the guest's reading starts at no (Theorems C.5, C.4, C.7 and C.8). That's a statement about which reading a step rule prices, not a claim that Turing machines can't price one.

Computing ran into this exact gap in hardware, and how it got out is worth the detour. In 1974 two computer scientists, Gerald Popek and Robert Goldberg, asked when a machine can run guests, a whole operating system inside another, efficiently and safely. Their answer is one sentence you could check against any instruction set. Call an instruction *sensitive* if it reads or changes the machine's own settings, the things the host has to keep for itself, and *privileged* if running it outside the host's own mode makes the machine stop and hand control to the host, which is called a trap. For the kind of machine they modelled: if every sensitive instruction is privileged, a host can be built. The guest runs at full speed, and whenever it touches something that matters, the machine itself stops it and lets the host decide, and a host like that is one whose step has an opinion.

Then the obvious next question: did the machines people actually bought pass the test? The most common one didn't. In 2000 John Scott Robin, an Air Force officer, and Cynthia Irvine, a computer-security researcher at the Naval Postgraduate School, went through every instruction of Intel's Pentium and found seventeen that are sensitive and not privileged; POPF, which loads the processor's flags, is the one usually pointed at. A guest could run them and the host would never hear about it. The fixes came two ways: VMware rewrote the guest's code on the fly so the troublesome instructions never ran as written, and in the middle of the 2000s Intel and AMD added hardware support so they'd trap. Does that change anything here? It doesn't change the theorems. Popek and Goldberg were after speed and isolation. A record and a price weren't their question, so their result isn't mine and mine isn't theirs. It changes the picture, though. The universal programs sit where they'd want a host to sit: the host's own steps collect each guest's toll and earn each guest's record again. And an industry learned the same lesson the expensive way: a rule binds the guest only if it lives in the step underneath.

The smallest version of "a ledger beside an ordinary machine" is a Turing machine configuration with a counter next to it, where each step adds one, plus one more if it writes a nonblank symbol. The counter doesn't disturb the machine.

Theorem 15.21 (A counter beside a Turing machine changes nothing)**.** Let a Turing machine with transition table δ step a configuration c to $\mathrm{tm}_n(\delta, c)$ after at most n steps (stopping when δ has no entry). Pair each configuration with a counter, and step the pair by stepping the configuration and adding the step's charge. Then for every n, δ and pair (c, m), the configuration part after at most n steps of the pair is $\mathrm{tm}_n(\delta, c)$. In particular this holds from $(c, 0)$. □

Proof. Put the ordinary run and the run with the counter side by side. Both start with configuration c. Whenever their configurations agree, they look up the same entry in the same transition table. If the entry is absent, both stop. If it is present, both take the same next configuration; only the paired run also changes its counter. Their configurations therefore agree again, ready for the next comparison. This covers every step up to the chosen limit n, as well as a stop before that limit. The counter's value never enters a lookup. ■

The pair is a Turing configuration with a number riding beside it, and nothing more. Its counter is kept in the state, and a ledger kept in the state isn't a price on the move: Definition 2.2 puts the cost on the move's name, and this counter's charge depends on what the step writes. This theorem therefore does not supply exact accounting for the basic instruction-price signature, or a reading with an earned-history meaning. It does not rule out equipping the same state space with other instruction prices and a Boolean reading satisfying the basic toll. The projection claim is simply that this additional count leaves the tape computation alone.

One limit on all of this needs saying, so it isn't read as more. Certification is one event. If you wanted a machine to carry some other kind of reading, you'd need an event for it, evidence for it and a price law for it, and nothing here supplies any of those.

15.11 The small machine's merges

And the whole small machine isn't the eight-state piece it runs. It merges some states without adding anything to the ledger.

Theorem 15.22 (The small machine's merges)**.** (1) A move of the small machine that takes the flag from down to up is CERTIFY; CERTIFY merges two different full states into one (a state with the flag down and the same state with the flag up land on the same state), and it costs at least one. (2) DEC A 2 also merges two different □

full states, and it costs nothing. So it is false of the small machine that every move that merges is priced.

Proof. (1) Only CERTIFY changes the flag, and it costs one. The two states with the flag down and up, and a committed claim, are the same state after CERTIFY. (2) Take a core at line 1 with A at 1 at version 0, and the same core with A at 0 at version 1. DEC A 2 takes the first to $A = 0$ at version 1, line 2, a write; it takes the second to line 2 with A still 0 at version 1, because DEC on zero is no write. Same state, zero cost. ∎

The theorem puts names to the last item on the list of what this machine doesn't do, near the end of this chapter. Part I's counting of states wants a finite machine that puts a price on every merge. This machine has counters with no top, and it prices the three record acts and leaves its counting merges at a price of zero, so Part I's counting of states doesn't reach it whole, and that's why the eight-state piece is a piece. The record's part of the state closes the rest: asked only of merges between states with the same line and counters, the merge premise holds for this machine's own prices and forces the toll on it with no finiteness premise (Theorem 3.19).

15.12 Reading the instruction rules

The instruction rules settle some concrete questions about running this machine. Follow them through a check, a zero decrement and a trapped trace.

- **The check price is flat.** CHECK costs 1 whether it's asking "is this 0" or "is this at least a billion." The property language is fixed and each property is one evaluation, so "pays for its work" here means that flat price, the same however hard the question was. A machine that charged by the difficulty of the check would be a different machine, and I haven't built it.
- **DEC on zero isn't a write.** It doesn't change the counter, so it doesn't change the version. That's the right call for versions (nothing changed), but it means DEC on zero is a way to move the line number that adds nothing to the ledger and leaves no trace in the record. The idle run in the subsection on the window, earlier in this chapter, uses exactly that.
- **After a trap, a trace still pays.** On programs, a trap stops the run. On arbitrary traces, instructions after a trap are each charged their cost and none of them changes the core. That isn't a separate choice. A price belongs to the instruction, the same from every state (Definition 2.2), so a CERTIFY on a trapped machine costs what every CERTIFY costs, and the ledger adds it; Theorem 15.6 has no exceptions for that reason. The choice is one level up, in putting the price on the

instruction and not on the instruction at a state, and what it brings with it is that a trapped machine is still charged for instructions that change nothing.

- **"Checked" means what the property language can say.** On this machine that's *is zero*, *is even* and *is at least* n, and a live fact means one of those holds of one counter's current value. It doesn't mean the program is correct, or that anything follows from the property, or that the property was worth checking. So can a proved machine of this kind check "sorted," the claim the sorter-and-copier table of Section 14 runs on? It can, through an encoding. A copy of this machine, instruction for instruction, with its property language left open, has provenance, checker soundness, no forging and the floor of three proved once for every property language that comes with an exact equality test and a checker proved equal to its meaning. Give it "the list this counter encodes is sorted" as a property and it certifies a sorted list for three and refuses an unsorted one forever. The instruction rules are those already given, with the chosen property language in place of the three original properties. For sorted lists, decode 0 as the empty list and $2^h(2t+1)$ as the list with head h and tail decoded from t. Every positive code has t smaller than itself, so decoding terminates. The checker returns yes on an empty list or singleton, and on $a, b, \ldots$ exactly when $a \leq b$ and it returns yes on $b, \ldots$. Induction on the list proves that this is exactly adjacent nondecreasing order. The provenance proof uses only equality of properties, and soundness only that the checker's yes means the property holds, so the same instruction-by-instruction argument applies. What makes that a check of the list is the proof that the checker means sortedness. Without that proof, "sorted" would be a number some unconstrained code wrote into a counter, and the record would be honest about the counter and silent about the list.
- **The certificate is historical.** Between the COMMIT and the CERTIFY, the counter can change, and the flag still goes up. The flag says a claim was checked and committed about an edition. It says nothing about whether the claim is still true.
- **One latch, and a channel that moves on.** The flag records that an earned chain happened, once. After that it stays up, and later CERTIFYs change nothing. The channel is the part that names a fact: with the flag up from a clean start, it names a fact that some earlier passing COMMIT committed. But a later commitment replaces it, so at the end it names the last earned commitment, which needn't be the one the flag rose on. Two runs from start$(0,0)$ show it, worked from the instruction table. First run: CHECK *is zero* A; CHECK *is zero* B; COMMIT *is zero* A; CERTIFY; COMMIT *is zero* B. Second run: the same five instructions with the last two swapped, so COMMIT *is zero* B comes before CERTIFY. Both end in the same state: both counters 0 at version 0, both facts in the table, the channel holding (*zero*, B, 0), the trap down, a ledger of five and the flag up. In

the first run the flag rose at step four, on the commitment about A. In the second it rose at step five, on the commitment about B. So the flag is an ever-certified latch: it says an earned chain happened at some step, and the end state doesn't say which claim or which step. A receipt naming the claim the flag first rose on would need that claim kept where nothing can overwrite it; a receipt with a time on it would also need the step number kept; and several certifications that each last would need a record richer than one flag. The publication file constructed earlier in this chapter keeps the set of claims certified at least once. That extension shows exactly what the native state left out. It still does not record which certification came first or at what step; those questions would need their own retained answers.

- **Sixteen checks per run, ever.** Facts never leave and the table holds sixteen, so a run gets at most sixteen passing CHECKs in its whole life, and the seventeenth traps. A certificate needs only one, so this costs nothing for the theorems, but it's a hard wall for any program that wants to keep checking forever, and it's this machine's small version of "you can't keep all of it" from Section 14. A CHECK that repeats a claim already in the table, same property, same counter, same edition, passes again and takes another entry. So the wall is sixteen passing CHECKs, and sixteen different claims is a different count. Removing duplicates, evicting old facts or collecting dead ones would each be a different design, and each would need its own invariants proved.
- **The clean start matters.** Provenance, soundness, no forging and the price of three are all stated from a clean start. Hand the machine a table pre-loaded with facts nobody checked, and of course none of that follows. The theorems are about what the machine produces. What you smuggle in at the door is outside them. The door is narrower than clean for some of them: a commitment's provenance and checker soundness use only the table, and hold from any start whose table holds nothing but facts about older editions; the empty channel and the lowered flag come in for the flag's provenance and the price of three. Spell the door out once: the theorems cover any list of instructions anybody can issue, from a clean start, on a step that runs as written, with nothing else writing to the machine. They say nothing about whoever sets or restores the start, about a reset, about a device that misbehaves, or about whoever carries the flag's value to you.
- **The universal programs have stated guests.** U (Appendix C.1) runs every one of this machine's programs from every clean start start(x, y), with halting, answers, flag and ledger matched (the flag over the whole run, the ledger at matching points). U_P (Appendix C.2) runs every computably presented Thiele machine, with its reading decided by a program, its record carried as a latch, and its ledger matched to within a surcharge of at most two when its reading starts

at no, and of three when it starts at yes. When the guest certifies with a ledger below three, no program on that host, run from a clean start, matches its ledger exactly. A machine outside that class, with a reading no program decides, isn't covered by either. The host with a guest step built in, charging one per guest step whatever the step was, would have a ledger at least the guest's and usually more; U's is exact, and U_P's is exact up to the surcharge.

- **Encoding and running time.** The universality result takes its inputs already packed into the counters. Getting there from an arbitrary Turing machine goes through Minsky's exponential packing, and that packing is what it takes to reach two counters this way. Anyone who wanted this to run fast would have to start the analysis over.
- **The step charges for the act, not for what the claim is worth.** CHECK *at least 0* passes on every value, so CHECK, COMMIT, CERTIFY on it raises the flag for three from any fresh loaded start (the note after Theorem 15.20), and that's by design. The step checks order, aboutness and an honest checker. What a claim is worth is the entitlement count of Section 9, a number computed over the rivals a run narrows. A step sees one state, so it cannot compute that count. A check that cannot fail narrows nothing and costs three, and the book keeps both numbers, apart.
- **The toll on this machine comes from the record's part of the state, with no finiteness premise on the whole machine.** Section 3's counting of states forces a toll on a finite machine that prices every merge. This machine is neither. Its counters have no ceiling, as a machine that runs every program has to have, and its editions can't have one either (the version rule says why), so it's infinite. And, by design, its unpriced instructions merge states: take a core at line ℓ with c at 1 and version w, and a core at line $j-1$ with c at 0 and version $w+1$, everything else alike, and DEC c j sends both to line j with c at 0 and version $w+1$, at a price of zero. A price of zero doesn't mean the hardware pays nothing. Any physical heat belongs to the implementation's account. A positive floor requires the thermal premises and an input distribution with entropy loss; merely taking a step establishes neither. This ledger does not measure that heat. The ledger leaves those merges out on purpose. It counts the keeping in the record, not what computing spends. Computing doesn't have to merge; a reversible base computes by paying in fresh states. On this machine full merge pricing would charge DEC, an ordinary step, and clause (c) asks for computing to cost nothing. Charging every move would instead give a clock's schedule. Its total counts moves, rather than isolating record acts. The machine can still retain a record; this total alone does not say which acts established it. So merge pricing on this machine applies to the record's part of the state only: a move that sends two different states with

the same line and counters to one state costs at least one. This machine's own prices meet it. The endless counters would allow a raise that merges nothing, and the flag doesn't take it: CERTIFY lands a state with the flag down and the same state with the flag up on one state, a merge of exactly that kind, in plain view. So every price list that meets the premise charges the flag-raising step at least one, with no finiteness premise, and through the trap it charges every CHECK and COMMIT at least one too, so a certificate from a clean start costs at least three on every such list (Theorem 3.19). That's a price I assume for losing distinctions in the record, and assuming it is all it is. Turning it into heat takes the physical conditions of Section 3; a unit on a ledger isn't heat on its own. The exact price of one, the least the premise allows on each of the three acts, is still a design choice.

15.13 What this chapter adds up to

So this chapter has one machine, and it isn't the model. It's a Thiele machine in the opening's sense, a Thiele machine with the toll proved (Theorem 15.7), and its toll charges the blank cell's 0: raising its flag is a merge in the record's part of its state (Theorem 3.19). Its flat part computes whatever a Turing machine can, under Minsky's 2^n encoding, done outside the machine, its computing adds nothing to the ledger and can't touch the record, and, from a clean start, its record is earned.

If you ever find yourself wondering whether a property of this machine is the model's or just the machine's, hold it up against Section 2's definition of a Thiele machine, the latch of Section 11, and the four things certified asks for (Section 14.15). What follows from those holds of every machine that meets them; the rest (a table of sixteen facts, three properties) is this machine's choice. The flat price of one is the third of the four, and at least one of it is what the record's merge premise asks of each act on this machine. There are six instruction kinds, and a seventh, PAY, for a host that has to charge somebody else's prices (Appendix C.2). If the model needs more than the five parts and the toll, this is where it will show.

CHAPTER 16

How far it reaches

The small machine earns its flag, and pays for it, on its own programs. Everything in this chapter asks how far that carries. Every statement in it is proved, and the runs are reported as runs. I give each statement here and leave the constructions to the appendices, because they're about the reach of the case, not the case.

Other machines as guests. One fixed program, U, written in the small machine's own six kinds of instruction on a host with more counters, runs every program of the small machine as a guest, from every clean start. It fetches, decodes and carries out each guest instruction itself, the way Turing's universal machine does; nothing in the host means "run one guest step." It halts exactly when its guest halts, with the guest's answer in its counters; its flag goes up during its run exactly when the guest's goes up during the guest's, by its own check, its own commitment and its own certificate; and its ledger equals the guest's at every point where the two runs match. The host is a certified Thiele machine, and whether U halts is undecidable. Add one move, PAY, that costs one and does nothing else, and the same kind of program, U_P, runs every Thiele machine whose next move, step, prices and reading are computed by programs that always answer: it halts when the guest's run stops, its flag goes up exactly when the guest's reading is yes at some state of its run, on a chain whose check is the guest's own reading, and its ledger is the guest's prices plus a surcharge of at most two at the raise when the guest's reading starts at no, and exactly three when it starts at yes, a surcharge that no program on that host, run from a clean start, can always avoid. Appendix C has both.

Other flat cases. Lay the toll over a register machine, a Turing machine or the lambda calculus and none of them computes one function more; what each gains is a record and a cost. Each is the flat case with its price column put back (Section 11).

Machines from parts. Wired side by side, end to end, or one inside the other, a Thiele machine stays a Thiele machine, and the tolls add the way money adds. What breaks, breaks for reasons you could name in advance: a single flag for two claims costs two claims' worth, a clock is still a clock, a shared value without a version can't be trusted, and a hand-off that costs nothing can't be allowed to carry a certificate (Appendix D).

CHAPTER 17

Five small models

So far there's been a toll, a counter, two kinds of wall and a latch, every one of them shown on paper, on cups and cards and dice I made up, and then a small machine to run them on. That is the model on paper and on one small example. It's time to go and look at what people actually do. I picked five systems that keep records other people rely on, with real money or real security behind them, and asked each the pricing question from the paper models: *is there a price on the moment the record is written?* The question isn't about the work around the write; it's about the write, the moment the 0 goes on the page and stays. I'll tell you the answer first, because it's a single sentence and it surprised me: of the five specifications, read through the transcriptions I made of them, only one takes a fee at the moment of the write, and that one has a leak at exactly the place Section 7 said a counter leaks, where it goes down.

"Price" needs one meaning for all five, or the answer is whatever I want it to be, so I'll fix it here, and every system below gets classified by it. A specification *prices* an event when it attaches one of three costs to it. A *fee*: something taken at the event itself, whatever happens afterward. A *bond*: value put at risk by the event and lost only if some later event happens, a contradiction, say. A *budget*: a finite allowance the event draws down, so that only so many such events can ever happen. A wall isn't a price under this definition. Nobody pays a hash; reversing one is just infeasible. Neither is work a protocol needs and never charges for. The table at the end of the section has a column for each of the three kinds, and what it shows is a finding about the five, because the definition came first.

Three things are easy to blur in a section like this, so I'll name them now and keep them apart all the way through. There's the *real specification*: the document real money and real security run on. There's my *transcription*: one mechanism copied out of that document into a mathematical model, small enough to examine. And for some of the five there's a *wrapper*: a tiny abstract model that takes one premise as given and says what follows from it. A claim proved at one level isn't a claim at the level above it. Near the end I'll walk one system, gas, through all three, so you can see exactly where each claim stops.

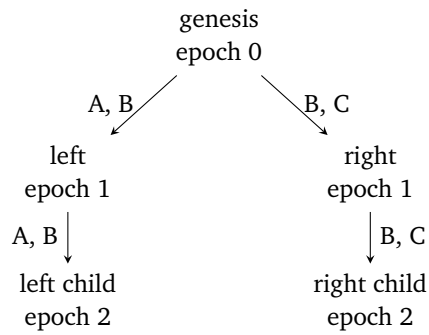
For each system I isolate the mechanism the question concerns: a *fragment* with its own states and operations, transcribed from the system's specification where one was available. I will put that mechanism on the page, state the guarantee it carries, and show how the argument follows its rules.

17.1 Proof-of-stake finality: the price is on going back

Ethereum and chains like it decide which blocks are final by having validators, people who've locked up money (their *stake*), vote on checkpoints: blocks picked out at regular numbered rounds called *epochs*. A vote isn't for one checkpoint. It's a *link* from a checkpoint the validator already trusts, the source, to a later one, the target, and every chain starts from one first block, its *genesis*, which counts as trusted from the start. In the design Vitalik Buterin and Virgil Griffith published in 2017, called Casper, a checkpoint is *justified* when validators holding two thirds of the stake vote a link to it from a justified source, and *finalized* when it is justified and a two-thirds link runs from it to its direct child. The record is "this checkpoint is final."

Two rules carry the price: don't vote for two different targets in the same epoch, and don't cast a vote whose span, source to target, surrounds the span of another of your votes. Break either and your deposit can be destroyed, which is called *slashing*. The theorem they prove, called accountable safety, says that if two *conflicting* checkpoints are ever both finalized, at least a third of the stake broke a rule and can be slashed. So the price is on *reversal*, on making the record contradict itself. The write itself, finalizing, need not incur a slashing penalty. This is not a claim that running validators has no operating costs or other protocol charges. In the terms from the start of the section, Casper's price is a bond. The deposit stands behind every vote, the finalizing votes included, and it's taken only from a validator whose own votes contradict each other, which is what making the record contradict itself takes from a third of the stake.

Do it with three validators of equal stake. Draw genesis at epoch 0, two competing children at epoch 1, and one child of each at epoch 2. Let A and B vote both links on the left branch, from genesis to its first child and from that child to the next. Let B and C do the same on the right. Each link has two of the three votes.



B appears on both branches at each target epoch. A and C do not. The diagram records votes; it has no step deducting a balance. Both epoch-1 blocks are justified and have the required link to their child, so both finalize. B voted for different targets at the same epoch, which makes B's third of the stake slashable, while A and C have broken neither voting rule. The votes supply evidence for a penalty; this example does not execute the transaction that collects it, and Theorem 17.2 below has exactly this instance, so the theorem's premise isn't vacuous: forks of that kind can be built, and the theorem fires on them.

The fragment is a reconstruction of the published casper-proofs argument from Runtime Verification and the University of Texas at Austin, based on Yoichi Hirai's proof, with validator sets and quorum intersection as a premise. A *quorum* is a set of validators big enough to act, here two thirds of the stake, and *quorum intersection* is the arithmetic of the three-validator example: two thirds plus two thirds is four thirds, so any two quorums share at least a third. The fragment takes the overlap as given; it doesn't do that arithmetic. It has no deposits, no rewards, no fork choice (the rule for picking a branch to build on), and no connection to anything else in this book. A finalization fork implies that a set of validators the size of that overlap, at least a third of the stake, has committed a slashable offence. A one-validator chain finalizes without any such offence, which is the "write costs no stake" sentence, proved here in a degenerate setting.

The fragment is small enough to read in one sitting, and every name below is from the transcription.

Definition 17.1 (Casper setting and votes)**.** A *setting* has a type of validators, a type of block hashes with a parent relation and a genesis hash, and two classes of validator sets, the "two-thirds" sets Q_1 and the "one-third" sets Q_2. It carries two premises: *quorum intersection*, for any two sets in Q_1 some set in Q_2 lies inside both; and *at most one parent* per hash. A *state* says, for each validator, target hash h, target epoch t and source epoch u, whether that validator voted the link (h, t, u). A link from p at epoch u to h at epoch t is *justified by* q when $q \in Q_1$, every member of q voted (h, t, u), h is the $(t - u)$-th descendant of p, and $t > u$. Genesis is justified

at epoch 0, and a block reached from a justified block by a justified link landing at epoch t is justified at t. Block h is *finalized* at epoch v with child c when c is a child of h, h is justified at v, and the link from h at v to c at $v+1$ is justified. A *finalization fork* is two finalized blocks $h_1 \neq h_2$, neither an ancestor of the other. The transcription calls a validator *slashed* when it voted two different targets with the same target epoch (a double vote), or voted links (h_1, t_1, u_1) and (h_2, t_2, u_2) with $t_1 > t_2$ and $u_2 > u_1$ (a surround vote). The state is *quorum-slashed* when some set in Q_2 consists of such validators. These names describe offending votes, not a balance deduction: there are no account balances in this state.

Theorem 17.1 (Accountable safety)**.** In every setting, every state with a finalization fork is quorum-slashed. □

Proof. Let the fork be h_1 finalized at v_1 and h_2 at v_2. If $v_1 = v_2$, the epoch is above 0, since genesis is the only block justified at 0, so the links that justified h_1 and h_2 there are each voted by a set in Q_1; quorum intersection gives a set in Q_2 that voted both, and their targets h_1 and h_2 differ. That set double-voted. If $v_1 \neq v_2$, say $v_1 < v_2$, walk down the chain of justified links that justifies h_2. Every block on that chain is an ancestor of h_2, so none descends from h_1. (So $v_1 > 0$: the only block justified at epoch 0 is genesis, an ancestor of every justified block.) Some link in the chain starts at or below v_1 and lands above v_1. If it lands at $v_1 + 1$, it and the finalization link of h_1 have distinct targets at that epoch, and the intersection set double-voted. If it starts at v_1, its source is a justified block at v_1 other than h_1, and the links that justified those two blocks give a double vote the same way. Otherwise it starts below v_1 and lands above $v_1 + 1$, so it surrounds the finalization link of h_1, and the intersection set made a surround vote. ■

Everything the fragment promises stands on the overlap, and with the overlap taken away it promises nothing. The one parent is in the setting because the original proof uses it, and the proof above never does.

Theorem 17.2 (The premise can hold, and the write costs no stake)**.** (1) In the three-validator setting where each validator has stake 1, Q_1 is the sets of stake at least two thirds and Q_2 the sets of at least one third, the state where A and B vote one branch and B and C the other has a finalization fork; B is slashed, A and C are not, and $\{B\}$ is in Q_2. (2) In the one-validator chain setting, one vote finalizes genesis and no validator is slashed. □

Proof. Both are computations on fixed finite data. In (1) every link carries two of the three votes, so stake two thirds, so both epoch-1 blocks are justified and each has a justified link to its child at epoch 2: both are finalized, and as siblings neither is an ancestor of the other. B voted two different targets at epoch 1, which is the double vote; A's and C's votes are one link at epoch 1 from source 0 and one at epoch 2 from source 1, which neither double-vote nor surround. In (2) the one validator's vote from genesis at 0 to its child at 1 is a justified link, so genesis is finalized, and a single vote can neither double-vote nor surround. ■

For the record itself, one thing that means is that finality is revocable at a price, the price is named, and a final block can hold a false statement.

The obvious next question is whether the 2017 design is what Ethereum actually runs. The answer is partly, and the paper's own title is the clue. It's called *Casper the Friendly Finality Gadget*, and its first sentence says it's a finality system that *overlays* a chain still choosing its blocks by proof of work, so it's a gadget, bolted on top. In 2020 a larger group, Buterin among them, wrote up *Gasper*, which joins Casper's finality to a fork-choice rule called LMD GHOST, and on 15 September 2022, in the switch people called the Merge, Ethereum stopped choosing blocks by proof of work at all and went over to proof of stake. So the checkpoints and the two slashing rules above are one part of a system that holds real money today, and the transcription is of that part's safety proof as the 2017 design states it. Everything between the two, the fork choice and the rest of the running protocol, is the gap between my transcription and the running system, which I walk through near the end of this section with a photocopied recipe and a kitchen. None of that moves the price: finalizing still costs an honest validator no stake, and what costs is contradicting yourself.

17.2 Gas: the one that charges a fee at the write

Every operation on Ethereum costs *gas*, paid in ether by whoever sent the transaction. The yellow paper, Ethereum's formal specification, gives the reason in one line: every computation is fee-bearing so the network can't be abused and halting questions are sidestepped. The record I care about is a nonzero slot in the shared storage, which every full node in the world (every computer keeping its own complete copy of the chain) has to keep until it's cleared. Clearing a slot is allowed, and the refund rules below exist to reward it.

Writing a fresh storage slot from zero to nonzero costs twenty thousand gas, by the table in the paper's appendix, plus twenty-one hundred for touching a *cold* slot, one the transaction hasn't touched yet (after the first touch it's *warm*). As I read it, the point is to discourage storage growth, since storage grows every node's database. That charge is a fee on the write, at the moment of the write, and it's the only fee in

the five. Gas has a budget too, in the sense from the start of the section: every block has a gas limit, and the gas its transactions use can't exceed it, so only so many writes fit in a block.

Where does a number like twenty-one hundred come from? Somebody picked it, so the obvious next question is who, and why, and the answer has a date on it. In September 2016, just as Ethereum's developers were gathering in Shanghai for Devcon 2, their developer conference (the third one, since they started counting at zero), attackers started filling blocks with an operation that was cheap in gas and expensive in fact: it sent every node off to read data from its disk, and nodes fell behind the chain. The first repair, a fork that October called Tangerine Whistle, raised the prices of the operations the attack had leaned on. The cold-slot charge came later, in a proposal by Vitalik Buterin and Martin Swende called EIP-2929, switched on in the Berlin upgrade of April 2021, and the motivation it gives is still those 2016 attacks. So twenty-one hundred isn't a law of nature or a measurement of anything deep. It's a scar: a price set after somebody found the place where the old price was wrong. That doesn't change the argument; the write is still priced at the moment of the write. It does change how I read the rest of that table: some of its numbers, at least, are somebody's answer to "what did this cost last time," which is a record of a different kind.

Now for the leak, which is Section 7's leak. Set a slot and then clear it *in the same transaction* and you get a refund. Count it through: first write twenty-two thousand one hundred; clearing it costs a hundred and credits a refund of nineteen thousand nine hundred; net, twenty-three hundred. The other half is that if you leave it set, you pay at least twenty thousand. The write is priced, and the *revoked* write nearly isn't. But I have to add what the fragment leaves out, because twenty-three hundred on its own misleads: the realized refund is capped at one fifth of the gas the whole transaction used (that's the 2021 change from Section 7, the one that ended the refund-hoarding token's advantage). So a real set-then-clear pays a lot more than twenty-three hundred. As for how much more, the cap is a fifth of everything the transaction used, and that includes the twenty-one thousand every transaction pays just for existing. Take a transaction that does nothing but set and clear the slot, and count only that base and the two stores (the instructions that load their arguments add about a dozen more): it uses 43,200 gas, so the cap is 8,640, so it pays 34,560, which is 13,560 on top of the base. (Look at the slot alone and forget the base, which the rule doesn't do, and you'd get 17,760.) Either way it's far above twenty-three hundred. The counter-level net is twenty-three hundred, and what the transaction pays is not.

The arithmetic below is the way the specification has it, so you can redo it with a pencil.

Definition 17.2 (One slot in one transaction)**.** A slot run records the slot's value at the start of the transaction (original), its current value, whether it has been touched (warm), the gas charged so far, and the refund counter. A store of a new value charges 2100 if the slot is cold, plus 20000 if the slot still holds its original value, the new value differs, and the original is 0 (2900 if the original is nonzero), and otherwise 100. If the value is unchanged, the refund does not change. Otherwise add together these applicable adjustments: add 4800 when the original and current values are nonzero and the new value is zero; subtract 4800 when the original is nonzero and the current value is zero; and, when the new value restores the original, add 19900 if that original is zero, or 2800 if it is nonzero. Store the new current value, mark the slot warm, and keep the original value unchanged. The run starts fresh: original and current 0, cold, gas and refund 0. The *net* charge is gas minus refund.

□ **Theorem 17.3** (Pricing the persistent write and the revoked one)**.** (1) For every list of stores from a slot whose original and current values are 0, warm or cold, with gas and refund 0, if the slot ends nonzero then the net charge is at least 20000, and from a fresh slot, which is cold, at least 22100. (2) Storing 1 then 0 leaves the slot at 0 with net charge 2300; storing 1 alone has net charge 22100.

Proof. (1) Keep two lines on the sheet: "current value zero: net at least zero," and "current value nonzero: net at least 20000." The starting slot meets the first line. I will check that every store leaves it meeting whichever line describes its new value. The original value remains zero throughout, so there are only four changes to check. Zero to zero adds 100 to net; zero to nonzero adds 20000; nonzero to nonzero adds 100; and nonzero to zero adds $100 - 19900 = -19800$. The last case starts with net at least 20000 and so ends at least 200, still nonnegative. So every store leaves the appropriate line true. Carry the two-line test through the list of stores: if the final value is nonzero, the net charge is at least 20000. A cold access adds another 2100, once, on the first store. Keep that extra charge in a separate column; later updates are exactly the ones just checked and do not depend on the accumulated total. A nonempty run from a cold slot therefore pays that 2100 on top of the bound, giving at least $20000 + 2100$ when it ends nonzero. (2) Direct computation: $2100 + 20000 = 22100$ for the first store, 100 for the second, refund 19900, and $22200 - 19900 = 2300$. ■

The fragment covers one slot in one transaction, with the gas and refund arithmetic copied out of the Ethereum execution specification's storage instruction and

gas table, at a named commit. It has no stack, no call context, no other slots, and *no refund cap*, the one-fifth rule applied at the end of a transaction.

Then comes the wrapper, which is a different kind of object, and the walk near the end is about exactly that difference. It's a *gas schedule* in the abstract: any states, any steps, a cost per step, a yes/no commitment flag, and a charge predicate saying which steps get charged. It formalises no EVM (the Ethereum virtual machine) at all. It is Section 8's result with the whole-run ceiling, the one that pins the amount as well as which steps pay, under a new name. Its two halves are worth reading separately. Leave one committing step uncharged, and a one-step run commits at no charge, so the floor fails outright; one missed commit is enough. Charge one step that doesn't commit, and the schedule charges more than the number of commits on that one-step run, so the ceiling fails. And the class isn't empty: in any system, the price that charges one exactly on a step that raises the flag and nothing on any other meets both halves (Section 8's step price).

The wrapper itself is the cleanest thing in this section and has the least Ethereum in it.

Definition 17.3 (A gas schedule)**.** A *gas schedule* has states, instructions, a step, a cost $c(s,i) \in \mathbb{N}$ that may depend on the state, a Boolean commitment flag on states, and a Boolean charge predicate $\chi(s,i)$, with two premises: a charged step costs at least 1, and an uncharged step costs 0. A step *commits* when it takes the flag from false to true. For a run t from s, let $c(t)$ be its total cost and $n(t)$ its number of committing steps. The schedule has the *floor* when $c(t) \geq n(t)$ for every run, and *no overcharge* when $c(t) \leq n(t)$ for every run. It has *unit pricing* when $c(s,i) = 1$ if $\chi(s,i)$ and 0 otherwise.

Theorem 17.4 (Gas schedule exactness)**.** For every gas schedule: it has the floor and no overcharge if and only if $\chi(s,i)$ holds exactly when the step from s by i commits, and it has unit pricing. Moreover, (1) if some committing step from an uncommitted state is uncharged, that one-step run costs 0 and the schedule fails the floor; (2) if some charged step does not commit, the schedule overcharges. □

Proof. The equivalence is Section 8's characterization with the whole-run ceiling, under another name. From right to left, each step's cost is 1 on a committing step and 0 otherwise, so a run's cost counts its commitments. From left to right, apply the floor and no overcharge to one-step runs: a committing step must cost at least 1, so it is charged; a charged step costs at least 1 and may not exceed its commitment count, so it commits and costs exactly 1. Parts (1) and (2) are those one-step runs read the other way. ■

Gas prices storage growth and computation. It says nothing about whether what's stored is true.

17.3 Attestation: hash chains, and a quote about the past

A trusted platform module keeps platform configuration registers, or PCRs. An extension hashes the old value together with a new measurement digest. A quote signs selected PCR values and challenge data supplied by the asker; the verifier must check the signature, challenge and expected measurements. That can establish a fresh report of those values. It does not establish that every relevant fact about the machine was measured.

PCR reset permissions depend on the register, locality and platform rules. Some reset at startup; some support permitted resets during a boot. Thus "you can only extend" describes an interval with those resets excluded. Intel SGX uses enclave measurements and attestation reports for a related purpose, but it does not use this PCR-extension interface.

What does the specification price on the registers? Nothing: no fee, no bond, no budget. Within the stated reset and access restrictions, forging a different measurement history that passes verification relies on defeating the cryptographic assumptions or trusted components. Authorized resets are a separate operation, not an attack that requires breaking a hash. Expensive against infeasible is the difference between a price and a wall, and it's easy to blur, which is why the definition at the start of the section leaves walls out. What stands behind a cost can come in three currencies. It can be economic, a fee or a deposit the protocol takes (Casper and gas are here). It can be cryptographic, a reversal made infeasible by a hardness assumption, which makes it a wall with no exact price on it (attestation and transparency are here). Or it can be physical, energy spent or hardware worn, which none of these five specifications charges as a fee or a bond; the nearest thing is the chip's write budget, below. And the record's permanence is only as long as the register lives: a register's continuity lasts only through the interval allowed by its reset rules, not necessarily an entire boot.

Why the registers, then, when something that lasts would make a better record? Because the quote is what a remote asker reads, and the registers are what it signs. The same chip has longer-lived records, and they're closer to the latch than anything else in this section. Version 2.0 of its specification has storage slots of a kind it calls a bit field, changed only by a command that ORs new bits into the old value. During that index's lifetime the command can set a bit and cannot clear one, the OR rule from Section 11. Deleting and redefining an index is governed by separate authorization and policy; it must be included in any claim about persistence across that lifetime. It also has counters advanced by an increment command. On first use,

a new counter starts above the values previously held by counters on that TPM, so deleting an index and making a new one does not restart the count at zero. This is a guarantee of the specified chip, not protection against replacing it with another chip. And those slots are rationed. Bryan Parno and his coauthors, building a system called Memoir in 2011, noted that the specification of the day only required a counter to be able to tick once every five seconds, and that the chip's storage was expected to survive about a hundred thousand writes in its whole life, so writing once a second would wear it out in under twenty-eight hours. Memoir writes to it twice per boot. Those figures describe the hardware assumptions discussed in Memoir, not a universal write quota for every TPM. Endurance and throttling constrain real use, but an expected endurance figure is not itself an enforced protocol budget. The quoted PCR interface supplies no such numerical price. On the bit field it's the budget of Theorem 11.10's chain of k-bit words, which can only climb k times. Phones and small controllers go one floor further down, to fuses like the PROMs in Section 11: a version number kept as a row of fuses, one more blown at each security update, and boot code that refuses anything older. An industry guide on it, Rob Smart's 2021 explainer for PSA Certified, a security certification scheme for chips and devices, tells designers to make sure there are enough fuse bits to last the number of updates expected over the device's lifetime, which is part (5) of that same theorem, written as a design rule.

What all of this is for is *rollback*. Memoir's example is a password checker restored to an old saved state to undo a user's password change, and in 2017 a system called ROTE did the same job for Intel's enclaves by having several machines vouch for each other's counters, with the stated result that the only way left to break it is to reset every one of them to its start. In 2016 Raoul Strackx and Frank Piessens's Ariadne got that job down to flipping a single bit of non-volatile memory per state update, which they call the theoretical lower limit. Notice who the attacker is in every one of these: whoever gets to put an old state back. A bad program doesn't come into it. A record kept by the step can't see that attacker. Every guarantee the small machine gives about a record's earned history (Section 15) is stated from a clean start; put back a state the machine really reached and each one still holds of it, through the run that reached it first, and none of them can say that nothing came after it. The per-step toll still holds after a restore; it just can't tell you what happened before it.

Replicated services put the same kind of record somewhere else: inside each replica, where a replica that's been taken over still can't make it say two different things. Byung-Gon Chun and coauthors' attested append-only memory (A2M, 2007) is a log a replica can only add to; Dave Levin and coauthors' TrInc (2009) shrinks that to a trusted counter; MinBFT (Giuliana Veronese and coauthors, 2013) uses a trusted counter to reach agreement with $2f + 1$ replicas when f of them can lie,

where the usual count is $3f + 1$; and Damysus (Jérémie Decouchant, David Kozhaya, Vincent Rahli and Jiangshan Yu, 2022) has a small trusted service, the Checker, that keeps a monotonic counter of where the protocol is. Allen Clement, Flavio Junqueira, Aniket Kate and Rodrigo Rodrigues (2012) worked out, with proofs, what such a record buys, and as I read their result it's a good warning for this book. On its own, stopping a replica from saying two different things doesn't lower the $3f + 1$ bound in an asynchronous network at all. Add signatures that a third party can check, and it turns any protocol that survives crashes into one that survives liars, with no extra replicas. The way I read it, a record helps the others only as far as they can check what it says, which is Section 8's point from the other side.

There's a fourth currency under those three, the logical kind, and it's the one every permanent record on a finite chip pays in: Section 3's counting of used and fresh states applies to any of these records. With only finitely many states holding the record on, a step that switches it on somewhere, and never switches it off anywhere, forgets something. Forgetting alone doesn't single out a record write: any step that overwrites a field able to hold two values forgets something too. What singles it out is where the forgetting lands. Take the states the record already reads yes on, together with the one the step switches on. The step sends every one of them to a yes state, and it can't keep them apart, so what it destroys is a distinction on the record's own yes side: two states it already vouched for, or the state it just raised and one that was up already. Any overwrite merges states somewhere; a permanent write is forced to merge them on that side. That needs no premise about money, keys or heat. The economic, cryptographic and physical payments each need a premise of their own about their kind (a fee rule, a hardness assumption, Landauer's principle), and they hold exactly where that premise holds.

□ **Theorem 17.5** (A permanent write forgets on its own yes side)**.** For any states S, instructions I, step and Boolean reading: suppose a duplicate-free list names every state, the instruction i never turns the reading from on to off, and i turns it on at some state s. Then the step by i is not injective. More: the step by i sends every state that reads on, and s, to a state that reads on, and it is not injective even on those states.

Count the on-states and call their number K. The list of all states is finite, so this count is finite too. The instruction keeps each of those K sources on, and it also sends the additional off-state s to on. That gives $K + 1$ distinct sources with only K on-states available as landing places. Two sources must land together. Both came from the stated collection, so the failure to keep sources apart already occurs there. ■ Permanence of any other instruction was never needed.

What the quote can do is say what was *measured*. It can't say the measured code is correct, and it can't say anything about runtime state that was never folded in. A correctly measured buggy program attests beautifully. The fragment proves that exact limit: one register, a hash about which nothing is assumed, a quote as a pair, and no function of the quote decides a runtime flag the quote ignores, which is Section 8's cube again. And a second small result shows that a sign-and-verify interface *alone* gives no authenticity (a scheme where verify always says yes passes the interface), so a cryptographic premise has to be added; the fragment doesn't model signatures, formats, or a freshness check (the nonce is just a field the verifier fixes).

The same limit, turned outward, is the honest edge of the four conditions of Section 14. Every one of them is about a thing the machine holds: same property, same thing, same edition, honest check. For a claim about the world (the oven is hot, the parcel arrived) there's no edition the machine controls. Whatever reads the world is a window onto it, and "nothing changed between the check and the commit" stops being something a step can enforce and becomes a premise about the sensor. So a record can establish claims about the machine's own state, and claims about the world only through named premises about whatever measured them. A correctly recorded sensor reading establishes what the sensor reported. Whether the oven was hot still depends on the sensor and the connection between its report and the world.

Definition 17.4 (Platform and quote)**.** Fix any functions $H : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ (the extend hash) and $\delta : \mathbb{N} \to \mathbb{N}$ (the digest); nothing is assumed about either. A *platform* is a measurement log (a list of numbers) and a Boolean runtime flag. Its register is the log folded from 0 by H. The *quote* with nonce n is the pair $(\delta(\text{register}), n)$.

Theorem 17.6 (The quote cannot attest unmeasured state)**.** For every nonce n there is no function D from quotes to Booleans with $D(\text{quote}_n(P)) = \text{flag}(P)$ for every platform P. □

Proof. Take two platforms with the empty log, one flagged true and one false. Their quotes are equal, since the quote never reads the flag, so D would have to return both values on one input. ■

Anything that depends only on $\delta(\text{register})$, the quote can answer; that's what it's for. And the second result is the one I mentioned: a "signature" scheme whose check

always says yes fits the sign-and-verify interface perfectly and accepts a signature no key ever made, so the interface on its own promises nothing.

17.4 Certificate transparency: visible, unpriced

The certificates that make your browser show a padlock are issued by hundreds of authorities, and a bad or compromised one can issue a certificate for a domain it shouldn't. Certificate Transparency puts certificates in public append-only logs built as Merkle trees. An inclusion check ties a certificate to a particular root; a consistency check ties a later root to one the reader already has. It was specified in RFC 6962 in 2013 and revised as RFC 9162 in 2021. I went looking for which one the running logs follow, and it's still 6962: Mozilla's reference for web developers says browser implementations are based on 6962, and as late as November 2025 Chrome's team was only proposing to drop its rule that every certificate carry a timestamp from at least one 6962 log. The fragment I transcribe is 9162's. A Merkle tree is Section 2's hash chain folded into a tree: hash each certificate (those are the *leaves*), hash the leaf hashes in pairs, hash those in pairs, and so on up to one *root* hash. To prove one leaf is in, the log hands you only the few hashes sitting beside the climb from that leaf to the root, the *path*, and you redo the climb yourself. A second check, *consistency*, shows that a bigger tree is a smaller one with leaves added on the end and nothing rewritten. The record is a leaf in the tree.

The data structure has a history of its own in security. The computer scientists Scott Crosby and Dan Wallach, at Rice University, built tamper-evident logs on the same kind of tree in 2009, with a proof that an entry is in the log and a proof that a later version of the log extends an earlier one, each a logarithmic number of hashes. A rewrite that fails to extend a previously witnessed root is caught by the consistency check.

The RFC prices nothing, and it says in its first section that logs let interested parties *detect* misissuance afterward. They don't *prevent* it. And a log that shows different views to different people (a *split view*) can defeat the auditing, so logs are still trusted parties. What it buys is visibility.

Which raises the obvious question: if the specification prices nothing, why does any authority bother logging? Nothing in the RFC makes it. The answer is a browser. Chrome's team set the rule, enforced from Chrome 68 in July 2018, that every certificate issued after 30 April 2018 had to meet its transparency policy, and a publicly trusted certificate that doesn't is simply not trusted by Chrome: the visitor gets a full-page warning in front of the site. So there is a price after all, though it isn't anywhere in the specification: it lives in one vendor's browser, and it's the price of being shown to that browser's users at all. Is that the bottom? Not quite, because it means the strongest lever on this record is a policy one company can change,

which is a fact about who holds the stamp. The tree has nothing to do with it. That policy changes nothing in the table at the end of this section, which asks what each specification prices. But it's worth keeping two sentences apart: "the specification prices nothing" and "nothing is priced."

Do the inclusion proof by hand. Lay out seven cards d_0 through d_6, each standing for one leaf hash. Join neighbours with paper brackets: $a = \nu(d_0, d_1)$, $b = \nu(d_2, d_3)$ and $c = \nu(d_4, d_5)$. The last card stands alone. Next join a to b, giving $L = \nu(a, b)$, and c to d_6, giving $R = \nu(c, d_6)$. The root is $\nu(L, R)$. Here ν means "put these two expressions, in this order, inside one labelled bracket." Equal symbolic hashes must have exactly the same bracket structure.

To check d_3, keep only that card and the three siblings d_2, a, R. Combine d_2 on its left to recover b, then a on the left to recover L, then R on the right to recover the root. You never needed to open the cards inside a or R. Those three siblings are the inclusion path.

Step	What is supplied	What is rebuilt
Start	card d_3	starting leaf hash d_3
1	d_2 on the left	$b = \nu(d_2, d_3)$
2	a on the left	$L = \nu(a, b)$
3	R on the right	root $= \nu(L, R)$

Swapping left and right changes the bracket. The path has to carry the correct order as well as the correct siblings. The hashes here are brackets, not numbers, so what you just checked is how the tree is put together; a real log uses short hashes, and "two different trees can't give the same root" is then a separate promise from the cryptography. Nothing in this fragment signs anything, no auditors talk to each other, and no log misbehaves.

Definition 17.5 (Inclusion check, RFC 9162 section 2.1.3.2)**.** Given a leaf index m, a tree size N, a leaf hash, a root hash and a path, the check rejects unless $m < N$. Otherwise start counters $f = m$, $s = N - 1$ and an accumulated hash r equal to the leaf hash. For each path element p, reject if $s = 0$. If f is odd or $f = s$, replace r by $\nu(p, r)$; in this branch, while f is positive and even, halve both counters with rounding down. In the other branch replace r by $\nu(r, p)$. After either branch halve both counters once more. At the end accept exactly when $s = 0$ and r is the claimed root. A *symbolic hash* makes a leaf hash $\ell(x)$ and a node hash $\nu(a, b)$ distinct constructors: two hashes are equal only when their entire constructions agree.

The consistency check used here takes old and new sizes M, N, roots u, v and a path, and rejects unless $0 < M < N$. If M is a power of two, prepend u to the path. Reject an empty resulting path; otherwise its first element seeds two accumulated hashes $r_{\text{old}}, r_{\text{new}}$. Start $f = M - 1$, $s = N - 1$, and halve both while f is odd. Process each remaining path element p: reject if $s = 0$; when f is odd or $f = s$, put

p on the left of both accumulated hashes and halve both counters while f is positive and even; otherwise leave r_{old} alone and put p on the right of r_{new}. After either branch halve both counters once. Accept exactly when $s = 0$, $r_{\text{old}} = u$ and $r_{\text{new}} = v$. All halvings round down.

□ **Theorem 17.7** (The RFC's own examples pass)**.** With the symbolic hash, in the RFC's seven-leaf tree with leaves $d_0, \ldots, d_6$ (writing d_k also for its leaf hash $\ell(d_k)$), the inclusion check accepts leaf 3 with path $[d_2, \nu(d_0, d_1), \nu(\nu(d_4, d_5), d_6)]$, and likewise leaves 0, 4 and 6 with their paths; the consistency check from size 4 to size 7 accepts. For every input with $m = N$, the inclusion check rejects.

Proof. Use the brackets a, b, c, L, R built above. For leaf 3, the counters start at $(3, 6)$: put d_2 on the left, leaving $(1, 3)$; put a on the left, leaving $(0, 1)$; put R on the right, leaving $(0, 0)$ and root $\nu(L, R)$. Leaf 0 has path $[d_1, b, R]$ and three right combinations. Leaf 4 has path $[d_5, d_6, L]$: right, right, left. Leaf 6 has path $[c, L]$: left, then left; its first branch shifts $(6, 6)$ to $(3, 3)$ before the ordinary halving, which accounts for the unpaired last leaf. Each reconstructs the same root. For consistency from 4 to 7, the old root is L and the supplied path is $[R]$. Prepending L seeds both hashes with L; shifting the initial $(3, 6)$ gives $(0, 1)$, so the sole remaining element leaves the old root alone and makes the new root $\nu(L, R)$, with final counters $(0, 0)$. The boundary case $m = N$ fails the first test $m < N$. ■

And the append-only part of the fragment is as plain as it sounds: the history is a list, and adding to the end of a list leaves every old entry where it was.

17.5 Proof-carrying code: no price, no permanence

Necula's idea, from Section 9, is to ship untrusted code with a proof that it obeys the host's policy; the host computes what would need to be true and checks the proof with a small checker. The record is the checked proof or the result of checking it. An implementation may retain either. Proof-carrying code does not by itself require a permanent latch or assign an abstract toll to acceptance. Producing and checking the proof both take computational work; that work has not been identified with the unit charge in this book.

17.6 The same table, five ways

Lay them out by the one definition of price from the start of the section. Casper puts a bond behind the write and takes it on *reversal* (slashing). Gas takes a fee at *the*

write (and refunds the revocation, capped), and rations writes with a gas limit per block. Attestation prices *nothing* on the registers the quote signs, and puts a budget on its longer-lived slots and counters. Transparency prices *nothing*. Attestation and transparency make reversal infeasible or visible. Proof-carrying code prices nothing and keeps nothing.

System	Fee at the write	Bond	Budget	What the transcription proves
Casper finality	none	the deposit, taken on a contradicting vote (slashing)	none	a finalization fork means a slashable third
Gas	20000 for a fresh slot, mostly refunded on revocation, up to a cap	none	the gas limit of each block	a persistent write to an empty slot nets at least 20000
Attestation	none; reversal is infeasible	none	none on the registers; a write budget on the non-volatile slots and counters	the quote can't decide what it never measured
Transparency	none; misissuance is visible	none	none	the RFC's own inclusion examples pass
Proof-carrying code	no required persistent latch	none	none	the checker is the verification condition

Each row proves what its own transcription says, and no more. To read a row as a claim about the running system you'd need evidence from that system, and no row shows that certification is the one event every model has to price.

So the thing the toll charges for, the moment the record is written, is charged a fee in one of the five specifications I read, and in that one the designers had to patch the place where the charge could be clawed back. I find that neither a vindication nor a refutation. It's a fact about what people built, and what they built was mostly walls (hash chains, signatures) and penalties for contradiction. Tolls on commitment were rare. Whether a toll on commitment is a *better* design than a wall is not something I can prove, and I won't pretend the five say so. What I read

in them, and it's a reading of five fragments I chose, is that each has a condition treated as a record, though the conditions differ in durability, and that the price axis is mostly empty. With one definition of price fixed before the survey, that last part is a count: of fifteen cells in the table, four are filled (one fee, one bond, two budgets), and only gas takes a payment at the moment of the write.

That leaves the question Section 11 raised and Section 18 has to face. The five systems have five different events chosen as "the record": finalized, committed, attested, logged, checked. Is there anything that picks out one event as *the* one worth recording, or is it an unforced choice every time? The pointer thesis is my proposed answer, and the next section gives the evidence it asks for.

CHAPTER 18

The pointer thesis

Everything so far leaves one choice open. A permanent record driven by the machine is a latch on an event, and the event is yours to pick (Section 11, the two cups on one die). Five real systems each picked a different one (Section 17), and for the machine in this book I picked certification. So I owe you an answer to the obvious question: is that just my choice, or is there something about how people and machines lean on records that picks this event out?

What I have is a thesis, in the sense the Church-Turing thesis is one (Section 6): a claim tying defined things to the world, which no proof can settle. Whether it stands or falls is what decides whether the choice of event has a reason behind it. Nothing before it depends on it; the latch, the windows and the counting arguments hold whichever event a system records. What follows is the claim, where I got it, and how its terms apply to the systems being described.

18.1 The claim

Of all the events a machine could put on its ledger, the ones its price list charges for and its counter adds up, certification is the one whose records get *copied and relied on by parties who didn't produce them*: checked, stored, and acted on across independent places that share no other state. Copying alone can't be the test. A replicated system copies everything it holds to every node, the gas used in a block header as faithfully as the word final, so rival events get copied too. Reliance does not automatically separate them either. In Ethereum, the previous block's gas use helps determine the next block's base fee: another party reads it and acts on it. That is an explicit rule in EIP-1559.

So the examples cannot establish that certification alone is copied and acted on. The thesis asks whether a separately justified class of reliance selects it, and whether that selection gives a reason to charge its establishment. The formal toy below fixes its readers and actions in advance. It can test those choices, but it cannot choose the real readers for us. Both the claimed selection and its connection to prices remain to be justified.

There's a second handle on the same question, and it comes from Section 3. The permanent-certificate theorem (Theorem 3.1) never uses the word certification for anything special: given Section 3's rule that forgetting is charged, it prices any reading that, once up, stays up, on a machine with finite memory. So it points at a narrower candidate: the events to price are the ones that write a record no later step can take back. Certification is one of them. As an exact answer that candidate is false, because a merging step can also revoke (Section 3's three cards). Whether the records that are copied and relied on are the permanent ones, and whether being copied and relied on picks out anything the merge criterion doesn't, is what the two handles have to settle between them.

I can see two holes in that from here, so you don't have to find them for me. The first is a missing premise. Even if the world copies certification records more than any other kind, "copied" doesn't give you "priced" on its own. Something has to connect them, and the best candidate I have is a premise about incentives: a record that many independent parties copy and lean on is exactly the record worth forging, so it's the one where a price buys the most protection. That's a reasonable premise, but it's still a premise. The thesis's second half states the connection as a claim to be tested; it doesn't explain it, and nothing in this book supplies the explanation. The second hole is a circle. If "certification" just means "the record other people rely on," then of course it's the one that gets relied on and copied, and the claim says nothing. So the event has to be picked out without mentioning reliance: here it's a reading going up for good right after a check of the same claim passed, which you can find in a run without asking who reads it. Whether the world copies *that* event more than its rivals is then a real question, and it could come out no.

The thesis says no more than that, and what's left is the rest of what a thesis owes you.

□ **Thesis 18.1** (The pointer thesis)**.** Let C be the class of ecosystems with reliance that faithfully model deployed record-keeping disciplines, each with its reachable states, its observers reading independent carriers, its certification event, and the records its discipline prices. Then for every member of C: (1) the certification event is the pointer on the reachable states, so it is copied and relied on by parties who didn't produce it, and every event copied and relied on in that sense holds wherever certification does; and (2) every record the discipline prices is copied and relied on by parties who didn't produce it.

The thesis leans on two notions with no precise mathematical definition: what it is to "faithfully model" a system, and which systems count as "deployed." Those two tie the definitions to the world, the way "computable in the plain-English sense"

does in the Church-Turing thesis, and no definition can do that job, so the claim is a thesis. The other term it needs, independent carriers, has an exact definition: each observer's view depends only on its own part of the world, and any two observers' parts can be set independently of each other. The schema takes the class, the reachable states, the certification event and the priced records as inputs; it proves nothing about which real systems satisfy them.

18.2 The end of the logic

The logic that started in Part I ends here. Part I found what keeping takes: anything kept is paid in a merge or in fresh states, alongside the model-dependent thermal bounds (Section 1), and a toll on raising a flag is forced by three premises (Section 3). Learning doesn't have to merge anything, so it owes no floor; it borrows a blank page, which is a supply of fresh states, and can hand it back. Committing on a finite supply of fresh states has to merge (Section 5). Part II earned the rest. The counter is arithmetic once the schedule is fixed (Section 7). A window with a colliding pair in it can't price the flag exactly (Section 8). Entitlement is an object, a check, and a paid event, and, given lists, a tree and a covering that someone supplies, and a run that paid for the tree's depth, it costs at least the index bits it commits to (Section 9). Insight is a distinction made to stay, and a committed relation is that at its best (Section 10). A feeling doesn't qualify. Over any base a record that is driven by the computation and never comes down is a latch, with two unforced choices, the price list and the event (Section 11). With more than two places, a record that is driven by the computation and climbs a partial order is a row of cups, each one a latch; the toll over the whole row charges at least one on any step that turns up some cup, and a straight line of stages needs exactly one cup fewer than it has stages, cups that only turn up (Section 4). When the machine has finitely many states, its record only grows, and the ordinary machine forgets nothing, the toll prices crushing two states of one fibre, and a window can price exactly by seeing the cups move (Section 12). No inside procedure whose flip a substrate can represent decides a shortcut property, given a recursion theorem. What a record can do is carry the commitment (Section 13). This part made "established" a record that a check of the claim passed (Section 14), and the small machine earns one by a check, a commitment and a certificate, one unit each (Section 15). Of five specifications, read through my transcriptions, one charges a fee at the write (Section 17). For the choice of event, the pointer thesis proposes certification and lists how that proposal could fail (Section 18).

One question about the price itself is still open: do the requirements I've put on a price pick out the toll and nothing else? If a short list of them left only one price standing, that would settle it. The lists I tried don't, and the no is small enough

to check on three states. Call them blank, once and twice, with once and twice reading yes. The flip takes blank to once and leaves the other two where they are. The smooth takes both once and twice to twice and leaves blank alone. The smooth forgets something, so the price on merging from Section 3 charges it one unit. (The flip forgets too, blank and once both land on once, so the price on merging charges it as well, as the list requires.) It raises nothing, so the toll charges it nothing. Now try requirements that name no reading: no charge on a move that changes nothing, none on a move that forgets nothing, the same cost when you rename the states, a total that adds up along a run, a cost that looks only at the step, and a cost of one on the flip from blank. The toll, read as the price that charges the flip from blank and nothing else, meets every one of them on this machine. So does the price on merging. They disagree on the smooth, so that list doesn't pick between them. What picks the toll is a requirement that names the record: over any run the total is never more than the number of times the reading went up. With the floor, that leaves exactly one price, one unit per raise and nothing else (Section 8), and it rules the price on merging out, because the smooth costs that price one unit and raises nothing. The only renaming of the three states that respects the moves is the identity, so the renaming requirement asks nothing there. So the uniqueness is real and it's small. It holds once you've said which reading you're charging for, and which reading deserves the charge is the pointer thesis, which is still a thesis.

The uniqueness theorems that do exist are about pricing information loss, and they're worth a look because they say what kind of charge the price on merging is. John Baez, Tobias Fritz and Tom Leinster (2011), mathematicians who work in category theory, the mathematics Lawvere's diagonal theorem came from, asked what a measure of information loss has to look like. As I read their paper, they proved that, for maps that preserve a probability measure, Shannon entropy is up to a constant factor the only measure of information loss that is functorial, convex-linear and continuous. János Aczél and Bruno Forte, mathematicians at the University of Waterloo who worked on functional equations, and C. T. Ng, who had taken his doctorate there under Aczél in 1971, asked in a paper of 1974 which entropies deserve to be called natural, and proved, by their account, that the entropies with a short list of natural properties are exactly the linear combinations of Shannon's and Hartley's, where Hartley's is the log of the count, so even for merges the choice between the average and the worst case is a real choice. Both price merges. Neither prices raises, so neither picks out the toll, and that's the three-state example again, at a larger size. On the other side of the fence, Luciano Pomatto, Philipp Strack and Omer Tamuz (2023), economists at Caltech and Yale who asked what learning something should cost, proved, by their account, a uniqueness result for what it costs to *acquire* information, with constant marginal costs. That result

prices producing a signal, which is a different matter from what Section 5 reads off a copy onto a blank page; the copy owes no floor and borrows fresh states.

Most of that isn't a machine. The theorems of Parts I and II quantify over "any states, any steps," and the five models are small copies of somebody else's systems. The example machine is the small one of Section 15, the least I could make that still shows the whole model, with every part named and every price written down, and the program that runs every program of it and, with one paid move added, every Thiele machine whose workings a program can compute, is in Appendix C. That's where everything above has something to bump into. If a statement up there doesn't mean what I think it means, that's where it shows, and Part IV is how you'd go looking.

CHAPTER 19

The counter against physics

Section 3 did the physics argument once: on a finite machine a permanent flag merges states, a price list that charges merges (a named premise, with Landauer's principle, another named premise, as the physics behind it) charges the flag, and the toll follows. I'm not going to do it again. This section is about what a machine's counter, the small machine's μ included, has to do with any of that. The counter is a count in the machine's own unit. Heat is a separate premise, and here is exactly where the two touch and where they don't.

19.1 The premises have to be able to hold together

The one piece of this section that I think is good physics discipline is a consistency test. Suppose you want to claim the counter tracks dissipation. You need two premises: that every step you've chosen to count dissipates at least one unit, and an upper calibration that bounds dissipation by the charge. Can those two hold at once? There's a theorem saying exactly when: a model satisfying both exists precisely when every operation you've chosen to count (the statement below calls those *eligible*) costs at least one. I should say how little that shows. The model the proof builds takes the dissipation to be the cost itself, so the theorem shows the two premises don't contradict each other. It doesn't show that the counter tracks any dissipation. Still, it does bite on a schedule: if you count a zero-cost step, no such model can exist. One example where the premises hold is a one-bit restore, where restoring the bit to one costs one and idling costs nothing; the small machine's flag-raising steps, below, are another.

Now point the test at the small machine of Section 15, counting every step that collapses some yes-or-no property, and it fails. Read without the last clause of the floor, it would fail at a step that forgets nothing: INC A raises "$A \geq 1$" one way, is one-to-one, and costs zero. With the clause in place the failure sits at a merge. Its unpriced instructions merge states (the last item of the list at the end of that chapter has the pair: two cores that one DEC sends to the same core, at zero cost), so DEC A 2, which collapses the property "A is zero or the line number is 2" by sending a state where it's false and a state where it's true to the same state, costs zero there,

and (1) below says no dissipation can charge it one and stay under its cost. The machine prices the certifying chain and leaves its other merges unpriced, so the premise pair holds for the steps the toll is about and fails for the whole instruction set. That follows from computing at no cost on a base that keeps no history, and the base is the part that's by design. The hardware pays for those unpriced merges in heat, off the ledger. The ledger counts the record only. A unit of cost is not a joule.

Theorem 19.1 (When the premises can hold together)**.** (1) For any set of operations, any predicate "eligible" on them, and any cost, a dissipation with $\delta(i) \geq 1$ on every eligible i and $\delta(i) \leq \text{cost}(i)$ on every i exists if and only if every eligible operation costs at least one. (2) On a one-bit machine with a reset of cost 1 and an idle of cost 0, taking "eligible" to mean "sends two different states to one", such a dissipation exists. □

Proof. (1) If such a δ exists, $1 \leq \delta(i) \leq \text{cost}(i)$. Conversely, take $\delta = \text{cost}$. (2) The reset is the only operation that sends two states to one, and it costs 1; apply (1). ■

Three statements are in play, and each points its own way. Energy at a scale, $\mu \cdot c$, is a conversion and points nowhere. The calibration, $\delta(i) \leq \text{cost}(i)$, puts the charge above the dissipation: an upper bound. Landauer's principle puts the heat above an entropy loss: a lower bound. The theorem above says the assumptions can hold together, which is compatibility, and it isn't evidence. It doesn't make the ledger the least energy anything needs, and it doesn't make it the heat anything gives off.

Now point the first way out of Section 3, keep everything and pay by taking fresh states, at a machine. Keep the previous state in a history list (push it onto the front each step; below, $s :: h$ is the list h with s put on the front) and every fixed-instruction step becomes injective, while the step you can see stays the same; the line number can collapse to one place while the bigger state keeps its past. The unpriced merge, lifted to a machine with history, isn't a merge. It still pays, though: the list gets one state longer every step, so the run never comes back to a state it has been in, and that's the fresh-state column of what keeping takes. On any memory you can build the list fills it, and then the merge is back. So "this step merged, so it dissipated" can't be said of the visible step alone. It's a sentence about the whole state including whatever history is retained, and a physical model has to say what's retained. The permanent-certificate theorem in Section 3 supplies that for the certifying step on finite memory. For the rest of a machine's steps it's unsaid.

Theorem 19.2 (Keeping history makes every step injective)**.** Let the history step send a pair (s, h), a state and a list of past states, to $(\text{apply}(s, i), s :: h)$. For any step function, the history step with a fixed instruction is injective, and its first component is the original step. Each step makes the list one longer, so a run of history steps never visits the same pair twice.

Proof. Undoing the history step, by popping the list, returns the old pair, so two pairs with the same image are equal. The list after n steps has n more entries than at the start, so two pairs at different times of one run differ in the length of their lists. ∎

19.2 Permanence and the heat premise

Is there a heat prediction the permanent-certificate theorems make that Landauer's principle alone doesn't? No, and the reason is short. The smallest case, one certified state and one flipping in, is "set a bit to one whatever it held," which is the reset Landauer described and Bérut measured; Landauer's own name for it is "restore to one," and Section 2 put it beside the toll. The general bound, the larger of kT ln((m plus k) over m) and 2k over (m plus k) times kT ln 2, for squeezing m plus k equally likely states into m, is what Landauer's principle gives for that squeeze, at kT ln 2 for each bit removed. What the permanence theorems add is the structural half: finiteness and a permanent reading force any flag-raising instruction to merge some full states. They don't say the newly certified state is one of the states that collide, and they don't supply the spread over inputs or the physical conditions a heat bound needs. Supply those and Landauer's premise prices the uncertainty that was actually removed. The machine result and the physical premise do different jobs, and neither does the other's.

19.3 At the bottom: what I suspect and can't prove

This is underneath the proofs, so read it with the lights on. None of it is a theorem, and I'm not going to dress it up as one. It's why I care, and I'm putting it here, after the main text's proofs and apart from them, so you can check that none of it got into them.

Physics has a word for the moment a question closes: *measurement*. And the people who thought hardest about it keep describing it in this book's shape. They ran together for me until I put them on one picture. Flip a coin in a crowded room. Before it lands, it could go either way. After, it's settled, somebody chalks it on a

board, and the result gets copied out into the world, the light and the air, so many times that it can't be taken back and everybody who looks agrees. There are five pieces: the question, the landing, the chalk mark, the copies, the agreeing. Each person leans on a different one. Zurek, in Section 18, leans on the copies. Rovelli's coin has to hit the ground, so his piece is the landing.

Niels Bohr, the Danish physicist whose institute in Copenhagen was where quantum mechanics got argued into shape, leans on the mark. In a 1954 talk he said seeing anything at that scale depends on an apparatus that works irreversibly, like permanent marks on a photographic plate: a door that swings one way. I found that line quoted in a recent paper and haven't seen the page in Bohr's book myself. John Wheeler, the Princeton physicist who made the name "black hole" stick, went further in a 1989 talk in Tokyo and leaned on the question: every physical thing gets its existence from yes-or-no questions answered and registered, boiled down to "it from bit." As I read him, the coin has no face until something asks and something writes the answer down. How literally he meant that, I can't tell. And Peirce, way before any of them, leans on the agreeing: the kind of settling worth trusting is the kind that has to answer to everybody. Laid out like that, they pair up. Bohr and Rovelli lean on a step that only goes one way, in the plate and in the coin. Zurek and Peirce lean on many onlookers who agree, one with light and one with people.

Lay that next to everything in this book and the parts line up: a check that could have come out either way, a result that's now settled, a price paid in forgetting or in fresh states, a record that stays. I suspect that isn't a coincidence. I suspect the event this book prices on a machine is the same kind of event physics calls a measurement, showing up in a different place. The forgetting is my step. Bohr only says irreversible, and getting from a record that stays to a merge or to fresh states is Part I's counting of states. Push on it and it gives at Zurek, for the reason Section 18 gave: copying onto fresh light merges nothing and uses up fresh states, which is Bennett's measurement that owes no floor. So physics' own word lands on both sides of the line between Bennett and Rovelli: the chalk on the paying side, the copies on the borrowing side. The landing I'd put with the chalk, and when Section 5 pushes on Rovelli's paper it slides toward the borrowing side, the side of finding out. So two pieces already stick out, and I'll leave them sticking out. Which picture a settling on a machine matches is part of the suspicion, and the suspicion doesn't settle it. That's as far as I'll take it here: a suspicion.

John Bell asks for precision in those words. He was the Northern Irish physicist at CERN whose 1964 theorem is behind the CHSH game. In 1990 Bell wrote a short, furious essay against the word "measurement," with a list of words he said have no place in a physics that wants to be precise. On the list: system, apparatus, environment, irreversible, information, and measurement, which he called the worst of all. I used half of them in the last few paragraphs. To turn that suspicion

into a physical claim, each of those words would need a specified meaning and a connection to the device. The proofs here work with the models and machines I have defined; the physical applications state the further premises they use.

PART IV

Results together



Each opening leaves something out.

Part III followed one example machine from its first step to the host running a guest. Bring those pieces back to the bench. The holding count says what the arrows must do. The price rule says what goes on the ledger. The window says which of those distinctions a reader receives. The earned chain says how the flag got there.

CHAPTER 20

The claims and their conditions

Start with a box holding an answer. I need to know which full states count as holding it, which moves keep it there, and whether the box has finitely many such states. Those choices make the counting argument possible. Add a price on merging and the same count gives a charge. Give a reader only the tabletop picture and there is another question: how much of that event reaches the reader?

Here I put the connections in one place. The statements and their proofs stay in the chapters where I built them; this is a way to follow what each piece contributes to the next.

20.1 Keeping a distinction

One next state. Follow one deterministic rule from a start. Until the first repeated state, every visit is new. At the first repeat, either you have returned to the start, or two different predecessors have reached the same state. From then on the rule repeats the same successors. Those are the three columns of the step law (Section 1). A choice between several successors would change the argument at precisely the point where you follow the next arrow.

A finite place to hold the answer. Suppose a move keeps every YES state at YES and brings a NO state in. With K YES states, those states and the entrant give $K + 1$ sources landing in K places. Two arrows meet. The entering arrow need not be one of them: a chain can run through the held states before meeting a fixed endpoint. Section 3 gives the general count and its attaining constructions.

Each part of that picture matters. An endless strip can keep moving to a different YES state forever. A two-state toggle keeps the arrows one-to-one by turning YES back to NO. A finite permanent holder has neither route available. For a single rule, if its entered holding run were to repeat while remaining one-to-one, cancelling the equal successors backwards would identify the unfinished entrant with a held state. That is why the one-to-one alternative needs a fresh configuration at every later tick.

Section 1 also follows finite noisy and unitary systems using their own preservation conditions. For a table of chances, both row and column sums enter the count. For a unitary map, preservation of inner products supplies it. For the zoom argument,

keep track of the full states behind the coarse picture. The finite collection being counted must remain closed under the step; otherwise the next tick can move into a state the count left out.

20.2 Putting a price beside the arrows

A graph tells you which states meet. A price schedule tells you what a move costs. The finite holding count and a rule charging merging moves together force a charge on a permanent raise. Once the one-step toll is part of the machine, every run from NO to YES includes a charged crossing; adding nonnegative costs gives the universal floor (Section 2).

There are two useful schedules to keep separate. Charge by instruction name and repeating that instruction has the same price even after the flag is up. Charge by the source state and the schedule can ask whether this particular step raises the flag. The floor together with a ceiling of one unit per raise selects the latter charge, one for a raise and zero otherwise. When the same instruction performs the raise and the later idle, this state-sensitive policy lets the raise pay once and the idle stay free. The first is the cost signature in the basic definition; the second is the broader interface used in the source-charge and exact-pricing comparisons.

The three-card example makes the choice visible. Call the states blank, once and twice; once and twice read YES. A move sending once to twice and fixing blank and twice merges two states without raising the reading. A merge price charges it. A raise price leaves it free. The chosen reading and the chosen schedule specify which question the ledger answers.

With a halving allowance on each move, the list of possible starts can shrink only as far as the accumulated allowance permits. The entropy form additionally counts a supplied probability distribution. A physical heat reading uses Landauer's premise for the device and process being described. These are the successive inputs to the arguments in Sections 3 and 5.

The four-state demon shows the distinction between learning and wiping. MEASURE copies onto a blank page reversibly; applied again it can return that page. WIPE sends different page contents to the same blank state. The first operation uses the prepared page; the second merges the alternatives. The price rule can therefore charge them differently.

20.3 What reaches the reader

Put a raising step and a quiet step behind a window that shows the same before-and-after pictures for both. Any price computed only from those pictures gives the

same answer for both. It cannot both charge the raise and leave the quiet step free. Let the window show the raise and that ambiguity disappears (Section 8).

The same argument applies to a question about an earlier value. If a good and a bad value leave the same complete retained observation, a reader of that observation has one input for two required answers. Inspect the value while it is live and retain an adequate answer, or retain the distinctions the later questions will need. Which questions are allowed determines how much must survive.

A permanent base-driven record has the latch form: it switches on at the first occurrence of an event and stays on (Section 11). A toggle changes that task by allowing the record to fall; a hidden clock changes what drives it. For a record anywhere in an order, the readings “at least this place” give the row of cups in Section 4, and the toll over the whole order is the toll on each cup. A chain with k cups can have at most $k + 1$ different arrangements if cups only turn up.

Entitlement adds a particular claim and the evidence permitting a later step to use it. Its index count uses a covering tree whose depth the paid steps cover. The uncovered narrowing example in Section 9 shows why that connection belongs in the statement: narrowing a candidate list alone supplies no such bound on the ledger.

20.4 The concrete machine and its guests

From a clean start, only CERTIFY raises the flag, only a successful COMMIT supplies its channel, and only a successful CHECK supplies the matching fact. The counter’s edition must still match at commitment. Following those entry points backwards gives the earned chain and its minimum charge of three (Section 15). A claim such as “this counter is at least zero” passes everywhere and still uses the same three acts. The flag records their occurrence; the claim says what they established.

The ledger starts at zero and each step adds its scheduled price. Induction on the run then gives both its total and its uniqueness: two ledgers following that same start and schedule agree before the first step and still agree after each addition (Section 7).

The universal program follows a guest through matching points, preserving its halting, flag and ledger guarantees (Appendix C.1). The presented-machine interpreter extends this to guests with computable transitions, prices and readings, with the surcharge stated in Appendix C.2. Each interpreter earns its own flag through the host’s instructions.

Independent components compose by interleaving their moves. Each move changes its own component, so the pair’s cost is the sum of the two component costs. Keep each component’s own move order fixed and every interleaving has the same

final state. Nesting instead follows a guest through a host map; the record-order condition carries the guest's crossings into the host's accounting (Appendix D).

CHAPTER 21

The claim ledger

This section lists everything this book claims, one line each, with what kind of claim it is and where it lives. “Proved” means proved under the premises printed in that section, and nothing more. If a line here ever says more than its section does, the section is right and the line is wrong. “Directly from the rules” means the claim can be read off the stated operations by checking their cases. “Directly from the interface” means its requirements already include the claimed behavior. These claims are true, but carry almost nothing beyond what those definitions already say. If you’re counting results, count those differently. I mark them so they don’t pad the list, and where they’re stated in the book they carry the label *observation*, with two exceptions: Theorem 15.6 and part (3) of Theorem C.8.

Ledger is a good word for it, and it’s older than bookkeeping. Around 1400 a *ledger* was a book that lay permanently in one place, a big church breviary, say, from the old verb *leggen*, to lay; the account-book sense turns up in the 1500s. Staying put is the job here too. This is the one place where every claim lies still, so you can come back and find it where you left it.

21.1 Claims about the model

The model comes first. Every line in this table is about any system that has the pieces the line names: a state and a step, plus whatever the result asks for, a reading and the toll, a counter and a price list, a window, programs. None of them is about my machine, except the lines that run these results on it, which say so. The table runs over several pages, in reading order, except that a line running a result on the small machine sits beside the result it runs.

There’s a way to use it that takes five minutes. Pick the sentence in this book you trust least. Find its line. Read the Kind column before anything else. Then go to the home section and check that the line says no more than the section does. If it says more, you’ve found my mistake, and the end of this section says what I’d like done about it.

Claim	Kind	Home
Every run of a machine with one next state for every state takes fresh states until its first return, which lands on its start or is a merge, and after it only repeats steps already taken, so it merges at most once; the only steps that spend neither a fresh state nor a merge are on a loop, which makes nothing new	proved	Section 1, Theorem 1.1
A step that may have no next state can raise a permanent reading with no merge; with each halted state stepping to itself, any state that steps onto a halted state shares that landing with it, a merge	proved	Section 1, Theorem 1.2

Claim	Kind	Home
Writing a 0 on a blank cell, with the write defined on written cells too, is a merge of blank with zero; a remembering state added to the cell still merges; the endless tape and the bilateral shift avoid the merge by taking a fresh state every step; one Thiele machine runs computably presented Thiele machines as guests, paying at least one unit for each rise of a guest's record, and no level codes all its own readings	proved	Section 1, Theorems 1.4 and 1.5; the guests, Theorems C.7 and D.4
A reversible step on finitely many states returns every run to its start and keeps nothing	proved	Section 1, Theorem 1.6
A noisy step that preserves the even spread cannot raise a permanent reading; one that can raise such a reading strictly concentrates the even spread, raising its collision weight and leaving its Shannon entropy below the maximum	proved	Section 1, Theorem 1.7

Claim	Kind	Home
A finite-dimensional unitary that keeps a subspace keeps its complement, so no unitary step raises a permanent reading in finite dimension	proved; finite dimension is used once, for “onto”	Section 1, Theorem 1.8
A biased walk hands its surroundings strictly positive entropy per tick at any nonzero speed, and heat times time is at least the distance squared	proved; Schnakenberg’s formula taken as the model	Section 1, Theorem 1.9
The measured floor, the measured time-heat rate, the measured recurrence and the collapse-heat searches	cited, not reproved; the bead’s erasure law reproduced by simulation	Section 1
Every scale pays the same way: when a coarse description is exact on a set of fine states and its rule merges, the fine rule merges there, steps out of that set, or the set is endless	proved	Section 1, Theorem 1.10
Room to hold an answer: a rule one-to-one on the states it has used can hold a newly entered answer in a region of K states for at most K steps before two states share a successor; with endless room and a one-to-one rule the orbit never repeats	proved	Section 1, Theorem 1.3

Claim	Kind	Home
Thiele machine: states, moves, step, cost, and a record in any order; the two-place record is the smallest case	definition	Section 2, Definitions 2.1 and 2.2
The toll: a move that climbs the order costs at least one	the one rule of the definition; derived from three premises in the next chapter	Section 2, Definition 2.1
The floor over any order: a run that ends above where it started costs at least one	proved (premise: the toll), directly from the rules	Section 2, Observation 2.1
The universal floor: no to yes costs at least one	proved (premise: the toll), directly from the rules	Section 2, Observation 2.2
One unit per crossing: a run costs at least its number of steps from no to yes, the converse gives the toll, and the count is reached	proved (premise: the toll), directly from the rules	Section 2, Observation 2.7
With a price per move name, no system whose flag rises keeps every run at most its number of raises and still obeys the toll	proved	Section 2
The floor speaks about the world, or computing power	not claimed	Section 2
A computation is in scope just by being one	not claimed; someone has to supply the five parts and prove the toll	Section 2

Claim	Kind	Home
Charging every question stops free changes: one price list meets the toll for every Boolean reading exactly when every free move leaves every full state unchanged	proved (premise: equality of states decidable)	Section 2, Theorem 2.3
Free moves leave a permanent reading alone	proved (premises: the toll, permanence)	Section 2, Theorem 2.4
What the tabletop can tell: horizontal moves follow the view's own rule and keep the record, so no function of the view recovers the record on a pair that a vertical move supplies	proved (the sliced interface)	Section 2, Theorem 2.5
The least instruction price: one on each move that raises the reading somewhere and zero elsewhere is the least schedule meeting the toll	proved (premise: a finite list covering the states)	Section 2, Theorem 2.6
An erased distinction cannot answer its old question; read before the loss, the enriched observation can	proved	Section 2, Theorem 2.8
On a finite machine, a permanent flip merges	definition; proved (no price premise)	Section 3, Definition 3.1, Theorem 3.1
Merge pricing	assumed (premise)	Section 3, Premise 3.2
Finite, permanent and merge pricing give the toll	proved (premise: merge pricing), directly from the rules	Section 3, Observation 3.3

Claim	**Kind**	**Home**
On any machine, a raise by a move that never turns the reading off and merges nothing sends the yes-states one-to-one into a part of themselves that leaves one out (Hilbert's hotel), so it needs infinitely many yes-states; with finitely many, that raise merges	proved (premise: the move never turns the reading off; nothing else beyond telling states apart)	Section 3, Proposition 3.4
Anything kept is paid for in a merge or in fresh states (finite-time dissipation bounds for specified thermal stochastic models are cited physics, proved for the walker): any run of one rule that raises a record and keeps it either merges two of its own states or is in a new state at every step, so on M states a non-merging run that raises it has fewer than M steps, and a bounded memory only delays the merge	proved; Bennett's trade-off cited, not reproved	Section 3, Proposition 3.5
Retention bound $M \geq 1 + (T+1)(N-1)$, exact, and $M \geq N + T(N-L)$ with L fixed places	proved; the fibre form by the same count, on the page	Section 3, Proposition 3.5
On a finite machine, a flip merges or revokes	proved	Section 3, Theorem 3.7

Claim	Kind	Home
Forced to be priced exactly when it merges	definition; proved	Section 3, Definition 3.3, Theorem 3.8
On a finite machine, permanent records are forced; the converse fails	proved (the three cards)	Section 3, Corollary 3.9
Each premise of the toll has a machine that walks past it	proved (the strip, which pays by taking states it has never used; the coin; the stamp); certifying at zero cost on the ledger is proved too, and on the strip it means the price list charges nothing for fresh states	Section 3
Landauer's principle is true of the world	not claimed; a named premise, with its critics named	Section 3
Halving price	assumed (premise)	Section 3, Premise 3.4
The squeeze bound	proved (premise: halving price)	Section 3, Theorem 3.10
The halving price holds exactly when no pile-up exceeds 2^{cost}; the eight-state price list is the least that meets it; the small machine is not halving-priced	proved (finite machines; Landauer's price premise stays named)	Section 3

Claim	Kind	Home
Forgetting nothing removes no bits	proved	Section 3, Theorem 3.11
A permanent flip from the even spread removes at least $\log_2 \frac{m+k}{m}$ bits and at least $\frac{2k}{m+k}$ bits	proved	Section 3, Theorem 3.12
Landauer's principle	assumed (premise)	Section 3, Premise 3.5
Heat floor of a permanent flip	proved (premises: Landauer, $k_B T \geq 0$, the even spread)	Section 3, Corollary 3.13

Claim	Kind	Home
In the worst case over spreads a permanent flip removes at least a full bit, whatever the number of yes-states; from the even spread it removes at least $\log_2 \frac{m+k}{m}$ and at least $\frac{2k}{m+k}$ bits	proved	Section 3
Positive heat for any full spread	proved (premises: Landauer, $k_B T > 0$)	Section 3, Corollary 3.14
Entropy pricing gives the toll	definition; proved (premise: entropy pricing)	Section 3, Definition 3.6, Theorem 3.15
Entropy pricing is equivalent to the halving price; merge pricing is their weaker restriction to pairs. The largest entropy loss is $\log_2$ of the largest pile-up	proved	Section 3, Theorem 3.16
On a known state the premise sets no positive heat floor; the distinction was kept by whoever knows the state	proved; the second half is a reading	Section 3, Theorem 3.17
The bits removed, one term per state	proved	Section 3, Theorem 3.18
Any machine worth building is of this kind	not claimed	Section 3
An eight-state machine meets every premise, so the toll holds on it outright	proved	Section 3

Claim	Kind	Home
On the small machine CERTIFY, COMMIT and CHECK merge, every merge-priced cost obeys the toll, and record-layer merge pricing holds for its own prices and gives the toll, with no finiteness premise	definition; proved (premise: record-layer merge pricing, for the toll)	Section 3, Premise 3.7, Theorem 3.19
Permanence is the job to pick	choice	Section 3
The alternatives balance exactly: the losses along T steps plus the endpoints equal the number of starts, and endpoints in fewer states than starts force an identification	proved	Section 3, Theorem 3.2
The exact room for exclusive resets: $T+3$ states, $2T+2$ with commuting hidden levers, $2T+3$ with levers one-to-one everywhere	proved	Section 3, Proposition 3.6
In a partial order the cups fix the place; in a preorder they need not	proved	Section 4, Theorem 4.1

Claim	Kind	Home
The toll over any direction: every step that turns a cup over costs at least one exactly when every run that ends somewhere not at or below its start does, and exactly when every cup obeys the toll on its own; a run then costs at least its number of such steps	proved	Section 4, Theorem 4.2
A chain of $n + 1$ stages needs n write-once Boolean cups	proved; ordinary rewritable binary storage is a different question	Section 4, Theorem 4.3
On a finite machine, a step that turns a cup over merges or revokes; with a growing record it merges, and merge pricing gives the toll	proved	Section 4, Theorem 4.4
Finiteness, growth and merge pricing each do work for the direction toll	proved (three machines)	Section 4, Theorem 4.5
The machine's own narrowing is priced	proved (premise: halving price)	Section 5, Theorem 5.1
An observer can learn with nothing merged: on the four-state demon, a run of cost zero under a price list meeting the halving price narrows two candidates to one, and WIPE costs at least one	definition; proved (one price list that allows it)	Section 5, Definition 5.1, Theorem 5.2

Claim	Kind	Home
Learning borrows: the copy uses up a blank page and can be handed back (a second MEASURE clears it), and the page is paid for when it's wiped	proved on the four-state demon; the wipe merges	Section 5
Learning costs nothing on every price list, or takes no fresh state	not claimed; a price list may charge it, and it uses up a blank page	Section 5

Claim	Kind	Home
Three states are the minimum for teaching at zero cost after the first look	proved	Section 5, Theorem 5.3
The certainty bound: the uncertainty about a start splits into the part the run destroyed, at most its cost in bits, and the part the end state holds; whatever the end state can't tell you about what an observer saw was paid for in units of cost	proved (premise: entropy pricing)	Section 5, Theorem 5.5
"No Free Insight," the toll's first name, names a distinction made to stay	choice (the name kept, the scope stated)	Section 5
Watching narrows like a quantum measurement	not claimed	Section 5

Claim	**Kind**	**Home**
Room for synchronized first completion: N starts that complete at the same tick T, under a rule one-to-one on each tick's states, need $N(T+1)$ configurations	proved	Section 5, Theorem 5.4
Structure: relations, kept under chosen changes, carried by any stuff, relative to a frame	choice (my overlay of the sources; they never proved it together)	Section 6
Turing's four parts have no slot for "these belong together"	not claimed; I looked and didn't find one, which is weaker	Section 6
Equivalence relation, partition	definition	Section 6, Definition 6.1
Sameness and sorting are one object	proved (textbook, Halmos)	Section 6, Theorem 6.1
Preorder, symmetric part, strict part, cut	definition	Section 6, Definition 6.2
An order is a sorting with doors: a preorder is exactly a partition with a one-way order on its blocks, a cut has two buckets and one door, and the record's order pulled back to the states is a preorder whose buckets the toll ignores and whose climbs it charges	proved	Section 6, Theorem 6.2

Claim	Kind	Home
A cut, a sorting with one door, is the start, smallest by the count of names and by the count of rules; a partition is its symmetric case	choice, with both rulers stated	Section 6
Nobody prices vouching for a sorting	not claimed; only that I found no one who does	Section 6
A counter kept by a price list	definition	Section 7, Definition 7.1
Conservation	proved	Section 7, Theorem 7.2
Only one honest count	proved	Section 7, Theorem 7.3
The fold on lists is unique; it reaches states only under a condition	proved	Section 7
Initiality picks the prices, or makes units of cost physical	not claimed	Section 7
The floor and a ceiling (a run never costs more than its raises) leave exactly one cost function, one unit per raise and nothing else, when the price may depend on where the step starts; the small machine is not priced that way, by design	proved	Section 7
The toll is the potential method	definition; proved	Section 7, Definition 7.2, Theorem 7.4
The potential method is new here	not claimed (it's Tarjan's)	Section 7
The counter is the cost-semantics frameworks' writer	proved for the writer: a run in the writer over whole numbers returns the final state and the total cost	Section 7

Claim	Kind	Home
The counter embeds the cost-semantics frameworks' type systems and their before-the-run checking of grades	not claimed; by design the grade is written by the step during the run	Section 7
A window that forgets the counter can't recover it on every state; on reached states it takes two runs with one view and two counters	proved (premise: the counter can be changed on its own); the premise is needed	Section 7
Receipts covering the observed mergers: with a source charge positive on one member of every observed colliding pair, the ledger's rise bounds the number of observed mergers	proved	Section 7, Theorem 7.1
A decoder needs constant fibres	proved (any types)	Section 8, Theorem 8.1
No exact window price across a collision	proved (premise: a collision), directly from the rules	Section 8, Observation 8.2

Claim	Kind	Home
On the toy, no verifier on bare transcripts is sound and complete, and none that reads only the bare part of a richer transcript is either, for any pair like it	proved, over arbitrary types	Section 8

Claim	Kind	Home
Full-state, commitment and response verifiers are sound and complete for their explicit explanation relations; commitment is sound and complete exactly when binding and honesty hold	proved; verification cost is stipulated, and no proof reads it	Section 8, Definition 8.1, Theorem 8.3
On the small machine, no verifier on bare transcripts is sound and complete for the record, or for a ledger rise of 3 from a clean loaded state	proved	Section 8, Theorem 8.4
The one-round "ask" models a cheating prover	not claimed	Section 8
The window fact is mysticism, or Rice	not claimed	Section 8
Entitlement: object, checked relation, paid event	definition (the book's word)	Section 9, Definition 9.1
Structural entitlement over any Thiele machine: a strictly stronger test, a paid crossing, a cost that covers the index bits	proved (premises: covering tree, paid steps)	Section 9, Theorem 9.1
A reading that a grouping was checked and committed sees only its partition and entitles, uniformly over keys, exactly the groupings that refine it	proved	Section 9, Theorem 9.2
Entitlement is truth, or permission	not claimed	Section 9

Claim	Kind	Home
The count prices why candidates were dropped	not claimed; it prices the size of the drop	Section 9
Raise, non-raising move	definition	Section 10, Definition 10.1
The toll prices only the crossing, flat in depth	fact read off the definitions	Section 10, Remark 10.1
Sorting moves by whether the reading moves	choice	Section 10
Nothing makes the crossing wait for a check	fact read off the definitions	Section 10, Remark 10.2
A paid flag proves its claim true	not claimed	Section 10

Claim	Kind	Home
Base, cover, base-driven record, latch	definition	Section 11, Definition 11.1, Definition 11.2
Over any base, a base-driven record is a latch	proved, directly from the rules	Section 11, Observation 11.1
Every reachable latch is a base-driven record	proved	Section 11, Theorem 11.2
A finite reversible machine cannot raise a permanent reading	proved; an unbounded-history extension can, by taking fresh states	Section 11, Theorem 11.3
Two records are two latches	proved	Section 11, Lemma 11.4
The toggle: driven, can switch off, no latch	proved	Section 11, Lemma 11.5
The timestamp, a record switched on by a hidden clock: permanent, undriven	proved	Section 11, Lemma 11.6

Claim	Kind	Home
The timestamp is undriven on the states its runs reach, while the timestamp started in sync is a latch there and is rejected only by (R1) over every state	proved	Section 11, Lemma 11.7
Base-driven records agree up to the schedule	proved	Section 11
The event is an unforced choice	proved, the converse applied twice	Section 11, Corollary 11.8
The schedule doesn't fix the probabilities	proved, with the branching latch refuted	Section 11, Theorem 11.9
One state space up to renaming	false: an extension may carry any hidden field	Section 11
Weak base equivalence connects those bases	not claimed	Section 11
A growing record is a family of latches	definition; proved, and one latch isn't enough	Section 11, Definition 11.3, Theorem 11.10
A window can price rises exactly without determining the record value	proved, under the collision and decidability conditions stated there	Section 12, Theorem 12.1
With an injective shadow step, merged states lie in one shadow fibre	proved; the shadow-injectivity premise is needed	Section 12, Theorem 12.2

Claim	Kind	Home
Substrate; shortcut predicate	definition	Section 13, Definition 13.1, Definition 13.2
No inside decider with a representable flip	proved (premise: the substrate's recursion theorem)	Section 13, Theorem 13.1
Undecidable means unknowable; the wall separates Turing degrees	not claimed	Section 13

Claim	Kind	Home
Casper setting; one storage slot; a gas schedule	definition (transcribed; the gas schedule's charge premise stipulated)	Section 17, Definition 17.1, Definition 17.2, Definition 17.3
Accountable safety	proved (premise: quorum intersection; the one parent the setting carries is never used)	Section 17, Theorem 17.1
The quorum premise can hold; on a one-validator chain one vote finalizes genesis and nobody is slashed	proved	Section 17, Theorem 17.2
A persistent write to a fresh slot nets at least 22100, and to a warm one at least 20000	proved, on the transcription	Section 17, Theorem 17.3
Gas is exact exactly when commits pay one	proved (premise: the stipulated charge predicate)	Section 17, Theorem 17.4
A permanent write on a finite machine merges, and the merge lands on the record's yes side	proved	Section 17, Theorem 17.5
The quote can't attest what it never measured	proved	Section 17, Theorem 17.6

Claim	Kind	Home
The RFC's inclusion examples pass	proved (symbolic hash)	Section 17, Theorem 17.7
A transcription is the real system	not claimed	Section 17
A toll is a better design than a wall	not claimed	Section 17
Certification is the event whose records are copied and relied on by parties who didn't produce them	thesis	Section 18, Thesis 18.1

Claim	Kind	Home
The formal objects prove the thesis	not claimed; “independent” is defined, “deployed” and “faithfully model” are defined nowhere	Section 18
The early engine derived physics	not claimed; it ran a simulation	Section 24
The floor raised to a threshold	proved (premises: the witness starts at zero, grows by at most the cost, and meets the threshold)	Appendix B, Theorem B.1

21.2 Claims about the machine

Part III built one small machine, with the hosts that run it in an appendix, and the ledger keeps them apart from the model. The machine is the small one of Section 15, two counters and six instruction kinds, where commitments are earned and universality is proved, with the universal program U on a host made of the same instruction kinds, and the priced program U_P that runs every computably presented Thiele machine. Every line in the next tables is about those machines, or about the physics section that closes Part III, or about the hosts and the machines built from parts in the appendices, and nothing else, under the premises printed in its section.

Claim	Kind	Home
Checker and arithmetic meaning agree	proved on the small machine, under the stated premises	Section 15, Lemma 15.2
Two-counter simulation and halting correspondence	proved on the small machine, under the stated premises	Section 15, Theorem 15.3 (step), Theorem 15.4 (halting)
Halting is undecidable by the two-counter reduction	proved on the small machine, under the stated premises	Section 15, Theorem 15.5
Exact ledger, permanent certification and toll	proved on the small machine, under the stated premises, directly from the rules	Section 15, Theorem 15.6
A Thiele machine	proved on the small machine, under the stated premises	Section 15, Theorem 15.7
Certification factors as a latch over the core base	proved on the small machine, under the stated premises	Section 15, Theorem 15.8

Claim	Kind	Home
A passing commitment has a prior matching check at the same version	proved on the small machine, under the stated premises	Section 15, Theorem 15.10
Certification has a check, commitment and certification in that order	proved on the small machine, under the stated premises	Section 15, Theorem 15.11
Unearned commitment and uncommitted certification trap	proved on the small machine, under the stated premises	Section 15, Theorem 15.12
Current-version checked claims are true	proved on the small machine, under the stated premises	Section 15, Theorem 15.13
Equal computational windows can hide different records	proved on the small machine, under the stated premises	Section 15, Theorem 15.15
The window cannot recover the ledger, flag or commitment permission	proved on the small machine, under the stated premises	Section 15, Theorem 15.16
From a clean start, a run ending with the flag up adds at least three to the ledger	proved on the small machine, under the stated premises, directly from the rules	Section 15, Observation 15.19
That minimum of three is reached by a run whose check could have failed, and when that check fails the program is refused forever	proved on the small machine, under the stated premises	Section 15, Theorem 15.20
The step refuses trivial checks	not claimed; “is at least 0” passes every value and certifies for three too	Section 15

Claim	Kind	Home
From a clean start, provenance, checker soundness, no forging and the floor of three hold for any property language with an exact equality test and a checker proved to say yes exactly when the claim holds, a sorted list read out of a counter included	proved on the generic machine	Section 15
With the flag up from a clean start, the channel names a fact an earlier passing COMMIT committed, and a later commit can replace it	proved on the small machine	Section 15
The guarantees hold against whoever sets the start, a reset, or a second writer that doesn't bump versions	not claimed; they are about programs from a clean start, one step at a time	Section 15, Section 22
A projection that hides every CHECK and gives each free instruction its own rule on the picture makes every free move stand still	proved on the small machine	Section 15, Theorem 15.1
On clean runs the flag is exactly the nonempty reading of the publication archive	proved on the small machine	Section 15, Theorem 15.9
No stored fact is forged on a clean-start trace	proved on the small machine, under the stated premises	Section 15, Theorem 15.14

Claim	Kind	Home
Same final counters and bill, different certification: two clean starts and one word end on $(0, 0)$ with ledger 3, one certificate true and the other false	proved on the small machine	Section 15, Theorem 15.17
Exactly 65,535 publication histories can lie above one reachable native endpoint; sixteen extra bits encode them and fifteen do not	proved on the small machine	Section 15, Theorem 15.18
CERTIFY merges and is priced; DEC A 2 merges and costs nothing, so not every merge of the small machine is priced	proved on the small machine	Section 15, Theorem 15.22
A counter beside a Turing machine changes nothing	proved	Section 15.10, Theorem 15.21
A Turing machine can't price a reading	not claimed	Section 15.10
The premises fit when every counted step costs at least one, and fit a one-bit reset	proved	Section 19, Theorem 19.1
The small machine's whole instruction set fails the premise pair	proved; its unpriced steps merge	Section 19
Keeping history makes every step injective; the history takes fresh states, one more entry every step	proved	Section 19, Theorem 19.2

Claim	Kind	Home
The canonical reset's rates aren't in detailed balance at any gap or temperature; with rates that are, a fixed-gap relaxation moves toward $1/(1+e^{\Delta/k_BT})$, which is $1/5$ at the gap $2k_BT\ln 2$	proved	Section 19
The counter is physical dissipation, in joules	not claimed	Section 19
Permanence predicts more than Landauer	not claimed	Section 19
The theorems prove physical reality	not claimed	Section 19
No fixed host program can keep the guest's facts on an exact copy of a guest counter	proved, for any host property language and any translation into a finite list of host properties	Appendix C.1, Theorem C.1
A universal Thiele machine for a class: one host and one program, any guest loaded into a start state, halting matched, the guest's states decoded in order, the host's flag the guest's reading latched, the host's ledger the guest's plus a surcharge, never negative and bounded for the class, and the flag raised only by the host's own moves under its own toll	definition (surcharge 0 for U; for U_P at most 2 when the guest's reading starts at no and exactly 3 when it starts at yes, so 3 over the class)	Appendix C.1, Definition C.1

Claim	Kind	Home
One fixed program U, in the small machine's own instruction kinds and with no guest step built in, runs every small-machine program from every clean start start(x, y), step for step, with halting and answers matched	proved on the host machine	Appendix C.1, Theorem C.2, Theorem C.3
U's flag rises at some step if and only if the guest's does, and its ledger equals the guest's at every matching point	proved on the host machine	Appendix C.1, Theorem C.4
U's raised flag stands on U's own check, commit and certificate on a slot, and on the guest's own earned chain for the same claim	proved on the host machine	Appendix C.1, Theorem C.5
U's host is certified, and whether U halts on a loaded guest is undecidable	proved on the host machine; undecidable in the reduction sense of Theorem 15.5	Appendix C.1, Theorem C.6
One fixed program U_P, U with one paid move added, runs every computably presented Thiele machine: halting matched, the state decoded at every matching point, the host's flag up exactly when the guest's reading has been yes, and the host's ledger the guest's plus a surcharge	proved on the priced host	Appendix C.2, Theorem C.7

Claim	Kind	Home
U_P's raised flag stands on its own check, commit and certificate on a slot whose claim is the guest's own reading of a state where it said yes	proved on the priced host	Appendix C.2, Theorem C.7
The surcharge is at most two when the reading starts at no, and zero when the first raising move costs at least three	proved	Appendix C.2, Theorem C.8
No program on the host machine, run from a clean start, exactly matches a guest certified with its whole ledger below three	proved, for every program from a clean start, directly from the rules	Appendix C.2, Theorem C.8
The priced host is certified, with the paid move read as a check that never passes	proved	Appendix C.2
A demonstration machine (one move of price three, reading "at least three") runs on U_P, with the host's ledger exactly $3n$ at the host time matching move n	proved; never run	Appendix C.2
A machine whose reading no program decides, or whose next move no program computes, runs on U_P	not claimed; outside the class	Appendix C.2
A guest's reading that goes back to no is carried as no	not claimed; the host's flag is the latch of the reading	Appendix C.2

Claim	Kind	Home
Products and sequential hand-offs preserve the stated structure; shared writers and unsuitable record readings can break it	proved under the stated interfaces and loading conditions	Appendix D, Theorems D.1–D.3
A following host run pays for the guest's record rises; presented nesting surcharges add	proved; every host run is presented on its states with finitely many live registers, so towers go to any depth	Appendix D, Theorem D.4

Claim	Kind	Home

21.3 What I propose

What ties all of those results together is a proposal, and it's mine: put the relations a computation carries, and what keeping them takes, into the state and the step. The items below spell that out, along with one narrower bet inside it, the pointer thesis about certification. Every one of them reaches past the theorems, and none of them adds to the physics, which stays in the premises the book names and the results it cites.

- That the axis belongs in the step (Section 22, Section 23).
- That the usual account of a computation is the flat case of the step, the shadow a window on the flat part leaves, for machines whose control never reads their record (Section 12). The proposal is that this is how to see computation in general. The small machine's control does read its own record: a commitment that finds no fact traps. So for it the claim is about its flat part, not the whole machine.
- Within the record construction, that certification is the event selected by the reliance criterion of the pointer thesis (Section 18).
- That this is a structural account of computation, with a cost model as one part of it (Section 22).
- That the toll is the right thing to price. The alternatives are a wall or a penalty for contradiction (Section 17, Section 23).

And there are three things this book doesn't settle, each with what would settle it. Whether Landauer's critics are right, which would leave every theorem standing and one premise unowned by the world: that takes measurements of real devices (Section 3). Whether certification really is the pointer event: that takes a way to tie the definitions to the systems the thesis is about, and Section 18 sets out the three comparisons needed. Whether there's a smaller start than a partition has its answer inside the definition, the two-rule order with the partition as its symmetric case (Section 6); whether anything sits below that takes a ruler someone can state and a smaller structure measured by it.

That's the whole ledger. If you find a claim in this book that isn't on it, or a line that says more than its home does, that's my mistake, and I'd want to hear about it.

CHAPTER 22

Scope

The definition asks for states, moves, readings and charges. To use a result elsewhere, put that other system beside the drawing and match the pieces. Which states carry the record? Which operation establishes it? What does the reader see, and what does the price count?

The following comparisons work through those questions for heat, computation and the choice of record. They show which further connections an application needs.

Physical interpretation of the counter. Choose a start and a price schedule and the ledger is determined on reachable states. To read it as a physical quantity, specify which states carry information and how the scheduled charge relates to dissipation, under physical premises that hold together. The permanent-certificate theorems supply the first for an abstract finite machine with a permanent reading. The two premises are shown able to hold together whenever every counted step costs at least one, and for a one-bit reset. The small machine's unpriced steps merge states and are charged nothing, so its ledger is not a heat meter for computing: whatever those merges cost the hardware is off the ledger, and the two premises hold together for the record steps only (Section 19). No joule value attaches to one unit of cost without thermal and device premises (Section 3, Section 19). Once you pick the start and the price list, the number is forced by arithmetic, and it isn't energy. What I'd need before it could be is the list at the top of this paragraph.

Class separation. The witnesses show specific windows losing specific fields. They don't stop a conventional simulator from keeping those fields in its encoding. No separation of standard complexity classes follows from the collisions (Section 8, Section 15). I'll be blunt about why I'm careful here. Whether P equals NP is one of the seven problems the Clay Mathematics Institute put a million dollars on, at a meeting in Paris in May 2000. One of the seven has been solved since, the Poincaré conjecture, by the mathematician Grigori Perelman, who posted the proof as preprints in 2002 and 2003. P versus NP hasn't. If a two-state collision on my machine settled it, I'd be owed a million dollars. Nobody owes me anything. A window that loses a field is a fact about that window. A complexity class is a fact

about what every possible program can do within a budget, and nothing in these collisions reaches that far.

Trace initiality, and what it doesn't settle. The universal property of the trace fold is about evaluating lists of instructions: pick where a target starts, and there's exactly one way to fold a list into it (Section 7). The counter's uniqueness assumes a schedule. Rise-only pricing assumes an event, and a reference count that says what charging too much means (Section 8). Those results make the construction precise. None of them shows that the schedule I chose, or the event I chose, is physically necessary.

Other observables. Certification is one worked example. A family of other observations could turn up other structural laws, but each would need its own events, its own evidence, and its own cost rules, justified on their own terms. The window and oracle results are about what specific maps keep, and nothing more.

Program predicates versus state observations. The state-level collision (Section 8's wall) and the program-level diagonal (Section 13's wall) live in different domains. Linking them would take a theorem of its own. The diagonal (Section 13) keeps its explicit quantifiers and representability premises.

Entitlement bounds. The count is handed its tree and its covering, along with the payment condition. It doesn't dig the tree out of an arbitrary run, and it isn't a Shannon bound on every trace (Section 9). On the small machine the payment is the record moves, each costing exactly one; a branch that costs nothing can still narrow, the lists, tree and fibres are supplied, and the covering can't be dropped.

The record axis. The proved structural result is: a permanent record driven by a base computation is a latch on an event; on a finite machine its write merges states; and the named windows don't recover it. Certification is one instance. No theorem selects it as the unique event every system must record (Section 11, Section 18). A finite monotone multi-valued record decomposes into threshold latches. One Boolean latch can't hold it. Revocation, including revocation priced from the other side as in Casper, lies outside the permanent-record axis; a price on revocation doesn't transfer to a price on the write (Section 17). Probabilistic records don't inherit the uniqueness result either. Two honest random records can have the same branches and the same charges and still give those branches different weights, one choosing evenly and one two to one, so the schedule doesn't fix the probabilities. And no single deterministic latch explains a record that can branch both ways from the same observation. Records that can come back down lie outside all of this: they're limit computability's, and Section 11 points at Ershov's ladder.

Who it's safe against. Every guarantee about the record is about programs: any list of instructions, run from a clean start on a step that does what its definition says, one writer stepping at a time. That covers the buggy program and the hostile one. It doesn't cover whoever sets the start (a snapshot put back, a table loaded with facts

nobody checked), a reset or a power cycle, the wire that carries the report, someone who flips the flag with a probe, the tools that built the step, or a second writer that changes a counter without bumping its edition. Those are the operator, the device and the tool chain, and they're outside this fence. A record read by someone who didn't watch it being made is only as good as their reason to believe the box that shows it. The ways across that I know of, a quote over measured state and a proof that travels on its own, are in Section 8 and Section 17, and this book builds neither. What the guarantee gives inside the fence is safety: no program forges the chain. Availability is a different promise, and it isn't given. A hostile program can put the trap up or fill the table, and after that nothing useful gets done. The mathematical machine uses whole numbers, which never overflow, so versions and the ledger grow without bound. A build with fixed-width counters, versions or ledger has to stop before any of them wraps, failing safe with the trap up, because a wrapped version could make an old fact look live again and a wrapped ledger would fall. Starting, restoring, resetting and that overflow stop belong to the implementation, and none of them is proved here.

The diagonal on the machine. What's proved for the small machine's programs is that halting is undecidable in the reduction sense of Theorem 15.5, for them and for the universal program (Section 15; for the universal program, Appendix C.1). The wall of Section 13 is stated over any substrate with a recursion theorem and a representable flip, and uses neither the counter nor the toll.

The five models. Each is a fragment transcribed from a specification, with a named theorem and a named limit: the Casper transcription has no deposits or fork choice; the gas fragment omits the refund cap; the attestation fragment has no signatures; the transparency verifier covers control flow and leaves collision resistance out; the proof-carrying checker is a three-instruction memory-bounds language. None is a claim about the real system beyond its fragment (Section 17).

The machine. Part III has one small machine, with the hosts that run it in an appendix, and this paragraph is its fence. The small machine of Section 15 (two counters, six instruction kinds) is where commitments are earned: from a clean start, a raised flag went through a passing check, a commitment and a certification of the same claim, nothing is forged, and the cheapest such run adds exactly three to the ledger, a price reached by a check that could have failed (the step doesn't police trivial checks: a check that can't fail certifies for three too). It's also where universality is proved, by matching Minsky's two-counter machines step for step and using the halting reduction of Forster, Larchey-Wendling and coauthors. It's certified, in the sense Section 14.15 pins down, and it hides its record and ledger from a window on its counters. The universal Thiele machine for the small machine's programs (Definition C.1) is a host with the small machine's instruction kinds and more counters, running one fixed program, U, with no guest step built in: it runs

every program of the small machine from every clean start start(x, y), halts when the guest halts, raises its own flag by its own check, commit and certificate if and only if the guest's flag rises somewhere in the guest's run, and keeps a ledger equal to the guest's at every matching point. With one paid move added, one fixed program, U_P, runs every computably presented Thiele machine: its step, prices and reading computed by programs, the reading decided on every state, the run fixed by a driver. The host halts when the guest stops, its flag is the latch of the guest's reading, raised on its own earned chain whose claim is that reading, and its ledger is the guest's plus a surcharge of at most two when the reading starts at no, a surcharge no program on that host, run from a clean start, can avoid when the guest certifies with its whole ledger below three. A machine whose reading no program decides is outside. The small machine's check price is flat, its fact table holds sixteen, and its certificate is historical (Section 15 lists the rest).

The comparison machines. The reversible result (Theorem 11.3) covers any base whose step is injective, and on a finite state space says only that no injective step raises a permanent reading. It is not a theorem about real memories or full reversible languages. The cost-semantics results are local correspondences, short of full cost-framework embeddings: the trace's cost is a writer cost and a grade, and the toll is the potential method with one potential (Section 7). Nobody's whole type system gets embedded either.

Outward consequences. I looked for results about real systems that would need the record axis. What's proved are the five fragments of Section 17, each a transcription with its own named limit, and none is a theorem about the deployed system. Stronger candidates would need protocol or hardware semantics I haven't formalised, and I've proved nothing about them. So none of this yields a new outside result that both needs the record axis and is ready to use.

Where each result lives is the premise column of the claims section (Section 21).

Models and processes. For a model of computation written down as states, a step and a reading that stays, what keeping takes is a theorem about the model, not a thesis. The thesis in the Conclusion is only the tie from a physical process to such a description. Here's how far the theorems reach, and where they stop. A step that gives every state one next state, on any number of states: a run that raises a reading and keeps it up merges two of its own states or never visits a state twice (Proposition 3.5), and with finitely many states reading yes the move that raises it merges (Theorem 3.1, Proposition 3.4). That takes in a Turing machine or a counter machine stepping by its program, and a cellular automaton stepping its whole grid at once. A model that halts has no next state at its halt, and slips past with no merge; give the halt a next state and the run is one rule again, so keeping the answer merges or takes fresh states, and with the halted state stepping to itself the step onto it is a merge (Theorem 1.2). Steps by chance on finitely many states: one

whose chances into each state add to one, which is what keeping the even spread takes, never enters a finite set nothing leaves from a state outside it (Theorem 1.7). A unitary step that keeps a finite-dimensional part of the space keeps the part at right angles to it, so a state at right angles to the kept part stays at right angles (Theorem 1.8).

A model whose step may offer several next states, or none, comes in too: a nondeterministic machine, a lambda term reduced at whichever redex you like, a rewriting system, a proof system read as derivations made one rule at a time. Nothing earlier in the book is stated for such steps, so here are the two arguments. The first is the pigeonhole of Proposition 3.4 on one choice of next state, and the second is the walk back of Proposition 3.5, stopped at the raise. First, suppose only finitely many states read yes, none of them can step to a state reading no, each has at least one next state, and some state reading no can step to one of them. Pick one next state for each yes-state, and the outside state's step into the yes-states: that's one more step than there are yes-states, each leaving a different state and landing on a yes-state, so two land on the same one, and two different states share a next state. That needs the yes-states finite; Hilbert's hotel, everybody moving up one, keeps a reading with no two states sharing a next state. Second, along any one run, however its choices fall, take the step where a reading goes up, and suppose it stays up for the rest of that run. If the run is in one state at two times after the raise, step both times back together. They can't stay equal past the raise, because the state at the raise reads no and every later one reads yes, so at the first pair that differs, two different states of that very run step onto one. So after the raise the run merges or never repeats a state, on any number of states. Before the raise it can come back to a state and then leave it for good with no merge, which one rule can't do. A normal form, with no next state, is the halt again: a reading kept by stopping is kept with no merge, as Theorem 1.2 says. For such steps the first argument says little by itself: in the lambda calculus every term N is one step from two different terms, $(\lambda x.x)N$ and $(\lambda z.N)M$ with z not free in N, so two states sharing a next state is no news there, and the second argument is the one that carries weight. And when a step can branch, a shared next state doesn't by itself bunch the crowd up, since a branch spreads it out again; with chances on the steps and finitely many states, the count that does is Theorem 1.7.

Where they stop. Steps in continuous time have no next state to count, and nothing here covers them. Steps by chance where endlessly many states read yes, and unitary steps whose kept part has endlessly many dimensions, get none of the finite results above, and the bilateral shift keeps a reading with no merge, a fresh state every step. What still reaches them is the run: a unitary step is one rule, so its runs are Proposition 3.5's, and one run of steps by chance is a run the second argument covers. And a truth isn't a process at all: that a statement is provable is a

fact with no step in it, and the theorems are about a run that derives it. What they give is the merge or the fresh states. The heat stays Landauer's premise.

What I'm proposing beyond that. A way of describing computation in which the relations a computation carries live in the state, what each move does to them lives in the step, and a price list is one part of the description, not an afterthought. The permanent record is the one case I took apart all the way. The pointer thesis asks whether certifying is the thing that deserves that record in the systems it talks about, and it can be wrong without touching the latch or the window arguments, which never use it.

Underneath that is a question about which distinctions a description of a computation ought to keep. The constructions show particular ways of keeping some, and what follows once you do, under premises I've printed. What that's worth depends on what you need the description to answer. A program log with a proof about its updates may recover the required distinctions. A host can instead enforce those updates through every guest program. A system may prevent an act with a wall, penalise it afterwards, or charge it at the step. Comparing those choices means following what each one guarantees for the task.

CHAPTER 23

Conclusion

Put the blank cell, the triangle, the cups and the two computers back on the table. The cell went from a specified blank state to a 0 and stayed under its write-once rule. The rule merges blank and zero; a heat floor still needs the input preparation and the thermal premises. Turning the triangle preserves relations you can check with your fingers. Turning a cup records an act; a row of cups can distinguish stages that one cup cannot. The two computers can leave the same answer while differing in what they did to reach it. In each case I had to say what the things were, which moves were allowed, which relations survived, and what my way of looking could tell apart.

Part I started at the bottom, with the cell. On any machine run by one rule, every step lands on a fresh state, merges two into one, or goes round a loop, and a run that raises a reading that stays either merges somewhere or never repeats a state, so it's paid in a merge or in fresh states, alongside the model-dependent thermal bounds, more the faster it goes. The counting needs nothing but one rule and a reading that stays. A single 0 written on a blank cell is already a merge, and every zoom pays the same way. Then the model got an exact form: five parts, one rule, one theorem you proved at the table, what keeping takes written into a step. Then the rule stopped being only my preference. With finitely many yes-states and a flag that stays up, the fresh states run out, every move that raises the flag merges, and under the merge-pricing rule the toll follows. The collision belongs to the instruction, over the states it's allowed to act on; the state that just came in needn't be one of the two that collide. Turning that into heat takes a spread over inputs and Landauer's premise on top, its critics named; from a start somebody already knows, the premise puts no floor under it, because that prepared distribution has no entropy to remove; the step can dissipate under the stated thermal-model assumptions. The toll is a derivation under three premises you can refuse one at a time, each with the machine that walks past it built; what keeping takes, underneath it, isn't. Learning borrows: copying onto a blank page merges nothing, the copy can be handed back, and the page is paid for when it's wiped. Committing makes a record that can't be handed

back, so it pays. That's as far as physics goes in this book, and I've tried to keep it exactly that far.

Part II gave the triangle and the cups their words: structure is relations among things, kept under the changes you choose, carried by whatever stuff happens to be there, relative to a frame. A cut, a sorting with a door that swings one way, turned out to be the smallest structure by the rulers I picked, a partition is the cut with its door taken off, and a cell that goes from blank to 0 and stays is that smallest cut, one yes-or-no question, kept. And making a grouping, keeping it, checking it and standing behind it came apart into four different acts. So the question became how an account of computation could carry what keeping the cell's 0 takes, and the difference between those acts, in its step.

The rest of Part II was the logic, and what it earned comes next, in the order it was earned. The counter is arithmetic once the schedule is chosen. A window in which a raising step and a quiet one look the same can't price the raise exactly, by a fact about functions far older than the machine. Entitlement is defined as an object, a checked relation, and a paid event; given lists, a tree and a covering somebody supplies, a run that paid a charged step for every level of the tree has paid for the index bits it commits to (that count doesn't show a run has to pay them, the run doesn't recover the tree, the lists count repeats, and the index bits aren't the questions anybody asked); when the run makes the narrowing itself, every candidate dropped was dropped by a test the run paid for, and the index bits are at most k times the cost when each test keeps a 2^{-k} share; on the small machine the same count, for a run from a clean start, is a theorem about its record moves, with the earned chain inside the run, and the count can't do without the covering; and it is a license to lean on a claim. It doesn't prove the claim true. Insight, in the sense the toll prices, is a distinction made to stay. A feeling doesn't count. Its floor is flat: at least one, however deep. The claim behind the flag is something a check could have refused, and in the bare model nothing makes the commit wait for the check. Over any base, a base-driven record is a latch, with two unforced choices, the event and the price list; "only one substrate" is false and "one structure, many price lists" is true once the event is fixed. No procedure whose flip the substrate itself can represent decides the shortcut predicate, given a recursion theorem, by the same diagonal that broke Frege's floor, and the machine's answer is to record the commitment. It leaves the judgement unautomated.

Part III built an example that meets the definition. The small machine's state ties a checked claim to an edition of a counter, and its step uses the tie: change the counter and the old check no longer lets you commit. So from a clean start a raised flag needs a passing check, a commitment about the same edition and a certificate, in that order. A cost of three is enough, and no such run pays less. Those are facts

about the example, the check that can't fail included, and its two-counter base is universal under Minsky's packing.

The universal programs ask whether those relations survive somebody else's machine doing the work. On the host with more counters, U runs the small machine's programs from clean starts, halts when they halt with their answers, earns its own flag exactly when the guest raises one, and matches the ledger wherever the two line up. With PAY added, U_P runs every computably presented Thiele machine, latches the guest's reading through its own earned chain, and carries the guest's ledger to within the stated surcharge: at most two at the raise when the reading starts at no. The theorems say which states match; this particular build takes a long time to get there.

Then I joined machines up, to see what carried over. Two machines side by side need both records kept; one total loses which side did the work. Hand-offs and nested runs each come with conditions of their own (Appendix D). Every one of those constructions makes you say which relation survives the change and which is lost, and that saying is the work.

Of five record disciplines I modelled in fragments, one charges a fee at the write, and which event to record is a thesis, with its refutations listed. And then came what the counter has to do with physics. Two facts about the machine sit where they belong. Its unpriced steps merge states, because that's what forgetting scratch work is; whatever those merges cost the hardware is off the ledger, so the toll on this machine comes from the record's merge premise, with no finiteness premise on the whole machine; the only finiteness in it is the flag's two readings. And a check that cannot fail still certifies for three, because the step charges for the act, and the entitlement count says what the claim bought.

Part IV brings those pieces together (Section 20). A finite permanent holder has to merge; the named price rule puts that merger on a ledger; a window determines which part of it a reader can recover. The example machine then supplies the earned path from a passing check to commitment and certification. Each argument has its place in that chain, and the constructions show how to carry it out.

Four contributions bring that chain into focus, each alongside the earlier work it builds on. First, the toll isn't only my preference: on a finite machine a reading nothing turns off can only be turned on by a step that merges states, so if merges are priced, raising it costs (Theorem 3.1, Observation 3.3); and on any machine a raise that merges nothing needs endlessly many yes-states, so it pays in fresh states instead (Proposition 3.4). The counting under it is the finite cousin of the recurrence argument Zermelo threw at Boltzmann in 1896, and the abstract merge price is motivated by Landauer's reset example. Identifying that price with heat needs the further premises stated in Part I. Ouldridge and Wolpert (2023) had already put a physical price on that merge for finite automata, and Li and Vitányi

(1996) had already assumed a unit price on each irreversibly erased bit in a model of computation, and Demaine and his coauthors (2016) a price on instructions that destroy bits, so what's here is the toll such a price forces on a reading that stays. Second, a window that shows a raising step and a quiet one the same can't price the raise exactly (Observation 8.2); that a price computed from what a window shows can't tell apart two steps the window shows the same is elementary, and what's here is the pricing form. Third, a record driven by the computation that never switches off is a latch on one event, over any base (Observation 11.1); latches and history variables are old, and the theorem covers only records that never come down. Beside it sits the flat case: forget a machine's record and its prices and what's left is a computer, blind to what was kept and what it cost, and a window on the flat part can't see either (Section 12). That forgetting a layer gives the runs back, where the step doesn't read it, is the ghost-code erasure of Section 7; what's here is that the forgotten layer still earns its record, and that what's left can't see it. Fourth, a record that's priced, and on the small machine earned (Theorem 15.11), under the move of guaranteeing something about every program. That move isn't mine at all. It's the reference monitor's (Anderson, 1972), made precise for monitors that watch a run (Schneider, 2000), and proved for monitors checked at every instruction (Azevedo de Amorim and coauthors, 2015). What I put together under it is built, and its idea isn't new: the check bound to the edition it looked at, the toll charged on top, and universal programs that carry both to every program of the small machine and to every computably presented Thiele machine, earning their own flag as they go.

And there is one thing I'd point at if you asked how far the proofs reach. It isn't a fifth new thing. In the model, the diagonal rules out a correct decider whenever its flip can be expressed inside the substrate and the substrate supplies the fixed point, and it uses neither the counter nor the toll. On the small machine the wall stands by proof: halting for its programs is undecidable in the reduction sense of Theorem 15.5.

There are things this book doesn't settle, by my count. It doesn't settle whether the toll is the *right* thing to price; the five real systems mostly built a wall or a penalty for contradiction. It doesn't settle whether certification is the pointer event. And it doesn't settle whether Landauer's critics are right, which would leave every theorem standing and one premise unowned by the world. And the question I used to leave open, whether there is a smaller start than a partition, has its answer from inside the definition: drop symmetry, and the two-rule order the record already reads into is the smaller start, with the partition as its symmetric case (Section 6). What I'd most like someone to take from me now is whether anything sits below that, by a ruler they can state. It would change this book.

One more claim ties the counting to the world, the way Church and Turing's ties computing to it, and I'll state it as what it is, a thesis, the *Thiele thesis*: every

physical process that computes with final answers has an exact description as a machine run by one rule, with a next state for every state, computably presented (its states written as whole numbers, and its step, its reading and its start for each input computed by programs that halt on every input), on which the answer is a reading that no run from any input ever turns off, on inputs accepted and rejected alike. No proof settles it, because "a physical process that computes" isn't a mathematical object. A model of computation is one, and for a model written down as states, a step and a reading that stays, what keeping takes needs no thesis: it's a theorem, and Section 22 says how far it reaches. The thesis is about what a physical computation pays for what it keeps, which is Part I. A description run by one rule, with its program loaded, is a Thiele machine in the flat case: either every move costs the same, a clock, or its reading never rises, and either way the toll asks nothing of it. The price shows only when the moves are offered before a program is chosen, and with the program in the state a reading is a record, not a simulated bit, exactly when it never comes down from any state. So the thesis is about physical processes and the one-rule descriptions of them. The Thiele Machine is the model that prices what such a description keeps. There is an established route to a conditional physical description. Gandy's work on mechanisms, restated by Arrighi and Dowek, starts with homogeneity in space and time, finite local state sets, bounded propagation and an initially finite departure from a preserved quiescent background. Their classical theorem makes the resulting evolution computable. In the construction, a cell consults finitely many neighbours through one finite table; a finite active region can therefore be advanced by a program. These are substantive modeling premises. They do not select a certification event or a price in joules, and they are not asserted of every physical system here. The quantum extension uses different premises. See Arrighi and Dowek, *The physical Church–Turing thesis and the principles of quantum theory*, Section 2, `https://arxiv.org/abs/1102.1612`.

What is proved sits on either side of the thesis. On one side, Part I applies to every such description on the states its inputs reach, where the answer never comes down: a run that raises it merges two of its own states or never visits a state twice (Proposition 3.5). On the other, seen only through which inputs it accepts, the thesis implies that every set a physical process accepts with final answers is one a Turing machine can list. At that extensional level it has the physical Church-Turing thesis's conclusion. Run the description from every input, a step at a time across all of them, and list an input the first time its reading says yes; no run turns its answer off, so a yes seen once is the answer. The other way, a set a Turing machine lists is accepted by the machine that searches the list for its input and then steps from its halted configuration to itself, with the reading "halted," which nothing turns off. This converse constructs a machine with the same accepted set; it does not prove an exact state-by-state description of the original physical process. Nothing about

how the keeping is paid survives that view: the same search held at its halt by a step to itself, which merges, or by a counter that goes on ticking, which takes fresh states, accepts the same inputs. And "final" has to cover the rejected inputs too. A machine that starts at no, reads yes while program n is still running and falls to no for good if it halts reads yes for good on exactly the programs that never halt, and no Turing machine lists those, since one that did, run beside the search for a halt, would decide halting.

And then there are the two computers of Section 14, the sorter and the copier, holding the same list. Walk the warehouse again. The list is in order on both. On the machine with the earned layer, a different part of the state tells you whether a check happened and a matching commitment followed it, and the rules keep that tied to the edition that was checked. Put your hand over that part of the machine and the difference between the two computers vanishes from where you're standing. Change the counter and you can watch the old check go stale. Those are different operations with different consequences, and you can follow each one with your finger.

Carry a report out of the warehouse and you've got another question: what connects the report to the machine? The flag on its own doesn't answer it. A window, a copy, a second machine and a reader each bring relations of their own, and the constructions in this book ask, one at a time, which of those relations hold. That's how the example gets past the lamp: by what survives a change of view, a change of state, or a change of machine.

At the bottom of the Thiele Machine is one cell going from blank to 0, and the question of what that write keeps and what the chosen price rule charges. Built on that cell, the machine is structure held as state, acted on by a step, with an account of the changes and a view that may keep or lose their distinctions. What I'm arguing is that those relations are part of computation itself, and not something you bolt on afterwards. The record and its toll are the part of that I've worked all the way through, and there this much is proved: forget a Thiele machine's record and its prices and what's left is the flat case, a computer, blind to what was kept and what it cost; on the small machine a window on its counters can't see either (Theorem 15.15). Run as one rule, a machine is a Thiele machine only as a clock or with a reading that never rises. The small machine and its interpreters are things you can take apart. Each theorem answers a stated question under stated premises. Put the computation, the record and its account on the same drawing, and you can follow what changes, what stays, and what the next reader has to work with.

CHAPTER 24

Afterword: how I got here

What I am is curious, past the point of good sense.

I pick a thing up and cannot set it down until I have turned it over enough times to see how it holds together, and this time it did not let go.

I am not a programmer and I was never formally taught any of this.

I was not trying to build a new model of computation.

I would not have known what one was if it bit me.

I was trying to build a 3D renderer.

There was no plan behind it.

Just a pull to make something real with my own hands, and the kind of stubbornness that will not let a thing go until it stands up on its own.

I have worked on it in off hours every day since.

Once I started digging in, two things came clear fast. I knew nothing about computers, and the field was already saturated, every obvious road paved and crowded. So I needed an angle nobody else was standing on, and the angle I had wasn't a strategy. It was just the only way my head works. I did not know what compositional meant. I had never heard of category theory. I called the thing I was building digital DNA, because that is what it felt like in my hands, and I thought about it in the only language I had: gravity pipes and directions and vectors, things that were linear and somehow not linear at the same time, structure that flowed and joined and came apart without my owning one proper word for any of it. I built a custom rendering pipeline out of those intuitions, by feel, throwing out pieces that did not sit right even when I could not have told you the rule they were breaking. And then, the way I always do, I went looking for what I was actually doing, and found out it already had a name. An old one. A deep one. What I had been reaching for in the dark was composition. It was category theory, a whole field I had never once heard of, sitting there describing the exact shapes I had been shoving around. The moment that landed, the rendering pipeline stopped being a rendering pipeline and started turning into a categorical engine, and I started swallowing the real theory as fast as I could find the words for things I had already built.

I took it one concept at a time, and I would not move to the next until I could watch the current one actually run. Category. Object. Morphism. Functor. Monoidal product. It came together in pieces and never in chapters, the way a picture resolves out of static. The Yoneda lemma (a result buried deep in category theory) was the piece that made the rest snap into place.

Then within weeks I found out the engine I had built to draw pictures was general enough to run a physics simulation on the exact same primitives, without my rewriting any of it. I wrote Newton's laws as a program in it, half expecting nothing, and it ran. I want to be careful about what that was and what it was not, because this is exactly the kind of thing that gets misread. It was not me deriving physics, and I am not claiming the machine explains gravity. It was the compositional structure being general enough to express a physics simulation the same way it had expressed a renderer, and that generality is what told me the structure was the point and the pixels never were. Somewhere in those weeks I crossed from debugging why one pixel would not show up to understanding that I had quietly built a categorical computing engine, and that the rendering had only ever been the first thing I aimed it at. The pixel did eventually show up, for what it is worth.

I kept going, because by then I could not not keep going. The night the engine first ran a program declaratively, meaning I told it what should happen and let it work out the how without my spelling out each step, I told my wife, Julie, I thought I was onto something. She said okay.

Then I hit a wall, and it is the wall this whole document is about. Every tool I had for settling questions about what I'd built could settle the small ones and fell over on the exact ones I needed answered, which left me staring down an infinite hallway of guess-and-test with no proof waiting at the end of it, ever. One night, not even working, I was watching a video about Flatland, the old story about a creature living in two dimensions who cannot so much as picture a third. And it landed on me sideways, all at once, the way the real ones always do. I said it out loud: I get it now. There is another axis. Classical computation only ever sees what it can read and write off its own surface, and there is a whole direction it is blind to. It isn't broken. It was just never built with an eye that points that way. That's how it landed that night, and it didn't survive in that form: the direction turned out to be one an ordinary computer can reach, and what the usual rules leave out is the price (Section 2.3 says where the axis ended up). The two computers in Section 14 are my attempt to show you that blindness with something you can hold. That same night I gave the idea a name: the Thiele Machine. The name began as a working title and became the name of the model. It carries the larger claim plainly: the machine treats structural state as part of computation.

What survived the whole trip from there to here is not the renderer. Not one piece of it, and good riddance. What survived was a single commitment I made early

and never once let anyone talk me out of: structure is the state itself, the actual thing the machine runs on. It is no metaphor laid over some realer state underneath. Category theory was the first language I found for that structure. It is not what the model is. The basic model asks for states, instructions, a step, prices and a reading. The ledger and the window are further structure used in the constructions. None of that needs category theory. The morphisms in that first engine, the night they first composed correctly at two in the morning, were objects that remembered where they came from; the records in the small machine I ended up with remember where they came from too, and you can walk up to one and challenge it. The receipt grew teeth. But the shape itself never moved. The shape was the substrate the entire time, and everything else, every renderer and every proof, was me slowly learning to see what I had already built. By the shape I mean structure held as state. Not category theory. Category theory was just the first language I had for it.

Every step pulled back to show another layer underneath that I had not known was there, and I mean that plainly: I did not know what a Turing machine was when I started, I did not know most of the words in this document existed. I learned each one the same way, by walking face-first into a wall that needed it and not being let past until I had it. The needing always came first. The learning came after, dragged along behind the needing because it had no say in the matter.

Large language models were my hands the whole way through, and I want the division of labor exact, because it is the thing people most want to get wrong about work like this. The hands had nothing like this to draw on: no reference implementation, no prior project to point at. The ideas, the architecture, the direction, the demand that the thing be true: those are mine, all of them, no asterisk. The symbol-pushing, the code and the formal derivations, the models produced. So I read every statement. A derivation can settle an empty question, and I had to refuse that too. However the symbols got onto the page, the argument on it is checkable by reading, and it would be exactly as right or as wrong if a million monkeys had typed it. The monkeys could not have refused.

A word on what I actually do, because it runs under everything after it. I do not write code or math by hand. I do not pick constants. I do not grind through derivations. I have never written a line of code on paper or in a notepad in my life. What I do is see, and direct, and refuse. When I take hold of an idea it does not stay a point. It opens up. It becomes a shape I can blow up and turn over and take apart, lay across another shape to find where the two of them touch, run forward and then back to where it started, zoom into until it is the only thing in the world and pull back until it is one dot in a field of other dots, and the whole time it stays wired to everything I have ever connected it to, the right connections and the wrong ones both, all of them live at once. I am not reading that field neutrally, the way a search would. The thing doing the looking is me, the whole of me, my gut and my mood

and my stubbornness picking what lights up and which way I move through it, and that is the part you cannot lift out without turning me back into a search engine. So I see where a thing has to fit, the shape it is obligated to have, usually well before I could write it down in a single symbol. Then the model gives me the symbols, and I refuse anything whose argument does not hold or does not match the shape I am holding. That refusal is the one thing in this entire document that is mine and only mine: the demand that it be true, and the throwing-out of everything that wasn't.

I am telling you all of this because it's the engine. I did not come up through anywhere that hands you a foundation and says trust us, this part is settled. No professors, no colleagues, no field full of authority to lean back on when a claim got heavy. From the outside that is a deficit. From the inside it is the whole reason any of this is solid: when I meet a claim I either find the proof or I do not believe it, and that goes double for my own, because my own are the ones I most want to wave through. It is how I am built. It is the only way I know to hold a thing and actually trust that I am holding it.

So everything in here gets built from the ground up. It isn't a courtesy to you. I build it because I do not believe I understand a thing until I have built it with my own hands, watched it stand up on its own, and tried to knock it over. I built this document the exact way I built the machine, brick by brick, refusing each brick until it held weight. That includes the parts you already knew cold before you opened this. I did not skip them for you, and I could not skip them for me.

The vocabulary I had to go look up to do any of this sits at the back of the book, so the argument comes first. If a word stops you, it is there. If you already know this material, you never have to open it. But if you catch something in the basics that is wrong, I want to know, and I mean that as a real ask. It isn't politeness. Nothing in here is correct just because I wrote it. Especially the parts I wrote.

On the name. I wanted to call it the meta machine. Then, in October 2021, Facebook renamed its company Meta: a company taking a term for self-reference as its name and draining the word of meaning in the same motion. So I went one level up, where this machine already lives. The most meta move available, for a machine that runs on meta-level reasoning, was to put my own name on the cover and take the word back. That's the whole of it, a meta move on a meta machine. It happens to be my dad's name too, and my wife's now. So I asked them before I did it. They said go ahead.

APPENDIX A

Words this book builds

These are mine, or mine in the sense that I pinned them down for this book. Each is built properly in the chapter named; this is only a reminder.

A few words first, because everything else hangs off them and each has exactly one meaning. The *model* (or the abstract model) is the definition of a Thiele machine, Definition 2.1, five parts and the toll, with the two-place record of Definition 2.2 as its smallest case, plus exactly the extra pieces a given theorem asks for: a counter in the state, or a window, or programs written as states and a recursion theorem. A few theorems, the step law among them, use only the states and the step. The name Thiele Machine belongs to the model. Parts I and II develop those general arguments, and Part III applies them to the machine it builds.

A *substrate* is the model's shape with whatever extension a theorem names, and nothing else; the word never means my machine, and never the side of a simulation that does the stepping (Section 14.15).

The example machine (also the machine I build, the build, the small machine, the smallest honest machine, Section 15) is one small machine I defined to show the definition can be met. It is an example of a Thiele machine, not the model. It fills in those pieces at once: two counters with their versions, a table of checked facts, a channel, a trap, a ledger and a flag, six instruction kinds and their prices, with its hosts and universal programs.

The *scaffolding* is the example machine and the constructions run on it, and the model does not depend on any of it. It lives in Section 15 and in the appendices that carry the machine on, the universal machines (Appendix C.1).

A Thiele machine, lower case, is any system proved to fit that definition (Definition 2.1), five parts and the toll, on any order of records or on the two-place one of Definition 2.2; the paper toys are, and so is the small machine. That bar is low on purpose: a clock meets it, and so does every machine in the flat case, whose moves never climb.

Certified is what the small machine is held to beyond the definition (Section 14.15): its flat moves cost nothing and can't touch the record, a record that stays up once it's up and is raised from a clean state only after a passing check,

which means its claim holds, a commitment to the same claim with the thing it's about unchanged in between, and a certificate, at a cost of one for each of those three acts, and a claim whose check, commit and certificate raise the record from a loaded start exactly when the claim holds there, true at some loaded starts and false at others. The small machine meets it, and it hides its record and ledger from a window on its counters.

A *universal* Thiele machine is one fixed machine running one fixed program that runs any machine of a stated class as a guest, with every step of the guest's computing done by the host's counter moves and every rise of its record put where its program put it, and, in the certified case, each guest's record earned again by the host's own check, commit and certificate (Definition C.1). The ones in this book are U (Appendix C.1), a fixed program in the small machine's instruction kinds, with no guest step built in, that runs every small-machine program with halting and flag exact and its ledger equal to the guest's at every matching point, and U_P (Appendix C.2), the same program with one paid move added, that runs every *computably presented* Thiele machine, one whose driver, step, prices and reading are computed by programs with the reading decided on every state, keeping its reading as a latch and, when the reading starts at no, its ledger to within two of the guest's, a surcharge no program on that host, run from a clean start, can avoid when the guest certifies with its whole ledger below three.

A computer running the small machine isn't the small machine; it implements it, inside bounds, the way a laptop implements a Turing machine. When a word below belongs to only one of the two layers, the entry says which.

A *partition* is a sorting of a pile into groups, nothing in two groups and nothing left over; a *cut* is a sorting in two with a door between the halves that swings one way, the order a yes-or-no reading puts on the pile; and an *order*, a preorder, is a partition with a one-way order on its blocks, the thing the record reads into (Section 6). A *Thiele machine* is a state type, moves, a step, a cost per move, and a record read into an order, plus one rule, the *toll*: a move that carries the record somewhere not at or below where it stood costs at least one; on the two-place record, a move that turns no into yes (Section 2). The rule is called A2 in the conditions table (Appendix B). A move *merges* when two different starting states land on the same end; one that never does is *injective*, it forgets nothing. A reading is *permanent* when no move turns it from yes back to no (Section 3).

A *distinction that stays* is something the world tells apart from something else and keeps telling apart: a cell that reads 0 and not 1 and not blank, a bead in the left well and not the right, a flag that's up (Section 1). The *step law* says what any step of any machine with one next state for each state does to the states its run has used, with no finiteness anywhere (Theorem 1.1). It lands on a state the run hasn't used yet, which uses up a *fresh state*. Or, the first time it comes back to a

used state other than the start, it comes from a different state than the one that first got there, which is a *merge*, two states sharing a next state, and under Landauer's premise a merge of states that were really possible carries a *heat* floor. Or it comes back to the start, or retakes a step it already took, which is all a run does after its first return: it's going round a loop and makes nothing new. *Merge, or a fresh state every step* is that law aimed at a reading that stays up: a run that raises one either merges two of its states somewhere or never repeats a state (Proposition 3.5), and that's what *keeping* takes. Physical dissipation is additional only under physical premises. In the specified thermal models a nonzero change of distribution has a finite-time entropy-production bound, with fixed finite activity or mobility, and slowness does nothing to the floor a merge adds. The book proves that for Bennett's walker (Theorem 1.9); the general finite-time bounds are physics (Schmiedl and Seifert, 2007; Shiraishi, Funo and Saito, 2018). A *reversible* machine never merges, so it owes no floor, but that doesn't mean it pays nothing: each step can dissipate under the stated thermal-model assumptions, and it pays in fresh states; on finitely many states it comes back to its start and keeps nothing (Theorem 1.6). *Finiteness* is what takes the column of fresh states away: when only finitely many states read yes, a move that raises a reading that stays up has to merge (Proposition 3.4).

The *counter*, which I also call the *ledger*: in the model, a number carried in the state that starts at zero and that every step raises by exactly that step's charge (Section 7). My machine has exactly one, the ledger μ (Section 15). One other thing gets called a counter, and each chapter says which: Minsky's two bins on the small machine. They aren't the ledger. A *price list* says what each move costs; the model takes it as given, and my machine's table of what each instruction costs is in Section 15. A *receipt* is a total that lives in the state, only climbs, equals the sum of the charges on every path including the failures, and is the only such total, on the states a run from the start reaches, for its price list (Section 7 proves it for any system with a counter and a price list; on my machine the sum is a theorem too, Section 15). In the model there is a *reading* into an order, two places at the smallest (Section 2); on the small machine it's the *flag*, set by a CERTIFY that passes and never cleared.

These words describe the example machine and its universal programs, and none of them is part of the definition of a Thiele machine (Section 15). A *two-counter machine* is Minsky's: two counters holding whole numbers with no upper limit, and two instructions: add one, and subtract one and jump if the counter is above zero, or else go on to the next line; already enough that nobody can decide in general whether one halts. The small machine is a two-counter machine plus HALT and three instructions for the record, six in all: INC, DEC, HALT, CHECK, COMMIT and CERTIFY. A counter's *version* is its edition number: it starts at 0 and goes up by one every time the counter is written, so an edition never comes back. A *fact* is

a property, a counter and a version together, such as "A is even, as of edition 3"; the properties come from a fixed language with three forms, *is zero*, *is even* and *is at least* n. The same theorems also hold for any property language with an exact equality test whose checker is proved to say yes exactly when the claim holds, and one such language reads a counter as a list and asks whether it's sorted. The *fact table* holds up to sixteen facts, and the only way in is a CHECK that passed; nothing ever leaves it. The small machine's *channel* holds one committed fact or nothing, and COMMIT puts a fact there only if the table has it about the counter's current edition. With the flag up from a clean start it names a commitment earned that way, and a later COMMIT replaces it, as a two-commit run shows. The *trap* is a bit that goes up when an instruction refuses, and then nothing in the core moves again. A commitment is *earned* when a passing check put its fact in the table first, about the same edition, with no write to that counter in between; on the small machine every flag raised from a clean start is earned, and that's a theorem. CHECK, COMMIT and CERTIFY cost one each and everything else costs nothing, so from a clean start a run ending with the flag up adds at least three to the ledger, and a run whose check could have failed adds exactly three: CHECK *is zero* on A, COMMIT, CERTIFY, from a start where A is 0. The step doesn't police trivial checks; *is at least 0* passes every value and certifies for three too. The *host machine* is the small machine with a counter, called a *register*, for every whole number, and one property, Slot, true of a number that packs a property of the small machine together with a value it holds of. A *slot* is a register of the host used for nothing but holding one checked fact about the guest, and its *mirror* is a register beside it, safe to read, that says which claim the slot holds while its fact is live. U is the fixed program of host instructions that runs every small-machine program; it reads guest instructions out of one register, keeps the guest's counters in two others, and puts every checked fact in a slot, because on this machine reading a counter above 0 into a program's control flow writes it. The universal program's *guest* is a program of the small machine, held as one number in a register of the host and run one instruction at a time by U's fetch and decode. U_P is U with one more handler, for PAY, an instruction that costs one and does nothing else; its guests are programs compiled from computably presented Thiele machines (Appendix C.2).

A *window* is any function that shows you part of a state (Section 8); a *shadow*, said loosely, is what a window leaves you with. A *collision* is two steps that look the same through a window, before and after, where one raises the flag and the other doesn't (Section 8). *Entitlement* is what a run has when it carries an object, a checked relation between that object and a claim, and a paid event that put the object in place: a license to lean on the claim. It doesn't prove the claim true (Section 9). A *latch* is a record that switches on the first time some event holds and never switches off (Section 11). The diagonal's *substrate* (Section 13) is the model's

shape extended with programs, a run that may give nothing back, a counter that never falls, a way to write programs as states, the transformations the substrate counts as its own, and its own recursion theorem for those; the proof of the diagonal uses neither the counter nor the toll.

APPENDIX B

Every assumption in one place

This appendix gathers the conditions used throughout the book and shows what each supplies. Use it to follow the connections between the results: which count needs finiteness, which record needs permanence, and which charge follows from which price rule. Each theorem also states its own hypotheses where the argument is made.

The labels A2 to A6 in this appendix are shorthand for the conditions listed below. They are not additional claims: each label names the rule written beside it. There is no A1: A2 is the toll's name in the mathematical specification, and the others follow it.

B.1 Systems and their conditions

Definition B.1 (Thiele machine with a two-place record)**.** A Thiele machine (Definition 2.1) has a record in any order. The results of this appendix are stated on the two-place record of Definition 2.2: a set of states S, a set of moves I, a step function $\text{step} : S \times I \to S$, a cost $\text{cost} : I \to \mathbb{N}$, and a reading $\text{cert} : S \to \{\text{no}, \text{yes}\}$, satisfying the toll: every move that turns the reading from no to yes costs at least one. This is the condition of Section 2, called A2 in the table below. Every result stated "for any Thiele machine" in this appendix assumes it.

Definition B.2 (Quantitative Thiele machine)**.** A quantitative Thiele machine is a Thiele machine with a *witness*, a function W from states to $\mathbb{N}$, and a threshold $N \in \mathbb{N}$, satisfying: (A3) $W(s) + \text{cost}(i) \geq W(\text{step}(s, i))$, the witness grows by at most what the move cost; and (A5) a state that reads yes has $W(s) \geq N$. An optional additional condition is (A4) $W(s) \leq W(\text{step}(s, i))$.

Theorem B.1 (The floor raised to a threshold)**.** In a quantitative Thiele machine, every run from a state with witness w to a state that reads yes has total cost at least $N \dot{-} w$; from a state with witness 0 (this starting condition is called A6) that is at least N. □

Proof. Put the witness readings in a row: $w_0 = w, w_1, \ldots, w_n$. Beside the move from w_j to w_{j+1} write its charge c_j. A3 says that move can add no more than c_j to the witness: $w_{j+1} - w_j \leq c_j$. Add these inequalities. Each intermediate reading appears once with a plus and once with a minus, so only the last reading minus the first survives:

$$w_n - w \leq c_0 + \cdots + c_{n-1}.$$

At the end the reading says yes, so A5 puts w_n at least at N. The bill is therefore at least $N - w$. If the witness started above N, that subtraction is negative and supplies no positive floor; the bill is still at least zero. That is what the truncated subtraction $N \dot{-} w$ says. At $w = 0$ the floor is N. For an empty run the same argument reads $w_n = w$ and bill zero. Nothing here required the witness to rise on every move (A4), or used the toll. ■

A3 and A4 bound opposite directions, and neither implies the other: witness 1, cost 0, witness 0 satisfies A3 and breaks A4, and witness 0, cost 0, witness 1 does the reverse. These conditions describe a general setting. The small machine's own floor of three is proved directly (Observation 15.19) and never goes through it.

The table below puts the labels side by side, so none of them is left without a meaning.

Label	Condition
A2	the toll
A3	the witness grows by at most the cost
A4	the witness never drops
A5	yes needs a witness at the threshold
A6	the run starts with witness zero

B.2 The premises the derivations take

These are definitions used as hypotheses. Nothing proves them in general; each theorem that uses one says so in its statement.

Premise	What it says	Where it's used
One next state	every state has exactly one next state, and states can be told apart	the step law, Theorem 1.1, and merge or a fresh state, Proposition 3.5(1); no finiteness
Finite	a list with no repeats contains every state	the permanent-flip theorem, Theorem 3.1, where only the states reading yes need to be finitely many, and what's built on it: merge or revoke, the squeeze bound, the entropy road and its heat floor, the certainty bound and the direction (Theorems 3.7, 3.10, 3.15, 5.5 and 4.4); the noisy and unitary steps; the exact count of states, Proposition 3.5(2), for bounded memories; recurrence, Theorem 1.6
Permanent	no move turns the reading from yes to no	the permanent-flip theorem; the toll derived, Observation 3.3; the squeeze bound and the entropy road; a raise with no merge, Proposition 3.4; merge or a fresh state; the latch, as (R4); the direction, as growth
Merge pricing	every move that merges costs at least one	the toll derived, Section 3; forced pricing, Theorem 3.8; the direction, Theorem 4.4(3)
Halving price	each unit of cost pays for at most one halving of distinct states	the squeeze bound; narrowing along a run, Section 5
Entropy pricing	on a finite machine, each move costs at least the bits it removes from any spread	the third road to the toll; the certainty bound, Theorem 5.5
Landauer's principle	for a stated physical memory whose logical states are alike in their own entropy, removing ΔH bits gives off mean heat $\geq k_B T \ln 2 \cdot \Delta H$	the heat floor; with $k_B T \geq 0$ (or > 0) as a separate hypothesis on each theorem
Recursion (generic)	every transformation the substrate can represent has a fixed point	the diagonal, Section 13

Premise	**What it says**	**Where it's used**
Commitment contract	a commitment bit is set only when the claim holds, and whenever it holds	the commitment escape, Section 8
Computably presented	four programs compute the driver, step, price and reading on valid codes; the reading program also halts and returns false on junk codes	the universal host for presented machines, Appendix C.2
Clean start	an empty fact table, an empty channel and the flag down	every provenance theorem about the small machine, Section 15; the floor of three needs less, and that section says exactly what
Record-layer merge pricing	a move that sends two different states with the same base part to one state costs at least one	the toll on the example machine, Theorem 3.19
Representable flip	the substrate can represent the flip of the candidate decider	the diagonal, Theorem 13.1
Quorum intersection; charge predicate	as transcribed in Section 17 (the setting also carries at most one parent, which no proof uses)	Theorems 17.1 and 17.4

Four remarks on the table. First, the recursion premise is what the diagonal asks of a substrate (Section 13). Second, the substrate used for the diagonal also carries a different rule from the toll: the counter never goes down on a step. Neither implies the other, and the diagonal uses neither. Third, merge pricing is an abstract lower bound on every merging instruction. It is the pairwise restriction of worst-case entropy pricing, which is equivalent to the halving price. Landauer's premise instead bounds heat from the uncertainty actually removed under its thermal conditions. A known starting state supplies no positive floor from that premise (Theorem 3.17). Preparing or erasing an uncertain record of that start would be a separate physical process with its own premises; knowing the start alone proves no payment. Under the fixed-protocol entropy-balance assumptions in Section 3.7, a protocol merging K inputs has a worst input with at least $k_B \ln K$ of entropy production. This compares inputs to one protocol rather than charging every known-state execution. Riechers, Boyd, Wimsatt and Crutchfield (2020) derive additional work bounds under time-symmetric control of metastable memories. Their low-error bound depends on the initial distribution and the memory's time-reversal behaviour; it is not a universal heat charge on each logical move.

Fourth, finiteness, because it's the premise people most often think the book leans on, and it leans on it less than it looks. The step law and the first half of merge or a fresh state need none: on any machine run by one rule, finite or endless, a run that raises a reading that stays up either merges two of its states somewhere or takes a fresh state at every step. Finiteness is what takes the second option away. With infinitely many states reading yes, a price list that charges only merges lets a machine certify at a cost of zero, the machine that keeps every history, the first way out (Section 3), by taking a fresh state at every step. That isn't a way around what keeping takes; it's a price list that charges nothing for the fresh states. And the finite set the count needs can be very small. When the record steers nothing and every move adds its own price to the ledger, the count runs over one state of the rest of the machine, on the record's own readings, and for a flag that's a set of two. That's how the machine of Part III, which has no end, still pays for its raise in a merge (Theorem 3.19). Applying a finite-state count to a device requires a justified finite set of distinguishable states. The abstract theorem does not supply that physical premise.

APPENDIX C

The universal machines

This appendix carries the two universal programs of Section 16 in full: U, which runs every program of the small machine, and U_P, which runs every computably presented Thiele machine, with the simpler host that has the guest step built in, the diagonal on the small machine's programs, and exactly what the universal machines cover.

C.1 One machine that runs them all

The small machine raises its flag only by checking, committing and certifying, and it pays for that. The next question is whether one machine can stand in for a whole class of machines, the way Turing's universal machine stands in for every Turing machine. What I mean by that is a *universal Thiele machine*: one fixed machine running one fixed program that runs any machine of a stated class as a guest, with every step of the guest's computing done by the host's own counter moves and every rise of the host's record put where the host's program put it. No program running on the host has to keep a convention for it. In full:

Definition C.1 (Universal Thiele machine)**.** Fix a class C of Thiele machines, each with its runs from given start states. A *universal Thiele machine for* C is a Thiele machine H, the *host*, with one program P fixed for the whole class, a way to load any guest M in C with any start s_0 into a start state of H, and a way to read a guest's state off the host's registers, such that: (1) H is a Thiele machine whose counter moves cost nothing and can't touch its record, and those are its computing moves; (2) the run of P from the loaded state halts exactly when M's run from s_0 stops, and then shows the observable part of M's last state, as the class names it, its reading kept as a latch, and its ledger up to the surcharge; (3) the run passes, in order, through host states that show the observable part of M's states along its run, at which the host's record equals M's reading kept as a latch and the host's ledger equals M's ledger plus a surcharge, never negative and bounded by a number fixed for the class; (4) *the rise is the program's*: from every untrapped state, a record move selected by P either traps, or leaves the flag up exactly when it was already

up or the current line is marked as raising by P. With the flag down and the move succeeding, that line decides whether it rises; (5) H is a *register host*: its counter moves are INC, DEC and HALT on registers, its record moves leave the registers alone and go to the next line or trap, and which move comes next depends only on the line and whether the trap is up; (6) each guest is loaded with the record down, no trap, a fixed line and the guest written into the registers, and the guest's state is read off the registers in a coding fixed with the class, before any guest is named. A *certified* universal Thiele machine also has its host certified, with clean loads, and its record, whenever it's up, standing on the host's own check of the guest's own justification, as the class supplies it.

For U the guest's justification is its chain claim; for U_P, the guest's reading at a state of the run. For U the observable part I've proved it matches with all six clauses is the guest's two counters, and in order it matches everything but the guest's version numbers, which no run of the guest can show; for U_P it's the whole state.

The guest lives only in the loaded state; the machine and the program are fixed before any guest is named. For U the surcharge is zero; for U_P it is at most two when the guest's reading starts at no, and exactly three when it starts at yes, so three bounds it over the whole class. Both are certified universal Thiele machines, by the theorems of this appendix put together: each meets the six clauses, each host is certified, with clean loads, and each one's flag stands on its own check of the guest's justification.

The two that U_P adds once a guest's record is up is, at worst, the price of certification, not of universality. A certified host raises its record from a clean load only after three marks, so on a guest whose whole ledger is one when its reading first says yes it falls two behind at least. A host without certification can keep up exactly: on a host with registers, INC, DEC, HALT, a paid move TICK that always raises the record, a paid move PAY that never does, and a paid move MARK that raises it when register 0 is at least 1, which no program below runs and which a record that tells starts apart needs, since without it no run's record depends on where it started, the program "count register 0 down to zero; TICK; HALT" runs the guests that count a pile down to zero and then raise their reading at a cost of one, each from its own pile, with surcharge zero. A guest whose reading is already yes at its start costs any host loaded with its record down at least one, and one is reached. And from a start at no one host keeps up exactly on the whole class: U_P's compiler on that host, with the three-act chain at the first yes replaced by one TICK and the rest of that move's price paid in PAYs. The toll on the guest makes that move cost at least one, so the TICK always fits inside it, and the host's ledger is the guest's at every matching point. On a host without certification the surcharge is zero from no and one from yes, and neither can be lowered. (And PAY is needed on such a host:

a guest that pays while its reading is down makes the host run a record move that doesn't raise, which is what PAY is.)

Each clause stops something. For clauses (1), (4) and (5) there's a host that meets the others and fails it; (2) and (3) together stop a host that ignores its guest; (6) I state in words below. Clause (1) stops a host that raises its record early and lowers it again before anyone looks. Clause (4) stops a mark that reads the registers, or a copy of the guest kept outside them, and raises when the guest would: the same move on the same line would raise from one state and not from another. Clause (5) stops a host with a move that takes one step of the guest. In the one I built that move costs nothing on the guest's unpriced instructions and traps on its paid ones; costing nothing, it would have to be a counter move, and a counter move never traps. Make such a move cost one instead and it's a record move, which has to leave the registers alone, so it can't move the guest's counters, which (6) puts there. Clause (5) also stops a move costing nothing that fetches, decodes and carries out a guest instruction on the registers, because that move adds one to one register from one state and to another register from another, which no single INC or DEC does. Clause (6) makes the guest come in through the registers. It's the one clause I've stated in words without a theorem that it discriminates: a reading of the registers that adds part of the guest back in isn't excluded by anything I've proved. What stops a host that ignores its guest is (2) and (3), on any class whose guests differ in when they halt or what they record, which a universal class does. What the clauses allow is exact too. By (1), (4) and (5), the line, the registers, the record and the ledger of a run before any trap depend only on the line, registers, record and ledger it was loaded with: each counter move acts on those alone, each record move leaves the registers and moves the line, and its effect on the record is fixed by its line. So a host may carry anything else along, even a copy of the guest that its counter moves step, and what it carries along that way can't touch anything the host shows. One route I haven't closed: a move that traps may read anything, and stopping the run is all it can do. U_P never traps on its class, so for it the route is shut. For U it's shut too: U is loaded with every version at 0 and an empty table and channel, so whatever a trapping move of U reads was written by the run itself, from the registers it was loaded with. For a host in general it stays open.

None of this needs certification. U_P's own program, read on a host with no claims, no checker, no table and no channel but with that MARK, with CHECK, COMMIT and PAY read as PAY and CERTIFY as TICK, meets (1) to (6) with the same surcharge: U_P never traps on its class, and while nothing traps the two hosts agree, step for step, on the line, the registers, the record and the ledger. Certification adds the tie to the guest's own justification, and it needs (4) as much as anything: a host that carries out a hidden copy's own check, commit and certificate meets the tie and fails only (4) and (5).

And I want it the hard way. The easy way is to give the host a move that means "take one step of the guest," and then "it runs any guest" is true because I wrote it that way. There's a host like that two subsections on, after the two programs, and it's useful as a picture. It isn't Turing's kind of universal. Turing's universal machine has no instruction for "do what the machine on the tape would do." It has the same kind of instructions every other Turing machine has, and one fixed table, and the table does the reading and the deciding and the carrying out. The bar here is the same, and clauses (4) and (5) make it exact: one fixed program, every computing step a counter move, and every rise performed by that program. The table, channel and trap still govern whether record operations succeed; the restricted computing-step claim is not independence from all auxiliary state.

Turing's own explanation is worth having in your head, because the program below copies its shape. In the 1936 paper the universal machine's tape starts with the machine to imitate, written out as a string of letters, its standard description, and after it the imitated machine's complete configurations, one after another: where its head is, what state it's in, what's on its tape. To take one step, the universal machine finds the last configuration, looks through the description line by line, comparing, until it finds the line for that state and that symbol, and then writes out the next configuration the way that line says. Then it goes back and does it again. It leaves marks on squares so it doesn't lose its place. Every bit of that is ordinary instructions on an ordinary tape, and nothing in its table knows any particular machine.

Turing's universal program came first on paper, and one that people actually came to use arrived through the side door. John McCarthy, who had started the artificial intelligence project at MIT with Minsky, was writing up his new language Lisp around 1960 and wanted to show it was neater than Turing machines, so he wrote a universal function for it, *eval*, the language describing how to evaluate itself, and argued it was shorter and easier to follow than a universal Turing machine. In his own history of Lisp, that was for the paper, a thing to read; the notation for writing programs as data was made up for the paper with no thought of using it in practice. Then Steve Russell (S. R. Russell in McCarthy's account), a programmer on that MIT project, noticed that eval could serve as an interpreter and coded it by hand, and there was a running language with an interpreter that nobody had planned. McCarthy adds that the surprise froze some decisions he'd made "rather lightheartedly" for the paper. So the trick has a habit: written to be read, it ends up being run. The program below goes the other way round, on purpose. It's written to run, and the proofs show that what runs is what the page says.

The host. The program runs on a machine with the small machine's instructions and the small machine's prices: INC, DEC, HALT costing nothing, and CHECK, COMMIT, CERTIFY costing one each, with the same table of sixteen facts, the same channel, trap, ledger and flag. One change: it has a counter for every whole number,

each with its own version, where the small machine had two. There's no new kind of instruction and no new price. Take away the table, the channel and the flag and what's left is old: a machine with as many registers as it needs, adding one, taking one away and jumping on zero, is the unlimited register machine of John Shepherdson and H. E. Sturgis, two logicians who proved in 1963 that such machines compute every partial recursive function.

Definition C.2 (The host machine)**.** The *host machine* is the machine of Definition 15.3 with one change: it has a counter, with its version, for every whole number r (a *register*), and INC r, DEC r j, CHECK p r and COMMIT p r name one register each. Its property language has one property, Slot. Write $\langle m, n \rangle$ for the number $(2n+1) \cdot 2^m$; every positive number is $\langle m, n \rangle$ for exactly one pair, and 0 is no pair. Number the small machine's properties Zero $\mapsto$ 0, Even $\mapsto$ 1, Ge(n) $\mapsto$ $n+2$. The checker of Slot says yes at a value x exactly when $x = \langle m, v \rangle$ and the small machine's checker says yes to the property numbered m at v; its meaning is the same with holds in place of the checker.

There's one place the guest's machine shows up in the host. The host's single property knows how to read the small machine's three-word property language: "this number is the pair (property, value) and the property holds of the value." It doesn't know the small machine's step. Knowing the language and not the step is the counter version of Turing's universal machine knowing the alphabet its guests write in.

Two facts that force the design. Why more counters? The answer is two facts, and the program looks the way it does because of them.

The first is about reading. On this machine a program that wants a counter's value in its own control flow has to take pebbles out (DEC) and put them back (INC), and every INC, and every DEC on a counter above 0, is a write, so the counter's version goes up. A fact is about one version, so a read done that way kills every live fact about the counter. Not every look is a write: CHECK reads a value without writing it, a DEC on 0 changes no version, and a fresh CHECK after such a read can make a new live fact. But the guest's counters are exactly the ones the host has to read by taking pebbles out, again and again, to decide which way a DEC jumps and to work out a property's answer, and each of those reads kills whatever fact was kept there. So the host can't keep the guest's facts on the copies it reads, and a copy it only writes is what the second fact rules out.

The second says that even if reading wrote nothing, a fixed program couldn't keep them by sending each guest property to one of a fixed, finite list of host properties and checking it on the same register the guest's fact is about. The theorem below rules out that and nothing more, and the pigeonhole is what rules it out.

□ **Theorem C.1** (No host fact on an exact copy of the guest's counter)**.** Take any host property language with an exact equality test and any checker, a finite list Q of host properties, and any translation of the small machine's properties into members of Q whose host check passes whenever the guest's check passes on the same number. Then there are $n \neq m$, with $\mathrm{Ge}(n)$ and $\mathrm{Ge}(m)$ translated to the same host property, such that the guest program CHECK $\mathrm{Ge}(n)$ A; COMMIT $\mathrm{Ge}(m)$ A traps within two steps from every start, while on every host state with the trap down and room in its table, and some register holding a number at least n, the translated CHECK and COMMIT on that register both pass and put the committed claim in the channel.

So the host keeps the guest's facts somewhere else: in *slots*, extra registers used for nothing but holding facts, one per checked fact, each holding the pair (property number, value) that the guest checked. The pigeonhole doesn't bite a slot, because the property lives in the slot's number. The instruction doesn't have to carry it: CHECK Slot on a slot asks about whatever pair is sitting there, and there are infinitely many pairs. A slot's number changes only when the program is moving a fact into it. And when the guest writes the counter a fact was about, the program gives every slot about that counter a nudge, one INC and one DEC, which leaves the number alone and pushes the version up, so the host's facts go stale at exactly the moment the guest's do.

The stored fact still names edition v. The two writes restore 18 but do not restore that edition. The intermediate row is inside the handler; the simulation promise is at the completed handler's endpoint.

Count what costs anything. Fetching, unpacking, copying, nudging and branching are all INC and DEC, and all cost nothing, as computing does on the small machine. The only paid instructions are the CHECK, COMMIT and CERTIFY sites, and each guest CHECK, COMMIT or CERTIFY makes the program run exactly one of them. So the host's ledger lands on the guest's, unit for unit. And when the guest's flag goes up, the host's goes up by the host's own CERTIFY, standing on the host's own COMMIT and CHECK on a slot. The host isn't copying a bit; it runs the same chain on its own registers.

Definition C.3 (The program U, and loading a guest)**.** U is the fixed list of 3728 host instructions specified by its block expansions, starting at line 1 and mentioning registers 0 to 80; exactly 69 of them cost anything. For a guest program P and a start (x, y), the *loaded host* $\mathrm{hload}(P, x, y)$ has register 0 at x, register 1 at y, register 2 at the number of P, register 3 at 1 and every other register at 0, every version 0, line 1, an empty table and channel, the trap down, ledger 0 and flag down. Write

grun(m) for m program steps of P from start(x, y), and hrun(n) for n program steps of U from hload(P, x, y).

Theorem C.2 (U keeps pace with its guest)**.** For every guest program P, start (x, y) and number of guest steps m, there's a number of host steps n such that one of two things holds. Either the host is back at line 1 with $m \leq n$ and *related* to the guest: registers 0 and 1 hold the guest's A and B, register 3 holds the guest's line, the guest's trap is down, the two ledgers are equal, the two flags are equal, and the host's table and channel hold, fact for fact, slot facts matching the guest's facts and channel. Or both have stopped, with the same counters, the same trap, the same ledger, the same flag, and matching tables and channels. And from any related pair where the guest hasn't stopped, one guest step is matched by at least one host step that lands related again, or with both stopped. □

Theorem C.3 (Halting and answers match)**.** For every P, x and y, the guest's run of P from start(x, y) halts if and only if U's run from hload(P, x, y) halts. And whenever the guest has halted at step m and U at step n, the host's registers 0 and 1 hold the guest's A and B, and the two traps, the two ledgers and the two flags are equal. □

Theorem C.4 (The host's flag is the guest's, and its ledger is exact)**.** (1) The guest's flag is up at some step of its run if and only if the host's flag is up at some step of U's run. (2) For every m there's an n at which the two are related, with $m \leq n$, or both stopped, with $\mu(\text{hrun}(n)) = \mu(\text{grun}(m))$. And for every n there's an m with $\mu(\text{grun}(m)) \leq \mu(\text{hrun}(n)) \leq \mu(\text{grun}(m+1))$: in between matching points, the host's ledger never gets ahead of the guest's next step. □

Equal ledgers at the matching points are a third sense of "exact," and the plainest of the three (Section 15 names all three): two ledgers holding one number.

Theorem C.5 (The host earns its own flag)**.** If the host's flag is up after n steps of U from hload(P, x, y), then the instructions U executed contain, in order, a passing CHECK Slot on a slot, then a passing COMMIT Slot on the same slot, with the slot untouched and its version unchanged in between, then a passing CERTIFY; the slot held $\langle$number of p, $v\rangle$ at the CHECK, with holds(p, v). And the guest's own run from □

start(x, y) contains a passing CHECK p c, then, with c untouched and its version unchanged, a passing COMMIT p c, then a passing CERTIFY, with c holding v at its CHECK.

□ **Theorem C.6** (The host is certified, and U's halting is undecidable)**.** (1) The host machine, with Slot as its property and one move per instruction, is certified in the sense of Section 14.15, over its own claim language, whose claims are "Slot holds of register r," whose meanings are written without the host's checker, and whose "unchanged" is "same value and same version of register r"; and U's run from hload(P, x, y) is a run of it. (2) Read as a Thiele machine it obeys the universal floor; a run of U that raises the host's flag costs at least 3 there. (3) U halts from hload(P, x, y) exactly when P halts from start(x, y), so the question "does U halt from this loaded host?" is undecidable in the reduction sense of Theorem 15.5.

Put it together, plainly. One fixed program, 3728 instructions of the small machine's six kinds, with no guest step built into anything, runs every program of the small machine from every clean start start(x, y). It halts exactly when its guest halts, with the guest's answer in its first two registers. Its flag goes up at some step of its run exactly when the guest's goes up at some step of the guest's run, by its own check, its own commit and its own certificate. Its ledger is the guest's, unit for unit, wherever the two line up. And the machine it runs on is certified. Since the small machine runs every two-counter program, through those guests U runs every computation, under Minsky's 2^n encoding, done outside the machine. That's a universal Thiele machine for the small machine's programs, and the definition allows any other host that meets it; the next subsection carries it to every computably presented Thiele machine whose reading a program decides on every state.

There's fine print, and it comes before you take that anywhere. The guests start clean, from start(x, y); the whole-run theorems are about loaded hosts, and only the one-step theorem is about any related pair. The host's property reads the small machine's three-word language, so a guest with a different property language needs a different host property; the host of the next subsection has one, a single property that reads a universal claim language. Sixteen slots per counter is the guest's whole table size, reserved for each counter in case every fact is about that one. And it's slow: for every guest step it unpacks the program number from the front again, and it copies a number one pebble at a time. That's where its running time comes from. The theorems above say nothing about time; they match states and ledgers at the points where the two line up.

Enforcement in the step is the reference monitor's move from Section 14, Anderson's 1972 move: one mechanism everything has to pass through. What's put

together here is a priced, earned record under that move, carried to every guest by one fixed program. The move itself is Anderson's. Kevin Hamlen, Greg Morrisett and Fred Schneider (2006) worked out, in their model, which policies mechanisms like that can enforce at all, and their paper is where I'd go to see why enforcing a property of every run escapes the Rice-style walls that deciding one runs into.

Why make such a fuss about proofs that walk every case? Because the first universal machine had bugs. It's the machine in Turing's 1936 paper, and nobody gets to call Turing careless. In 1947 Emil Post, in an appendix to a paper on a problem of Thue, pointed out that the construction leaned on a convention it hadn't earned, and he patched a small defect in how the universal machine writes out each complete configuration. Around the same time Donald Davies, who joined the National Physical Laboratory in 1947, where Turing was designing a computer, worked through the universal machine's tables and found more mistakes, much to Turing's annoyance, as the accounts of Davies's life have it. (It's the same Davies who later named the *packet* in packet switching.) His corrections were finally printed in a collection edited by Jack Copeland, a philosopher and historian of computing who runs an archive of Turing's papers, *The Essential Turing*, in 2004. They may be some of the first bugs ever written, in a program for a computer that didn't exist yet. That's a lovely claim, and I haven't been able to confirm it, so take it as a lead. What it changes here: a universal machine is exactly the kind of construction where a careful person leaves a hole, U has 3728 lines for a hole to hide in, and each case of its simulation argument has to hold. The definitions are on the page so you can examine what those cases establish.

C.2 Every computably presented Thiele machine, on one host: driven, decided, priced in whole units, carried as a latch

U runs the small machine's programs, and through them every computation, under Minsky's 2^n encoding, done outside the machine. But a Thiele machine from somewhere else has its own states, its own moves, its own prices and its own reading, and running it through the two-counter encoding keeps its computation and nothing else. So the question Turing's machine raises for this book is the bigger one. Take any Thiele machine whose workings can be computed. Is there one fixed program that runs every such machine as a guest, keeping its halting, raising its own flag exactly when the guest's reading would rise, on an earned chain of its own, and keeping a ledger equal to the guest's own prices, with the host's flat ones nowhere in it? There is, within limits I'll list, and this subsection gives it.

First the class, said plainly. A *computably presented* Thiele machine is a Thiele machine in Section 2's sense (states, moves, a step, a price for each move, a yes/no reading, and the toll: a move that turns the reading from no to yes costs at least one)

together with a *driver* that picks the next move from the current state or says stop, and a way of writing states and moves as whole numbers. “Computably” means four little programs, written in the plainest language of computable functions there is, the one built from zero, plus one, picking an argument, composing, recursion and search: one computes the driver, one the step, one the price, and one the reading. The driver, step and price routines are required to halt and give the specified result on valid state and move codes; their behavior on other numbers is not constrained by the presentation. The reading routine is required to halt on every number, returning the reading on valid state codes and no on other numbers. State and move encodings have left-inverse decoders, so distinct states or moves cannot share a code. Run from a start s_0, the driver's moves give one run. Its *ledger* after n moves is the sum of their prices, and its *latch* after n moves says whether the reading has been yes at any state so far.

The host gets one thing the small machine doesn't have: a paid move that certifies nothing. On the small machine every charge comes from a CHECK, a COMMIT or a CERTIFY, and those can charge without raising the flag: one passing CHECK and six COMMITs of the same claim cost seven. But every passing CHECK takes one of the table's sixteen entries for good, every COMMIT rewrites the channel, and a failing CHECK traps. So the host machine gets a seventh instruction kind, PAY: it costs one, goes to the next line, and changes nothing else; it never raises the flag, takes no table entry and leaves the channel alone. That's why the compiler uses it.

How far the small machine gets without it is proved, and the count is short. Take a run that never traps and ends with the flag down. No CERTIFY ran in it, since a passing one raises the flag and a failing one traps. Every CHECK in it passed, since a failing one traps, and each passing CHECK filled one of the table's sixteen entries, so there were at most sixteen. Every COMMIT in it committed a claim the table holds, and the table holds at most sixteen, so at most sixteen COMMITs can be of claims the run hadn't committed before. Sixteen and sixteen make 32, and the only way past it is a COMMIT of a claim already committed earlier in the run. A claim names its counter's version, and the version changes at every write, so a repeat needs its counter left alone since the earlier COMMIT. A guest that writes both counters between any two of its payments can't do that, and it can't commit at all, since a COMMIT needs its counter left alone since its CHECK, so it runs out of ways to pay after sixteen. Without that limit, 32 is reached: sixteen rounds of CHECK *at least 0* A; COMMIT *at least 0* A; INC A pay 32 with no trap and the flag down. A guest that writes both counters at every move of the machine it simulates needs a fresh CHECK for each move it pays for, so it can pay for at most sixteen of them. A simulated machine that charges more often than that has to be paid for somewhere, and PAY is where. The 32 isn't a ceiling on either machine: a program that checks one fact about a counter it never writes can commit that fact again and

again, at a cost of one each, with the flag down and no trap, because the counter's version never changes. On the small machine that leaves one counter to compute with while it repeats, which is too few; the host has a counter for every whole number, so on it the repeats cost the computing nothing. A program written for the host's registers doesn't need PAY: it checks one true fact once, about a register it never writes, and pays every later unit by committing that fact again. Those commits overwrite the channel, so at the move where the reading first says yes it runs the chain's CHECK, COMMIT and CERTIFY back to back and pays the rest of that move after the CERTIFY. U_P can do without it on its class, and PAY is what it needs beyond the class. U_P names no register above 80 and its guest can't name a host register, so two sit untouched. Pay the first unit by raising one of them to 1, which reads "zero holds at 0," and checking it; pay every later unit by committing that fact again, which passes because the register's version never moves. A compiled guest runs its chain back to back and pays after it, so the chain's own commitment is what its certificate finds, and its one check leaves the table at two facts. Every payment costs one, the surcharge is unchanged, and the host is the register host with no seventh instruction. Outside the class the trick fails: a guest that pays twice and then certifies with nothing committed traps, and this host, with the pay-fact sitting in its channel, would raise its flag instead. PAY leaves the channel alone, and that is what those guests need. On a host without certification PAY is a necessity: a guest that pays while its reading is down makes the host run a paid move that doesn't raise its record.

Read as a Thiele machine, the host needs no filing at all: PAY is a paid move that never raises the record. Certified, as Section 14.15 pins it down, sorts every paid move into CHECK, COMMIT or CERTIFY, so PAY has to be filed under one of them, and it's filed as a CHECK of the empty claim, a claim that never holds. The filing is for the interface. It isn't what PAY does on the machine: a CHECK that fails on this machine traps, and PAY goes on to the next line. With PAY filed that way the host stays certified, over its own claim language, in which the empty claim holds of no state. PAY costs exactly one, as the third of the four asks of a CHECK; its check never passes, so it can't stand in a chain; and it leaves the table, the channel and the flag alone, so it can't add to a chain or raise the flag either. For the same reason the floor of three for a flag raised from a clean start still holds with PAY present: the flag still needs a CHECK, a COMMIT and a CERTIFY.

The program is U with one more handler: a seventh branch on the guest's opcode, for PAY, that runs the host's own PAY once. Call it U_P; it's 3735 instructions, with 70 paid sites, the 69 of U plus the PAY. The guest it runs is a program of the small machine with PAY added, and that guest is where the presented machine goes. Its four programs are compiled into two counters, by the packing Minsky used, using the standard translations from computable functions to counter programs; the guest

keeps the presented machine's state coded in its counter B, takes one of its moves by running the compiled driver and step, runs the compiled reading on the new state, and pays the move's price in PAYs; the first time the reading says yes, it checks, commits and certifies, and those three paid instructions count toward that move's price. The host's one property is fixed too: a claim is either one of the small machine's three words about a number, or "run reading program r on the coded state v," and the host's checker evaluates that with one fixed routine for every presented machine there is. So nothing in U_P knows any particular presented machine. The machine arrives as data, the way Turing's guests arrived on the tape.

A fixed checker that runs whatever program it's handed and always answers needs saying how, so here is the routine. A claim of the second kind carries a number r. The checker decodes r as a counter-machine routine: its instructions, where they sit, which register holds the answer, which register holds the fuel, and how many registers to load. If r doesn't decode to a routine, the answer is no. Otherwise the checker loads the routine's registers from v, reads a fuel number f out of v too (the fuel register's entry in v's prime factorisation), runs the routine for at most f steps, and says yes exactly when by then the routine has left its code with 1 in its answer register. Every input gets an answer within f steps, so the checker is total, and what it means is that bounded run: yes if and only if the routine halts within the fuel the value carries, with answer 1. That's how it stays clear of the diagonal. A total evaluator E that agreed with every total yes-or-no program on that program's code alone would let $g(n) = \neg E(n, n)$ contradict itself at its own code, as long as E is itself computable. This checker never evaluates a program on its code alone: the fuel comes with the claim's value. The guest compiled from M runs M's reading routine itself and puts at least the step count it took into the fuel register, and at such a value the checker says yes if and only if the reading says yes.

For the fuel rule, take a routine containing just one instruction: increment an answer register initially holding 0, then leave its one-line code. Give it fuel 0 and the check says no: it has not completed. Give it fuel 1 and the check says yes: it has left the code with answer 1. The first no did not mean the routine would never answer yes. It meant "not within this allowance." You can run that distinction with a pebble and one instruction card. The real checker does the same job for any list of instructions, with the allowance handed in alongside the claim.

□ **Theorem C.7** (One host runs every computably presented Thiele machine)**.** Let M be a computably presented Thiele machine, so that a driver picks its next move (one run per start), a program decides its reading on every state, and its prices are whole numbers; and let s_0 be a start state. Load U_P with the guest compiled from M's presentation and s_0 coded in. What the host carries of M's reading is its latch.

Then: (1) U_P halts exactly when M's run stops. (2) For every n such that M hasn't stopped before move n, there's a host time t at which the host's register for the guest's counter B decodes to M's state after n moves, the host's ledger is M's ledger after n moves plus a *surcharge*, and the host's flag is M's latch after n moves; and if M stops at n, such a t has U_P halted as well. (3) The host's flag is up at some time exactly when M's reading is yes at some state of its run. (4) If the host's flag is up at time t, the host's own instructions up to t contain a passing CHECK on a slot, then a passing COMMIT on the same slot with the slot untouched in between, then a passing CERTIFY; and the slot held the claim "reading program r says yes of v" for M's own reading program r, that claim holds, and v decodes to a state of M's run where the reading is yes.

There are two things the theorem doesn't say. First, the host's flag is M's latch. If M's reading says yes at some state and no at a later one, the host's flag stays up, so it says the reading was yes somewhere in the run, and it doesn't say the reading is yes now. Second, what the host re-establishes is the reading: the claim it checked is "M's reading program says yes of this coded state," and that's all. A bare Thiele machine needn't tie its reading to anything true about sorting, proofs or the world outside. If M's reading is meant to say a list is sorted, that it does is a separate correctness theorem about the reading, and the host doesn't supply it. The two universal programs also keep their own guests: U runs the small machine's programs over its three words about numbers, and U_P runs computably presented machines over the universal claim language.

The surcharge comes next, because that's where the ledger stops being exact, and below three units it can't be helped. The host raises its flag the only way the host machine allows, by a CHECK, a COMMIT and a CERTIFY, and those cost three. If M's first move that turns its reading to yes costs at least three, the host pays the difference in PAYs and nothing more is owed. If it costs less, the host pays three there anyway, which is the larger of three and the move's price. The surcharge starts at that move: while M's latch is down it's zero, so a run that never certifies is matched exactly. Whether anything could do better depends on what M paid before. When M's whole ledger at the raise is below three, no program on the host machine can match it, since a raised host flag costs three from a clean start. When M's ledger at the raise is three or more, this compiler isn't always the cheapest. Take a guest whose first move costs two and leaves its reading at no, and whose second costs one and turns it to yes. This compiler carries it at 2 and then 5, against the guest's 2 and 3. A program that looks ahead carries it exactly: inside the first move's price of two it works out the guest's state two moves on, puts it on B, runs the CHECK and the COMMIT of the reading's claim about it, and puts the guest's state back; inside the second move's price of one it runs the CERTIFY, which needs only a fact in the

channel, and nothing has committed since. Its totals are 2 and 3, and its flag stands on the guest's reading at the first state where it said yes. Part (1) of the theorem below bounds this compiler, and part (3) is the only lower bound proved.

□ **Theorem C.8** (Exact, within two, and no better)**.** Let M, s_0 and the loaded host be as in Theorem C.7, and let M's reading be no at s_0. (1) For every n before M stops there's a host time at which the state is decoded and the flag matched as before, and the host's ledger is M's ledger after n moves while M's latch is down, and that plus the larger of $3 - c$ and 0 once it's up, where c is the price of the move that first turned M's reading to yes; so the surcharge is at most two. (2) If moreover every move that first turns M's reading to yes costs at least three, the surcharge is zero and the host's ledger equals M's exactly at those times. (3) If M's latch is up after n moves while its ledger is below three, then no run of the host machine from a clean start, under any program at all, ends with its flag equal to that latch and its ledger raised by exactly M's ledger. (4) If instead M's reading is yes at s_0, then at the host time matching each move n before M stops, the host's ledger is M's ledger after n moves plus exactly three.

Part (3) is an observation in the sense of Section 2: it reads off Observation 15.19, which holds on the host machine with PAY for the reason given above. Part (4) is the same three paid at once: if the reading is already yes at the start, the host has to raise its flag before M has paid anything, and at that point no program can do better, since M's ledger there is nothing. After it the three stay on only because this compiler doesn't count them against M's later prices.

And one machine is carried all the way, by proof, so the class isn't empty. Its states are whole numbers, its one move adds one and costs three, its driver always picks that move, and its reading is "at least three." The driver returns the fixed move; the step adds one; the price returns three; and the reading compares the state with three. Counter programs implement all four. Loaded into U_P from 0, the host never halts, its flag goes up, at the host time matching move n its ledger is exactly $3n$ and its flag is up exactly when $n \geq 3$, and the raised flag stands on a host chain whose slot claims the machine's own reading of a state at least three. That follows from the two theorems above: the reading is no at 0, and the one move that turns it to yes, from 2 to 3, costs three, so part (2) of the surcharge theorem makes the ledger exact.

Now the limits, each of which is in the statement, said plainly.

- **Readings that are decided.** The reading's program has to answer yes or no on every state. A reading a program could only confirm, waiting forever on the no cases, isn't covered, and neither is a reading no program computes at all.

- **One run per start.** The driver picks the move, so the guest is deterministic. A machine whose next move is somebody's unforced choice gets in only with that choice written into its driver.
- **The record is the latch.** The host's flag is up once M's reading has been yes at any state of the run, which is exactly the latch of Section 11. A reading that goes yes and then back to no is carried as "it was yes once."
- **The ledger is exact up to the surcharge.** When the reading starts at no, at most two, and zero when the first raising move costs three or more; when it starts at yes, three. And no program on this host, run from a clean start, can do better when the guest certifies with its whole ledger below three.
- **Prices in whole units, paid one at a time.** A move that costs a million is a million PAYs. The compiler also uses Minsky's packing. Between them, those two choices set how long the host takes to pay for the guest's moves.
- **Proved; never run on a presented machine.** The demonstration machine's numbers come from the two theorems above. U_P itself has run on guests small enough to finish; a presented machine's compiled guest is far too large for that, and nobody has run U_P on one.

APPENDIX D

Built from parts

Back in Section 2 I asked a question and only got halfway: bolt two Thiele machines together, and do you get one? Everything real is built out of parts. A computer is a processor and memory and a disk, a bank is branches, a proof is lemmas. If the record and its toll only held for one machine at a time and fell apart the moment you put two together, they'd be a property of toys.

So put two Thiele machines on a bench, each box with its own panel, and wire them up the three ways anybody wires machines. Side by side, with no wire between them at all, so that every move is a lever pulled on one box or the other. End to end, with the first box's bins tipped into the second's when the first is done. And one inside the other, with the inner box running as a guest on the outer one. Some of it holds and some of it provably doesn't, and the places where it doesn't were where I learned the most.

D.1 Side by side

Two boxes on the bench, no wire between them. A move of the pair is a lever pulled on one box or the other. The question is what the pair's record should be, and my first answer was the wrong one: a single count of how many certifications the bench has made, one number, easy to read. I'll come back to why it fails, because it's in the list of things that break. What works is to keep both panels showing. The pair's record is the pair of records, a place for the left machine and a place for the right one, compared one side at a time.

Since that record has more than two places, "certified" in this appendix means the four things of Section 14.15 with the record allowed to be a place along a direction, the way Section 4 set it up. Read on a direction, clean states all sit at one place, the floor; a record never moves below where it was; and in the second of the four a raised record is one that isn't at or below where the run began, with the CERTIFY the move that first takes it out. On two places this is exactly the two-place reading.

Theorem D.1 (Side by side)**.** (1) The cost of any run of the pair is the left machine's cost of its moves plus the right machine's cost of its moves, and the order in which levers on the two boxes were interleaved doesn't change where the pair ends up or what it paid. (2) If both component state spaces are nonempty, the pair obeys the toll exactly when both machines do. (3) Read through "either flag is up," two Thiele machines side by side are a Thiele machine with floor one; read through "both flags are up," a run that starts with both flags down and ends with both up costs at least two, and two is reached when each part can certify for one. (4) Two certified machines side by side are a certified machine. (5) In that pair, call a side's *floor* the place on its record axis where its clean states sit. A run from a clean start that ends with each side at or above a point that isn't at or below that side's floor makes at least six record moves and raises the ledger by at least six. Six is reached, from suitable loaded starts, by a three-act certificate on each side; a particular pair of target points can need more. (6) The pair of records is the product in the precise sense: for any machine mapping into both, preserving records, there's exactly one map into the pair that does both jobs. □

Proof. (1) A move of the pair is a lever of one box. It moves that box's component by that box's step, leaves the other component alone, and costs what that box charges for it. So the pair's run is the pair of the two boxes' runs along the projections of the trace, each box's moves in the order they came, and the cost is the left projection's cost plus the right's. Two interleavings with the same two projections end in the same state at the same cost. (2) The toll prices a step that takes the record out of the down-set of where it stood. A pair is at or below another when both sides are, so the down-set of a pair is the two down-sets side by side, and a move of the left box leaves the pair's down-set exactly when it leaves the left down-set, the right side not having moved. So the pair's exits are the left exits and the right exits, each priced by its own box, and the pair obeys the toll exactly when both parts do; for the converse, an exit at zero cost on one side is an exit at zero cost of the pair with any state at all on the other. (3) Read through "either flag is up," a step from neither to either raises one side's flag, and that side charges it at least one, so the pair is a Thiele machine with floor one. Read through "both flags are up," a run from both down to both up raises each side's flag on some step, no move touches both, and each such step costs at least one; when each part can certify for one, the two certificates one after the other cost two. (4) Give the pair the interface that reads a move as its own box reads it, takes a claim to be a claim of one side, about that side's component and unchanged when that component is unchanged, with both sides clean as clean, both sides loaded as loaded, the sum of the ledgers as ledger, and the left box's

window as window. Clause (a) is the left box's clause (a), since a left base move acts on the left window as its instruction does and a right move doesn't touch the left component, and no base move of either side changes either record. Clause (c) is the two price lists added. Clause (d) is the left box's witness claim with the right box idle. Clause (b) is the one with work in it. Take a run from both clean that ends with the pair's record raised, and cut it just after the first move that takes the pair's record out of where it began. That move is one side's, and since pairs compare side by side it takes that side's record out of where that side began. Project the cut run onto that side: a run from clean that ends with that side's record raised, so by that side's clause (b) it contains that side's chain, a passing CHECK of a claim c, a COMMIT of c with c's component unchanged in between, and a CERTIFY. Put the other side's moves back where they were. None of them touches the component c is about, so the chain is a chain of the interleaved run. (5) If a side ends at or above a point that isn't at or below its floor, its record ends not at or below its floor either, or the point would be at or below the floor by passing through it. The clean start puts that side at its floor, so the projection onto that side is a run from clean with its record raised, which by that side's clause (b) has three record moves of its own, a CHECK, a COMMIT and a CERTIFY. The two sides' moves are different moves, so six record moves at least, and by (c) a ledger rise of at least six; a three-act certificate on each side, from loaded starts where each side's witness claim holds, makes exactly six. (6) A record-preserving map from a machine Z into a box is a map of states with a monotone map of records, the record of the image being the image of the record, and every move of Z simulated by some run of the box. Given one into each box, send a state of Z to the pair of its images. The record of the pair is the pair of the records, and a move of Z is simulated by the left box's run followed by the right box's, which is a run of the pair by (1). That map followed by either projection is the map you started with, and any map into the pair with those two projections agrees with it on every state, which is what equal maps means here. ■

Most of that is what you'd guess from the bench. Each box keeps its own ledger and nobody else pulls its levers, so the costs add, and it can't matter which box you happened to work on first. The six in (5) is two certifications costing three each, one per box, with no discount for doing them on the same bench.

Machines, with maps that keep the record, form a category, and item (6) says these pairs are its products. Ask the maps to keep the cost too and the pair stops being a product: a machine mapped into both boxes would pay its cost once on each side, twice in all. What survives is a symmetric monoidal structure, which is all that side by side needs. Its pentagon, triangle and hexagon laws say that regrouping four boxes, inserting an idle box, or exchanging groups of boxes gives the same final wiring whichever permitted route you take. Label the boxes and follow each wire:

every rewiring only moves boxes around, so both routes put each wire at the same labelled terminal, at the same cost.

That paragraph is the most formal way I know of saying "side by side composes, and nothing about the order of the parts matters." It's a lot of category theory to confirm what the bench told me in a glance, and I'm glad to have it anyway. A glance is where I'd have missed something.

D.2 One after the other

Now wire them end to end. Run the first box, and when it's done, tip its two bins into the second box's start, at no charge on the ledger, and run the second. Programs get chained that way everywhere, and my first thought was that the tipping couldn't matter, because it's plumbing: pebbles go from one bin into another, nobody checks anything, and plumbing has no business on a record. That turned out to be half right, and the wrong half is item (4).

Theorem D.2 (One after the other)**.** (1) Running one certified machine and then handing its two counters to another through loads that cost nothing gives a certified machine. (2) For any two programs of the two-counter machine, it computes the composite: some schedule leaves the first machine's window where the first program takes the input, and the second's where the second program takes that result. (3) The hand-off keeps the toll when every state it can load into sits at one level of the second machine's record. (4) If some load lands the second machine at a record that isn't at or below another load's, the hand-off loses the toll: it's a certification nobody paid anything for. □

Proof. (1) Give the second machine a loadable form. A state is either pristine, holding the second machine's loaded state for some (a, b), or running, a state of the second machine. A move LOAD (a, b) turns a pristine state into the pristine state for (a, b) and does nothing to a running one; the second machine's own moves move a running state as before and turn a pristine state into a running one. LOAD is a base move and costs nothing. When the second machine is certified its loaded states are clean and stand at the floor, so a LOAD leaves the record alone; the base, the toll and the witness claim are the second machine's; and the chain in front of a raise lies after the last LOAD before the second machine's first move, so it lifts through the projection onto the second machine's own moves. So the loadable form is certified, and the composite, the first machine side by side with it, is certified by Theorem D.1(4). (2) A wired schedule is the first machine's moves, then a LOAD of the two counters its window shows at that moment, then the second machine's

moves. The first stretch runs the first program on the first window; the LOAD puts the second window at $(1, a, b)$ for the counters (a, b) the first stretch left; the last stretch runs the second program on it. If both programs halt, the two windows after the schedule are the two halting configurations, the second's on the first's output, and the wire cost nothing. (3) and (4) are one fact read two ways. A LOAD from the pristine state for one input to the pristine state for another costs nothing, and carries the record from where the first loaded state stands to where the second stands. If every loaded state stands at one level, no LOAD moves the record, the composite's exits are the two machines' own, and the toll is kept by Theorem D.1(2). If some loaded state stands at a record not at or below another's, the LOAD from that other one to it, at zero cost, leaves a down-set, which is an exit nobody paid for, and the toll fails. ■

Item (4) is the one to remember. The tipping costs nothing, so it had better not tip in anything that counts as established. If pouring pebbles into the second box can land its panel somewhere that isn't at or below where some other pouring would land it, the second machine gets a raised record without ever having checked anything, and that's a forged certificate passed off as plumbing. My first thought was right about the pebbles and wrong about where they land.

D.3 Connecting records and writers

Joining two boxes also means deciding how to read their flags and who may write their workspace. The following examples trace those connections and the guarantees they affect.

□ **Theorem D.3** (What breaks)**.** (1) Read the pair through "both flags are up" as a single flag, and any run that raises it from a clean start makes at least six record moves; certified asks for a claim that three moves can certify, so read that way the pair isn't certified. (2) Two clocks side by side obey the toll with a two-counter machine inside. No move of the pair costs nothing, so composing that easy bar composes nothing. (3) Give the small machine a second writer that changes counter A without raising its version, and the run "check that A is 0, let the other writer change A, commit, certify" raises the flag, so the earned-record clause fails when the machine is read the small machine's way, with the second writer as a base move. (4) Two windows that each price their own machine exactly can collide as a pair, so exact pricing isn't preserved by putting machines together. (5) Count the pair's certifications with one number, and the count can't tell which side certified. The pair of records can.

Proof. (1) Through the pair's interface, a run from a clean start that raises "both flags are up" ends with each side's record raised, and by Theorem D.1(5) that takes at least six record moves. Certified asks for a claim whose CHECK, COMMIT and CERTIFY, those three moves alone, raise the record from a loaded state. Three moves aren't six. (2) A clock is the two-counter machine with every move priced at one and a reading over it. Two side by side have a two-counter machine running inside, the left's, and obey the toll by Theorem D.1(2); and every move of the pair is a move of one clock, costing one. (3) Add to the small machine a writer that changes counter A and leaves its version alone, read as a base move. The run CHECK (A is 0), the writer's move, COMMIT (A is 0), CERTIFY, from a start with A at 0, passes its CHECK, finds its fact at the unchanged version at the COMMIT, and raises the flag at the CERTIFY. The base and toll clauses still hold, and so does the checker's promise. What fails is clause (b). The small machine reads "unchanged" as same value and same version, and the only chain in that run has the writer's move between its CHECK and its COMMIT, where counter A, the thing the claim is about, changed its value. The COMMIT didn't notice, because it looks only at the version, and the version didn't move. So a run from a clean start raises the record with no earned chain in it. (4) Take a left box whose only step its window doesn't see is a certifying one, so the window still prices it exactly, charging that unseen transition one; and a right box whose only step is a stutter, at zero cost, leaving its down-set alone and its window unchanged, priced exactly at zero. In the pair, the left box's unseen certifying step and the right box's stutter show the same pair of windows before and after. One costs at least one and leaves the pair's down-set; the other costs nothing and doesn't. So a price computed from the pair of windows is short on the first or over on the second. (5) Replace the pair of records by the number of sides that have certified. A run in which only the left side certifies and one in which only the right side does both reach the count one, with the left record raised in the first and at its floor in the second. The count can't tell them apart; the pair of records can. ∎

Item (1) puts one flag over two boxes. It costs what two certifications cost, six record moves, and the definition wants a claim that three can settle, so a single flag over both panels, read through the pair's interface, is too expensive to meet the definition. That holds for this interface; nothing here shows that no other interface on the pair could meet it, and Theorem D.1(4) shows the pair meeting it through a richer record. Item (4) says that a window exact for each box alone can be fooled by the pair: two pair states with different costs can show the same pair of readings.

Item (3) is the one with the most teeth, because it's the bug from Section 14, the file checked and then swapped. On the bench it's a second hand reaching into a bin while the box isn't looking: two writers on one value, and a check that's out of date

by the time anybody leans on it. The small machine's version rule is exactly the fix, and this is what happens without it: the commitment goes through on a claim about a thing that changed underneath it. Database people and the people who build processors have been fighting this one since at least Kung and Robinson's paper of 1981 and the Livermore report of 1987, and that section gave it the security people's name, time of check to time of use. Here it's a theorem about the record, and the version number is what makes it go away. The second hand can still reach in, but with the version its reach shows on the tally, and the COMMIT traps.

Item (5) is my first guess from the side-by-side wiring, coming back to fail on schedule. It's why the pair's record compares one side at a time. Count only how many sides certified and the record forgets which side did it. That count answers "how many?" but cannot answer "did the left side certify?" An ordinary number could encode the full pair too: use 0 for neither, 1 for left only, 2 for right only and 3 for both. It is the counting rule that loses the distinction, not the fact that its answer is one number.

D.4 One inside another

The last way to wire the bench: put one box *inside* the other, as a guest on a host. Part of this the book already has. The appendix on the universal machines (Appendix C.2) showed the fixed program U_P runs any computably presented Thiele machine, and the small machine's own programs, with the record earned again by the host. What's new is putting those links in a row, a box in a box in a box, and asking what the outermost one pays.

My first guess was that it could pay less, even nothing for the guest's climbs, since a host that's clever about how it runs the guest could skip the guest's costs. The first item below puts a floor under that: the host pays at least one for each step where the guest's record went up. It's a floor on rises, and it doesn't keep the guest's whole cost.

□ **Theorem D.4** (Nesting)**.** (1) Let a host that obeys the toll simulate a guest faithfully: each guest state has an image among the host's states and each guest record an image among the host's records, the host's record at the image of a state is the image of the guest's record there, and each guest move is carried by some host run from the image of the state before it to the image of the state after it. Let the record map also reflect the order: when the image of one guest record is at or below the image of another, the first record is at or below the second. Say a run of the host *follows* a run of the guest when it splits into one stretch of host moves for each guest step, each stretch carrying the image of the state before that step to the image of the

state after it. Every run of the guest has a host run that follows it, every such host run ends on the image of the guest's end state, and every host run that follows a guest run costs at least the number of the guest's steps that turned a cup over. A host run that only ends on the image of the guest's end state can cost less. (2) In each link the surcharge is 0 until the guest's reading first says yes, and the bounds below hold from then on. A small-machine guest on U_P is carried with surcharge exactly 0. A computably presented machine whose reading starts at no is carried, through its compiled guest, with surcharge at most 2, and, when every move costs at most one, exactly 2 once its reading has first said yes. One whose reading starts at yes is carried with surcharge exactly 3. (3) Links compose, and their surcharges add: a tower of k links has surcharge between the sums of their lower and upper bounds. (4) A tower of presented machines, each level running the next on U_P, adds exactly 2 for every level above the first, on top of the first level's own surcharge; every level's host run is presented by the next.

Proof. (1) The simulation gives, for each guest move, a list of host moves carrying the image of the state before it to the image of the state after it. String those lists together along the guest run: that host run follows it, and it ends on the image of the guest's end state, because each stretch ends where the next begins. Now take any host run that follows the guest run, and a guest step that turned a cup over, leaving the down-set of its record. Its stretch carries the image of the state before to the image of the state after, and since the record map reflects the order, the image after isn't at or below the image before, or the guest's record after would be at or below the record before. If every host move in that stretch ended at or below where it started, then chaining them, the end of the stretch would be at or below its start; so some host move in it ends not at or below where it started, leaving the down-set of where it stood, and the host, obeying the toll, charges that move at least one. Different guest steps have different stretches, so the charges add up to at least the number of such steps. The end alone isn't enough: a guest on one cup that turns it over and turns it back has one rise and ends where it started, and the host run that does nothing reaches the image of the end state at no cost. (2) A link from a guest to a host matches every point of the guest's run with a point of the host's, with the record bits equal there and the host's ledger the guest's plus a surcharge, the surcharge 0 while the record is down and between the link's two bounds once it's up, and halting and the record's first rising matching. A small-machine guest run by U_P is a link with surcharge exactly 0: that is the ledger equality of Appendix C.1, carried to U_P. A computably presented machine run as its compiled guest is a link with surcharge at most 2 from a reading that starts at no and exactly 3 from one that starts at yes, which is Theorem C.8; once the reading has first said yes the surcharge is the larger of $3 - c$ and 0, with c what the raising move cost, since the host pays

the larger of c and 3 there, and when every move costs at most one, c is exactly one, the toll making it at least one, so the surcharge is exactly 2. (3) Match the points through the middle: a point of the bottom run has a matching point of the middle run, which has a matching point of the top run; the record bits agree along the way; the top ledger is the bottom ledger plus both surcharges, which is 0 while the record is down and between the sum of the lower bounds and the sum of the upper bounds once it's up; halting and the first rising pass through. A tower of k links is that, $k - 1$ times. (4) Each level above the first is, by the presentation after this proof, a presented machine whose run is the level below's host run, carried as its compiled guest on U_P. Its moves are the host's moves, which cost 0 or 1, so by (2) that link adds exactly 2 once the record is up, and by (3) the additions stack on the first level's own surcharge. ■

Item (1) is the cleanest sentence in the appendix. However you build the outer box, as long as it obeys its own toll and doesn't lie about the inner box's record, it pays at least one for every step where the inner record climbed, on every run that keeps pace with the inner box step by step. The keeping pace is needed. A guest that turns a cup over and then turns it back ends where it started, and the host can reach that image by doing nothing at all, at no charge on its ledger. The step by step floor is the guest's rises paid one by one, and the end alone can't see them. It's a floor on the climbs, and it isn't the inner box's whole cost. A guest can charge a hundred for one rise while a host charges one for the matching rise, and both obey their tolls. Keeping the whole cost takes a further demand on the simulation, that it carry the guest's costs too, and for U_P that's the separate ledger theorem of Appendix C.2, with its surcharge.

Part (4) looks as if it needs a premise, and it doesn't. Two links are plainly unconditional: from a presented machine to its compiled guest, and from that guest to U_P running it. Going deeper needs the host's own run, at each level, to be presented as a machine the next level up can run. All the host's states together have no one-to-one code in one number, since there's a register for every whole number. But a loaded state has only finitely many registers that aren't at 0 with version 0, and no move changes more than one of them, so the states like that are a machine of their own, closed under every move, and each has a code: its line, its finitely many live registers with their versions, its table, channel, trap, ledger and flag. Programs that always answer compute its step, price and flag from that code, and the driver reads U_P's instruction off the line and says stop when the host has stopped. Its run is the host's run, and its toll is the host's. So every level is presented by the next, and the tower adds 2 a level as deep as you like. The 2 comes from the compiler's rule of stopping at zero; a compiler that carried the excess forward would add 2 at that level's raise and give it back as the level below paid on.

D.5 What this adds up to

Wired side by side, end to end, or one inside the other, a Thiele machine on the bench stays a Thiele machine, and the tolls add the way money adds. What breaks, breaks for reasons you could name before you saw the proof: a single flag for two claims costs two claims' worth, a clock is still a clock, a shared value without a version can't be trusted, and a hand-off that costs nothing can't be allowed to carry a certificate. None of those is a surprise in hindsight. All of them are now theorems, and the composition question from Section 2 has an answer. Two Thiele machines side by side are one under both readings I tried, "either flag" and "both flags." The "both flags" reading still fails certified through the pair's interface, the second writer breaks the earned-record clause through the small machine's own reading, the clocks and the two counterexamples fail outright, and simulation in general has a proved floor.

Three of my first guesses on this bench were wrong: the single count, the harmless tipping, and the clever host that pays nothing for a climb. The last two leaned toward a record that says more than anybody paid for, and the first toward one that says less than it should. I didn't see any of the three coming; the proofs did.

APPENDIX E

Sources

These are the works I leaned on, for their results, their history and the company they keep. A few words on how to read the list. The counter and lambda references are the constructions the universal programs stand on (Appendices C.1 and C.2); the credit for those constructions is theirs. The thermodynamics references don't take the physical premise out of the heat bound; it stays a premise. And the papers on records in the environment are where the pointer thesis got its shape; none of them proves it.

E.1 Structure, information and knowledge

Edwin A. Abbott (1884). Flatland: A Romance of Many Dimensions, by A Square. Seeley and Co., London.

Rudolf Ahlswede and Gunter Dueck (1989). Identification via channels. IEEE Transactions on Information Theory 35(1), pp. 15–29. DOI: 10.1109/18.42172.

David Z. Albert (2000). Time and Chance. Harvard University Press, Cambridge, Massachusetts. The name "past hypothesis" for an assumed low-entropy beginning; Stanford Encyclopedia of Philosophy, Thermodynamic Asymmetry in Time, section 3.1, Entry.

Hans Albert (1968). Traktat über kritische Vernunft. J. C. B. Mohr (Paul Siebeck), Tübingen. English translation: Treatise on Critical Reason, Princeton University Press, 1985. The Münchhausen trilemma.

Aristotle. Physics II.1; Metaphysics VII.3 and VII.11. Matter, form and substance; VII.11 distinguishes the material of a circle from the functioning parts of an animal (MIT Classics text). Also Metaphysics I.5 (985b–986b), on the Pythagoreans: "they supposed the elements of numbers to be the elements of all things" (trans. W. D. Ross, MIT Classics text); and Prior Analytics I.23 (41a23–30), the proof by contradiction that if the diagonal were commensurable with the side, odd numbers would equal even ones.

Sergei N. Artemov (1995). Operational Modal Logic. Technical Report MSI 95-29, Cornell University. The Logic of Proofs, the first justification logic, surveyed in the Stanford Encyclopedia of Philosophy entry on justification logic. Entry.

Sergei N. Artemov (2001). Explicit provability and constructive semantics. Bulletin of Symbolic Logic 7(1), pp. 1–36. Explicit evidence ("this is a proof") against implicit provability ("there is a proof"), and why reflection is harmless for the first.

Sergei Artemov and Roman Kuznets (2014). Logical omniscience as infeasibility. Annals of Pure and Applied Logic 165(1), pp. 6–25.

Mark van Atten (2018). The Creating Subject, the Brouwer–Kripke Schema, and infinite proofs. Indagationes Mathematicae 29(6), pp. 1565–1636. DOI: 10.1016/j.indag.2018.06.005; arXiv: 1805.00404. The reading of the creating subject is in Section 4.2, as read in the arXiv preprint; I didn't see the journal page.

Robert J. Aumann (1976). Agreeing to disagree. The Annals of Statistics 4(6), pp. 1236–1239. DOI: 10.1214/aos/1176343654. The partition model of knowledge, Section 8.

Steve Awodey (1996). Structure in mathematics and logic: a categorical perspective. Philosophia Mathematica 4(3), pp. 209–237. DOI: 10.1093/philmat/4.3.209.

Alexandru Baltag, Bryan Renne and Sonja Smets (2014). The logic of justified belief, explicit knowledge, and conclusive evidence. Annals of Pure and Applied Logic 165(1), pp. 49–81. DOI: 10.1016/j.apal.2013.07.005.

Thomas F. Banchoff (n.d.). Edwin Abbott Abbott (biographical sketch). PDF.

Michael J. Barany (2019). Making a name in mid-century mathematics: individuals, institutions and the open secret of Nicolas Bourbaki. Talk, History and Philosophy of Science, University of Cambridge, 17 October 2019. Abstract. Bourbaki's 1948 application to the American Mathematical Society and the second one a year later.

Pietro Baroni and Massimiliano Giacomin (2007). On principle-based evaluation of extension-based argumentation semantics. Artificial Intelligence 171(10–15), pp. 675–700. DOI: 10.1016/j.artint.2007.04.004.

Paul Benacerraf (1965). What numbers could not be. The Philosophical Review 74(1), pp. 47–73. DOI: 10.2307/2183530. Structuralism, Section 6.

Charles H. Bennett (1988). Logical Depth and Physical Complexity. In Rolf Herken (ed.), The Universal Turing Machine: A Half-Century Survey, Oxford University Press, pp. 227–257.

Max Black (1952). The Identity of Indiscernibles. Mind 61(242), pp. 153–164.

Boston University (2019). Leonid Levin, BU computer scientist, elected to National Academy of Sciences. Article. Kolmogorov as his advisor, his leaving the Soviet Union, and Boston University from 1980.

Nicolas Bourbaki (1950). The Architecture of Mathematics. American Mathematical Monthly 57(4), pp. 221–232. The mother structures. “Whose nature has not been specified” I have secondhand, from J. L. Bell’s “Reflections on Bourbaki’s notion of structure,” which reproduces the passage without a page.

Nicolas Bourbaki (1968; reprinted 2004). Theory of Sets. Chapter IV, sections 2–3: morphisms, derived structures and universal mappings. The table of contents already rules out saying Bourbaki omitted maps. DOI: 10.1007/978-3-642-59309-3.

Robert B. Brandom (1994). Making It Explicit: Reasoning, Representing, and Discursive Commitment. Cambridge, MA: Harvard University Press.

Robert Brandom. Course pages, University of Pittsburgh. Home page.

Tyler Burge (1993). Content preservation. The Philosophical Review 102(4), pp. 457–488. DOI: 10.2307/2185680. The acceptance principle.

Walter Burkert (1972). Lore and Science in Ancient Pythagoreanism. Translated by Edwin L. Minar Jr. Harvard University Press. Cited here through Huffman’s entry.

Martin Caminada (2014). Strong admissibility revisited. In Simon Parsons, Nir Oren, Chris Reed and Federico Cerutti (eds.), Computational Models of Argument: Proceedings of COMMA 2014, Frontiers in Artificial Intelligence and Applications 266, IOS Press, pp. 197–208. DOI: 10.3233/978-1-61499-436-7-197. Journal version with Paul E. Dunne (2019), Argument & Computation 10(3), pp. 277–300, DOI: 10.3233/AAC-190463.

Gregory J. Chaitin (1966). On the Length of Programs for Computing Finite Binary Sequences. Journal of the ACM 13(4), pp. 547–569.

Roderick M. Chisholm (1957). Perceiving: A Philosophical Study. Cornell University Press, p. 16. One of the two definitions Gettier targets.

R. Cockett, I. J. Geng, C. M. Scandolo and P. V. Srinivasan (2023). Extending resource monotones using Kan extensions. In Proceedings of the Fifth International Conference on Applied Category Theory (ACT 2022), Electronic Proceedings in Theoretical Computer Science 380, pp. 203–223. arXiv: 2206.09784.

Leo Corry (1992). Nicolas Bourbaki and the Concept of Mathematical Structure. Synthese 92(3), pp. 315–348. The sentence quoted, “plays no actual role in the presentation of theories within the treatise,” I read in Chapter 7 of his book Modern Algebra and the Rise of Mathematical Structures (Birkhäuser), p. 338 of the chapter as posted on his website. Author’s chapter.

Claude Crépeau, Rafael Dowsley and Anderson C. A. Nascimento (2020). On the commitment capacity of unfair noisy channels. IEEE Transactions on Information Theory 66(6), pp. 3745–3752. DOI: 10.1109/TIT.2020.2967048; arXiv: 1905.10921.

Ho Ngoc Duc (1997). Reasoning about rational, but not logically omniscient, agents. Journal of Logic and Computation 7(5), pp. 633–648. DOI: 10.1093/logcom/7.5.633.

Karl Duncker (1935; English translation 1945). Zur Psychologie des produktiven Denkens. Julius Springer, Berlin. English translation by Lynne S. Lees: On Problem-Solving, Psychological Monographs 58(5), whole no. 270, American Psychological Association, Washington, 1945. The candle problem.

Phan Minh Dung (1995). On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games. Artificial Intelligence 77(2), pp. 321–357. DOI: 10.1016/0004-3702(94)00041-X.

Julien Dutant (2015). The Legend of the Justified True Belief Analysis. Philosophical Perspectives 29(1), pp. 95–145. DOI: 10.1111/phpe.12061. Known here from its abstract and summaries. Dutant's page at King's College London: kcl.ac.uk/people/julien-dutant.

Kenny Easwaran (2009). Probabilistic proofs and transferability. Philosophia Mathematica (III) 17(3), pp. 341–362. DOI: 10.1093/philmat/nkn032.

Samuel Eilenberg and Saunders Mac Lane (1945). General Theory of Natural Equivalences. Transactions of the American Mathematical Society 58(2), pp. 231–294. Original paper.

Euclid. Elements, Books I and X. Definitions, postulates, common notions and commensurability. Quoted in T. L. Heath's translation (Definition 1, Postulate 1, Common Notion 1).

Ronald Fagin, Joseph Y. Halpern, Yoram Moses and Moshe Y. Vardi (1995). Reasoning About Knowledge. Cambridge, MA: MIT Press.

Dean P. Foster and Robert A. Stine (2008). α-investing: a procedure for sequential control of expected false discoveries. Journal of the Royal Statistical Society, Series B 70(2), pp. 429–444. DOI: 10.1111/j.1467-9868.2007.00643.x.

Gottlob Frege (1884). Die Grundlagen der Arithmetik, section 64. Directions and parallel lines.

Gottlob Frege (1903). Grundgesetze der Arithmetik, vol. II. Hermann Pohle, Jena. The appendix answering Russell's paradox.

Milton Friedman (1975). There's No Such Thing as a Free Lunch: Essays on Public Policy. Open Court, LaSalle, Illinois.

Peter Gärdenfors (1986). Belief revisions and the Ramsey test for conditionals. The Philosophical Review 95(1), pp. 81–93. The impossibility theorem.

Aurélien Garivier, Tor Lattimore and Emilie Kaufmann (2016). On explore-then-commit strategies. Advances in Neural Information Processing Systems 29 (NeurIPS 2016), pp. 784–792. arXiv: 1605.08988.

Murray Gell-Mann (1964). A schematic model of baryons and mesons. Physics Letters 8(3), pp. 214–215. The paper that named the quark. DOI: 10.1016/S0031-9163(64)92001-3.

Edmund L. Gettier (1963). Is Justified True Belief Knowledge? Analysis 23(6), pp. 121–123. JSTOR: 3326923. The remark on Plato is in footnote 1, p. 121.

Sam Glucksberg (1962). The influence of strength of drive on functional fixedness and perceptual recognition. Journal of Experimental Psychology 63(1), pp. 36–41. DOI: 10.1037/h0044683.

Joseph A. Goguen and José Meseguer (1982). Security policies and security models. Proceedings of the 1982 IEEE Symposium on Security and Privacy, pp. 11–20. DOI: 10.1109/SP.1982.10014. Noninterference.

Sanford C. Goldberg (2010). Relying on Others: An Essay in Epistemology. Oxford University Press.

Alvin I. Goldman (1976). Discrimination and Perceptual Knowledge. Journal of Philosophy 73(20), pp. 771–791. The fake-barn case, credited there to Carl Ginet.

Oded Goldreich, Brendan Juba and Madhu Sudan (2012). A theory of goal-oriented communication. Journal of the ACM 59(2), article 8.

I. J. Good (1979). Studies in the History of Probability and Statistics XXXVII: A. M. Turing's statistical work in World War II. Biometrika 66(2), pp. 393–396. JSTOR: 2335677.

Peter D. Grünwald (2005). A Tutorial Introduction to the Minimum Description Length Principle. In Peter D. Grünwald, In Jae Myung and Mark A. Pitt (eds.), Advances in Minimum Description Length: Theory and Applications, MIT Press. arXiv: math/0406077.

Paul R. Halmos (1960). Naive Set Theory. D. Van Nostrand. Section 7, Relations: equivalence relations and partitions.

C. L. Hamblin (1970). Fallacies. Methuen, London. Chapter 8, formal dialectic, where each speaker has a commitment store.

G. H. Hardy (1940). A Mathematician's Apology. Cambridge University Press. The essay as a defence of mathematics pursued for its own sake; summary at Wikipedia, A Mathematician's Apology.

Ralph V. L. Hartley (1928). Transmission of Information. Bell System Technical Journal 7(3), pp. 535–563. DOI: 10.1002/j.1538-7305.1928.tb01236.x.

Hartley (unit). IEC 80000-13:2008, Quantities and units—Part 13: Information science and technology, items 13-24 and 13-24.b, pp. 18–19: the hartley (Hart) is the base-ten information unit. IEC publication record; the table is available in the distributor's standard preview. This cites the 2008 edition, superseded in 2025. The alternative names ban and dit, and naming after Ralph Hartley: Wikipedia, Hartley (unit).

Chris Heegard (1985). On the capacity of permanent memory. IEEE Transactions on Information Theory 31(1), pp. 34–42. DOI: 10.1109/TIT.1985.1056994.

Geoffrey Hellman (1989). Mathematics without Numbers: Towards a Modal-Structural Interpretation. Oxford: Clarendon Press. DOI: 10.1093/0198240341.001.0001.

Martin E. Hellman and Thomas M. Cover (1970). Learning with finite memory. The Annals of Mathematical Statistics 41(3), pp. 765–782. DOI: 10.1214/aoms/1177696958.

Jaakko Hintikka (1962). Knowledge and Belief: An Introduction to the Logic of the Two Notions. Cornell University Press. Logical omniscience is surveyed in the Stanford Encyclopedia of Philosophy entry on epistemic logic. Entry.

Wesley H. Holliday (2009). Dynamic testimonial logic. In Xiangdong He, John Horty and Eric Pacuit (eds.), Logic, Rationality, and Interaction, LORI 2009, Lecture Notes in Computer Science 5834, Springer, pp. 161–179. DOI: 10.1007/978-3-642-04893-7_13. Authority graphs for trust in testimony, with complete reduction axioms.

Carl Huffman (2006, revised 2024). Pythagoreanism. Stanford Encyclopedia of Philosophy. Entry. The source for the remark on Hippasus.

David Hume (1748). An Enquiry Concerning Human Understanding, Section X, "Of Miracles," paragraph 4: "A wise man, therefore, proportions his belief to the evidence." Hume Texts Online, E 10.4. Hume as a figure of the Scottish Enlightenment: Stanford Encyclopedia of Philosophy, "David Hume," Entry.

Iamblichus (c. AD 300). On the Pythagorean Life. Translated by Thomas Taylor as Iamblichus' Life of Pythagoras (1818), chapters XVIII and XXXIV. Project Gutenberg 63300.

Christoph Jäger and Nicholas Shackel (2025). Testimonial authority and knowledge transmission. Social Epistemology 39(6), pp. 636–651. DOI: 10.1080/02691728.2025.2449608.

Hirofumi Katsuno and Alberto O. Mendelzon (1991). On the difference between updating a knowledge base and revising it. Proceedings of the Second International Conference on Principles of Knowledge Representation and Reasoning (KR'91), pp. 387–394; reprinted in Peter Gärdenfors (ed.), Belief Revision, Cambridge University Press (1992), pp. 183–203. DOI: 10.1017/CBO9780511526664.007.

Arthur M. Keller and Marianne Winslett Wilkins (1985). On the use of an extended relational model to handle changing incomplete information. IEEE Transactions on Software Engineering 11(7), pp. 620–633.

Felix Klein (1872). Vergleichende Betrachtungen über neuere geometrische Forschungen (the Erlangen program). Erlangen: A. Deichert.

Wolfgang Köhler (1917). Intelligenzprüfungen an Anthropoiden. Abhandlungen der Königlich Preussischen Akademie der Wissenschaften, Physikalisch-mathematische Klasse, 1917. English translation by Ella Winter, The Mentality of Apes, London: Kegan Paul, Trench, Trubner, 1925.

Andrei N. Kolmogorov (1965). Three approaches to the quantitative definition of information. Problems of Information Transmission 1(1), pp. 1–7; reprinted in International Journal of Computer Mathematics 2 (1968), pp. 157–168. DOI of the reprint: 10.1080/00207166808803030.

Lewis A. Kornhauser. Faculty page, New York University School of Law. Page; Lawrence G. Sager, faculty publications, University of Texas School of Law, Unpacking the Court (1986).

Lewis A. Kornhauser and Lawrence G. Sager (1986). Unpacking the court. Yale Law Journal 96(1), beginning p. 82. DOI: 10.2307/796436.

Oliver Kosut and Jörg Kliewer (2018). Authentication capacity of adversarial channels. 2018 IEEE Information Theory Workshop (ITW), Guangzhou, pp. 1–5. Authentication capacity is zero or the capacity without an adversary, according to overwritability.

Erik C. W. Krabbe (2001). The problem of retraction in critical discussion. Synthese 127(1–2), pp. 141–159. DOI: 10.1023/A:1010318403544.

Georg Kreisel (1967). Informal rigour and completeness proofs. In Imre Lakatos (ed.), Problems in the Philosophy of Mathematics, North-Holland, Amsterdam, pp. 138–186. DOI: 10.1016/S0049-237X(08)71525-8. The creating subject's evidence at a stage.

Saul Kripke (1975). Outline of a theory of truth. Journal of Philosophy 72(19), pp. 690–716. Grounded and ungrounded sentences.

Ralf Krömer (2006). La “machine de Grothendieck” se fonde-t-elle seulement sur des vocables métamathématiques? Bourbaki et les catégories au cours des années cinquante. Revue d’histoire des mathématiques 12(1), pp. 119–162. Numdam. Why categories never entered the Éléments.

Jennifer Lackey (2008). Learning from Words: Testimony as a Source of Knowledge. Oxford University Press. The case of the teacher who doesn’t believe what she teaches.

James Ladyman (1998). What is structural realism? Studies in History and Philosophy of Science Part A 29(3), pp. 409–424. DOI: 10.1016/S0039-3681(98)80129-5.

Gottfried Wilhelm Leibniz (1679). Letter to Christiaan Huygens, 8 September 1679, with the enclosed essay on the analysis of situation. In C. I. Gerhardt (ed.), Mathematische Schriften, vol. 2, Halle, 1850; the essay is on pp. 20–27. Huygens replied on 22 November 1679. Discussion with the dated correspondence.

Gottfried Wilhelm Leibniz (1716). Fourth paper to Samuel Clarke, 2 June 1716, section 4. In A Collection of Papers which passed between the late learned Mr. Leibnitz and Dr. Clarke, trans. Samuel Clarke, James Knapton, London, 1717. Text.

Isaac Levi (1980). The Enterprise of Knowledge. MIT Press. Where routine and deliberate expansion first appear.

Isaac Levi (1991). The Fixation of Belief and Its Undoing: Changing Beliefs through Inquiry. Cambridge University Press. Chapter 3, routine and deliberate expansion. DOI: 10.1017/CBO9780511663819.

Leonid A. Levin (1984). Randomness Conservation Inequalities; Information and Independence in Mathematical Theories. Information and Control 61(1), pp. 15–37. The line about the uninformed witness is Levin’s, quoted as the epigraph of Luis Antunes, Lance Fortnow, Dieter van Melkebeek and N. V. Vinodchandran, Computational Depth: Concept and Applications, Theoretical Computer Science 354(3), 2006, pp. 391–404. Paper.

Dennis V. Lindley (1956). On a Measure of the Information Provided by an Experiment. Annals of Mathematical Statistics 27(4), pp. 986–1005. DOI: 10.1214/aoms/1177728069.

Christian List and Philip Pettit (2002). Aggregating sets of judgments: an impossibility result. Economics and Philosophy 18(1), pp. 89–110. DOI: 10.1017/S0266267102001098.

John Locke (1690). An Essay Concerning Human Understanding, Book I, chapter iv, section 23. The attack on taking belief from authority. Modernized text of Book I.

Saunders Mac Lane (1971; 2nd edition 1998). Categories for the Working Mathematician. Graduate Texts in Mathematics 5, Springer, New York. The notes at the end of Chapter I: *category* borrowed from Aristotle and Kant, *functor* from Carnap.

MacTutor History of Mathematics. Arend Heyting. University of St Andrews. Biography. Doctorate in 1925 under Brouwer.

MacTutor History of Mathematics. Emmy Amalie Noether. University of St Andrews. Biography. Habilitation closed to women; courses advertised under Hilbert's name; habilitation in 1919.

Robert K. Merton (1957). Priorities in Scientific Discovery: A Chapter in the Sociology of Science. American Sociological Review 22(6), pp. 635–659.

Janet Metcalfe (1986). Premonitions of insight predict impending error. Journal of Experimental Psychology: Learning, Memory, and Cognition 12(4), pp. 623–634. DOI: 10.1037/0278-7393.12.4.623. High warmth before an insight answer goes with the answer being wrong.

Janet Metcalfe and David Wiebe (1987). Intuition in insight and noninsight problem solving. Memory and Cognition 15(3), pp. 238–246. The warmth ratings.

Emmy Noether (1918). Invariante Variationsprobleme. Nachrichten von der Gesellschaft der Wissenschaften zu Göttingen, Mathematisch-Physikalische Klasse, pp. 235–257. Original paper, EuDML 59024.

Robert Nozick (1981). Philosophical Explanations. Harvard University Press. Chapter 3, the sensitivity condition on knowledge.

James R. O'Shea (2010). Review of Bernhard Weiss and Jeremy Wanderer (eds.), Reading Brandom: On Making It Explicit. Notre Dame Philosophical Reviews, 13 December 2010. Review. Brandom's deontic scorekeeping of commitments and entitlements, with default, inferential and testimonial (deferential) authority.

David L. Parnas (1972). On the Criteria To Be Used in Decomposing Systems into Modules. Communications of the ACM 15(12), pp. 1053–1058. DOI: 10.1145/361598.361623.

Charles S. Peirce (1877). The Fixation of Belief. Popular Science Monthly 12, pp. 1–15. Quoted: p. 6.

Plato. Meno 97d–98a: Daedalus's statues, and true opinions tied down by reasoning; trans. W. R. M. Lamb, Plato in Twelve Volumes, vol. 3, Harvard University Press, 1967. Republic III, 403e: Glaucon on a guardian needing a guardian; trans. Paul

Shorey, Plato in Twelve Volumes, vols. 5–6, Harvard University Press, 1969, text. Republic, Book VII, 514a–517c, translated by Benjamin Jowett: the cave; the prisoners' honours for whoever is quickest at the passing shadows and best at guessing what comes next (516c–d); Socrates' own reading of the cave as the world of sight (517b); MIT Internet Classics Archive. Timaeus; Theaetetus: the receptacle and the discussion of knowledge as true judgement with an account. Philosophical antecedents. None of the formal results takes them as a premise.

Karl R. Popper (1959). The Logic of Scientific Discovery. Hutchinson. Translation of Logik der Forschung (1934). Section 30: basic statements, the jury, and the piles driven into a swamp.

Henry Prakken (2005). Coherence and flexibility in dialogue games for argumentation. Journal of Logic and Computation 15(6), pp. 1009–1040. DOI: 10.1093/logcom/exi046.

Michael D. Resnik (1997). Mathematics as a Science of Patterns. Oxford: Clarendon Press. DOI: 10.1093/0198250142.001.0001.

Jorma Rissanen (1978). Modeling by shortest data description. Automatica 14(5), pp. 465–471. DOI: 10.1016/0005-1098(78)90005-5.

Bertrand Russell (1902). Letter to Gottlob Frege, 16 June 1902, and Frege's reply. In Jean van Heijenoort (ed.), From Frege to Gödel: A Source Book in Mathematical Logic, 1879–1931. Harvard University Press, 1967, pp. 124–125 and 127–128. Stanford Encyclopedia of Philosophy, Russell's paradox. Russell as a British philosopher and logician: Stanford Encyclopedia of Philosophy, "Bertrand Russell," Entry.

Bertrand Russell (1948). Human Knowledge: Its Scope and Limits. George Allen and Unwin, London. The stopped clock: a true belief about the time from a clock that has stopped, which is not knowledge. Quoted in the Stanford Encyclopedia of Philosophy entry "The Analysis of Knowledge" (which misdates the book 1923). SEP entry.

Girolamo Saccheri (1733). Euclides ab omni naevo vindicatus. Milan.

Krister Segerberg (1998). Irrevocable Belief Revision in Dynamic Doxastic Logic. Notre Dame Journal of Formal Logic 39(3), pp. 287–306. DOI: 10.1305/ndjfl/1039182247.

Sextus Empiricus. Outlines of Pyrrhonism I.164–177. The five modes attributed to Agrippa.

Claude E. Shannon (1948). A Mathematical Theory of Communication. Bell System Technical Journal 27, pp. 379–423 and 623–656. Bell Labs publication record. Quoted: "irrelevant to the engineering problem," p. 379; "allows one to make a

saving in time," p. 385; half of English "determined by the structure of the language," p. 399. Paper text.

Claude E. Shannon, interviewed by Robert Price (1982). Oral history, 28 July 1982, IEEE History Center. Transcript.

Stewart Shapiro (1997). Philosophy of Mathematics: Structure and Ontology. New York: Oxford University Press.

Ray J. Solomonoff (1964). A Formal Theory of Inductive Inference, Parts I and II. Information and Control 7(1), pp. 1–22, and 7(2), pp. 224–254.

Sophia of Hanover. Wikipedia. Article. Her friendship with Leibniz from 1676 to 1714 and her part in laying out the Herrenhausen Gardens.

Stanford Encyclopedia of Philosophy. Gottlob Frege. Entry. German logician and mathematician at Jena.

Stephen M. Stigler (1980). Stigler's Law of Eponymy. Transactions of the New York Academy of Sciences 39(1), pp. 147–157. DOI: 10.1111/j.2164-0947.1980.tb02775.x.

Alfred Tarski (1954). Contributions to the theory of models, I and II. Indagationes Mathematicae 16, pp. 572–581 and 582–588. The first part proposes the name theory of models.

Alfred Tarski (1986). What are logical notions? Edited by John Corcoran. History and Philosophy of Logic 7(2), pp. 143–154. DOI: 10.1080/01445348608837096. From a lecture given at Bedford College, University of London, on 16 May 1966; Klein's Erlangen program is its starting point.

Neil Tennant (2008). Belief-revision, the Ramsey test, monotonicity, and the so-called impossibility results. The Review of Symbolic Logic 1(4), pp. 402–423. DOI: 10.1017/S1755020308090023.

Anvith Thudi, Hengrui Jia, Ilia Shumailov and Nicolas Papernot (2022). On the Necessity of Auditable Algorithmic Definitions for Machine Unlearning. Proceedings of the 31st USENIX Security Symposium. arXiv: 2110.11891.

Myron Tribus and Edward C. McIrvine (1971). Energy and Information. Scientific American 225(3), pp. 179–188.

Trinity College, Cambridge. Godfrey Harold Hardy. Page. Hardy as Sadleirian Professor at Cambridge, and the 1940 essay.

Anne S. Troelstra (1969). Principles of Intuitionism. Lecture Notes in Mathematics 95, Springer, Berlin. DOI: 10.1007/BFb0080643. The creating subject who never forgets.

Nikolai K. Vereshchagin and Paul M. B. Vitányi (2004). Kolmogorov's Structure Functions and Model Selection. IEEE Transactions on Information Theory 50(12), pp. 3265–3290. arXiv: cs/0204037; DOI: 10.1109/TIT.2004.838346. "Meaningless information, random noise" is in Section II.D, as read in the arXiv preprint; I didn't see the journal page.

Heinrich Weber (1893). Leopold Kronecker. Jahresbericht der Deutschen Mathematiker-Vereinigung 2, pp. 5–23. The remark attributed to a lecture of 1886 is on p. 19.

Andreas Winter, Anderson C. A. Nascimento and Hideki Imai (2003). Commitment capacity of discrete memoryless channels. Cryptography and Coding 2003, LNCS 2898, pp. 35–51. DOI: 10.1007/978-3-540-40974-8_4; arXiv: cs/0304014.

Linda Zagzebski (1994). The inescapability of Gettier problems. The Philosophical Quarterly 44(174), pp. 65–73. DOI: 10.2307/2220147.

Ernst Zermelo (1908). Untersuchungen über die Grundlagen der Mengenlehre I. Mathematische Annalen 65(2), pp. 261–281. DOI: 10.1007/BF01449999.

E.2 Cost, records and thermodynamics

American Physical Society (2006). May, 1911: Rutherford and the Discovery of the Atomic Nucleus. APS News 15(5), This Month in Physics History. Article.

Will Arthur and David Challener, with Kenneth Goldman (2015). A Practical Guide to TPM 2.0: Using the Trusted Platform Module in the New Age of Security. Apress. Chapter 11, NV indexes (bit-field indexes and counters); Chapter 12, Platform Configuration Registers (read, extend and reset; an extend can't be undone), DOI: 10.1007/978-1-4302-6584-9_12.

Erik Aurell, Carlos Mejía-Monasterio and Paolo Muratore-Ginanneschi (2011). Optimal protocols and optimal transport in stochastic thermodynamics. Physical Review Letters 106, 250601. DOI: 10.1103/PhysRevLett.106.250601; arXiv: 1012.2037.

J. L. Austin (1962). How to Do Things with Words. Clarendon Press, Oxford. The William James Lectures, Harvard, 1955.

Adam Back (2002). Hashcash: A Denial of Service Counter-Measure. Technical report, 1 August 2002; the scheme was first proposed in 1997. Report.

John C. Baez, Tobias Fritz and Tom Leinster (2011). A characterization of entropy in terms of information loss. Entropy 13(11), pp. 1945–1957. DOI: 10.3390/e13111945; arXiv: 1106.1791.

Mihir Bellare and Bennet S. Yee (1997). Forward integrity for secure audit logs. Technical Report CS98-580, Department of Computer Science and Engineering, University of California, San Diego.

Charles H. Bennett (1973). Logical Reversibility of Computation. IBM Journal of Research and Development 17(6), pp. 525–532. Original paper.

Charles H. Bennett (1982). The thermodynamics of computation: a review. International Journal of Theoretical Physics 21, pp. 905–940. DOI: 10.1007/BF02084158.

Charles H. Bennett (1989). Time/Space Trade-Offs for Reversible Computation. SIAM Journal on Computing 18(4), pp. 766–776. DOI: 10.1137/0218053.

Charles H. Bennett (2003). Notes on Landauer's principle, reversible computation, and Maxwell's Demon. Studies in History and Philosophy of Modern Physics 34(3), pp. 501–510. arXiv: physics/0210005v2; DOI: 10.1016/S1355-2198(03)00039-X. Quoted from the arXiv preprint: "the erasure of a bit or the merging of two computation paths" (abstract) and "in my opinion the one of greatest merit" (p. 1); "the most important logically reversible operations" and "erasure of one of two copies of data known to be identical" (p. 2). I didn't see the journal pages.

Charles H. Bennett, Péter Gács, Ming Li, Paul M. B. Vitányi and Wojciech H. Zurek (1998). Information distance. IEEE Transactions on Information Theory 44(4), pp. 1407–1423. DOI: 10.1109/18.681318. The irreversibility cost of turning one string into another.

Antoine Bérut, Artak Arakelyan, Artyom Petrosyan, Sergio Ciliberto, Raoul Dillenschneider and Eric Lutz (2012). Experimental verification of Landauer's principle linking information and thermodynamics. Nature 483, pp. 187–189. DOI: 10.1038/nature10872.

Antoine Bérut, Artyom Petrosyan and Sergio Ciliberto (2015). Information and thermodynamics: experimental verification of Landauer's erasure principle. Journal of Statistical Mechanics: Theory and Experiment, P06015. DOI: 10.1088/1742-5468/2015/06/P06015. The long version, with the protocol in full and the fit $\langle Q \rangle = kT \ln 2 + B/\tau$, $B = 8.15\,kT \cdot \mathrm{s}$.

Laurent Bindschaedler, Quentin Botha and Christoph Siebenbrunner (2026). Agent Flight Recorder: Tamper-Evident Audit Trails with On-Chain Anchoring for Long-Horizon Tool-Using Agents. Preprint. arXiv: 2609.01931.

Matt Bishop and Michael Dilger (1996). Checking for race conditions in file accesses. Computing Systems 9(2), pp. 131–152. PDF.

Thomas Blanchard (2024). Causation and the Time-Asymmetry of Knowledge. Australasian Journal of Philosophy 102(4), pp. 959–977. DOI: 10.1080/00048402.2024.2328629.

P. Bocchieri and A. Loinger (1957). Quantum Recurrence Theorem. Physical Review 107, 337. DOI: 10.1103/PhysRev.107.337.

Ludwig Boltzmann (1896). Entgegnung auf die wärmetheoretischen Betrachtungen des Hrn. E. Zermelo. Annalen der Physik und Chemie, Neue Folge 57, pp. 773–784.

Walter J. Boyne (2013). The Checklist. Air Force Magazine, August 2013, pp. 52–56. The Model 299 crash of 30 October 1935: the locked controls, the earlier written routines from 1911 on, and the judgement that "it took the Model 299 crash to institutionalize its use." Article.

Jeffrey Bub (2001). Maxwell's Demon and the Thermodynamics of Computation. Studies in History and Philosophy of Modern Physics 32(4), pp. 569–579. Szilard and Brillouin on the cost of acquiring information, at least $k \log 2$ per bit.

Eric Budish (2025). Trust at Scale: The Economic Limits of Cryptocurrencies and Blockchains. The Quarterly Journal of Economics 140(1), pp. 1–62. DOI: 10.1093/qje/qjae033.

Jesse Bull and Joel Watson (2007). Hard evidence and mechanism design. Games and Economic Behavior 58(1), pp. 75–93. DOI: 10.1016/j.geb.2006.03.003.

Vitalik Buterin (2016). Transaction spam attack: next steps. Ethereum Foundation Blog, 22 September 2016. Post.

Vitalik Buterin and Martin Swende (2020). EIP-2929: Gas cost increases for state access opcodes. Ethereum Improvement Proposals. eips.ethereum.org/EIPS/eip-2929.

Vitalik Buterin and Martin Swende (2021). EIP-3529: Reduction in refunds. Ethereum Improvement Proposals. eips.ethereum.org/EIPS/eip-3529.

Vitalik Buterin and Virgil Griffith (2017). Casper the Friendly Finality Gadget. arXiv: 1710.09437v1.

Vitalik Buterin, Diego Hernandez, Thor Kamphefner, Khiem Pham, Zhi Qiao, Danny Ryan, Juhyeok Sin, Ying Wang and Yan X. Zhang (2020). Combining GHOST and Casper. arXiv: 2003.03052.

Wen Tsing Chow and William H. Henrich (1962). Storage matrix. US patent 3,028,659, filed 27 December 1957, granted 10 April 1962. Patent text. The invention in 1956 by Chow, of the Arma Division of the American Bosch Arma Corporation in Garden City, New York, at the request of the United States Air Force, to store the targeting constants of the Atlas E/F's airborne digital computer: Wikipedia, Programmable ROM, History section, entry.

Byung-Gon Chun, Petros Maniatis, Scott Shenker and John Kubiatowicz (2007). Attested append-only memory: making adversaries stick to their word. Proceedings

of the 21st ACM Symposium on Operating Systems Principles (SOSP '07), pp. 189–204. DOI: 10.1145/1294261.1294280.

Michael T. Clanchy (1979). From Memory to Written Record: England 1066–1307. London: Edward Arnold; third edition, Wiley-Blackwell, 2012.

Allen Clement, Flavio Junqueira, Aniket Kate and Rodrigo Rodrigues (2012). On the (limited) power of non-equivocation. Proceedings of the 2012 ACM Symposium on Principles of Distributed Computing (PODC '12), pp. 301–308. DOI: 10.1145/2332432.2332490.

Bob Coecke, Tobias Fritz and Robert W. Spekkens (2016). A mathematical theory of resources. Information and Computation 250, pp. 59–86. DOI: 10.1016/j.ic.2016.02.008.

Scott A. Crosby and Dan S. Wallach (2009). Efficient data structures for tamper-evident logging. Proceedings of the 18th USENIX Security Symposium, Montreal. Conference page.

Nicholas of Cusa (1433). De concordantia catholica. Where he calls the Donation of Constantine a forgery.

Norman Danner and Daniel R. Licata (2022). Denotational semantics as a foundation for cost recurrence extraction for functional languages. Journal of Functional Programming 32, e8. DOI: 10.1017/S095679682200003X.

Norman Danner, Daniel R. Licata and Ramyaa Ramyaa (2015). Denotational cost semantics for functional languages with inductive types. Proceedings of ICFP 2015, pp. 140–151. DOI: 10.1145/2784731.2784749.

Erik D. Demaine, Jayson Lynch, Geronimo J. Mirano and Nirvan Tyagi (2016). Energy-Efficient Algorithms. Proceedings of the 2016 ACM Conference on Innovations in Theoretical Computer Science (ITCS), pp. 321–332. DOI: 10.1145/2840728.2840756. Energy complexity beside time and space: energy-aware random-access machines that count destroyed bits.

Deuteronomy 19:15. Two or three witnesses for a conviction. NRSV text.

David Deutsch and Chiara Marletto (2015). Constructor theory of information. Proceedings of the Royal Society A 471(2174), 20140540. DOI: 10.1098/rspa.2014.0540; arXiv: 1405.5563. Section 8.6, the irreducible perturbation caused by measurement.

David Deutsch and Chiara Marletto (2025). Constructor theory of time. Preprint. arXiv: 2505.08692.

Sandro Donadi, Kristian Piscicchia, Catalina Curceanu, Lajos Diósi, Matthias Laubenstein and Angelo Bassi (2021). Underground test of gravity-related wave function

collapse. Nature Physics 17, pp. 74–78. DOI: 10.1038/s41567-020-1008-4. The radiation a gravity-driven collapse would emit, listened for at Gran Sasso and not found at the level the parameter-free model predicts.

Cynthia Dwork and Moni Naor (1992). Pricing via Processing or Combatting Junk Mail. Advances in Cryptology, CRYPTO '92, LNCS 740 (Springer, 1993), pp. 139–147. DOI: 10.1007/3-540-48071-4_10.

John Earman and John D. Norton (1998, 1999). Exorcist XIV: The Wrath of Maxwell's Demon. Part I: From Maxwell to Szilard, Studies in History and Philosophy of Modern Physics 29, pp. 435–471; Part II: From Szilard to Landauer and Beyond, 30, pp. 1–40. Part I; Part II.

Aram Ebtekar and Marcus Hutter (2024). Modeling the Arrows of Time with Causal Multibaker Maps. Entropy 26(9), article 776. DOI: 10.3390/e26090776.

Arthur S. Eddington (1928). The Nature of the Physical World. Cambridge University Press. Chapter IV, The Running-Down of the Universe. Quotation with chapter.

Massimiliano Esposito and Christian Van den Broeck (2010). Three detailed fluctuation theorems. Physical Review Letters 104, 090601. DOI: 10.1103/PhysRevLett.104.090601. The split of entropy production into adiabatic and nonadiabatic parts.

Paul N. Fahn (1996). Maxwell's demon and the entropy cost of information. Foundations of Physics 26, pp. 71–93. DOI: 10.1007/BF02058888.

Philippe Faist, Frédéric Dupuis, Jonathan Oppenheim and Renato Renner (2015). The minimal work cost of information processing. Nature Communications 6, article 7669. arXiv: 1211.1037.

FIDE (2023). FIDE Laws of Chess taking effect from 1 January 2023, Article 9.2.3. FIDE Handbook.

Forrester (n.d.). Books as Evidence, before and after Pacioli (1494 AD). Paper on the University of Strathclyde accounting history pages; the page gives no initials or date. Mirror of the paper.

Edward Fredkin and Tommaso Toffoli (1982). Conservative logic. International Journal of Theoretical Physics 21(3–4), pp. 219–253. DOI: 10.1007/BF01857727.

Tobias Fritz (2017). Resource convertibility and ordered commutative monoids. Mathematical Structures in Computer Science 27(6), pp. 850–938. arXiv: 1504.03661.

Gaius. Institutes, book 2, sections 104, 119 and 147 (second century). The will by bronze and balance before five witnesses, a scale-holder and the buyer of the estate (104); a will sealed by seven witnesses (119, 147). Text and translation.

Momčilo Gavrilov and John Bechhoefer (2016). Erasure without Work in an Asymmetric Double-Well Potential. Physical Review Letters 117, 200601. DOI: 10.1103/PhysRevLett.117.200601. Average erasure work below $kT \ln 2$ when the two wells are not alike.

Rocco Gaudenzi, Enrique Burzurí, Satoru Maegawa, Herre S. J. van der Zant and Fernando Luis (2018). Quantum Landauer erasure with a molecular nanomagnet. Nature Physics 14, pp. 565–568. DOI: 10.1038/s41567-018-0070-7. Landauer's floor on a register of molecular spins at one kelvin.

Atul Gawande (2007). The Checklist. The New Yorker, 10 December 2007. The earliest place the line "too much airplane for one man to fly" turns up that I could find; it names no newspaper. Article.

Sebastian Goldt and Udo Seifert (2017). Stochastic thermodynamics of learning. Physical Review Letters 118, 010601. DOI: 10.1103/PhysRevLett.118.010601.

Tomáš Gonda and Robert W. Spekkens (2023). Monotones in general resource theories. Compositionality 5, article 7. DOI: 10.32408/compositionality-5-7.

Jim Gray (1981). The Transaction Concept: Virtues and Limitations. Proceedings of the 7th International Conference on Very Large Data Bases, pp. 144–154. Paper.

Jerry R. Green and Jean-Jacques Laffont (1986). Partially verifiable information and mechanism design. Review of Economic Studies 53(3), pp. 447–456.

Stuart Haber and W. Scott Stornetta (1991). How to Time-Stamp a Digital Document. Journal of Cryptology 3(2), pp. 99–111.

Andreas Haeberlen and Petr Kuznetsov (2009). The fault detection problem. Principles of Distributed Systems, OPODIS 2009, Lecture Notes in Computer Science, Springer, pp. 99–114. DOI: 10.1007/978-3-642-10877-8_10. Faults divided into classes; only omission and commission faults are detectable.

Andreas Haeberlen, Petr Kouznetsov and Peter Druschel (2007). PeerReview: practical accountability for distributed systems. Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP '07), pp. 175–188. DOI: 10.1145/1294261.1294279.

Douglas Harper (n.d.). Online Etymology Dictionary, entry "ledger". Entry.

Joseph M. Hellerstein and Peter Alvaro (2020). Keeping CALM: When Distributed Consistency Is Easy. Communications of the ACM 63(9), pp. 72–81. DOI: 10.1145/3369736.

Maurice Herlihy (1991). Wait-free synchronization. ACM Transactions on Programming Languages and Systems 13(1), pp. 124–149. DOI: 10.1145/114005.102808.

Yoichi Hirai (2017). Development of accountable safety for one-message Casper. Repository.

Anson Ho, Ege Erdil and Tamay Besiroglu (2023). Limits to the Energy Efficiency of CMOS Microprocessors. arXiv: 2312.08595. Table I: switching a CMOS gate, 100 aJ to 100 fJ.

Geoffrey M. Hodgson (2025). The Secret History of the Tally Stick. Web page. Henry I and tallies from about 1100; the stock and the stockholder.

Jan Hoffmann, Klaus Aehlig and Martin Hofmann (2011). Multivariate Amortized Resource Analysis. Proceedings of POPL 2011, pp. 357–370. DOI: 10.1145/1926385.1926427. Journal version: ACM Transactions on Programming Languages and Systems 34(3), 2012, article 14, DOI: 10.1145/2362389.2362393.

Martin Hofmann and Steffen Jost (2003). Static prediction of heap space usage for first-order functional programs. Proceedings of the 30th ACM Symposium on Principles of Programming Languages (POPL 2003), pp. 185–197. DOI: 10.1145/604131.604148. The start of type-based amortized analysis.

Jeongmin Hong, Brian Lambson, Scott Dhuey and Jeffrey Bokor (2016). Experimental test of Landauer's principle in single-bit operations on nanomagnetic memory bits. Science Advances 2(3), e1501492. DOI: 10.1126/sciadv.1501492. The floor on a real memory technology, within two standard deviations.

Horace (c. 20 BC). Epistles, book 1, epistle 1, line 14. The source of *Nullius in verba*.

Radha Jagadeesan, Alan Jeffrey, Corin Pitcher and James Riely (2009). Towards a theory of accountability and audit. Computer Security, ESORICS 2009, Lecture Notes in Computer Science 5789, Springer, pp. 152–167. DOI: 10.1007/978-3-642-04444-1_10.

Edwin T. Jaynes (1965). Gibbs vs Boltzmann entropies. American Journal of Physics 33(5), pp. 391–398. DOI: 10.1119/1.1971557. "An anthropomorphic concept" is in Section VI, p. 398, and the credit to Wigner in the acknowledgments on the same page.

Edwin T. Jaynes (1989). Clearing up Mysteries: The Original Goal. In John Skilling (ed.), Maximum Entropy and Bayesian Methods, Kluwer, pp. 1–27. DOI: 10.1007/978-94-015-7860-8_1. Where the mind projection fallacy is named.

E. Jensen, G. Hagensen and J. Broughton (1987). A New Approach to Exclusive Data Access in Shared Memory Multiprocessors. Technical Report UCRL-97663, Lawrence Livermore National Laboratory, November 1987. The load-linked and store-conditional pair.

Yonggun Jun, Momčilo Gavrilov and John Bechhoefer (2014). High-precision test of Landauer's principle in a feedback trap. Physical Review Letters 113, 190601. DOI: 10.1103/PhysRevLett.113.190601. Erasure at $0.71 \pm 0.03\,kT$ and the control that squashes nothing at $0.05 \pm 0.03\,kT$, on the same bead.

Ruth E. Kastner (2025). Maxwell's demon is foiled by the entropy cost of measurement, not erasure. Foundations 5(2), article 16. DOI: 10.3390/foundations5020016; arXiv: 2503.18186.

Shin-ya Katsumata (2014). Parametric effect monads and semantics of effect systems. Proceedings of POPL 2014, pp. 633–645. DOI: 10.1145/2535838.2535846.

Narayana R. Kocherlakota (1998). Money Is Memory. Journal of Economic Theory 81(2), pp. 232–251. DOI: 10.1006/jeth.1997.2357.

Artemy Kolchinsky and David H. Wolpert (2017). Dependence of dissipation on the initial distribution over states. Journal of Statistical Mechanics: Theory and Experiment 2017, 083202. DOI: 10.1088/1742-5468/aa7ee1.

Artemy Kolchinsky and David H. Wolpert (2020). Thermodynamic costs of Turing machines. Physical Review Research 2, 033312. DOI: 10.1103/PhysRevResearch.2.033312.

H. T. Kung and John T. Robinson (1981). On optimistic methods for concurrency control. ACM Transactions on Database Systems 6(2), pp. 213–226.

Robert Künnemann, Ilkan Esiyok and Michael Backes (2019). Automated verification of accountability in security protocols. 2019 IEEE 32nd Computer Security Foundations Symposium (CSF), IEEE. DOI: 10.1109/CSF.2019.00034; arXiv: 1805.10891.

Ralf Küsters, Tomasz Truderung and Andreas Vogt (2010). Accountability: definition and relationship to verifiability. Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS '10), pp. 526–535. DOI: 10.1145/1866307.1866366.

James Ladyman, Stuart Presnell, Anthony J. Short and Berry Groisman (2007). The connection between logical and thermodynamic irreversibility. Studies in History and Philosophy of Modern Physics 38(1), pp. 58–79. DOI: 10.1016/j.shpsb.2006.03.007.

Leslie Lamport (1978). Time, Clocks, and the Ordering of Events in a Distributed System. Communications of the ACM 21(7), pp. 558–565. DOI: 10.1145/359545.359563. "Implemented by counters with no actual timing mechanism" is on p. 559.

Leslie Lamport (1983). What Good Is Temporal Logic? In R. E. A. Mason (ed.), Information Processing 83, Elsevier, pp. 657–668. Author paper. The paper that introduces invariance under stuttering.

Leslie Lamport (n.d.). The Writings of Leslie Lamport, note on "Time, Clocks and the Ordering of Events in a Distributed System." Web page. The Johnson and Thomas note and the comments of Jim Gray.

Rolf Landauer (1961). Irreversibility and Heat Generation in the Computing Process. IBM Journal of Research and Development 5(3), pp. 183–191. DOI: 10.1147/rd.53.0183. Quoted: "requires a minimal heat generation," "typically of the order of kT," "serves the purpose of standardizing signals. . . " and "show, or at least very strongly suggest," all p. 183; "no guarantee that this minimum is in fact achievable," p. 187. Paper scan.

Yves Lecerf (1963). Machines de Turing réversibles. Récursive insolubilité en $n \in \mathbb{N}$ de l'équation $u = \theta^n u$, où θ est un "isomorphisme de codes." Comptes Rendus de l'Académie des Sciences de Paris 257, pp. 2597–2600.

Geoffrey A. Lee (1977). The Coming of Age of Double Entry: The Giovanni Farolfi Ledger of 1299–1300. Accounting Historians Journal 4(2), pp. 79–96. DOI: 10.2308/0148-4184.4.2.79.

Tom Leinster. Wikipedia. Article. A British mathematician known for his work on category theory, at Glasgow and then Edinburgh. John C. Baez, Wikipedia, Article: mathematical physicist at the University of California, Riverside, working on higher categories and applied category theory. Tobias Fritz's category-theoretic approach to probability: arXiv listing.

Dave Levin, John R. Douceur, Jacob R. Lorch and Thomas Moscibroda (2009). TrInc: small trusted hardware for large distributed systems. Proceedings of the 6th USENIX Symposium on Networked Systems Design and Implementation (NSDI '09), pp. 1–14.

Ming Li and Paul Vitányi (1996). Reversibility and adiabatic computation: trading time and space for energy. Proceedings of the Royal Society of London A 452, pp. 769–789. DOI: 10.1098/rspa.1996.0039; arXiv: quant-ph/9703022. Axioms 1 and 2: reversible computation costs nothing, and each irreversibly provided or deleted bit costs one unit.

Elliott H. Lieb and Jakob Yngvason (1999). The physics and mathematics of the second law of thermodynamics. Physics Reports 310(1), pp. 1–96. DOI: 10.1016/S0370-1573(98)00082-9.

Jacques-Louis Lions (chair) et al. (1996). ARIANE 5 Flight 501 Failure: Report by the Inquiry Board. Paris, 19 July 1996. Copy of the report.

Josef Loschmidt (1876). Über den Zustand des Wärmegleichgewichtes eines Systems von Körpern mit Rücksicht auf die Schwerkraft. Sitzungsberichte der Kaiserlichen

Akademie der Wissenschaften in Wien, Mathematisch-Naturwissenschaftliche Classe 73, Abt. II, pp. 128–142.

Aleksandr M. Lyapunov (1892). The general problem of the stability of motion. Doctoral thesis, Kharkov Mathematical Society (in Russian). English translation by A. T. Fuller, International Journal of Control 55(3) (1992), pp. 531–773.

Henry Sumner Maine (1861). Ancient Law. John Murray, London, chapter VI. The seven seals of the praetor's will as the seven people of the older mock sale. Project Gutenberg 22910.

Gonzalo Manzano, Gülce Kardeş, Édgar Roldán and David Wolpert (2024). Thermodynamics of Computations with Absolute Irreversibility, Unidirectional Transitions, and Stochastic Computation Times. Physical Review X 14, 021026. DOI: 10.1103/PhysRevX.14.021026.

Chiara Marletto (2016). Constructor theory of thermodynamics. Preprint, first posted in 2016. arXiv: 1608.02625.

Chiara Marletto, Vlatko Vedral, Laura T. Knoll, Fabrizio Piacentini, Ettore Bernardi, Enrico Rebufello, Alessio Avella, Marco Gramegna, Ivo Pietro Degiovanni and Marco Genovese (2022). Emergence of constructor-based irreversibility in quantum systems: theory and experiment. Physical Review Letters 128, 080401. DOI: 10.1103/PhysRevLett.128.080401; arXiv: 2009.14649.

O. J. E. Maroney (2009). Generalizing Landauer's Principle. Physical Review E 79, 031105. arXiv: quant-ph/0702094v3; DOI: 10.1103/PhysRevE.79.031105.

Owen Maroney (2009). Information Processing and Thermodynamic Entropy. Stanford Encyclopedia of Philosophy, first published 15 September 2009. Entry. It ends without a verdict on Landauer's principle.

James Clerk Maxwell (1865). A Dynamical Theory of the Electromagnetic Field. Philosophical Transactions of the Royal Society of London 155, pp. 459–512. DOI: 10.1098/rstl.1865.0008.

James Clerk Maxwell (1867, 1871). Letter to Peter Guthrie Tait, 11 December 1867, printed in Cargill Gilston Knott, Life and Scientific Work of Peter Guthrie Tait, Cambridge University Press, 1911, pp. 213–214 ("very observant and neat-fingered being," p. 214), with the undated note "Concerning Demons" on pp. 214–215 ("Call him no more a demon but a valve," p. 215); and Theory of Heat, London: Longmans, Green, 1871, the section on the limitation of the second law of thermodynamics. Text of the letter.

Ralph C. Merkle (1979). Secrecy, Authentication, and Public Key Systems. PhD thesis, Stanford University. Thesis. Where tree authentication, now called the Merkle tree, is introduced.

Alfred Mitchell-Innes (1913). What is Money? The Banking Law Journal, May 1913, pp. 377–408. The exchequer tally and "raising a tally."

Kenichi Morita (1996). Universality of a reversible two-counter machine. Theoretical Computer Science 168(2), pp. 303–320. DOI: 10.1016/S0304-3975(96)00081-3.

Mozilla. Certificate Transparency. MDN Web Docs. Page.

Moni Naor and Guy N. Rothblum (2005). The complexity of online memory checking. Proceedings of the 46th IEEE Symposium on Foundations of Computer Science (FOCS 2005), pp. 573–584; journal version in Journal of the ACM 56(1), 2009. IACR ePrint 2006/091.

National Institute of Standards and Technology (2019). The International System of Units (SI). NIST Special Publication 330, 2019 edition. NIST page on the kelvin. The 2019 redefinition fixing the Boltzmann constant.

National Museum of Australia. Defining Moments: Black box flight recorder invented. Museum page. David Warren's flight memory recorder, Aeronautical Research Laboratories, Melbourne: the idea in 1953, the first prototype in 1956, the combined voice and data recorder in 1958.

Tobias Nipkow (2015). Amortized Complexity Verified. Interactive Theorem Proving, ITP 2015, LNCS 9236, pp. 310–324. Author paper. Extended treatment: Tobias Nipkow and Hauke Brinkop (2019), same title, Journal of Automated Reasoning 62(3), pp. 367–391. DOI: 10.1007/s10817-018-9459-3.

Yue Niu, Jonathan Sterling, Harrison Grodin and Robert Harper (2022). A Cost-Aware Logical Framework. Proceedings of the ACM on Programming Languages 6(POPL), article 9. DOI: 10.1145/3498670.

John D. Norton (2005). Eaters of the Lotus: Landauer's Principle and the Return of Maxwell's Demon. Studies in History and Philosophy of Modern Physics 36, pp. 375–411. Author's papers and discussion.

John D. Norton (2013). The end of the thermodynamics of computation: a no-go result. Philosophy of Science 80(5), pp. 1182–1192. DOI: 10.1086/673714.

John D. Norton (2018). Maxwell's demon does not compute. In M. E. Cuffaro and S. C. Fletcher (eds.), Physical Perspectives on Computation, Computational Perspectives on Physics, Cambridge University Press, pp. 240–256. PhilSci-Archive 14808. The demon exorcised with Liouville's theorem alone.

John D. Norton (2026). Too good to be true: entropy, information, computation, and the uninformed thermodynamics of erasure. Philosophy of Science, published online 22 May 2026; manuscript dated 2025. DOI: 10.1017/psa.2026.10221.

Daniel Oberhaus (2018). The World's Oldest Blockchain Has Been Hiding in the New York Times Since 1995. Vice (Motherboard), 27 August 2018. Article. Surety's weekly hash under "Notices & Lost and Found."

Dominic Orchard, Vilem Liepelt and Harley Eades III (2019). Quantitative program reasoning with graded modal types. Proceedings of the ACM on Programming Languages 3(ICFP), article 110. DOI: 10.1145/3341714.

Thomas E. Ouldridge and David H. Wolpert (2023). Thermodynamics of deterministic finite automata operating locally and periodically. New Journal of Physics 25, 123013. DOI: 10.1088/1367-2630/ad1070; arXiv: 2208.06895. The sixth main result: if a language's minimal automaton is non-invertible, every automaton for it is, and recognising it with a local, periodic process produces entropy; worked example, the strings with no b.

Thomas E. Ouldridge and Pieter Rein ten Wolde (2017). Fundamental Costs in the Production and Destruction of Persistent Polymer Copies. Physical Review Letters 118, 158103. DOI: 10.1103/PhysRevLett.118.158103.

Thomas E. Ouldridge, Christopher C. Govern and Pieter Rein ten Wolde (2017). Thermodynamics of Computational Copying in Biochemical Systems. Physical Review X 7, 021004. DOI: 10.1103/PhysRevX.7.021004.

Luca Pacioli (1494). Summa de arithmetica, geometria, proportioni et proportionalità. Venice. The bookkeeping treatise is Distinctio IX, Tractatus XI, Particularis de computis et scripturis.

Bryan Parno, Jacob R. Lorch, John R. Douceur, James Mickens and Jonathan M. McCune (2011). Memoir: practical state continuity for protected modules. Proceedings of the 2011 IEEE Symposium on Security and Privacy, pp. 379–394. DOI: 10.1109/SP.2011.38.

Juan M. R. Parrondo, Jordan M. Horowitz and Takahiro Sagawa (2015). Thermodynamics of information. Nature Physics 11(2), pp. 131–139. DOI: 10.1038/nphys3230.

Max Planck (1920). The genesis and present state of development of the quantum theory. Nobel Lecture, 2 June 1920. Quoted in the translation by H. T. Clarke and L. Silberstein, The Origin and Development of the Quantum Theory, Oxford: Clarendon Press, 1922, p. 11. 1922 translation.

Henri Poincaré (1890). Sur le problème des trois corps et les équations de la dynamique. Acta Mathematica 13, pp. 1–270.

Luciano Pomatto, Philipp Strack and Omer Tamuz (2023). The cost of information: the case of constant marginal costs. American Economic Review 113(5), pp. 1360–1393. DOI: 10.1257/aer.20190185.

Jenny M. Poulton, Pieter Rein ten Wolde and Thomas E. Ouldridge (2019). Nonequilibrium correlations in minimal dynamical models of polymer copying. Proceedings of the National Academy of Sciences 116(6), pp. 1946–1951. arXiv: 1805.08502.

Peter E. Prosser (2001). Church History's Biggest Hoax. Christian History 72. Article. Cusa in 1433, before Valla.

Bernhard Rauer, Sebastian Erne, Thomas Schweigler, Federica Cataldini, Mohammadamin Tajik and Jörg Schmiedmayer (2018). Recurrences in an isolated quantum many-body system. Science 360, pp. 307–310. DOI: 10.1126/science.aan7938. A collective quantity of a sealed gas of thousands of atoms coming back around.

Hans Reichenbach (1956). The Direction of Time. University of California Press; Dover reprint. Stanford Encyclopedia of Philosophy entry on Reichenbach. Cited by Rovelli (2024) as the first to notice that any measurement is dissipative.

Richard Rhodes (1986). The Making of the Atomic Bomb. Simon and Schuster, New York.

David Reeb and Michael M. Wolf (2014). An improved Landauer Principle with finite-size corrections. arXiv: 1306.4352v3. The system/reservoir assumptions and the exact entropy-balance identity are stated explicitly.

Paul M. Riechers, Alexander B. Boyd, Gregory W. Wimsatt and James P. Crutchfield (2020). Balancing error and dissipation in computing. Physical Review Research 2, 033524. DOI: 10.1103/PhysRevResearch.2.033524; arXiv: 1909.06650. Time-symmetric control and nonreciprocal transitions.

James Ritty and John Ritty (1879). Cash Register and Indicator. US patent 221,360, 4 November 1879. See also Ritty's Incorruptible Cashier, American Table. The Dayton saloon, the 1878 steamboat trip and the propeller counter, the dial with no drawer, and the sale through Jacob H. Eckert to John H. Patterson in 1884: Wikipedia, James Ritty.

Ronald L. Rivest and Adi Shamir (1982). How to reuse a write-once memory. Information and Control 55, pp. 1–19. DOI: 10.1016/S0019-9958(82)90344-8.

Carlo Rovelli (2022). Memory and Entropy. Entropy 24(8), article 1022. DOI: 10.3390/e24081022; arXiv: 2003.06687.

Carlo Rovelli (2024). The Thermodynamic Cost of Choosing. Foundations of Physics 54, article 28. arXiv: 2309.08557; DOI: 10.1007/s10701-024-00765-7. "Stabilization must be irreversible" is on p. 3 of the arXiv preprint.

The Royal Society. History of the Royal Society. Web page. First meeting 28 November 1660; the motto adopted in the First Charter, 1662.

Runtime Verification, Inc. and The University of Texas at Austin (2018). casper-proofs: a development of Casper FFG accountable safety and plausible liveness. Version at commit d8fe05df57e59909bf5c3392579e66ac1b78dab8. Repository.

Takahiro Sagawa (2014). Thermodynamic and Logical Reversibilities Revisited. Journal of Statistical Mechanics: Theory and Experiment, P03025. arXiv: 1311.1886v2.

Takahiro Sagawa and Masahito Ueda (2009). Minimal Energy Cost for Thermodynamic Information Processing: Measurement and Information Erasure. Physical Review Letters 102, 250602. arXiv: 0809.4098v3; DOI: 10.1103/PhysRevLett.102.250602. See also the erratum, Physical Review Letters 106, 189901 (2011).

Denise Schmandt-Besserat (1992). Before Writing, volume 1: From Counting to Cuneiform. University of Texas Press.

Denise Schmandt-Besserat. Tokens and Writing: the Cognitive Development. Author's page, University of Texas at Austin. Web page. Hollow clay envelopes with the tokens impressed on the outside, around 3300 BC.

Tim Schmiedl and Udo Seifert (2007). Optimal finite-time processes in stochastic thermodynamics. Physical Review Letters 98, 108301. DOI: 10.1103/PhysRevLett.98.108301; arXiv: cond-mat/0701554.

Bruce Schneier and John Kelsey (1999). Secure audit logs to support computer forensics. ACM Transactions on Information and System Security 2(2), pp. 159–176. DOI: 10.1145/317087.317089.

Alex Shamis and coauthors (2021). IA-CCF: individual accountability for permissioned ledgers. Preprint, arXiv: 2105.13116; published in Proceedings of the 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI '22), 2022, pp. 467–491. USENIX page.

Steven Shapin (1984). Pump and Circumstance: Robert Boyle's Literary Technology. Social Studies of Science 14(4), pp. 481–520. DOI: 10.1177/030631284014004001.

Naoto Shiraishi, Ken Funo and Keiji Saito (2018). Speed limit for classical stochastic processes. Physical Review Letters 121, 070601. DOI: 10.1103/PhysRevLett.121.070601; arXiv: 1802.06554.

James Six (1782). Account of an improved Thermometer. Philosophical Transactions of the Royal Society of London 72, pp. 72–81. DOI: 10.1098/rstl.1782.0012. Quoted: "if it be not immediately noticed, it can never after be known," p. 72.

Rob Smart (2021). Anti-rollback explained. PSA Certified, 27 April 2021. Article.

Raoul Strackx and Frank Piessens (2016). Ariadne: a minimal approach to state continuity. Proceedings of the 25th USENIX Security Symposium, pp. 875–892. USENIX page.

Supreme Court of the United States (2005). Arthur Andersen LLP v. United States, 544 U.S. 696. The conviction reversed over the jury instructions.

Leo Szilard (1929). Über die Entropieverminderung in einem thermodynamischen System bei Eingriffen intelligenter Wesen. Zeitschrift für Physik 53, pp. 840–856. DOI: 10.1007/BF01341281.

Omer Tamuz, Caltech, faculty page; Philipp Strack, Yale, faculty page; Luciano Pomatto, Caltech, HSS new faculty 2016–17. The economists behind the 2023 cost-of-information paper.

Robert Endre Tarjan (1985). Amortized Computational Complexity. SIAM Journal on Algebraic and Discrete Methods 6(2), pp. 306–318. DOI: 10.1137/0606031.

William Thomson (1874). Kinetic Theory of the Dissipation of Energy. Nature 9, pp. 441–444. Nature page.

Tommaso Toffoli (1980). Reversible computing. Automata, Languages and Programming (ICALP 1980), Lecture Notes in Computer Science, pp. 632–644. DOI: 10.1007/3-540-10003-2_104.

Alin Tomescu, Vivek Bhupatiraju, Dimitrios Papadopoulos, Charalampos Papamanthou, Nikos Triandopoulos and Srinivas Devadas (2019). Transparency logs via append-only authenticated dictionaries. Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19), pp. 1299–1316. DOI: 10.1145/3319535.3345652.

Shoichi Toyabe, Takahiro Sagawa, Masahito Ueda, Eiro Muneyuki and Masaki Sano (2010). Experimental demonstration of information-to-energy conversion and validation of the generalized Jarzynski equality. Nature Physics 6, pp. 988–992. DOI: 10.1038/nphys1821.

Timo Treitz and Robert Künnemann (2026). Accountability in Certificate Transparency and variants. Preprint. arXiv: 2609.11552.

Trusted Computing Group (n.d.). Trusted Platform Module Library Specification, Family 2.0, Part 1: Architecture. The platform configuration registers and the extend operation. Specification page.

U.S. Department of Justice (2002). Press release, 15 June 2002, on the jury verdict against Arthur Andersen LLP. Press release.

U.S. Department of Justice (2002). Press release, 2 October 2002, on the criminal complaint against Andrew Fastow; Enron's special purpose entities and off-balance-sheet treatment. Press release.

Jos Uffink (2004, revised 2024). Boltzmann's Work in Statistical Physics. Stanford Encyclopedia of Philosophy. SEP entry. Zermelo as Planck's assistant in 1896; Boltzmann's reply "drenched in sarcasm and bitterness."

University of Waterloo, Department of Pure Mathematics (2020). We remember Distinguished Professor Emeritus János Aczél. Notice. Aczél at Waterloo from 1965, founder of Aequationes Mathematicae. Bruno Forte at Waterloo: Mathematics Genealogy Project.

Lorenzo Valla (c. 1440). De falso credita et ementita Constantini donatione declamatio. The Donation of Constantine. English translation: The Treatise of Lorenzo Valla on the Donation of Constantine, trans. Christopher B. Coleman, Yale University Press, 1922.

Giuliana Santos Veronese, Miguel Correia, Alysson Neves Bessani, Lau Cheuk Lung and Paulo Verissimo (2013). Efficient Byzantine fault-tolerance. IEEE Transactions on Computers 62(1), pp. 16–30. DOI: 10.1109/TC.2011.221.

Andrea Vinante, Rima Mezzena, Paolo Falferi, Matteo Carlesso and Angelo Bassi (2017). Improved noninterferometric test of collapse models using ultracold cantilevers. Physical Review Letters 119, 110401. DOI: 10.1103/PhysRevLett.119.110401. A millikelvin cantilever with a non-thermal force noise of unknown origin, compatible with one model's collapse heating.

Maria Violaris and Chiara Marletto (2022). Irreversibility beyond Landauer's principle: is it possible to perform erasure using the quantum homogenizer? New Journal of Physics 24, 113030. DOI: 10.1088/1367-2630/ac9e13.

David H. Wolpert (1992). Memory systems, computation, and the second law of thermodynamics. International Journal of Theoretical Physics 31(4), pp. 743–785. DOI: 10.1007/BF00670829.

David H. Wolpert (2019). The stochastic thermodynamics of computation. Journal of Physics A: Mathematical and Theoretical 52, 193001. DOI: 10.1088/1751-8121/ab0850; arXiv: 1905.05669. Review of distribution-dependent costs and restrictions imposed by computational architecture.

David H. Wolpert and Jens Kipper (2024). Memory Systems, the Epistemic Arrow of Time, and the Second Law. Entropy 26(2), article 170. DOI: 10.3390/e26020170.

David H. Wolpert and William G. Macready (1997). No free lunch theorems for optimization. IEEE Transactions on Evolutionary Computation 1(1), pp. 67–82. DOI: 10.1109/4235.585893.

Gavin Wood (2014). Ethereum: A Secure Decentralised Generalised Transaction Ledger (the yellow paper). Current version.

Ernst Zermelo (1896). Ueber einen Satz der Dynamik und die mechanische Wärmetheorie. Annalen der Physik und Chemie, Neue Folge 57, pp. 485–494. A separate work from the 1908 set-theory paper listed under the first heading.

Wojciech H. Zurek (1989). Thermodynamic cost of computation, algorithmic complexity and the information metric. Nature 341, pp. 119–124. DOI: 10.1038/341119a0.

E.3 Computation, self-reference and checking

Martín Abadi and Leslie Lamport (1991). The Existence of Refinement Mappings. Theoretical Computer Science 82(2), pp. 253–284. Earlier versions appeared at LICS 1988 and as SRC Research Report 29. Author report; DOI: 10.1016/0304-3975(91)90224-P.

Alok Aggarwal and Jeffrey Scott Vitter (1988). The input/output complexity of sorting and related problems. Communications of the ACM 31(9), pp. 1116–1127. DOI: 10.1145/48529.48535. The I/O model.

Danel Ahman, Cédric Fournet, Cătălin Hriţcu, Kenji Maillard, Aseem Rastogi and Nikhil Swamy (2018). Recalling a witness: foundations and applications of monotonic state. Proceedings of the ACM on Programming Languages 2(POPL), 65:1–65:30. arXiv: 1707.02466.

American Institute of Physics. Labor status of women computers. In Harvard College Observatory history pages. AIP page. Hourly pay of the women computers, and of the men, nearly five times as much.

Arthur Azevedo de Amorim, Maxime Dénès, Nick Giannarakis, Cătălin Hriţcu, Benjamin C. Pierce, Antal Spector-Zabusky and Andrew Tolmach (2015). Micro-policies: formally verified, tag-based security monitors. Proceedings of the 2015 IEEE Symposium on Security and Privacy, pp. 813–830.

James P. Anderson (1972). Computer Security Technology Planning Study. Technical Report ESD-TR-73-51, Electronic Systems Division, Air Force Systems Command, Hanscom Field, Bedford, MA. Early computer security papers.

Andrew W. Appel (2001). Foundational proof-carrying code. Proceedings of LICS 2001, pp. 247–256. DOI: 10.1109/LICS.2001.932501.

Kenneth Appel and Wolfgang Haken (1976). Every planar map is four colorable. Bulletin of the American Mathematical Society 82(5), pp. 711–712.

François Bancilhon and Nicolas Spyratos (1981). Update semantics of relational views. ACM Transactions on Database Systems 6(4), pp. 557–575. DOI: 10.1145/319628.319634.

Henk Barendregt (1984). The Lambda Calculus: Its Syntax and Semantics. Revised edition. Studies in Logic and the Foundations of Mathematics 103. Amsterdam: North-Holland. The second fixed-point theorem.

Francisco Bavera and Eduardo Bonelli (2018). Justification logic and audited computation. Journal of Logic and Computation 28(5), pp. 909–934; first published online in 2015.

Guy Blanc, Caleb Koch, Jane Lange and Li-Yang Tan (2022). The query complexity of certification. Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing (STOC 2022), pp. 623–636. DOI: 10.1145/3519935.3519993; arXiv: 2201.07736.

Manuel Blum (1967). A machine-independent theory of the complexity of recursive functions. Journal of the ACM 14(2), pp. 322–336. DOI: 10.1145/321386.321395. The two axioms for a complexity measure; cited beside the definition of a model of computation in the opening section, The point.

Manuel Blum (1983). Coin Flipping by Telephone: A Protocol for Solving Impossible Problems. ACM SIGACT News 15(1), pp. 23–27. Presented at CRYPTO 1981.

Manuel Blum and Sampath Kannan (1989). Designing programs that check their work. Proceedings of the 21st Annual ACM Symposium on Theory of Computing (STOC '89), pp. 86–97. DOI: 10.1145/73007.73015. Journal version: Journal of the ACM 42(1), 1995, pp. 269–291, DOI: 10.1145/200836.200880.

George Boole (1847). The Mathematical Analysis of Logic, Being an Essay towards a Calculus of Deductive Reasoning. Macmillan, Barclay and Macmillan, Cambridge.

George Boole (1854). An Investigation of the Laws of Thought. Walton and Maberly, London. Chapter III.

Alex Brodsky and Nicholas Pippenger (2002). Characterizations of 1-way quantum finite automata. SIAM Journal on Computing 31(5), pp. 1456–1478. DOI: 10.1137/S0097539799353443. Measure-once automata with bounded error accept exactly the group languages; measure-many automata accept every piecewise testable language.

Samuel Bucheli, Meghdad Ghari and Thomas Studer (2017). Temporal justification logic. Methods for Modalities (M4M9), EPTCS 243, pp. 59–74. arXiv: 1703.02195.

John P. Burgess (1986). The truth is never simple. Journal of Symbolic Logic 51(3), pp. 663–681. DOI: 10.2307/2274021. Quasi-inductive definitions, in the setting of revision theories of truth.

Michael Burrows, Martin Abadi and Roger Needham (1990). A logic of authentication. ACM Transactions on Computer Systems 8(1), pp. 18–36. DOI: 10.1145/77648.77649. First presented at the 12th ACM Symposium on Operating Systems Principles, 1989.

Georg Cantor (1874). Über eine Eigenschaft des Inbegriffes aller reellen algebraischen Zahlen. Journal für die reine und angewandte Mathematik 77, pp. 258–262. DOI: 10.1515/crll.1874.77.258.

Georg Cantor (1891). Über eine elementare Frage der Mannigfaltigkeitslehre. Jahresbericht der Deutschen Mathematiker-Vereinigung 1, pp. 75–78. English: On an elementary question in the theory of manifolds, translated by Anthony Phillips, translation. The symbols m and w, and the comparison with the 1874 proof, are on p. 75 of the original.

Mario Carneiro (2019). Formalizing computability theory via partial recursive functions. Interactive Theorem Proving, ITP 2019, LIPIcs 141, article 12. arXiv: 1810.08380; DOI: 10.4230/LIPIcs.ITP.2019.12.

Ethan Cecchetti, Andrew C. Myers and Owen Arden (2017). Nonmalleable information flow control. CCS 2017, pp. 1875–1891. DOI: 10.1145/3133956.3134054.

K. Mani Chandy and Leslie Lamport (1985). Distributed snapshots: determining global states of distributed systems. ACM Transactions on Computer Systems 3(1), pp. 63–75. DOI: 10.1145/214451.214456. Stable properties.

Alessandro Chiesa and Eran Tromer (2010). Proof-carrying data and hearsay arguments from signature cards. Proceedings of Innovations in Computer Science (ICS 2010), pp. 310–331.

Alonzo Church (1936). An Unsolvable Problem of Elementary Number Theory. American Journal of Mathematics 58(2), pp. 345–363. DOI: 10.2307/2371045.

Clay Mathematics Institute (n.d.). The Millennium Prize Problems. Web page.

Clay Mathematics Institute (n.d.). The Poincaré Conjecture. Web page. Perelman's preprints of 2002 and 2003, and the prize of 2010.

Edgar F. Codd (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the ACM 13(6), pp. 377–387. DOI: 10.1145/362384.362685.

Thomas Colcombet and Daniela Petrişan (2020). Automata minimization: a functorial approach. Logical Methods in Computer Science 16(1), paper 32. DOI: 10.23638/LMCS-16(1:32)2020. Journal version of the CALCO 2017 paper.

Stephen A. Cook (1971). The complexity of theorem-proving procedures. Proceedings of STOC 1971, pp. 151–158. DOI: 10.1145/800157.805047.

Stephen A. Cook and Robert A. Reckhow (1973). Time bounded random access machines. Journal of Computer and System Sciences 7(4), pp. 354–375. DOI: 10.1016/S0022-0000(73)80029-7.

Patrick Cousot and Radhia Cousot (1977). Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. Proceedings of POPL 1977, pp. 238–252. Author publication record and paper.

Vincent Danos and Jean Krivine (2004). Reversible Communicating Systems. CONCUR 2004, Lecture Notes in Computer Science 3170, pp. 292–307. DOI: 10.1007/978-3-540-28644-8_19.

Vincent Danos and Jean Krivine (2005). Transactions in RCCS. CONCUR 2005, Lecture Notes in Computer Science 3653, pp. 398–412. DOI: 10.1007/11539452_31.

Martin Davis (1958). Computability and Unsolvability. New York: McGraw-Hill. Reprinted by Dover, 1982.

Richard Dedekind (1888). Was sind und was sollen die Zahlen? Vieweg, Braunschweig. The definition of infinite is in Section 5, item 64: a system is infinite when it is similar (maps one-to-one) onto a proper part of itself, and finite otherwise.

Dorothy E. Denning (1976). A lattice model of secure information flow. Communications of the ACM 19(5), pp. 236–243.

Jack B. Dennis and Earl C. Van Horn (1966). Programming semantics for multiprogrammed computations. Communications of the ACM 9(3), pp. 143–155. DOI: 10.1145/365230.365252. Capabilities.

Harry Deutsch and Oliver Marshall (2021, revised 2022). Alonzo Church. Stanford Encyclopedia of Philosophy. Entry.

Edsger W. Dijkstra (1982). Why numbering should start at zero. EWD 831, 11 August 1982. Transcription.

Wolfgang Doeblin. His life (birth in Berlin in 1915, doctorate at the Sorbonne in 1938, work on Markov chains, death in June 1940) is from J. J. O'Connor and E. F. Robertson, "Wolfgang Doeblin," MacTutor History of Mathematics archive, University of St Andrews, mathshistory.st-andrews.ac.uk.

Rod Downey, Carl Jockusch, Timothy H. McNicholl and Paul Schupp (2014). Asymptotic density and the Ershov hierarchy. arXiv: 1309.0137. States that the 1-c.e. sets (one change of guess, from no to yes) are precisely the c.e. sets.

Andrej Dudenhefner (2022). Certified Decision Procedures for Two-Counter Machines. FSCD 2022, LIPIcs 228, article 16. The successful-decrement jump convention used in the counter guest. DOI: 10.4230/LIPIcs.FSCD.2022.16.

W. H. Eccles and F. W. Jordan (1919). A trigger relay utilizing three-electrode thermionic vacuum tubes. The Electrician 83, p. 298; reprinted in Radio Review 1(3), pp. 143–146.

Yu. L. Ershov (1968). A hierarchy of sets. I. Algebra and Logic 7, pp. 25–43; translated from Algebra i Logika 7(1), pp. 47–74. DOI: 10.1007/BF02218750.

Benja Fallenstein and Nate Soares (2015). Vingean reflection: reliable reasoning for self-improving agents. Technical Report 2015–2, Machine Intelligence Research Institute. Report.

Solomon Feferman (2006). Are There Absolutely Unsolvable Problems? Gödel's Dichotomy. Philosophia Mathematica (III) 14(2), pp. 134–152. Author's text; DOI: 10.1093/philmat/nkj003.

Solomon Feferman (2006). The nature and significance of Gödel's incompleteness theorems. Lecture, Institute for Advanced Study, Gödel Centenary Program, 17 November 2006. Author's text.

Robert W. Floyd (1967). Assigning Meanings to Programs. In J. T. Schwartz (ed.), Mathematical Aspects of Computer Science, Proceedings of Symposia in Applied Mathematics 19, American Mathematical Society, pp. 19–32.

Lester R. Ford Jr. and Selmer M. Johnson (1959). A Tournament Problem. American Mathematical Monthly 66(5), pp. 387–389.

Yannick Forster and Dominique Larchey-Wendling (2019). Certified undecidability of intuitionistic linear logic via binary stack machines and Minsky machines. Certified Programs and Proofs, CPP 2019, pp. 104–117. DOI: 10.1145/3293880.3294096.

Yannick Forster and Fabian Kunze (2019). Interactive Theorem Proving, ITP 2019, LIPIcs 141, article 17. DOI: 10.4230/LIPIcs.ITP.2019.17.

Yannick Forster and Gert Smolka (2017). Interactive Theorem Proving, ITP 2017, LNCS 10499, pp. 189–206. Author paper; DOI: 10.1007/978-3-319-66107-0_13.

Yannick Forster, Dominik Kirst and Gert Smolka (2019). Certified Programs and Proofs, CPP 2019, pp. 38–51. DOI: 10.1145/3293880.3294091.

Yannick Forster, Dominique Larchey-Wendling, Andrej Dudenhefner, Edith Heiter, Dominik Kirst, Fabian Kunze, Gert Smolka, Simon Spies, Dominik Wehr and Maximilian Wuttke (2020). Workshop contribution and accompanying software library of undecidability results, January 2020.

Galileo Galilei (1586). La Bilancetta (The Little Balance). Described at the Galileo Project, Rice University: page. The words quoted are from the English translation in Laura Fermi and Gilberto Bernardini, Galileo and the Scientific Revolution, Basic Books, 1961, as reproduced at math.nyu.edu; I didn't see the printed page.

The Galileo Project. Galileo Galilei. Rice University. Biography.

George Gamow (1947). One Two Three... Infinity. Viking Press, New York.

Deepak Garg and Frank Pfenning (2010). A proof-carrying file system. IEEE Symposium on Security and Privacy 2010, pp. 349–364.

Carl Friedrich Gauss (1831). Untersuchungen über die Eigenschaften der positiven ternären quadratischen Formen von Ludwig August Seeber. Göttingische gelehrte Anzeigen, 9 July 1831; reprinted in Journal für die reine und angewandte Mathematik 20 (1840), pp. 312–320.

Roberto Giacobazzi, Francesco Ranzato and Francesca Scozzari (2000). Making abstract interpretations complete. Journal of the ACM 47(2), pp. 361–416. Author publication record.

Jean-Yves Girard (1987). Linear logic. Theoretical Computer Science 50(1), pp. 1–101. DOI: 10.1016/0304-3975(87)90045-4.

Kurt Gödel (1931). Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. Monatshefte für Mathematik und Physik 38, pp. 173–198. DOI: 10.1007/BF01700692.

Kurt Gödel (1951). Some basic theorems on the foundations of mathematics and their implications. Gibbs Lecture, American Mathematical Society, Brown University, 26 December 1951. In Kurt Gödel, Collected Works, Vol. III: Unpublished Essays and Lectures, S. Feferman et al. (eds.), Oxford University Press, 1995, pp. 304–323.

Joseph A. Goguen (1972). Realization is universal. Mathematical Systems Theory 6, pp. 359–374. DOI: 10.1007/BF01843493.

E. Mark Gold (1965). Limiting recursion. Journal of Symbolic Logic 30, pp. 28–48.

Shafi Goldwasser, Silvio Micali and Charles Rackoff (1985). The Knowledge Complexity of Interactive Proof-Systems. Proceedings of the 17th Annual ACM Symposium on Theory of Computing, pp. 291–304.

Georges Gonthier (2008). Formal proof: the four-color theorem. Notices of the American Mathematical Society 55(11), pp. 1382–1393. The derivation discussed there was completed in 2005.

Andrew D. Gordon and Alan Jeffrey (2003). Authenticity by typing for security protocols. Journal of Computer Security 11(4), pp. 451–519. DOI: 10.3233/JCS-2003-11402.

Michael J. C. Gordon, Robin Milner and Christopher P. Wadsworth (1979). Edinburgh LCF: A Mechanised Logic of Computation. Lecture Notes in Computer Science 78, Springer. DOI: 10.1007/3-540-09724-4.

Michael J. C. Gordon (2000). From LCF to HOL: a short history. In Gordon Plotkin, Colin Stirling and Mads Tofte (eds.), Proof, Language, and Interaction: Essays in Honour of Robin Milner, MIT Press, pp. 169–185. Paper.

David Alan Grier (2005). When Computers Were Human. Princeton University Press.

Thomas C. Hales (2005). A proof of the Kepler conjecture. Annals of Mathematics 162(3), pp. 1065–1185. DOI: 10.4007/annals.2005.162.1065.

Thomas Hales (2024). The Formal Proof of the Kepler Conjecture: a critical retrospective. arXiv: 2402.08032. The panel of twelve referees, their exhaustion, and the formal proof begun to get past them are on p. 2.

Thomas Hales and others (2017). A formal proof of the Kepler conjecture. Forum of Mathematics, Pi 5, e2. DOI: 10.1017/fmp.2017.1.

Joseph Y. Halpern, Yoram Moses and Mark R. Tuttle (1988). A knowledge-based analysis of zero knowledge. Proceedings of the 20th Annual ACM Symposium on Theory of Computing (STOC '88), pp. 132–147. DOI: 10.1145/62212.62224.

Joel David Hamkins and Andy Lewis (2000). Infinite time Turing machines. Journal of Symbolic Logic 65(2), pp. 567–604. DOI: 10.2307/2586556; arXiv: math/9808093.

Joel David Hamkins and Theodor Nenu (2026). Did Turing prove the undecidability of the halting problem? Journal of Logic and Computation 36(1), exaf075. DOI: 10.1093/logcom/exaf075; arXiv: 2407.00680.

Kevin W. Hamlen, Greg Morrisett and Fred B. Schneider (2006). Computability classes for enforcement mechanisms. ACM Transactions on Programming Languages and Systems 28(1), pp. 175–205.

Klaus Havelund and Grigore Roşu (2002). Synthesizing monitors for safety properties. Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2002), Lecture Notes in Computer Science 2280, pp. 342–356. DOI: 10.1007/3-540-46002-0_24.

Percy John Heawood (1890). Map-colour theorem. Quarterly Journal of Pure and Applied Mathematics 24, pp. 332–338. I haven't seen the original; "the now apparently recognised proof" is as quoted in secondary accounts, one of which spells it "recognized."

David Hilbert (1900). Mathematische Probleme. Nachrichten von der Königlichen Gesellschaft der Wissenschaften zu Göttingen, pp. 253–297. English translation by Mary Winston Newson, Mathematical Problems, Bulletin of the American Mathematical Society 8 (1902), pp. 437–479.

C. A. R. Hoare (1969). An Axiomatic Basis for Computer Programming. Communications of the ACM 12(10), pp. 576–580. DOI: 10.1145/363235.363259.

Douglas R. Hofstadter (1979). Gödel, Escher, Bach: An Eternal Golden Braid. Basic Books.

Kohei Honda (1993). Types for dyadic interaction. CONCUR '93, LNCS 715, pp. 509–523. DOI: 10.1007/3-540-57208-2_35.

William Stanley Jevons (1864). Pure Logic, or the Logic of Quality apart from Quantity. Edward Stanford, London.

Juvenal. Satires, Satire VI, lines 346–348, and the Oxford fragment found by E. O. Winstedt in a Bodleian manuscript in 1899, published as E. O. Winstedt (1899), A Bodleian MS. of Juvenal, The Classical Review 13, pp. 201–205. The debate over whether the lines are Juvenal's is summarized at Wikipedia, Quis custodiet ipsos custodes?.

Rudolf E. Kalman (1960). A New Approach to Linear Filtering and Prediction Problems. Journal of Basic Engineering 82(1), pp. 35–45. DOI: 10.1115/1.3662552.

Rudolf E. Kalman (1960). Contributions to the Theory of Optimal Control. Boletín de la Sociedad Matemática Mexicana 5, pp. 102–119. Reprinted paper.

Paul A. Karger and Roger R. Schell (1974). Multics Security Evaluation: Vulnerability Analysis. Technical Report ESD-TR-74-193, Vol. II, Electronic Systems Division, Air Force Systems Command, June 1974.

Alfred Bray Kempe (1879). On the Geographical Problem of the Four Colours. American Journal of Mathematics 2(3), pp. 193–200. DOI: 10.2307/2369235. Quoted: p. 194.

Johannes Kepler (1611). Strena seu de Nive Sexangula. Frankfurt am Main: Godefrid Tampach. English: The Six-Cornered Snowflake, translated by Colin Hardie, with essays by L. L. Whyte and B. J. Mason, Oxford: Clarendon Press, 1966. I haven't seen the Hardie translation; the words quoted from it are as given in secondary accounts.

Stephen Cole Kleene (1938). On notation for ordinal numbers. Journal of Symbolic Logic 3(4), pp. 150–155. DOI: 10.2307/2267778. The second recursion theorem is stated and proved at the end of section 2.

Stephen Cole Kleene (1952). Introduction to Metamathematics. North-Holland and D. Van Nostrand. Partial recursive functions, parameterisation and recursion theorems.

Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch and Simon Winwood (2009). seL4: formal verification of an OS kernel. Proceedings of SOSP 2009, pp. 207–220. DOI: 10.1145/1629575.1629596.

Donald E. Knuth (1998). The Art of Computer Programming, Volume 3: Sorting and Searching, second edition. Addison-Wesley. Section 5.3.1, merge insertion, pp. 184–186.

Donald E. Knuth (n.d.). Home page, Stanford University: "Professor Emeritus of The Art of Computer Programming at Stanford University." Page.

Donald E. Knuth (n.d.). The Art of Computer Programming (TAOCP) home page, on the reward for errors. Page. See also Knuth's news for 2008, page, on the Bank of San Serriffe.

Helge Kragh (2014). The True (?) Story of Hilbert's Infinite Hotel. arXiv: 1403.0059.

Lindsey Kuper and Ryan R. Newton (2013). LVars: lattice-based data structures for deterministic parallelism. Proceedings of the 2nd ACM SIGPLAN Workshop on Functional High-Performance Computing (FHPC 2013), pp. 71–84. DOI: 10.1145/2502323.2502326.

Joachim Lambek (1961). How to Program an Infinite Abacus. Canadian Mathematical Bulletin 4(3), pp. 295–302. Paper. "Our method is probably not new" is on p. 295.

Joachim Lambek and Philip J. Scott (1986). Introduction to Higher Order Categorical Logic. Cambridge Studies in Advanced Mathematics 7. Cambridge University Press.

F. William Lawvere (1969). Adjointness in foundations. Dialectica 23(3–4), pp. 281–296. DOI: 10.1111/j.1746-8361.1969.tb01194.x. The quantifiers as the two adjoints of substitution.

F. William Lawvere (1969). Diagonal arguments and cartesian closed categories. Lecture Notes in Mathematics 92, pp. 134–145. Reprinted with author commentary in Reprints in Theory and Applications of Categories 15 (2006), pp. 1–13. Authorized reprint.

Xavier Leroy (2009). Formal verification of a realistic compiler. Communications of the ACM 52(7), pp. 107–115. DOI: 10.1145/1538788.1538814.

Leonid A. Levin (1973). Universal sequential search problems. Problemy Peredachi Informatsii 9(3), pp. 115–116; English translation in Problems of Information Transmission 9(3), pp. 265–266. Record.

Orna Lichtenstein, Amir Pnueli and Lenore Zuck (1985). The glory of the past. Logics of Programs, Lecture Notes in Computer Science 193, pp. 196–218. DOI: 10.1007/3-540-15648-8_16. Past-time temporal logic.

Martin H. Löb (1955). Solution of a problem of Leon Henkin. Journal of Symbolic Logic 20(2), pp. 115–118.

J. R. Lucas (1961). Minds, Machines and Gödel. Philosophy 36(137), pp. 112–127. DOI: 10.1017/S0031819100057983.

Charles E. Mackenzie (1980). Coded Character Sets: History and Development. Addison-Wesley. The design history of ASCII, including the placing of the lower-case letters so that each differs from its capital by one bit.

Elizabeth J. Magie (1904). Game-Board (The Landlord's Game). US patent 748,626, filed 23 March 1903, issued 5 January 1904. Her description of the game as "a practical demonstration of the present system of land-grabbing" is from The Single Tax Review, Autumn 1902, as transcribed at landlordsgame.info; I didn't see the printed page. Parker Brothers' purchase of her 1923 patent for $500 in 1935: Wikipedia, The Landlord's Game.

John McCarthy (1979). History of Lisp. Author's version dated 12 February 1979. Section "The implementation of LISP".

Leonard A. McGee and Stanley F. Schmidt (1985). Discovery of the Kalman Filter as a Practical Tool for Aerospace and Industry. NASA Technical Memorandum 86847, Ames Research Center. NTRS.

Z. A. Melzak (1961). An Informal Arithmetical Approach to Computability and Computation. Canadian Mathematical Bulletin 4(3), pp. 279–293.

Marvin L. Minsky (1961). Recursive Unsolvability of Post's Problem of Tag and Other Topics in Theory of Turing Machines. Annals of Mathematics 74(3), pp. 437–455. Original paper; DOI: 10.2307/1970290.

Marvin L. Minsky (1967). Computation: Finite and Infinite Machines. Prentice-Hall. Two-counter machines are universal, so their halting problem is undecidable.

Torben Æ. Mogensen (1992). Efficient Self-Interpretation in Lambda Calculus. Journal of Functional Programming 2(3), pp. 345–364. DOI: 10.1017/S0956796800000423. Scott's encoding carried from data to lambda terms themselves.

Liesbeth De Mol (2008). Tag systems and Collatz-like functions. Theoretical Computer Science 390(1), pp. 92–101. DOI: 10.1016/j.tcs.2007.10.020.

Edward F. Moore (1956). Gedanken-Experiments on Sequential Machines. In Claude E. Shannon and John McCarthy (eds.), Automata Studies, pp. 129–153. Princeton University Press. Original paper.

Augustus De Morgan (1864). On the Syllogism, No. IV, and on the Logic of Relations. Transactions of the Cambridge Philosophical Society 10. Sources disagree on the page range and on the date it was read.

F. L. Morris and C. B. Jones (1984). An early program proof by Alan Turing. Annals of the History of Computing 6(2), pp. 139–143. Paper.

James H. Morris, Jr. (1973). Protection in programming languages. Communications of the ACM 16(1), pp. 15–21. DOI: 10.1145/361932.361937.

Yiannis N. Moschovakis (2009). Kleene's Amazing Second Recursion Theorem. Computer Science Logic, LNCS 5771, pp. 24–39. "This little gem is stated unbilled and proved (completely) in the last two lines of §2 of the short note Kleene (1938)." Longer version: Bulletin of Symbolic Logic 16(2) (2010), pp. 189–239. Author lecture text.

George C. Necula (1997). Proof-Carrying Code. Proceedings of POPL 1997, pp. 106–119. Author publication list.

George C. Necula and Peter Lee (1996). Safe Kernel Extensions Without Run-Time Checking. Proceedings of the 2nd USENIX Symposium on Operating Systems Design and Implementation (OSDI '96), pp. 229–243. USENIX page.

Kyndylan Nienhuis, Alexandre Joannou, Thomas Bauereiss, Anthony Fox, Michael Roe, Brian Campbell, Matthew Naylor, Robert M. Norton, Simon W. Moore, Peter G. Neumann, Ian Stark, Robert N. M. Watson and Peter Sewell (2020). Rigorous engineering for hardware security: formal modelling and proof in the CHERI design and implementation process. 2020 IEEE Symposium on Security and Privacy, pp. 1003–1020. DOI: 10.1109/SP40000.2020.00055. Technical report version: University of Cambridge UCAM-CL-TR-940, which states the reachable-capability monotonicity property.

Michael Norrish (2011). Mechanised Computability Theory. Interactive Theorem Proving, ITP 2011, LNCS 6898, pp. 297–311. Author paper; DOI: 10.1007/978-3-642-22863-6_22.

Caspar Oesterheld (2019). Robust program equilibrium. Theory and Decision 86, pp. 143–159. DOI: 10.1007/s11238-018-9679-3.

Susan Owicki and David Gries (1976). An axiomatic proof technique for parallel programs I. Acta Informatica 6(4), pp. 319–340. DOI: 10.1007/BF00268134.

Catuscia Palamidessi, Vijay Saraswat, Frank D. Valencia and Björn Victor (2006). On the expressiveness of linearity vs persistence in the asynchronous pi-calculus. Proceedings of the 21st Annual IEEE Symposium on Logic in Computer Science (LICS 2006), pp. 59–68. DOI: 10.1109/LICS.2006.39.

Parker Brothers (Hasbro) (n.d.). Monopoly, official rules, the section on the Bank.

Bryan Parno, Jon Howell, Craig Gentry and Mariana Raykova (2013). Pinocchio: nearly practical verifiable computation. Proceedings of the 2013 IEEE Symposium on Security and Privacy, pp. 238–252. IACR ePrint 2013/279.

Giuseppe Peano (1889). Arithmetices principia, nova methodo exposita. Bocca, Turin.

Charles S. Peirce (1870, printed 1873). Description of a Notation for the Logic of Relatives, Resulting from an Amplification of the Conceptions of Boole's Calculus of Logic. Memoirs of the American Academy of Arts and Sciences 9(2), pp. 317–378.

Roger Penrose (1989). The Emperor's New Mind: Concerning Computers, Minds, and the Laws of Physics. Oxford University Press.

Roger Penrose (1994). Shadows of the Mind: A Search for the Missing Science of Consciousness. Oxford University Press.

Oskar Pfungst (1907). Das Pferd des Herrn von Osten (Der Kluge Hans). J. A. Barth, Leipzig. English translation: Clever Hans (The Horse of Mr. von Osten), trans. C. L. Rahn, Henry Holt, New York, 1911. Wilhelm von Osten as a mathematics teacher, the exhibitions, Pfungst's tests, and von Osten never persuaded, as summarized at Wikipedia, Clever Hans.

Jean-Éric Pin (1992). On reversible automata. In LATIN '92, 1st Latin American Symposium on Theoretical Informatics, Lecture Notes in Computer Science 583, Springer, pp. 401–416. DOI: 10.1007/BFb0023844.

Serge A. Plotkin (1989). Sticky Bits and Universality of Consensus. Proceedings of PODC 1989, pp. 159–175. DOI: 10.1145/72981.72992.

Johannes Åman Pohjola and Joachim Parrow (2016). The expressive power of monotonic parallel composition. Programming Languages and Systems, ESOP 2016, Lecture Notes in Computer Science, Springer, pp. 780–803. DOI: 10.1007/978-3-662-49498-1_30.

Gerald J. Popek and Robert P. Goldberg (1974). Formal Requirements for Virtualizable Third Generation Architectures. Communications of the ACM 17(7), pp. 412–421.

Emil L. Post (1947). Recursive Unsolvability of a Problem of Thue. Journal of Symbolic Logic 12(1), pp. 1–11. The defect in Turing's appendix and its correction are

described in Liesbeth De Mol, Turing Machines, Stanford Encyclopedia of Philosophy, entry.

Hilary Putnam (1965). Trial and error predicates and the solution to a problem of Mostowski. Journal of Symbolic Logic 30, pp. 49–57.

W. V. O. Quine (1962). Paradox. Scientific American 206(4), April 1962, beginning p. 84. Reprinted as "The Ways of Paradox" in The Ways of Paradox and Other Essays, Harvard University Press, 1966. I haven't seen the original printing; the quoted phrase is as the essay is usually quoted.

Jean-Jacques Quisquater, Myriam Quisquater, Muriel Quisquater, Michaël Quisquater, Louis Guillou, Marie Annick Guillou, Gaïd Guillou, Anna Guillou, Gwenolé Guillou, Soazig Guillou and Thomas A. Berson (1990). How to Explain Zero-Knowledge Protocols to Your Children. Advances in Cryptology, CRYPTO '89, LNCS 435, pp. 628–631. Springer. DOI: 10.1007/0-387-34805-0_60.

Henry Gordon Rice (1953). Classes of Recursively Enumerable Sets and Their Decision Problems. Transactions of the American Mathematical Society 74(2), pp. 358–366. DOI: 10.1090/S0002-9947-1953-0053041-6.

Jules Richard (1905). Les principes des mathématiques et le problème des ensembles. Revue générale des sciences pures et appliquées 16(12), pp. 541–543. English translation in Jean van Heijenoort (ed.), From Frege to Gödel, Harvard University Press, 1967, pp. 142–144. His life (school posts, doctorate, geometry) is from J. J. O'Connor and E. F. Robertson, "Jules Antoine Richard," MacTutor History of Mathematics archive, University of St Andrews, mathshistory.st-andrews.ac.uk.

Wayne Richter and Peter Aczel (1974). Inductive definitions and reflecting properties of admissible ordinals. In J. E. Fenstad and P. G. Hinman (eds.), Generalized Recursion Theory, North-Holland, Studies in Logic and the Foundations of Mathematics 79, pp. 301–381. DOI: 10.1016/S0049-237X(08)70592-5.

John Scott Robin and Cynthia E. Irvine (2000). Analysis of the Intel Pentium's Ability to Support a Secure Virtual Machine Monitor. Proceedings of the 9th USENIX Security Symposium, Denver, pp. 129–144. Paper.

Hartley Rogers, Jr. (1967). Theory of Recursive Functions and Effective Computability. New York: McGraw-Hill. Acceptable numberings and the recursion theorem.

Morgan Rogers (2019). Toposes of discrete monoid actions. arXiv: 1905.10277.

Morgan Rogers (2021). Toposes of monoid actions. arXiv: 2112.10198.

J. Barkley Rosser (1936). Extensions of some theorems of Gödel and Church. Journal of Symbolic Logic 1(3), pp. 87–91. DOI: 10.2307/2269028. Rosser's trick, which

weakens Gödel's ω-consistency premise to consistency. Rosser as a student of Alonzo Church at Princeton: Wikipedia, J. Barkley Rosser.

James S. Royer and John Case (1994). Subrecursive Programming Systems. Boston: Birkhäuser. DOI: 10.1007/978-1-4612-0249-3.

Bertrand Russell (1903). The Principles of Mathematics. Cambridge University Press. Chapter X, The Contradiction, §100: "I was led to it in the endeavour to reconcile Cantor's proof that there can be no greatest cardinal number with the very plausible supposition that the class of all terms ... has necessarily the greatest possible number of members." Text of §100.

Bertrand Russell (1906). Les paradoxes de la logique. Revue de métaphysique et de morale 14, pp. 627–650.

A. Saboori and C. N. Hadjicostis (2007). Notions of security and opacity in discrete event systems. Proceedings of the 46th IEEE Conference on Decision and Control, pp. 5056–5061. DOI: 10.1109/CDC.2007.4434515.

Anooshiravan Saboori (2011). Verification and Enforcement of State-Based Notions of Opacity in Discrete Event Systems. PhD thesis, University of Illinois at Urbana-Champaign; adviser Christoforos N. Hadjicostis. IDEALS.

M. Sampath, R. Sengupta, S. Lafortune, K. Sinnamohideen and D. C. Teneketzis (1996). Failure diagnosis using discrete-event models. IEEE Transactions on Control Systems Technology 4(2), pp. 105–124. Its first page gives Sampath, Sengupta, Lafortune and Teneketzis at the University of Michigan and Sinnamohideen at Johnson Controls. DOI: 10.1109/87.486338.

M. Sampath, R. Sengupta, S. Lafortune, K. Sinnamohideen and D. Teneketzis (1995). Diagnosability of discrete-event systems. IEEE Transactions on Automatic Control 40(9), pp. 1555–1575. DOI: 10.1109/9.412626.

Fred B. Schneider (2000). Enforceable security policies. ACM Transactions on Information and System Security 3(1), pp. 30–50. DOI: 10.1145/353323.353382.

Rich Schroeppel (1972). A Two Counter Machine Cannot Calculate 2^N. MIT Artificial Intelligence Laboratory, AI Memo 257, 1 May 1972. MIT DSpace.

Marc Shapiro, Nuno Preguiça, Carlos Baquero and Marek Zawirski (2011). A comprehensive study of Convergent and Commutative Replicated Data Types. INRIA Research Report RR-7506. HAL: inria-00555588.

J. C. Shepherdson and H. E. Sturgis (1963). Computability of recursive functions. Journal of the ACM 10(2), pp. 217–255. DOI: 10.1145/321160.321170. The unlimited register machine.

Geoffrey Smith (2009). On the foundations of quantitative information flow. Foundations of Software Science and Computational Structures, FoSSaCS 2009, LNCS 5504, pp. 288–302. DOI: 10.1007/978-3-642-00596-1_21.

Raymond M. Smullyan (1993). Recursion Theory for Metamathematics. New York: Oxford University Press. DOI: 10.1093/oso/9780195082326.001.0001.

Raymond M. Smullyan (1994). Diagonalization and Self-Reference. Oxford: Clarendon Press. DOI: 10.1093/oso/9780198534501.001.0001.

Dava Sobel (2016). The Glass Universe: How the Ladies of the Harvard Observatory Took the Measure of the Stars. Viking.

William E. Story (1879). Note on the Preceding Paper. American Journal of Mathematics 2(3), from p. 201. DOI: 10.2307/2369236. “Substantially proved” is on p. 201.

George G. Szpiro (2003). Does the proof stack up? Nature 424, pp. 12–13. The Annals editor’s letter as Hales reported it, ending “run out of energy . . . ”.

George G. Szpiro (2006). The Secret Life of Numbers. Washington, DC: Joseph Henry Press. Chapter 12, “Proving the Proof.”

Chamseddine Talhi, Nadia Tawbi and Mourad Debbabi (2008). Execution monitoring enforcement under memory-limitation constraints. Information and Computation 206, pp. 158–184.

Ken Thompson (1984). Reflections on Trusting Trust. Communications of the ACM 27(8), pp. 761–763. Delivered as the 1983 Turing Award lecture. DOI: 10.1145/358198.358210.

Amin Timany and Lars Birkedal (2021). Reasoning about monotonicity in separation logic. Proceedings of the 10th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2021), pp. 91–104. DOI: 10.1145/3437992.3439931.

Alan M. Turing (1936–1937). On Computable Numbers, with an Application to the Entscheidungsproblem. Proceedings of the London Mathematical Society, series 2, volume 42, pp. 230–265. Publisher’s DOI record. The “note of instructions” is in Section 9, argument III, p. 253.

Alan M. Turing (1937). Computability and λ-definability. Journal of Symbolic Logic 2(4), pp. 153–163. DOI: 10.2307/2268280. The computable functions are the λ-definable functions of Church and the general recursive functions.

Alan M. Turing (1949). Checking a large routine. In Report of a Conference on High Speed Automatic Calculating Machines, University Mathematical Laboratory, Cambridge, pp. 67–69. Turing Digital Archive. “Definite assertions which can be

checked individually” is in the opening paragraph; I read it in the reprint by F. L. Morris and C. B. Jones, Annals of the History of Computing 6(2), 1984, p. 139.

United States of America Standards Institute (1967). USA Standard Code for Information Interchange, USAS X3.4-1967. The revision that added the lower-case letters, one bit away from the capitals.

Leslie G. Valiant (1990). A bridging model for parallel computation. Communications of the ACM 33(8), pp. 103–111. DOI: 10.1145/79173.79181. The bulk-synchronous parallel (BSP) model.

Vitruvius. The Ten Books on Architecture, Book IX, Introduction. Trans. Morris Hicky Morgan, Harvard University Press, 1914. Passage.

Hao Wang (1957). A variant to Turing’s theory of computing machines. Journal of the ACM 4(1), pp. 63–92. DOI: 10.1145/320856.320867. Machines that mark a square and never erase one.

Mark B. Wells (1965). Applications of a language for computing in combinatorics. Information Processing 65, Proceedings of IFIP Congress 1965, pp. 497–498.

David A. Wheeler (2005). Countering Trusting Trust through Diverse Double-Compiling. Proceedings of the 21st Annual Computer Security Applications Conference (ACSAC), Tucson, pp. 28–40. Author’s paper.

David A. Wheeler (2009). Fully Countering Trusting Trust through Diverse Double-Compiling. PhD dissertation, George Mason University. Author’s page.

Stephen Wolfram (2021). After 100 Years, Can We Finally Crack Post’s Problem of Tag? A Story of Computational Irreducibility, and More. Stephen Wolfram Writings, March 2021. Article.

Wai Wong (1995). Recording and checking HOL proofs. In Higher Order Logic Theorem Proving and Its Applications (TPHOLs 1995), Lecture Notes in Computer Science 971, Springer, pp. 353–368.

Thomas Y. C. Woo and Simon S. Lam (1993). A semantic model for authentication protocols. Proceedings of the 1993 IEEE Computer Society Symposium on Research in Security and Privacy, pp. 178–194. DOI: 10.1109/RISP.1993.287633. Correspondence and secrecy as the two basic properties.

Noson S. Yanofsky (2003). A Universal Approach to Self-Referential Paradoxes, Incompleteness and Fixed Points. Bulletin of Symbolic Logic 9(3), pp. 362–386. arXiv: math/0305282; DOI: 10.2178/bsl/1058448677.

Andrew Chi-Chih Yao (1981). Should tables be sorted? Journal of the ACM 28(3), pp. 615–628. DOI: 10.1145/322261.322274. The cell-probe model; an earlier version appeared at FOCS 1978.

Eliezer Yudkowsky and Marcello Herreshoff (2013). Tiling agents for self-modifying AI, and the Löbian obstacle. Machine Intelligence Research Institute, draft report. Report.

E.4 Correlations and environmental records

Rotem Arnon-Friedman, Frédéric Dupuis, Omar Fawzi, Renato Renner and Thomas Vidick (2018). Practical device-independent quantum cryptography via entropy accumulation. Nature Communications 9, 459. DOI: 10.1038/s41467-017-02307-4.

Alain Aspect, Jean Dalibard and Gérard Roger (1982). Experimental Test of Bell's Inequalities Using Time-Varying Analyzers. Physical Review Letters 49, pp. 1804–1807. DOI: 10.1103/PhysRevLett.49.1804. The group was at the Institut d'Optique in Orsay, as Aspect tells it in his 2022 interview with Phillips and Dalibard (arXiv: 2212.04737).

Philip Ball (2019). Quantum Darwinism, an Idea to Explain Objective Reality, Passes First Tests. Quanta Magazine, 22 July 2019. Article.

Jonathan Barrett, Roger Colbeck and Adrian Kent (2013). Memory attacks on device-independent quantum cryptography. Physical Review Letters 110, 010503. DOI: 10.1103/PhysRevLett.110.010503.

John S. Bell (1964). On the Einstein Podolsky Rosen paradox. Physics Physique Fizika 1(3), pp. 195–200. DOI: 10.1103/PhysicsPhysiqueFizika.1.195.

John S. Bell (1975). On wave packet reduction in the Coleman-Hepp model. Helvetica Physica Acta 48, p. 93.

John S. Bell (1990). Against "measurement". Physics World 3(8), pp. 33–40.

Robin Blume-Kohout and Wojciech H. Zurek (2006). Quantum Darwinism: Entanglement, branches, and the emergent classicality of redundantly stored quantum information. Physical Review A 73, 062310. arXiv: quant-ph/0505031; DOI: 10.1103/PhysRevA.73.062310.

Niels Bohr (1958). Atomic Physics and Human Knowledge. Wiley. The essay Unity of Knowledge (1954). The page number, p. 73, is the one Mauro Dorato and Jan Faye (2026) give when they quote the line; I haven't seen the page myself.

Fernando G. S. L. Brandão, Marco Piani and Paweł Horodecki (2015). Generic emergence of classical features in quantum Darwinism. Nature Communications 6, 7908. DOI: 10.1038/ncomms8908.

Nicolas Brunner, Daniel Cavalcanti, Stefano Pironio, Valerio Scarani and Stephanie Wehner (2014). Bell nonlocality. Reviews of Modern Physics 86, pp. 419–478. arXiv: 1303.2849.

Anubhav Chaturvedi (2026). No Finite NPA Level Characterizes the Complete Quantum Set in the Simplest Bell Scenario. arXiv: 2607.14569.

Boris S. Cirel'son (Tsirelson) (1980). Quantum generalizations of Bell's inequality. Letters in Mathematical Physics 4, pp. 93–100. DOI: 10.1007/BF00417500.

John F. Clauser, Michael A. Horne, Abner Shimony and Richard A. Holt (1969). Proposed Experiment to Test Local Hidden-Variable Theories. Physical Review Letters 23, pp. 880–884. DOI: 10.1103/PhysRevLett.23.880.

Wim van Dam (2005). Implausible consequences of superstrong nonlocality. arXiv: quant-ph/0501159.

Wim van Dam. Scientific biography. University of California, Santa Barbara. Web page. Research on the theory of quantum computation and quantum communication; professor of computer science and physics at UC Santa Barbara.

Wim van Dam, Richard D. Gill and Peter D. Grünwald (2005). The statistical strength of nonlocality proofs. IEEE Transactions on Information Theory 51(8), pp. 2812–2835. DOI: 10.1109/TIT.2005.851738. "Much stronger" is on p. 9 of the arXiv preprint quant-ph/0307125; I didn't see the journal page.

Mnacho Echenim and Mehdi Mhalla (2023). Journal of Automated Reasoning 68(1), article 2 (print 2024). arXiv: 2306.12535v1; DOI: 10.1007/s10817-023-09689-9.

Albert Einstein, Boris Podolsky and Nathan Rosen (1935). Can Quantum-Mechanical Description of Physical Reality Be Considered Complete? Physical Review 47, pp. 777–780. DOI: 10.1103/PhysRev.47.777.

Arthur Fine (1982). Hidden Variables, Joint Probability, and the Bell Inequalities. Physical Review Letters 48, pp. 291–295. DOI: 10.1103/PhysRevLett.48.291.

Arthur Fine (2004, substantive revision 2017). The Einstein-Podolsky-Rosen Argument in Quantum Theory. Stanford Encyclopedia of Philosophy. Entry. Einstein's "smothered by formalism" (letter to Schrödinger, 19 June 1935) is Fine's translation, quoted from Section 1.1 of this entry, which gives it as p. 35 of Fine's The Shaky Game (2nd edition, 1996); I have it secondhand.

Stuart J. Freedman and John F. Clauser (1972). Experimental Test of Local Hidden-Variable Theories. Physical Review Letters 28, pp. 938–941. DOI: 10.1103/PhysRevLett.28.938.

Murray Gell-Mann and James B. Hartle (1995, printed 1998). Strong decoherence. In D.-H. Feng and B.-L. Hu (eds.), Proceedings of the 4th Drexel Conference on Quantum Non-Integrability: The Quantum-Classical Correspondence, International Press, Hong Kong. arXiv: gr-qc/9509054. The permanence of the past.

Iulia Georgescu (2026). John Bell and the most obscure journal in physics. Physics World. Article.

Richard D. Gill. Biography, Mathematical Institute, Leiden University. Page. Chair of Mathematical Statistics at Leiden from 2006.

Marissa Giustina and 21 coauthors (2015). Significant-Loophole-Free Test of Bell's Theorem with Entangled Photons. Physical Review Letters 115, 250401. DOI: 10.1103/PhysRevLett.115.250401.

Philippe Allard Guérin, Veronika Baumann, Flavio Del Santo and Časlav Brukner (2021). A no-go theorem for the persistent reality of Wigner's friend's perception. Communications Physics 4, 93. DOI: 10.1038/s42005-021-00589-1.

J. J. Halliwell (1999). Arrival Times in Quantum Theory from an Irreversible Detector Model. Progress of Theoretical Physics 102(4), pp. 707–717. DOI: 10.1143/PTP.102.707.

J. J. Halliwell (1999). Somewhere in the universe: Where is the information stored when histories decohere? Physical Review D 60, 105031. DOI: 10.1103/PhysRevD.60.105031.

B. Hensen and 18 coauthors (2015). Loophole-free Bell inequality violation using electron spins separated by 1.3 kilometres. Nature 526, pp. 682–686. DOI: 10.1038/nature15759.

Klaus Hepp (1972). Quantum theory of measurement and macroscopic observables. Helvetica Physica Acta 45, p. 237. The Coleman-Hepp model.

Gautam A. Kavuri and coauthors (2025). Traceable random numbers from a nonlocal quantum advantage. Nature 642, pp. 916–921. DOI: 10.1038/s41586-025-09054-3.

L. A. Khalfin. Person page, Math-Net.Ru. Entry. Steklov Mathematical Institute, St. Petersburg (Leningrad) branch.

Leonid A. Khalfin and Boris S. Tsirelson (1985). Quantum and quasi-classical analogs of Bell inequalities. In P. Lahti and P. Mittelstaedt (eds.), Symposium on the Foundations of Modern Physics: 50 Years of the Einstein-Podolsky-Rosen Gedankenexperiment, Joensuu, Finland, 16–20 June 1985. World Scientific, Singapore, 1985, p. 441.

Lawrence J. Landau (1988). Empirical two-point correlation functions. Foundations of Physics 18, pp. 449–460. DOI: 10.1007/BF00732549. Landau's 1969 Berkeley thesis, "Some Aspects of the Quantum Theory of Local Observables": Mathematics Genealogy Project.

Harold Ollivier, David Poulin and Wojciech H. Zurek (2004). Objective Properties from Subjective Quantum States: Environment as a Witness. Physical Review Letters 93, 220401. arXiv: quant-ph/0307229v2.

Helfried Peyrl and Pablo A. Parrilo (2008). Computing sum of squares decompositions with rational coefficients. Theoretical Computer Science 409(2), pp. 269–281. Author paper; DOI: 10.1016/j.tcs.2008.09.025.

William D. Phillips and Jean Dalibard (2022). Experimental tests of Bell's inequalities: A first-hand account by Alain Aspect. European Physical Journal D 77, article 8 (2023). arXiv: 2212.04737; DOI: 10.1140/epjd/s10053-022-00557-6.

Stefano Pironio, Antonio Acín, Serge Massar, A. Boyer de la Giroday, Dzmitry N. Matsukevich, Peter Maunz, Steven Olmschenk, David Hayes, Le Luo, T. Andrew Manning and Christopher Monroe (2010). Random numbers certified by Bell's theorem. Nature 464, pp. 1021–1024. DOI: 10.1038/nature09008.

Sandu Popescu and Daniel Rohrlich (1994). Quantum nonlocality as an axiom. Foundations of Physics 24, pp. 379–385. DOI: 10.1007/BF02058098.

Xiao-Liang Qi and Daniel Ranard (2021). Emergent classicality in general multipartite states and channels. Quantum 5, article 555. DOI: 10.22331/q-2021-09-28-555.

Peter Rastall (1985). Locality, Bell's theorem, and quantum mechanics. Foundations of Physics 15, pp. 963–972. DOI: 10.1007/BF00739036.

C. Jess Riedel, Wojciech H. Zurek and Michael Zwolak (2016). Objective past of a quantum universe: Redundant records of consistent histories. Physical Review A 93, 032126. DOI: 10.1103/PhysRevA.93.032126; arXiv: 1312.0331.

Lynden K. Shalm and 33 coauthors (2015). Strong Loophole-Free Test of Local Realism. Physical Review Letters 115, 250402. DOI: 10.1103/PhysRevLett.115.250402.

James Joseph Sylvester (1850). Additions to the articles "On a new class of theorems" and "On Pascal's theorem". Philosophical Magazine, series 3, 37(251), pp. 363–370, DOI: 10.1080/14786445008646629; reprinted in The Collected Mathematical Papers of James Joseph Sylvester, vol. 1, Cambridge University Press, 1904, pp. 145–151. The "oblong arrangement of terms" passage is quoted in Jeff Miller's Earliest Known Uses of Some of the Words of Mathematics (M), entry "matrix". "Oblong arrangement of terms" is on p. 150 of the Collected Papers, vol. 1.

Boris Tsirelson. Wikipedia. Article. Born in Leningrad in 1950; professor at Tel Aviv University from 1991 to 2017.

Jean Ville (1939). Étude critique de la notion de collectif. Gauthier-Villars, Paris. The maximal inequality for nonnegative martingales.

Stephanie Wehner (2006). Tsirelson bounds for generalized Clauser-Horne-Shimony-Holt inequalities. Physical Review A 73, 022110. arXiv: quant-ph/0510076v2; DOI: 10.1103/PhysRevA.73.022110.

Reinhard F. Werner and Michael M. Wolf (2001). Bell inequalities and entanglement. Quantum Information and Computation 1(3), pp. 1–25. DOI: 10.26421/QIC1.3-1.

John A. Wheeler (1990). Information, Physics, Quantum: The Search for Links. In Wojciech H. Zurek (ed.), Complexity, Entropy and the Physics of Information, Addison-Wesley. First given at the 3rd International Symposium on Foundations of Quantum Mechanics, Tokyo, 1989.

Andrew Whitaker (2016). John Stewart Bell and Twentieth Century Physics: Vision and Integrity. Oxford University Press.

Yanbao Zhang, Scott Glancy and Emanuel Knill (2011). Asymptotically optimal data analysis for rejecting local realism. Physical Review A 84, 062118. DOI: 10.1103/PhysRevA.84.062118.

Yanbao Zhang, Scott Glancy and Emanuel Knill (2013). Efficient quantification of experimental evidence against local realism. Physical Review A 88, 052119. DOI: 10.1103/PhysRevA.88.052119.

Wojciech H. Zurek (1981). Pointer basis of quantum apparatus: Into what mixture does the wave packet collapse? Physical Review D 24(6), pp. 1516–1525. DOI: 10.1103/PhysRevD.24.1516.

Wojciech H. Zurek (2009). Quantum Darwinism. Nature Physics 5, pp. 181–188. DOI: 10.1038/nphys1202; arXiv: 0903.5082v1.